5

운영체제 개요

만일 당신이 학부 운영체제 수업을 듣고 있다면 컴퓨터 프로그램이 실행될 때 어떤 일을 하는지 대략은 알고 있을 것이다. 그렇지 않다면 이 책을 읽는 것과 관련 수업을 듣는 것은 힘든 일이 될 것이다. 일단 읽기를 멈추고 가까운 서점에 가서 필요한 배경 지식을 제공하는 책을 구입하여 읽은 후 다시 시작하기 바란다. Patt/Patel [PP03]과 특히 Bryant/O'Hallaron [BO10]은 매우 훌륭한 책이다.

프로그램이 실행될 때 어떤 일이 일어날까?

프로그램은 매우 단순한 일을 한다: 명령어를 실행한다. 프로세서는 명령어를 초당 수백만 번 (요즘은 수십억 번) **반입**(**fetch**)하고, **해석**(**decode**)하고 (즉, 무슨 명령어인지 파악하고), **실행**(**execute**)한다(즉, 두 수를 더하고, 메모리에 접근하고, 조건을 검사하고, 함수로 분기하는 등의 정해진 일을 한다). 명령어 작업을 완료한 후 프로세서는 다음 명령어로, 또 그 다음 명령어로 프로그램이 완전히 종료될 때까지 실행을 계속한다[1].

우리는 방금 **Von Neumann** 컴퓨팅 모델의 기초를 설명하였다[2]. 간단하게 들리지, 그렇지? 우리는 이 수업에서 시스템을 사용하기 **쉽게하기 위해**(**easy to use**) 프로그램 실행 시 다양한 일들이 발생한다는 것을 배우게 될 것이다.

프로그램을 쉽게 실행하고 (심지어 동시에 여러 개의 프로그램을 실행시킬 수도 있음), 프로그램 간의 메모리 공유를 가능케 하고, 장치와 상호작용을 가능케하고, 다양 흥미로운 일을 할 수 있게 하는 소프트웨어가 있다. 시스템을 사용하기 편리하면서 정확하고 올바르게 동작시킬 책임이 있기 때문에 소프트웨어를 **운영체제**(**operating system**, **OS**)라고 부른다[3].

---

1) 물론 현대 프로세서는 프로그램을 빨리 실행시키기 위해서 안 보이는 곳에서 이상하고 기괴한 일을 한다. 예를 들면, 동시에 여러 명령어를 실행시키고 심지어는 명령어의 순서를 바꾸어서 실행시키기도 한다! 이런 일들은 우리의 관심사가 아니다. 우리는 대부분의 프로그램이 가정하고 있는 한 번에 하나의 명령어만 실행되고 순차적으로 실행되는 단순한 모델에만 관심을 둔다.

2) Von Neumann은 컴퓨팅 시스템의 초기 개척자 중의 한 명이었다. 그는 게임 이론과 원자 폭탄 분야에서도 선구자적 역할을 하였으며 NBA에서 6년 동안 선수 생활도 하였다. 이 중 하나는 사실이 아니다. 뭘까?

3) OS를 부르는 다른 이름에는 슈퍼바이저 (supervisor) 또는 심지어 주 제어 프로그램 (master control program)도 있다. 분명히 후자는 약간 열정이 지나친 것처럼 들리기 때문에 (자세한 내용은 영화 Tron을 보시오) 따라서 대신 감사하게도 "운영체제"라는 용어가 유행하게 되었다.

운영체제는 앞에서 언급한 일을 하기 위하여 **가상화(virtualization)**라고 불리는 기법을 사용한다. 운영체제는 프로세서, 메모리, 또는 디스크와 같은 **물리적 (physical)** 인 자원을 이용하여 일반적이고, 강력하고, 사용이 편리한 가상 (virtual) 형태의 자원을 생성한다. 때문에 운영체제를 때로는 **가상 머신(virtual machine)**이라고 부른다.

사용자 프로그램의 프로그램 실행, 메모리 할당, 파일 접근과 같은 가상 머신과 관련된 기능들을 운영체제에게 요청할 수 있도록, 운영체제는 사용자에게 API를 제공한다. 보통 운영체제는 응용 프로그램이 사용 가능한 수백 개의 **시스템 콜**을 제공한다. 운영체제가 프로그램 실행, 메모리와 장치에 접근, 기타 이와 관련된 여러 작업을 진행하기 위해 이러한 시스템 콜을 제공하기 때문에, 우리는 운영체제가 표준 라이브러리 (standard library)를 제공한다고 일컫기도 한다.

마지막으로, 가상화는 많은 프로그램들이 CPU를 공유하여, 동시에 실행될 수 있게 한다. 프로그램들이 각자 명령어와 데이터를 접근할 수 있게 한다. 프로그램들이 디스크 등의 장치를 공유할 수 있게 한다. 이러한 이유로 운영체제는 **자원 관리자(resource manager)**라고도 불린다. CPU, 메모리, 및 디스크는 시스템의 **자원**이다. 효율적으로, 공정하게, 이들 자원을 **관리**하는 것이 운영체제의 역할이다. 운영체제의 역할을 좀 더 잘 이해하기 위해서 몇 가지 예를 살펴 보기로 하자.

5.1 CPU 가상화

핵심 질문: 자원을 어떻게 가상화시키는가

이 책에서 우리가 대답할 가장 중요한 질문은 간단하다: 운영체제는 어떻게 자원을 가상화하는가? 이게 바로 우리의 핵심 문제이다. 답변이 너무 뻔하기 때문에 운영체제가 가상화시키는 *이유*는 궁금하지 않다. 시스템을 사용하기 편리하게 만들기 때문이다. 우리는 그 *방법에* 초점을 맞춘다. 가상화 효과를 얻기 위하여 운영체제가 구현하는 기법과 정책은 무엇인가? 운영체제는 이들을 어떻게 효율적으로 구현하는가? 어떤 하드웨어 지원이 필요한가?

운영체제를 구현할 때 해결해야 하는 특정 문제를 강조하는 방법으로 지금과 같이 음영 상자 형식으로 "*핵심 질문*"을 나타낼 것이다. 특정 주제를 다루면서 하나 이상의 핵심 질문을 만날 것이다. 그리고 그 장에서는 해당 질문에 답할 수 있는 해결책 또는 해결책의 기본 특징을 제시한다.

그림 5.1은 우리의 첫 번째 프로그램이다. 이 프로그램은 많은 일을 하지 않는다. 하는 일은 `Spin()`을 호출하는 것이다. `Spin()`은 1초 동안 실행된 후 리턴하는 함수이다. 그런 후 사용자가 명령어 라인으로 전달한 문자열을 출력한다. 이러한 일련의 작업을 무한히 반복한다.

이 코드를 `cpu.c`라는 이름으로 저장하고 단일 프로세서(또는 **CPU** 라는 용어도 섞어서 사용할 것임) 시스템에서 컴파일하고 실행시킨다고 가정하자. 다음과 같은 출력을 볼 수 있을 것이다.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <sys/time.h>
4  #include <assert.h>
5  #include "common.h"
6  int
7  main(int argc, char *argv[])
8  {
9      if (argc != 2) {
10         fprintf(stderr, "usage: cpu <string>\n");
11         exit(1);
12     }
13     char *str = argv[1];
14     while (1) {
15         Spin(1);
16         printf("%s\n", str);
17     }
18     return 0;
19 }
```

argc != 2

〈그림 5.1〉 **간단한 예 : 반복해서 출력하는 코드 (cpu.c)**

```
prompt> gcc -o cpu cpu.c -Wall
prompt> ./cpu "A"
A
A
A
A
^C
prompt>
```

그렇게 재미있는 결과는 아니다—시스템은 프로그램 실행 후 1초가 지나면 사용자가 전달한 입력 문자열을 (이 경우, 문자 "A") 출력한다. 출력 후 실행을 계속한다. 프로그램은 계속 실행된다는 것에 주목하자. "Control-c"를 (UNIX-계열 시스템에서 전위 프로그램을 종료시키는 키) 눌러 프로그램을 종료시킬 수 있다.

이번에는 같은 작업에 대한 여러 인스턴스를 동시에 실행시켜 보자. 그림 5.2에 약간 복잡해 보이는 실행 결과가 나와 있다.

```
prompt> ./cpu A & ; ./cpu B & ; ./cpu C & ; ./cpu D &
[1] 7353
[2] 7354
[3] 7355
[4] 7356
A
B
D
C
A
B
D
C
A
C
B
D
...
```

〈그림 5.2〉 **동시에 많은 프로그램 실행시키기**

이제 좀 더 흥미로워졌다. 프로세서가 하나밖에 없음에도 프로그램 4개 모두 동시에

실행되는 것처럼 보인다! 어떻게 이런 마술 같은 일이 일어날까?[4]

하드웨어의 도움을 받아 운영체제가 시스템에 매우 많은 수의 가상 CPU가 존재하는 듯한 **환상(illusion)**을 만들어 낸 것이다. 하나의 CPU 또는 소규모 CPU 집합을 무한 개의 CPU가 존재하는 것처럼 변환하여 동시에 많은 수의 프로그램을 실행시키는 것을 **CPU 가상화(virtualizing the CPU)**라 한다. 이 책의 첫 부분의 주제이다.

프로그램을 실행하고, 멈추고, 어떤 프로그램을 실행시킬 것인가를 운영체제에게 알려주기 위해서는 원하는 바를 운영체제에 전달할 수 있는 인터페이스 (API)가 필요하다. 이 책 전반에 걸쳐 이런 API들에 대해 논의한다. API는 운영체제와 사용자가 상호작용할 수 있는 주된 방법이다.

다수의 프로그램을 동시에 실행시킬 수 있는 기능은 새로운 종류의 문제를 발생시킨다는 것을 인지했을 것이다. 예를 들어, 특정 순간에 두 개의 프로그램이 실행되기를 원한다면, *누가 실행되어야 하는가*? 이 질문은 운영체제의 정책 (policy)에 달려있다. 운영체제의 여러 부분에서 이러한 유형의 문제에 답하기 위한 정책들이 사용된다. 운영체제가 구현한 동시에 다수의 프로그램을 실행시키는 **기본적인 기법(mechanism)**에 대해 다룰 것이다. 즉, **자원 관리자**로서의 운영체제의 역할을 다룬다.

```
1  #include <unistd.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include "common.h"
5  int
6  main(int argc, char *argv[])
7  {
8      int *p = malloc(sizeof(int)); // a1
9      assert(p != NULL);
10     printf("(%d) memory address of p: %08x\n",
11         getpid(), (unsigned) p); // a2
12     *p = 0; // a3
13     while (1) {
14         Spin(1);
15         *p = *p + 1;
16         printf("(%d) p: %d\n", getpid(), *p); // a4
17     }
18     return 0;
19 }
```

〈그림 5.3〉 **메모리 접근 프로그램 (mem.c)**

5.2 메모리 가상화

메모리에 대해 생각해 보자. 현재 우리가 사용하고 있는 컴퓨터에서의 **물리 메모리(physical memory)** 모델은 매우 단순하다. 바이트의 배열이다. 메모리를 읽기 위

4) & 문자를 사용하여 동시에 4개의 프로그램을 실행시키는 방법에 주목하자. 이렇게 하면 tcsh 쉘 상에서 작업을 백그라운드로 실행시킬 수 있다. 이는 사용자가 다음 명령어를 곧바로 실행시킬 수 있다는 것을 의미하고, 이 경우에는 다른 인스턴스를 실행시켰다. 명령어 사이의 세미콜론은 `tcsh`에서 동시에 여러 프로그램을 실행시킬 수 있게 한다. `bash`와 같은 다른 쉘을 사용하고 있다면 약간은 다르게 실행시켜야 한다. 자세한 사항에 대해서는 온라인 문서를 읽기 바란다.

해서는 데이터에 **주소(address)**를 명시해야 한다. 메모리에 **쓰기** (혹은 **갱신**) 위해서는 주소와 데이터를 명시해야 한다.

메모리는 프로그램이 실행되는 동안 항상 접근된다. 프로그램은 실행 중에 자신의 모든 자료 구조를 메모리에 유지하고 load와 store 또는 기타 메모리 접근을 위한 명령어를 통하여 자료 구조에 접근한다. 명령어 역시 메모리에 존재한다는 사실을 잊지 말자. 명령어를 반입할 때마다 메모리가 접근된다.

`malloc()`을 호출하여 메모리를 할당하는 그림 5.3의 프로그램을 살펴보자. 이 프로그램의 출력은 다음과 같다.

```
prompt> ./mem
(2134) memory address of p: 00200000
(2134) p: 1
(2134) p: 2
(2134) p: 3
(2134) p: 4
(2134) p: 5
^C
```

이 프로그램은 몇 가지 작업을 수행한다. 우선 메모리를 할당받는다 (a1 행). 그런 후 할당받은 메모리의 주소를 출력한다 (a2). 새로 할당받은 메모리의 첫 슬롯에 숫자 0을 넣는다 (a3). 마지막으로 루프로 진입하여 1초 대기 후, 변수 `p`가 가리키는 주소에 저장되어 있는 값을 1 증가시킨다. 출력할 때마다 실행 중인 프로그램의 프로세스 식별자(PID)라 불리는 값이 함께 출력된다. PID는 프로세스의 고유의 값이다.

```
prompt> ./mem &; ./mem &
[1] 24113
[2] 24114
(24113) memory address of p: 00200000
(24114) memory address of p: 00200000
(24113) p: 1
(24114) p: 1
(24114) p: 2
(24113) p: 2
(24113) p: 3
(24114) p: 3
(24113) p: 4
(24114) p: 4
...
```

〈그림 5.4〉 **메모리 프로그램 여러 번 실행하기**

첫 번째 실행 결과는 별로 흥미롭지 않다. 새로 할당된 메모리의 주소는 `00200000`이다. 프로그램이 진행되면서 천천히 값을 갱신하고 그 결과를 출력한다.

같은 프로그램을 여러 번 실행시켜 보자(그림 5.4). 프로그램들은 같은 주소에 메모리를 할당받지만 (`00200000`), 각각이 독립적으로 `00200000` 번지의 값을 갱신한다. 각 프로그램은 물리 메모리를 다른 프로그램과 공유하는 것이 아니라 각자 자신의 메모리를 가지고 있는 것처럼 보인다[5].

5) 이 예제 코드가 동작하려면, 주소 공간 난수화 기능을 꺼야 한다. 앞으로 보겠지만 난수화는 특정 유형의 보안 취약점을 방어하는 데 좋은 방어법이다. 스택-스매싱 공격(stack-smashing attack)을 이용하여

운영체제가 **메모리 가상화**(**virtualizing memory**)를 하기 때문에 이런 현상이 생긴다. 각 프로세스는 자신만의 **가상 주소 공간**(**virtual address space**, 때로 그냥 **주소 공간**(**address space**)이라고 불림)을 갖는다. 운영체제는 이 가상 주소 공간을 컴퓨터의 물리 메모리로 매핑(mapping)한다. 하나의 프로그램이 수행하는 각종 메모리 연산은 다른 프로그램의 주소 공간에 영향을 주지 않는다. 실행 중인 프로그램의 입장에서는, 자기 자신만의 물리 메모리를 갖는 셈이다. 실제로는 물리 메모리는 공유 자원이고, 운영체제에 의해 관리된다. 이러한 일들이 정확히 어떻게 일어나는지 역시 이 책의 첫 주제 **가상화**(**virtualization**)에 포함된다.

5.3 병행성

이 책의 다른 주요 주제는 **병행성**(**concurrency**)이다. 프로그램이 한 번에 많은 일을 하려 할 때 (즉, 동시에) 발생하는 그리고 반드시 해결해야 하는 문제들을 가리킬 때 이 용어를 사용한다. 병행성 문제는 우선 운영체제 자체에서 발생한다. 가상화에 관한 앞의 예에서 알 수 있듯이 운영체제는 한 프로세스 실행, 다음 프로세스, 또 다음 프로세스 등의 순서로 여러 프로세스를 실행시켜 한 번에 많은 일을 한다. 이러한 행동은 심각하고 흥미로운 문제를 발생시킨다.

병행성 문제는 운영체제만의 문제는 아니다. **멀티 쓰레드** 프로그램도 동일한 문제를 드러낸다. **멀티 쓰레드** 프로그램을 예로 들어 확인해 보자 (그림 5.5).

기본 아이디어는 간단하다(이에 대해서는 이 책의 병행성 부분에서 더 자세히 배우게 될 것이다). 메인 프로그램은 `Pthread_create()`를 사용하여 두 개의 쓰레드를 생성한다[6]. 쓰레드를 동일한 메모리 공간에서 함께 실행 중인 여러 개의 함수라고 생각할 수 있다. 이 예제 코드에서 각 쓰레드는 `worker()`라는 루틴을 실행한다. `worker()` 루틴은 `loops`번 만큼 루프를 반복하면서 카운터 값을 증가시킨다.

`loops` 변수를 1000으로 설정하여 프로그램을 실행시켰을 때 어떤 일이 일어나는지 아래에 나와 있다. `loops` 값은 각 쓰레드가 카운터를 증가시키는 횟수다. `loops` 값을 1000으로 지정한 후 프로그램을 실행시키면, `counter` 변수의 최종 값은 얼마가 될까?

```
prompt> gcc -o thread thread.c -Wall -pthread
prompt> ./thread 1000
Initial value : 0
Final value   : 2000
```

예상했겠지만 각 쓰레드가 1000번씩 counter 값을 증가시켰기 때문에 counter의 최종 값은 2000이 된다. 사실 `loops` 값이 N이면 프로그램의 최종 출력은 $2N$이 되리라고 예상할 수 있다. 곧 밝혀지겠지만 인생은 그리 간단하지 않다. 동일한 프로그램을 다시 실행시켜 보자. 이번에는 `loops`의 값을 더 큰 값으로 지정해 보자.

컴퓨터 시스템에 침입하는 법에 대해 알아보고 싶은 사람은 스스로 관련 자료를 읽어 보기 바란다. 비록 우리는 그러한 공격을 추천하지는 않지만...

6) 실제는 소문자로 시작하는 `pthread_create()`가 호출되어야 한다. 대문자로 시작하는 버전은 `pthread_create()`를 호출하고 반환 코드가 호출 성공을 나타내는지 검사하도록 우리가 만든 래퍼 (wrapper) 함수이다. 자세한 내용은 코드를 확인하기 바란다.

```
#include <stdio.h>
#include <stdlib.h>
#include "common.h"
volatile int counter = 0;
int loops;
void *worker(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        counter++;
    }
    return NULL;
}

int
main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "usage: threads <value>\n");
        exit(1);
    }
    loops = atoi(argv[1]);
    pthread_t p1, p2;
    printf("Initial value : %d\n", counter);

    Pthread_create(&p1, NULL, worker, NULL);
    Pthread_create(&p2, NULL, worker, NULL);
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("Final value : %d\n", counter);
    return 0;
}
```

〈그림 5.5〉 **멀티 쓰레드 프로그램 (threads.c)**

```
prompt> ./thread 100000
Initial value : 0
Final value   : 143012 // 어?
prompt> ./thread 100000
Initial value : 0
Final value   : 137298 // 뭐라고?
```

이번 실행에서는 입력 값을 100,000으로 주었더니 최종 값이 200,000이 아니라 143,012가 되었다. 다시 한 번 동일한 조건으로 실행시켰을 때에는 또 잘못된 값이 출력되었을 뿐 아니라 직전 실행과도 다른 결과가 출력되었다. 사실 `loops` 값을 큰 값으로 지정하여 반복해서 실행시키면 몇 번은 올바른 결과가 출력된다는 것을 발견할 수 있을 것이다. 왜 이런 일이 생기는 걸까?

예상하지 못한 결과의 원인은 명령어가 한 번에 하나씩만 실행된다는 것과 관련 있다. 앞 프로그램의 핵심 부분인 counter를 증가시키는 부분은 세 개의 명령어로 이루어진다. counter 값을 메모리에서 레지스터로 탑재하는 명령어 하나, 레지스터를 1 증가시키는 명령어 하나, 레지스터의 값을 다시 메모리에 저장하는 명령어 하나 이렇게 3개의 명령어로 구성된다. 이 세 개의 명령어가 **원자적(atomically)으로** (한 번에 3개 모두) 실행되지 않기 때문에 이상한 일이 발생할 수 있다. 이 책의 후반부에서 자세하게 논의할 주제가 바로 이 **병행성(concurrency)** 문제이다.

핵심 질문: 올바르게 동작하는 병행 프로그램은 어떻게 작성해야 하는가

같은 메모리 공간에 다수의 쓰레드가 동시에 실행한다고 할 때, 올바르게 동작하는 프로그램을 어떻게 작성할 수 있는가? 운영체제로부터 어떤 기본 기법들을 제공받아야 하는가? 하드웨어는 어떤 기능을 제공해야 하는가? 병행성 문제를 해결하기 위하여 기본 기법들과 하드웨어 기능을 어떻게 이용할 수 있는가?

5.4 영속성

이 책의 세 번째 주요 주제는 **영속성**(**persistence**)이다. DRAM과 같은 장치는 데이터를 **휘발성**(**volatile**) 방식으로 저장하기 때문에 메모리의 데이터는 쉽게 손실될 수 있다. 전원 공급이 끊어지거나 시스템이 갑자기 고장나면(crash) 메모리의 모든 데이터는 사라진다. 데이터를 영속적으로 저장할 수 있는 하드웨어와 소프트웨어가 필요하다. 저장 장치는 모든 시스템에 필수적이다.

하드웨어는 **입력/출력**(**input/output**) 혹은 **I/O** 장치 형태로 제공된다. 요즘에는 **solid-state drives**(**SSDs**)가 많이 사용되고 있기는 하지만 장기간 보존할 정보를 저장하는 장치로는 일반적으로 **하드 드라이브**(**hard drive**)가 사용된다.

디스크를 관리하는 운영체제 소프트웨어를 **파일 시스템**(**file system**)이라고 부른다. 파일 시스템은 사용자가 생성한 **파일**(**file**)을 시스템의 디스크에 안전하고 효율적인 방식으로 저장할 책임이 있다.

CPU나 메모리 가상화와는 달리 운영체제는 프로그램 별로 가상 디스크를 따로 생성하지 않는다. 오히려 사용자들이 종종 파일 정보를 공유하기 원한다고 가정한다. 예를 들어, C 프로그램을 작성할 때, 우선 에디터(예, Emacs[7])를 사용하여 C 파일을 생성하고 편집한다(`emacs -nw main.c`). 편집이 끝나면 소스 코드를 컴파일한다(예, `gcc -o main main.c`). 컴파일이 완료되면 새로 생성된 실행 파일을 실행할 수 있다(예, `./main`). 파일이 여러 다른 프로세스 사이에서 공유된다. 우선, emacs라는 편집기가 컴파일러가 사용할 파일을 생성한다. 컴파일러는 입력 파일을 사용하여 새로운 실행 파일을 생성한다(여러 단계를 거쳐서, 자세한 사항은 컴파일러 강좌를 수강하기 바란다). 마지막으로 실행 파일이 실행된다. 자 새로운 프로그램이 탄생했다!

핵심 질문: 데이터를 영속적으로 저장하는 방법은 무엇인가

파일 시스템은 데이터를 영속적으로 관리하는 운영체제의 일부분이다. 올바르게 일하기 위해서는 어떤 기법이 필요할까? 이러한 작업의 성능을 높이기 위해서 어떤 기법과 정책이 필요한가? 하드웨어와 소프트웨어가 실패하더라도 올바르게 동작하려면 어떻게 해야 하는가?

7) 반드시 Emacs를 사용하고 있어야 한다. 만일 vi를 사용하고 있는 중이라면 문제가 있을 가능성이 있다. 코드 에디터가 아닌 일반 편집기를 사용하는 중이라면 문제는 더 심각하다.

코드를 살펴보자. 그림 5.6은 문자열 "hello world"를 포함한 파일 `/tmp/file`을 생성하는 코드이다.

```
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <assert.h>
4  #include <fcntl.h>
5  #include <sys/types.h>
6  int
7  main(int argc, char *argv[])
8  {
9      int fd = open("/tmp/file", O_WRONLY | O_CREAT | O_TRUNC, S_IRWXU);
10     assert(fd > -1);
11     int rc = write(fd, "hello world\n", 13);
12     assert(rc == 13);
13     close(fd);
14     return 0;
15 }
```

〈그림 5.6〉 **입출력을 수행하는 프로그램 (`io.c`)**

여기서 프로그램은 운영체제를 세 번 호출한다. 첫째, `open()` 콜은 파일을 생성하고 연다. 둘째, `write()` 콜은 파일에 데이터를 쓴다. 셋째, `close()` 콜은 단순히 파일을 닫는데, 프로그램이 더 이상 해당 파일을 사용하지 않는다는 것을 나타낸다. 이들 시스템 콜(**system call**)은 운영체제에서 **파일 시스템**(**file system**)이라 불리는 부분으로 전달된다. 파일 시스템은 요청을 처리하고 경우에 따라 사용자에게 에러 코드를 반환한다.

데이터를 디스크에 쓰기 위해서 운영체제가 실제로 하는 일이 무엇인지 궁금할 것이다. 설명하겠지만, 그렇게 간단하지 않다. 파일 시스템은 많은 작업을 해야 한다. 먼저 새 데이터가 디스크의 어디에 저장될지 결정해야 하고, 파일 시스템이 관리하는 다양한 자료 구조를 통하여 데이터의 상태를 추적해야 한다. 이런 작업을 하기 위해서는 저장 장치로부터, 기존 자료 구조를 읽거나 갱신해야 한다. **장치 드라이버**(**device driver**)[8]를 작성해 본 사람은 알겠지만 당신을 대신하여 장치가 무언가 하게 하는 일은 복잡한 작업이다. 이를 위해서는 저수준의 장치 인터페이스와 그 시맨틱에 대한 깊은 이해가 필요하다. 운영체제는 시스템 콜이라는, 표준화된 방법으로 장치들을 접근할 수 있게 한다. 운영체제는 **표준 라이브러리**(**standard library**)처럼 보이기도 한다.

장치를 접근하는 방법과 파일 시스템이 데이터를 영속적으로 관리하는 방법은 이보다 훨씬 더 복잡하다. 성능향상을 위해서 대부분의 파일 시스템은 쓰기요청을 지연시켜 취합된 요청들을 한 번에 처리한다. 쓰기 중에 시스템의 갑작스런 고장에 대비해서 많은 파일 시스템들이 **저널링**(**journaling**)이나 **쓰기-시-복사**(**Copy-On-Write**)와 같은 복잡한 쓰기 기법을 사용한다. 이런 기법들은 쓰기 순서를 적절히 조정하여 고장이 발생하더라도 정상적인 상태로 복구될 수 있게 한다. 효율적인 디스크 작업을 위해 단순 리스트에서 복잡한 B-트리까지 다양한 종류의 자료 구조를 사용한다. 지금까지 설명한 내용을 이해 못해도 괜찮다! 이 책의 **영속성**(**persistence**) 부분에서 자세히 설명할

8) 디바이스 드라이버는 운영체제 코드의 일부분으로, 특정 장치의 처리를 담당한다. 장치와 장치 드라이버에 대해서는 나중에 더 자세하게 논의할 것이다.

것이다. 장치와 입출력에 관한 전반적인 내용과 디스크, RAID와 파일 시스템에 대해 매우 상세하게 다룰 것이다.

5.5 설계 목표

이제 운영체제가 실제로 어떤 일을 하는지 어느 정도 감을 잡았을 것이다. 운영체제는 CPU, 메모리, 디스크와 같은 물리 **자원**을 **가상화**(**virtualize**)한다. 운영체제는 병행성과 관련된 복잡한 문제를 처리한다. 파일을 **영속적**으로 저장하여 아주 오랜 시간 동안 안전한 상태에 있게 한다. 그런 시스템을 구현하려면 몇 가지를 목표를 세워야 한다. 이러한 목표는 설계와 구현에 집중하고, 필요한 경우 절충안을 찾는 데 필수적이다. 시스템 개발 시 적절한 절충안을 찾는 것은 매우 중요하다.

가장 기본적인 목표는 시스템을 편리하고 사용하기 쉽게 만드는 데 필요한 **개념**(abstraction)[9]들을 정의하는 것이다. 컴퓨터 과학에서 추상화는 모든 일에 근간이다. 추상화를 통해 큰 프로그램을 이해하기 쉬운 작은 부분들로 나누어 구현할 수 있다. 어셈블리 코드를 몰라도 C[10]와 같은 고수준 언어로 큰 프로그램을 작성하는 것이 가능하다. 추상화는 논리 게이트를 고려하지 않고도 어셈블리 코드를 작성할 수 있게 하며, 트랜지스터에 대한 지식없이도 게이트를 이용하여 프로세서를 만들 수 있게 한다. 추상화는 너무 근본이기 때문에 중요성을 쉽게 잊을 수 있다. 이 강의를 진행하는 동안은 잊지 않도록 하자. 각 절에서 오랫동안 개발되어 온 주요 개념을 설명하며, 운영체제의 각 부분을 쉽게 이해할 수 있는 단서를 제공할 것이다.

운영체제의 설계와 구현에 중요한 목표는 **성능**이다. 다른 말로 표현하면 **오버헤드를 최소화**(**minimize the overhead**)하는 것이다. "가상화"와 "사용하기 쉬운 시스템을 만드는것"은 의미가 있지만 반드시 해야 하는 것은 아니다. 가상화 및 다른 운영체제 기능을 과도한 오버헤드 없이 제공해야 한다. 오버헤드는 시간(더 많은 명령어)과 공간(메모리 또는 디스크)의 형태로 나타난다. 시간이나 공간 혹은 둘 다 최소로 하는 해결책을 찾을 것이다. 항상 완벽한 해답을 얻을 수는 없다. 앞으로 완벽한 해결책을 찾는 법과 절충하는 것에 대해 배우게 될 것이다.

또 다른 목표는 응용 프로그램 간의 **보호**, 그리고 운영체제와 응용 프로그램 간의 **보호**이다. 다수 프로그램들이 동시에 실행되기 때문에, 운영체제는 한 프로그램의 악의적인 또는 의도치 않은 행위가 다른 프로그램에게 피해를 주지 않는다는 것을 보장해야 한다. 당연히 응용 프로그램이 운영체제에게 해를 끼치지 않기를 원한다(운영체제에게 해를 가하면 모든 프로그램에게 영향을 주기 때문이다). 보호는 운영체제의 원칙 중 하나인 **고립**(**isolation**) 원칙의 핵심이다. 프로세스를 다른 프로세스로부터 고립시키는 일은 보호의 핵심이고 운영체제가 해야 하는 일 중 많은 부분의 근간이 된다.

운영체제는 계속 실행되어야 한다. 운영체제가 실패하면 그 위에서 실행되는 모든 응용 프로그램도 실패하게 된다. 이러한 종속성 때문에 운영체제는 높은 수준의 **신뢰**

9) 역자 주: abstraction은 추상화라는 사전적 의미보다 "개념"에 더 가까움.

10) 어떤 일부는 C를 고수준 언어라고 부르는 데에 반대할지 모른다. 그러나 이 수업은 운영체제 수업이라는 것을 기억하라. 우리는 단지 어셈블리 언어로만 코딩하지 않아도 된다는 것만으로도 너무 행복하다.

성(**reliability**)을 제공해야 한다. 운영체제가 복잡해질수록 (때론 수백만 라인의 코드로 이루어짐), 신뢰성 있는 운영체제를 구현하는 일이 매우 어려워진다. 운영체제 분야의 많은 연구(우리 연구를 포함하여 [Bai+09; Sun+10])들이 바로 이 문제에 초점을 맞추고 있다.

다른 중요한 목표들도 존재한다. **에너지-효율성**(**energy-efficiency**)은 녹색 세상을 위해 중요하다. 악의적인 응용 프로그램에 대한 **보안**(**security**, 사실은 보호의 확장)은 현재와 같은 네트워크 환경에서 특히 중요하다. **이동성**(**mobility**)은 운영체제가 작은 장치에서 사용될수록 중요해지고 있다. 시스템의 목적에 따라, 운영체제는 다른 목표를 지향하게 되고, 구현이 달라진다. 이 책에서 다루는 많은 주제들은 다양한 기기에서 모두 유용하다.

5.6 역사 약간

운영체제 소개를 마치기 전에 운영체제가 발전해 온 간단한 역사를 살펴보기로 하자. 인간이 만든 다른 것들(예: 자동차, 신발 등 아무거나)과 마찬가지로 이전의 사례로부터 학습하기 때문에, 시간이 지나면서 좋은 아이디어가 운영체제에 축적되었다. 몇 가지 주요 발전에 대해서 논의한다. 더 자세한 내용은 Brinch Hansen의 훌륭한 운영체제의 역사를 보기 바란다 [Han00].

초창기 운영체제: 단순 라이브러리

초창기 운영체제는 그렇게 많은 일을 하지 않았다. 기본적으로는 자주 사용되는 함수들을 모아 놓은 라이브러리에 불과했다. 예를 들어 프로그래머들이 각자 저수준 입출력 처리 코드를 작성하는 것이 아니라 운영체제가 그런 API를 제공하였다. 이러한 방식은 개발자들의 삶을 편하게 해준다.

옛날 메인프레임 시스템에서는 컴퓨터를 조작하는 사람이 프로그램을 한 번에 하나씩 실행하였다. 현대 운영체제가 하는 많은 작업, 예를 들어 작업의 순서를 정하는 것과 같은 것들을 컴퓨터 관리자가 담당했다. 현명한 개발자라면 컴퓨터 관리자에게 잘 보여야 한다. 그래야 당신의 작업을 우선 실행해주니까 말이다.

작업들이 준비되면 컴퓨터 관리자가 일괄적으로 처리한다. 이러한 방식의 컴퓨팅을 **일괄 처리**(**batch**)라고 부른다. 그 당시 컴퓨터는 비용 때문에 대화 방식으로 사용되지 않았다. 컴퓨터가 너무 고가였기 때문에 사용자가 그 앞에 앉아서 사용하게 할 수 없었다. 아무 것도 하지 않은 채 대부분의 시간을 보내게 되고 그러면 설치 기관은 시간당 수만 달러씩 낭비하게 되기 때문이다 [Han00].

라이브러리를 넘어서: 보호

단순한 라이브러리를 넘어서 운영체제는 컴퓨터 관리 면에서 더 중심적인 역할을 하게 된다. 이러한 방식이 등장하게 된 원인 중 하나는 운영체제가 실행하는 코드는 특별하다는 인식이었다. 운영체제 코드는 장치를 제어하였 때문에 일반 응용 프로그램 코드와는

다르게 취급되어야 한다. 왜 이래야 할까? 모든 응용 프로그램이 디스크에서 원하는 지점을 읽을 수 있다고 상상해 보자. 어떤 프로그램이든 원하는 모든 파일을 읽을 수 있기 때문에 사생활이라는 개념은 사라지게 될 것이다. 때문에, 라이브러리 형태로 **파일 시스템**을 구현하는 것은 의미가 없다. 다른 대체 방안이 필요하게 되었다.

Atlas 컴퓨팅 시스템 [Kil+61; Lav78]에 의해 **시스템 콜**이라는 아이디어가 발명되었다[11]. 운영체제를 라이브러리가 아니라 특별한 하드웨어 명령어와 하드웨어 상태를 결합하여 운영체제로 전환하기 위해서는 정해진 규칙에 따라 제어 가능한 과정을 거치도록 만들었다.

시스템 콜과 프로시저 호출의 결정적 차이는 시스템 콜은 제어를 운영체제에게 넘길 때 (예, 분기) **하드웨어 특권 수준**(**hardware privilege level**)을 상향 조정한다는 것이다. 사용자 응용 프로그램은 **사용자 모드**(**user mode**)라고 불리는 상태에서 실행된다. 사용자 모드에서는 응용 프로그램이 할 수 있는 일을 하드웨어적으로 제한한다. 예를 들어 사용자 모드에서 실행 중인 응용 프로그램은 디스크 입출력, 물리 메모리 페이지 접근 또는 네트워크 패킷 송신 등의 작업을 할 수 없다. 시스템 콜은 보통 **trap**이라고 불리는 특별한 하드웨어 명령어를 이용하여 호출된다. 시스템 콜 시작 시, 하드웨어는 미리 지정된 **트랩 핸들러**(**trap handler**) 함수에게 제어권을 넘기고 특권 수준을 **커널 모드**(**kernel mode**)로 격상시킨다. 트랩 핸들러 함수는 운영체제가 미리 구현해 놓는다. 커널 모드에서 운영체제는 시스템의 하드웨어를 자유롭게 접근할 수 있으며 입출력 또는 메모리 할당 등과 같은 작업을 할 수 있다. 운영체제가 서비스를 완료하면 **return-from-trap** 특수 명령어를 사용하여 제어권을 다시 사용자에게 넘긴다. 이 명령어는 응용 프로그램이 출발했던 지점으로 제어권을 넘기는 동시에 사용자 모드로 다시 전환한다.

멀티프로그래밍 시대

운영체제 다운 운영체제의 실제 등장은 메인프레임 이후 **미니컴퓨터**(**minicompu-ter**) 시대에 이루어졌다. Digital Equipment 사의 PDP 계열 컴퓨터의 등장으로 컴퓨터 가격이 크게 낮아졌다. 큰 기관이 하나의 메인프레임 컴퓨터를 갖는 대신 기관의 소그룹마다 하나의 컴퓨터를 가질 수 있었다. 컴퓨터 가격의 하락으로 개발자들의 활동이 활발해졌다[12]. 더 많은 유능한 인력이 컴퓨터 분야로 모여 들었고 컴퓨터 시스템이 더 흥미롭고 멋진 일들을 하게 만들었다.

컴퓨터 자원의 효율적 활용을 위해 **멀티프로그래밍**(**multiprogramming**)기법이 대중으로 사용되었다. 한 번에 하나의 프로그램만 실행시키는 대신 운영체제는 여러 작업을 메모리에 탑재하고 작업들을 빠르게 번갈아 가며 실행하여 CPU 사용률을 향상시킨다. 입출력 장치가 느리기 때문에 전환(switching) 능력이 특히 중요하였다. 입출력 요청이 서비스 되고 있는 동안 CPU가 대기하는 것은 CPU 시간 낭비를 초래한다. 대신 그동안 다른 작업으로 전환하여 잠시라도 실행하는 것이 낫지 않을까?

11) 역자 주: 실로 역사적인 사건이 아닐 수 없다.
12) 역자 주: 매우 중요한 현상이다.

멀티프로그래밍 지원의 필요와 인터럽트를 통한 입출력 작업 처리 등이 운영체제의 발전에 여러 가지의 혁신을 가져왔다. **메모리 보호**(**memory protection**)와 같은 주제가 중요하게 되었다. 한 프로그램이 다른 프로그램의 메모리에 접근하는 것을 원하지 않는다. 멀티프로그래밍에서 발생하는 **병행성**(**concurrency**) 문제에 대한 이해도 중요하다. 인터럽트가 발생하더라도 운영체제가 올바르게 동작한다는 것을 보장하는 것도 매우 어려운 일이다. 이와 관련된 여러 주제에 대해서는 이 책의 후반부에서 살펴본다.

그 당시 사건 중 하나는 Unix 운영체제의 등장이었다. Unix 운영체제는 Bell 연구소(예, 전화 회사 맞음)의 Ken Thompson과 Dennis Ritchie가 만들었다. Unix는 다른 운영체제로부터 좋은 아이디어를 많이 도입하였다. 특히 Multics [Org72], TENEX [Bob+72]와 Berkeley Time-Sharing System [Inc68] 같은 시스템으로부터도 아이디어를 받아들였다. Unix는 이 아이디어들을 더욱 단순하고 사용하기 쉽도록 변형하였다. 얼마 있어 이 팀은 Unix 소스 코드가 담긴 테이프를 전세계 사람들에게 보내기 시작했다. 소스 코드를 받은 사람 중 많은 사람들이 구현에 참여했으며, 시스템에 새로운 기능을 추가하였다. 더 자세한 사항에 대해서는 다음 페이지의 **여담** 상자를 보기 바란다[13].

현대

미니컴퓨터 시대를 지나 더 싸고, 더 빠르고 대중적인 컴퓨터가 등장하였다. 현재 **개인용 컴퓨터**(**personal computer**) 또는 **PC**라고 불리는 컴퓨터이다. Apple사의 초기 컴퓨터와 (예를 들면 Apple II) IBM PC에 의해 주도된 이 새로운 종류의 컴퓨터는 그룹당 하나의 미니컴퓨터 대신 책상 마다 하나의 컴퓨터를 놓을 수 있을만큼 가격이 쌌기 때문에 컴퓨팅의 주도 세력이 되었다.

불행하게도 운영체제의 입장에서는 PC의 등장이 퇴보를 의미하였다. 초기 PC들은 미니컴퓨터 기술로부터 얻었던 소중한 경험들을 무시하거나 아예 이에 대해 무지했기 때문이다. 예를 들어, **DOS**(**Disk Operating System**, **Microsoft** 제품) 같은 초기 운영체제는 메모리 보호가 중요하다는 생각조차 하지 않았다. 악의적인 혹은 잘못 개발된 응용 프로그램이 메모리 전체를 손상시킬 수 있었다. 1세대 **Mac OS**(v9과 그 이전 버전)는 작업 스케줄링에 협업 스케줄링(Cooperative Scheduling)을 채용하였다. 쓰레드가 우연히 무한루프에 빠지면, 전체 시스템을 정지시키게 되고, 결국 재부팅을 할 수밖에 없었다. 이 세대에 사라져버린 운영체제 기술들이 너무 많아 여기서 전부 논의할 수 없을 정도이다.

다행히 암흑기가 지난 뒤 예전 미니컴퓨터 운영체제 기법들이 데스크톱 컴퓨터용 운영체제에 등장하기 시작했다. 예를 들면, Mac OS X는 Unix를 근간으로 하며 Unix의 모든 특성을 가지고 있다. Windows 역시 컴퓨팅 역사의 위대한 아이디어 중 많은 것들을 채택하였다. 이러한 노력은 특히 Widows NT부터 시작되었는데, NT는 Microsoft

13) 우리는 여담 상자와 다른 관련 텍스트 상자들을 텍스트의 주제와는 맞지 않는 다양한 사항에 대해 주의를 환기하기 위해 사용한다. 때론 단지 농담하기 위해서도 그 상자들을 사용할 것이다. 진행하면서 약간의 재미를 즐기는 것은 괜찮지 않나? 예, 많은 농담들이 그리 재미있지는 않다.

여담: Unix의 중요성

운영체제의 역사에서 Unix의 중요성은 아무리 강조해도 지나치지 않다. 초기 시스템들로부터, 특히 MIT의 **Multics**는 위대한 아이디어들을 집대성하여 간단하고 강력한 시스템을 만들었다.

"Bell Labs"에서 시작된 Unix의 핵심은 서로 통합 사용이 가능한 작고 강력한 프로그램들이다. 명령어를 입력하는 **쉘**(**shell**)은 그러한 메타-수준 프로그래밍을 할 수 있는 **pipes**와 같은 기본 기법을 제공한다. 프로그램을 연결하는 것이 용이하다. 예를 들어, 텍스트 파일에서 "foo"라는 단어를 포함하고 있는 행을 찾고 행의 개수를 세고 싶은 경우, 다음과 같이 입력하면 된다: `grep foo file.txt | wc -l`. `grep`과 `wc`(word count) 프로그램을 이용하여 목적을 달성할 수 있다. Unix 환경은 프로그래머와 개발자에 친화적이며 새로운 C 언어를 위한 컴파일러를 제공하였다. 프로그래머가 프로그램을 쉽게 작성하고 공유할 수 있었기 때문에 Unix는 엄청난 인기를 끌었다. 저자들이 소스 코드를 요청한 누구에게나 공짜로 배포한 것도 크게 한 몫 하였다. 이는 **공개-소스 소프트웨어**(**open-source software**)의 초창기 형태라고 할 수 있다.

코드를 쉽게 접할 수 있고 가독성이 좋았다는 것도 매우 중요한 특징이다. C 언어로 작성된 아름답고 작은 크기의 커널은 많은 사람들이 커널에 새로운 기능과 멋진 특징을 추가할 수 있게 하였다. 예를 들어, **Bill Joy**가 이끌었던 Berkeley의 사업화 그룹은 환상적인 배포판(**Berkeley Systems Distribution**, **BSD**)을 만들었다. 이 배포판은 발전된 가상 메모리, 파일 시스템 및 네트워킹 서브시스템을 가지고 있었다. Joy는 나중에 **Sun Microsystems**를 설립하였다.

불행하게도 회사들이 소유권을 주장하고 이익을 추구하면서 조금씩 느려졌다. 변호사가 관여하면 항상 발생하는 흔한 결과였다. 많은 회사들이 자신의 고유 버전을 소유하고 있었다. Sun Microsystems의 **SunOS**, IBM의 **AIX**, HP의 **HPUX**("H-Pucks"라고도 불림), SGI의 **IRIX** 등이 대표적인 예이다. AT&T/Bell Labs와 다른 회사 간의 법적 다툼은 Unix에 먹구름을 드리웠다. 많은 사람들은 Windows가 등장하여 PC 시장을 장악함에 따라 Unix가 살아남을 수 있을까 하는 의문을 가지게 되었다.

운영체제 기술을 크게 발전시켰다. 오늘날 이동전화조차 Linux와 같은 운영체제를 실행하고 있다. 이 운영체제는 1980년대 PC 용 운영체제보다 1970년대 미니컴퓨터 용 운영체제에 가깝다 (정말 다행이다). 운영체제 전성기에 개발된 좋은 아이디어들이 현재 시스템에서도 여전히 적용되는 것을 보면 사뭇 즐겁다. 더욱이 이 아이디어들은 계속 발전하고 있다. 이러한 발전으로 운영체제에 많은 기능들이 추가되고 사용하기 좋은 시스템이 되고 있다.

5.7 요약

운영체제에 대한 소개를 마쳤다. 오늘날 운영체제는 시스템을 사용하기 쉽게 만들었다. 거의 모든 운영체제가 이 책에서 논의하게 될 기술을 사용한다.

시간적인 제약 때문에 이 책에서 다루지 못한 운영체제 내용이 존재한다. 예를

여담: 그리고 LINUX가 나왔다

UNIX에게는 다행하게도 **Linus Torvalds**라는 이름의 젊은 핀란드 해커가 자신만의 UNIX 버전을 개발하기로 마음 먹었다. UNIX의 원리와 개념은 사용하였지만 코드는 사용하지 않아 법적 문제는 발생하지 않았다. 그는 전세계의 개발자들에게 도움을 요청하였고 곧 **Linux**가 탄생하였다. 동시에 현재의 공개-소스 소프트웨어 운동도 시작되었다.

인터넷 시대가 도래함에 따라 Google, Amazon, Facebook 등 대부분의 회사가 공짜이면서 자신들의 입맛대로 쉽게 수정할 수 있는 Linux를 사용하기로 결정하였다. 사실 이런 시스템이 없었다면 이런 새 회사들의 성공은 상상하기 힘들다. 스마트폰(Android)이 사용자가 직접 사용하는 주된 플랫폼이 되면서 역시 같은 이유 때문에 Linux는 입지를 확장하였다. 그리고 Steve Jobs는 자신의 UNIX-기반 NeXTStep 운영 환경을 Apple에 가지고 가서 UNIX를 데스크톱에서도 대중적으로 만들었다. 많은 Apple 사용자들은 이러한 사실 조차 알지 못할 것이다. 이리하여 UNIX는 그 어느 때보다도 건재하다. 이런 환상적인 결과에 대해 감사해야 한다.

들면, 운영체제에는 많은 **네트워킹** 코드가 있다. 네트워킹 수업의 수강 여부는 당신에게 맡기겠다. 비슷하게 **그래픽**(**graphic**) 장치는 특히 중요하다. 이 분야의 지식을 쌓고 싶으면, 그래픽 수업을 듣기 바란다. 마지막으로 어떤 운영체제 교재는 **보안**을 상세히 다룬다. 운영체제는 실행 중인 프로그램들 간에 보호를 제공해야 하고 응용 프로그램들이 자신의 파일을 보호할 수 있는 능력을 주어야 한다는 의미에서 보안을 다룬다. 그러나 보안 수업에서 발견할 수 있는 심화된 보안 쟁점에 대해서는 깊이 들어가지 않는다.

다루어야 할 중요한 주제가 많다. CPU와 메모리 가상화의 기본 원리, 병행성 및 장치와 파일 시스템을 통한 영속성이 바로 그것이다. 걱정하지 말기 바란다! 공부해야 할 것이 많지만 강좌가 끝나면 컴퓨터 시스템의 실제 동작 방식에 관한 새로운 이해를 가지게 될 것이다. 자 출발하자!

참고 문헌

[Bai+09] **"Tolerating File-System Mistakes with EnvyFS"**
LakshmiN. Bairavasundaram, Swaminathan Sundararaman, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau
USENIX '09, San Diego, CA, June 2009
동시에 여러 파일 시스템을 사용하면 그 중 하나의 실수를 감내할 수 있다는 재미있는 논문

[Bob+72] **"TENEX, A Paged Time Sharing System for the PDP-10"**
Daniel G. Bobrow, Jerry D. Burchfiel, Daniel L. Murphy, and Raymond S. Tomlinson
CACM, Volume 15, Number 3, March 1972
TENEX는 현대 운영체제에서 발견되는 많은 기법을 가지고 있다; 얼마나 많은 혁신이 이미 1970년대에 일어났는지 알고 싶은 사람은 읽어 보기 바란다.

[BO10] **"Computer Systems: A Programmer's Perspective"**
Randal E. Bryant and David R. O'Hallaron
Addison-Wesley, 2010
컴퓨터 시스템이 어떻게 동작하는가에 관한 또 하나의 훌륭한 소개. 이 책과 약간 겹치는 부분이 존재하므로 원한다면 그 책의 마지막 몇 장을 건너 뛰거나 또는 그 책을 읽어서 같은 주제에 대해 다른 시각을 경험하기 바란다. 결국 자신만의 지식을 쌓는 좋은 방법은 가능한 많은 다른 사람의 관점을 듣고 자신만의 의견과 생각을 개발해야 함. 생각 또 생각.

[Han00] **"The Evolution of Operating Systems"**
P. Brinch Hansen
In Classic Operating Systems: From Batch Processing to Distributed Systems Springer-Verlag, New York, 2000
이 에세이는 역사적으로 중요한 시스템에 관한 논문 모둠을 소개한다.

[Inc68] **"SDS 940 Time-Sharing System"**
Scientific Data Systems Inc.
TECHNICAL MANUAL, SDS 90 11168 August 1968
URL: `http://goo.gl/EN0Zrn`
예, 찾을 수 있는 가장 좋은 기술 매뉴얼. 옛날 시스템 문서를 읽고 이미 1960년대 후반에 얼마나 많은 것들이 나타났는지 알아보는 것은 환상적인 일이다. Berkeley Time-Sharing System(후에 SDS 시스템이 됨)을 만든 사람 중에 하나는 Butler Lampson이었다. 그는 나중에 시스템 분야에 공헌한 공로로 Turing-award를 수상한다.

[Kil+61] **"One-Level Storage System"**
T. Kilburn, D.B.G. Edwards, M.J. Lanigan, and F.H. Sumner
IRE Transactions on Electronic Computers, April 1962
Atlas는 현대 시스템에서 볼 수 있는 많은 작품들을 만들었다. 그러나 이 논문이 최고는 아니다. 하나만 읽어야 한다면 아래의 역사적 관점을 설명한 책 [Lav78]을 읽는 것이 낫다.

[Lav78] **"The Manchester Mark I and Atlas: A Historical Perspective"**
S.H. Lavington
Communications of the ACM archive Volume 21, Issue 1 (January 1978), pages 4-12
컴퓨터 시스템의 초창기 발전 역사와 Atlas의 선구적 노력에 관한 좋은 논문. 물론 Atlas 원 논문을 읽는 것도 괜찮지만 이 논문은 좋은 개관과 역사적 관점을 추가하였다.

[Org72] **"The Multics System: An Examination of its Structure"**
Elliott Organick

*Multics의 좋은 개괄. 너무 좋은 아이디어들이 많았지만 과설계된 시스템. 너무 많은 목적을 가지고 있어 실제로는 하나도 제대로 동작하지 않음. Fred Brooks가 "second-system effect"라고 부른 고전적인 예 [**B75**].*

[PP03] **"Introduction to Computing Systems: From Bits and Gates to C and Beyond"**
Yale N. Patt and Sanjay J. Patel
McGraw-Hill, 2003
컴퓨팅 시스템에 대한 소개로 저자들이 좋아하는 책 중 하나. 트랜지스터와 게이트로부터 출발하여 C까지 차례대로 설명함. 앞 부분 내용이 특히 좋음.

[RT74] **"The Unix Time-Sharing System"**
Dennis M. Ritchie and Ken Thompson
CACM, Volume 17, Number 7, July 1974, pages 365-375
컴퓨팅 세계를 장악하고 나서 개발자들에 의해 쓰여진 훌륭한 요약

[Sun+10] **"Membrane: Operating System Support for Restartable File Systems"**
Swaminathan Sundararaman, Sriram Subramanian, Abhishek Rajimwale, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, and Michael M. Swift
FAST '10, San Jose, CA, February 2010
강의 노트를 쓸 때 좋은 점은 당신의 연구를 홍보할 수 있다는 것이다. 그러나 이 논문은 실제 매우 잘된 논문이다. 파일 시스템이 버그를 만나 갑자기 고장날 때, Membrane은 마술과 같이 재시동 된다. 이때 응용 프로그램이나 시스템의 다른 부분에는 전혀 영향을 주지 않는다.

7

프로세스의 개념

이 장에는 운영체제가 제공하는 핵심 개념 중 하나인 **프로세스**(**process**)에 대해 논의한다. 일반적으로 프로세스는 **실행 중인 프로그램**으로 정의한다 [VCG65; Han70]. 프로그램 자체는 생명이 없는 존재다. 프로그램은 디스크 상에 존재하며 실행을 위한 명령어와 정적 데이터의 묶음이다. 이 명령어와 데이터 묶음을 읽고 실행하여 프로그램에 생명을 불어넣는 것이 운영체제이다.

사용자는 하나 이상의 프로그램을 동시에 실행시키기를 원한다. 예를 들어, 웹 브라우저, 메일 프로그램, 게임, 음악 플레이어 등을 실행하는 데스크톱 또는 랩톱 컴퓨터를 생각해 보라. 컴퓨터들은 동시에 수십 혹은 수백 개의 프로세스를 실행하는 것처럼 보인다. 여러 프로그램을 동시에 실행할 수 있으면, 시스템을 쉽게 사용할 수 있다. 사용자는 사용 가능한 CPU가 있는지 신경쓰지 않고 그저 프로그램만 실행시키면 된다. 우리의 도전 과제는 다음과 같다.

핵심 질문: CPU가 여러 개 존재한다는 환상을 어떻게 제공하는가
적은 개수의 CPU 밖에 없더라도, 운영체제는 어떻게 거의 무한개에 가까운 CPU가 있는 듯한 환상을 만들 수 있을까?

운영체제는 CPU를 **가상화**하여 이러한 환상을 만들어 낸다. 하나의 프로세스를 실행하고, 얼마 후 중단시키고 다른 프로세스를 실행하는 작업을 반복하면서 실제 하나 또는 소수의 CPU로 여러 개의 가상 CPU가 존재하는 듯한 환상을 만들어 낸다. **시분할**(**time sharing**)이라 불리는 이 기법은 원하는 수 만큼의 프로세스를 동시에 실행할 수 있게 한다. 시분할 기법은 CPU를 공유하기 때문에, 각 프로세스의 성능은 낮아진다.

운영체제에서 CPU 가상화를 잘 구현하기 위해, 저수준의 도구와 고차원적인 "지능"이 필요하다. 저수준 도구를 **메커니즘**(**mechanism**)이라 한다. 메커니즘은 필요한 기능을 구현하는 방법이나 규칙을 의미한다. 예를 들어, 나중에 **문맥 교환**(**context switch**)의 구현에 대해 배우게 될텐데, CPU에서 프로그램 실행을 잠시 중단하고 다른 프로그램을 실행하는 것을 문맥 교환이라고 한다. 이 **시분할** 기법은 모든 현대 운영체제들이 채택하고 있다.

팁: 시분할과 공간 분할 이용

시분할은 자원 공유를 운영체제가 사용하는 가장 기본 기법 중 하나이다. 한 개체가 잠깐 자원을 사용한 후, 다른 개체가 또 잠깐 자원을 사용하고, 그 다음 개체가 사용하면서 이 자원(CPU 또는 네트워크 링크 등)을 많은 개체들이 공유한다. 시분할과 자연스럽게 대응되는 개념은 **공간 분할**(**space sharing**)이 될 것이다. 공간 분할은 개체에게 공간을 분할해 준다. 공간 분할의 예로 디스크가 있다. 디스크는 자연스럽게 공간 분할할 수 있는 자원으로, 블럭이 하나의 파일에 할당되면 파일을 삭제하기 전에는 다른 파일이 할당될 가능성이 낮다.

운영체제의 지능은 **정책**(**policy**)의 형태로 표현된다. 정책은 운영체제 내에서 어떤 결정을 내리기 위한 알고리즘이다. 예를 들어, 실행 가능한 여러 프로그램들이 있을 때, 운영체제는 어느 프로그램을 실행시켜야 하는가? 운영체제의 **스케줄링 정책**(**scheduling policy**)이 이러한 결정을 내린다. 이러한 결정을 내리기 위하여 스케줄링 정책은 과거 정보(예, 직전 1분 동안 어떤 프로그램이 자주 실행되었는지), 워크로드에 관한 지식(예, 어떤 유형의 프로그램들이 실행되었는지), 및 성능 측정 결과(예, 시스템이 대화 성능 혹은 처리량을 높이려 하는지)를 이용한다.

7.1 프로세스의 개념

운영체제는 **실행 중인 프로그램**의 개념을 제공하는데, 이를 **프로세스**(**process**)라 한다. 전술한 바와 같이 프로세스는 실행 중인 프로그램이다. (특정 순간의) 프로세스를 간단하게 표현하려면, 실행되는 동안 접근했거나 영향을 받은 자원의 목록을 작성하면 된다.

프로세스의 구성 요소를 이해하기 위해서 **하드웨어 상태**(**machine state**)를 이해해야 한다. 프로그램이 실행되는 동안 하드웨어 상태를 읽거나 갱신할 수 있다. 이때 가장 중요한 하드웨어 구성 요소는 무엇일까?

프로세스의 하드웨어 상태 중 가장 중요한 구성 요소는 *메모리이다*. 명령어는 메모리에 저장된다. 실행 프로그램이 읽고 쓰는 데이터 역시 메모리에 저장된다. 프로세스가 접근할 수 있는 메모리(**주소 공간**(**address space**)이라 불림)는 프로세스를 구성하는 요소이다.

레지스터도 프로세스의 하드웨어 상태를 구성하는 요소 중 하나이다. 많은 명령어들이 레지스터를 직접 읽거나 갱신한다. 프로세스를 실행하는 데 레지스터도 빠질 수 없다.

프로세스의 하드웨어 상태를 구성하는 레지스터 중에 특별한 레지스터들이 존재한다. **프로그램 카운터**(**program counter**, **PC**)는 프로그램의 어느 명령어가 실행 중인지를 알려준다. 프로그램 카운터는 **명령어 포인터**(**instruction pointer**, **IP**)라고도 불린다. **스택 포인터**(**stack pointer**)와 **프레임 포인터**(**frame pointer**)는 함수의 변수와 리턴 주소를 저장하는 스택을 관리할 때 사용하는 레지스터이다.

프로그램은 영구 저장장치(persistent storage)에 접근하기도 한다. 이 입출력 *정보*는 프로세스가 현재 열어 놓은 파일 목록을 가지고 있다.

팁: 정책과 구현의 분리

많은 운영체제에서 공통된 설계 패러다임은 고수준 정책을 저수준 기법으로부터 분리하는 것이다 [Lev+75]. 기법은 시스템에 관한 "어떻게"라는 질문에 답을 제공하는 것이라고 생각할 수 있다. 예를 들어, 운영체제는 어떻게 문맥 교환을 하는가? 등이 해당된다. 정책은 "어느 것"이라는 질문에 답한다. 예를 들어, 운영체제는 지금 당장 어느 프로세스를 실행시켜야 하는가? 등이 해당된다. 둘을 분리하면 정책을 변경할 때 기법의 변경을 고민하지 않아도 된다. 따라서 일반적인 소프트웨어 설계 원칙인 **모듈성(modularity)**의 한 형태이다.

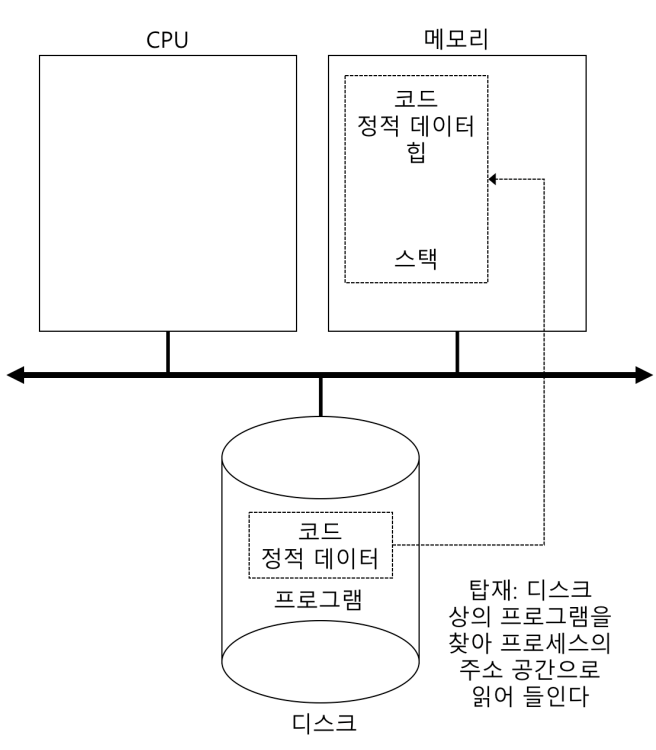
7.2 프로세스 API

실제 프로세스 API에 대한 논의는 다음의 장으로 미루겠지만, 운영체제가 반드시 API로 제공해야 하는 몇몇 기본 기능에 대해 간단히 살펴본다. 이 API들은 형태는 다르지만 모든 현대 운영체제에서 제공된다.

- 생성(Create): 운영체제는 새로운 프로세스를 생성할 수 있는 방법을 제공해야 한다. 쉘에 명령어를 입력하거나, 응용 프로그램의 아이콘을 더블-클릭하여 프로그램을 실행시키면, 운영체제는 새로운 프로세스를 생성한다.
- 제거(Destroy): 프로세스 생성 인터페이스를 제공하는 것처럼 운영체제는 프로세스를 강제로 제거할 수 있는 인터페이스를 제공해야 한다. 물론, 많은 프로세스는 실행되고 할 일을 다하면 스스로 종료한다. 그러나 프로세스가 스스로 종료하지 않으면 사용자는 그 프로세스를 제거하길 원할 것이고, 필요없는 프로세스를 중단시키는 API는 매우 유용하다.
- 대기(Wait): 때론 어떤 프로세스의 실행 중지를 기다릴 필요가 있기 때문에 여러 종류의 대기 인터페이스가 제공된다.
- 각종 제어(Miscellaneous Control): 프로세스의 제거, 대기 이외에, 여러 가지 제어 기능들이 제공된다. 예를 들어, 대부분의 운영체제는 프로세스를 일시정지하거나 재개(일시정지되었던 프로세스의 실행을 다시 시작)하는 기능을 제공한다.
- 상태(Status): 프로세스 상태 정보를 얻어내는 인터페이스도 제공된다. 상태 정보에는 얼마 동안 실행되었는지 또는 프로세스가 어떤 상태에 있는지 등이 포함된다.

7.3 프로세스 생성: 좀 더 자세하게

우리가 밝혀내야 할 한 가지 미스테리는 프로그램이 어떻게 프로세스로 변형되는가이다. 운영체제는 어떻게 프로그램을 준비하고 실행시키는가? 실제로 어떻게 프로세스를 생성하는가?



〈그림 7.1〉 **탑재 : 프로그램에서 프로세스로**

프로그램 실행을 위하여 운영체제가 하는 첫 번째 작업은 프로그램 코드와 정적 데이터(static data, 예를 들어, 초기값을 가지는 변수)를 메모리, 프로세스의 주소 공간에 **탑재**(**load**)하는 것이다. 프로그램은 **디스크** 또는 요즘 시스템에서는 **플래시-기반 SSD**에 특정 **실행 파일 형식**으로 존재한다. 코드와 정적 데이터를 메모리에 탑재하기 위해서 운영체제는 디스크의 해당 바이트를 읽어서 메모리의 어딘가에 저장해야 한다 (그림 7.1을 보시오).

초기 운영체제들은 프로그램 실행 전에 코드와 데이터를 모두 메모리에 탑재하였다. 현대의 운영체제들은 이 작업을 늦추었다. 즉, 프로그램을 실행하면서 코드나 데이터가 필요할 때 필요한 부분만 메모리에 탑재한다. 코드와 데이터의 늦은 탑재의 동작을 정확하게 이해하기 위해서는 **페이징**(**paging**)과 **스와핑**(**swapping**) 동작의 이해가 필요하다. 이에 대해서는 나중에 메모리 가상화를 논의할 때 자세히 다루게 될 것이다. 지금은 어떤 프로그램이든 실행시키기 전에 운영체제는 프로그램의 중요 부분을 디스크에서 메모리로 탑재해야 한다는 것만 기억하자.

코드와 정적 데이터가 메모리로 탑재된 후, 프로세스를 실행시키기 전에 운영체제가 해야 할 일이 몇 가지 있다. 일정량의 메모리가 프로그램의 **실행시간 스택**(run-time stack, 혹은 그냥 **스택**) 용도로 할당되어야 한다. 이미 알고 있겠지만 C 프로그램은 지역 변수, 함수 인자, 리턴 주소 등을 저장하기 위해 스택을 사용한다. 운영체제는 스택을 주어진 인자로 초기화한다. 특히, `main()`함수의 인자인 `argc`와 `argv` 벡터를

사용하여 스택을 초기화한다.

운영체제는 프로그램의 **힙**(**heap**)을 위한 메모리 영역을 할당한다. C 프로그램에서 힙은 동적으로 할당된 데이터를 저장하기 위해 사용된다. 프로그램은 `malloc()`을 호출하여 필요한 공간을 요청하고 `free()`를 호출하여 사용했던 공간을 반환하여 다른 프로그램이 사용할 수 있도록 한다. 힙은 연결 리스트, 해시 테이블, 트리 등 크기가 가변적인 자료 구조를 위해 사용된다. 프로그램이 실행되면 `malloc()` 라이브러리 API를 호출하여 메모리를 요청하고, 운영체제가 이를 충족하도록 메모리를 할당한다.

운영체제는 또 입출력과 관계된 초기화 작업을 수행한다. 예를 들어, Unix 시스템에서 각 프로세스는 기본적으로 표준 입력(`STDIN`), 표준 출력(`STDOUT`), 표준 에러(`STDERR`) 장치에 해당하는 세 개의 **파일 디스크립터**(**file descriptor**)를 갖는다. 이 디스크립터들을 사용하여 프로그램은 터미널로부터 입력을 읽고 화면에 출력을 프린트하는 작업을 쉽게 할 수 있다. 입출력, 파일 디스크립터 등에 관해서는 이 책의 세 번째 부분인 **영속성**에서 다룬다.

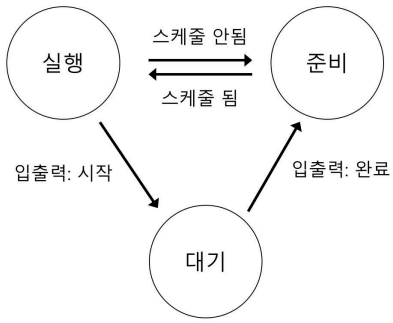
코드와 정적 데이터를 메모리에 탑재하고, 스택과 힙을 생성하고 초기화하고, 입출력 셋업과 관계된 다른 작업을 마치게 되면, 운영체제는 프로그램 실행을 위한 준비를 마치게 된다. 프로그램의 시작 지점(entry point), 즉 `main()`에서부터 프로그램 실행을 시작하는 마지막 작업만이 남는다. `main()` 루틴으로 분기함으로써 (다음 장에서 이를 가능하게 하는 특수 기법을 설명할 것이다), 운영체제는 CPU를 새로 생성된 프로세스에게 넘기게 되고 프로그램 실행이 시작된다.

7.4 프로세스 상태

프로세스가 무엇인지(이 개념에 대해 명확히 정의하겠지만), 어떻게 생성되는지(대강이라도) 알게 되었으므로, 프로세스의 **상태**(**state**)에 대해 논의해 보자. 프로세스 상태의 개념은 초기 컴퓨터 시스템에서 등장하였다 [DH66; VCG65]. 프로세스 상태를 단순화하면 다음 세 상태 중 하나에 존재할 수 있다.

- 실행(Running): 실행 상태에서 프로세스는 프로세서에서 실행 중이다. 즉, 프로세스는 명령어를 실행하고 있다.
- 준비(Ready): 준비 상태에서 프로세스는 실행할 준비가 되어 있지만 운영체제가 다른 프로세스를 실행하고 있는 등의 이유로 대기 중이다.
- 대기(Blocked): 프로세스가 다른 사건을 기다리는 동안 프로세스의 수행을 중단시키는 연산이다. 흔한 예: 프로세스가 디스크에 대한 입출력 요청을 하였을 때 프로세스는 입출력이 완료될 때까지 대기 상태가 되고, 다른 프로세스가 실행 상태로 될 수 있다.

이러한 상태를 그림으로 표현하면 그림 7.2와 같이 될 것이다. 그림에서 보듯이 프로세스는 준비 상태와 실행 상태를 운영체제의 정책에 따라 이동한다. 프로세스는 운영체제의 스케줄링 정책에 따라 **스케줄**이 되면 준비 상태에서 실행 상태로 전이한다.



〈그림 7.2〉 **프로세스 : 상태 전이**

실행 상태에서 준비 상태로의 전이는 프로세스가 나중에 다시 스케줄 될 수 있는 상태가 되었다는 것을 의미한다. 프로세스가 입출력 요청 등의 이유로 대기 상태가 되면 요청 완료 등의 이벤트가 발생할 때까지 대기 상태로 유지된다. 이벤트가 발생하면 프로세스는 다시 준비 상태로 전이되고 운영체제의 결정에 따라 바로 다시 실행될 수도 있다.

두 개의 프로세스가 어떻게 전이될 수 있는지를 보여주는 예를 보자. 실행 중인 두 프로세스가 있다고 하자. 각 프로세스는 오직 CPU만 사용하고 입출력을 행하지 않는다. 이 경우, 각 프로세스의 상태 전이는 그림 7.3과 같은 모양이 될 수 있다.

시간	Process0	Process1	비고
1	실행	준비	
2	실행	준비	
3	실행	준비	
4	실행	준비	Process0 종료
5		실행	
6		실행	
7		실행	
8		실행	Process1 종료

〈그림 7.3〉 **프로세스 상태 추이 : CPU만 이용할 때**

다음 예에서는 첫 번째 프로세스가 어느 정도 실행한 후에 입출력을 요청한다. 그 순간 프로세스는 대기 상태가 되고 다른 프로세스에게 실행 기회를 준다. 그림 7.4가 시나리오의 진행 과정을 보인다.

자세히 살펴보면, Process0은 입출력을 요청하고 요청한 작업이 완료되기를 기다린다. 프로세스는 디스크를 읽거나 네트워크로부터 패킷을 기다릴 때 대기 상태로 전이한다. 운영체제는 Process0이 CPU를 사용하지 않는다는 것을 감지하고 Process1을 실행시킨다. Process1이 실행되는 동안 입출력은 완료되고 Process0은 준비 상태로 다시 전이된다. 결국 Process1은 종료되고 Process0이 실행되어 종료된다.

위와 같은 간단한 예에서조차 운영체제가 내려야 할 결정이 매우 많다는 사실에

시간	Process0	Process1	비고
1	실행	준비	
2	실행	준비	
3	실행	준비	Process0이 입출력을 시작한다.
4	대기	실행	Process0은 대기 상태가 되고,
5	대기	실행	Process1이 실행된다.
6	대기	실행	
7	준비	실행	Process0 입출력 종료
8	준비	실행	Process1 종료
9	실행	-	
10	실행	-	Process0 종료

〈그림 7.4〉 **프로세스 상태 추이 : CPU 이용과 입출력 작업을 할 때**

주목해야 한다. 우선 시스템은 Process0이 입출력을 요청할 때 Process1의 실행여부를 결정해야 한다. Process1을 실행키로한 결정은 CPU를 계속 동작시키므로 자원 이용률을 높인다. 또한, 시스템은 Process0이 요청한 입출력이 완료되었을 때, Process0을 바로 실행하지 않고 실행 중이던 Process1을 계속 실행하였다. 이러한 결정이 좋은 결정이었는지는 확실하지 않다. 당신은 어떻게 생각하는가? 운영체제는 스케줄러를 통해 이러한 결정을 내린다. 운영체제의 스케줄러는 차후에 자세히 다룰 것이다.

여담 : 자료 구조—프로세스 리스트

운영체제에는 다양한 중요 **자료 구조**들이 많이 존재한다. 프로세스 리스트가 그 중 첫 번째이다. 이 자료 구조는 단순하다. 다수의 프로그램을 동시에 실행할 수 있는 모든 운영체제는 이와 유사한 자료 구조를 가지고 있고, 이 자료 구조를 이용하여 시스템에서 실행 중인 프로그램을 관리한다. 프로세스의 관리를 위한 정보를 저장하는 자료 구조를 **프로세스 제어 블럭**(**Process Control Block**, **PCB**)이라 부른다. 각 프로세스에 관한 정보를 저장하는 C 자료 구조를 이야기 할 때 부르는 멋진 이름이다.

7.5 자료 구조

운영체제도 일종의 프로그램이다. 다른 프로그램들과 같이 다양한 정보를 유지하기 위한 자료 구조를 가지고 있다. 예를 들어, 프로세스 상태를 파악하기 위해 준비 상태의 프로세스들을 위한 **프로세스 리스트**(**process list**)와 같은 자료 구조를 유지한다. 또한, 어느 프로세스가 실행 중인지를 파악하기 위한 부가적인 자료 구조도 유지한다. 운영체제는 또 대기 상태인 프로세스도 파악해야 한다. 입출력 요청이 완료되면 운영체제는 적절한 프로세스를 깨워 준비 상태로 다시 전이시킬 수 있어야 한다.

그림 7.5는 xv6 커널에서 각 프로세스를 추적하기 위해 운영체제가 필요로 하는 정보를 보이고 있다 [Cox+08]. Linux, Mac OS X, 또는 Windows 같은 운영체제들도 이와 비슷한 프로세스 구조를 가지고 있다. 찾아 보면 훨씬 복잡하다는 것을 알 수 있을 것이다.

그림을 통해서 운영체제가 관리하는 있는 프로세스 정보를 알 수 있다. **레지스터 문맥**(**register context**) 자료 구조는 프로세스가 중단되었을 때 해당 프로세스의 레지스터값들을 저장한다. 이 레지스터값들을 복원하여(예, 해당 값을 실제 물리 레지스터에 다시 저장함으로써) 운영체제는 프로세스 실행을 재개한다. **문맥 교환**(**context switch**)이라고 알려진 이 기법에 관해서 나중에 좀 더 자세히 다루기로하자.

```
// 프로세스를 중단하고 이후에 재개하기 위해
// xv6가 저장하고 복원하는 레지스터
struct context {
    int eip;
    int esp;
    int ebx;
    int ecx;
    int edx;
    int esi;
    int edi;
    int ebp;
};

// 가능한 프로세스 상태
enum proc_state { UNUSED, EMBRYO, SLEEPING,
                  RUNNABLE, RUNNING, ZOMBIE };
// 레지스터 문맥과 상태를 포함하여
// 각 프로세스에 대하여 xv6가 추적하는 정보
struct proc {
    char *mem;                // 프로세스 메모리 시작 주소
    uint sz;                  // 프로세스 메모리의 크기
    char *kstack;             // 이 프로세스의 커널 스택의 바닥 주소
    enum proc_state state;    // 프로세스 상태
    int pid;                  // 프로세스 ID
    struct proc *parent;      // 부모 프로세스
    void *chan;               // 0이 아니면, chan에서 수면
    int killed;               // 0이 아니면 종료됨
    struct file *ofile[NOFILE];   // 열린 파일
    struct inode *cwd;        // 현재 디렉터리
    struct context context;   // 프로세스를 실행시키려면 여기로 교환
    struct trapframe *tf;     // 현재 인터럽트에 해당하는 트랩 프레임
};
```

〈그림 7.5〉 **xv6 Proc 구조**

그림에서 실행, 준비, 대기 외에 다른 상태들이 존재하는 것을 볼 수 있다. **초기**(**initial**) 상태를 가지는 시스템도 있다. 프로세스가 생성되는 동안에는 초기 상태에 머무른다. 프로세스는 종료되었지만 메모리에 남아있는 상태인 **최종**(**final**)상태도 있다 (Unix-기반 시스템에서 이 상태는 **좀비**(**zombie**) 상태[1]라고 불린다). 이 상태는 프로세스가 성공적으로 실행했는지를 다른 프로세스(보통은 **부모**(**parent**) 프로세스)가 검사하는 데 유용하다. 이를 위하여 최종 상태를 활용한다(Unix-기반 시스템에서는 프로세스가 성공적으로 종료되었으면 0을, 그렇지 않으면 0이 아닌 값을 반환한다). 부모

1) 이 좀비들은 제거 명령으로 쉽게 제거할 수 있으나 일반적으로 다른 방법을 사용하여 제거한다.

프로세스는 자식 프로세스의 종료를 대기하는 시스템 콜을 호출(예, `wait()`) 한다. 이 호출은 종료된 프로세스와 관련된 자원들을 정리할 수 있다고 운영체제에 알리는 역할도 한다.

7.6 요약

운영체제의 기본 개념인 프로세스를 소개하였다. 프로세스는 간단히 말하면 실행 중인 프로그램이다. 이 개념을 염두에 두고 이제 좀 더 구체적인 핵심 사항을 살펴볼 것이다. 여기서 핵심 사항은 프로세스 구현에 필요한 기법, 구현한 프로세스를 스케줄링하는 정책 등을 뜻한다. 이러한 여러 방법들을 조합하여 운영체제가 CPU를 가상화하는 방식을 이해하자.

여담: 시뮬레이션 숙제

시뮬레이션 숙제는 논의된 주제를 이해하였는지 확인할 수 있도록 실행 가능한 시뮬레이터 형태로 제공된다. 시뮬레이터는 통상 python 프로그램 형태로 제공된다. 이 프로그램을 사용하면 랜덤 시드를 이용하여 다른 문제를 출제할 수 있고 `-c` 플래그를 지정하면 주어진 문제에 대한 해답을 만들 수 있다. 이렇게 생성된 해결책과 비교하여 당신의 해결책을 점검할 수 있다. `-h` 또는 `--help` 옵션을 지정하고 시뮬레이터를 실행시키면 시뮬레이터가 제공하는 모든 옵션에 대한 정보를 얻을 수 있다.

각 시뮬레이터와 함께 제공되는 README 파일은 시뮬레이터를 실행시키는 방법에 대해 상세히 설명한다. 각 플래그에 대한 상세한 설명을 확인할 수 있다.

참고 문헌

[Cox+08] **"The xv6 Operating System"**
Russ Cox, Frans Kaashoek, Robert Morris, and Nickolai Zeldovich
URL: `http://pdos.csail.mit.edu/6.828/2008/index.html`
세상에서 가장 멋지고 작은 실제 운영체제. 운영체제의 상세한 동작을 배우려면 다운로드하여 가지고 놀아 보아라.

[DH66] **"Programming Semantics for Multiprogrammed Computations"**
Jack B. Dennis and Earl C. Van Horn
Communications of the ACM, Volume 9, Number 3, March 1966
이 논문은 멀티프로그래밍 시스템과 관련된 많은 초기 용어와 개념을 정의한다.

[Han70] **"The Nucleus of a Multiprogramming System"**
Per Brinch Hansen
Communications of the ACM, Volume 13, Number 4, April 1970
이 논문은 운영체제 역사상 첫 번째 마이크로커널 중의 하나인 Nucleus를 소개한다. 크기가 작고 꼭 필요한 기능만 가진 시스템을 구현하려는 아이디어는 운영체제 역사에서 반복해서 등장하는 주제이다. 그 모든 것이 이 논문에서 설명된 Brinch Hansen의 연구와 함께 시작되었다.

[Lev+75] **"Policy/mechanism separation in Hydra"**
R. Levin, E. Cohen, W. Corwin, F. Pollack, and W. Wulf
SOSP 1975
Hydra로 알려진 연구 운영체제의 구조에 관한 초기 논문. Hydra는 주류 운영체제가 된 적은 없지만 몇몇 아이디어는 운영체제 설계자에게 많은 영향을 주었다.

[VCG65] **"Structure of the Multics Supervisor"**
V.A. Vyssotsky, F.J. Corbato, and R.M. Graham
Fall Joint Computer Conference, 1965
Multics에 관한 초기 논문. 이 논문은 현대 시스템에서 발견되는 많은 기본 아이디어와 용어를 설명한다. 유틸리티 컴퓨팅의 배경이 되는 몇몇 선견지명은 마침내 현대 클라우드 시스템에서 구현되었다.

숙제

process-run.py 프로그램은 프로세스가 실행되면서 변하는 프로세스의 상태를 추적할 수 있고, 프로세스가 CPU를 사용하는지(예, add 명령어 실행) 입출력을 하는지(예, 디스크에 요청을 보내고 완료되기를 기다린다)를 알아볼 수 있다. 상세한 사항은 README 파일을 확인하기 바란다.

문제

1. 다음과 같이 플래그를 지정하고 프로그램을 실행시키시오: ./process-run.py -l 5:100, 5:100. CPU 이용률은 얼마가 되어야 하는가(예, CPU가 사용 중인 시간의 퍼센트?) 그러한 이용률을 예측한 이유는 무엇인가? -c 플래그를 지정하여 예측이 맞는지 확인하시오. 100
2. 이제 다음과 같이 플래그를 지정하고 실행시키시오: ./process-run.py -l 4:100, 1:0. 이 플래그는 4개의 명령어를 실행하고 모두 CPU만 사용하는 하나의 프로세스와 오직 입출력을 요청하고 완료되기를 기다리는 하나의 프로세스를 명시한다. 두 프로세스가 모두 종료되는 데 얼마의 시간이 걸리는가? -c 플래그를 사용하여 예측한 것이 맞는지 확인하시오. 11틱
3. 옵션으로 지정된 프로세스의 순서를 바꾸시오: ./process-run.py -l 1:0, 4:100. 이제 어떤 결과가 나오는가? 실행 순서를 교환하는 것은 중요한가? 이유는 무엇인가? (언제나처럼 -c 플래그를 사용하여 예측이 맞는지 확인하시오.) 7틱
4. 자, 다른 플래그에 대해서도 알아보자. 중요한 플래그 중 하나는 -S로서 프로세스가 입출력을 요청했을 때 시스템이 어떻게 반응하는지를 결정한다. 이 플래그가 SWITCH_ON_END로 지정되면 시스템은 요청 프로세스가 입출력을 하는 동안 다른 프로세스로 전환하지 않고 대신 요청 프로세스가 종료될 때까지 기다린다. 입출력만 수행하는 프로세스와 CPU 작업만 하는 프로세스 두 개를 실행시키면 어떤 결과가 발생하는가?(-l 1:0,4:200 -c -S SWITCH_ON_END) 11틱
5. 이번에는 프로세스가 입출력을 기다릴 때마다 다른 프로세스로 전환하도록 플래그를 지정하여 같은 프로세스를 실행시켜 보자 (-l 1:0,4:100 -c -S SWITCH_ON_IO). 이제 어떤 결과가 발생하는가? -c를 사용하여 예측이 맞는지 확인하시오. 7틱 7
6. 또 다른 중요한 행동은 입출력이 완료되었을 때 무엇을 하느냐이다. -I IO_RUN_LATER가 지정되면 입출력이 완료되었을 때 입출력을 요청한 프로세스가 바로 실행될 필요가 없다. 완료 시점에 실행 중이던 프로세스가 계속 실행된다. 다음과 같은 조합의 프로세스를 실행시키면 무슨 결과가 나오는가? (./process-run.py -l 3:0, 5:100, 5:100, 5:100 -S SWITCH_ON_IO -I IO_RUN_LATER -c -p) 시스템 자원은 효과적으로 활용되는가? 3틱
7. 같은 프로세스 조합을 실행시킬 때 -I IO_RUN_IMMEDIATE를 지정하고 실행시키시오. 이 플래그는 입출력이 완료되었을 때 요청 프로세스가 곧바로 실행되는 동작을

의미한다. 이 동작은 어떤 결과를 만들어 내는가? 방금 입출력을 완료한 프로세스를 다시 실행시키는 것이 좋은 생각일 수 있는 이유는 무엇인가?

8. 이제 다음과 같이 무작위로 생성된 프로세스를 실행시켜 보자. 예를 들면, `-s 1 -l 3:50,3:50`, `-s 2 -l 3:50,3:50`, `-s 3 -l 3:50,3:50`. 어떤 양상을 보일지 예측할 수 있는지 생각해 보시오. `-I IO_RUN_IMMEDIATE`를 지정했을 때와 `-I IO_RUN_LATER`를 지정했을 때 어떤 결과가 나오는가? `-S SWITCH_ON_IO` 대 `-S SWITCH_ON_END`의 경우에는 어떤 결과가 나오는가?

1 p1번째 p3-5 ⋮ p1-3번

2 p2-1 p4-1 ⋮

3 p2-2 p4-2 ⋮

4 p2-3 p4-3 p1-2번

5 p2-4 p4-4 p1-3번

6 p2-5 p4-5 ⋮

7 p3-1 p1-1번 ⋮

8 p3-2 p1-2번 ⋮

9 p3-3 ⋮ ⋮

10 p3-4 ⋮ ⋮

8

막간: 프로세스 API

여담: 막간

이번 절에서는 운영체제 API와 그의 사용법을 포함한 시스템의 실제적인 측면을 다룰 것이다. 실제적인 측면에 관심이 없다면 이 막간 절은 넘어갈 수 있다. 실생활에서는 이런 실제적인 측면이 유용하기 때문에 관심을 가지고 좋아해야 한다. 회사는 사용할 수 없는 지식을 가진 사람을 채용하지 않는다.

이번 절에서는 UNIX 시스템의 프로세스 생성에 관해 논의한다. UNIX는 프로세스를 생성하기 위하여 `fork()`와 `exec()` 시스템 콜을 사용한다. `wait()`는 프로세스가 자신이 생성한 프로세스가 종료되기를 기다리기 원할 때 사용된다. 이제부터 이 인터페이스에 대해 간단한 예제를 이용하여 더 자세하게 설명한다. 우리의 문제는 다음과 같다.

핵심 질문: 프로세스를 생성하고 제어하는 방법

프로세스를 생성하고 제어하려면 운영체제가 어떤 인터페이스를 제공해야 하는가? 유용하고 편하게 사용하기 위해서 이 인터페이스는 어떻게 설계되어야 하는가?

8.1 `fork()` 시스템 콜

프로세스 생성에 `fork()` 시스템 콜이 사용된다 [Con63]. 미리 경고하자면, 이 시스템 콜이 당신이 사용할 시스템 콜 중에서 가장 이해하기 힘든 시스템 콜이다[1].

그림 8.1에 실행 가능한 프로그램이 나와 있다. 코드를 분석해 보라. 아니면 타이핑해서 직접 실행시켜 보기 바란다!

이 프로그램을 실행하면(`p1.c`) 다음과 같은 출력을 보게 될 것이다.

1) 물론 가장 이해하기 힘들다는 것을 보장할 수는 없다. 어쨌거나 `fork()` 시스템 콜은 매우 특이하다.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4
5  int
6  main(int argc, char *argv[])
7  {
8      printf("hello world (pid:%d)\n", (int) getpid());
9      int rc = fork();
10     if (rc < 0) {           // fork 실패; 종료
11         fprintf(stderr, "fork failed\n");
12         exit(1);
13     } else if (rc == 0) {   // 자식 (새 프로세스)
14         printf("hello, I am child (pid:%d)\n", (int) getpid());
15     } else {                // 부모 프로세스는 이 경로를 따라 실행한다 (main)
16         printf("hello, I am parent of %d (pid:%d)\n",
17                rc, (int) getpid());
18     }
19     return 0;
20 }
```

〈그림 8.1〉 fork() 호출 (p1.c)

```
prompt> ./p1
hello world (pid:29146)
hello, I am parent of 29147 (pid:29146)
hello, I am child (pid:29147)
prompt>
```

p1.c 프로그램에서 어떤 일들이 벌어지는지 더 자세히 보도록 하자. 실행이 시작될 때 프로세스는 "hello world ..." 메시지를 출력한다. 이 메시지에는 **PID**로도 알려진 **프로세스 식별자**(**process identifier**)가 포함된다. 이 프로세스는 29146이라는 PID를 가진다. UNIX 시스템에서 PID는 프로세스의 실행이나 중단과 같이 특정 프로세스를 대상으로 작업을 해야 할 경우 프로세스를 지칭하기 위해 사용된다. 지금까지는 오케이다.

재미있는 부분은 지금부터 시작이다. 프로세스는 `fork()` 시스템 콜을 호출한다. 운영체제는 프로세스 생성을 위해 이 시스템 콜을 제공한다. 이상한 부분이 있다: 생성된 프로세스가 *호출한 프로세스의 복사본이라*는 것이다. `fork()` 호출 직후를 살펴보자. 운영체제 입장에서 보면, 프로그램 `p1`이 2개가 존재한다. 두 프로세스가 모두 `fork()`에서 리턴하기 직전이다. 새로 생성된 프로세스는(일반적으로 **자식** 프로세스, 생성한 프로세스는 **부모** 프로세스라 불린다) `main()`함수 첫 부분부터 시작하지 않았다는 것을 알 수 있다 ("hello, world" 메시지가 한 번만 출력되었다는 것에 유의하기 바란다). 자식 프로세스는 `fork()`를 호출하면서부터 시작되었다.

자식 프로세스는 부모 프로세스와 완전히 동일하지는 않다. 자식 프로세스는 자신의 주소 공간, 자신의 레지스터, 자신의 PC 값을 갖는다. 매우 중요한 차이점이 있다. 아무리 강조해도 지나치지 않다. `fork()` 시스템 콜의 반환 값이 서로 다르다. `fork()`로 부터 부모 프로세스는 생성된 자식 프로세스의 PID를 반환받고, 자식 프로세스는 0을 반환받는다. 이 반환 값의 차이로 인해, 그림 8.1과 같이 부모와 자식 프로세스가 서로 다른 코드를 실행하는 프로그램을 쉽게 작성할 수 있다.

이 프로그램의 출력 결과가 항상 동일하지는 않다. 단일 CPU 시스템에서 이 프로그램을 실행하면, 프로세스가 생성되는 시점에는 2개(부모와 자식) 프로세스 중 하나가

실행된다. 위의 출력 예에서는 부모 프로세스 실행 후에 자식 프로세스가 실행되었다. 예에서 볼 수 있는 바와 같이, 그 반대 경우도 발생할 수 있다.

```
prompt> ./p1
hello world (pid:29146)
hello, I am child (pid:29147)
hello, I am parent of 29147 (pid:29146)
prompt>
```

CPU **스케줄러**(**scheduler**)는 실행할 프로세스를 선택한다. 스케줄러에 관해서는 곧 자세하게 논의한다. 스케줄러의 동작은 일반적으로 상당히 복잡하고 상황에 따라 다른 선택이 이루어지기 때문에, 어느 프로세스가 먼저 실행된다라고 단정하는 것은 매우 어렵다. 이 **비결정성**(**nondeterminism**)으로 인해 **멀티 쓰레드 프로그램** 실행 시 다양한 문제가 발생한다. 이 책의 제2편 **병행성** 부분에서 비결정성에 대해 자세히 다룰 것이다.

8.2 wait() 시스템 콜

아직 배울 것이 많이 남았다. 이제까지 메시지를 출력하는 자식 프로세스를 만들어 보았다. 부모 프로세스가 자식 프로세스의 종료를 대기해야 하는 경우도 발생할 수 있다. 이러한 작업을 위해 `wait()` 시스템 콜이 (혹은 더 많은 기능을 가진 `waitpid()`) 있다. 자세한 사항은 그림 8.2를 보라.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5  int
6  main(int argc, char *argv[])
7  {
8      printf("hello world (pid:%d)\n", (int) getpid());
9      int rc = fork();
10     if (rc < 0) {           // fork 실패; 종료
11         fprintf(stderr, "fork failed\n");
12         exit(1);
13     } else if (rc == 0) {   // 자식 (새 프로세스)
14         printf("hello, I am child (pid:%d)\n", (int) getpid());
15     } else {                // 부모 프로세스는 이 경로를 따라 실행한다 (main)
16         int wc = wait(NULL);
17         printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
18                rc, wc, (int) getpid());
19     }
20     return 0;
21 }
```

〈그림 8.2〉 `fork()`와 `wait()` 호출 (`p2.c`)

이 예제에서 (`p2.c`) 부모 프로세스는 `wait()` 시스템 콜을 호출하여 자식 프로세스 종료 시점까지 자신의 실행을 잠시 중지시킨다. 자식 프로세스가 종료되면 `wait()`는 리턴한다.

`wait()` 호출을 위와 같이 코드에 추가하면 프로그램은 항상 동일한 결과를 출력한다. 왜 그런지 알 수 있겠지? 좋아, 생각해 보도록.

(생각하는 동안 기다림 .... 끝)

자 이제 생각해 보았으니, 출력 결과를 보자.

```
prompt> ./p2
hello world (pid:29266)
hello, I am child (pid:29267)
hello, I am parent of 29267 (wc:29267) (pid:29266)
prompt>
```

이 코드에서는 항상 자식 프로세스가 먼저 출력을 수행한다는 것을 알 수 있다. 왜 그럴까? 자식 프로세스가 부모 프로세스보다 먼저 실행되면 당연히 자식 프로세스가 먼저 출력된다. 부모 프로세스가 먼저 실행되면 곧바로 `wait()`를 호출한다. 이 시스템 콜은 자식 프로세스가 종료될 때까지 리턴하지 않는다[2]. 부모가 먼저 실행되더라도 자식 종료 후 `wait()`가 리턴한다. 그런 후 부모 프로세스가 출력한다.

8.3 드디어, `exec()` 시스템 콜

프로세스 생성 관련 API 중에서 마지막으로 중요한 시스템 콜은 `exec()` 시스템 콜이다[3]. 이 시스템 콜은 자기 자신이 아닌 다른 프로그램을 실행해야 할 때 사용한다. `p2.c`의 `fork()` 시스템 콜은 자신의 복사본을 생성하여 실행한다. 자신의 복사본이 아닌 다른 프로그램을 실행해야 할 경우에는 바로 `exec()` 시스템 콜이 그 일을 한다 (그림 8.3).

이 예에서 자식 프로세스는 `wc` 프로그램을 실행하기 위해 `execvp()` 시스템 콜을 호출한다. `wc` 프로그램은 단어의 개수를 세는 프로그램이다. 사실 예제 프로그램은 자신의 소스 파일인 `p3.c`를 인자로 하여 `wc`를 실행하고 소스 코드의 행 개수, 단어의 개수, 바이트의 개수를 알려 준다.

```
prompt> ./p3
hello world (pid:29383)
hello, I am child (pid:29384)
      29     107     1030    p3.c
hello, I am parent of 29384 (wc:29384) (pid:29383)
prompt>
```

`fork()` 시스템 콜만큼 `exec()` 시스템 콜도 이해하기 어렵다. `exec()` 시스템 콜은 다음과 같은 과정으로 수행된다. 실행 파일의 이름과(예, `wc`) 약간의 인자가(예, `p3.c`) 주어지면 해당 실행 파일의 코드와 정적 데이터를 읽어 들여 현재 실행 중인 프로세스의 코드 세그멘트와 정적 데이터 부분을 덮어 쓴다. 힙과 스택 및 프로그램 다른

2) `wait()`가 자식 프로세스가 종료하기 전에 리턴하는 몇 가지 경우가 있다. 자세한 사항은 `man` 페이지를 참조하기 바란다. 이 책의 주장에(예를 들어, "자식이 항상 먼저 프린트 한다" 혹은, "UNIX가 최고다, 아이스크림보다 더 좋다" 등) 대해서는 주의해야 한다.

3) 사실 `exec()` 시스템 콜은 6가지 변형이 존재한다. `execl()`, `execle()`, `execlp()`, `execv()`, `execvp()`, `execve()`가 그것이다. 더 자세한 사항은 `man` 페이지를 읽어보기 바란다.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <string.h>
5  #include <sys/wait.h>
6  int
7  main(int argc, char *argv[])
8  {
9      printf("hello world (pid:%d)\n", (int) getpid());
10     int rc = fork();
11     if (rc < 0) { // fork 실패함; exit
12         fprintf(stderr, "fork failed\n");
13         exit(1);
14     } else if (rc == 0) {            // 자식 (새 프로세스)
15     printf("hello, I am child (pid:%d)\n", (int) getpid());
16     char *myargs[3];
17     myargs[0] = strdup("wc");      // 프로그램: "wc"(단어 세기)
18     myargs[1] = strdup("p3.c");    // 인자: 단어 셀 파일
19     myargs[2] = NULL;          // 배열의 끝 표시
20     execvp(myargs[0], myargs);  // "wc" 실행
21     printf("this shouldn't print out");
22     } else {           // 부모 프로세스는 이 경로를 따라 실행한다 (main)
23         int wc = wait(NULL);
24         printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
25             rc, wc, (int) getpid());
26     }
27     return 0;
28 }
```

〈그림 8.3〉 fork(), wait(), 및 exec() 호출하기 (p3.c)

주소 공간들로 새로운 프로그램의 실행을 위해 다시 초기화된다. 그런 다음 운영체제는 프로세스의 argv와 같은 인자를 전달하여 프로그램을 실행시킨다. 새로운 프로세스를 생성하지는 않는다. 현재 실행 중인 프로그램을(p3) 다른 실행 중인 프로그램으로(wc) 대체하는 것이다. 자식 프로세스가 exec()을 호출한 후에는 p3.c는 전혀 실행되지 않은 것처럼 보인다. exec() 시스템 콜이 성공하게 되면 p3.c는 절대로 리턴하지 않는다.

8.4 왜, 이런 API를?

여기서 이런 의문이 생길 수 있다. 새로운 프로세스를 생성하는 간단한 작업 같은데, 왜 이런 이상한 인터페이스를 사용할까? UNIX의 쉘을 구현하기 위해서는 fork()와 exec()을 분리해야 한다. 그래야만 쉘이 fork()를 호출하고 exec()를 호출하기 전에 코드를 실행할 수 있다. 이때 실행하는 코드에서 프로그램의 환경을 설정하고, 다양한 기능을 준비한다.

쉘은 단순한 사용자 프로그램이다[4]. 쉘은 **프롬프트**를 표시하고 사용자가 무언가 입력하기를 기다린다. 그리고 명령어를 입력한다(예, 실행 프로그램의 이름과 필요한 인자). 대부분의 경우 쉘은 파일 시스템에서 실행 파일의 위치를 찾고 명령어를 실행하기

4) 굉장히 많은 종류의 쉘이 존재한다. 몇 가지 꼽아 보면 tcsh, bash, 그리고 zsh 등이 있다. 그 중 하나를 골라 man 페이지를 읽어 보라. 모든 UNIX 전문가들이 그렇게 한다.

팁: 올바른 선택하기 (Lampson's Law)

그 유명한 "Hints for Computer Systems Design" [Lam83]에서 Lampson이 말한바 같이 "**올바르게 선택하라**. 추상화도 단순함도 올바른 선택을 대체할 수 없다." 올바른 선택을 해야만 한다. 올바른 선택은 다른 어떤 대안보다 나아야 한다. 프로세스 생성을 위한 API를 설계하는 방법은 많다. 그러나 `fork()`와 `exec()`의 조합은 단순하면서 매우 강력하다. Unix 설계자들은 단순하고 올바른 방법으로 구현하였다. Lampson은 매우 자주 올바른 선택을 했기 때문에 그를 기념하여 이 법칙에 그의 이름을 붙였다.

위하여 `fork()`를 호출하여 새로운 자식 프로세스를 만든다. 그런 후 `exec()`의 변형 중 하나를 호출하여 프로그램을 실행시킨 후 `wait()`를 호출하여 명령어가 끝나기를 기다린다. 자식 프로세스가 종료되면 쉘은 `wait()`로부터 리턴하고 다시 프롬프트를 출력하고 다음 명령어를 기다린다.

`fork()`와 `exec()`을 분리함으로써 쉘은 많은 유용한 일을 조금 쉽게 할 수 있다. 다음 예를 생각해 보자.

```
prompt> wc p3.c > newfile.txt
```

위의 예제에서 `wc` 프로그램의 출력은 `newfile.txt`라는 출력 파일로 방향이 **재지정**된다 (〉 표시가 재지정을 나타낸다). 이러한 작업을 수행하는 방법은 간단하다. 자식이 생성되고 `exec()`이 호출되기 전에 **표준 출력**(**standard output**) 파일을 닫고 `newfile.txt` 파일을 연다. 이런 작업을 해 놓으면 곧 실행될 프로그램인 `wc`의 출력은 모두 화면이 아니라 파일로 보내진다.

그림 8.4에 작업을 수행하는 프로그램이 나와 있다. Unix 시스템은 미사용 중인 파일 디스크립터를 0번부터 찾아 나간다. 이 경우, `STDOUT_FILENO`가 첫 번째 사용 가능 파일 디스크립터로 탐색되어, `open()`이 호출될 때 할당된다. 표준 출력 파일을 닫았기 때문이다. 이후 자식 프로세스가 표준 출력 파일 디스크립터를 대상으로 하는 모든 쓰기, 예를 들어 `printf()`에 의한 쓰기는 화면이 아니라 새로 열린 파일로 향하게 된다.

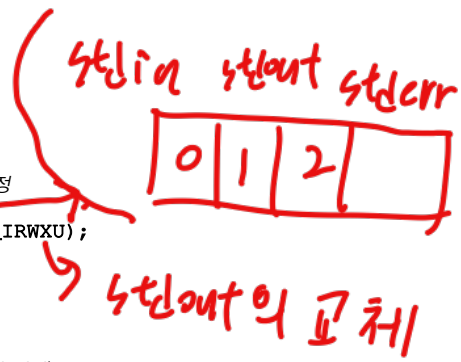
`p4.c`의 실행 결과는 다음과 같다.

```
prompt> ./p4
prompt> cat p4.output
      32      109     846    p4.c
prompt>
```

이 출력 결과에는 두 가지의 흥미로운 점이 있다. 첫째, `p4`를 실행하면, 화면에 아무 일도 일어나지 않는다. 그러나 실제로는 다음과 같은 일이 발생하였다. 프로그램 `p4`는 `fork()`를 호출하여 새로운 자식 프로세스를 생성하고 `execvp()`를 호출하여 `wc` 프로그램을 실행시킨다. 출력이 `p4.output` 파일로 재지정되었기 때문에 화면에는 아무 것도 출력되지 않는다. 출력 파일을 `cat`해 보면 `wc`를 실행시켰을 때 얻을 수 있는 모든 출력이 파일에 저장되어 있는 것을 발견하게 될 것이다. 멋지지 않나?

Unix 파이프가 이와 유사한 방식으로 구현되지만 `pipe()` 시스템 콜을 통하여 생성된다. 이 경우, 한 프로세스의 출력과 다른 프로세스의 입력이 동일한 **파이프**에

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <string.h>
5  #include <fcntl.h>
6  #include <sys/wait.h>
7  int
8  main(int argc, char *argv[])
9  {
10     int rc = fork();
11     if (rc < 0) { // fork 실패함; exit
12         fprintf(stderr, "fork failed\n");
13         exit(1);
14     } else if (rc == 0) {          // 자식: 표준 출력 파일로 재지정
15         close(STDOUT_FILENO);
16         open("./p4.output", O_CREAT|O_WRONLY|O_TRUNC, S_IRWXU);
17
18         // 자 exec "wc"...
19
20
21         char *myargs[3];
22         myargs[0] = strdup("wc");      // 프로그램: "wc"(단어 세기)
23         myargs[1] = strdup("p4.c");    // 인자: 단어 셀 파일
24         myargs[2] = NULL;          // 배열의 끝 표시
25         execvp(myargs[0], myargs);  // wc 실행
26     } else {     // 부모 프로세스는 이 경로를 따라 실행한다 (main)
27         int wc = wait(NULL);
28     }
29     return 0;
30 }
```



〈그림 8.4〉 **입출력 재지정의 모든 것 (p4.c)**

연결된다. 한 프로세스의 출력은 자연스럽게 다음 프로세스의 입력으로 사용되고, 명령어 체인이 형성된다. 파일에서 특정 단어가 몇 번 나오는지 세는 예제를 생각해 보자. 파이프와 `grep`과 `wc`를 사용하면 쉽게 할 수 있다. `grep foo file | wc -l`을 명령어 프롬프트에 입력하고 결과에 놀라기만 하면 된다.

프로세스 API를 간략하게 설명하였지만, 아직 실제로 배우고 이해해야 할 것들이 많이 남아있다. 예를 들면, 파일 디스크립터에 관해서는 이 책의 세 번째 영역인 파일 시스템에 관해서 논의할 때 다루게 될 것이다. 지금 당장은 `fork()`/`exec()` 조합이 프로세스를 생성하고 조작하는 강력한 방법이라는 정도만 이해하면 충분하다.

8.5 여타 API들

UNIX 시스템에는 `fork()`, `exec()`, 및 `wait()` 외에 많은 프로세스 관련 인터페이스가 있다. 예를 들면, `kill()` 시스템 콜은 프로세스에게 **시그널**(signal)을 보내는 데 사용된다. 시그널은 프로세스를 중단시키고(block), 삭제하는 등의 작업에 사용된다. 시그널이라는 운영체제의 메터니즘은 외부 사건을 프로세스에게 전달하는 토대이다. 이 기반 구조는 시그널을 보내거나 전달받는 방법을 모두 포함한다.

유용한 명령어들이 많이 있다. 예를 들면, `ps` 명령어는 어떤 프로세스가 실행 중인지 알아보기 위하여 사용된다. `ps`의 유용한 플래그를 알기 위해서는 **man** 페이지를 읽어 보라. `top` 역시 시스템에 존재하는 프로세스와 그 프로세스가 CPU 및 다른 자원들을

여담: RTFM — man 페이지를 읽어라

특정 시스템 콜이나 라이브러리 콜을 언급할 때 **매뉴얼 페이지** 또는 간략하게 **man** 페이지를 읽으라는 말을 여러 번 들었을 것이다. `man` 페이지는 UNIX 시스템에 존재하는 문서의 원형이다. 이른바 **web**이라고 불리는 것이 존재하기 전에 만들어졌다는 것을 기억하라.

man 페이지를 읽는 것은 시스템 프로그래머로서 성장하는 데 매우 중요한 단계이다. 이 페이지들 속에는 매우 유용한 정보가 셀 수 없이 숨어 있다. 특히, 유용한 페이지는 사용 중인 쉘(예, **tcsh** 또는 **bash**)의 페이지와 프로그램에서 사용한 시스템 콜에(반환값이 무엇이고 어떤 에러 조건이 존재하는지 보기 위하여) 관한 페이지다.

마지막으로 `man` 페이지를 읽으면 당황하는 경우를 줄일 수 있다. `fork()`의 복잡함에 대하여 동료에게 물어보면 "RTFM"이라는 아주 간단한 대답이 돌아올 것이다. `man` 페이지를 읽어야 한다는 것을 강조하는 동료들의 점잖은 표현이다. RTFM의 F는 표현을 더 맛깔나게 한다.

얼마나 사용하고 있는지를 보여 주기 때문에 매우 유용하다. `top` 명령어를 여러 번 실행할 경우 자기 스스로가 가장 많은 자원을 사용하고 있다고 지적하는 것이 재밌는 부분이다. 매우 자기중심적인 명령어라고 할 수 있다. 마지막으로 다양한 CPU 측정기가 제공되어 시스템의 부하 정도를 알 수 있다. 예를 들어, Raging Menace software 사의 **MenuMeter**를 Macintosh의 toolbar에 실행시키면 어느 때건 CPU의 이용률을 점검할 수 있다. 일반적으로 현재 벌어지고 있는 일에 대해 더 많은 정보를 가질수록 더 유리하다.

8.6 요약

UNIX 프로세스를 다루는 API 중 몇 개를 소개하였다. `fork()`, `exec()` 및 `wait()`이 바로 그것이다. 매우 기본적인 것만 살펴보았다. 더 자세한 사항에 관해서는 Stevens and Rago [SR05]를 읽기 바란다. 특히, 프로세스 제어, 프로세스 관계 및 시그널 부분을 읽어 보기 바란다. 그 책에 유용한 내용들이 많이 있다.

참고 문헌

[Con63] **"A Multiprocessor System Design"**
Melvin E. Conway
AFIPS, '63 Fall Joint Computer Conference
New York, USA 1963
멀티 프로세스 시스템의 설계 방법에 관한 초기 논문. 아마도 새 프로세스 생성에 관한 논의 중 `fork()`라는 용어가 처음으로 사용된 논문일 것이다.

[DH66] **"Programming Semantics for Multiprogrammed Computations"**
Jack B. Dennis and Earl C. Van Horn
Communications of the ACM, Volume 9, Number 3, March 1966
멀티 프로그램 컴퓨터 시스템의 기본에 관한 개략을 보여주는 고전 논문. MAC, Multics 및 궁극적으로 Unix 시스템에 영향을 주었다.

[Lam83] **"Hints for Computer Systems Design"**
Butler Lampson
ACM Operating Systems Review, 15:5, October 1983
컴퓨터 시스템의 설계에 관한 Lampson의 그 유명한 조언. 언젠가 한 번은 읽어야 할 논문이자 아마 여러 번 읽어야 할 논문일 것이다.

[SR05] **"Advanced Programming in the Unix Environment"**
W. Richard Stevens and Stephen A. Rago
Addison-Wesley, 2005
Unix API 사용의 미묘한 차이와 절묘함을 발견할 수 있는 책. 책을 사서 읽어라. 그리고 더 중요한 건 항상 곁에 두어야 한다.

숙제 (코드)

이 숙제에서는 방금 읽은 프로세스 관리 API와 친숙해 질 것이다. 걱정하지 마라. 들리는 것 보다는 훨씬 재미있으니까! 일반적으로 코드를 작성하기 위해 더 많은 시간을 투자하면 잘 하게 될 것이다[5]. 자 당장 시작하자.

여담: 코딩 숙제

코딩 숙제는 코드를 작성하여 실제 컴퓨터에서 실행시켜 봄으로써 현대 운영체제가 제공해야 하는 기본 API의 일부를 경험할 수 있는 작은 연습이다. 어쨌든 당신은 (아마도) 컴퓨터 과학자이니까 코딩을 좋아하겠지? 물론 진짜 전문가가 되려면 컴퓨터를 분해하는 시간보다는 더 많은 시간을 투자해야 한다. 사실 코드를 작성하여 실행시켜 동작을 이해할 수 있는 모든 구실을 찾아보아라. 시간을 투자해라. 그리고 될 수 있는 한 현명한 마스터가 되라.

문제

1. `fork()`를 호출하는 프로그램을 작성하라. `fork()`를 호출하기 전에 메인 프로세스는 변수에 접근하고 (예, x) 변수에 값을 지정하라 (예, 100). 자식 프로세스에서 그 변수의 값은 무엇인가? 부모와 자식이 변수 x를 변경한 후에 변수는 어떻게 변했는가?
2. `open()` 시스템 콜을 사용하여 파일을 여는 프로그램을 작성하고 새 프로세스를 생성하기 위하여 `fork()`를 호출하라. 자식과 부모가 `open()`에 의해 반환된 파일 디스크립터에 접근할 수 있는가? 부모와 자식 프로세스가 동시에 파일에 쓰기 작업을 할 수 있는가?
3. `fork()`를 사용하는 다른 프로그램을 작성하라. 자식 프로세스는 "hello"를 출력하고 부모 프로세스는 "goodbye"를 출력해야 한다. 항상 자식 프로세스가 먼저 출력하게 하라. 부모가 `wait()`를 호출하지 않고 할 수 있는가?
4. `fork()`를 호출하고 `/bin/ls`를 실행하기 위하여 `exec()` 계열의 함수를 호출하는 프로그램을 작성하라. `exec()`의 변형 `execl()`, `execle()`, `execlp()`, `execv()`, `execvp()`, `execve()` 모두를 사용할 수 있는지 시도해 보라. 기본적으로는 동일한 기능을 수행하는 시스템 콜에 여러 변형이 있는 이유를 생각해 보라.
5. `wait()`를 사용하여 자식 프로세스가 종료되기를 기다리는 프로그램을 작성하라. `wait()`가 반환하는 것은 무엇인가? 자식 프로세스가 `wait()`를 호출하면 어떤 결과가 발생하는가?

5) 코딩이 싫지만 컴퓨터 과학자가 되기 원한다면 (a) 컴퓨터 과학 이론 분야에서 잘 하거나 (b) 지금까지 들어 왔던 "컴퓨터 과학"이라는 것에 대해 다시 생각해 보아야 한다.

6. 위 문제에서 작성한 프로그램을 수정하여 `wait()` 대신에 `waitpid()`를 사용하라. 어떤 경우에 `waitpid()`를 사용하는 것이 좋은가?

7. 자식 프로세스를 생성하고 자식 프로세스가 표준 출력(`STDOUT_FILENO`)을 닫는 프로그램을 작성하라. 자식이 설명자를 닫은 후에 아무거나 출력하기 위하여 `printf()`를 호출하면 무슨 일이 생기는가?

8. 두 개의 자식 프로세스를 생성하고 `pipe()` 시스템 콜을 사용하여 한 자식의 표준 출력을 다른 자식의 입력으로 연결하는 프로그램을 작성하라.

dup() dup2() 를 알아야 한다

9

제한적 직접 실행 원리

CPU를 가상화하기 위해서 운영체제는 여러 작업들이 동시에 실행되는 것처럼 보이도록 물리적인 CPU를 공유한다. 기본적인 아이디어는 간단하다. 한 프로세스를 잠시 동안 실행하고 다른 프로세스를 또 잠깐 실행하고, 이런 식으로 계속해서 잠깐씩 실행시키면 된다. 이러한 방식처럼 **CPU 시간을 나누어 씀**으로써 가상화를 구현할 수 있다.

그러나 이러한 가상화 기법을 구현하기 위해서는 몇 가지 문제를 해결해야 한다. 첫 번째는 *성능 저하*이다. 시스템에 과중한 오버헤드를 주지 않으면서 가상화를 구현할 수 있을까? 두 번째는 *제어 문제*이다. CPU에 대한 통제를 유지하면서 프로세스를 효율적으로 실행시킬 수 있는 방법은 무엇인가? 운영체제의 입장에서는 자원 관리의 책임자로서 특히 제어 문제가 중요하다. 제어권을 상실하면 한 프로세스가 영원히 실행을 계속할 수 있고 컴퓨터를 장악하거나 접근해서는 안 되는 정보에 접근하게 된다. 제어권을 유지하면서 성능 저하가 없도록 하는 것이 운영체제를 구축하는 데 핵심적인 도전 과제이다.

핵심 질문: 제어를 유지하면서 효과적으로 CPU를 가상화하는 방법

운영체제는 효율적인 방식으로 CPU를 가상화하지만 시스템에 대한 제어를 잃지는 않는다. 제어를 잃지 않기 위해서 하드웨어와 운영체제의 지원이 필수적이다. 종종 운영체제는 작업을 효과적으로 수행하기 위하여 하드웨어가 제공하는 기능을 신중하게 사용한다.

9.1 기본 원리: 제한적 직접 실행

운영체제 개발자들은 프로그램을 빠르게 실행하기 위하여 **제한적 직접 실행**(**Limited Direct Execution**) 이라는 기법을 개발하였다. 이 아이디어의 "직접 실행"에 해당하는 부분은 간단하다. 프로그램을 CPU 상에서 그냥 직접 실행시키는 것이다. 따라서 운영체제가 프로그램을 실행하기 시작할 때 프로세스 목록에 해당 프로세스 항목을 만들고 메모리를 할당하며 프로그램 코드를 디스크에서 탑재하고 진입점(예, `main()` 루틴 혹은 유사한 무엇)을 찾아 그 지점으로 분기하여 사용자 코드를 실행하기 시작한다. 그림

운영체제	프로그램
프로세스 목록의 항목을 생성	
프로그램 메모리 할당	
메모리에 프로그램 탑재	
`argc/argv`를 위한 스택 셋업	
레지스터 내용 삭제	
`call main()` 실행	
	`main()` 실행
	`main`에서 `return` 명령어 실행
프로세스 메모리 반환	
프로세스 목록에서 항목 제거	

〈그림 9.1〉 **직접 실행 프로토콜(제한 없음)**

9.1은 이 기본적인 직접 실행 프로토콜(아무 제한이 없는)을 보이고 있다. 그림 9.1은 프로그램의 `main()`으로 분기하고 커널로 되돌아 가기 위해 일반적인 호출과 리턴을 사용하였다.

간단하다, 그렇지? 그러나 이 접근법은 CPU를 가상화함에 있어 몇 가지 문제를 일으킨다. 첫 번째는 간단한 문제이다. 프로그램을 직접 실행시킨다면 프로그램이, 운영체제가 원치않는 일을 하지 않는다는 것을 어떻게 보장할 수 있는가? 두 번째 문제는 프로세스 실행 시, 운영체제는 어떻게 프로그램의 실행을 중단하고 다른 프로세스로 전환시킬 수 있는가, 즉, CPU를 가상화하는 데 필요한 **시분할(time sharing)** 기법을 어떻게 구현할 수 있느냐는 것이다.

아래에서 이와 같은 질문에 답하면서 CPU를 가상화하는 데 필요한 사항을 더 잘 이해하게 될 것이다. 이러한 기법을 발전시키는 과정 중에 "제한적"이라는 이름이 어디서 비롯되었는지 보게 될 것이다. 프로그램 실행에 제한을 두지 않으면 운영체제는 어떠한 것도 제어할 수 없으며 따라서 단순한 라이브러리일 뿐이다. 장차 운영체제로 발전하기 위해서는 매우 슬픈 상황이다.

9.2 문제점 1: 제한된 연산

직접 실행의 장점은 빠르게 실행된다는 것이다. 기본적으로 프로그램이 하드웨어 CPU에서 실행되기 때문이다. 그러나 CPU에서 직접 실행시키면 새로운 문제가 발생한다. 만일 프로세스가 특수한 종류의 연산을 수행하길 원한다면 어떻게 될 것인가? 이러한 연산에는 디스크 입출력 요청이나 CPU 또는 메모리와 같은 시스템 자원에 대한 추가할당 요청 등이 포함된다.

프로세스가 원하는 대로 할 수 있게 방치하는 방안이 있다. 그러나 이 방안은 바람직한 시스템을 구축하는 데에는 방해 요인이다. 파일에 대한 접근을 허용하기 전에 접근 권한을 검사하는 파일 시스템을 구현하는 것을 예를 들어 보자. 프로세스가 디스크에

핵심 질문: 제한 연산을 수행하는 방법

프로세스는 입출력 연산을 비롯한 다른 제한된 연산을 수행해야 한다. 그러나, 프로세스는 시스템에 대한 권한이 없기 때문에 제한된 연산을 수행할 수 없다. 이 일을 위해 운영체제와 하드웨어가 할 일은 무엇인가?

팁: 보호된 제어 양도

하드웨어는 두 가지 실행 모드를 제공하여 운영체제를 돕는다. **사용자 모드(user mode)**에서 응용 프로그램은 하드웨어 자원에 대한 접근 권한이 일부 제한되어 있다. 운영체제는 컴퓨터의 모든 자원에 대한 접근 권한을 **커널 모드(kernel mode)**에서 가진다. 이를 위하여 커널 모드로 진입하기 위한 trap명령어와 사용자 모드로 돌아가기 위한 `return-from-trap` 명령어가 제공된다. 또한, 운영체제가 하드웨어에게 트랩 테이블(trap table)의 메모리 주소를 알려주기 위한 명령어도 함께 제공된다.

대하여 입출력하는 것을 제한하지 않으면 프로세스는 전체 디스크를 읽고 쓸 수 있기 때문에 접근 권한을 검사하는 기능이 아무런 의미가 없다.

이 때문에 **사용자 모드(user mode)**라고 알려진 새로운 모드가 도입되었다. 사용자 모드에서 실행되는 코드는 할 수 있는 일이 제한된다. 예를 들어, 프로세스가 사용자 모드에서 실행 중이면 입출력 요청을 할 수 없도록 설정한다. 이때 입출력 요청을 하면 프로세서가 예외를 발생시키고, 운영체제는 해당 프로세스를 제거한다.

커널 모드(kernel mode)는 사용자 모드와 대비되는 모드로서 운영체제의 중요한 코드들이 실행된다. 이 모드에서 실행되는 코드는 모든 특수한 명령어를 포함하여 원하는 모든 작업을 수행할 수 있다.

그러나 우리에겐 아직 해결해야 할 한 가지 문제가 남아 있다. 사용자 프로세스가 디스크를 읽기와 같은 특권 명령어를 실행해야 할 때는 어떻게 해야 하는가? 이런 제한 작업의 실행을 허용하기 위하여 거의 모든 현대 하드웨어는 사용자 프로세스에게 **시스템 콜**을 제공한다. Atlas [Kil+62; Lav78]와 같은 커널은 시스템 콜을 통하여 자신의 주요 기능을 사용자 프로그램에게 제공한다. 이러한 기능에는 파일 시스템 접근, 프로세스 생성 및 제거, 다른 프로세스와의 통신 및 메모리 할당 등이 포함된다. 대부분의 운영체제는 수백 개의 시스템 콜을 제공한다(자세한 사항에 관해서는 POSIX 표준을 참조하시오 [P10]). 초기 Unix 시스템은 약 20개 정도의 시스템 콜을 제공하였다.

시스템 콜을 실행하기 위해 프로그램은 **trap** 특수 명령어를 실행해야 한다. 이 명령어는 커널 안으로 분기하는 동시에 특권 수준을 커널 모드로 상향 조정한다. 커널 모드로 진입하면 운영체제는 모든 명령어를 실행할 수 있고 이를 통하여 프로세스가 요청한 작업을 처리할 수 있다. 완료되면 운영체제는 **return-from-trap** 특수 명령어을 호출한다. 예상하는 것처럼 이 명령어는 특권 수준을 사용자 모드로 다시 하향 조정하면서 호출한 사용자 프로그램으로 리턴한다.

하드웨어는 `trap` 명령어를 수행할 때 주의가 필요하다. 호출한 프로세스의 필요한

여담: 왜 시스템 콜은 프로시저 콜과 비슷하게 보일까

`open()` 또는 `read()`와 같은 시스템 콜을 호출하는 형태는 C 프로그램의 전형적인 프로시저 호출의 형태와 똑같다. 즉, 만약 프로시저 호출과 똑같다면 운영체제는 시스템 콜을 어떻게 구분할 수 있으며 그에 맞는 작업을 할 수 있는가? 간단한 이유: 진짜 프로시저 호출이지만 유명한 trap 명령어가 그 안에 감추어져 있다. 더 정확하게 설명하면, 예를 들어, `open()`을 호출할 때, C 라이브러리의 프로시저 호출을 수행하고 있는 것이다. `open()`(다른 시스템 콜들도 마찬가지이다)이 호출되면, 라이브러리는 커널과 약속된 호출 규약을 사용하여 `open`에게 전달할 인자와 시스템 콜 번호를 지정된 장소(스택 또는 특정 레지스터)에 저장한다. 그런 다음 `trap` 명령어를 실행한다. 그 다음에 반환 값을 정리하고 시스템 콜을 호출한 프로그램에게 제어를 넘긴다. 이를 위해서 C 라이브러리에서 시스템 콜을 호출하는 부분의 코드는 어셈블리어로 작성된다. 인자의 반환 값을 올바르게 처리하고, 하드웨어마다 다른 `trap` 명령어를 실행하기 위해 꼭 필요한 일이다. 자 이제 운영체제로 진입하기 위해서 어셈블리 코드 작업을 직접할 필요가 없다는 것을 알았다. 어셈블리 코드는 이미 만들어져 있다.

레지스터들을 저장해야 한다. 운영체제가 `return-from-trap` 명령어 실행 시 사용자 프로세스로 제대로 리턴할 수 있도록 하기 위함이다. 예를 들면, x86에서는 프로그램 카운터, 플래그와 다른 몇 개의 레지스터를 각 프로세스의 **커널 스택**(**kernel stack**)에 저장한다. `return-from-trap` 명령어가 이 값들을 스택에서 팝(pop)하여 사용자 모드 프로그램의 실행을 다시 시작한다(Intel systems manual [Int11]을 참조하시오). 다른 하드웨어 시스템은 다른 규약을 사용하긴 하지만 기본적인 개념은 모두 비슷하다.

현재까지의 논의에서 다루지 않은 중요한 사항이 있다. 그것은 `trap`이 운영체제 코드의 어디를 실행할지 어떻게 아느냐는 것이다. 호출한 프로세서는 분기할 주소를 명시할 수 없다. 주소를 명시한다는 것은 커널 내부의 원하는 지점을 접근할 수 있다는 것이기 때문에 위험하다. 커널이 임의의 코드를 실행하기 위해서는 접근 권한 검사가 끝난 후 분기해야 한다 [Sha07]. 이러한 문제 때문에 커널은 trap 발생 시 어떤 코드를 실행할지 신중하게 통제해야 한다.

커널은 부팅 시에 **트랩 테이블**(**trap table**)을 만들고 이를 이용하여 시스템을 통제한다. 컴퓨터가 부트될 때는 커널 모드에서 동작하기 때문에 하드웨어를 원하는 대로 제어할 수 있다. 운영체제가 하는 초기 작업 중 하나는 하드웨어에게 예외 사건이 일어났을 때 어떤 코드를 실행해야 하는지 알려주는 것이다. 예를 들어, 하드 디스크 인터럽트가 발생하면, 키보드 인터럽트가 발생하면, 또는 프로그램이 시스템 콜을 호출하면 무슨 코드를 실행해야 하는가? 운영체제는 특정 명령어를 사용하여 하드웨어에게 **트랩 핸들러**(**trap handler**)의 위치를 알려준다. 하드웨어는 이 정보를 전달받으면 해당 위치를 기억하고 있다. 따라서 시스템 콜과 같은 예외적인 사건이 발생했을 때 하드웨어는 무엇을 해야 할지(즉, 어느 코드로 분기하여 실행할지) 알 수 있다.

마지막 하드웨어에게 트랩 테이블의 위치를 알려주는 것은 매우 **강력한** 기능이다. 당연히 이 역시 특권 명령어이다. 사용자 모드에서 이 명령어를 실행하려고 하면 실행할

운영체제 @부트 **(커널 모드)**	**하드웨어**	
트랩 테이블을 초기화한다		
	syscall 핸들러의 주소를 기억한다	
운영체제 @실행 **(커널 모드)**	**하드웨어**	**프로그램** **(사용자 모드)**
프로세스 목록에 항목을 추가한다 프로그램을 위한 메모리를 할당한다 프로그램을 메모리에 탑재한다 argv를 사용자 스택에 저장한다 레지스터와 PC를 커널 스택에 저장한다 **return-from-trap**		
	커널 스택으로부터 레지스터를 복원한다 사용자 모드로 이동한다 main으로 분기한다	
		`main()`을 실행한다 ... 시스템 콜을 호출한다 운영체제로 **트랩**한다
	레지스터를 커널 스택에 저장한다 커널 모드로 이동한다 트랩 핸들러로 분기한다	
트랩을 처리한다 syscall의 임무를 수행한다 **return-from-trap**		
	커널 스택으로부터 레지스터를 복원한다 사용자 모드로 이동한다 트랩 이후의 PC로 분기한다	
		main에서 리턴한다 **trap**(**`exit()`**를 통하여)
프로세스의 메모리를 반환한다 프로세스 목록에서 제거한다		

〈그림 9.2〉 **제한적 직접 실행 프로토콜**

수 없고, 어떤 결과가 올지 알고 있을 것이다(힌트, 잘 가거라. 프로그램). 숙고해 볼 주제: 만약 당신이 자신의 트랩 테이블을 설치할 수 있다면, 시스템에 어떤 황당한 일이 일어날까? 컴퓨터를 장악할 수 있는가?

그림 9.2는 이 메커니즘을 요약해서 나타내고 있다. 연대표는 시간의 흐름에 따라 아래 방향으로 진행된다. 프로세스는 커널 스택을 각자 가지고 있다. 커널 모드로 진입하거나 진출할 때 하드웨어에 의해 프로그램 카운터와 범용 레지스터 등의 레지스터가 저장되고 복원되는 용도로 사용된다.

LDE 프로토콜은 두 단계로 진행된다. 전반부에서(부팅 시) 커널은 트랩 테이블을 초기화하고 CPU는 나중에 사용하기 위하여 테이블의 위치를 기억한다. 커널은 이러한 작업을 커널 모드에서만 사용할 수 있는 명령어를 이용하여 수행한다(이러한 명령어는 굵게 강조하였다).

후반부에서(프로세스를 실행할 때) return-from-trap을 이용하여 사용자 프로세스를 시작할 때 몇 가지 작업을 수행한다. 새로운 프로세스를 위한 노드를 할당하여 프로세스 리스트에 삽입하고, 메모리를 할당하는 등의 작업이 포함된다. `return-from-trap` 명령어는 CPU를 사용자 모드로 전환하고 프로세스 실행을 시작한다. 프로세스가 시스템 콜을 호출하면 운영체제로 다시 트랩된다. 운영체제는 시스템 콜을 처리하고 `return-from-trap` 명령어를 사용하여 다시 제어를 프로세스에게 넘긴다. 프로세스는 이후 자신의 할 일을 다하면 `main()`에서 리턴한다. 이때 일반적으로 스텁 코드로 리턴하고 스텁 코드가 프로그램을 종료시킨다. 종료시킬 때 `exit()` 시스템을 호출하고 다시 운영체제로 트랩된다. 이 시점에 운영체제는 정리 작업을 하게 되어 모든 일이 완료된다.

9.3 문제점 2: 프로세스 간 전환

직접 실행의 두 번째 문제점은 프로세스 간 전환을 할 수 있어야 한다는 점이다. 프로세스 간 전환은 당연히 간단해야 한다. 맞지? 운영체제는 실행 중인 프로세스를 계속 실행할 것인지, 멈추고 다른 프로세스를 실행할 것인지를 결정해야 한다. 이게 뭐가 문제지? 실제로는 까다로운 문제이다. CPU에서 프로세스가 실행 중이라는 것은 운영체제는 실행 중이지 않다는 것을 의미한다. 운영체제가 실행하고 있지 않다면 어떻게 이런 일들을 할 수 있을까? (힌트: 할 수 없다). 거의 철학적으로 들릴지 모르겠지만 이건 진짜 심각한 문제다. CPU에서 실행하고 있지 않다면 운영체제는 어떠한 조치도 취할 수 없다. 이제 핵심 문제에 도달했다.

핵심 질문: CPU를 어떻게 다시 획들할 수 있는가
운영체제는 어떻게 CPU를 **다시 획득하여** 프로세스를 전환할 수 있는가?

협조 방식: 시스템 콜 기다리기

협조(cooperative) 방식으로 알려진 방법은 과거의 몇몇 시스템에서 채택되었던 방식이다. 예를 들어, Macintosh 운영체제의 초기 버전 [Mac11], 또는 오래 전 Xerox Alto 시스템 [Alt79]이 채택했던 방식이다. 이 방식에서는 운영체제가 프로세스들이 합리적으로 행동할 것이라고 신뢰한다. 너무 오랫동안 실행할 가능성이 있는 프로세스는 운영체제가 다른 작업을 실행할 결정을 할 수 있도록 주기적으로 CPU를 포기할 것이라고 가정한다.

그러면 이렇게 질문할 수 있을 것이다: 이 이상적인 환경에서 우호적인 프로세스는 어떻게 CPU를 포기할 수 있을까? 알려진 것처럼 대부분의 프로세스는 자주 **시스템 콜**을 호출하여 CPU의 제어권을 운영체제에게 넘겨준다. 예를 들어, 파일을 열고 읽는 작업을 한다거나 다른 컴퓨터에게 메시지를 송신하거나 또는 새 프로세스를 생성하는 등의 시스템 콜을 호출한다. 이런 유형의 운영체제는 **yield** 시스템 콜을 제공하는데,

이 시스템 콜은 운영체제에게 제어를 넘겨 운영체제가 다른 프로세스를 실행할 수 있게 한다.

팁: 응용 프로그램의 오작동 처리하기

운영체제는 종종 오작동 프로세스를 처리해야 한다. 오작동하는 프로세스란 악의적으로 설계되었거나 실수로 인한 버그 때문에 해서는 안 될 무언가를 하려는 프로세스를 말한다. 현대 시스템에서 운영체제가 그러한 부정 행위를 처리하는 방법은 단순히 행위자를 종료시키는 것이다. 원 스트라이크에 아웃! 조금 가혹하다고 할지는 모르겠지만, 만약 메모리에 불법적으로 접근하거나 불법적인 명령어를 실행할 때 운영체제가 취할 수 있는 일은 무엇이 있을까?

응용 프로그램이 비정상적인 행위를 하게 되면 운영체제에게 제어가 넘어간다. 예를 들어 응용 프로그램이 어떤 수를 0으로 나누는 연산을 실행하거나 접근할 수 없는 메모리에 접근하려고 하면 운영체제로의 **트랩**이 일어난다. 그러면 운영체제는 다시 CPU를 획득하여 해당 행위를 하는 프로세스를 종료할 수 있다.

협조 방식의 스케줄링 시스템에서 운영체제는 시스템 콜이 호출되기를 기다리거나 불법적인 연산이 일어나기를 기다려서 CPU의 제어권을 다시 획득한다. 이렇게 생각할 수도 있다: 이상적이라기보다 수동적인 방법 아닐까? 예를 들어 악의적이든 버그로 인한 것이든 프로세스가 무한 루프에 빠져 시스템 콜을 호출할 수 없을 때에는 어떤 일이 벌어지는가? 운영체제가 할 수 있는 일은 무엇인가?

비협조 방식: 운영체제가 전권을 행사

프로세스가 시스템 콜을 호출하기를 거부하거나 실수로 호출하지 않아 운영체제에게 제어를 넘기지 않을 경우 하드웨어의 추가적인 도움없이는 운영체제가 할 수 있는 일은 거의 없다. 사실, 협조적 방법에서 프로세스가 무한 루프에 빠졌을 경우 할 수 있는 일은 컴퓨터 시스템의 모든 문제를 해결해 왔던 아주 오래된 방법을 동원하는 수밖에 없다. 바로 **컴퓨터를 다시 부팅**하는 것이다. CPU의 제어권을 획득하기 위한 과정에서 해결해야 할 사소한 문제를 만나게 된다.

핵심 질문: 협조 없이 제어를 얻는 방법

프로세스가 비협조적인 상황에서도 CPU의 제어를 획득하는 방법은 무엇인가? 악의적인 프로세스가 컴퓨터를 장악하지 않도록 보장하기 위하여 운영체제는 무엇을 할 수 있을까?

그 해결책은 간단한 것으로 수십 년 전에 컴퓨터 시스템을 만들었던 많은 사람들에 의해 발명되었다. 해결책은 **타이머 인터럽트**(**timer interrupt**)를 이용하는 것이다 [McC+63]. 타이머 장치는 수 밀리 초마다 인터럽트를 발생시키도록 프로그램

가능하다. 인터럽트가 발생하면 현재 수행 중인 프로세스는 중단되고 미리 구성된 운영체제의 **인터럽트 핸들러(interrupt handler)**가 실행된다. 이 시점에 운영체제는 CPU 제어권을 다시 얻게 되고 자신이 원하는 일을 할 수 있다. 현재의 프로세스를 중단하고 다른 프로세스를 실행시키는 작업 등이 해당된다.

팁: 제어를 다시 획득하기 위해 타이머 인터럽트 사용하기

타이머 인터럽트 기능을 추가하면 운영체제는 프로세스가 비협조적으로 행동하는 상황에서도 CPU 상에서 실행될 수 있는 능력을 가지게 된다. 타이머 인터럽트 하드웨어 기능은 운영체제가 컴퓨터의 제어를 유지하는 핵심적인 기능이다.

운영체제는 하드웨어에게 타이머 인터럽트가 발생했을때 실행해야 할 코드[1]를 알려주어야 한다. 부팅될 때 운영체제가 이런 준비를 한다. 부팅 과정 진행 중에 운영체제는 타이머를 시작한다. 타이머가 시작되면 운영체제는 자신에게 제어가 돌아올 것이라는 것을 알고 부담 없이 사용자 프로그램을 실행할 수 있다. 또한, 타이머는 특정 명령어를 수행하여 끌 수도 있다. 이에 대한 자세한 사항은 병행성을 설명할 때 논의할 것이다.

인터럽트 발생 시 하드웨어에게도 약간의 역할이 있다. 인터럽트가 발생했을 때 실행중이던 프로그램의 상태를 저장하여 나중에 `return-from-trap` 명령어가 프로그램을 다시 시작할 수 있도록 해야 한다. 이러한 일련의 동작은 시스템 콜이 호출되었을 때 하드웨어가 하는 동작과 매우 유사하다. 다양한 레지스터가 커널 스택에 저장되고, `return-from-trap` 명령어를 통하여 복원된다.

문맥의 저장과 복원

시스템 콜을 통하여 협조적으로 하던, 또는 타이머 인터럽트를 통하여 약간은 강제적으로 하던, 운영체제가 제어권을 다시 획득하면 중요한 결정을 내려야 한다. 즉, 현재 실행중인 프로세스를 계속 실행할 것인지 아니면 다른 프로세스로 전환할 것인지를 결정해야 한다. 이 결정은 운영체제의 **스케줄러(scheduler)**라는 부분에 의해 내려진다. 우리는 이후의 장에서 스케줄링 정책에 관해 상세하게 논의할 것이다.

다른 프로세스로 전환하기로 결정되면 운영체제는 **문맥 교환(context switch)**이라고 알려진 코드를 실행한다. 문맥 교환은 개념적으로는 간단하다. 운영체제가 해야 하는 작업은 현재 실행 중인 프로세스의 레지스터 값을 커널 스택 같은 곳에 저장하고 곧 실행될 프로세스의 커널 스택으로부터 레지스터 값을 복원하는 것이 전부다. 그렇게 함으로써 운영체제는 `return-from-trap` 명령어가 마지막으로 실행될 때 현재 실행중이던 프로세스로 리턴하는 것이 아니라 다른 프로세스로 리턴하여 실행을 다시 시작할 수 있다.

프로세스 전환을 위하여 운영체제는 저수준 어셈블리 코드를 사용하여 현재 실행중인 프로세스의 범용 레지스터, PC뿐 아니라 현재 커널 스택 포인터를 저장한다.

1) 특정 작업을 대처하는 코드

그리고 곧 실행될 프로세스의 범용 레지스터, PC를 복원하고 커널 스택을 이 프로세스의 커널 스택으로 전환한다. 이로써 운영체제는 인터럽트된 프로세스 문맥에서 전환 코드를 호출하고, 실행될 프로세스 문맥으로 리턴할 수 있다. 운영체제가 마지막으로 `return-from-trap` 명령어를 실행하면 곧 실행될 프로세스가 현재 실행 중인 프로세스가 된다. 그래서 문맥 교환이 마무리 된다.

이 모든 과정의 연대표가 그림 9.3에 나와 있다. 이 예에서 프로세스 A가 실행 중이고 타이머 인터럽트에 의해 중단된다. 하드웨어는 A의 레지스터를 커널 스택에 저장하고 커널 모드로 진입한다. 타이머 인터럽트 핸들러에서 운영체제는 프로세스 B로 전환하기로 결정한다. 이 시점에 `switch()` 루틴을 호출한다. 이 루틴이 A의 레지스터의 현재 값을 A의 프로세스 구조체에 저장하고 B의 프로세스 구조체에서 B의 레지스터를 복원한다. 그런 후에 A의 커널 스택이 아니라 B의 커널 스택을 사용하도록 스택 포인터를 바꾸어서 **문맥 교환**을 수행한다. 마지막으로 운영체제는 `return-from-trap`을 수행하여 B의

운영체제 @부트 (커널 모드)	**하드웨어**	
트랩 테이블을 초기화 한다		
	syscall 핸들러, 타이머 핸들러의 주소를 기억한다	
인터럽트 타이머를 시작시킨다		
	타이머를 시작시킨다 X msec 지난 후 CPU를 인터럽트한다	
운영체제 @실행 (커널 모드)	**하드웨어**	**프로그램 (사용자 모드)**
		프로세스 A …
	타이머 인터럽트 A의 레지스터를 A의 커널 스택에 저장 커널 모드로 이동 트랩 핸들러로 분기	
트랩을 처리한다 **switch()** 루틴 호출 A의 레지스터를 A의 proc 구조에 저장 B의 proc 구조로부터 B의 레지스터를 복원 B의 커널 스택으로 전환 **return-from-trap (B 프로세스로)**		
	B의 커널 스택을 B의 레지스터로 저장 사용자 모드로 이동 B의 PC로 분기	
		프로세스 B …

〈그림 9.3〉 **제한적 직접 실행 프로토콜(타이머 인터럽트)**

```
1  # void switch(struct context **old, struct context *new);
2  #
3  # Save current register context in old
4  # and then load register context from new.
5  .globl switch
6  switch:
7      # Save old registers
8      movl 4(%esp), %eax      # old포인터를 eax에 넣는다
9      popl 0(%eax)            # old IP를 저장한다
10     movl %esp, 4(%eax)      # 그리고 스택
11     movl %ebx, 8(%eax)      # 그리고 다른 레지스터
12     movl %ecx, 12(%eax)
13     movl %edx, 16(%eax)
14     movl %esi, 20(%eax)
15     movl %edi, 24(%eax)
16     movl %ebp, 28(%eax)
17
18     # Load new registers
19     movl 4(%esp), %eax      # new포인터를 eax에 넣는다
20     movl 28(%eax), %ebp     # 다른 레지스터를 복원한다
21     movl 24(%eax), %edi
22     movl 20(%eax), %esi
23     movl 16(%eax), %edx
24     movl 12(%eax), %ecx
25     movl 8(%eax), %ebx
26     movl 4(%eax), %esp      # 스택은 이 지점에서 전환된다
27     pushl 0(%eax)           # 리턴 주소를 지정된 장소에 넣는다
28     ret                     # 마지막으로 new문맥으로 리턴한다
```

〈그림 9.4〉 **xv6 문맥 교환 코드**

레지스터를 복원하고 실행을 시작한다.

이 과정이 실행되는 동안 두 번의 레지스터의 저장/복원이 일어난다는 것에 주의하자. 첫 번째는 타이머 인터럽트가 발생했을 때 일어난다. 이 경우 실행 중인 프로세스의 *사용자 레지스터가 하드웨어에* 의해 암묵적으로 저장되고 저장 장소로 해당 프로세스의 커널 스택이 사용된다. 두 번째는 운영체제가 A에서 B로 전환하기로 결정했을 때 일어난다. 이 경우 *커널 레지스터는 운영체제에* 의하여 해당 프로세스의 프로세스 구조체에 저장된다. 이것은 운영체제가 A가 아닌 B로부터 커널로 트랩된 것처럼 만든다.

이러한 문맥 교환이 어떻게 일어나는지 잘 이해할 수 있도록 xv6의 문맥 교환 코드를 그림 9.4에 보이고 있다. 우선 이해할 수 있는지 살펴보기 바란다. 물론 xv6와 x86에 관한 얼마간의 지식을 가지고 있어야 한다. `context` 구조체 `old`와 `new`는 구 프로세스와 새 프로세스의 프로세스 구조체 안에 위치해 있다.

9.4 병행성이 걱정

집중력과 사고력이 좋은 몇몇 친구는 지금 이런 생각을 하고 있을지 모른다. "음, 만약 시스템 콜을 처리하는 도중에 타이머 인터럽트가 발생하면 어떤 일이 발생하는가?" 또는 "하나의 인터럽트를 처리하고 있을 때 다른 인터럽트가 발생하면 어떤 일이 생기는가? 커널에서 처리하기 더 어려워지는 것은 아닌가?" 좋은 질문. 우리에겐 아직 희망이 있다!

대답은 "그렇다" 이다. 운영체제는 인터럽트 또는 트랩을 처리하는 도중에 다른 인터럽트가 발생할 때 어떤 일이 생기는가에 대해 신중하게 고려할 필요가 있다. 사실

여담: 문맥 교환하는 데 걸리는 시간은 얼마인가

떠올릴 수 있는 자연스러운 질문은 문맥 교환과 같은 작업의 처리 소요 시간이다. 혹은 시스템 콜의 처리 소요 시간은? 호기심 많은 당신을 위해 lmbench [MS96]라는 도구가 있다. 이 도구는 언급한 그러한 작업의 시간을 정확하게 측정할 뿐 아니라 연관된 다른 성능 수치도 측정한다.

결과는 시간이 지나면서 대략 프로세서의 성능개선 추이와 비슷하게 점점 좋아졌다. 예를 들어, 1996년에 200 MHz P6 CPU에서 실행되는 Linux 1.3.37의 경우 시스템 콜은 4마이크로초 그리고 문맥 교환에는 대략 6마이크로초가 소요되었다 [MS96]. 현대 시스템에서는 자릿수가 달라질 정도로 성능이 좋아져서 2 또는 3 GHz 프로세서의 경우 1마이크로초 미만이 소요된다.

운영체제의 모든 동작이 CPU 성능에 따라 좋아지는 것이 아니라는 것에 주의하자. Ousterhout가 발견한 것처럼 운영체제의 많은 연산은 주로 메모리를 접근하는 연산이며 메모리의 대역폭은 프로세서 속도가 발전하는 것만큼 극적으로 향상되지 않았기 때문이다 [Ous90]. 사용한 워크로드의 특성에 따라 최신의 좋은 프로세서를 구매하더라도 운영체제의 속도가 기대만큼 증가하지 않을 수 있다.

이 내용이 바로 이 책의 두 번째 부분에서 논의할 주제인 **병행성**에 관한 것으로서 자세한 논의는 그때까지 미룰 것이다.

흥미를 돋우기 위하여 운영체제가 이 다루기 힘든 상황을 어떻게 처리하는지 간략히 알아보자. 운영체제가 할 수 있는 간단한 해법은 인터럽트를 처리하는 동안 **인터럽트를 불능화**시키는 것이다. 이럴 경우 하나의 인터럽트가 처리되고 있는 동안 다른 어떤 인터럽트도 CPU에게 전달되지 않는다. 물론, 운영체제는 이 작업을 신중하게 해야 한다. 인터럽트를 너무 오랫동안 불능화시키면 인터럽트를 놓치게 되고 기술적으로도 좋지 않다.

운영체제는, 또한 내부 자료 구조에 동시에 접근하는 것을 방지하기 위해 많은 정교한 **락(lock)** 기법을 개발해 왔다. 이 잠금 기법은 커널 안에서 동시에 다수의 활동이 진행될 수 있게 허용한다. 이 책의 병행성에 관한 부분에서 살펴보겠지만 이런 잠금 기법은 복잡해질 수 있으며 그로 인해 흥미로우면서도 발견하기 힘든 버그를 만들어 낸다.

9.5 요약

우리는 CPU 가상화를 구현하기 위한 핵심적인 저수준 기법에 관해 설명하였다. 이런 기법들을 묶어서 **제한적 직접 실행**이라고 부른다. 기본적인 아이디어는 간단하다. CPU에서 실행하고 싶은 프로그램을 실행시킨다. 그러나 운영체제가 CPU를 사용하지 못하더라도 프로세스의 작업을 제한할 수 있도록 하드웨어를 셋업해야 한다.

이러한 일반적인 방법은 실생활에서도 사용된다. 예를 들어, 아이가 있거나 적어도 아이들에 관해 들어 보았다면 방에 **아기 보호 장치**를 설치하는 일에 친숙할 것이다. 위험한 물건이 들어 있는 캐비닛을 잠그고 전기 소켓의 두껑을 덮는다거나 하는 일이

팁: 리부팅은 유용하다

협조적 선점 모드에서 무한루프와 같은 행동의 유일한 해결책은 컴퓨터를 **리부팅**하는 것이다. 단순 무식한 방법이지만, 연구가들은 리부팅이나 소프트웨어를 다시 시작하는 방법이 강건한 시스템 구축에 있어 매우 유용하다는 것을 보였다 [Can+04].

구체적으로 리부트는 소프트웨어를 이미 알려진 상태 그리고 검증된 상태로 되돌리기 때문에 유용하다. 리부트는 오래된 자원 또는 제어를 벗어난 자원을 시스템에 반환한다. 이러한 자원은 반환하지 않으면 처리하기 곤란하다. 마지막으로 리부트는 자동화하기 쉽다. 이러한 이유들 때문에 시스템 관리 소프트웨어를 위한 대규모 클러스터 인터넷 서비스에서는 컴퓨터 일부를 리셋시키기 위하여 주기적으로 리부팅하는 것이 흔한 일이다. 이렇게 함으로써 앞서 언급한 모든 이점을 얻을 수 있다.

따라서 리부팅한다고 해서 그것이 단순무식한 방법은 아니다. 오히려 컴퓨터 시스템의 동작을 향상시키기 위하여 충분히 검증된 방법을 사용하고 있는 것이다. 잘했어!

포함된다. 이렇게 방이 준비되면 대부분의 위험한 요소들을 막아 놓았기 때문에 아기가 자유롭고 안전하게 돌아다니게 할 수 있다.

비슷한 방식으로 운영체제는 CPU에 안전 장치를 준비해 놓을 수 있다. 우선 부팅할 때 트랩 핸들러 함수를 셋업하고 인터럽트 타이머를 시작시키고 그런 후에 제한된 모드에서만 프로세스가 실행되도록 한다. 이로써 운영체제는 프로세스를 효율적으로 실행할 수 있고, 특별한 연산을 수행할 때, 즉 프로세스가 CPU를 독점하거나, 다른 프로세스로 전환해야 할 때만 개입한다.

이리하여 우리는 CPU 가상화를 위한 기본 개념을 학습하였다. 중요한 질문이 그대로 남았다. 특정 시점에 어떤 프로세스를 실행시켜야 하는가? 바로 스케줄러가 답해야 하는 질문이고 우리가 공부할 다음 주제이다.

참고 문헌

[Alt79] **"Alto User's Handbook"**
Xerox Palo Alto Research Center, September 1979
URL: `http://history-computer.com/Library/AltoUsersHandbook.pdf`
시대를 앞선 환상적인 시스템. Steve Jobs가 방문하고 기록하고 Lisa, 궁극적으로 Mac을 구현하였기 때문에 유명해졌다.

[Can+04] **"Microreboot---A Technique for Cheap Recovery"**
George Candea, Shinichi Kawamoto, Yuichi Fujiki, Greg Friedman, and Armando Fox
OSDI '04, San Francisco, CA, December 2004
강인한 시스템을 구축하는 데 리부트를 어디까지 활용할 수 있는지를 지적한 훌륭한 논문.

[Int11] **"Intel 64 and IA-32 Architectures Software Developer's Manual"**
Volume 3A and 3B: System Programming Guide, Intel Corporation, January 2011

[Kil+62] **"One-Level Storage System"**
T. Kilburn, D.B.G. Edwards, M.J. Lanigan, and F.H. Sumner
IRE Transactions on Electronic Computers, April 1962
Atlas는 현대 시스템에서 우리가 볼 수 있는 많은 것들을 개척하였다. 그러나 이 논문이 읽을 만한 가장 좋은 논문은 아니다. 한편의 논문을 읽어야 한다면 아래에 있는 역사적 관점의 논문을 읽는 것이 좋을 것이다 [Lav78].

[Lav78] **"The Manchester Mark I and Atlas: A Historical Perspective"**
S.H. Lavington
Communications of the ACM, 21:1, January 1978
초창기 컴퓨터 개발과 Atlas의 개척적인 노력에 관한 역사.

[Mac11] **"Mac OS 9"**
January 2011
URL: `http://en.wikipedia.org/wiki/Mac_OS_9`

[McC+63] **"A Time-Sharing Debugging System for a Small Computer"**
J. McCarthy, S. Boilen, E. Fredkin, and J.C.R. Licklider
AFIPS '63 (Spring), May, 1963, New York, USA
타이머 인터럽트 사용에 관해 언급한 시분할에 관한 초기 논문. 그 논의를 인용하면 "채널 17 클록 루틴의 기본 임무는 코어에서 현재 사용자를 제거할지 결정하는 것이고 제거하기로 결정한 경우 대신에 어느 사용자 프로그램이 스왑-인 될지 결정하는 것이다."

[MS96] **"lmbench: Portable tools for performance analysis"**
Larry McVoy and Carl Staelin
USENIX Annual Technical Conference, January 1996
운영체제와 그의 성능에 관해 여러 측면을 어떻게 측정하느냐에 관한 흥미로운 논문. lmbench를 다운로드하여 한 번 측정해 보라.

[Ous90] **"Why Aren't Operating Systems Getting Faster as Fast as Hardware?"**
J. Ousterhout
USENIX Summer Conference, June 1990
운영체제 성능의 본질에 관한 고전적인 논문.

[P10] **"The Single Unix Specification, Version 3"**
The Open Group, May 2010
URL: `http://www.unix.org/version3/`
읽기에 어렵고 힘든 문서. 가능하다면 읽지 말도록.

[Sha07] **"The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls (on the x86)"**
Hovav Shacham
CCS '07, October 2007
연구 중에 가끔 만나게 되는 경이로우면서 놀라운 아이디어 중 하나. 저자는 임의의 위치로 분기할 수 있다면 주어진 커다란 코드 베이스에서 원하는 코드 시퀀스를 만들어 낼 수 있다는 것을 보인다. 자세한 사항은 논문을 읽도록. 안타깝게도 이 기법은 악의적인 공격에 대한 방어를 훨씬 어렵게 만든다.

숙제 (측정)

여담 : 측정 숙제

측정 숙제는 운영체제 또는 하드웨어 성능의 특정 측면을 측정하기 위해 실제 컴퓨터에서 실행시킬 코드를 작성하는 작은 연습이다. 이 숙제의 의도에는 운영체제를 직접 경험해 보자는 의미가 있다.

이 숙제에서 시스템 콜과 문맥 교환의 비용을 측정할 것이다. 시스템 콜의 비용을 측정하는 것은 상대적으로 쉽다. 예를 들어, 간단한 시스템 콜을(0 바이트 읽기 등) 반복적으로 호출하여 통틀어 걸린 시간을 측정한다. 이 시간을 반복 횟수로 나누면 시스템 콜의 비용을 추정할 수 있다.

한 가지 고려해야 할 사항은 타이머의 정확도와 정밀도이다. 사용할 수 있는 전형적인 타이머는 `gettimeofday()`이다. 자세한 사항에 대해서는 `man` 페이지를 참조하도록. `man` 페이지에서 볼 수 있는 정보는 `gettimeofday()`이 1970년 이후 경과한 시간을 마이크로초 단위로 반환한다는 것이다. 그러나 이 사실이 타이머가 마이크로초 수준의 정밀도를 가진다는 것을 의미하지는 않는다. 타이머가 얼마나 정밀한지를 알고 싶다면 `gettimeofday()`를 연속해서 호출하여 측정하라. 이 결과로부터 만족스러운 측정 결과를 얻기 위해서는 널 시스템 콜 테스트를 몇 번 반복해야 하는지 알 수 있을 것이다. `gettimeofday()`가 원하는 만큼 정밀도를 보이지 않는다면 x86 CPU에서 제공되는 `rdtsc` 명령어를 사용하는 것을 검토해 보아야 한다.

문맥 교환의 비용을 측정하는 것은 더 곤란하다. lmbench 벤치마크는 두 개의 프로세스를 하나의 CPU에서 실행시키고 둘 사이에 Unix 파이프를 설정하여 문맥 교환 비용을 측정한다. 파이프는 Unix 시스템에서 프로세스끼리 통신할 수 있는 여러 방법 중의 하나이다. 첫 번째 프로세스는 첫 번째 파이프에 데이터를 쓰고, 두 번째 파이프로부터 데이터가 전달되기를 기다린다. 첫 번째 프로세스가 두 번째 파이프로부터 읽을 무언가를 기다린다는 사실을 알게 되면 운영체제는 첫 번째 프로세스를 봉쇄 상태로 만들고 다른 프로세스로 전환한다. 이 프로세스는 첫 번째 파이프로부터 데이터를 읽고 두 번째 파이프에 데이터를 쓴다. 두 번째 프로세스가 첫 번째 파이프로부터 다시 데이터를 읽으려고 하면 봉쇄 상태로 들어간다. 이런 식으로 주고 받는 통신 사이클이 계속된다. 이러한 통신 비용을 반복적으로 측정하여 lmbench는 문맥 교환의 비용에 대해 훌륭한 추정치를 구할 수 있다. 이 숙제에서 파이프를 사용하거나 Unix 소켓과 같은 다른 통신 기법을 사용하여 유사한 상황을 만들어 볼 수 있다.

문맥 교환의 비용을 측정하는 데 있어 어려움은 하나 이상의 CPU를 가진 시스템에서 발생한다. 그런 시스템에서 해야만 하는 일은 문맥 교환되는 프로세스들이 같은 프로세서 상에 위치하는 것을 보장하는 것이다. 다행히도 대부분의 운영체제는 프로세스를 특정 프로세서에 묶어두는 시스템 콜을 가진다. 예를 들어, Linux에서는 `sched_setaffinity()` 시스템 콜이 기대하는 일을 한다. 두 프로세스를 같은 프로세

서에 존재하도록 보장하면 같은 프로세서에 존재하는 한 프로세스를 중단시키고 다른 프로세스를 복원하는 운영체제의 비용을 확실하게 측정할 수 있다.

10

스케줄링 : 개요

이제 프로세스를 실행시키기 위한 문맥 교환 등의 저수준 **기법**에 대해서는 분명하게 이해하고 있어야 한다. 그렇지 않다면 한 두 장 이전으로 돌아가서 이러한 기법들이 어떻게 동작하는지에 대한 설명을 다시 읽기 바란다. 운영체제 스케줄러의 고수준 **정책**에 관해서는 이해가 필요하다. 이제부터 다양한 **스케줄링 정책**(**scheduling policy**)을 소개하고 그에 관한 이해를 높일 것이다. 이러한 정책은 **원칙**(**discipline**)이라고도 불리며 많은 똑똑하고 부지런한 사람들이 오랫동안 개발한 정책들이다.

사실 스케줄링의 기원은 컴퓨터 시스템 개발 이전으로 거슬러 올라간다. 초기의 방법들은 생산 관리 분야에서 개발되었으며 컴퓨터에 적용되었다. 이 사실은 그렇게 놀라운 일은 아니다. 생산 공정과 기타 많은 인간의 행동 역시 스케줄링이 필요하고, 효율성에 대한 요구 등, 공통된 해결 사항이 존재하기 때문이다. 우리의 문제는 다음과 같다.

핵심 질문 : 스케줄링 정책은 어떻게 개발하는가
스케줄링 정책을 생각하기 위한 기본적인 프레임워크를 어떻게 만들어야 하는가? 핵심 가정은 무엇인가? 어떤 평가 기준이 중요한가? 컴퓨터 시스템의 초창기에 사용되었던 기본 접근법은 무엇인가?

10.1 워크로드에 대한 가정

가장 먼저 프로세스에 대하여 몇 가지 가정을 할 것이다. 일련의 프로세스들이 실행하는 상황을 **워크로드**(**workload**)라고 부르기로 한다. 워크로드를 결정하는 것은 정책 개발에 매우 중요한 부분이다. 워크로드에 관해 더 잘 알수록 그에 맞게 정책을 정교하게 손질할 수 있다.

우리가 지금 설명에서 사용할 워크로드에 대한 가정은 비현실적이다. 하지만, 당장 논의하는 데 아무 문제 없다. 앞으로 논의가 진행됨에 따라 가정을 완화시킬 것이고 최종적으로 **제대로 동작하는 스케줄링 정책**을 만들게 될 것이다.

우리는 시스템에서 실행 중인 프로세스 혹은 작업(job)에 대해 다음과 같은 가정을 한다.

1. 모든 작업은 같은 시간 동안 실행된다.

2. 모든 작업은 동시에 도착한다.

3. 각 작업은 시작되면 완료될 때까지 실행된다.

4. 모든 작업은 CPU만 사용한다(즉, 입출력을 수행하지 않는다).

5. 각 작업의 실행 시간은 사전에 알려져 있다.

이러한 가정들이 비현실적이라고 이미 언급하였다. Animal Farm [Orw45]에서 어떤 동물은 다른 동물보다 더 평등하다고 한 것처럼 몇몇 가정은 다른 가정에 비해 더 비현실적이다. 특히, 각 작업의 실행 시간이 미리 알려져 있다는 가정은 더욱 그렇다. 이 가정은 스케줄러가 모든 것을 다 알 수 있다는 것을 의미한다. 좋은 일이기는 하지만 가까운 미래에 나올 것 같지는 않다.

10.2 스케줄링 평가 항목

워크로드에 대한 가정 이외에 스케줄링 정책의 비교를 위해 **스케줄링 평가 항목**(**scheduling metric**)을 결정해야 한다. 스케줄링의 경우 다양한 평가 기준이 존재한다.

우선, 문제를 간단하게 하기 위해 **반환 시간**(**turnaround time**)이라는 하나의 평가 기준을 사용한다. 작업 반환 시간은 작업이 완료된 시각에서 작업이 시스템에 도착한 시각을 뺀 시간으로 정의된다. 반환 시간 $T_{turnaround}$는

$$T_{turnaround} = T_{completion} - T_{arrival} \tag{10.1}$$

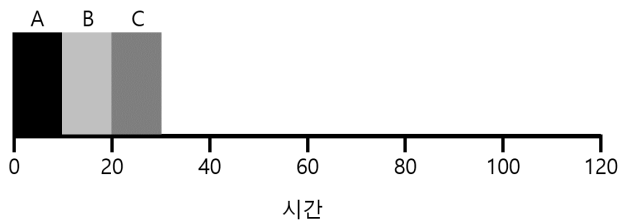
모든 작업은 동시에 도착한다고 가정했으므로 $T_{arrival}$=0으로 생각할 수 있다. $T_{turnaround} = T_{completion}$이다. 이 가정은 완화해 나갈 것이다.

반환 시간은 **성능** 측면에서의 평가 기준이라는 것을 주목해야 한다. 다른 평가 기준으로 **공정성**(**fairness**)이다. 예를 들면 **Jain's Fairness Index** [Jai91]에 따라 측정된다. 성능과 공정성은 스케줄링에서는 서로 상충되는 목표이다. 예를 들어, 스케줄러는 성능을 극대화하기 위해 몇몇 작업의 실행을 중지하며, 결과적으로 공정성이 악화된다. 인생은 대부분 완벽하지 않다.

10.3 선입선출

가장 기초적인 알고리즘은 **선입선출**(**First In First Out**, **FIFO**) 또는 **선도착선처리**(**First Come First Served**, **FCFS**) 스케줄링이라고 알려져 있다. FIFO는 많은 장점을 가진다. 단순하고 구현하기 쉽다. 우리의 가정 하에서는 매우 잘 동작한다.

간단한 예를 스케줄 해 보자. 시스템에 3개의 작업 A, B, C가 거의 동시에 도착했다고 가정하자 ($T_{arrival} = 0$). 간발의 차이로 A, B, C 순서대로 도착했다고 가정하자. 또한, 각 작업은 10초 동안 실행된다고 가정하자. 이 작업들의 평균 반환 시간은 얼마인가?



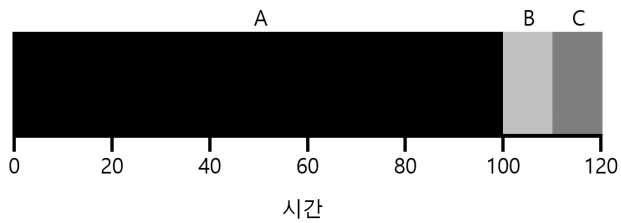
〈그림 10.1〉 **FIFO 간단한 예**

그림 10.1에서 A는 10, B는 20, C는 30에 종료했다는 것을 알 수 있다. 세 작업의 평균 반환 시간은 $\frac{(10+20+30)}{3} = 20$이다. 반환 시간의 계산은 쉽다.

자 이제 가정 중 한 가지를 완화하자. 구체적으로 1번 가정을 완화하여 작업을 실행 시간이 모두 같지 않다고 가정하자. FIFO는 어떻게 동작하는가? FIFO는 어떤 워크로드에서 좋지 않은 성능을 보일까?

(다음으로 넘어가기 전에 생각해 보자... 생각에 생각... 알았지?!)

FIFO 스케줄링이 어떤 문제를 야기하는지 같이 생각해 보자. 구체적으로 다시 A, B, C 세 개의 작업을 가정하자. 그러나 이번에는 A는 100초, B와 C는 10초 동안 실행된다.



〈그림 10.2〉 **FIFO가 그렇게 좋은 스케줄링이 아닌 이유**

그림 10.2에서 볼 수 있듯이, 작업 A가 B와 C 보다 먼저 100초 동안 실행된다. 따라서 시스템의 평균 반환 시간은 110초로 늘어난다. $\frac{(100+110+120)}{3} = 110$.

이 문제점은 **convoy effect** [Bla+79]라고 칭해지며 짧은 시간 동안 자원을 사용할 프로세스들이 자원을 오랫동안 사용하는 프로세스의 종료를 기다리는 현상을 말한다. 이 스케줄링 시나리오는 슈퍼마켓에서 줄 서서 계산을 기다릴 때, 앞사람이 카트 세 개에 물건을 가득 싣고 있는 경우와 느낌이 비슷하다. 한참 걸리겠지?[1]

그러면 어떻게 해야 할까? 작업 실행 시간이 다른 경우 더 좋은 알고리즘은 무엇인가? 먼저 생각해 보아라. 그런 후 다음 내용을 읽기 바란다.

1) 이 경우 빨리 다른 줄에 가서 서거나 길고 깊게 심호흡을 하라고 권하고 싶다. 오케이, 들이쉬고 내쉬고.

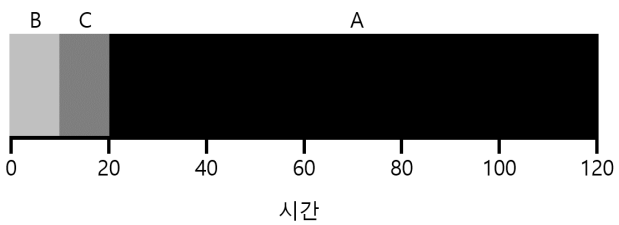
팁 : SJF의 원칙

최단 작업 우선 원칙은 평균 반환 시간이 (혹은 우리의 경우, 작업당) 강조되는 모든 시스템에 적용될 수 있는 일반적인 스케줄링 원리이다. 당신이 기다린 경험이 있는 모든 줄을 생각해 보라. 문제의 장소가 고객 만족을 중요시한다면 SJF의 사용을 고려했을 가능성이 높다. 예를 들어, 식료품 가게들은 적은 개수의 물건을 구매한 사람들이 다가올 겨울을 준비하는 가족 뒤에 서서 마냥 기다리지 않도록 "소량 구매 계산대"를 갖추고 있다.

10.4 최단 작업 우선

이 문제는 간단히 해결할 수 있다. 사실 이 아이디어는 오퍼레이션 리서치 분야에서 차용하여 [Cob54; JV56] 컴퓨터 시스템의 작업 스케줄링에 적용한 것이다. 이 새로운 스케줄링 개념용 **최단 작업 우선**(**Shortest Job First**, **SJF**)으로 이름 자체가 정책을 설명하고 있기 때문에 기억하기 쉽다. 이 원칙은 가장 짧은 실행 시간을 가진 작업을 먼저 실행시킨다.

위의 예를 이번에는 SJF로 스케줄 해 보자. 그림 10.3이 A, B, C를 실행시킨 결과를 보이고 있다. SJF가 평균 반환 시간 기준으로 더 나은 성능을 보이는지 보인다. B, C, A 순서로 실행시킴으로써 SJF는 평균 반환 시간을 110초에서 50초($\frac{(10+20+120)}{3} = 50$)로 2배 이상 향상시켰다.



〈그림 10.3〉 **SJF 간단한 예**

모든 작업이 동시에 도착한다면 SJF가 **최적**(**optimal**)의 스케줄링 알고리즘임을 증명할 수 있다. 지금은 이론이나 오퍼레이션 리서치 수업이 아니라 시스템 수업을 듣고 있기 때문에 이에 대한 증명은 하지 않을 것이다.

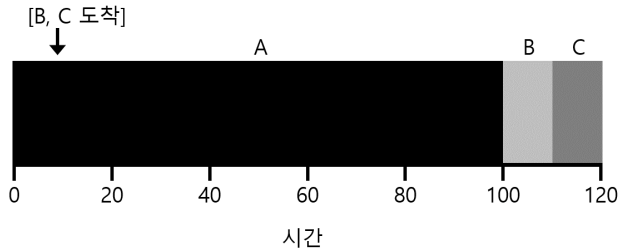
SJF라는 좋은 스케줄링 기법을 개발하였지만 가정이 아직 비현실적이다. 이제 가정을 완화해 보자. 특히, 가정 2를 완화하여 이제 모든 작업은 동시에 도착하는 것이 아니라 임의의 시간에 도착할 수 있다고 가정한다. 이러면 어떤 문제가 발생하는가?

(또 생각할 시간... 생각하고 있는 거지?... 자, 할 수 있어!)

자 문제가 무엇인지 설명하기 위하여 예를 들어 보자. 이번에는 A는 t = 0에 도착하고 100초의 실행 시간을 가지고 있고, 반면에 B와 C는 t = 10에 도착하고 각각 10초의 실행

별 문제 아니다, 걱정하지말자.

시간을 가진다고 가정하자. 순수한 SJF로 스케줄 했을 경우의 스케줄이 그림 10.4에 나와 있다.



〈그림 10.4〉 **B와 C가 늦게 도착한 경우의 SJF 스케줄**

그림에서 알 수 있듯이 B와 C가 A 바로 뒤에 도착한다고 하더라도 A가 끝날 때까지 기다릴 수밖에 없어서 이전의 convoy 문제가 다시 발생한다. 이 경우 평균 반환 시간은 103.33($\frac{(100+(110-10)+(120-10))}{3}$)이다. 스케줄러는 어떻게 스케줄링 해야 할까?

여담: 선점형 스케줄러

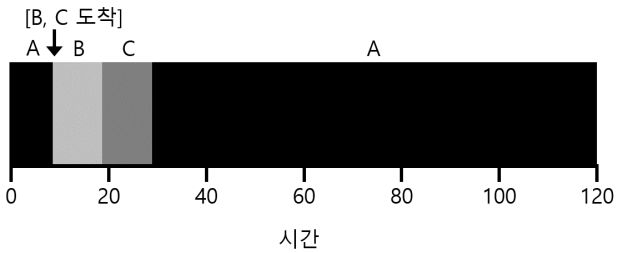
예전 일괄처리 시절에 많은 **비선점**(**non-preemptive**) 스케줄러가 개발되었다. 이 시스템은 각 작업이 종료될 때까지 계속 실행한다. 이론적으로 모든 현대 스케줄러는 **선점**형이고, 다른 프로세스를 실행시키기 위하여 필요하면 현재 프로세스의 실행을 중단한다. 이 사실은 스케줄러가 앞에서 배웠던 기법을 채용하고 있다는 것을 의미한다. 구체적으로 스케줄러는 **문맥 교환**을 수행할 수 있고, 실행 중인 프로세스를 잠시 중단시킬 수 있으며 다른 프로세스를 재개 또는 시작시킬 수 있다.

10.5 최소 잔여시간 우선

이 문제를 해결하기 위해서는 가정 3 (작업은 끝날 때까지 계속 실행된다)을 완화해야 한다. 이제 그 작업을 해 보자. 타이머 인터럽트와 문맥 교환을 고려하면 B와 C가 도착했을 때 스케줄러는 다른 일을 할 수 있다. 작업 A를 **중지**하고 지금 도착한 B나 C를 실행하기로 결정할 수도 있다. 물론, 선점된 A는 나중에 다시 실행된다. SJF는 **비선점**형 스케줄러이기 때문에 이와 같은 동작을 하지 못한다.

다행히도 이렇게 동작할 수 있는 스케줄러는 이미 존재한다. SJF에 선점 기능을 추가한 **최단 잔여시간 우선**(**Shortest Time-to-Completion First**, **STCF**) 또는 선점형 최단 작업 우선 (PSJF)으로 알려진 스케줄러이다 [CK68]. 언제든 새로운 작업이 시스템에 들어오면, 이 스케줄러는 남아 있는 작업과 새로운 작업의 잔여 실행 시간을 계산하고 그 중 가장 적은 잔여 실행 시간을 가진 작업을 스케줄한다. 우리의 예에서는

STCF는 A를 선점하고 B와 C를 끝날 때까지 실행시킨다. 두 작업이 모두 끝난 후에 A가 스케줄되어 남은 실행 시간 만큼 실행된다. 그림 10.5의 스케줄 예를 보라.



〈그림 10.5〉 **STCF의 간단한 예**

그 결과 평균 반환 시간이 단축되어 50초 ($\frac{((120-0)+(20-10)+(30-10))}{3}$)가 된다. 먼저와 같이, 새로운 가정 하에서 STCF가 최적의 스케줄러이다. 작업들이 동시에 도착할 경우, SFJ가 최적의 결과를 낸다는 것을 고려하면, STCF가 최적의 스케줄링이 되는 이유를 쉽게 알 수 있을것이다.

10.6 새로운 평가 기준 : 응답 시간

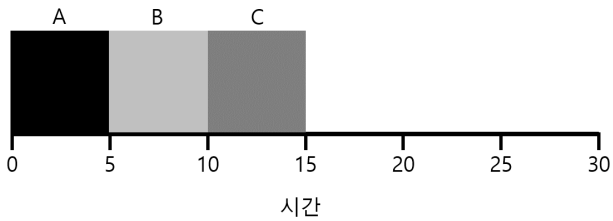
작업의 길이를 미리 알고 있고, 작업이 오직 CPU만 사용하며, 평가 기준이 반환 시간 하나라면, STCF는 매우 훌륭한 정책이다. 초기 일괄처리 컴퓨터 시스템에서는 이러한 스케줄링 알고리즘이 의미가 있었다. 그러나 시분할 컴퓨터의 등장이 모든 것을 바꾸었다. 이제 사용자는 터미널에서 작업하게 되어 시스템에게 상호작용을 원활히 하기 위한 성능을 요구하게 되었다. **응답 시간**(**response time**)이라는 새로운 평가 기준이 태어나게 된다.

응답 시간은 작업이 도착할 때부터 처음 스케줄 될 때까지의 시간으로 정의된다. 다음과 같다.

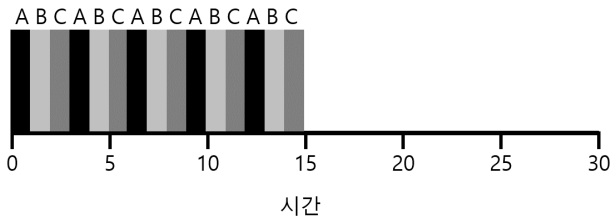
$$T_{response} = T_{firstrun} - T_{arrival} \tag{10.2}$$

예를 들어, 위와 같은 스케줄의 (A는 시간 0, B 와 C는 시간 10에 도착) 경우, 각 작업의 응답 시간은 A는 0, B는 0, C는 10, 평균 3.33이 된다.

이미 생각하고 있는지 모르지만, STCF를 비롯하여 비슷한 류의 정책들은 응답 시간이 짧다고 할 수 없다. 예를 들어, 3개의 작업이 동시에 도착한 경우, 세 번째 작업은 딱 한 번 스케줄 되기 위해 먼저 실행된 두 작업이 *완전히 끝날 때까지* 기다린다. 반환 시간 기준으로는 매우 훌륭한 반면, 응답 시간과 상호작용 측면에서는 매우 나쁜 방법이다. 터미널 앞에 앉아 입력한 후, 단지 다른 작업이 당신의 작업보다 먼저 스케줄 되었다는 이유로 시스템의 응답이 올 때까지 10초 기다리는 것을 상상해 보자. 별로 즐거운 일은 아니다.



〈그림 10.6〉 **다시 SJF 스케줄링(응답 시간은 좋지 않음)**



〈그림 10.7〉 **라운드 로빈(응답 시간이 좋음)**

우리는 다른 문제를 해결해야 한다. 응답 시간에 민감한 스케줄러를 어떻게 만들 수 있는가?

10.7 라운드 로빈

응답 시간 문제를 해결하기 위하여 **라운드 로빈**(**Round-Robin**, **RR**) 스케줄링 [Kle64]이라 불리는 스케줄링 알고리즘을 도입한다. 기본 발상은 간단하다. RR은 작업이 끝날 때까지 기다리지 않는다. 대신 일정 시간 동안 실행한 후 실행 큐의 다음 작업으로 전환한다. 이때 작업이 실행되는 일정 시간을 **타임 슬라이스**(**time slice**) 또는 **스케줄링 퀀텀**(**scheduling quantum**)이라 부른다. 작업이 완료될 때까지 이런 식으로 계속 진행된다. 이러한 이유로 RR은 때때로 **타임 슬라이싱**이라고 불린다. 타임 슬라이스의 길이는 타이머 인터럽트 주기의 배수여야 한다. 타이머가 10 msec 마다 인터럽트를 발생시킨다면 타임 슬라이스는 10, 20 등 10 msec의 배수가 될 수 있다.

RR을 더 자세하게 이해하기 위해 하나의 예를 살펴보자. 3개의 작업 A, B, C가 시스템에 동시에 도착하고, 각각 5초간 실행된다고 가정한다. SJF 스케줄러는 다른 작업을 실행하기 전에 각 작업을 종료할 때까지 실행한다(그림 10.6). 대조적으로, 1초의 타임 슬라이스를 가지는 RR은 작업을 빠르게 순환하게 된다(그림 10.7). RR의 평균 응답 시간은 $\frac{(0+1+2)}{3} = 1$이고, SJF의 평균 응답 시간은 $\frac{(0+5+10)}{3} = 5$이다.

쉽게 알 수 있듯이, 타임 슬라이스의 길이는 RR에게 매우 중요하다. 타임 슬라이스가 짧을수록, 응답 시간 기준으로 RR의 성능은 더 좋아진다. 그러나 타임 슬라이스를 너무 짧게 지정하면 문제가 생긴다. 짧게 지정하면 문맥 교환 비용이 전체 성능에 큰 영향을 미치게 된다. 시스템 설계자는 시스템이 최적의 상태로 동작할 수 있도록 타임 슬라이스의

팁 : 비용의 상쇄

일반적인 **상쇄**(**amortization**) 기술은 어떤 연산에 고정 비용이 존재하는 시스템에서 흔히 사용된다. 그 비용이 적은 횟수로 발생하게 하여, 즉 해당 연산을 적게 실행함으로써 시스템의 전체 비용이 감소된다. 예를 들어, 타임 슬라이스는 10 msec로 설정되고 문맥 교환 비용이 1 msec라면 대강 10%의 시간이 문맥 교환에 사용되고 따라서 낭비된다. 이 비용을 상쇄하고 싶은 경우, 타임 슬라이스를 예로 들어 100 msec로 늘릴 수 있다. 이러면 문맥 교환에 1% 미만의 시간이 소모되고 타임 슬라이싱의 비용이 상쇄된다.

길이를 결정해야 한다. 문맥 교환 비용을 **상쇄**할 수 있을 만큼 길어야 하지만 그렇다고 응답 시간이 너무 길어지면 안 된다.

문맥 교환 비용에는 레지스터를 저장/복원하는 작업만 있는 것이 아니다. CPU 캐시, TLB, 분기 예측 등을 비롯하여 다른 하드웨어에도 프로그램과 관련된 다양한 작업 정보들이 저장되어 있다. 작업이 전환되면 이 정보들은 모두 갱신되어야 한다. 갱신 작업이 매우 큰 성능 비용을 유발한다 [MB91].

적당한 길이의 응답 시간이 유일한 평가 기준인 경우 타임 슬라이스를 가진 RR은 매우 훌륭한 스케줄러이다. 반환 시간 기준으로는 어떤가? 앞의 예를 다시 한 번 살펴보자. A, B, C 각 작업은 5초간 실행되고, 동시에 도착하며, RR은 1초의 타임 슬라이스를 가진다. A는 13, B는 14, C는 15에 종료하고 평균 14의 반환 시간을 보인다는 것을 알 수 있다. 매우 안 좋다!

반환 시간이 유일한 측정 기준인 경우 RR이 확실히 최악의 정책이라는 것은 놀라운 일이 아니다. 직관적으로, 충분히 이해 가능하다. RR은 각 작업을 잠깐 실행하고 다음 작업으로 넘어가고 하면서 가능한 한 각 작업을 늘리는 것이 목표다. 반환 시간은 작업 완료 시간만을 고려하기 때문에, RR은 거의 최악이며, 많은 경우 단순한 FIFO보다도 성능이 좋지 않게 된다.

일반적으로 RR과 같은 **공정**한 정책, 즉 작은 시간 단위로 모든 프로세스에게 CPU를 분배하는 정책은 반환 시간과 같은 평가 기준에서는 성능이 나쁘다. 이건 당연하다. 불공정하게 한다면 하나의 작업을 끝까지 실행하고 종료할 수 있지만 나머지 작업들에 대한 응답 시간은 포기해야 한다. 반대로 공정성을 더 중히 여긴다면 응답 시간은 줄어들지만 반환 시간은 나빠지게 된다. 이러한 유형의 절충은 시스템에서는 흔히 있는 일이다. 당신은 케익을 먹으면서 동시에 케익을 보관할 수는 없다[2].

우리는 2종류의 스케줄러를 개발하였다. 첫째 유형은 (SJF, STCF) 반환 시간을 최적화하지만 응답 시간은 나쁘다. 두 번째 유형은 (RR) 응답 시간을 최적화하지만 반환 시간이 나쁘다. 아직 완화해야 하는 가정 두 개가 남아 있다. 가정 4 (작업은 입출력을 하지 않는다)와 가정 5 (각 작업의 실행 시간은 알려져 있다)이다. 이제 이 가정들을 정복해 보자.

2) 사람을 혼란시키는 속담. 원래는 "당신은 케익을 먹으면서 보존할 수 없다"여야 하기 때문이다. (분명하지 않은가, 아닌가?). 놀랍게도 이 속담에 관한 Wikipedia 페이지가 있다. 더욱 놀라운 것은 재미있는 읽을거리라는 것이다. `http://en.wikipedia.org/wiki/You_can't_have_your_cake_and_eat_it`

팁: 중첩은 높은 이용률을 가능하게 한다

가능하면 시스템의 활용도를 극대화하기 위해 연산을 **중첩**되게 실행한다. 중첩은 디스크 입출력을 할 때나 원격 컴퓨터에 메시지를 보낼 때 등 다양한 영역에서 유용하다. 어느 경우라도 이 작업들을 시작한 후에 다른 작업으로 전환하는 것은 좋은 생각이다. 시스템의 전반적인 사용률과 효율성이 향상된다.

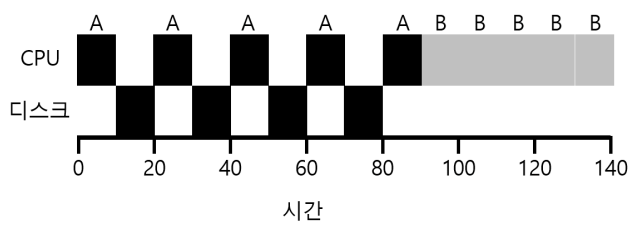
10.8 입출력 연산의 고려

우선 가정 4를 완화하자. 모든 프로그램은 입출력 작업을 수행한다. 아무런 입력도 받아들이지 않는 프로그램을 상상해 보자. 매번 같은 출력을 만들어 낼 것이다. 출력하지 않는 프로그램을 상상해 보자. 그건 속담에도 나올 법한 아무도 보는 사람이 없이 숲에서 잎이 떨어지는 것과 같다. 실행되었다는 사실이 아무 의미가 없다.

입출력 작업을 요청한 경우 스케줄러는 다음에 어떤 작업을 실행할지를 결정해야 한다. 현재 실행 중인 작업은 입출력이 완료될 때까지 CPU를 사용하지 않을 것이기 때문이다. 입출력 요청을 발생시킨 작업은 입출력 완료를 기다리며 대기 상태가 된다. 입출력이 하드 디스크 드라이브에 보내진 경우, 프로세스는 드라이브의 현재 입출력 워크로드에 따라 몇 초 또는 좀 더 긴 시간 동안 블록될 것이다. 스케줄러는 그 시간 동안 실행될 다른 작업을 스케줄 해야 한다.

마찬가지로 스케줄러는 입출력 완료 시에도 의사 결정을 해야 한다. 입출력이 완료되면 인터럽트가 발생하고 운영체제가 실행되어 입출력을 요청한 프로세스를 대기 상태에서 준비 상태로 다시 이동시킨다. 물론, 인터럽트가 발생했을 때 요청 프로세스를 즉시 실행시키기로 결정할 수도 있다. 운영체제는 각 작업을 어떻게 처리해야 하는가?

이 쟁점을 더 잘 이해하기 위해서 두 개의 작업 A와 B가 있다고 하자. 각 작업은 50 msec의 CPU 시간을 필요로 한다. 그러나 둘은 한 가지 큰 차이를 가진다. A는 10 msec 동안 실행된 후, 입출력 요청을 한다(여기서 입출력 시간은 10 msec라고 가정한다). 반면에 B는 입출력을 수행하지 않는다. 스케줄러는 A를 먼저 실행시키고 B를 다음에 실행시킨다(그림 10.8).

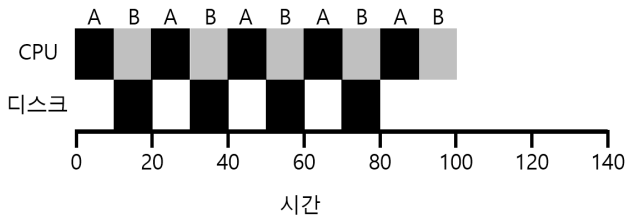


〈그림 10.8〉 **자원의 비효율적인 활용**

STCF 스케줄러를 구축하려고 한다고 가정하자. A가 5개의 10 msec 작업으로 분할되는 반면에 B는 하나의 50 msec의 CPU를 요청한다는 사실을 어떻게 활용할까?

분명히 입출력을 고려하지 않고 작업을 하나씩 실행시키는 것은 의미가 없다.

일반적인 접근 방식은 A의 각 10 msec 하위 작업을 독립적인 작업으로 다루는 것이다. 시스템이 시작할 때 10 msec 작업들과 50 msec를 스케줄하는 것이다. STCF의 경우 선택은 자명하다. 가장 짧은 작업을 선택하라. 이 경우에는 A가 된다. 그러면 A의 첫 번째 소 작업이 완료되면 B만 남게 되어 실행을 시작한다. A의 다음 작업이 제출되고 B를 선점하여 10 msec 동안 실행한다. 이렇게 하면 프로세스의 입출력이 끝나기를 기다리는 동안 CPU는 다른 프로세스에 의해 사용되어 연산의 중첩이 가능해진다. 시스템은 좀 더 잘 활용된다 (그림 10.9).



〈그림 10.9〉 **중첩을 통해 효율적 자원 활용이 가능하다**

입출력을 고려한 스케줄러 방법에 대해서 알아보았다. 각 CPU 버스트를 하나의 작업으로 간주함으로써 스케줄러는 대화형 프로세스가 더 자주, 즉 유리하게 실행되는 것을 보장한다. 이러한 대화형 작업이 입출력을 실행하는 동안 다른 CPU-집중 작업들이 실행되고 CPU의 이용률은 더 높아진다.

10.9 만병통치약은 없다(No More Oracle)

입출력을 고려한 기본적인 접근 방식에 대해 논의하였으므로 마지막 가정에 대해 생각해 보자. 스케줄러가 각 작업의 실행 시간을 알고 있다는 가정이다. 전에 얘기했듯이, 이 가정은 아마 우리가 할 수 있는 최악의 가정이다. 사실 범용 운영체제에서(우리가 현재 고려 중인 것과 같은) 작업의 길이에 대해서 알 수 있는 길은 없다. 따라서 아무런 사전 지식 없이 SJF/STCF 처럼 행동하는 알고리즘을 구축할 수 있을까? 게다가, 응답 시간도 좋게 하기 위하여 RR 스케줄러의 경우에 보았던 아이디어를 어떻게 하면 포함시킬 수 있을까?

10.10 요약

우리는 스케줄링의 기본적인 개념과 두 가지 부류의 접근법을 살펴보았다. 첫 번째 부류는 남아 있는 작업 중 실행 시간이 제일 짧은 작업을 수행하고, 반환 시간을 최소화한다. 두 번째 부류는 모든 작업을 번갈아 실행시키고 응답 시간을 최소화한다. 안타깝게도 반환 시간과 응답 시간 중 하나를 최적화하면 나머지 하나는 좋지 않은 특성을 나타낸다. 이는 시스템에서 흔히 보이는 절충을 요구하는 상황이다. 전체적인 그림에 입출력을

어떻게 통합해야 하는지도 보았다. 그렇지만 미래를 예측할 수 없는 운영체제의 근본적인 문제는 해결할 수 없었다. 가까운 과거를 이용하여 미래를 예측하는 스케줄러를 구현하여 이 문제를 어떻게 해결하는지 곧 보게 될 것이다. 이 스케줄러는 **멀티 레벨 피드백 큐**(**multi-level feedback queue**)라고 불리며 다음 장의 주제이다.

참고 문헌

[Bla+79] **"The Convoy Phenomenon"**
M. Blasgen, J. Gray, M. Mitoma, and T. Price
ACM Operating Systems Review, 13:2, April 1979
아마도 운영체제뿐 아니라 데이터베이스에서 발생한 호위함 효과에 대한 첫 번째 참고 문헌.

[Cob54] **"Priority Assignment in Waiting Line Problems"**
A. Cobham
Journal of Operations Research, 2:70, pages 70-76, 1954
기계 수리 일정을 정하는 데 SJF 방식을 사용한 선구적인 논문.

[CK68] **"Computer Scheduling Methods and their Countermeasures"**
Edward G. Coffman and Leonard Kleinrock
AFIPS '68 (Spring), April 1968
다양한 기본 스케줄링 알고리즘에 대한 우수한 초기 소개 및 분석.

[Jai91] **"The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling"**
R. Jain
Interscience, New York, April 1991
컴퓨터 시스템 측정에 관한 표준 교과서. 당신의 라이브러리를 빛낼 훌륭한 참고 문헌.

[JV56] **"Machine Repair as a Priority Waiting-Line Problem"**
Thomas E. Phipps Jr. and W.R. Van Voorhis
Operations Research, 4:1, pages 76-86, February 1956
기계 수리 스케줄링에 사용된 Cobham의 SJF를 일반화한 후속 연구. 또한 이러한 환경에서 STCF 방식의 유용성을 생각한 논문. 구체적으로 "수리 작업에는 특정 유형이 존재한다 ... 많은 해체와 마루를 너트와 볼트로 뒤덮음을 동반하기 때문에 한 번 시작하면 중단되서는 안 된다. 다른 경우, 하나 이상의 짧은 작업들이 존재할 때 긴 작업을 계속하는 것은 권하고 싶지 않을 것이다 (p.81)"

[Kle64] **"Analysis of a Time-Shared Processor"**
Leonard Kleinrock
Naval Research Logistics Quarterly, 11:1, pages 59-73, March 1964
라운드 로빈 스케줄링 알고리즘에 대한 첫 번째 참고 문헌. 시분할 시스템을 스케줄링 할 때 라운드 로빈 스케줄링 알고리즘 사용에 대한 분석을 제시한 논문 중 하나.

[Lav78] **"The Manchester Mark I and Atlas: A Historical Perspective"**
S.H. Lavington
Communications of the ACM, 21:1, January 1978
초창기 컴퓨터 개발과 Atlas의 개척적인 노력에 관한 역사.

[MB91] **"The effect of context switches on cache performance"**
Jeffrey C. Mogul and Anita Borg
ASPLOS, 1991
문맥 교환이 캐시의 성능에 미치는 영향에 관한 좋은 연구. 프로세서는 초당 수십억 명령어를 실행하지만 문맥 교환은 msec 시간 단위로 발생하기 때문에 현재 시스템에서는 크게 문제가 되지 않는다.

[Orw45] **"Animal Farm"**
George Orwell

Secker and Warburg (London), 1945
권력과 그의 변질에 관한 슬프지만 우울한 우화책. 어떤 이는 제2차 세계대전 전 스탈린 시대의 소련에 대한 비판이라고 말한기도 한다. 우리는 그저 돼지에 대한 비판이라고 말한다.

숙제

`scheduler.py` 프로그램은 응답 시간, 반환 시간, 총 대기 시간의 관점에서 여러 스케줄러가 어떻게 동작하는지를 볼 수 있게 한다. 자세한 내용은 README를 참조하시오.

문제

1. 길이가 200인 세 개의 작업을 SJF와 FIFO 스케줄링 방식으로 실행할 경우 응답 시간과 반환 시간을 계산하시오.
2. 같은 조건이지만 작업의 길이가 각각 100, 200 및 300일 경우에 대해 계산하시오.
3. 2번과 같은 조건으로 타임 슬라이스가 1인 RR 스케줄러에 대해서도 계산하시오.
4. SJF와 FIFO가 같은 반환 시간을 보이는 워크로드의 유형은 무엇인가?
5. SJF가 RR과 같은 응답 시간을 보이기 위한 워크로드와 타임 퀀텀의 길이는 무엇인가?
6. 작업의 길이가 증가하면 SJF의 응답 시간은 어떻게 되는가? 변화의 추이를 보이기 위해서 시뮬레이터를 사용할 수 있는가?
7. 타임 퀀텀의 길이가 증가하면 RR의 응답 시간은 어떻게 되는가? N 개의 작업이 주어졌을 때, 최악의 응답 시간을 계산하는 식을 만들 수 있는가?

11

스케줄링 : 멀티 레벨 피드백 큐

이 장에서는 **멀티 레벨 피드백 큐**(**Multi-level Feedback Queue**, **MLFQ**)로 알려진 가장 유명한 스케줄링 기법에 대해 논의한다. 멀티 레벨 피드백 큐(MLFQ) 스케줄러는 Compatible Time-Sharing System(CTSS)에 사용되며 Corbato 등에 의해 1962년에 최초로 소개되었다 [CDD62] . 이 연구와 Multics에 대한 후속 연구로 Corbato는 최고 영예인 **Turing Award**를 안았다. 이 스케줄러는 수년 동안 다듬어져 일부 현대 시스템에까지 발전되었다.

MLFQ가 해결하려고 하는 기본적인 문제는 두 가지이다. 첫째, 짧은 작업을 먼저 실행시켜 *반환 시간을* 최적화하고자 한다. 이에 대해서는 전 장에서 논의한 바 있다. SJF나 STCF 같은 알고리즘은 작업의 실행 시간 정보를 필요로 하지만, 불행히도 운영체제는 이 실행 시간을 미리 알 수 없다. 둘째, MLFQ는 대화형 사용자(즉, 화면 앞에 앉아 바라보면서 프로세스의 종료를 기다리는 사용자)에게 응답이 빠른 시스템이라는 느낌을 주고 싶었기 때문에 응답 시간을 최적화한다. 불행히도 RR과 같은 알고리즘은 응답 시간을 단축시키지만 반환 시간은 거의 최악이다. 우리의 문제는 다음과 같다. 우리가 프로세스에 대한 정보가 없다면 이러한 스케줄러를 어떻게 만들 수 있을까? 실행 중인 작업의 특성을 알아내고 이를 이용하여 더 나은 스케줄링 결정을 하기 위한 방법은 무엇인가?

핵심 질문 : 정보 없이 스케줄 하는 방법은 무엇인가
작업의 실행 시간에 대한 선행 정보 없이 대화형 작업의 응답 시간을 최소화하고 동시에 반환 시간을 최소화하는 스케줄러를 어떻게 설계할 수 있는가?

11.1 MLFQ: 기본 규칙

멀티 레벨 피드백 큐의 기본 알고리즘을 설명한다. 현재 구현되어 있는 여러 MLFQ들은 자세하게 살펴보면 차이가 있지만 [Epe95], 기본적으로 비슷한 방법을 사용하고 있다.

MLFQ는 여러 개의 **큐**로 구성되며, 각각 다른 **우선순위**(**priority level**)가 배정된다. 실행 준비가 된 프로세스는 이 중 하나의 큐에 존재한다. MLFQ는 실행할 프로세스를

팁: 역사로부터 배우다

멀티 레벨 피드백 큐는 미래를 예측하기 위해 과거의 경험을 활용하는 훌륭한 예이다. 이러한 접근 방식은 운영체제 (하드웨어 분기 예측기와 캐시 알고리즘을 포함한 컴퓨터 과학의 다른 많은 분야에서도) 분야에서 흔히 사용된다. 이러한 방식은 작업이 단계별로 진행되어 예측 가능할 때 잘 동작한다. 물론, 신중하게 사용해야 한다. 잘못 동작하기 쉬우며, 이러한 정보가 없을 때 보다 더 나쁜 결정을 하게 만들기 때문이다.

결정하기 위하여 우선순위를 사용한다. 높은 우선순위를 가진 작업이, 즉 높은 우선순위 큐에 존재하는 작업이 선택된다.

큐에 둘 이상의 작업이 존재할 수 있다. 이들은 모두 같은 우선순위를 가진다. 이 작업들 사이에서는 라운드 로빈 (Round-Robin, RR) 스케줄링 알고리즘이 사용된다.

MLFQ 스케줄링의 핵심은 우선순위를 정하는 방식이다. MLFQ는 각 작업에 고정된 우선순위를 부여하는 것이 아니라 각 작업의 *특성에 따라* 동적으로 우선순위를 부여한다. 예를 들어, 어떤 작업이 키보드 입력을 기다리며 반복적으로 CPU를 양보하면 MLFQ는 해당 작업의 우선순위를 높게 유지한다. 이러한 패턴은 대화형 프로세스가 나타내는 패턴이다. 대신에 한 작업이 긴 시간 동안 CPU를 집중적으로 사용하면 MLFQ는 해당 작업의 우선순위를 낮춘다. 이렇게 MLFQ는 작업이 진행되는 동안 해당 작업의 정보를 얻고, 이 *정보를* 이용하여 *미래* 행동을 예측한다.

MLFQ의 두 가지 기본 규칙은 다음과 같다.

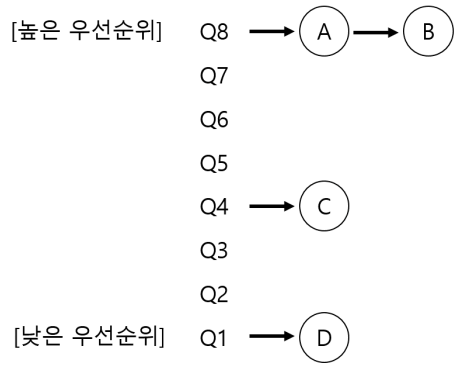
- **규칙 1:** Priority(A) > Priority(B) 이면, A가 실행된다 (B는 실행되지 않는다).
- **규칙 2:** Priority(A) = Priority(B) 이면, A와 B는 RR 방식으로 실행된다.

임의의 시간에 큐의 모양은 그림 11.1과 같다. 그림에서 두 작업(A와 B)이 가장 높은 우선순위에 존재하고, C는 중간, D는 가장 낮은 우선순위 큐에 존재한다. 우리가 알고 있는 MLFQ의 동작을 고려하면 스케줄러는 가장 높은 우선순위의 큐의 A와 B를 번갈아 실행할 것이다. 불쌍한 작업 C와 D는 실행되지도 않는다. 어떻게 이런 일이!!!

정적인 스냅 사진만으로는 MLFQ가 어떻게 동작하는지 알 수 없다. 작업 우선순위가 시간에 따라 어떻게 변화하는지 알아보자.

11.2 시도 1: 우선순위의 변경

MLFQ가 작업의 우선순위를 어떻게 바꿀 것인지 결정해야 한다. 작업의 우선순위를 변경하는 것은 작업이 존재할 큐를 결정하는 것과 마찬가지다. 이를 위해서 우리는 워크로드의 특성을 반영해야 한다. 짧은 실행 시간을 갖는 CPU를 자주 양보하는 대화형 작업과 많은 CPU 시간을 요구하지만 응답 시간은 중요하지 않은 긴 실행 시간의 CPU 위주 작업이 혼재되어 있다. 우선순위 조정 알고리즘을 위한 첫 번째 시도는 다음과 같다.

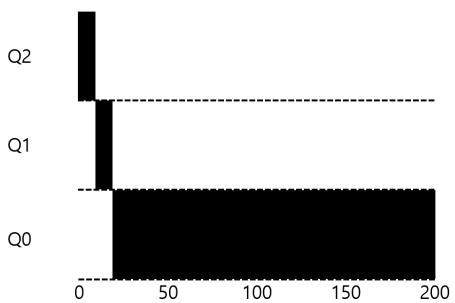


〈그림 11.1〉 **MLFQ 예**

- **규칙 3:** 작업이 시스템에 진입하면, 가장 높은 우선순위, 즉 맨 위의 큐에 놓여진다.
- **규칙 4a:** 주어진 타임 슬라이스를 모두 사용하면 우선순위는 *낮아진다*. 즉, 한 단계 아래 큐로 이동한다.
- **규칙 4b:** 타임 슬라이스를 소진하기 전에 CPU를 양도하면 *같은* 우선순위를 유지한다.

예 1: 한 개의 긴 실행 시간을 가진 작업

몇 가지 예를 살펴보자. 우선 긴 실행 시간을 가진 작업이 도착했을 때 어떤 일이 일어나는지 알아보자. 그림 11.2는 세 개의 큐로 이루어진 스케줄러에서 시간이 지남에 따라 작업의 우선순위가 어떻게 변하는지 보인다.

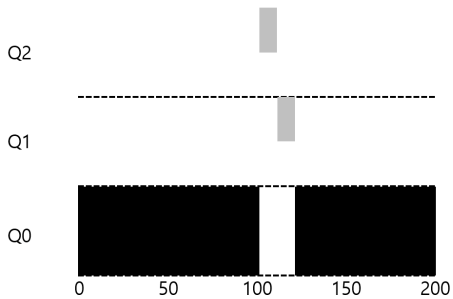


〈그림 11.2〉 **긴 실행 시간을 가진 작업의 우선순위 변화**

예에서 보는 것처럼 작업은 최고 우선순위로 진입한다(Q2). 10 msec 타임 슬라이스가 하나 지나면 스케줄러는 작업의 우선순위를 한 단계 낮추어 해당 작업을 Q1으로 이동시킨다. 다시 하나의 타임 슬라이스 동안 Q1에서 실행한 후 작업은 마침내 가장 낮은 우선순위를 가지게 되고 Q0로 이동된다. 이후에는 Q0에 계속 머무르게 된다. 간단하다. 그렇지 않은가?

예 2: 짧은 작업과 함께

좀 더 복잡한 예를 살펴보자. MLFQ가 어떻게 SJF에 근접할 수 있는지 이해하기 바란다. 이 예에서는 2개의 작업이 존재한다. A는 오래 실행되는 CPU 위주 작업이고 B는 짧은 대화형 작업이다. A는 얼마 동안 이미 실행해 온 상태이고 B는 이제 도착했다고 가정하자. 어떤 일이 벌어질까? MLFQ는 SJF와 근사하게 동작해서 B를 선호할 것인가?



〈그림 11.3〉 **대화형 작업이 들어온 경우**

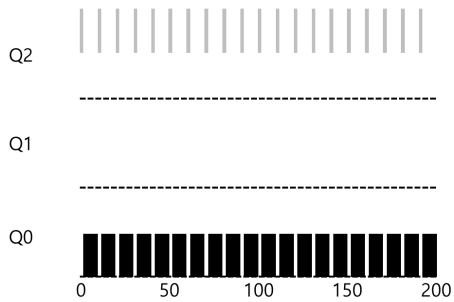
그림 11.3은 이 시나리오의 결과를 보인다. 다른 오래 실행되는 CPU 위주 작업들처럼 A(검은색)는 가장 낮은 우선순위 큐에서 실행되고 있다. B(회색)는 $T = 100$에 시스템에 도착하고 가장 높은 우선순위 큐에 놓여진다. 실행 시간이 짧기 때문에 (단지 20 ms), 두 번의 타임 슬라이스를 소모하면 B는 바닥의 큐에 도착하기 전에 종료한다. 그런 후에 A는 낮은 우선순위에서 실행을 재개한다.

이 예에서 이 알고리즘의 주요 목표를 알 수 있다. 스케줄러는 작업이 짧은 작업인지 긴 작업인지 알 수 없기 때문에 일단 짧은 작업이라고 *가정하여* 높은 우선순위를 부여한다. 진짜 짧은 작업이면 빨리 실행되고 바로 종료할 것이다. 짧은 작업이 아니라면 천천히 아래 큐로 이동하게 되고 스스로 긴 배치형 작업이라는 것을 증명하게 된다. 이러한 방식으로 MLFQ는 SJF를 근사할 수 있다.

예 3: 입출력 작업에 대해서는 어떻게?

다음으로 입출력 작업을 수행하는 예를 살펴보자. 규칙 4b가 말하는 것처럼, 프로세스가 타임 슬라이스를 소진하기 전에 프로세서를 양도하면 같은 우선순위를 유지하게 한다. 이 규칙의 의도는 간단하다. 대화형 작업이 키보드나 마우스로부터 사용자 입력을 대기하며 자주 입출력을 수행하면 타임 슬라이스가 종료되기 전에 CPU를 양도하게 될 것이다. 그런 경우 동일한 우선순위를 유지하게 하는 것이다.

그림 11.4는 이 규칙이 동작하는 예를 보이고 있다. B(회색)는 대화형 작업으로서 입출력을 수행하기 전에 1 msec 동안만 실행된다. A(검정색)는 긴 배치형 작업으로 B와 CPU를 사용하기 위하여 경쟁한다. B는 CPU를 계속해서 양도하기 때문에 MLFQ 방식은 B를 가장 높은 우선순위로 유지한다. B가 대화형 작업이라면 MLFQ는 대화형 작업을 빨리 실행시킨다는 목표에 더 근접하게 된다.



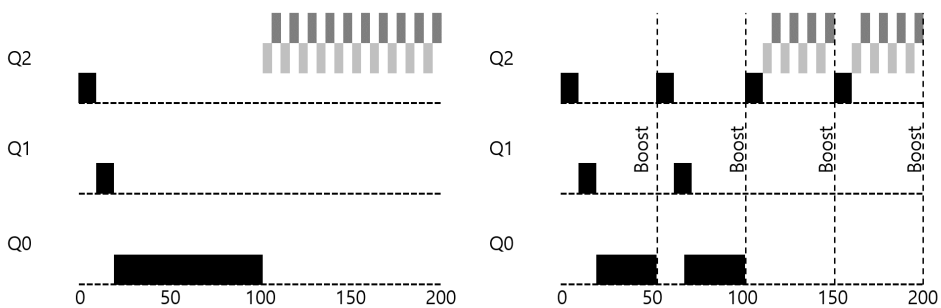
〈그림 11.4〉 **입출력-집중 작업과 CPU-집중 작업이 혼합된 워크로드**

현재 MLFQ의 문제점

현재 MLFQ는 단순하다. CPU를 긴 작업들과 짧은 작업들 사이에 잘 공유하고, 입출력-중점 대화형 작업을 빨리 실행시키기 때문에 잘 동작하는 것처럼 보인다. 이 방식은 심각한 결점을 가진다. 하나라도 생각해 낼 수 있겠는가?

(여기서 잠깐 중지하고 할 수 있는 한 모든 생각을 해 보도록)

첫째, **기아 상태(starvation)**가 발생할 수 있다. 시스템에 너무 많은 대화형 작업이 존재하면 그들이 모든 CPU 시간을 소모하게 될 것이고 따라서 긴 실행 시간 작업은 CPU 시간을 할당받지 못할 것이다(굶어 죽는다). 이러한 시나리오에서도 긴 실행 시간 작업들도 진행이 되도록 만들고 싶다.



〈그림 11.5〉 **우선순위 상향이 없는 경우(좌측)와 있는 경우(우측)**

둘째, 똑똑한 사용자라면 스케줄러를 자신에게 유리하게 동작하도록 프로그램을 다시 작성할 수 있다. 스케줄러를 자신에게 유리하게 동작시킨다는 것은 일반적으로 스케줄러를 속여서 지정된 몫보다 더 많은 시간을 할당하도록 하게 만드는 것을 가리킨다. 지금까지 논의한 알고리즘은 다음과 같은 공격에 취약하다. 타임 슬라이스가 끝나기 전에 아무 파일을 대상으로 입출력 요청을 내려 CPU를 양도한다. 그렇게 하면 같은 큐에 머무를 수 있고 따라서 더 높은 퍼센트의 CPU 시간을 얻게 된다. 제대로 된다면, 예를 들어 타임 슬라이스의 99%를 실행하고 CPU를 양도하게 되면 CPU를 거의 독점할 수 있다.

마지막으로 프로그램은 시간 흐름에 따라 *특성이 변할* 수 있다. CPU 위주 작업이 대화형 작업으로 바뀔 수 있다. 현재 구현 방식으로는 그런 작업은 운이 없게도 다른 대화형 작업들과 같은 대우를 받을 수 없다.

11.3 시도 2: 우선순위의 상향 조정

규칙을 보완하여 기아 문제를 방지할 수 있는지 살펴보자. CPU 위주 작업이 조금이라도 진행하는 것을 보장하기 위해서 무엇을 할 수 있는가?

간단한 아이디어는 주기적으로 모든 작업의 우선순위를 **상향 조정(boost)**하는 것이다. 목적을 달성하기 위해 여러 방법이 존재하지만 간단한 방법을 사용하기로 하자. 모두 최상위 큐로 보내는 것이다. 새로운 규칙은 다음과 같다.

- **규칙 5:** 일정 기간 S가 지나면, 시스템의 모든 작업을 최상위 큐로 이동시킨다.

새 규칙은 두 가지 문제를 한 번에 해결한다. 첫째, 프로세스는 굶지 않는다는 것을 보장받는다. 최상위 큐에 존재하는 동안 작업은 다른 높은 우선순위 작업들과 라운드 로빈 방식으로 CPU를 공유하게 되고 서비스를 받게 된다. 둘째, CPU 위주의 작업이 대화형 작업으로 특성이 변할 경우 우선순위 상향을 통해 스케줄러가 변경된 특성에 적합한 스케줄링 방법을 적용한다.

예를 들어 보자. 긴 실행 시간을 가진 작업이 두 개의 대화형 작업과 CPU를 두고 경쟁할 때의 행동을 보여주는 시나리오다. 이 두 개의 그래프가 그림 11.5에 나와 있다. 왼쪽 그래프는 우선순위 상향이 없는 경우를 보이고 있다. 긴 실행 시간 작업은 두 개의 짧은 작업이 도착한 이후에는 굶게 된다. 오른쪽 그래프는 50 msec 마다 우선순위 상향이 일어난다. 물론, 너무 짧은 시간이지만 예를 위해 사용되었다. 긴 실행 시간 작업도 꾸준히 진행된다는 것을 보장할 수 있으며, 50 msec 마다 상향되고 따라서 주기적으로 실행된다.

물론 S 값의 결정이 필요하다. S를 얼마로 해야 하는가? 존경받는 시스템 연구자인 John Ousterhout [Ous11]는 이러한 종류의 값을 **부두 상수(voo-doo constants)**라고 불렀다. 이러한 값을 정확하게 결정하기 위해서는 흑마술이 필요한 것처럼 보이기 때문이다. 불행하게도 S는 그러한 종류의 변수이다. 너무 크면 긴 실행 시간을 가진 작업은 굶을 수 있으며 너무 작으면 대화형 작업이 적절한 양의 CPU 시간을 사용할 수 없게 된다.

11.4 시도 3: 더 나은 시간 측정

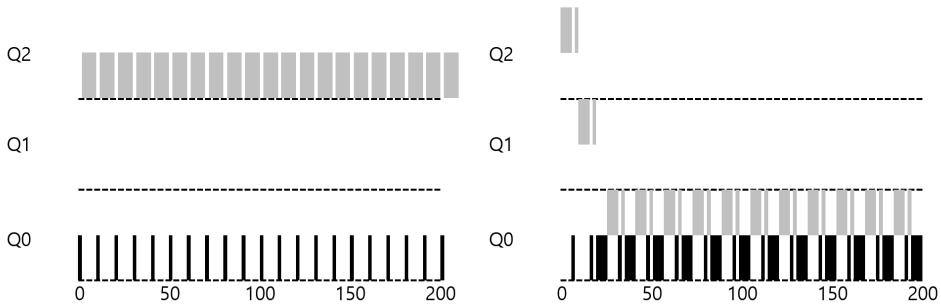
해결해야 할 문제가 하나 더 있다. 스케줄러를 자신에게 유리하게 동작시키는 것을 어떻게 막을 수 있는가? 이러한 일을 가능하게 만든 주범은 규칙 4a와 4b이다. 두 규칙은 작업이 타임 슬라이스가 끝나기 전에 CPU를 양보하여 우선 순위를 유지가 가능하게 한다. 우리는 무엇을 해야 하는가?

여기서의 해결책은 MLFQ의 각 단계에서 CPU 총 사용 시간을 **측정**하는 것이다. 스케줄러는 현재 단계에서 프로세스가 소진한 CPU 사용 시간을 저장한다. 프로세스가 타임 슬라이스에 해당하는 시간을 모두 소진하면 다음 우선순위 큐로 강등된다. 타임 슬라이스를 한 번에 소진하든 짧게 여러 번 소진하든 상관 없다. 규칙 4a와 4b를 합쳐 하나의 규칙으로 재정의한다.



규칙 4: 주어진 단계에서 시간 할당량을 소진하면 (CPU를 몇 번 양도하였는지 상관없이), 우선순위는 낮아진다 (즉, 아래 단계의 큐로 이동한다).

예를 살펴보도록 하자. 그림 11.6은 워크로드가 스케줄러를 자신에게 유리하게 동작시키려고 할 때 예전 규칙 4a와 4b일 때의 행동과(왼쪽) 새로운 조작 방지 규칙 4일 때의 행동 양식을 보이고 있다. 이런 자신에게 유리하도록 조작하는 데 대한 방지책이 없다면 프로세스는 타임 슬라이스가 끝나기 직전에 입출력 명령어을 내릴 수 있어서 CPU 시간을 독점할 수 있다. 방지책이 마련되면 프로세스의 입출력 행동과 무관하게 아래 단계 큐로 천천히 이동하게 되어 CPU를 자기 몫 이상으로 사용할 수 없게 된다.



〈그림 11.6〉 **조작에 대한 내성이 없는 경우(좌측)와 있는 경우(우측)**

11.5 MLFQ 조정과 다른 쟁점들

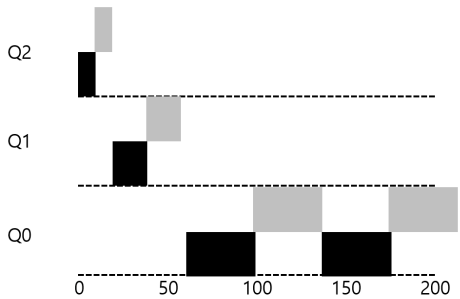
MLFQ 스케줄링에는 여러 다른 쟁점들이 남아 있다. 필요한 변수들을 스케줄러가 어떻게 설정해야 하는지도 중요한 문제다. 예를 들어, 몇 개의 큐가 존재해야 하는가? 큐당 타임 슬라이스의 크기는 얼마로 해야 하는가? 기아를 피하고 변화된 행동을 반영하기 위하여 얼마나 자주 우선순위가 상향 조정되어야 하는가? 이러한 질문들에 대해 쉽게 대답할 수는 없다. 워크로드에 대해 충분히 경험하고 계속 조정해 나가면서 균형점을 찾아야 한다.

예를 들어, 대부분의 MLFQ 기법들은 큐 별로 타임 슬라이스를 변경할 수 있다. 우선순위가 높은 큐는 보통 짧은 타임 슬라이스가 주어진다. 이 큐는 대화형 작업으로 구성되고, 결국 이 작업들을 빠르게 교체하는 것은 의미가 있다(예, 10 msec 이하). 낮은 우선순위는 반대로 CPU-중심의 오래 실행되는 작업들을 포함한다. 긴 타임 슬라이스

팁: 부두 상수 피하기 (Ousterhout's Law)

가능하다면 부두 상수를 피하는 것이 좋은 생각이다. 불행하게도, 앞의 예처럼, 피할 수 없는 경우가 더 자주 일어난다. 더 좋은 값을 찾고 그 방법도 쉽지는 않다. 흔한 결과: 디폴트 값으로 가득 찬 설정 파일. 설정 파일의 값들은 무언가 정확히 동작하지 않을 때 풍부한 경험의 관리자가 조정 가능하다. 상상할 수 있듯이, 이 값은 대부분 있는 그대로 사용되며 디폴트 값이 현장에서 잘 작동하기를 바랄 뿐이다. 이 팁은 오래된 운영체제 교수 John Ousterhout에 의해 알려졌기 때문에 **Ousterhout's Law**라고 부른다.

(예, 수백 msec)가 적합하다. 그림 11.7은 가장 높은 우선순위 큐는 10 ms, 중간 큐는 20 ms, 가장 낮은 큐는 40 ms의 스케줄러에서 두 개의 작업이 실행되는 모양을 보인다.



〈그림 11.7〉 **낮은 우선순위, 더 긴 할당 시간**

Solaris의 MLFQ 구현, 시분할 스케줄링 클래스 또는 TS는 설정이 특히 쉽다. Solaris는 프로세스의 우선순위가 일생 동안 어떻게 변하는지, 타임 슬라이스의 길이는 얼마인지, 작업의 우선순위는 얼마나 자주 상향되는지를 결정하는 테이블을 제공한다 [Arp00]. 관리자는 이 테이블을 수정하여 스케줄러의 동작 방식을 바꿀 수 있다. 테이블의 기본값은 큐의 개수는 60, 각 큐의 타임 슬라이스 크기는 가장 높은 우선순위 큐가 20 msec에서 가장 낮은 우선순위 큐가 수백 msec까지 천천히 증가하고, 우선순위 상향 조정은 1초 정도마다 일어난다.

다른 MLFQ 스케줄러는 테이블이나 이 장에서 설명한 정확한 규칙 같은 것은 사용하지 않는다. 수학 공식을 사용하여 우선순위를 조정한다. 예를 들어, FreeBSD의 스케줄러(버전 4.3)는 작업의 현재 우선순위를 계산하기 위하여 프로세스가 사용한 CPU 시간을 기초로 한 공식을 사용한다 [Lef+89]. CPU 사용은 시간이 지남에 따라 감쇠되어 이 장에서 설명한 방식과는 다른 방식으로 우선순위 상향을 제공한다. 이와 같은 **감쇠-사용**(**decay-usage**) 알고리즘과 특성에 대한 훌륭한 개요는 Epema의 논문을 참조하기 바란다 [Epe95].

마지막으로, 스케줄러들은 다른 여러 기능을 제공한다. 예를 들어, 일부 스케줄러의 경우 가장 높은 우선순위를 운영체제 작업을 위해 예약해 둔다. 일반적인 사용자 작업은 시스템 내에서 가장 높은 우선순위를 얻을 수 없다. 일부 시스템은 사용자가 우선순위를

팁 : 가능하면 조언을 이용하시오

운영체제 시스템이 모든 프로세스에게 어떤 것이 최선인지 알 수는 없다. 사용자 또는 관리자가 힌트를 전달할 수 있는 인터페이스를 제공하는 것이 도움이 된다. 우리는 그러한 **힌트(hint)**를 **조언(advice)**이라고 부른다. 운영체제가 힌트를 반드시 고려할 필요는 없지만 더 나은 결정을 내리는 데 힌트가 도움이 될 수는 있기 때문이다. 이러한 힌트는 스케줄러(예, `nice`), 메모리 관리자(예, `madvise`), 파일 시스템(예, 제공 정보에 기초한 선반입과 캐싱 [Pat+95]) 등을 포함한 운영체제의 여러 부분에서 유용하다.

정하는 데 도움을 줄 수 있도록 허용한다. 예를 들어, 명령어 라인 도구인 nice를 사용하여 작업의 우선순위를 높이거나 낮출 수 있다. 작업의 실행 순서를 바꿀 수 있다. 자세한 사항은 `man` 페이지를 보시오.

11.6 MLFQ: 요약

우리는 멀티 레벨 피드백 큐로 알려진 스케줄링 방법을 기술하였다. 이제 왜 그런 이름으로 불리는지 이해했을 것이다. 알고리즘은 *멀티 레벨* 큐를 가지고 있으며, 지정된 작업의 우선순위를 정하기 위하여 *피드백*을 사용한다. 과거에 보여준 행동이 우선순위 지정의 지침이 된다. 작업이 시간이 지남에 따라 어떻게 행동하고 그에 맞게 어떻게 처리하는지에 대해 주의를 기울여라.

이 장 전체에 산재해 있는 정교한 MLFQ 규칙의 집합을 보기 쉽게 여기에 다시 적어 보았다.

- 규칙 1 : 우선순위(A)> 우선순위(B) 일 경우, A가 실행, B는 실행되지 않는다.
- 규칙 2 : 우선순위(A) = 우선순위(B), A와 B는 RR 방식으로 실행된다.
- 규칙 3 : 작업이 시스템에 들어가면 최상위 큐에 배치된다.
- 규칙 4 : 작업이 지정된 단계에서 배정받은 시간을 소진하면(CPU를 포기한 횟수와 상관없이), 작업의 우선순위는 감소한다(즉, 한 단계 아래 큐로 이동한다).
- 규칙 5 : 일정 주기 S가 지난 후, 시스템의 모든 작업을 최상위 큐로 이동시킨다.

MLFQ는 다음과 같은 측면에서 매우 흥미로운 스케줄러이다. 작업의 특성에 대한 *정보 없이*, 작업의 실행을 관찰한 후 그에 따라 우선순위를 지정한다. MLFQ는 반환 시간과 응답 시간을 모두 최적화한다. 짧게 실행되는 대화형 작업에 대해서는 우수한 전반적인 성능을 제공한다(SJF/ STCF와 유사). 오래 실행되는 CPU-집중 워크로드에 대해서는 공정하게 실행하고 조금이라도 진행되도록 한다. 이런 이유로 BSD UNIX와 여기서 파생된 다양한 운영체제 [Lef+89; Bac86], Solaris [McD06], Windows NT 및 이후 Windows 운영체제 [CS97]를 포함한 많은 시스템이 기본 스케줄러로 MLFQ를 사용한다.

참고 문헌

[Arp00] **"Multilevel Feedback Queue Scheduling in Solaris"**
Andrea Arpaci-Dusseau
URL: `http://www.cs.wisc.edu/%CB%9Cremzi/solaris-notes.pdf`
Solaris 스케줄러에 대한 저자 중 하나의 위대한 짧은 노트 모음. 오케이, 아마 저자의 설명에는 편향된 의견이 들어 있지만 노트들은 정말 훌륭하다.

[Bac86] **"The Design of the Unix Operating System"**
M.J. Bach
Prentice-Hall, 1986
실제 Unix 운영체제가 어떻게 구축되었는지에 관한 고전적인 오래된 책들 중 하나. 커널 해커가 반드시 읽어야 할 필독서.

[CDD62] **"An Experimental Time-Sharing System"**
F.J. Corbato, M.M. Daggett, and R.C. Daley
IFIPS 1962
읽기는 약간 어렵지만 멀티 레벨 피드백 스케줄링의 초창기 아이디어의 원천. 이 중 많은 발상이 Multics에 포함되었으며, 논쟁이 있겠지만 Multics는 지금까지 가장 영향력 있는 운영체제이다.

[CS97] **"Inside Windows NT"**
Helen Custer and David A. Solomon
Microsoft Press, 1997
Unix 이외의 무언가를 배우고 싶다면 이 NT 책을 봐야 한다. 물론 왜 그래야 할까? OK, 농담. 당신이 Microsoft를 위해 언젠가 일할 수도 있으니까.

[Epe95] **"An Analysis of Decay-Usage Scheduling in Multiprocessors"**
D.H.J. Epema
SIGMETRICS '95
감쇠-사용 스케줄러의 배경이 되는 기본적인 방법에 관한 훌륭한 개요를 포함하여 1990년대 중반 당시 스케줄링 기술의 현황에 관한 멋진 논문.

[Lef+89] **"The Design and Implementation of the 4.3BSD Unix Operating System"**
S.J. Leffler, M.K. McKusick, M.J. Karels, and J.S. Quarterman
Addison-Wesley, 1989
BSD를 만든 주요 사람들 중 4인에 의해 쓰여진 또 다른 운영체제의 고전. 이후 버전들은 최신 내용을 담고 있기는 하지만 이 버전의 아름다움에는 미치지 못한다.

[McD06] **"Solaris Internals: Solaris 10 and OpenSolaris Kernel Architecture"**
Richard McDougall
Prentice-Hall, 2006
Solaris와 그의 동작에 관한 좋은 책.

[Ous11] **"John Ousterhout's Home Page"**
John Ousterhout
URL: `http://www.stanford.edu/%CB%9Couster/`
유명한 Ousterhout 교수의 홈페이지. 이 책의 공동저자 모두는 대학원에 있을 때 Ousterhout 교수의 운영체제 강의를 수강했던 즐거운 기억을 가지고 있다. 사실 거기서 서로를 알게 되었고 마침내 결혼, 자녀, 심지어 이 책까지 오게 되었다. 따라서, 당신이 경험하는 이 모든 혼란은 Ousterhout에게 항의할 수도 있겠다.

[Pat+95] **"Informed Prefetching and Caching"**
R.H. Patterson, G.A. Gibson, E. Ginting, D. Stodolsky, and J. Zelenka
SOSP '95
파일 시스템의 매우 근사한 아이디어에 관한 흥미로운 논문. 어떤 파일을 접근할 것인지와 어떻게 접근할 것인지에 관해 응용 프로그램이 운영체제에게 조언을 줄 수 있는 방법을 설명하고 있다.

숙제

프로그램 `mlfq.py`는 지금의 이 장에서 논의한 MLFQ 스케줄러가 어떻게 동작하는지를 살펴볼 수 있게 한다. 자세한 사항은 README를 참조하시오.

문제

1. 두 개의 작업과 두 개의 큐를 무작위로 구성하여 실행시켜 보시오. 각 문제에 대한 MLFQ 실행 추적을 계산하시오. 문제를 쉽게 하기 위해 각 작업의 길이를 제한하고 입출력은 하지 않는다고 가정하시오.
2. 이 장의 예제를 재현하려면 스케줄러를 어떻게 실행해야 하는가?
3. 라운드 로빈 스케줄러처럼 동작시키려면 스케줄러의 매개변수를 어떻게 설정해야 하는가?
4. 두 개의 작업과 스케줄러 매개변수를 가진 워크로드를 고안하시오. 두 작업 중 하나의 작업은 옛날 규칙 4a 및 4b를 이용하여(`-S` 플래그를 켠다) 스케줄러를 자신에게 유리하게 동작하도록 만들어 특정 구간에서 99%의 CPU를 차지하도록 고안해야 한다.
5. 가장 높은 우선순위 큐의 타임 퀀텀의 길이가 10 ms인 시스템이 있다고 하자. 하나의 장기 실행(및 잠재적인 기아 위험) 작업이 적어도 5%의 CPU를 사용할 수 있도록 보장하려면 얼마나 자주 가장 높은 우선순위로 이동시켜야 하는가(`-B` 플래그 켠다)?
6. 스케줄링에서 제기되는 질문 중 하나는, 입출력이 방금 종료된 작업은 큐의 어느 쪽에 추가해야 하는가이다. 플래그 `-I`가 시뮬레이터의 이 행동 양식을 변경한다. 몇 개의 워크로드를 가지고 실험하여 이 플래그의 영향을 확인할 수 있는지 보라.

12

스케줄링 : 비례 배분

이 장에서는 **비례 배분**(**Proportional Share**) 스케줄러, 혹은 **공정 배분**(**fair share**)이라고도 하는 유형의 스케줄러에 대해 다루도록 하겠다. 비례 배분의 개념은 간단하다. 반환 시간이나 응답 시간을 최적화하는 대신 스케줄러가 각 작업에게 CPU의 일정 비율을 보장하는 것이 목적이다.

비례 배분 스케줄링의 좋은 예가 Waldspurger and Weihl [WW94]의 연구다. **추첨 스케줄링**(**lottery scheduling**)으로 알려져 있다. 이 아이디어는 꽤 오래되었다 [KL98]. 기본 아이디어는 매우 간단하다. 다음 실행될 프로세스를 추첨을 통해 결정한다. 더 자주 수행되어야 하는 프로세스는 당첨 기회를 더 많이 준다. 쉽다. 아닌가? 이제 자세한 사항을 알아보자. 그러나 그 전에 핵심 문제를 살펴보자.

핵심 질문 : 어떻게 CPU를 정해진 비율로 배분할 수 있는가
특정 비율로 CPU를 배분하는 스케줄러를 어떻게 설계할 수 있는가? 그렇게 하기 위한 중요한 기법은 무엇인가? 그 기법은 얼마나 효과적인가?

12.1 기본 개념 : 추첨권이 당신의 몫을 나타낸다

추첨권(**티켓**)이라는 기본적인 개념이 추첨 스케줄링의 근간을 이룬다. 추첨권은 프로세스가 (또는 사용자 또는 그 무엇이든) 받아야 할 자원의 몫을 나타내는 데 사용된다. 프로세스가 소유한 티켓의 개수와 전체 티켓에 대한 비율이 자신의 몫을 나타낸다.

예를 살펴보자. A와 B 두 프로세스가 있다고 가정하자. A는 75장의 추첨권을, B는 25장의 추첨권을 가지고 있다. A에게 75%의 CPU를, B에게 남은 25%를 할당하는 것이 목적이다.

추첨 스케줄링은 이러한 목적을 (타임 슬라이스가 끝날 때마다) 확률적으로(하지만 결정적이지는 않게) 달성한다. 추첨 방식은 간단하다. 스케줄러는 전체 몇 장의 추첨권이 있는지 알아야만 한다(우리의 예에서는 100장이다). 스케줄러는 추첨권을 선택한다.

팁 : 무작위성의 이용

추첨권 스케줄링의 큰 장점 중 하나는 **무작위성**이다. 결정이 필요할 때, 이러한 무작위 방식은 강력하고 쉬운 방법이다.

무작위 방식은 전통적인 결정 방식에 비해 세 가지 장점이 있다. 첫째, 무작위 방식은 더 전통적인 방식이 잘 해결하지 못하는 특이 상황을 잘 대응한다. LRU 교체 정책을 생각해 보자(가상 메모리의 나중 장에서 더 자세히 검토). LRU는 좋은 교체 알고리즘이지만 반복되는 순차적인 접근 패턴을 보이는 오버헤드에 대해서는 최악의 성능을 보인다. 반면, 무작위 방식에서는 그러한 최악의 경우가 발생하지 않는다.

둘째, 무작위 추첨 방식은 매우 가볍다. 관리해야 할 상태 정보가 거의 없기 때문이다. 전통적인 공정 배분 스케줄링 알고리즘에서는 각 프로세스가 사용한 CPU 양을 기록해야 한다. 이 정보는 각 프로세스를 실행시킬 때마다 갱신된다. 무작위 추첨 방식에서는 프로세스의 상태 정보만 필요하다(예, 각 프로세스가 가진 추첨권의 개수).

마지막으로 무작위 추첨 방식은 매우 빠르다. 난수 발생 시간이 빠르기만 하면 결정 역시 빠르게 되고 따라서 속도가 요구되는 많은 경우에 사용될 수 있다. 물론, 속도를 증가시키기 위해서 추첨 과정을 덜 무작위(pseudo-random)하게 만들기도 한다.

추첨권은 0 에서 99의 숫자다[1]. A가 0에서 74까지의 티켓을, B는 75에서 99까지의 추첨권을 가지고 있다. 뽑힌 추첨권 값에 따라 실행할 프로세스가 결정된다. 스케줄러는 당첨된 프로세스를 실행한다.

추첨 스케줄러의 당첨 추첨권의 예가 다음에 나와 있다.

```
63 85 70 39 76 17 29 41 36 39 10 99 68 83 63 62 43  0 49 49
```

그에 따른 스케줄링 결과는 다음과 같다.

```
A  A  A  A  A  A  A  A  A  A  A  A  A  A  A  A  A  A  A  A
  B     B                    B  B
```

이 예에서 볼 수 있듯이 무작위성은 원하는 비율을 정확히 보장하지는 않는다. 앞의 예에서 B는 20개의 타임 슬라이스 중에 4개 타임 슬라이스를(20%) 획득한다(원래 목표인 25%가 아니라). 하지만, 두 작업이 장시간 실행될수록, 원하는 비율을 달성할 가능성이 높아진다.

12.2 추첨 기법

추첨권을 다루는 다양한 기법이 있다. 한 가지 기법은 **추첨권 화폐**(**ticket currency**)의 개념이다. 이 개념은 사용자가 추첨권을 자신의 화폐 가치로 추첨권을 자유롭게 할당할 수 있도록 허용한다. 시스템은 자동적으로 화폐 가치를 변환한다.

1) 컴퓨터 과학자들은 항상 0부터 세기 시작한다. 이 관습은 컴퓨터 전공자가 아닌 사람들에게는 매우 이상한 일로 받아들여지기 때문에 저명한 사람들은 항상 우리가 이렇게 하는 이유를 설명해야 한다는 의무감을 느껴 왔다 [Dij82].

팁 : 몫을 표현하기 위한 추첨권의 사용

추첨 (그리고 보폭(stride)) 스케줄링의 설계에서 가장 기본적인 개념은 **추첨권**이다. 이 예에서는 추첨권이 CPU를 사용할 수 있는 프로세스의 몫을 나타내는 데 사용되고 있지만 훨씬 더 널리 적용될 수 있다. 예를 들어, 하이퍼바이저의 가상 메모리 관리에 관한 최근의 연구에서 Waldspurger는 게스트 운영체제의 메모리 할당 지분을 나타내는 데 추첨권이 어떻게 사용될 수 있는지 보여 준다 [Wal02]. 현재 소유 지분을 표현하기 위한 기법이 필요하다면 이 개념은 아마 ... (잠시 기다림) ... 추첨권일 것이다.

예를 들어, 사용자 A, B가 각각 100장의 추첨권을 받았다고 가정하자. 사용자 A는 A1과 A2의 두 개의 작업을 실행 중이고 자신이 정한 화폐로 각각 500장의 추첨권을 할당하였다(전체 1000장의 추첨권 중에). 사용자 B는 하나의 작업을 실행 중이고 자신의 기준 화폐 10장 중에 10장을 할당하였다. 시스템은 A1과 A2의 몫을 A의 기준 화폐 500장에서 전역 기준 화폐 각각 50장으로 변환한다. 마찬가지로, B1의 추첨권 10장은 추첨권 100장으로 변환된다. 전역 추첨권 화폐량 (총 200장) 기준으로 추첨한다.

```
User A -> 500 (A's currency) to A1 -> 50 (global currency)
       -> 500 (A's currency) to A2 -> 50 (global currency)
User B -> 10 (B's currency) to B1 -> 100 (global currency)
```

다른 유용한 기법은 **추첨권 양도(ticket transfer)**이다. 양도를 통하여 프로세스는 일시적으로 추첨권을 다른 프로세스에게 넘겨줄 수 있다. 이 기능은 클라이언트/서버 환경에서 특히 유용하다. 클라이언트 프로세스는 서버에게 메시지를 보내 자신을 대신해 특정 작업을 해달라고 요청한다. 작업이 빨리 완료될 수 있도록 클라이언트는 서버에게 추첨권을 전달하고 서버가 자신의 요청을 수행하는 동안 서버의 성능을 극대화하려고 한다. 요청을 완수하면 서버는 추첨권을 다시 클라이언트에게 되돌려 주고 먼저와 같은 상황이 된다.

마지막으로, **추첨권 팽창(ticket inflation)**도 유용하게 사용된다. 이 기법에서 프로세스는 일시적으로 자신이 소유한 추첨권의 수를 늘이거나 줄일 수 있다. 물론 서로 신뢰하지 않는 프로세스들이 상호 경쟁하는 시나리오에서는 의미가 없다. 하나의 욕심 많은 프로세스가 매우 많은 양의 추첨권을 자신에게 할당하고 컴퓨터를 장악할 수 있다. 화폐 팽창 기법은 프로세스들이 서로 신뢰할 때 유용하다. 그런 경우 어떤 프로세스가 더 많은 CPU 시간을 필요로 한다면 시스템에게 이를 알리는 방법으로 다른 프로세스들과 통신하지 않고 혼자 추첨권의 가치를 상향 조정한다.

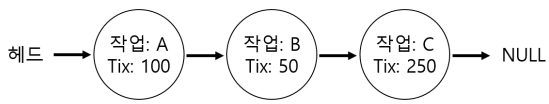
12.3 구현

추첨 스케줄링의 가장 큰 장점은 구현이 단순하다는 점이다. 필요한 것은 난수 발생기와 프로세스들의 집합을 표현하는 자료 구조(예, 리스트), 추첨권의 전체 개수 뿐이다.

리스트를 사용하여 프로세스를 관리한다고 가정하자. A, B 및 C 세 개의 프로세스로 구성되고 각자 몇 장의 추첨권을 가진다.

```
1  // counter: 당첨자를 발견했는지 추적하는 데 사용됨
2  int counter = 0;
3
4  // winner: 0부터 총 추첨권의 수 사이의 임의의 값을 얻기 위해
5  // 난수 발생기를 호출함
6  int winner = getrandom(0, totaltickets);
7
8  // current: 작업 목록을 탐색하는 데 사용
9  node_t *current = head;
10
11 // 티켓 값 > winner를 만족할 때까지 반복
12 while (current) {
13     counter = counter + current->tickets;
14     if (counter > winner)
15         break; // 당첨자 발견
16     current = current->next;
17 }
18 // "current"는 당첨자를 가리킴: 당첨자가 실행될 수 있도록 준비
```

〈그림 12.1〉 **추첨 스케줄링 결정 코드**



스케줄을 결정하기 위해 먼저 총 추첨권의 개수(400)[2] 중에서 한 숫자를 선택해야 한다. 300이 선택되었다고 하자. 리스트를 순회하며 카운터 값을 이용하여 당첨자를 찾아낸다(그림 12.1).

프로세스 리스트를 순회하면서 `counter`의 값이 `winner`의 값을 초과할 때까지 각 추첨권 개수를 `counter`에 더한다. 값이 초과하게 되면 리스트의 현재 원소가 당첨자가 된다. 당첨 번호가 300인 우리의 예에서는 다음과 같이 진행된다. 먼저 A의 추첨권 개수가 더해져서 `counter` 값이 100으로 증가한다. 100은 300보다 작기 때문에 계속 루프를 실행한다. 다음 `counter`는 150(B의 추첨권 개수)으로 갱신되고, 아직 300보다 작기 때문에 다시 순회를 계속한다. 마침내 `counter`는 400(확실히 300보다 큼)으로 갱신되고 루프를 빠져 나오게 되고 `current`는 프로세스 C(당첨자)를 가리키게 된다.

일반적으로 리스트를 내림차순으로 정렬하면 이 과정이 가장 효율적이 된다. 정렬 순서는 알고리즘의 정확성에 영향을 주지는 않는다. 그러나 리스트를 정렬해 놓으면 검색 횟수가 최소화되는 것을 보장한다. 특히, 적은 수의 프로세스가 대부분의 추첨권을 소유하고 있는 경우에 효과적이다.

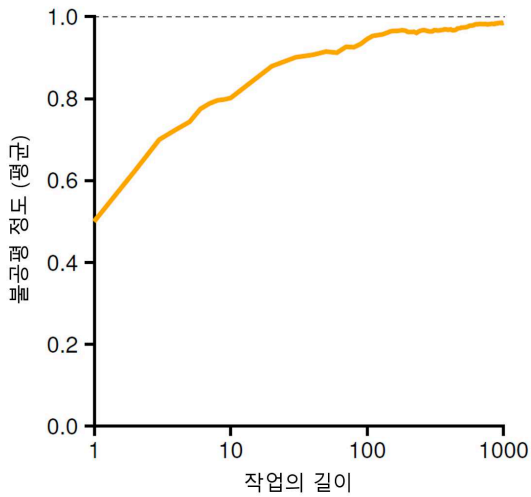
12.4 예제

추첨 스케줄링 동작을 쉽게 이해하기 위해서, CPU를 공유하는 두 개의 작업의 수행 시간을 살펴보자. 각 프로세스는 같은 개수의 추첨권(100)을 가지고 있으며 동일한 실행

2) Björn Lindberg가 지적한 것처럼 놀랍게도 범위 안의 난수를 정확하게 선택하는 것은 어려운 일이다. 자세한 내용은 `http://stackoverflow.com/questions/2509679/how-to-generate-a-random-number-from-within-a-range`

시간을 갖는다(R, 값을 변경할 수 있음).

우리는 두 작업을 거의 동시에 종료시키고자 한다. 그러나 추첨 스케줄링의 무작위성 때문에 한 작업이 다른 작업보다 먼저 종료될 수 있다. 이 차이를 정량화하기 위해, 간단한 **불공정 지표(unfairness metric)**인 U를 정의한다. U는 첫 번째 작업이 종료된 시간을 두 번째 작업이 종료된 시간으로 나눈 값이다. 예를 들어, $R = 10$, 첫 번째 작업은 시간 10에 종료했을 때(그리고 두 번째 작업은 20에서 종료), $U = 10/20 = 0.5$. 두 작업이 거의 동시에 종료하면 U는 1에 근접해진다. 이 시나리오에서는 바로 그게 목표다. 완벽한 공정 스케줄러에서는 $U = 1$을 얻게 된다.



〈그림 12.2〉 **추첨권 배분 방식의 공정성 분석**

그림 12.2은 평균적인 불공정 정도를 보이고 있다. 두 작업의 수행 시간은 1에서 1000까지 변경시켰으며 30번씩 실행시켰다. 결과는 이 장의 마지막에 제공되는 시뮬레이터를 통하여 생성되었다. 그래프에서 보듯 작업 길이가 길지 않은 경우, 평균 불공정 정도는 심각하다. 작업이 충분한 기간 동안 실행되어야 추첨 스케줄러는 원하는 결과에 가까워진다.

12.5 추첨권 배분 방식

추첨 스케줄링에서 아직 언급하지 않은 문제는 추첨권을 작업에게 나누어주는 방식이다. 작업들에게 추첨권을 몇 개씩 분배해야 하는가? 시스템 동작이 추첨권 할당 방식에 따라 크게 달라지기 때문에 상당히 어려운 문제이다. 한 가지 접근 방식은 사용자가 가장 잘 알고 있다고 가정하는 것이다. 각 사용자에게 추첨권을 나누어 준 후 사용자가 알아서 실행시키고자 하는 작업들에게 추첨권을 배분하는 것이다. 하지만 이건 해결책이 아니다. 어떤 일을 해야 하는지 전혀 제시하지 않는다. 주어진 작업 집합에 대한 "추첨권 할당 문제"는 여전히 미해결 상태다.

12.6 왜 결정론적(Deterministic) 방법을 사용하지 않는가

위에서 본 바와 같이, 무작위성을 이용하면 스케줄러를 단순하게(그러나 어느 정도 정확하게) 만들 수 있지만, 정확한 비율을 보장할 수 없다. 짧은 기간만 실행되는 경우는 더 그렇다. 이 때문에 Waldspurger는 결정론적 공정 배분 스케줄러인 **보폭 스케줄링(stride scheduling)** [Wal95]을 고안하였다.

보폭 스케줄링 역시 이해하기 쉽다. 시스템의 각 작업은 보폭을 가지고 있다. 보폭은 자신이 가지고 있는 추첨권 수에 반비례하는 값이다. 앞의 예에서 작업 A, B, C는 각각 100, 50, 250의 추첨권을 가지고 있으며 임의의 큰 값을 각자의 추첨권 개수로 나누어 보폭을 계산할 수 있다. 예를 들어 10,000을 각자의 추첨권 개수로 나누면 각 작업의 보폭은 100, 200 및 40이 된다. 이 값을 **보폭(stride)**이라고 부르며 프로세스가 실행될 때마다 **pass**라는 값을 보폭만큼 증가시켜 얼마나 CPU를 사용하였는지를 추적한다.

스케줄러는 보폭과 pass 값을 사용하여 어느 프로세스를 실행시킬지 결정한다. 기본적인 아이디어는 간단하다. 가장 작은 pass 값을 가진 프로세스를 선택한다. 프로세스를 실행시킬 때마다 pass 값을 보폭만큼씩 증가시킨다. Waldspurger [Wal95]가 작성한 의사코드는 아래와 같다:

```
current = remove_min(queue);   // pass 값이 최소인 클라이언트로 선택
schedule(current);             // 자원을 타임 퀀텀만큼 선택된 프로세스에게 할당
current->pass += current->stride; // 다음 pass 값을 보폭 값을 이용하여 갱신
insert(queue, current);        // 다시 큐에 저장한다
```

세 작업(A, B, C), 각자의 보폭은 각각 100, 200, 40 이다. 각자의 pass 값은 모두 0에서 시작한다. 처음에는 pass 값이 같기 때문에 아무 프로세스나 실행될 수 있다. A를 선택했다고 가정하자(임의로, 값이 제일 작은 프로세스 중에서 아무거나). A가 실행된다. 타임 슬라이스만큼 실행되고 종료될 때 pass 값을 100으로 갱신한다. 다음에 B가 실행되고 해당 pass 값이 200으로 갱신된다. 마지막으로 C를 실행하고, pass 값은 40으로 갱신된다. 이 시점에서 알고리즘은 가장 작은 pass 값을 가진 C를 선택하고 80으로 갱신한다(C의 보폭은 40이다). 다음 C가 다시 실행되고 (아직 최소 pass 값) 120으로 갱신된다. 이제 A가 실행되고 200으로 갱신된다(이제 B와 같아진다). 다음 C가 두 번 더 실행되고 pass 값은 160을 거쳐 200으로 갱신된다. 이 시점에서 모든 pass 값은 다시 같아지게 되고 이런 선택 과정이 계속 반복된다. 그림 12.3에 스케줄러의 동작이 나와 있다.

그림에서 알 수 있듯이, C는 5번, A는 2번, B는 한 번만 실행되었다. 이 횟수는 각자 가진 추첨권의 개수 250, 100, 50과 정확히 비례한다. 추첨 스케줄링은 정해진 비율에 따라 확률적으로 CPU를 배분한다. 보폭 스케줄링은 각 스케줄링 주기마다 정확한 비율로 CPU를 배분한다.

그렇다면 이런 질문을 던질지 모른다. 보폭 스케줄링의 정확도를 고려할 때, 추첨 스케줄링을 왜 사용하는가? 추첨 스케줄링은 보폭 스케줄링이 가지고 있지 않은 멋진 특성을 가지고 있다. 상태 정보가 필요 없다. 위 보폭 스케줄링의 예에서 스케줄링 중간에 새로운 작업이 시스템에 도착했다고 상상해 보자. pass 값은 얼마가 되어야 하는가? 0으로 지정되어야 하는가? 그렇다면 CPU를 독점하게 될 것이다. 추첨 스케줄링에서는

Pass(A) (보폭=100)	Pass(B) (보폭=200)	Pass(C) (보폭=40)	실행은?
0	0	0	A
100	0	0	B
100	200	0	C
100	200	40	C
100	200	80	C
100	200	120	A
200	200	120	C
200	200	160	C
200	200	200	...

〈그림 12.3〉 **보폭 스케줄링 : 추적**

프로세스 상태(CPU 사용 현황, pass 값)를 유지할 필요가 없다. 새 프로세스를 추가할 때, 새로운 프로세스가 가진 추첨권의 개수, 전체 추첨권의 개수만 갱신하고 스케줄한다. 이런 이유로 추첨 스케줄링에서는 새 프로세스를 쉽게 추가할 수 있다.

12.7 요약

우리는 비례 배분 스케줄링의 개념을 소개하고, 추첨권 및 보폭 스케줄링이라는 두 가지 구현 방식에 대해 간단하게 논의하였다. 추첨권 스케줄링은 무작위성(randomness)을 사용한다. 보폭 스케줄링은 같은 목적을 결정적 방법으로 달성한다. 둘 다 개념적으로 흥미롭지만 여러 이유로 CPU 스케줄러로서 널리 사용되고 있지 않다. 한 가지 이유는 이러한 접근 방식이 특히 입출력과 맞물렸을 때, 제대로 동작하지 않는다는 것이다 [AC97]. 또 다른 이유는 추첨권 할당이라는 어려운 문제가 미해결 상태로 남아 있다는 것이다. 즉, 브라우저에 얼마나 추첨권을 할당해야 하는가? 범용 스케줄러(앞서 언급한 MLFQ 그리고 다른 유사 Linux 스케줄러 같은)는 그러한 문제를 더 직관적으로 해결하고, 때문에 더 널리 사용되고 있다.

비례 배분 스케줄러는 추첨권 할당량을 비교적 정확하게 결정할 수 있는 환경에서 유용하게 사용된다. 예를 들어, **가상화** 데이터 센터에서 Windows 가상 머신에 CPU 사이클의 1/4을 할당하고 나머지는 Linux 시스템에 할당하고 싶은 경우 비례 배분이 간단하고 효과적이다. 이러한 기법이 VMware ESX 서버에서 메모리를 비례 배분하는 데 어떻게 사용되는지를 알고 싶으면 Waldspurger [Wal02]를 참조하시오.

참고 문헌

[AC97] **"Extending Proportional-Share Scheduling to a Network of Workstations"**
Andrea C. Arpaci-Dusseau and David E. Culler
PDPTA'97, June 1997
클러스터 환경에서 더 잘 동작하도록 비례 배분 스케줄링을 확장하는 방법에 대한 본 저자 중 하나의 논문.

[Dij82] **"Why Numbering Should Start At Zero"**
Edsger Dijkstra
URL: `http://www.cs.utexas.edu/users/EWD/ewd08xx/EWD831.PDF`
컴퓨터 과학의 선구자 중 한 사람인 E. Dijkstra의 짧은 메모. 병행성 섹션에서 이 사람에 대해 더 많이 듣게 될 것이다. 한편 이 노트를 즐기도록. 이 노트는 다음과 같은 인용을 포함한다. "내 동료 중 한 사람이—컴퓨터 과학자가 아닌—젊은 컴퓨터 과학자들을 현학적이라고 비난하였다. 이들이 숫자를 0부터 센다는 것이 비난의 이유였다." 이 노트는 0부터 세는 것이 논리적인 이유에 대해 설명한다.

[KL98] **"A Fair Share Scheduler"**
J. Kay and P. Lauder
CACM, Volume 31 Issue 1, January 1988
공정 배분 스케줄러에 대한 초기 참고 문헌.

[Wal95] **"Lottery and Stride Scheduling: Flexible Proportional-Share Resource Management"**
Carl A. Waldspurger
Ph.D. Thesis, MIT, 1995
추첨권과 보폭 스케줄링의 개요를 설명하는 Waldspurger의 수상 논문. 언젠가 박사 학위 논문을 쓸 생각이 있다면 주위에 좋은 본보기를 가지고 있어야 한다. 바로 이 논문이 그러한 좋은 본보기이다.

[Wal02] **"Memory Resource Management in VMware ESX Server"**
Carl A. Waldspurger
OSDI '02, Boston, Massachusetts
VMM (일명 hypervisor)에서의 메모리 관리에 관한 논문. 읽기 쉬울 뿐 아니라 논문에는 이 새로운 VMM-단계의 메모리 관리에 관한 수많은 멋진 아이디어가 포함되어 있다.

[WW94] **"Lottery Scheduling: Flexible Proportional-Share Resource Management"**
Carl A. Waldspurger and William E. Weihl
OSDI '94, November 1994
추첨 스케줄링에 관한 획기적인 논문. 이 논문은 시스템 커뮤니티가 스케줄링, 공정 배분 및 단순 무작위 알고리즘의 힘에 관한 연구를 다시 할 수 있도록 활력을 불어 넣었다.

숙제

lottery.py 프로그램은 추첨 스케줄러의 동작을 살펴볼 수 있게 한다. 자세한 사항은 README를 참조하시오.

문제

1. 3개의 작업 및 1, 2, 3의 랜덤 시드를 위한 시뮬레이션을 위한 해결책을 계산하시오.
2. 다음 두 가지 특정 작업을 실행시켜 보라. 각 작업의 길이는 10, 작업 0은 1장의 추첨권, 작업 1은 100장의 추첨권을 가진다(예, -l 10:1,10:100). 추첨권의 개수가 심하게 불균형을 이룰 경우 어떻게 동작하나? 작업 0은 작업 1이 종료하기 전에 실행될 수 있는가? 얼마나 자주 실행될 것인가? 일반적으로 이러한 추첨권의 불균형은 추첨 스케줄링의 동작에 어떤 영향을 미치는가?
3. 작업의 길이가 100이고 100장의 같은 추첨권을 가진 두 작업(-l 100:100,100:100)을 실행시킨다면 스케줄러는 얼마나 불공정한가? (확률적인) 답을 결정하기 위해 몇몇 다른 랜덤 시드를 가지고 실행해 보시오. 불공정성은 하나의 작업이 다른 작업보다 얼마나 일찍 종료하느냐를 기준으로 결정된다고 하자.
4. 타임 퀀텀의 크기(-q)가 커질수록 이전 질문에 대한 대답은 어떻게 바뀔까?
5. 이 장에서 나온 것 같은 그래프를 만들 수 있는가? 탐구해 볼만한 다른 것은 무엇인가? 보폭 스케줄러의 그래프는 어떻게 될까?

스케줄링: 비례 배분

영어판에만 있는 추가 내용 번역

대조 기준: 한국어판 전체와 Version 1.20 영어 원문을 문단 단위로 비교했습니다. 이미 한국어판에 있는 내용은 반복하지 않고, 영어판에만 있는 내용만 번역했습니다. 절 번호는 영어 원문의 번호를 유지했습니다.

9.7 Linux 완전 공정 스케줄러(CFS)

앞서 살펴본 공정 배분 스케줄링 연구와는 별개로, Linux의 한 가지 전통적인 방식은 비슷한 목표를 다른 방법으로 달성한다. 완전 공정 스케줄러(Completely Fair Scheduler, CFS)[J09]는 공정 배분 스케줄링을 매우 효율적이고 확장 가능한 방식으로 구현한다.

CFS는 설계 자체와 작업에 알맞은 자료 구조를 영리하게 활용하여 스케줄링 결정에 드는 시간을 최소화한다. 최근 연구에 따르면 스케줄러의 효율성은 놀라울 정도로 중요하다. Kanev 등의 Google 데이터 센터 연구에서는 적극적으로 최적화한 뒤에도 스케줄링에 전체 데이터 센터 CPU 시간의 약 5%가 사용되는 것으로 나타났다[K+15]. 따라서 이 오버헤드를 가능한 한 줄이는 것이 현대 스케줄러 구조의 핵심 목표다.

기본 동작

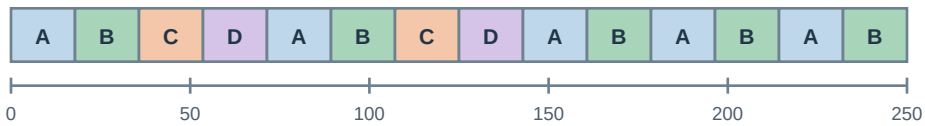
대부분의 스케줄러가 고정된 타임 슬라이스를 중심으로 동작하는 것과 달리 CFS는 조금 다르게 동작한다. 목표는 간단하다. 경쟁 중인 모든 프로세스가 CPU를 공평하게 나누어 사용하도록 하는 것이다. 이를 위해 가상 실행 시간(virtual runtime, vruntime)이라는 단순한 누적 기법을 사용한다.

각 프로세스는 실행될 때마다 vruntime이 누적된다. 가장 기본적인 경우 모든 프로세스의 vruntime은 실제 시간에 비례하여 같은 속도로 증가한다. 스케줄링 결정을 내릴 때 CFS는 vruntime이 가장 작은 프로세스를 다음 실행 대상으로 선택한다.

그러면 현재 실행 중인 프로세스를 언제 멈추고 다음 프로세스를 실행해야 할까? CFS가 너무 자주 전환하면 아주 짧은 시간 구간에서도 각 프로세스의 CPU 몫을 보장할 수 있어 공정성은 높아지지만, 문맥 교환이 지나치게 많아져 성능이 떨어진다. 반대로 전환 횟수를 줄이면 문맥 교환이 감소해 성능은 좋아지지만 단기적인 공정성이 낮아진다.

CFS는 여러 제어 매개변수로 이 균형을 조절한다. 첫 번째는 `sched_latency`다. CFS는 이 값으로 프로세스 하나를 얼마나 실행한 뒤 전환을 검토할지 결정한다. 즉, 동적으로 타임 슬라이스를 정한다. 일반적인 `sched_latency` 값은 48ms다. CFS는 이 값을 CPU에서 실행 중인 프로세스 수 n으로 나누어 프로세스별 타임 슬라이스를 계산하고, 이 기간 전체에서 완전한 공정성을 보장하려 한다.

예를 들어 실행 중인 프로세스가 n=4개라면 48ms를 4로 나누어 각 프로세스에 12ms의 타임 슬라이스를 준다. CFS는 첫 번째 작업을 12ms의 가상 실행 시간 동안 실행한 다음, vruntime이 더 작은 다른 작업이 있는지 확인한다. 이 경우 나머지 세 작업 중 하나로 전환한다. 아래 예에서는 A, B, C, D 네 작업이 각각 두 번의 타임 슬라이스를 사용한 뒤 C와 D가 종료된다. 이후 남은 A와 B는 각각 24ms씩 라운드 로빈 방식으로 실행된다.



[그림 9.4] CFS의 단순한 실행 예

실행 중인 프로세스가 너무 많다면 타임 슬라이스가 지나치게 짧아져 문맥 교환이 너무 잦아질 수 있다. 이 문제를 해결하기 위해 CFS는 보통 6ms 정도로 설정되는 `min_granularity`라는 매개변수를 추가로 사용한다. CFS는 프로세스의 타임 슬라이스를 이 값보다 짧게 설정하지 않는다.

예를 들어 프로세스가 10개라면 원래 계산에서는 48ms를 10으로 나눈 4.8ms가 타임 슬라이스가 된다. 그러나 `min_granularity` 때문에 각 프로세스의 타임 슬라이스는 6ms로 설정된다. 목표 스케줄링 지연 시간인 48ms 동안 완벽히 공정하지는 않지만, 공정성에 가깝게 동작하면서 높은 CPU 효율을 유지한다.

CFS는 주기적인 타이머 인터럽트를 사용하므로 고정된 시간 간격에서만 결정을 내릴 수 있다. 이 인터럽트는 보통 1ms처럼 매우 짧은 간격으로 발생하며, CFS는 현재 작업이 실행 구간의 끝에 도달했는지 확인한다. 타임 슬라이스가 타이머 인터럽트 간격의 정확한 배수가 아니어도 괜찮다. CFS가 vruntime을 정밀하게 추적하기 때문에 장기적으로는 이상적인 CPU 배분에 가까워진다.

가중치와 nice 값

CFS는 프로세스 우선순위를 조절할 수 있으며, 사용자나 관리자가 특정 프로세스에 더 많은 CPU 몫을 줄 수 있다. CFS는 추첨권 대신 UNIX의 전통적인 프로세스 nice 값을 사용한다. nice 값은 -20부터 +19까지 설정할 수 있고 기본값은 0이다. 양수는 낮은 우선순위, 음수는 높은 우선순위를 뜻한다. 너무 '친절한(nice)' 프로세스는 스케줄러의 관심을 덜 받는 셈이다.

CFS는 다음 표처럼 각 nice 값을 가중치로 변환한다.

```
static const int prio_to_weight[40] = {
 /* -20 */ 88761, 71755, 56483, 46273, 36291,
 /* -15 */ 29154, 23254, 18705, 14949, 11916,
 /* -10 */  9548,  7620,  6100,  4904,  3906,
 /*  -5 */  3121,  2501,  1991,  1586,  1277,
 /*   0 */  1024,   820,   655,   526,   423,
 /*   5 */   335,   272,   215,   172,   137,
 /*  10 */   110,    87,    70,    56,    45,
 /*  15 */    36,    29,    23,    18,    15
};
```

이 가중치를 사용하면 우선순위 차이를 반영한 각 프로세스의 실질적인 타임 슬라이스를 다음과 같이 계산할 수 있다. n은 프로세스 수다.

```
time_slice(k) = weight(k) / sum(weight(i), i=0..n-1) * sched_latency
```

예를 들어 작업 A와 B가 있다고 하자. 중요한 작업 A에는 nice 값 -5를 주고, B에는 기본값 0을 준다. 원문은 B를 싫어하기 때문이라며 일부러 'we hates it'이라는 어색한 문법을 쓰는데, 이는 『반지의 제왕』의 골룸을 가볍게 인용한 표현이다. 표에서 A의 가중치는 3121이고 B의 가중치는 1024다. 계산하면 A는 `sched_latency`의 약 3/4인 36ms를, B는 약 1/4인 12ms를 얻는다.

타임 슬라이스 계산을 일반화했으므로 vruntime 계산도 바뀌어야 한다. 프로세스 i가 실제로 사용한 실행 시간 `runtime(i)`에 기본 가중치 1024를 해당 프로세스의 가중치로 나눈 값을 곱한다.

```
vruntime(i) = vruntime(i) + weight(0) / weight(i) * runtime(i)
```

위 예에서 A의 vruntime은 B 속도의 약 1/3로 증가한다. 가중치 표의 영리한 점은 nice 값의 차이가 일정하면 CPU 배분 비율도 그대로 유지된다는 것이다. 예를 들어 A의 nice 값이 -5가 아니라 5이고, B가 0이 아니라 10이어도 두 프로세스는 이전과 똑같은 비율로 스케줄된다.

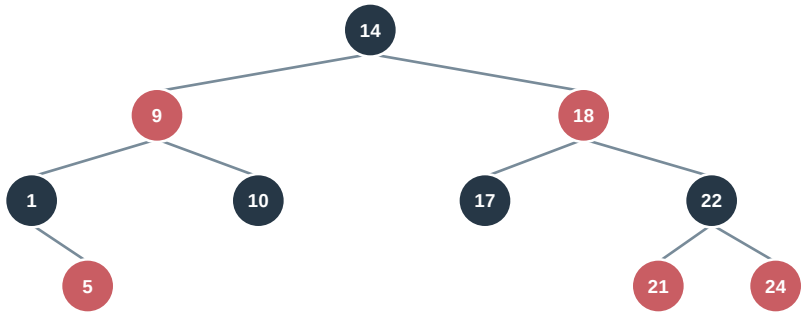
레드-블랙 트리 사용

CFS의 중요한 목표 중 하나는 효율성이다. 스케줄러가 다음 실행 작업을 찾을 때 가능한 한 빨리 찾아야 한다. 단순 리스트는 확장성이 좋지 않다. 현대 시스템에는 수천 개의 프로세스가 존재할 수 있기 때문에 몇 ms마다 긴 리스트를 검색하는 것은 낭비다.

CFS는 프로세스를 레드-블랙 트리(red-black tree)[B72]에 저장한다. 레드-블랙 트리는 균형 트리의 한 종류다. 최악의 삽입 순서에서 리스트처럼 길어질 수 있는 단순 이진 트리와 달리, 균형 트리는 깊이를 낮게 유지하는 추가 작업을 수행한다. 그 결과 연산 시간을 선형 시간이 아니라 로그 시간으로 보장한다.

모든 프로세스가 이 트리에 들어가는 것은 아니다. 실행 중이거나 실행 가능한 프로세스만 저장된다. 입출력 완료나 네트워크 패킷을 기다리며 잠든 프로세스는 트리에서 제거해 다른 곳에서 관리한다.

예를 들어 10개 작업의 vruntime이 1, 5, 9, 10, 14, 18, 17, 21, 22, 24라고 하자. 정렬 리스트라면 다음 작업은 첫 항목을 제거하기만 하면 되지만, 실행 후 제자리에 다시 삽입하려면 리스트를 훑어야 하므로 O(n)이 걸린다. 검색 역시 평균적으로 선형 시간이 든다.



[그림 9.5] vruntime을 기준으로 구성한 CFS 레드-블랙 트리

같은 값을 레드-블랙 트리에 저장하면 삽입과 삭제를 비롯한 대부분의 연산이 O(log n)에 수행된다. 프로세스 수가 수천 개에 이르면 로그 시간은 선형 시간보다 눈에 띄게 효율적이다.

입출력 및 잠든 프로세스 처리

vruntime이 가장 작은 프로세스를 선택하는 방식은 오랫동안 잠들었던 작업에서 문제가 된다. A는 계속 실행되고 B는 10초 동안 잠들었다고 하자. B가 깨어났을 때 B의 vruntime은 A보다 10초 뒤처져 있다. 조심하지 않으면 B가 그 차이를 따라잡는 동안 다음 10초 동안 CPU를 독점해 A를 굶길 수 있다.

CFS는 작업이 깨어날 때 vruntime을 조정하여 이 문제를 처리한다. 구체적으로 깨어난 작업의 vruntime을 트리에서 발견되는 최솟값으로 설정한다. 이 트리에는 실행 가능한 작업만 들어 있다는 점을 기억하자[B+18]. 이렇게 하면 기아 상태는 피할 수 있지만, 짧은 시간 동안 자주 잠드는 작업은 오히려 자신의 공정한 몫을 얻지 못할 수 있다[AC97].

그 밖의 CFS 기능

CFS에는 이 책의 이 지점에서 모두 다루기에는 너무 많은 기능이 있다. 캐시 성능을 높이는 여러 경험적 기법, 여러 CPU를 효과적으로 다루는 전략, 각 프로세스를 독립적으로 취급하는 대신 큰 프로세스 그룹 단위로 스케줄하는 기능 등이 포함된다. 자세한 내용은 Bouron 등의 최근 연구[B+18]에서 시작해 살펴볼 수 있다.

팁 - 상황에 맞는 효율적인 자료 구조를 사용하라. 많은 경우 리스트로 충분하지만 항상 그런 것은 아니다. 데이터 센터의 부하가 큰 서버에는 수천 개의 활성 프로세스가 있을 수 있다. 각 코어에서 몇 ms마다 긴 리스트를 검색하면 귀중한 CPU 사이클을 낭비한다. CFS는 레드-블랙 트리를 사용해 이 문제를 해결했다. 시스템을 설계할 때는 자료 접근 패턴과 사용 빈도를 함께 고려해야 작업에 맞는 자료 구조를 선택할 수 있다.

개정판 요약에 추가된 내용

개정 영어판은 비례 배분 방식으로 추첨 스케줄링, 보폭 스케줄링, CFS의 세 가지를 정리한다. 추첨 스케줄링은 무작위성을, 보폭 스케줄링은 결정론적 방법을 사용한다. CFS는 동적인 타임 슬라이스를 쓰는 가중 라운드 로빈과 비슷하지만, 높은 부하에서도 확장성과 성능을 유지하도록 설계된 실제 스케줄러이며 오랫동안 가장 널리 사용된 공정 배분 스케줄러였다.

개정판은 입출력을 수행하는 작업이 공정한 CPU 몫을 얻지 못할 수 있다는 점과, 추첨권 또는 우선순위 자체를 어떻게 정할지 여전히 어렵다는 점을 강조한다. 예를 들어 브라우저에 추첨권을 몇 장 주어야 하는지, 텍스트 편집기의 nice 값을 얼마로 해야 하는지는 자동으로 결정하기 어렵다.

Linux의 세계는 계속 변한다. Linux 6.6부터는 또 다른 공정 배분 방식인 EEVDF(Earliest Eligible Virtual Deadline First) 스케줄러가 기본값이 되었다. 자세한 내용은 EEVDF 자료[Z23, SA95]를 참고하라.

영어 개정판에서 추가된 참고문헌

[B+18] J. Bouron 외, ‘The Battle of the Schedulers: FreeBSD ULE vs. Linux CFS’, USENIX ATC 2018. Linux CFS와 FreeBSD ULE 스케줄러를 자세히 비교한 연구다. 두 스케줄러의 개요도 훌륭하다. 비교 결과는 한쪽의 완승이 아니라, 상황에 따라 CFS 또는 ULE가 더 나은 것으로 나타났다.

[B72] Rudolf Bayer, ‘Symmetric Binary B-Trees: Data Structure And Maintenance Algorithms’, Acta Informatica, 1972. 여러 균형 트리 가운데 하나를 소개한 초기 논문이다.

[K+15] S. Kanev 외, ‘Profiling A Warehouse-scale Computer’, ISCA 2015. 현대 데이터 센터에서 CPU 시간이 어디에 사용되는지 분석했다. CPU 시간의 약 20%가 운영체제에, 약 5%가 스케줄러에 사용되는 것으로 나타났다.

[J09] M. Tim Jones, ‘Inside The Linux 2.6 Completely Fair Scheduler’, 2009. 초기 CFS를 간단히 설명한다. CFS는 Ingo Molnar가 62시간 동안 개발한 약 10만 줄 규모의 커널 패치에서 시작되었다.

[SA95] Ion Stoica, Hussein M. Abdel-Wahab, ‘Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation’, Old Dominion University 기술 보고서 TR-95-22. EEVDF를 처음 소개한 기술 보고서다.

[Z23] Peter Zijlstra, ‘The EEVDF Scheduler’. Linux 커널 문서의 EEVDF 설명 자료다: https://docs.kernel.org/scheduler/sched-eevdf.html

13

멀티프로세서 스케줄링 (고급)

이 장에서는 **멀티프로세서 스케줄링**(**multiprocessor scheduling**)의 기본을 소개한다. 이 주제는 다소 고급에 속하기 때문에 병행성(concurrency) 주제를 어느 정도 깊게 공부한 이후에 다루는 것이 가장 좋다. 병행성은 이 책의 두 번째 주요 내용이다.

고사양 컴퓨터에만 존재했던 **멀티프로세서**(**multiprocessor**) 시스템은 일반적이 되었으며, 데스크톱 컴퓨터, 노트북, 심지어 모바일 장치에도 사용되고 있다. 여러 개의 CPU 코어가 하나의 칩에 내장된 **멀티코어**(**multicore**) 프로세서가 대중화의 근본 원인이다. 싱글코어 CPU의 성능 개선이 한계에 봉착하면서 멀티코어 기술이 각광을 받게 되었다. 우리는 다수의 CPU를 사용할 수 있게 되었다. 좋은 소식이지 않은가?

다중 CPU 시대가 오면서 많은 문제가 발생하였다. 가장 중요한 것은 전통적 응용 프로그램은 (예, 당신이 작성한 C 프로그램) 오직 하나의 CPU만 사용한다는 것이다. 더 많은 CPU를 추가해도 더 빨리 실행되지 않는다. 이 문제를 해결하려면 응용 프로그램을 **병렬**(**parallel**)로 실행되도록 다시 작성해야 한다. 보통 **쓰레드**를 이용한다. 쓰레드는 이 책의 두 번째 부분에서 매우 상세하게 설명된다. 멀티 쓰레드 응용 프로그램은 작업을 여러 CPU에 할당하며, 따라서 더 많은 수의 CPU가 주어지면 더 빠르게 실행된다.

응용 프로그램뿐 아니라 운영체제가 새로이 직면한 문제는 (놀랍지도 않게!) 멀티프로세서 스케줄링이다. 지금까지 우리는 단일프로세서 스케줄링의 많은 원칙들에 대해 논의하였다. 이 아이디어를 여러 CPU에서 동작하도록 어떻게 확장할 수 있을까? 우리가 해결해야 하는 새로운 문제는 무엇인가? 문제는 다음과 같다.

핵심 질문: 여러 CPU에 작업을 어떻게 스케줄 해야 하는가
운영체제는 어떻게 작업을 여러 CPU에 스케줄 해야 하는가? 어떤 새로운 문제가 등장하는가? 예전 기술을 적용할 것인가 아니면 새로운 아이디어가 필요한가?

13.1 배경: 멀티프로세서 구조

멀티프로세서 스케줄링에 대한 새로운 문제점을 이해하기 위해서는 단일 CPU 하드웨어와 멀티 CPU 하드웨어의 근본적인 차이에 대한 이해가 필요하다. 다수의 프로세서

여담: 고급 주제의 장들

고급 주제의 장을 제대로 이해하기 위해서는 책의 전반적인 내용에 대한 이해를 필요로 한다. 미리 알아야 할 선행 내용보다 먼저 나오는 것이 책의 구성상 더 적절한 경우도 많다. 예를 들어, **멀티프로세서 스케줄링**에 관한 이 장은 먼저 병행성에 관한 중간 부분을 읽은 후에 더 잘 이해가 될 것이다. 그러나 논리적으로는 가상화(일반적으로)와 CPU 스케줄링(구체적으로)에 관한 부분에 포함되는 것이 더 적합하다. 그러한 장들은 순서와 상관없이 먼저 읽을 것을 권한다. 이 경우에는 책의 두 번째 부분에 해당한다.

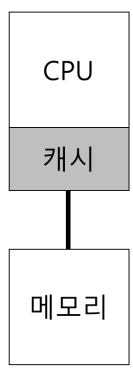
간에 데이터의 공유, 그리고 하드웨어 **캐시**의 사용 방식(예, 그림 13.1)에서 근본적인 차이가 발생한다. 우리는 이 문제를 개념적으로만 다루도록 하겠다. 구체적인 내용은 다른 과목 [CSG99], 특히 3, 4학년 혹은 대학원 수준의 컴퓨터 구조 수업에서 다룬다.

단일 CPU 시스템에는 **하드웨어 캐시** 계층이 존재한다. 이 캐시는 프로그램을 빠르게 실행하기 위해 존재한다. 캐시는 메인 메모리에서 *자주* 사용되는 데이터의 복사본을 저장하는 작고 빠른 메모리이다. 메인 메모리는 모든 데이터를 저장하지만 느리다. 자주 접근되는 데이터를 캐시에 임시로 가져다 둠으로써 시스템은 크고 느린 메모리를 빠른 메모리처럼 보이게 한다.

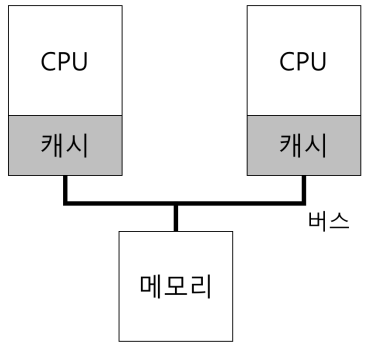
예를 들어, `load` 명령어를 수행하는 프로그램과 하나의 CPU만 있는 간단한 시스템을 생각해 보자. CPU는 작은 크기의 캐시(64 KB)와 큰 메인 메모리를 갖고 있다.

프로그램이 처음 이 `load` 명령어를 실행할 때, 데이터가 메인 메모리에 존재하므로 데이터를 가져오는 데 긴 시간이 소모된다(아마 수십 또는 수백 nano second). 데이터가 다시 사용될 것으로 예상한 프로세서는 읽은 데이터의 복사본을 CPU 캐시에 저장한다. 프로그램이 나중에 다시 같은 데이터를 가져오려고 하면, CPU는 우선 해당 데이터가 캐시에 존재하는지 검사한다. 캐시에 존재하기 때문에 데이터는 훨씬 빨리 접근되고(수 nano second) 프로그램은 빨리 실행한다.

캐시는 **지역성**(**locality**)에 기반한다. 지역성에는 **시간 지역성**(**temporal locality**)과 **공간 지역성**(**spatial locality**)의 두 종류가 있다. 시간적 지역성의 기본 아이디어는



〈그림 13.1〉 **캐시가 장착된 하나의 CPU**



〈그림 13.2〉 **메모리를 공유하는 캐시가 장착된 두 개의 CPU**

데이터가 한 번 접근되면 가까운 미래에 다시 접근되기 쉽다는 것이다. 루프에서 여러 번 반복해서 접근되는 변수 또는 명령어 자체를 상상해 보라. 공간적 지역성의 기본 아이디어는 프로그램이 주소 x의 데이터를 접근하면 x 주변의 데이터가 접근되기 쉽다는 것이다. 전체 배열을 차례대로 접근하는 프로그램 또는 순차적으로 실행되는 명령어들을 생각해 보자. 이런 유형의 지역성은 많은 프로그램에서 관측된다. 하드웨어 시스템은 캐시에 어떤 데이터를 저장할지 비교적 정확하게 추측을 할 수 있고, 캐시는 잘 작동한다.

자 이제부터 까다로운 부분: 그림 13.2에서 보는 바와 같이 하나의 시스템에 여러 프로세서가 존재하고 하나의 공유 메인 메모리가 있을 때 어떤 일이 일어나는가?

알려진 것처럼 멀티프로세서 시스템에서 캐시를 사용하는 것은 훨씬 복잡하다. 예를 들어, CPU 1에서 실행 중인 프로그램이 주소 A를(D 값을) 읽는다고 가정하자. 데이터가 CPU 1 캐시에 존재하지 않기 때문에 시스템은 메인 메모리로부터 데이터를 가져오고 값 D를 얻는다. 그런 후 프로그램은 주소 A의 값을 변경한다. 변경은 캐시에 존재하는 값만 D'으로 갱신한다. 메모리에 데이터를 쓰는 것은 시간이 오래 걸리므로 메인 메모리에 기록하는 것은 보통 나중에 한다. 운영체제가 프로그램의 실행을 중단하고 CPU 2로 이동하기로 결정한다고 가정하자. 프로그램은 주소 A의 값을 다시 읽는다. CPU 2의 캐시에는 그러한 데이터가 존재하지 않고 따라서 시스템은 메인 메모리에서 데이터를 가져온다. 이때 D'이 아니라 옛날 값인 D를 가져온다. 이런!

이 문제를 **캐시 일관성 문제**(**cache coherence**)라고 부른다. 이 문제의 해결에 관련된 방대한 내용을 잘 정리해 놓은 문헌이 있다 [SHW11]. 여기서는 모든 차이점을 생략하고 주요 쟁점에 대해서만 논의할 것이다. 더 많은 것을 알고 싶다면 컴퓨터 구조 수업을 수강하기 바란다.

기본적인 해결책은 하드웨어에 의해 제공된다. 하드웨어는 메모리 주소를 계속 감시하고, 항상 "제대로" 된 상황만 발생토록 시스템을 관리한다. 특히, 여러 개의 프로세스들이 하나의 메모리에 갱신할 때에는 항상 공유되도록 한다. 버스 기반 시스템에서는 **버스 스누핑**(**bus snooping**)이라는 오래된 기법을 사용한다 [Goo83]. 캐시는 자신과 메모리를 연결하는 버스의 통신 상황을 계속 모니터링한다. 캐시 데이터에 대한 변경이 발생하면, 자신의 복사본을 **무효화**(**invalidate**) 시키거나(즉, 자신의 캐시에서 삭제) **갱신**(새로운 값을 캐시에 기록) 한다. 나중 쓰기(write-back) 캐시는 메인 메모리에 쓰기

연산이 지연되기 때문에 캐시 일관성 유지 문제를 훨씬 복잡하게 만든다. 그러나 기본 기법이 어떻게 동작하는지는 대략 상상할 수 있을 것이다.

13.2 동기화를 잊지 마시오

일관성 유지에 대한 모든 일을 캐시가 담당한다면, 프로그램 또는 운영체제 자신은 공유 데이터를 접근할 때 걱정할 필요가 있을까? 불행하게도 대답은 예이고 이 책의 "병행성"에 관한 부분에서 자세하게 설명된다. 여기서 깊게 들어가지는 않지만 몇몇 기본 아이디어에 대해서는 간단하게 설명하겠다. 병행성에 관해 잘 알고 있다고 가정하고 진행할 것이다.

CPU들이 동일한 데이터 또는 구조체에 접근할 때(특히, 갱신), 올바른 연산 결과를 보장하기 위해 락과 같은 상호 배제를 보장하는 동기화 기법이 많이 사용된다. **락-프리(lock-free)** 데이터 구조 등의 다른 방식은 복잡할 뿐 아니라 특별한 경우에만 사용된다. 자세한 사항은 병행성에 관한 부분 중 교착상태 장을 참조하기 바란다. 예를 들어, 여러 CPU가 동시에 사용하는 공유 큐가 있다고 가정하자. 캐시의 일관성을 보장하는 프로토콜이 존재한다 하더라도 락이 없이는 항목의 추가나 삭제가 제대로 동작하지 않을 것이다. 구조체를 원자적으로 갱신하기 위해서는 락이 필요하다.

```
1  typedef struct __Node_t {
2      int value;
3      struct __Node_t *next;
4  } Node_t;
5
6  int List_Pop() {
7      Node_t *tmp = head;       // 이전 head를 기억...
8      int value = head->value; // ... 값도 기억
9      head = head->next;        // head 를 다음 포인터로 이동
10     free(tmp);                // 이전 head 해제
11     return value;             // head의 값을 반환
12 }
```

〈그림 13.3〉 **간단한 리스트의 삭제 코드**

연결 리스트에서 원소 하나를 삭제하는 코드를 생각해 보자. 코드가 그림 13.3에 나와 있다. 두 CPU의 쓰레드가 동시에 이 루틴으로 진입한다고 가정하자. 쓰레드 1이 첫 번째 행을 실행하면 `head`의 현재 값을 `tmp`에 저장한다. 그런 후 쓰레드 2가 첫 번째 행을 실행하면 역시 `head`의 같은 값을 `tmp`에 저장한다. `tmp`는 스택에 할당된다. 각 쓰레드는 자신만의 스택을 갖고 있다. 각 쓰레드는 동일한 헤드 원소를 제거하려고 한다. 제대로 된 상황에서는 각 쓰레드는 리스트의 첫 번째 원소를 한 번씩 제거해야 한다. 이러한 상황이 모든 문제를 (4행의 헤드 원소를 두 번 삭제하고 같은 데이터 값을 두 번 반환하는 등의) 야기한다.

물론 해결책은 **락(lock)**을 사용하여 올바르게 동작하도록 만드는 것이다. 이 경우 간단한 mutex를 할당하고 (예, `pthread_mutex_t m;`) 루틴의 시작에 `lock(&m)`, 마지막에 `unlock(&m)`을 추가하면 문제를 해결할 수 있다. 곧 보겠지만, 이러한 접근

방식이 문제가 없는 것은 아니다. 특히, 성능 측면에서 문제가 있다. CPU의 개수가 증가할수록 동기화된 자료 구조에 접근하는 연산은 매우 느리게 된다.

13.3 마지막 문제점: 캐시 친화성

멀티프로세서 캐시 스케줄러에서의 마지막 문제점은 **캐시 친화성(cache affinity)**이다. 개념은 간단하다. CPU에서 실행될 때 프로세스는 해당 CPU 캐시와 TLB에 상당한 양의 상태 정보를 올려 놓게 된다. 다음 번에 프로세스가 실행될 때 동일한 CPU에서 실행되는 것이 유리하다. 해당 CPU 캐시에 일부 정보가 이미 존재하고 있기 때문에 더 빨리 실행될 것이기 때문이다. 반면 프로세스가 매번 다른 CPU에서 실행되면 실행할 때마다 필요한 정보를 캐시에 다시 탑재해야만 하기 때문에 프로세스의 성능은 더 나빠질 것이다. 하드웨어의 캐시 일관성 프로토콜 덕분에 다른 CPU에서 실행되더라도 프로그램이 제대로 실행될 것이다. 멀티프로세서 스케줄러는 스케줄링 결정을 내릴 때 캐시 친화성을 고려해야 한다. 가능한 한 프로세스를 동일한 CPU에서 실행하려고 노력하는 방향으로 결정해야 한다.

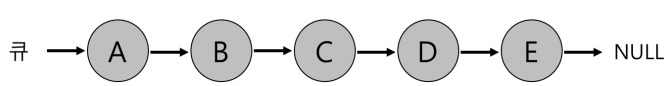
13.4 단일 큐 스케줄링

기본적인 지식을 학습했으므로 이제 멀티프로세서 시스템의 스케줄러 개발 방법에 대해 논의해 보자. 가장 기본적인 방식은 단일 프로세서 스케줄링의 기본 프레임워크를 그대로 사용 하는 것이다. 이러한 방식을 **단일 큐 멀티프로세서 스케줄링(single queue multiprocessor scheduling, SQMS)**이라고 부른다. 이 방식의 장점은 단순함이다. 기존 정책을 다수 CPU에서 동작하도록 하는 데는 많은 변경이 필요치 않다. 예를 들어, CPU가 2개라면 실행할 작업 두 개를 선택한다.

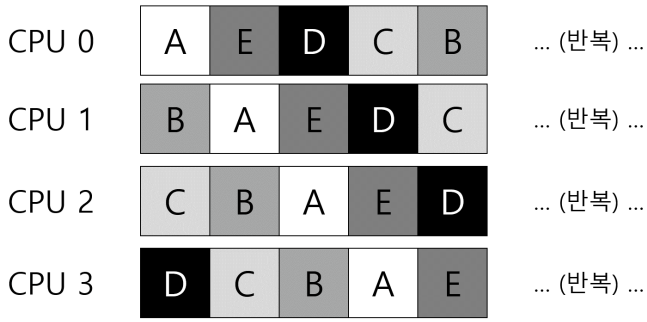
그러나 SQMS는 명백한 단점이 있다. 첫 번째 문제는 **확장성(scalability)** 결여이다. 스케줄러가 다수의 CPU에서 제대로 동작하게 하기 위해 코드에 일정 형태의 **락**을 삽입한다. 락은 SQMS 코드가 단일 큐를 접근할 때 (즉, 실행시킬 다음 작업을 찾을 때) 올바른 결과가 나오도록 한다.

불행히도 락은 성능을 크게 저하시킬 수 있고, 시스템의 CPU 개수가 증가할수록 더욱 그렇다 [And90]. 단일 락에 대한 경쟁이 증가할수록 시스템은 락에 점점 더 많은 시간을 소모하게 되고 실제 필요한 일에 쓰는 시간은 줄어들게 된다.

SQMS가 가지는 두 번째 주요 문제는 캐시 친화성이다. 예를 들어, 실행할 5개의 작업이 있고 (A, B, C, D, E) 4개의 프로세서가 있다고 가정하자. 스케줄링 큐는 다음과 같은 모양일 것이다.

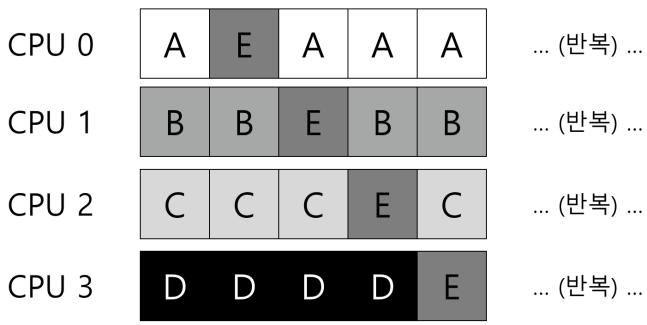


시간이 흐르면서 각 작업은 주어진 타임 슬라이스 동안 실행되고, 다음 작업이 선택되었다고 가정하자. 작업 스케줄은 다음과 같을 것이다.



각 CPU는 공유 큐에서 다음 작업을 선택하기 때문에 각 작업은 CPU를 옮겨 다니게 된다. 캐시 친화성 관점에서 보면 잘못된 선택을 하는 것이다.

이 문제의 해결을 위해 대부분의 SQMS 스케줄러는 가능한 한 프로세스가 동일한 CPU에서 재실행될 수 있도록 시도한다. 구체적으로, 특정 작업들에 대해서 캐시 친화성을 고려하여 스케줄링하고 다른 작업들은 오버헤드를 균등하게 하기 위해 여러 군데로 분산시키는 정책을 사용한다. 예를 들어 다음과 같은 동일한 5개의 작업에 대해 생각해 보자.



이 배열에서 A부터 D까지의 작업은 각각 자신의 프로세서에서 실행된다. 오직 E만이 하나의 프로세서에서 다른 프로세서로 **이동**한다. 그렇게 함으로써 대부분의 작업에게 친화성을 보존하고 있다. 다음에는 다른 작업을 이동시켜서 일종의 친화성에 대한 형평성도 추구할 수 있다. 그러나 이러한 기법은 구현이 복잡해질 수 있다.

SQMS 방식의 장단점을 볼 수 있다. 기존의 단일 CPU 스케줄러가 있다면 하나의 큐밖에 없기 때문에 구현이 간단하다. 그러나 동기화 오버헤드 때문에 이 방식은 확장성이 좋지 않고 캐시 친화성에 문제가 있다.

13.5 멀티 큐 스케줄링

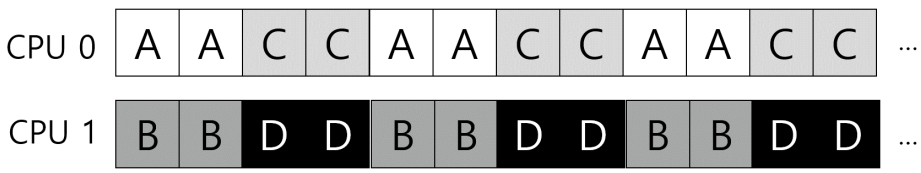
단일 큐 스케줄러로 인한 문제 때문에 일부 시스템은 멀티 큐, 예를 들어, CPU마다 큐를 하나씩 둔다. 이 방식을 **멀티 큐 멀티프로세서 스케줄링**(**multi-queue multiprocessor scheduling**, **MQMS**)이라고 부른다.

MQMS에서 기본적인 스케줄링 프레임워크는 여러 개의 스케줄링 큐로 구성된다. 각 큐는 아마도 라운드 로빈 같은 특정 스케줄링 규칙을 따를 것이고 물론 어떤 스케줄링 기법도 사용 가능하다. 작업이 시스템에 들어가면 하나의 스케줄링 큐에 배치된다. 배치될 큐의 결정은 적당한 방법을 따른다 (예를 들어, 무작위로 할 수도 있고 또는 다른 큐보다 작은 수의 작업이 있는 큐로 배치할 수도 있다). 그 후에는 각각이 독립적으로 스케줄 되기 때문에 단일 큐 방식에서 보았던 정보의 공유 및 동기화 문제를 피할 수 있다.

예를 들어, 2개의 CPU(CPU 0과 CPU 1)와 A, B, C, D 네 개의 작업이 시스템에 존재한다고 가정하자. CPU는 각자 하나씩의 스케줄링 큐를 가지고 있으므로 운영체제는 각 작업을 배치할 큐를 결정해야 한다. 다음과 같은 결정을 내렸다고 하자.



큐 스케줄링 정책에 따라 각 CPU는 현재 2개씩의 작업을 가지고 있다. 예를 들어, **라운드 로빈**의 경우 다음과 같은 스케줄이 생성될 수 있다.

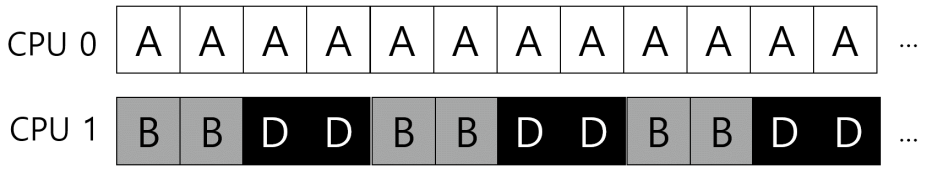


MQMS가 SQMS에 비해 가지는 명확한 이점은 확장성이 좋다는 것이다. CPU 개수가 증가할수록, 큐의 개수도 증가하므로 락과 캐시 경합(cache contention)은 더 이상 문제가 되지 않는다. 또한, MQMS는 본질적으로 캐시 친화적이다. 작업이 같은 CPU에서 계속 실행되기 때문에 캐시에 저장된 내용을 재사용하는 이점을 얻게 된다.

지금까지 주의를 집중해 왔다면 새로운 문제가 있다는 것을 알았을 것이다. 멀티 큐 기반 방식의 근본적인 문제는 **워크로드의 불균형**(**load imbalance**)이다. 앞의 예와 동일한 설정을 가정하자(4개의 작업, 2개의 CPU). 그러나 그 중 하나(예, C)가 종료되었다고 하자. 이제 다음과 같은 스케줄링 큐를 가진다.



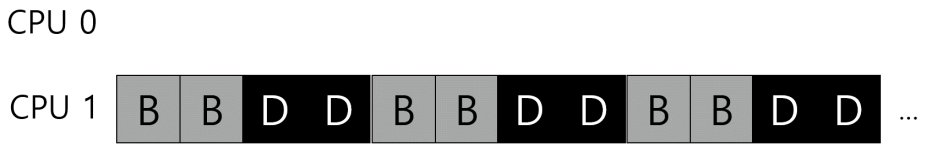
우리는 각 큐에 라운드 로빈 정책을 실행한 경우, 다음과 같은 스케줄을 얻을 수 있다.



이 그림에서 알 수 있듯이, A가 B와 D보다 2배의 CPU를 차지하고 이는 우리가 원하는 결과가 아니다. 설상가상으로 A와 C가 모두 종료하여 B와 D만 남게 되었다고 가정하자. 스케줄링 큐는 다음과 같을 것이다.



결과 CPU 0은 유휴 상태가 된다! *(이 시점에 극적이고 불길한 음악 삽입)* 그래서 CPU 사용 타임라인은 슬프게도 다음 모양이 된다.



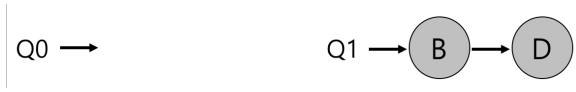
그렇다면 불쌍한 멀티 큐 멀티프로세서 스케줄러는 무엇을 해야 할까? 어떻게 하면 이 기분 나쁜 오버헤드 불균형 문제를 극복할 수 있고 나아가 Decepticon의 악의 세력을 물리칠 수 있을까[1]? 훌륭한 이 책과 관련 없는 질문을 중지하려면 어떻게 해야 하나?

핵심 질문: 워크로드 불균형에 대처하는 방법
원하는 스케줄링 목적을 더 잘 달성할 수 있기 위하여, 멀티 큐 멀티프로세서 스케줄러는 워크로드 불균형 문제를 어떻게 해결할 수 있을까?

이 질문에 대한 당연한 답은 작업을 이리저리로 이동시키는 것이다. 우리는 이 기술을 **이주**(**migration**)라고 부른다. 작업을 한 CPU에서 다른 CPU로 이주시킴으로써 워크로드 균형을 달성한다.

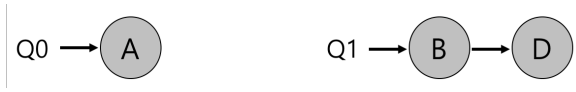
좀 더 명확하게 하기 위하여 몇 가지 예를 보도록 하자. 다시 한 번, 하나의 CPU는 유휴 상태이고 다른 CPU는 몇몇 작업을 가지고 있다고 하자.

1) 잘 알려지지 않은 사실은 사이버트론의 고향 행성이 나쁜 CPU 스케줄링 결정에 의해 파괴되었다는 것이다. 그리고 이제 이 얘기가 트랜스포머에 대한 처음이자 마지막이다. 물론, 이 점에 대해서는 진심으로 미안.

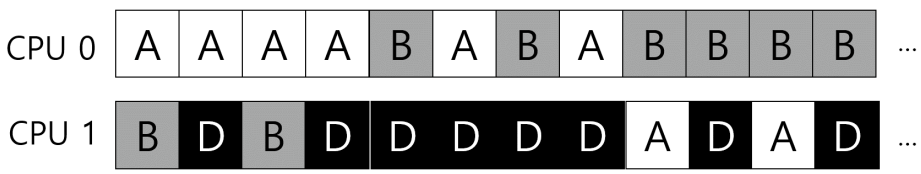


이 경우 어떻게 이주시켜야 하는지 이해하기 쉽다. 운영체제는 B 또는 D 중 하나를 CPU 0로 이동시킨다. 이 한 번의 작업 이주로 워크로드가 균형을 이루게 되고 모두가 행복하게 된다.

좀 더 까다로운 경우는 그 이전 예에서 발생한다. A는 혼자 CPU 0에 있었고 B와 D는 CPU 1에서 번갈아 실행되고 있었다.



이 경우 한 번의 이주만으로는 문제가 해결되지 않는다. 무엇을 해야 할까? 대답은 작업들을 지속적으로 이주시키는 것이다. 한 가지 가능한 해결책은 다음 타임라인에서 볼 수 있는 것처럼 계속해서 작업을 이주하는 것이다. 그림에서 처음에 A는 CPU 0에 혼자 있고 B와 D는 CPU 1에서 번갈아 실행되고 있다. 몇 번의 타임 슬라이스 후 B는 CPU 0로 이주하여 A와 경쟁한다. 그동안 D는 CPU 1에서 몇 개의 타임 슬라이스 동안 혼자 실행된다. 따라서 워크로드의 균형이 맞게 된다.



물론 가능한 많은 다른 이주 패턴이 존재한다. 그러나 지금의 더 까다로운 부분: 이주의 필요 여부를 어떻게 결정할까? 한 가지 기본적인 접근 방식은 **작업 훔치기(work stealing** [FLR88]) 라는 기술이다. 작업 훔치기에서는 작업의 개수가 낮은 (소스) 큐가 가끔 다른 (대상) 큐에 훨씬 많은 수의 작업이 있는지를 검사한다. 대상 큐가 소스 큐보다 더 가득 차 있다면 워크로드 균형을 맞추기 위해 소스는 대상에서 하나 이상의 작업을 가져온다.

물론 이러한 방식에는 자연스러운 문제가 있다. 큐를 너무 자주 검사하게 되면 높은 오버헤드로 확장성에 문제가 생기게 된다. 확장성은 멀티 큐 스케줄링의 가장 중요한 목적이다. 반면 다른 큐를 자주 검사하지 않으면 심각한 워크로드 불균형을 초래할 가능성이 있다. 시스템 정책 설계에 흔히 있는 일로서 적절한 값을 찾아내는 것은 마법의 영역이다.

13.6 Linux 멀티프로세서 스케줄러

Linux 커뮤니티에서는 멀티프로세서 스케줄러를 위한 단일화된 방식이 존재하지 않았다. 세 가지 스케줄러가 등장했다. O(1) 스케줄러, Completely Fair Scheduler(CFS) 및 BF 스케줄러 (BFS)가[2] 바로 그것이다. 언급한 스케줄러의 장점과 단점에 관한 훌륭한 개요를 보려면 Meehean의 학위 논문을 [Mee11] 참조하시오. 기본 아이디어의 일부만 요약한다.

O(1)과 CFS는 멀티 큐를, BFS는 단일 큐를 사용하기 때문에 두 방식 모두 실제 시스템에서 성공적으로 사용할 수 있다는 것을 보이고 있다. 물론, 이 스케줄러를 구분하는 다른 많은 분류 기준들이 존재한다. 예를 들어, O(1) 스케줄러는 우선순위 기반 스케줄러로서(전에 논의했던 MLFQ와 유사) 프로세스의 우선순위를 시간에 따라 변경하여 우선순위가 가장 높은 작업을 선택하여 다양한 목적을 만족시킨다. 특히, 상호작용을 가장 우선시 한다. 반면에 CFS는 결정론적(deteministic) 비례배분(proportional share) 방식이다(전에 논의했던 보폭 스케줄링에 가까움). BFS는 셋 중에서 유일한 단일 큐 방식이며 또한 비례배분 방식이다. 그러나 Earliest Eligible Virtual Deadline First(EEVDF) [SA96]라고 알려진 더 복잡한 방식에 기반을 둔다. 이 알고리즘들에 대해서는 스스로 학습하기 바란다. 이제는 스케줄링 알고리즘이 어떻게 동작하는지 이해할 수 있을 것이다.

13.7 요약

다양한 방식의 멀티프로세서 스케줄링을 살펴보았다. 단일 큐 방식(SQMS)은 구현이 용이하고 워크로드의 균형을 맞추기 용이하지만 많은 개수의 프로세서에 대한 확장성과 캐시 친화성이 좋지 못하다. 멀티 큐 방식(MQMS)은 확장성이 좋고 캐시 친화성을 잘 처리하지만 워크로드 불균형에 문제가 있고 구현이 복잡하다. 어떤 방식을 택하든지 쉬운 답은 없다. CPU 스케줄러는 사소한 코드 수정으로 시스템의 동작이 엄청나게 바뀌기 때문에 모든 경우에 다 잘 동작하는 범용 스케줄러를 구현하는 것은 매우 어렵다. 본인이 하는 작업이 가져올 결과를 정확히 이해하고 있거나, 괜찮은 보수를 받는다면 한 번 시도해 볼 만하다.

2) BF가 무엇을 나타내는지는 혼자 힘으로 찾아 보라. 미리 경고, 용기 없는 사람들을 위한 건 아니다.

참고 문헌

[And90] **"The Performance of Spin Lock Alternatives for Shared-Memory Multiprocessors"**
Thomas E. Anderson
IEEE TPDS Volume 1:1, January 1990
여러 기법들이 얼마나 확장성을 갖는지에 관한 고전적인 논문. 시스템과 네트워크 분야에서 매우 저명한 연구자인 Tom Anderson의 논문. 매우 훌륭한 운영체제 교재의 저자라는 것도 얘기해야겠다.

[Boy+10] **"An Analysis of Linux Scalability to Many Cores"**
Silas Boyd-Wickizer, Austin T. Clements, Yandong Mao, Aleksey Pesterev, Robert Morris M. Frans Kaashoek, and Nickolai Zeldovich
OSDI '10, Vancouver, Canada, October 2010
멀티코어로 Linux를 확장시키는 어려움에 대한 훌륭한 최근의 논문.

[CSG99] **"Parallel Computer Architecture: A Hardware/Software Approach"**
David E. Culler, Jaswinder Pal Singh, and Anoop Gupta
Morgan Kaufmann, 1999
병렬 시스템과 알고리즘의 상세 내용이 담겨져 있는 보물같은 책. Mark Hill이 표지에 익살스럽게 밝힌 바와 같이 대부분의 연구 논문보다 많은 정보를 담고 있다.

[FLR88] **"The Implementation of the Cilk-5 Multithreaded Language"**
Matteo Frigo, Charles E. Leiserson, and Keith Randall
PLDI '98, Montreal, Canada, June 1998
Cilk는 경량 언어이자 병렬 프로그램을 작성하기 위한 런타임이고, 작업 훔치기 패러다임의 좋은 사례이다.

[Goo83] **"Using Cache Memory To Reduce Processor-Memory Traffic"**
James R. Goodman
ISCA '83, Stockholm, Sweden, June 1983
버스 스누핑 사용에 관한, 즉 버스에 나타나는 요청에 주의를 기울여서 캐시 일관성 프로토콜을 구현하는 방법에 관한 선구적인 논문. Goodman이 Wisconsin에서 행한 오랫동안의 연구는 영리함으로 가득 차 있으며 이 논문도 사례 중의 하나.

[Mee11] **"Towards Transparent CPU Scheduling"**
Joseph T. Meehean
Doctoral Dissertation at University of Wisconsin-Madison, 2011
최근 Linux의 멀티프로세서 스케줄링이 어떻게 동작하는지를 자세하게 다루는 학위 논문. 매우 대단함! 우리가 Joe의 공동 지도교수이기 때문에 약간 편향되었을 수 있다.

[SHW11] **"A Primer on Memory Consistency and Cache Coherence"**
Daniel J. Sorin, Mark D. Hill, and David A. Wood
Synthesis Lectures in Computer Architecture, Morgan and Claypool Publishers, May 2011
메모리 일관성과 멀티프로세서 캐싱에 관한 최고의 개요. 언급된 주제에 관해 더 많이 알고 싶어하는 사람들을 위한 필독 논문.

[SA96] **"Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation"**
Ion Stoica and Hussein Abdel-Wahab
Technical Report TR-95-22, Old Dominion University, 1996

이 멋진 스케줄링 아이디어에 관한 Ion Stoica의 기술 보고서. 그는 현재 U.C. Berkeley의 교수로 재직 중이며 네트워킹, 분산 시스템 및 다른 분야의 세계적인 권위자이다.

멀티프로세서 스케줄링(고급)

영어판에만 있는 추가 내용 번역

대조 기준: 한국어판 전체와 Version 1.10 영어 원문을 문단 단위로 비교했습니다. 이미 한국어판에 있는 내용은 반복하지 않고, 영어판에만 있는 내용만 번역했습니다. 절 번호는 영어 원문의 번호를 유지했습니다.

영어판에만 있는 참고문헌

[TTG95] Josep Torrellas, Andrew Tucker, Anoop Gupta, 'Evaluating the Performance of Cache-Affinity Scheduling in Shared-Memory Multiprocessors', Journal of Parallel and Distributed Computing, Vol. 24, No. 2, 1995. 캐시 친화성 스케줄링을 처음 다룬 논문은 아니지만, 더 이른 연구에 대한 인용을 포함하며 앞선 큐잉 기반 분석 논문보다 읽기 쉽고 실용적이다.

숙제: 시뮬레이션

이 실습에서는 `multi.py`를 사용하여 멀티프로세서 CPU 스케줄러를 시뮬레이션하고 세부 동작을 학습한다. 시뮬레이터와 선택 사항에 관한 자세한 내용은 관련 README를 참고하라.

1. 먼저 시뮬레이터로 효과적인 멀티프로세서 스케줄러를 구성하는 방법을 알아보자. 첫 시뮬레이션에서는 실행 시간이 30이고 작업 집합 크기가 200인 작업 하나만 실행한다. 다음과 같이 하나의 가상 CPU에서 작업 a를 실행하라: `./multi.py -n 1 -L a:30:200`. 완료하는 데 얼마나 걸리는가? `-c` 플래그로 최종 답을 확인하고, `-t` 플래그로 매 틱마다 작업이 어떻게 스케줄되는지 추적하라.

2. 이제 기본 크기가 100인 캐시를 키워서 크기 200인 작업 집합이 캐시에 들어가게 하라. 예: `./multi.py -n 1 -L a:30:200 -M 300`. 작업이 캐시에 들어간 뒤 얼마나 빠르게 실행될지 예측할 수 있는가? 힌트: `-r` 플래그로 정하는 warm rate 매개변수를 기억하라. `-c` 플래그를 켜서 답을 확인하라.

3. `multi.py`의 장점 중 하나는 여러 추적 플래그로 내부 동작을 더 자세히 볼 수 있다는 점이다. 위와 같은 시뮬레이션을 실행하되 이번에는 남은 실행 시간 추적 옵션 `-T`를 켜라. 이 플래그는 각 시간 단계에서 CPU에 스케줄된 작업과, 한 틱 실행 후 그 작업에 남은 실행 시간을 함께 보여 준다. 두 번째 열이 감소하는 방식에서 무엇을 발견할 수 있는가?

4. 각 작업에 대한 CPU 캐시의 상태를 표시하도록 `-C` 플래그도 추가하라. 각 캐시는 해당 작업에 대해 차가운 상태면 빈칸, 따뜻한 상태면 w를 표시한다. 이 간단한 예에서 작업 a의 캐시는 언제 따뜻해지는가? 준비 시간 매개변수 `-w`를 기본값보다 작거나 크게 바꾸면 어떻게 되는가?

5 이제 단일 CPU에서 단일 작업을 실행하는 시뮬레이터의 동작을 이해했을 것이다. 멀티프로세서 CPU 스케줄링을 살펴보자. 두 CPU 시스템에서 다음 세 작업을 실행하라: `./multi.py -n 2 -L a:100:100,b:100:50,c:100:50`. 중앙 집중식 라운드 로빈 스케줄러라면 완료하는 데 얼마나 걸릴지 예측하라. `-c`로 답을 확인하고, `-t`로 단계별 동작을 살펴본 뒤 `-C`로 각 작업의 캐시가 효율적으로 따뜻해졌는지 확인하라. 무엇을 발견했는가?

6 이제 이 장에서 설명한 캐시 친화성을 연구하기 위해 명시적인 제어를 적용한다. `-A` 플래그는 스케줄러가 특정 작업을 배치할 수 있는 CPU를 제한한다. 작업 b와 c는 CPU 1에, a는 CPU 0에만 배치해 보자: `./multi.py -n 2 -L a:100:100,b:100:50,c:100:50 -A a:0,b:1,c:1`. 여러 추적 옵션도 함께 사용하라. 이 버전은 얼마나 빨리 실행될지 예측할 수 있는가? 왜 더 나은 성능을 내는가? a, b, c를 두 프로세서에 다르게 배치하면 더 빨라지거나 느려지는가?

7 캐시를 가진 멀티프로세서에서는 여러 CPU와 캐시를 사용할 때 작업이 예상보다 더 크게 가속될 수 있다. N개의 CPU에서 실행했을 때 속도가 N배를 넘는 경우를 초선형 가속(super-linear speedup)이라고 한다. 세 작업 `-L a:100:100,b:100:100,c:100:100`을 작은 캐시 `-M 50`에서 생성하고, CPU가 1개, 2개, 3개인 시스템(`-n 1`, `-n 2`, `-n 3`)에서 실행하라. 이후 CPU별 캐시 크기를 100으로 키워 같은 실험을 반복하라. CPU 수가 늘어날 때 성능에서 무엇을 발견할 수 있는가? `-c`로 예측을 확인하고 다른 추적 플래그로 더 자세히 분석하라.

8 시뮬레이터에서 살펴볼 또 다른 기능은 CPU별 스케줄링 옵션인 `-p` 플래그다. 다시 두 CPU와 세 작업 구성 `-L a:100:100,b:100:50,c:100:50`을 사용하라. 이 방식은 앞에서 직접 설정한 친화성 제한과 비교해 어떻게 동작하는가? 엿보기 간격(peek interval)인 `-P` 값을 더 작게 또는 크게 바꾸면 성능이 어떻게 달라지는가? CPU 수가 늘어날 때 CPU별 스케줄링 방식은 어떻게 동작하는가?

9 마지막으로 무작위 워크로드를 생성하고, 서로 다른 프로세서 수, 캐시 크기, 스케줄링 옵션에서 성능을 예측해 보라. 이 과정을 반복하면 멀티프로세서 스케줄링을 충분히 익힐 수 있을 것이다.

16

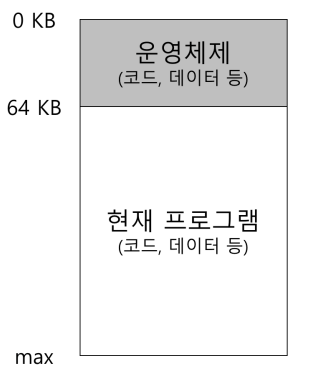
주소 공간의 개념

초기에는 컴퓨터 시스템을 구현하는 것이 쉬웠다. 왜냐고? 사용자가 많은 것을 기대하지 않았기 때문이다. 이 모든 두통거리를 만든 사람들은 바로 "사용의 편이", "고성능", "신뢰성" 등을 기대한 훌륭한 사용자이다. 다음에 그런 컴퓨터 사용자를 만나게 되면 그들이 제시한 모든 문제에 대해 감사하기 바란다.

16.1 초기 시스템

메모리 관점에서 초기 컴퓨터는 많은 개념을 사용자에게 제공하지 않았다. 컴퓨터의 물리 메모리는 그림 16.1에서 보는 것과 같이 생겼다.

운영체제는 메모리에(이 예에서는 물리 주소 0부터) 상주하는 루틴(실제로는 라이브러리)의 집합이었다. 물리 메모리에 하나의 실행 중인 프로그램(프로세스)이 존재하였고(이 예에서 물리 주소 64KB부터 시작하여) 나머지 메모리를 사용하였다. 특별한 가상화는 거의 존재하지 않았고 사용자는 운영체제로부터 그리 많은 것을 기대하지 않았다. 그 당시 운영체제 개발자의 삶은 쉬운 게 확실했다. 그렇지 않나?



〈그림 16.1〉 **운영체제 : 초기**

16.2 멀티프로그래밍과 시분할

시간이 흐른 후, 컴퓨터는 고가 장비였기 때문에, 사람들이 더 효과적으로 컴퓨터를 공유하기 시작했다. **멀티프로그래밍**(**multi-programming**) 시대가 도래하였다 [DH66]. 여러 프로세스가 실행 준비 상태에 있고 운영체제는 그들을 전환하면서 실행하였다. 예를 들어 한 프로세스가 입출력을 실행하면, CPU는 다른 프로세스로 전환하였다. 이런 전환은 CPU의 **이용률**을 증가시켰다. 당시에는 특히 이러한 **효율성**의 개선이 중요하였다. 시스템의 가격이 수십만 또는 수백만 달러나 하였다(그리고 당신은 Mac이 비싸다고 생각하고 있다!).

곧 사람들이 컴퓨터를 더 많이 사용하길 원하기 시작했으며 **시분할**(**time-sharing**) 시대가 시작되었다 [Str59; Lic60; McC62; McC83]. 많은 사람들이 일괄처리방식(batch computing) 컴퓨팅의 한계를 인식하였으며, 특히 오랜 시간이 걸리는 (따라서 비효과적인) 프로그램-디버그 사이클에 지친 프로그래머 자신들 [CV65]이 일괄처리방식 컴퓨팅의 한계를 느꼈다. 많은 사용자가 동시에 컴퓨터를 사용하고, 현재 실행 중인 작업으로부터 즉시 응답을 원하기 때문에 **대화식 이용**(**interactivity**)의 개념이 중요하게 되었다.

시분할을 구현하는 한 가지 방법은 하나의 프로세스를 짧은 시간 동안 실행시키는 것이다. 해당 기간 동안 프로세스에게 모든 메모리를 접근할 권한이 주어진다(그림 16.1). 그런 후에, 이 프로세스를 중단하고, 중단 시점의 모든 상태를 디스크 종류의 장치(모든 물리 메모리를 포함하여)에 저장하고 다른 프로세스의 상태를 탑재하여 또 짧은 시간 동안 실행시킨다. 시분할 시스템을 이런 식으로 엉성하게 구현하였다 [McC+63].

이 방법에는 커다란 문제가 있다. 너무 느리게 동작한다는 것이고, 특히 메모리가 커질수록 느리게 된다. 레지스터 상태를 저장하고 복원하는 것은 빠르지만 메모리의 내용 전체를 디스크에 저장하는 것은 엄청나게 느리다. 우리가 할 일은 프로세스 전환 시 프로세스를 메모리에 그대로 유지하면서, 운영체제가 시분할 시스템을 효율적으로 구현할 수 있게 하는 것이다(그림 16.2)



〈그림 16.2〉 **세 개의 프로세스 : 공유 메모리**

그림에 세 개의 프로세스(A, B, C)가 있고 각 프로세스는 512 KB 물리 메모리에서 각기 작은 부분을 할당받았다. 하나의 CPU를 가정할 때, 운영체제는 실행할 한 개의 프로세스(A라 하자)를 선택하고, 다른 프로세스들(B와 C)은 준비 큐에서 실행을 기다린다.

시분할 시스템이 대중화되면서 운영체제에게 새로운 요구 사항이 부과되었다. 여러 프로그램이 메모리에 동시에 존재하려면 **보호(protection)**가 중요한 문제가 된다. 우리는 한 프로세스가 다른 프로세스의 메모리를 읽거나 혹은 더 안 좋게는 쓸 수 있는 상황을 원하지 않는다.

16.3 주소 공간

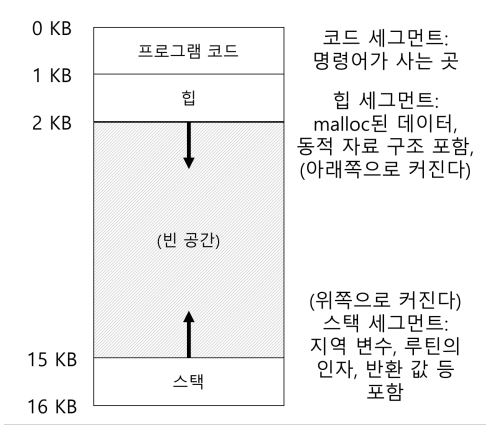
우리는 그런 위험한 행위를 하는 사용자를 염두에 두어야 한다. 그런 위험에 대비하여 운영체제는 **사용하기 쉬운(easy to use)** 메모리 개념을 만들어야 한다. 이 개념이 **주소 공간(address space)**이다. 실행 중인 프로그램이 가정하는 메모리의 모습이다. 운영체제의 메모리 개념을 이해하는 것이 메모리를 어떻게 가상화할지를 이해하는 핵심이다.

주소 공간은 실행 프로그램의 모든 메모리 상태를 갖고 있다. 예를 들어, 프로그램의 **코드(code)**, 명령어)는 반드시 메모리에 존재해야 하고 따라서 주소 공간에 존재한다. **스택**은 함수 호출 체인 상의 현재 위치, 지역 변수, 함수 인자와 반환 값 등을 저장하는 데 사용된다. 마지막으로 **힙(heap)**은 동적으로 할당되는 메모리를 위해 사용된다. C 언어의 `malloc()`, C++나 Java 같은 객체 지향 언어의 `new`를 통해 메모리를 동적으로 할당받는다. 주소 공간 구성 요소에는 정적으로 초기화된 변수 등의 다른 것들도 있지만, 현재로선 코드, 스택 및 힙의 세 가지만 있다고 가정하자.

그림 16.3은 아주 작은 주소 공간을 보이고 있다(겨우 16 KB)[1]. 프로그램 코드는 주소 공간의 위쪽에 위치한다. 이 예제에서는 주소 0부터 시작하고, 주소 공간의 첫 1 KB를 차지한다. 코드는 정적이기 때문에 메모리에 저장하기 쉽다. 따라서 주소 공간의 상단에 배치하고, 프로그램이 실행되면서 추가 메모리를 필요로 하지 않는다.

다음으로 프로그램 실행과 더불어 확장되거나 축소될 수 있는 두 종류의 주소 공간이 존재한다. 주소 공간의 상단에 존재하는 힙과 하단에 존재하는 스택이다. 두 메모리 영역은 확장할 수 있어야 하기 때문에 이런 방식으로 배치하고, 주소 공간의 양 끝단에 배치해야 두 영역 모두 확장하는 것이 가능하다. 두 영역은 확장 방향이 반대 방향일 수밖에 없다. 힙은 코드 바로 뒤 1 KB부터 시작하고 아래 방향으로 확장한다(예를 들면, 사용자가 `malloc()` 함수를 통해 더 많은 메모리를 요청할 때). 스택은 16 KB에서 시작하고 위쪽 방향으로 확장한다(예를 들어, 사용자가 프로시저를 호출할 때). 그러나 스택과 힙의 이러한 배치는 관례일 뿐이다. 원한다면 주소 공간을 다른 방식으로 배치할 수 있다. 나중에 보겠지만 주소 공간에 여러 **쓰레드**가 공존할 때는 이런 식으로 주소 공간을 나누게 되면 동작하지 않는다.

1) 우리는 종종 이런 작은 크기의 예제를 사용할 것이다. 왜냐하면 (a) 32비트 주소 공간을 표현하는 것은 고통이고 (b) 관련 수학은 더 어렵기 때문이다. 우리는 단순한 수학을 좋아한다.



〈그림 16.3〉 **주소 공간의 예**

주소 공간을 설명할 때, 운영체제가 실행 중인 프로그램에게 제공하는 **개념**(**abstraction**)을 설명한다. 실제로 프로그램이 물리 주소 0에서 16 KB 사이에 존재하는 것은 아니다. 실제로는 임의의 물리 주소에 탑재된다. 그림 16.2의 프로세스 A, B, C를 보자. 각 프로세스가 어떤 식으로 다른 주소에 탑재되었는지 알 수 있다. 우리의 문제는 다음과 같다.

핵심 질문: 메모리를 어떻게 가상화하는가
운영체제는 물리 메모리를 공유하는 다수의 프로세스에게 어떻게 프로세스 전용의 커다란 주소 공간이라는 개념을 제공할 수 있는가?

운영체제가 이 일을 할 때, 우리는 운영체제가 **메모리를 가상화**(**virtualizing memory**)한다고 말한다. 왜냐하면 실행 중인 프로그램은 자신이 특정 주소의 메모리에(예를 들어 0) 탑재되고 매우 큰 주소 공간을(예를 들어, 32비트 또는 64비트) 가지고 있다고 생각하기 때문이다. 현실은 다르다.

예를 들어, 그림 16.2의 프로세스 A가 주소 0으로부터(우리는 이를 **가상 주소**(**virtual address**)라고 부를 것이다) `load` 연산을 수행할 때, 운영체제는 하드웨어의 지원을 통해 물리 주소 0이 아니라 물리 주소 320 KB(A가 탑재된 메모리)를 읽도록 보장해야 한다. 이것이 메모리 가상화의 열쇠이고, 현대 모든 컴퓨터 시스템의 기저를 이룬다.

16.4 목표

이제 메모리 가상화라는 운영체제 기능을 논의할 시점이 되었다. 운영체제는 메모리를 가상화할 뿐 아니라 그 가상화를 멋진 방식으로 한다. 운영체제가 가상화를 멋지게 하기 위해서는, 몇 가지 목표가 필요하다. 이전에 목표들을 보았지만, 다시 보도록 하자. 충분히 반복할 가치가 있다.

팁: 고립의 원칙

고립은 신뢰할 수 있는 시스템을 구축하는 데 중요한 원칙이다. 두 개체가 서로 적절하게 고립된 경우, 한 개체가 실패하더라도 상대 개체에 아무 영향을 주지 않는다는 것을 암시한다. 운영체제는 프로세스를 서로 고립시키기 위해 노력하고 이런 방식으로 한 프로세스가 다른 프로세스에게 피해를 주는 것을 방지한다. 더 나아가 메모리 고립을 사용하여 운영체제는 프로그램이 운영체제 동작에 영향을 줄 수 없다는 것을 보장한다. 일부 현대 운영체제는 고립을 더 확장하여 운영체제의 구성 요소를 서로 고립시킨다. 이러한 **마이크로커널** [Han70; Ras+89; SBL03]은 전통적인 모놀리식 커널보다 더 큰 신뢰성을 제공할 수 있다.

가상 메모리 시스템(VM)의 주요 목표 중 하나는 **투명성(transparency)**이다[2]. 운영체제는 실행 중인 프로그램이 가상 메모리의 존재를 인지하지 못하도록 가상 메모리 시스템을 구현해야 한다. 프로그램은 메모리가 가상화되었다는 사실을 인지해서는 안 된다. 오히려 프로그램은 자신이 전용 물리 메모리를 소유한 것처럼 행동해야 한다. 많은 작업들이 메모리를 공유할 수 있도록, 무대 뒤에서 운영체제와 하드웨어가 모든 작업을 수행한다.

VM의 또 다른 목표는 **효율성(efficiency)**이다. 운영체제는 가상화가 시간과 공간 측면에서 효율적이도록 해야 한다. 시간적으로는 프로그램이 너무 느리게 실행되서는 안 되고 공간적으로는 가상화를 지원하기 위한 구조를 위해 너무 많은 메모리를 사용해서는 안 된다. 시간-효율적인 가상화를 구현할 때, 운영체제는 TLB 등의 하드웨어 기능을 포함하여 하드웨어의 지원을 받아야 한다. TLB에 대해서는 나중에 적절한 때에 배우게 될 것이다.

마지막으로 VM의 세 번째 목표는 **보호(protection)**이다. 운영체제는 프로세스를 다른 프로세스로부터 보호해야 하고 운영체제 자신도 프로세스로부터 보호해야 한다. 프로세스가 탑재, 저장, 혹은 명령어 반입 등을 실행할 때 어떤 방법으로든 다른 프로세스나 운영체제의 메모리 내용에 접근하거나 영향을 줄 수 있어서는 안 된다. 즉, 자신의 주소 공간 밖의 어느 것도 접근할 수 있어서는 안 된다. 보호 성질을 이용하여 우리는 프로세스들을 서로 **고립(isolate)**시킬 수 있다. 각 프로세스는 잘못된 혹은 악성 프로세스로부터 안전한 자신만의 보호막 안에서 실행되어야 한다.

다음 장에서 운영체제와 하드웨어 지원을 포함하여 메모리를 가상화하기 위해 필요한 기본적인 기법을 집중적으로 탐구한다. 또한, 빈 공간을 관리하는 방법과 공간이 부족할 때 어떤 페이지를 내보낼 것인가 등의 운영체제 정책들에 대해서도 학습한다. 그런 탐구와 조사를 수행하면서 현대 가상 메모리 시스템의 실제 동작에 대해 이해하게 될

2) 투명성의 이런 사용은 때때로 혼란스럽다. 일부 학생들은 "투명하다"가 모든 것을 숨김 없이 밝혀야 한다는 의미라고, 즉 정부가 그래야 한다는 것처럼 생각한다. 여기서는 그 반대를 의미한다. 운영체제에 의해 제공된 환상은 응용 프로그램에게 보여서는 안 된다. 일반적인 사용에서 투명한 시스템은 인지하기 어려운 시스템이지, Freedom of Information Act에 의해 명기된 것처럼 요청에 반응하는 시스템이 아니다.

여담: 당신이 보는 모든 주소는 가상 주소이다

포인터를 출력하는 C 프로그램을 작성한 적이 있는가? 당신이 보는 값은(좀 큰 변수, 종종 16진수로 출력되는) **가상 주소**(**virtual address**)이다. 프로그램 코드가 탑재된 주소가 어디인지 궁금한 적은 없는가? 그 주소를 출력할 수 있다. 그러나 주소를 출력했다면, 그 주소는 가상 주소이다. 사용자 프로그램(user level program)이 볼 수 있는 주소는 모두 가상 주소이다. 메모리 가상화의 기술 때문에, 명령어와 데이터가 탑재되어 있는 물리 메모리 주소를 알 수 있는 것은 오직 운영체제뿐이다. 결코 잊지 말아야 할 사실: 프로그램에서 주소를 출력하면, 그 주소는 가상 주소, 즉 메모리 배치에 대한 환상이다. 오직 운영체제와 하드웨어만이 진실을 알고 있다.

다음은 `main()` 함수, `malloc()`으로 할당된 메모리, 스택에 존재하는 정수의 위치를 출력하는 프로그램이다.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 int main(int argc, char *argv[]) {
4     printf("location of code : %p\n", (void *) main);
5     printf("location of heap : %p\n", (void *) malloc(1));
6     int x = 3;
7     printf("location of stack : %p\n", (void *) &x);
8     return x;
9 }
```

64비트 Mac OS X 컴퓨터에서 실행하면 우리는 다음과 같은 출력을 얻을 수 있다.

```
location of code : 0x1095afe50
location of heap : 0x1096008c0
location of stack : 0x7fff691aea64
```

이 예에서 주소 공간에서 코드가 제일 먼저 등장하고, 다음에 힙이 배치되고, 마지막으로 대규모 가상 공간의 반대편에 스택이 위치한다는 것을 알 수 있다. 이 모든 주소는 가상 주소이며, 저장된 실제 물리 메모리 위치에서 값을 반입하기 위해 운영체제와 하드웨어에 의해 물리 주소로 변환된다.

것이다[3].

16.5 요약

우리는 가상 메모리라는 운영체제의 구성 요소에 대해 소개를 마쳤다. VM 시스템은 프로세스 전용 공간이라는 환상을 프로그램에게 제공할 책임이 있다. 이 공간에 프로그램 명령어 전부와 데이터 전부가 저장된다. 하드웨어의 도움을 받아 운영체제는 가상 메모리 주소를 받아 물리 주소로 변환한다. 물리 주소는 원하는 정보를 반입하기 위하여 물리 메모리에게 전달된다. 운영체제는 많은 프로세스를 대상으로 이러한 작업을 수행하여

3) 그렇지 않다면 수강취소를 해야겠다는 확신을 주게 될 것이다. 그러나 잠깐, 당신이 VM을 잘 이해할 수 있다면 끝까지 내용을 다 소화할 가능성이 높다고 생각한다.

프로그램과 운영체제를 보호한다. 전체적인 접근 방식은 많은 기법들, 많은 저수준 하드웨어 기능과 핵심적인 정책들을 필요로 한다. 우리는 기초적인 필수 "기법"들에 대한 설명을 시작으로 점차 고수준의 정책을 설명할 것이다.

참고 문헌

[CV65] **"Introduction and Overview of the Multics System"**
F.J. Corbato and V.A. Vyssotsky
Fall Joint Computer Conference, 1965
훌륭한 초창기 Multics 논문. 시분할에 대한 좋은 인용이 여기 있다: "시분할에 대한 추진력은 배치 처리 기관에서 프로그램을 디버깅 할 때 경험한 끊임없는 좌절감 때문에 전문 프로그래머들로부터 처음 발생했다. 따라서 시분할의 원래 목표는 컴퓨터를 시분할 하여 많은 사람들이 동시에 사용할 수 있게 하면서 사용자에게는 자신이 컴퓨터 전부를 마음대로 할 수 있다는 환상을 주는 것이다."

[DH66] **"Programming Semantics for Multiprogrammed Computations"**
Jack B. Dennis and Earl C. Van Horn
Communications of the ACM, Volume 9, Number 3, March 1966
멀티프로그래밍에 관한 초기 논문(첫 번째 논문은 아님).

[Han70] **"The Nucleus of a Multiprogramming System"**
Per Brinch Hansen
Communications of the ACM, 13:4, April 1970
운영체제 또는 커널은 맞춤형 운영체제를 구현하기 위한 최소 기능과 융통성을 가지는 층이어야 한다는 첫 번째 논문. 이 주제는 운영체제 연구 역사상 전반에 걸쳐 재논의 된다.

[Lic60] **"Man-Computer Symbiosis"**
J.C.R. Licklider
IRE Transactions on Human Factors in Electronics, HFE-1:1, March 1960
컴퓨터와 사람이 어떻게 공존할 수 있는지에 관한 고풍스러운 논문. 시대를 앞서 나갔지만 재미있게 읽을 수 있다.

[McC62] **"Time-Sharing Computer Systems"**
J. McCarthy
Management and the Computer of the Future, MIT Press, Cambridge, Mass, 1962
아마도 McCarthy의 시분할에 관한 기록된 초기 논문. 그러나 다른 논문[McC83]에서 1957년부터 이 아이디어를 구상 중이었다고 말함. McCarthy는 시스템 분야를 떠나 Stanford에서 LISP 프로그래밍 언어를 만드는 등 인공 지능 분야의 거인이 되었다. 자세한 사항은 McCarthy의 홈페이지를 보시오: `http://www-formal.stanford.edu/jmc/`

[McC+63] **"A Time-Sharing Debugging System for a Small Computer"**
J. McCarthy, S. Boilen, E. Fredkin, and J.C.R. Licklider
AFIPS '63 (Spring), May, 1963, New York, USA
프로그램이 실행되지 않을 때 프로그램 메모리를 "drum"에 스왑-아웃하고 다시 실행되어야 할 때 "core" 메모리에 다시 탑재되는 시스템의 초기 훌륭한 예.

[McC83] **"Reminiscences on the History of Time Sharing"**
John McCarthy
Winter or Spring of 1983
URL: `http://www-formal.stanford.edu/jmc/history/timesharing/timesharing.html`
시분할 아이디어가 어디서 비롯되었는지에 관한 멋진 역사적인 노트. Strachey의 연구[Str59]를 이 분야의 선구적 연구라고 인용한 사람들에 대한 의문 제기도 포함하고 있음.

[Ras+89] **"Mach: A System Software kernel"**

Richard Rashid, Daniel Julin, Douglas Orr, Richard Sanzi, Robert Baron, Alessandro Forin, David Golub, and Michael Jones
COMPCON 89, February 1989
마이크로커널에 관한 첫 번째 프로젝트는 아닐지라도 CMU의 Mach 프로젝트는 유명하고 영향을 크게 미쳤다. 이 운영체제는 Mac OS X의 깊숙한 곳에 아직도 현존하고 있다.

[Str59] **"Time Sharing in Large Fast Computers"**
C. Strachey
Proceedings of the International Conference on Information Processing, UNESCO, June 1959
시분할에 관한 초기 참고 문헌의 하나.

[SBL03] **"Improving the Reliability of Commodity Operating Systems"**
Michael M. Swift, Brian N. Bershad, and Henry M. Levy
SOSP 2003
마이크로커널 사고 방식이 운영체제 신뢰성을 얼마나 증대시킬 수 있는지를 보여 주는 첫 번째 논문.

Homework (Code)

In this homework, we'll just learn about a few useful tools to examine virtual memory usage on Linux-based systems. This will only be a brief hint at what is possible; you'll have to dive deeper on your own to truly become an expert (as always!).

Questions

1. The first Linux tool you should check out is the very simple tool `free`. First, type `man free` and read its entire manual page; it's short, don't worry!
2. Now, run `free`, perhaps using some of the arguments that might be useful (e.g., `-m`, to display memory totals in megabytes). How much memory is in your system? How much is free? Do these numbers match your intuition?
3. Next, create a little program that uses a certain amount of memory, called `memory-user.c`. This program should take one command-line argument: the number of megabytes of memory it will use. When run, it should allocate an array, and constantly stream through the array, touching each entry. The program should do this indefinitely, or, perhaps, for a certain amount of time also specified at the command line.
4. Now, while running your `memory-user` program, also (in a different terminal window, but on the same machine) run the `free` tool. How do the memory usage totals change when your program is running? How about when you kill the `memory-user` program? Do the numbers match your expectations? Try this for different amounts of memory usage. What happens when you use really large amounts of memory?
5. Let's try one more tool, known as `pmap`. Spend some time, and read the `pmap` manual page in detail.
6. To use `pmap`, you have to know the **process ID** of the process you're interested in. Thus, first run `ps auxw` to see a list of all processes; then, pick an interesting one, such as a browser. You can also use your `memory-user` program in this case (indeed, you can even have that program call `getpid()` and print out its PID for your convenience).
7. Now run `pmap` on some of these processes, using various flags (like `-X`) to reveal many details about the process. What do you see? How many different entities make up a modern address space, as opposed to our simple conception of code/stack/heap?
8. Finally, let's run `pmap` on your `memory-user` program, with different amounts of used memory. What do you see here? Does the output from `pmap` match your expectations?

© 2008–23, ARPACI-DUSSEAU

Homework (Code) - 한국어 번역

이번 숙제에서는 Linux 기반 시스템에서 가상 메모리 사용량을 확인할 수 있는 몇 가지 유용한 도구를 간단히 배운다. 여기서는 가능한 기능의 일부만 살펴보며, 제대로 익히려면 직접 더 깊게 공부해야 한다.

문제

1. 가장 먼저 살펴볼 Linux 도구는 매우 간단한 free이다. 먼저 man free를 입력하고 매뉴얼 페이지 전체를 읽어보자. 내용이 짧으니 걱정하지 않아도 된다.

2. 이제 free를 실행해보자. 필요하다면 -m과 같은 옵션을 사용해도 된다. -m은 메모리 용량을 MB 단위로 표시한다. 시스템에는 전체 메모리가 얼마나 있는가? 사용 가능한 메모리는 얼마나 되는가? 이 수치들이 예상했던 것과 일치하는가?

3. 다음으로 일정량의 메모리를 사용하는 간단한 프로그램 memory-user.c를 작성하라. 이 프로그램은 명령행 인자 하나를 입력받아야 하며, 이 값은 프로그램이 사용할 메모리의 크기(MB)를 의미한다. 프로그램을 실행하면 지정된 크기의 배열을 할당하고, 배열 전체를 계속 순회하면서 각 원소에 접근해야 한다. 이 작업을 무한히 반복하도록 만들거나, 명령행에서 실행 시간을 추가로 입력받아 일정 시간 동안만 실행하도록 만들어도 된다.

4. memory-user 프로그램을 실행한 상태에서 같은 컴퓨터의 다른 터미널 창을 열어 free를 실행하라. 프로그램 실행 중 메모리 사용량이 어떻게 변하는가? memory-user 프로그램을 종료하면 어떻게 변하는가? 결과가 예상과 일치하는가? 서로 다른 크기의 메모리를 사용하도록 설정하여 실험해보라. 아주 큰 양의 메모리를 사용하면 어떤 일이 발생하는가?

5. 이번에는 pmap이라는 도구를 사용해보자. 시간을 들여 pmap의 매뉴얼 페이지를 자세히 읽어보라.

6. pmap을 사용하려면 조사하려는 프로세스의 프로세스 ID(PID)를 알아야 한다. 먼저 ps auxw를 실행하여 모든 프로세스의 목록을 확인한다. 그런 다음 웹 브라우저처럼 관심 있는 프로세스 하나를 선택한다. 앞에서 작성한 memory-user 프로그램을 사용해도 된다. 편의를 위해 memory-user 프로그램에서 getpid()를 호출하여 자신의 PID를 출력하도록 만들어도 된다.

7. 이제 선택한 프로세스에 대해 pmap을 실행해보라. -X 같은 여러 옵션을 사용하면 프로세스에 대한 더 자세한 정보를 확인할 수 있다. 어떤 정보들이 나타나는가? 우리가 단순하게 배운 코드(code) / 스택(stack) / 힙(heap) 구조와 비교했을 때, 실제 현대 프로세스의 주소 공간은 얼마나 다양한 영역들로 구성되어 있는가?

8. 마지막으로 서로 다른 크기의 메모리를 사용하도록 memory-user를 실행하고, 각각에 대해 pmap을 실행해보라. 어떤 차이가 나타나는가? pmap의 출력 결과가 예상했던 것과 일치하는가?

17

막간: 메모리 관리 API

이번 장에서는 UNIX의 메모리 관리 인터페이스에 대해 논의한다. 제공되는 인터페이스들은 비교적 간단하기 때문에 이 장은 짧고 핵심적인 내용만 다룬다[1]. 여기서의 메모리라 함은 사용자 주소 공간을 의미한다. 우리가 해결하고자 하는 문제는 다음과 같다:

핵심 질문: 어떻게 메모리를 할당하고 관리해야 하는가
UNIX/C 프로그램에서 메모리를 할당하고 관리하는 방법을 이해하는 것은 강력하고 안정적인 소프트웨어를 구축하는 데 중요하다. 일반적으로 어떤 인터페이스가 사용되는가? 어떤 실수를 해서는 안 되는가?

17.1 메모리 공간의 종류

C 프로그램이 실행되면, 두 가지 유형의 메모리 공간이 할당된다. 첫 번째는 스택(stack) 메모리라고 불리며 할당과 반환은 프로그래머를 위해 컴파일러에 의해 *암묵적으로* 이루어진다. 이러한 이유 때문에 때로는 **자동**(**automatic**) 메모리라고 불린다.

C 프로그램에서 스택에 메모리를 선언하는 것은 쉽다. 예를 들어, `func()`라는 함수 안에서 `x`라 불리는 정수를 위한 공간이 필요하다고 하자. 이러한 메모리를 선언하려면 다음과 같이 하면 된다.

```
void func() {
     int x; // 스택에 int 형을 선언
     ...
}
```

컴파일러가 나머지 작업을 수행하여, `func()`가 호출될 때 스택에 공간을 확보한다. 함수에서 리턴하면 컴파일러는 프로그래머 대신에 메모리를 반환한다. 함수 리턴 이후에도 유지되어야 하는 정보는 스택에 저장하지 않는 것이 좋다.

오랫동안 값이 유지되어야 하는 변수를 위해 힙(`heap`) 메모리라고 불리는 두 번째 유형의 메모리가 필요하다. 모든 할당과 반환이 프로그래머에 의해 명시적으로 처리된다.

1) 사실 우리는 모든 장이 그러길 바란다. 그러나 우리 생각엔 이 장이 유난히 짧고 더 핵심적이다.

당연히 무거운 책임! 그리고 많은 버그의 원인. 그러나 조심하고 주의를 기울이면 큰 문제 없이 올바르게 인터페이스를 사용할 수 있다. 다음 코드는 정수에 대한 포인터를 힙에 할당하는 예를 보여 준다.

```
void func() {
     int *x = (int *) malloc(sizeof(int));
     ...
}
```

이 코드에 대한 주의 사항이 몇 개 있다. 첫째, 한 행에 스택과 힙 할당이 모두 발생한다. 우선 컴파일러가 포인터 변수의 선언(`int *x`)을 만나면 정수 포인터를 위한 공간을 할당해야 한다는 것을 안다. 프로그램이 `malloc()`을 호출하여 정수를 위한 공간을 힙으로부터 요구한다. `malloc()`은 그 정수의 주소를 반환한다(성공한 경우, 실패한 경우에는 `NULL`을 반환). 이 반환된 주소는 스택에 저장되어 프로그램에 의해 사용된다.

이런 명시적 성질과 다양한 쓰임새 때문에 힙 메모리의 사용은 사용자와 시스템 모두에게 어려운 숙제다. 남은 논의는 힙 메모리에 초점을 맞출 것이다.

17.2 malloc() 함수

`malloc()` 호출은 매우 간단하다. 힙에 요청할 공간의 크기를 넘겨 주면, 성공했을 경우 새로 할당된 공간에 대한 포인터를 사용자에게 반환하고 실패했을 경우 `NULL`을 반환한다[2].

매뉴얼 페이지는 `malloc`을 사용하는 데 필요한 일을 보여 준다. 명령어 라인에 `man malloc`이라고 입력하면 다음과 같은 내용을 볼 것이다.

```
#include <stdlib.h>
...
void *malloc(size_t size);
```

이 정보로부터 `malloc`을 사용하기 위해서 할 일은 헤더 파일 `stdlib.h`를 소스 코드에 포함시키는 것 뿐이라는 것을 알 수 있다. 사실 이 일도 할 필요 없다. 모든 C 프로그램에 디폴트로 링크되는 C 라이브러리는 `malloc()` 함수를 가지고 있다. 헤더 파일을 추가하면 `malloc()`을 올바르게 호출했는지 컴파일러가 검사할 수 있다. 전달된 인자의 개수가 맞는지 올바른 데이터 타입의 인자를 전달했는지 검사하게 된다.

`malloc()`의 인자는 `size_t` 타입의 변수이고 이 변수는 필요 공간의 크기를 바이트 단위로 표시한 것이다. 대부분의 프로그래머는 숫자를 직접 입력하지 않는다(10 등). 사실 그런 식으로 입력하는 것은 좋지 않다. 대신 다양한 루틴과 매크로가 활용된다. 예를 들어, `double precision`의 부동 소수점 값을 위한 공간을 확보하기 위해서 다음과 같이 하면 된다.

```
double *d = (double *) malloc(sizeof(double));
```

2) C 언어에서 `NULL`은 특별한 것이 전혀 아니다. 단순히 0 값을 나타내는 매크로일 뿐이다.

팁: 의심스러우면 한 번 써 본다

당신이 사용하고 있는 루틴 또는 연산자가 어떻게 동작하는지 확신할 수 없으면, 실행시켜 보고 예상한 대로 동작하는지 확인하는 방법 이외의 대안은 없다. 매뉴얼 페이지나 기타 문서를 읽는 것이 유용하긴 하지만, 실제 동작을 확인하는 것이 중요하다. 코드를 작성해서 실험해 보라. 당연히 이 방법이 작성한 코드가 예상대로 동작하는지 확인하는 가장 최선의 방법이다. 사실 우리도 `sizeof()`에 대해 우리가 말했던 내용이 사실이라는 것을 재차 확인하기 위하여 실제로 실행시켜 보았다.

와, `double`이 엄청 많네! 이 `malloc()` 호출에서는 정확한 크기의 공간을 요청하기 위하여 `sizeof()` 연산자를 사용한다. C 언어에서 이 `sizeof()`는 통상 컴파일 시간 연산자이다. 인자의 실제 크기가 컴파일 시간에 결정된다. `sizeof()`는 숫자(이 경우 `double`의 크기인 8)로 대체되어 `malloc()`에 전달된다. 이러한 이유로, `sizeof()`는 연산자로 간주되는 게 맞으며 함수 호출이 아니다(함수 호출은 실행 시간에 일어난다).

데이터 타입뿐 아니라 변수의 이름도 `sizeof()`의 인자로 전달할 수 있다. 그러나 원하는 결과를 얻지 못할 때도 있으므로 조심해야 한다. 예를 들어, 다음과 같은 코드를 살펴보자.

```
int *x = malloc(10 * sizeof(int));
printf("%d\n", sizeof(x));
```

첫 번째 행에서, 정수형 원소 10개를 가지는 배열을 위한 공간을 선언하였다. 멋진 코드. 그러나 다음 줄에서 `sizeof()`를 사용하면 4(32비트 컴퓨터에서) 또는 8(64비트 컴퓨터에서) 값을 반환한다. 이 경우 `sizeof()`는 동적으로 할당받은 메모리의 크기가 얼마인지가 아니라, 정수를 가리키는 포인터의 크기가 얼마인지 물어본다고 생각하기 때문이다. 그러나 때때로 `sizeof()`는 기대한 대로 동작할 때도 있다:

```
int x[10];
printf("%d\n", sizeof(x));
```

이 경우에는 변수 x에 40바이트가 할당되었다는 것을 컴파일러가 알 수 있는 정적인 정보가 충분하다.

또 하나 조심해야 하는 경우는 문자열을 다룰 때이다. 문자열을 위한 공간을 선언할 때에는 다음과 같은 문장을 사용한다: `malloc(strlen(s) + 1)`. 이 문장은 `strlen()` 함수를 사용하여 문자열의 길이를 얻어낸 뒤 문자열-끝을 나타내는 문자를 위한 공간을 확보하기 위해 1바이트를 더한다. `sizeof()`의 사용은 여기서 문제를 일으킬 수 있다.

`malloc()`은 `void` 타입에 대한 포인터를 반환한다는 것을 알았을 것이다. 그렇게 하는 것은 주소만 넘겨주고 해당 주소 공간에 어떤 타입의 자료를 저장할 지는 프로그래머가 결정하게 하는 전형적인 C의 방식이다. 프로그래머는 **타입 변환**(**type casting**)을 이용하여 공간 활용을 결정한다. 위의 예에서 프로그래머는 `malloc()`이 반환한 데이터 타입을 `double` 형을 가리키는 포인터 타입으로 변환하였다. 캐스팅은 실제

무언가를 하지는 않고, 컴파일러와 다른 프로그래머에게 "예, 제가 알아서 할게요"라고 말할 뿐이다. `malloc()`의 결과에 타입을 명시함으로써, 프로그래머는 정확한 타입을 "확인"한 정도이며, 프로그램이 제대로 동작하는 데 있어서 캐스트가 반드시 필요한 것은 아니다.

17.3 `free()` 함수

"메모리 할당"은 우리가 고민하고 있는 문제들 중 쉬운 쪽이다. 할당된 메모리를 언제, 어떻게 해제하고 더욱이 해제 여부를 확인하는 것이 더 어려운 문제이다. 더 이상 사용되지 않는 힙 메모리를 해제하기 위해 프로그래머는 `free()`를 호출한다.

```
int *x = malloc(10 * sizeof(int));
...
free(x);
```

한 개의 인자, `malloc()`에 의해 반환된 포인터를 받는다. 할당된 영역의 크기는 전달되지 않는다. 할당된 메모리의 크기는 메모리 할당 라이브러리가 알고 있어야 한다.

17.4 흔한 오류

`malloc()`과 `free()`를 사용하는 데 흔히 발생하는 오류가 많다. 학부 운영체제를 가르치면서 반복적으로 보아 왔던 오류 중 일부를 소개하겠다. 모든 예들은 컴파일러 최적화 없이 컴파일되고 실행되었다. C 프로그램을 컴파일하는 것은 제대로 돌아가는 C 프로그램을 제작하는 데 반드시 필요하지만, 그것만으로는 충분하지 않다는 것을 배우게 될 것이다.

올바른 메모리 관리가 그렇다. 많은 새로운 언어들이 **자동 메모리 관리**(**automatic memory management**)를 지원한다. 그러한 언어들에서는 메모리를 할당하기 위하여 `malloc()`과 유사한, 보통 **new** 또는 새 객체를 할당하기 위한 비슷한 루틴을 호출하는 반면에, 공간을 해제하기 위해서는 아무것도 호출하지 않는다. 쓰레기 수집기(garbage collector)가 실행되어 참조되지 않는 메모리가 찾아 알아서 해제한다.

메모리 할당 잊어버리기

많은 루틴은 자신이 호출되기 전에 필요한 메모리가 미리 할당되었다고 가정한다. 예를 들어, `strcpy(dst, src)` 루틴은 소스 포인터에서 목적 포인터로 문자열을 복사한다. 그러나 주의하지 않으면, 다음과 같이 코드를 작성할 수 있다.

```
char *src = "hello";
char *dst;        // 아뿔싸! 할당이 안 되어 있네
strcpy(dst, src); // segfault 그리고 죽는다
```

팁: 컴파일되었다 또는 실행되었다 ≠ 원하는 결과다

단지 프로그램이 컴파일(!) 또는 심지어 한 번 또는 여러 번 정확하게 실행되었다고 하더라도 프로그램이 정확하다는 것을 의미하지는 않는다. 많은 사건이 어우러져 프로그램이 잘 동작한다고 믿게끔 만들 수 있지만, 뭔가가 바뀌면 동작을 중지하게 된다. 대부분 학생들의 반응은 "그러나 전에는 동작했단 말이에요"라고 말하거나 소리지르고, 컴파일러, 운영체제, 하드웨어, (감히 말하건대) 심지어는 교수를 비난한다. 그러나 문제는 보통 거기 있을 것이라고 생각하는 바로 그 곳, 당신의 코드 안에 있다. 다른 요소를 비난하기 전에 자리로 돌아가서 디버깅해라.

이 코드를 실행하면 **세그멘테이션 폴트(segmentation fault)**[3]가 발생할 가능성이 높다. 이 폴트는 "**네가 메모리 관련 무언가를 잘못했어, 이 바보 같은 프로그래머야, 그래서 나 화났거든**"이라는 메시지이다.

이 경우, 올바른 코드는 다음과 같을 수 있다:

```
char *src = "hello";
char *dst = (char *) malloc(strlen(src) + 1);
strcpy(dst, src); // 제대로 동작
```

대안으로, `strdup()`을 사용하여 훨씬 편하게 할 수 있다. 더 많은 정보를 원한다면 `strdup`의 `man` 페이지를 읽으시오.

메모리를 부족하게 할당받기

이 오류는 메모리를 부족하게 할당받는 것으로, 때때로 **버퍼 오버플로우(buffer overflow)**라고 불린다. 앞의 예에서, 흔한 오류는 목적지 버퍼 공간을 약간 부족하게 할당받는 것이다.

```
char *src = "hello"
char *dst = (char *) malloc(strlen(src)); // 너무 작다!
strcpy(dst, src); // 제대로 동작
```

이상하지만 `malloc`이 구현 방식과 기타 많은 세부 사항에 따라서는 이 프로그램이 제대로 동작하는 것처럼 보이는 경우가 종종 있다. 어떤 경우에는 문자열 복사가 실행될 때 할당된 공간의 마지막을 지나쳐 한 바이트 만큼 더 공간을 사용한다. 이 공간이 더 이상 사용되지 않는 변수 영역이기 때문에 덮어쓰더라도 아무 피해가 발생하지 않을 때도 있다. 다른 때에는 이러한 오버플로우가 매우 유해할 수 있고 사실 많은 시스템에서 보안 취약점의 원인이다 [Wer06]. 어떤 경우에는 `malloc` 함수 라이브러리가 여분의 공간을 할당하고 프로그램은 다른 변수의 값을 덮어쓰지 않고 잘 동작한다. 또 다른 경우에는 프로그램은 고장을 일으키고 크래시된다. 다음과 같은 또다른 중요한 교훈을 얻는다: 프로그램이 한 번 올바르게 실행된다고 하더라도, 프로그램이 올바르다는 것을

3) 용어가 신비롭게 들리겠지만, 이러한 불법적인 메모리 접근이 왜 세그멘테이션 폴트라고 불리는지 곧 배우게 될 것이다. 이게 계속 책을 읽을만한 인센티브가 아니라면, 뭐를 하면 계속 책을 읽을래?

의미하지는 않는다.

할당받은 메모리 초기화하지 않기

이 오류의 경우, `malloc()`을 제대로 호출했지만 새로 할당받은 데이터 타입에 특정 값을 넣는 것을 잊는 것이다. 절대로 이러지 마라. 초기화하지 않는다면, 프로그램은 결국 **초기화되지 않은 읽기**(**uninitialized read**), 즉 힙으로부터 알 수 없는 값을 읽는 일이 생긴다. 그 영역에 어떤 값이 들어 있을지 누가 알겠는가? 재수가 좋으면 프로그램이 여전히 잘 동작할 수 있는 값일테고, 그게 아니라면 임의의 값이나 해로운 값이 읽힐 것이다.

메모리 해제하지 않기

다른 일반적인 오류는 **메모리 누수**(**memory leak**)다. 메모리 해제를 잊었을 때 발생한다. 장시간 실행되는 응용 프로그램이나 또는 운영체제 자체와 같은 시스템 프로그램에서는 큰 문제이다. 메모리가 천천히 누수되면 결국 메모리가 부족하게 되고 시스템을 재시작 할 수밖에 없기 때문이다. 메모리 청크의 사용이 끝나면 반드시 해제해야 한다. 쓰레기 수집 기능이 있는 언어도 이 문제에는 도움이 되지 않는다. 메모리 청크에 대한 참조가 존재하면, 어느 쓰레기 수집기도 그 청크를 해제하지 않을 것이고, 따라서 이런 현대적인 언어에서도 메모리 누수는 여전히 문제가 된다.

경우에 따라서 `free()`를 호출하지 않는 것이 타당하게 보이는 경우도 있다. 예를 들어, 프로그램 실행 시간이 짧아, 실행 시작 후 곧 종료한다고 하자. 이 경우 프로세스가 죽으면, 운영체제는 할당된 페이지를 모두 정리하고 따라서 메모리 누수는 일어나지 않는다. 이 가정이 맞을 수도 있지만, 들여서는 안 되는 나쁜 버릇으로서, 이와 같은 전략은 매우 조심해서 사용해야 한다 (다음 여담을 참조하시오). 장기적으로 프로그래머로서 당신의 목표는 좋은 습관을 만드는 것이다. 중요한 습관 중 하나가 C 같은 언어에서 어떻게 메모리를 관리하는지 이해하고 할당받았던 메모리 블럭을 해제하는 것이다. 메모리 해제 없이도 그럭저럭 개발자로 지낼 수 있겠지만 한 바이트라도 명시적으로 할당받았으면 해제하는 습관을 들이도록 하자.

메모리 사용이 끝나기 전에 메모리 해제하기

때때로 프로그램은 메모리 사용이 끝나기 전에 메모리를 해제한다. 그런 실수는 **dangling pointer**라고 불리며 심각한 실수이다. 차후 그 포인터를 사용하면 프로그램을 크래시 시키거나 유효 메모리 영역을 덮어쓸 수 있다. 예를 들어, `free()`를 호출하고, 그 후 다른 용도로 `malloc()`을 호출하면 잘못 해제된 메모리를 재사용한다.

반복적으로 메모리 해제하기

프로그램은 가끔씩 메모리를 한 번 이상 해제하며 **이중 해제**(**double free**)라 불린다. 결과는 예측하기 어렵다. 상상할 수 있듯이 메모리 할당 라이브러리는 어찌할 바 모르게

여담: 프로세스가 종료하면 왜 메모리 누수가 일어나지 않는가

실행 시간이 짧은 프로그램을 작성할 때 `malloc()`을 이용하여 일부 공간을 할당할 수 있다. 프로그램은 실행 후 곧 종료할 것이다. 종료 직전에 `free()`를 호출할 필요가 있을까? 호출하지 않으면 잘못한 것처럼 보이지만 실제로는 손해보지 않는다. 이유는 간단하다. 시스템이 실제 두 단계로 메모리를 관리하기 때문이다. 메모리 관리의 첫 번째 단계는 운영체제에 의해 수행된다. 프로세스가 실행할 때 메모리를 프로세스에게 건네 주고 프로세스가 종료하거나 다른 이유로 죽을 때 메모리를 되돌려 받는다. 두 번째 단계는 각 프로세스 내에서, 예를 들면, `malloc()`과 `free()`를 호출할 때 힙 내에서 수행된다. `free()`를 호출하지 못했더라도 그래서 메모리가 누수되었더라도 프로세스가 종료할 때, 운영체제는 프로세스의 모든 메모리를 회수한다. 프로세스의 코드, 스택, 및 여기서 논의 중인 힙 등 모든 페이지가 포함된다. 프로세스 주소 공간의 힙이 어떤 상태가 되더라도, 운영체제는 프로세스가 죽을 때 그 모든 페이지를 회수한다. 당신이 해제하지 않았더라도 메모리가 누수되지 않는다.

이처럼 실행 시간이 짧은 프로그램의 경우, 나쁜 형태의 코드라고 생각되지만 메모리 누수는 어떠한 동작적인 문제도 일으키지 않는 경우가 대부분이다. Web 서버나 데이터베이스 관리 시스템 등의 결코 종료하지 않는 장시간 실행되는 서버(웹서버나 데이터베이스 관리 시스템과 같이 절대 종료하지 않는 것)의 코드를 작성할 때, 누수 메모리는 훨씬 심각한 문제이고 응용 프로그램이 메모리를 다 소진하게 되면 결국 크래시가 발생한다. 물론, 메모리 누수는 한 가지 특정 프로그램에서 더 큰 문제가 된다: 운영체제 자신. 이러한 사실은 커널 코드를 작성하는 사람들이 가장 힘들고 어려운 일을 한다는 걸 우리에게 다시 한 번 상기시킨다.

되고, 모든 종류의 이상한 일을 하게 된다. 가장 흔히 일어나는 결과는 크래시다.

`free()` 잘못 호출하기

마지막 문제점은 `free()`를 잘못 호출하는 것이다. `free()`는 `malloc()` 받은 포인터만 전달될 것으로 예상한다. 그 이외의 값을 전달하면 문제가 발생한다. **유효하지 않은 해제(invalid frees)**는 매우 위험하고 당연히 피해야 한다.

요약

메모리를 오용하는 방법은 많다. 메모리 관련 오류가 자주 발생하기 때문에, 코드에서 그런 오류를 찾아내는 도구의 생태계가 생겼다. **purify** [HJ92]와 **valgrind** [SN05]를 검토해 보라. 메모리 관련 문제의 원인을 찾아내는 훌륭한 도구들이다. 이 강력한 도구의 사용에 익숙해지면 그동안 그들없이 어떻게 살았나 하는 의문이 들 것이다.

17.5 운영체제의 지원

`malloc()`과 `free()`를 논의해 오면서 시스템 콜에 관해서는 한 번도 언급하지 않았다. 이유는 간단하다: 그들은 시스템 콜이 아니라 라이브러리 함수이기 때문이다! `malloc` 라이브러리가 프로세스 가상 주소 공간 안의 공간을 관리하지만 라이브러리 자체는 시스템에게 더 많은 메모리를 요구하고 반환하는 시스템 콜을 기반으로 구축된다.

그런 시스템 콜 중 하나가 `brk`라고 불리는 시스템 콜로서, 프로그램의 **break** 위치를 변경하는 데 사용된다. `break`는 힙의 마지막 위치를 나타낸다. `brk`는 새로운 `break` 주소를 나타내는 한 개의 인자를 받는다. 새로운 `break`가 현재 `break`보다 큰지 작은지에 따라 힙의 크기를 증가시키거나 감소시킨다. `sbrk`는 증가량만을 받아들이는 것을 제외하고 비슷한 용도로 사용된다.

`brk` 또는 `sbrk`를 직접 호출해서는 안 된다는 것에 주의하라. 이들은 메모리 할당 라이브러리에 의해 사용된다. 직접 사용하면 감당할 수 없는 결과가 발생할 것이다. 반드시 `malloc()`과 `free()`를 사용하라.

마지막으로, `mmap()` 함수를 사용하여 운영체제로부터 메모리를 얻을 수도 있다. 올바른 인자를 전달하면 `mmap()`은 프로그램에 **anonymous**의 메모리 영역을 만든다. `anonymous` 영역은 특정 파일과 연결되어 있지 않고 **스왑 공간**(**swap space**)에 연결된 영역을 말한다. 이 영역에 대해서는 나중에 가상 메모리에서 논의한다. 이 메모리는 힙처럼 취급되고 관리된다. 자세한 내용은 `mmap()`의 매뉴얼 페이지를 읽어라.

17.6 기타 함수들

메모리 할당과 관련하여 다른 함수들이 몇 개 더 있다. 예를 들어, `calloc()`은 메모리를 할당 영역을 0으로 채워서 반환한다. 이 함수는 초기화하는 것을 잊어버리는 오류를 방지한다(앞에서 설명한 "초기화되지 않은 읽기"에 관한 문단을 참조하라). `realloc()`은 이미 할당된 공간에 대해(예를 들어, 배열) 추가의 공간이 필요할 때 유용하다. `realloc()`은 더 큰 새로운 영역을 확보하고 옛 영역의 내용을 복사한 후에 새 영역에 대한 포인터를 반환한다.

17.7 요약

우리는 메모리 할당을 다루는 몇 가지 API를 소개하였다. 기본적인 내용만을 다루었다. 자세한 내용은 다른 곳에서 찾을 수 있다. 더 많은 정보를 원한다면 C 책 [KR88]과 Stevens [SR05] (7장)를 읽어라. 많은 오류들을 자동적으로 감지하고 수정하는 방법에 대한 Novark의 논문을 읽기 바란다 [NBZ07]. 이 논문에는 자주 발생하는 문제들과 그것들을 발견하고 수정하는 방법에 잘 요약해 놓았다.

참고 문헌

[HJ92] **"Purify: Fast Detection of Memory Leaks and Access Errors"**
R. Hastings and B. Joyce
USENIX Winter '92
멋진 Purify 도구에 관한 논문. 현재는 상업용 제품이다.

[KR88] **"The C Programming Language"**
Brian Kernighan and Dennis Ritchie
Prentice-Hall 1988
C 언어 개발자의 C 책. 한 번 읽은 후에 프로그래밍을 해 보라. 그런 후 다시 읽어라. 프로그램 할 때마다 책상 가까운 곳에 두어라.

[NBZ07] **"Exterminator: Automatically Correcting Memory Errors with High Probability"**
Gene Novark, Emery D. Berger, and Benjamin G. Zorn
PLDI 2007
메모리 오류를 자동적으로 찾아 수정하는 것에 관한 멋진 논문. 또한 C와 C++ 프로그램의 일반적인 오류에 관한 개괄도 제공한다.

[SN05] **"Using Valgrind to Detect Undefined Value Errors with Bit-precision"**
J. Seward and N. Nethercote
USENIX '05
특정 유형의 오류를 발견하는 것이 목적인 경우의 valgrind 사용법.

[SR05] **"Advanced Programming in the Unix Environment"**
W. Richard Stevens and Stephen A. Rago
Addison-Wesley, 2005
예전에도 이야기 했지만 다시 말한다. 이 책을 여러 번 읽어라 그리고 의문이 생길 때마다 참고 문헌으로 사용하라. C 프로그래밍 경험이 많음에도 불구하고 저자들은 책을 읽을 때마다 어떻게 새로운 것을 배울 수 있는지 항상 놀란다.

[Wer06] **"Survey on Buffer Overflow Attacks and Countermeasures"**
Tim Werthman
URL: `www.nds.rub.de/lehre/seminar/SS06/Werthmann_BufferOverflow.pdf`
버퍼 오버플로우와 그들이 일으키는 보안 문제에 관한 훌륭한 요약서. 유명한 취약점 공격을 보려면 참조하시오.

숙제 (코드)

이 과제에서 메모리 할당에 대한 어느 정도의 지식을 얻을 수 있다. 먼저 몇몇 버그 있는 프로그램을 작성한다(재미있겠지!). 다음, 당신이 삽입한 버그를 발견하는 데 도움을 주는 몇 가지 도구를 사용한다. 그러면 이러한 도구가 얼마나 대단한지 알게 될 것이고 미래에 그들을 사용하게 되어 행복하고 생산적인 프로그래머가 될 것이다.

사용할 첫 번째 도구는 `gdb` 디버거이다. 이 디버거에 대해 배울 것이 많다. 여기서는 단지 수박 겉핥기식으로 넘어갈 것이다.

두 번째 사용할 도구는 `valgrind` [SN05]이다. 이 도구는 프로그램의 메모리 누수와 다른 교묘한 메모리 문제를 발견하는 데 도움을 준다. 만약 설치되어 있지 않다면 다음 web 사이트에 가서 설치하라.

`http://valgrind.org/downloads/current.html`

문제

1. `null.c`라는 간단한 프로그램을 작성하라. 이 프로그램은 정수를 가리키는 포인터를 만들고 NULL로 초기화하고 그것을 역참조하려고 한다. `null`이라는 실행 파일이 생성되도록 컴파일하라. 프로그램을 실행시키면 어떤 일이 발생하는가? 프로그램 죽음
2. 다음에 이 프로그램을 심볼 정보가 포함되도록 컴파일하라(`-g` 플래그 사용). 실행 파일에 더 많은 정보를 추가하여 디버거가 변수 이름 등의 유용한 정보를 접근할 수 있게 한다. 디버거에서 프로그램을 실행하라(`gdb null`). `gdb`가 실행되면, `run` 명령어를 입력하라. `gdb`는 무엇을 보여주는가?
3. 마지막으로, 이 프로그램에 `valgrind` 도구를 사용하라. 우리는 `valgrind`의 일부인 `memcheck` 도구를 사용하여 어떤 일이 일어나는지 분석할 것이다. 다음과 같은 명령어를 실행시켜라: `valgrind --leak-check=yes null`. 이를 실행하면 어떻게 되는가? 출력을 해석할 수 있는가?
4. `malloc()`을 사용하여 메모리를 할당하지만 종료 전에 해제하는 것을 잊어버리는 간단한 프로그램을 작성하라. 이 프로그램을 실행시키면 무슨 일이 일어나는가? 이 프로그램의 문제를 gdb를 사용하여 발견할 수 있는가? `valgrind`는 어떤가(다시 `--leak-check=yes` 플래그를 주고 실행)?
5. 크기가 100인 `data`라는 이름의 정수 배열을 만드는 프로그램을 작성하라. 그런 후 `data[100]`을 0으로 클리어 하라. 이 프로그램을 실행하면 어떤 일이 벌어지는가? `valgrind`를 사용하여 프로그램을 실행하면 어떤 일이 일어나는가? 프로그램은 정확한가?
6. 정수 배열을 할당하고(위와 동일), 다시 해제하고, 배열의 원소 하나의 값을 출력하려고 시도하는 프로그램을 작성하라. 프로그램이 실행되는가? `Valgrind`를 사용하여 실행하면 어떤 일이 발생하는가?

7. `free`에 위에서 할당한 배열의 중간을 가리키는 포인터와 같은 황당한 값을 전달하라. 어떤 일이 벌어지는가? 이런 종류의 문제를 발견하기 위해 도구가 필요한가?

8. 메모리를 할당할 때 다른 인터페이스를 사용해 보라. 예를 들어, 간단한 벡터-비슷한 자료 구조를 만들고 벡터를 관리하기 위하여 `realloc()`을 사용하는 루틴을 만들어라. 벡터 원소를 저장하기 위하여 배열을 사용하라. 사용자가 벡터에 항목을 추가할 때, `realloc()`을 사용하여 필요 공간을 확보하라. 이 벡터 프로그램은 얼마나 잘 동작하는가? 연결 리스트와 비교하면 어떠한가? 버그를 발견하기 위하여 valgrind를 사용하라.

9. `gdb`와 `valgrind` 사용법을 배우기 위하여 시간과 노력을 투자하라. 도구를 잘 아는 것은 매우 중요하다. UNIX와 C 환경에서 디버거 전문가가 되기 위해 시간과 노력을 투자하라.

18

주소 변환의 원리

CPU 가상화 부분에서, **제한적 직접 실행**(**LDE**)이라는 기법을 집중적으로 다루었다. LDE의 아이디어는 간단하다. 대부분의 경우 프로그램은 하드웨어에서 직접 실행된다. 그러나, 프로세스가 시스템 콜을 호출하거나 타이머 인터럽트가 발생할 때 등의 특정 순간에는 운영체제가 개입하여 문제가 발생하지 않도록 한다. 운영체제는 약간의 하드웨어 지원을 받아 *효율적인* 가상화를 제공하기 위해 실행 프로그램에게 방해가 되지 않도록 최선을 다한다. 중요한 순간에 운영체제가 관여하여 하드웨어를 직접 제어한다. 효율성과 제어가 현대 운영체제의 목표이다.

메모리 가상화에서 비슷한 전략을 추구할 것이다. 가상화를 제공하는 동시에 효율성(efficiency)과 제어(control) 모두를 추구한다. 효율성을 높이려면 하드웨어 지원을 활용할 수밖에 없다. 처음에는 몇 개의 레지스터만 사용하는 정도부터 TLB, 페이지 테이블 등으로 점차 복잡한 하드웨어를 사용하게 될 것이다. 제어는 응용 프로그램이 자기자신의 메모리 이외에는 다른 메모리에 접근하지 못한다는 것을 운영체제가 보장하는 것을 의미한다. 프로그램을 다른 프로그램으로부터 보호하고 운영체제를 프로그램으로부터 보호하기 위하여 하드웨어 도움이 필요하다. 마지막으로, *유연성*(flexibility) 측면에서 VM 시스템에서 필요한 사항이 있다. 프로그래머가 원하는 대로 주소 공간을 사용하고, 프로그래밍하기 쉬운 시스템을 만들기 원한다. 핵심 문제는 다음과 같다.

핵심 질문: 어떻게 효율적이고 유연하게 메모리를 가상화하는가

어떻게 효율적인 메모리 가상화를 구축할 수 있을까? 프로그램이 필요로 하는 유연성을 어떻게 제공하는가? 프로그램이 접근할 수 있는 메모리의 위치에 대한 제어를 어떻게 유지하는가? 메모리 접근을 어떻게 적절히 제한할 수 있는가? 어떻게 이 모든 것을 효율적으로 할 수 있는가?

우리가 다룰 기법은 **하드웨어-기반 주소 변환**(**hardware-based address translation**) 또는 그냥 짧게 **주소 변환**(**address translation**)이다. 이 기술은 제한적 직접 실행 방식에 부가적으로 사용되는 기능이라고 생각할 수 있다. 주소 변환을 통해 하드웨어는 명령어 반입, 탑재, 저장 등의 **가상** 주소를 정보가 실제 존재하는 **물리** 주소로

변환한다. 프로그램의 모든 메모리 참조를 실제 메모리 위치로 재지정하기 위하여 하드웨어가 주소를 변환한다.

물론, 하드웨어에 의해 제공되는 저수준(low level) 기능들은 변환을 가속화 시키는 도움을 주지만, 하드웨어만으로 메모리 가상화를 구현할 수는 없다. 정확한 변환이 일어날 수 있도록 하드웨어를 셋업하기 위해 운영체제가 관여해야 한다. 운영체제는 메모리의 빈 공간과 사용 중인 공간을 항상 알고 있어야 하고, 메모리 사용을 제어하고 관리한다.

이 모든 작업의 목표는 다음과 같다. 프로그램이 자신의 전용 메모리를 소유하고 그 안에 자신의 코드와 데이터가 있다는 **환상**을 만드는 것이다. 이 가상 현실의 이면에는 불편한 진실이 존재한다. CPU가 실행 중인 한 프로그램에서 다음 프로그램으로 전환하는 것처럼, 많은 프로그램이 메모리를 공유한다. 운영체제는 하드웨어의 도움을 받아 쓸모없는 기계를 유용하고, 강력하며 사용하기 쉬운 개념으로 변환시킨다.

18.1 가정

메모리 가상화를 위한 첫 번째 시도는 매우 간단하다, 거의 터무니없을 정도다. 오케이, 실컷 웃어라. 잠시 후에 TLB, 멀티 레벨 페이지 테이블 및 기타 기법을 이해해야 할 때에는 운영체제가 당신을 비웃을 것이다. 운영체제가 비웃는다는 것이 별로 기분이 좋지 않겠지? 글쎄, 운이 없게도 그게 바로 운영체제의 구동방식이기 때문에 어쩔 수 없다.

당분간 사용자 주소 공간은 물리 메모리에 연속적으로 배치되어야 한다고 가정한다. 논의를 단순화하기 위해 주소 공간의 크기가 너무 크지 않다고 가정한다. 주소 공간은 *물리 메모리 크기보다 작다*. 우리는 또한 각 주소 공간의 *크기는 같다고* 가정한다. 이러한 가정이 비현실적이라고 걱정하지 말기 바란다. 논의를 진행하면서 가정들을 완화하여 실제적인 메모리의 가상화를 이끌어 낼 것이다.

18.2 사례

주소 변환 구현을 위해 어떤 것이 필요한지, 왜 그런 기법이 필요한지 알기 해 간단한 예를 살펴보자. 주소 공간이 그림 18.1과 같은 프로세스가 있다고 가정하자. 여기서 우리가 검토하려고 하는 코드는 메모리에서 값을 탑재하고, 3을 증가시키고, 다시 메모리에 저장하는 짧은 코드이다. 이 코드의 C 언어 표현은 다음과 같다고 예상할 수 있다.

```
void func() {
     int x = 3000;
     x = x + 3; // 우리가 관심있는 코드
```

컴파일러는 이 코드를 어셈블리 코드로 변환하고 그 결과는 x86 어셈블리로 다음과 같을 것이다. Linux 상에서는 `objdump` 도구 또는 Mac OS X 상에서는 `otool`을 사용하여 디스어셈블한다.

```
128: movl 0x0(\%ebx), \%eax ; 0+ebx를 eax에 저장
```

팁: 개입은 강력하다

개입은 컴퓨터 시스템에 큰 효과를 주기 위해 종종 사용되는 일반적이고 강력한 기술이다. 메모리를 가상화할 때 하드웨어는 모든 메모리 접근에 개입하여 프로세스에 의해 발생하는 가상 주소를 원하는 정보가 실제로 저장되어 있는 물리 주소로 변환할 것이다. 개입이라는 일반적인 기술은 훨씬 더 광범위하게 적용 가능하다. 사실 거의 모든 잘 정의된 인터페이스는 새 기능을 추가하기 위해 또는 시스템의 다른 측면을 개선하기 위해 개입을 사용할 수 있다. 이러한 접근 방식의 장점 중 하나는 **투명성**이다. 개입은 종종 인터페이스 사용자인 클라이언트를 변경하지 않고 이루어지고 따라서 클라이언트는 수정할 필요가 없다.

```
132: addl \$0x03, \%eax     ; eax레지스터에 3을 더한다
135: movl \%eax, 0x0(\%ebx) ; eax를 메모리에 다시 저장
```

이 코드는 비교적 이해하기 쉽다. **x**의 주소는 레지스터 **ebx**에 저장되어 있다고 가정하고 이 주소에 저장되어 있는 값을 `movl` 명령어(`longword`를 이동하는 명령어)를 사용하여 범용 레지스터 **eax**에 넣는다. 다음 명령은 **eax**에 3을 더하고, 마지막 명령은 **eax**의 값을 같은 위치의 메모리에 저장한다.

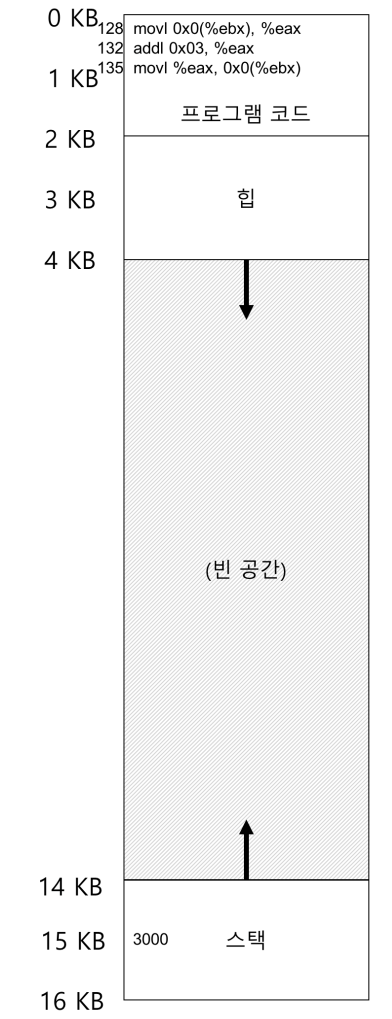
그림 18.1에서 코드와 데이터가 프로세스 주소 공간에 어떻게 배치되어 있는지 볼 수 있다. 세 개 명령어의 코드는 주소 128에 위치하고(상단 코드 섹션에), 변수 **x**의 값은 주소 15 KB(아래 쪽 스택에)에 위치한다. 그림에서 **x**의 초기 값은 3000이다(스택 영역 참조).

이 명령어가 실행되면 프로세스의 관점에서 다음과 같은 메모리 접근이 일어난다.

- 주소 128의 명령어를 반입
- 이 명령어 실행(주소 15 KB에서 탑재)
- 주소 132의 명령어를 반입
- 이 명령어 실행(메모리 참조 없음)
- 주소 135의 명령어를 반입
- 이 명령어 실행(15 KB에 저장)

프로그램 관점에서 **주소 공간**은 주소 0부터 시작하여 최대 16 KB까지이다. 프로그램이 생성하는 모든 메모리 참조는 이 범위 내에 있어야 한다. 메모리 가상화를 위해 운영체제는 프로세스를 물리 메모리 주소 0이 아닌 다른 곳에 위치시키고 싶다. 어떻게 하면 프로세스 모르게 메모리를 다른 위치에 **재배치** 하느냐가 우리가 해결해야 할 문제이다. 프로세스 주소 공간이 실제로는 다른 물리 주소에 배치되어 있을 때, 주소 0번지부터 시작하는 가상 주소 공간의 환상을 어떻게 제공할 수 있을까?

이 프로세스의 주소 공간이 메모리에 배치되었을 때 가능한 물리 메모리 배치의 한 예가 그림 18.2에 나와 있다. 그림에서 물리 메모리의 첫 번째 슬롯은 운영체제 자신이



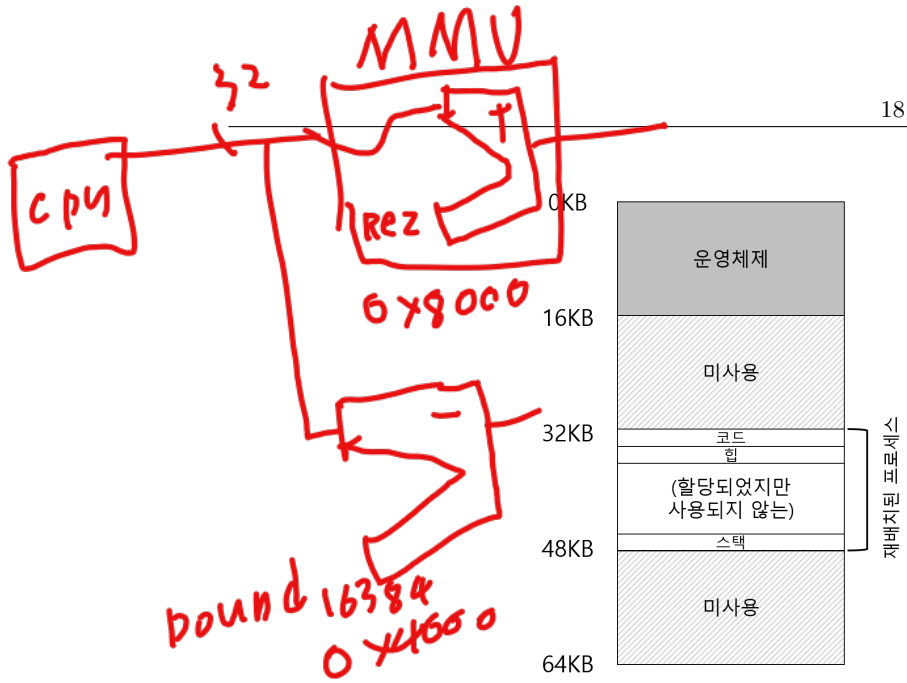
〈그림 18.1〉 **프로세스와 그 주소 공간**

사용하는 것을 볼 수 있다. 위 예의 프로세스는 물리 주소 32 KB에서 시작하는 슬롯에 재배치된 것을 볼 수 있다. 다른 두 슬롯(16 KB-32 KB와 48 KB-64 KB)은 비어 있다.

18.3 동적(하드웨어-기반) 재배치

하드웨어 기반 주소 변환을 이해하기 위하여 먼저 첫 번째 실현 사례를 설명한다. 1950년대 후반의 첫 번째 시분할 컴퓨터에서 **베이스와 바운드**(**base and bound**)라는 간단한 아이디어가 채택되었다. 이 기술은 또한 **동적 재배치**(**dynamic relocation**)라고 한다. 우리는 두 용어를 혼용한다 [SS74].

각 CPU마다 2개의 하드웨어 레지스터가 필요하다. 하나는 **베이스**(**base**) 레지스터라고 불리고, 다른 하나는 **바운드**(**bound**) 레지스터 혹은 **한계**(**limit**) 레지스터라고



〈그림 18.2〉 **하나의 재배치된 프로세스를 가진 물리 메모리**

불린다. 이 베이스와 바운드 쌍은 우리가 원하는 위치에 주소 공간을 배치할 수 있게 한다. 배치와 동시에 프로세스가 오직 자신의 주소 공간에만 접근한다는 것을 보장한다.

이 설정에서 각 프로그램은 주소 0에 탑재되는 것처럼 작성되고 컴파일된다. 프로그램 시작 시, 운영체제가 프로그램이 탑재될 물리 메모리 위치를 결정하고 베이스 레지스터를 그 주소로 지정한다. 위의 예에서 운영체제는 프로세스를 물리 주소 32 KB에 저장하기로 결정하고 베이스 레지스터를 이 값으로 설정한다.

프로세스가 실행되면서 재미있는 일이 발생한다. 프로세스에 의해 생성되는 모든 주소가 다음과 같은 방법으로 프로세서에 의해 **변환**된다.

```
physical address = virtual address + base
```

프로세스가 생성하는 메모리 참조는 **가상 주소**이다. 하드웨어는 베이스 레지스터의 내용을 이 주소에 더하여 **물리 주소**를 생성한다.

이를 이해하기 위해 명령어가 실행될 때 무슨 일이 벌어지는지 추적해 보자. 우리의 예전 코드 중 하나를 살펴보자.

```
128: movl 0x0(%EBX), % eax
```

프로그램 카운터(PC)는 128로 설정된다. 하드웨어가 이 명령어를 반입할 때, 먼저 PC 값을 베이스 레지스터의 값 32 KB(32768)에 더해 32896의 물리 주소를 얻는다. 그런 후 하드웨어는 해당 물리 주소에서 명령어를 가져온다. 그리고 프로세서는 명령어의 실행을 시작한다. 얼마 후 프로세스는 가상 주소 15 KB의 값을 탑재하라는 명령어를 내린다. 이 주소를 프로세서가 받아 다시 베이스 레지스터(32 KB)를 더하고 물리 주소 47 KB에서 원하는 내용을 탑재한다.

가상 주소에서 물리 주소로의 변환이 **주소 변환**이라고 부르는 바로 그 기술이다. 하드웨어는 프로세스가 참조하는 가상 주소를 받아들여 데이터가 실제로 존재하는 물리 주소로

여담: 소프트웨어-기반 재배치

초창기, 하드웨어 지원이 제공되기 전, 일부 시스템은 소프트웨어만으로 엉성하게 재배치를 수행하였다. 이 기술은 **정적 재배치(static relocation)**라고 불리고, **로더(loader)**라 불리는 소프트웨어가 실행하고자 하는 실행 파일의 모든 주소를 원하는 물리 메모리 오프셋으로 변경한다.

예를 들어, 명령어가 주소 1000번지로부터 레지스터로 탑재하는 경우(예를 들어, `movl 1000, %eax`) 그리고 프로그램 주소 공간이 주소 3000부터 탑재되었다면(프로그램이 생각하는 주소 0이 아닌), 로더는 명령어의 모든 주소를 3000씩 이동한 주소로 다시 작성한다(예를 들어, `movl 4000, %eax`). 이런 식으로 간단하게 프로세스 주소 공간의 정적 재배치가 이루어진다.

정적 재배치는 문제가 많다. 첫 번째 가장 중요한 문제는 보호 기능이 없다는 것이다. 잘못된 주소를 생성하여 다른 프로세스나 운영체제의 메모리를 불법적으로 접근할 수 있다. 제대로 보호하기 위해서는 하드웨어 지원이 필요하다 [Wah+93]. 또 다른 단점은 한 번 배치되면 추후 주소 공간을 재배치하는 것이 어렵다는 것이다 [McG65].

팁: 하드웨어 기반의 동적 재배치

동적 재배치를 구현하기 위해서는 작은 하드웨어면 큰 성공을 거둔다. 즉, 베이스 레지스터는 프로그램에 의해 생성된 가상 주소를 물리 주소로 변환하는 데 사용된다. **베이스**(또는 **한계**) 레지스터는 이 가상 주소가 주소 공간의 범위 내에 있다는 것을 보장한다. 두 레지스터는 함께 간단하고 효율적인 메모리의 가상화를 제공한다.

변환한다. 이 주소의 재배치는 실행 시에 일어나고, 프로세스가 실행을 시작한 이후에도 주소 공간을 이동할 수 있기 때문에, **동적 재배치(dynamic relocation)** [McG65]라고도 불린다.

이제 다음과 같은 의문을 가질 수 있다. 바운드(한계) 레지스터는 어디에 쓰는 거지? 베이스와 바운드 방식이라고 하지 않았나? 사실 맞다. 예측했을지 모르지만, 바운드 레지스터는 보호를 지원하기 위해 존재한다. 프로세서는 먼저 메모리 참조가 합법적인가를 확인하기 위해 가상 주소가 바운드 안에 있는지 확인한다. 앞에서 다룬 간단한 예에서, 바운드 레지스터는 항상 16 KB로 설정된다. 프로세스가 바운드보다 큰 가상 주소 또는 음수인 가상 주소를 참조하면 CPU는 예외를 발생시키고 프로세스는 종료될 것이다. 바운드의 요점은 프로세스가 생성한 모든 주소가 합법적이고 프로세스의 "범위"에 있다는 것을 확인하는 것이다.

베이스와 바운드 레지스터는 CPU 칩 상에 존재하는 하드웨어 구조임을 주의하라(CPU당 1쌍). 주소 변환에 도움을 주는 프로세서의 일부를 **메모리 관리 장치(memory management unit, MMU)**라고 부르기도 한다. 더 정교한 메모리 관리 기법을 개발할수록 MMU에 더 많은 회로를 추가하게 될 것이다.

바운드 레지스터는 두 가지 방식 중 하나로 정의될 수 있다. 한 가지 방법은 앞에서처

럼 주소 공간의 *크기*를 저장하는 방식으로 하드웨어는 가상 주소를 베이스 레지스터에 더하기 전에 먼저 바운드 레지스터와 비교한다. 두 번째 방식은 주소 공간의 마지막 *물리 주소*를 저장하는 방식으로 하드웨어는 먼저 베이스 레지스터를 더하고 그 결과가 바운드 안에 있는지 검사한다. 두 방법 모두 논리적으로는 동일하다. 논의를 간단하게 하기 위해 전자의 방식을 사용한다고 가정한다.

예제

베이스와 바운드를 이용한 주소 변환을 상세하게 이해하기 위해 하나의 예를 살펴보자. 주소 공간의 크기가 4 KB(비현실적으로 작기는 하다)인 프로세스가 물리 주소 16 KB에 탑재되어 있다고 가정하자. 주소 변환의 결과는 다음과 같다.

가상 주소		물리 주소
0	→	16 KB
1 KB	→	17 KB
3000	→	19384
4400	→	폴트(바운드 벗어남)

예에서 볼 수있는 것처럼 물리 주소를 얻기 위해서는 간단히 가상 주소에 베이스 주소를 더하기만 하면 된다 (가상 주소를 주소 공간 내에서의 오프셋을 나타낸다고 생각할 수 있음). 가상 주소가 너무 크거나 음수일 경우에 폴트를 일으키고 예외가 발생하게 된다.

18.4 하드웨어 지원: 요약

필요한 하드웨어 지원을 요약해 보자(그리고 그림 18.3을 보자). 먼저 CPU 가상화 장에서 설명했듯이 두 가지 CPU 모드가 필요하다. 운영체제는 **특권 모드**(또는 **커널 모드**)에서 실행하며 컴퓨터 전체에 대한 접근 권한을 가진다. 응용 프로그램은 **사용자 모드**에서 실행되며, 할 수 있는 일에 제한이 있다. **프로세서 상태 워드**(**processor status word**) 레지스터의 한 비트가 CPU의 현재 실행 모드를 나타낸다. 특정한 순간에 (예를 들면, 시스템 콜 또는 다른 종류의 예외나 인터럽트 발생 시) CPU는 모드를 전환한다.

하드웨어는 **베이스와 바운드 레지스터**를 자체적으로 제공한다. CPU는 **메모리 관리 장치**(**MMU**)의 일부인 추가의 레지스터 쌍을 가진다. 프로그램이 실행 중인 경우, 하드웨어는 프로그램이 생성한 가상 주소에 베이스 값을 더하여 주소를 변환한다. 하드웨어는 주소가 유효한지 검사할 수 있어야 한다. 이 검사는 바운드 레지스터와 CPU 내의 일부 회로를 사용하여 이루어진다.

하드웨어는 베이스와 바운드 레지스터 값을 변경하는 명령어를 제공해야 한다. 다른 프로세스를 실행시킬 때 운영체제가 이 명령어를 사용하여 베이스와 바운드 레지스터 값을 변경할 수 있다. 이 명령어들은 특권 명령어이다. 커널 모드(또는 **특권** 모드)에서만

여담: 자료 구조—빈 공간 리스트

운영체제는 프로세스에게 메모리를 할당할 수 있도록 사용되지 않는 메모리 공간의 리스트를 유지한다. 다양한 자료 구조가 사용될 수 있다. 가장 간단한 (여기서 가정하고 있는) 자료 구조는 **빈 공간 리스트(free list)**이다. 이 리스트는 현재 사용 중이지 않은 물리 메모리 영역의 리스트이다.

하드웨어 요구사항	노트
특권 모드	사용자 모드 프로세스가 특권 연산을 실행하는 것을 방지하기 위해 필요
베이스/바운드 레지스터	주소 변환과 범위 검사를 지원하기 위하여 CPU 당 한 쌍의 레지스터가 필요
가상 주소를 변환하고 범위 안에 있는지 검사하는 능력	주소 변환과 범위 검사를 위한 회로. 매우 간단함
베이스/바운드를 갱신하기 위한 특권 명령어	프로그램 시작 전에 운영체제가 베이스와 바운드 레지스터 값을 지정할 수 있어야 함
예외 핸들러 등록을 위한 특권 명령어	운영체제가 예외 처리 코드를 하드웨어에게 알려줄 수 있어야 함
예외 발생 기능	프로세스가 특권 명령어 실행을 시도하거나 범위를 벗어난 메모리의 접근을 시도할 때 예외를 발생시킬수 있어야 함

〈그림 18.3〉 **동적 재배치: 하드웨어 요구사항**

레지스터를 변경할 수 있다. 사용자 프로세스가 실행 중에 베이스 레지스터를 임의대로 변경하여 야기할 혼란[1]을 상상해 보라. 악몽과 같다. 즉시 이러한 어두운 생각을 머릿속에서 지워 버려라.

마지막으로, CPU는 사용자 프로그램이 바운드를 벗어난 주소로 불법적인 메모리 접근을 시도하려는 상황에서 **예외**를 발생시킬 수 있어야 한다. 이 경우 CPU는 사용자 프로그램의 실행을 중지하고 운영체제의 "바운드 벗어남" **예외 핸들러(exception handler)**가 실행되도록 조치해야 한다. 운영체제 핸들러는 어떻게 대처할지 결정할 수 있다. 이 경우에 프로세스를 종료시킬 확률이 매우 높다. 마찬가지로 사용자 프로그램이 특권이 필요한 베이스와 바운드 레지스터 값의 변경을 시도하면 CPU는 예외를 발생시키고 "사용자 모드에서의 특권 연산 발생" 핸들러를 실행시켜야 한다. CPU는 이 핸들러들의 주소를 파악하기 위한 방법을 제공해야 한다. 몇 개의 특권 명령어가 더 필요하다.

1) 피해를 입힐 게 "대규모 피해" 밖에 없나?

18.5 운영체제 이슈

동적 재배치 지원을 위해 하드웨어가 새로운 기능을 제공하는 것과 마찬가지로, 운영체제에도 새로운 이슈가 등장한다. 하드웨어 지원과 운영체제 관리가 결합되면 간단한 가상 메모리를 구현할 수 있다. 베이스와 바운드 방식의 가상 메모리 구현을 위해서 운영체제가 반드시 개입되어야 하는 중요한 세 개의 시점이 존재한다.

첫째, 프로세스가 생성될 때 운영체제는 주소 공간이 저장될 메모리 공간을 찾아 조치를 취해야 한다. 다행히도 각 주소 공간은 (a) 물리 메모리의 크기보다 작고 (b) 크기가 일정하다라는 가정하에서는 운영체제가 쉽게 처리할 수 있다. 운영체제는 물리 메모리를 슬롯의 배열로 보고 각 슬롯의 사용여부를 관리한다. 새로운 프로세스가 생성되면 운영체제는 새로운 주소 공간 할당에 필요한 영역을 찾기 위해 (흔히 **빈 공간 리스트**(**free list**)라고 불리는) 자료 구조를 검색해야 한다. 검색을 통해 선택된 공간을 사용 중이라고 표시한다. 가변 크기 주소 공간의 경우, 더 복잡해질 테지만 그에 대한 걱정은 미래의 장으로 넘기자.

예를 들어 살펴보자. 앞서 살펴 본 그림 18.2에서 물리 메모리의 첫 번째 슬롯을 운영체제 자신이 사용하는 것을 보았다. 운영체제는 예의 프로세스의 위치를 물리 주소 32 KB에서 시작하는 슬롯으로 재배치한 것을 볼 수 있다. 다른 두 슬롯(16 KB-32 KB와 48 KB-64 KB)은 비어 있다. 따라서 **빈 공간 리스트**는 이 두 항목으로 구성된다.

둘째, 프로세스가 종료할 때, 즉 정상적으로 종료될 때 또는 잘못된 행동을 하여 강제적으로 죽게될 때 프로세스가 사용하던 메모리를 회수하여 다른 프로세스나 운영체제가 사용할 수 있게 해야 한다. 프로세스가 종료하면, 운영체제는 종료한 프로세스의 메모리를 다시 빈 공간 리스트에 넣고 연관된 자료 구조를 모두 정리한다.

셋째, 운영체제는 문맥 교환이 일어날 때에도 몇 가지 추가 조치를 취해야 한다. CPU마다 한 쌍의 베이스-바운드 레지스터만 존재하고 각 프로그램은 다른 물리 주소에 탑재되어야 하기 때문에 실행 중인 프로그램마다 다른 값을 가진다. 운영체제는 프로세스 전환 시 베이스와 바운드 쌍을 저장하고 복원해야 한다. 운영체제가 실행 중인 프로세스를 중단시키기로 결정하면 운영체제는 메모리에 존재하는 프로세스 별 자료 구조 안에 베이스와 바운드 레지스터의 값을 저장해야 한다. 이 자료 구조는 **프로세스 구조체**(**process structure**) 또는 **프로세스 제어 블럭**(**process control block**, **PCB**)이라고 불린다. 마찬가지로 운영체제는 실행 중인 프로세스를 다시 시작할 때 또는 처음 실행시킬 때, 이 프로세스에 맞는 값으로 CPU의 베이스와 바운드 값을 설정해야 한다.

프로세스가 중단되면(즉, 실행 상태가 아닐 때에는), 운영체제가 메모리의 현 위치에서 다른 위치로 주소 공간을 비교적 쉽게 옮길 수 있다는 사실에 주목해야 한다. 프로세스의 주소 공간을 이동시키려면 운영체제는 먼저 프로세스의 실행을 중지시킨다. 그런 후 운영체제는 현재 위치에서 새 위치로 주소 공간을 복사한다. 마지막으로, 운영체제는 프로세스 구조체에 저장된 베이스 레지스터를 갱신하여 새 위치를 가리키도록 한다. 프로세스가 실행을 재개하면 새로운 베이스 레지스터가 복원되고 다시 실행을 시작하고, 명령어와 데이터가 전혀 다른 새 위치에 존재한다는 사실을 인식하지 못한다.

넷째, 위에서 언급한 것처럼 운영체제는 예외 핸들러 또는 호출될 함수를 제공해

운영체제 요구사항	노트
메모리 관리	새 프로세스의 메모리 할당에 필요;
	종료 프로세스로부터 메모리 회수;
	빈 공간 리스트를 통한 일반적인 메모리 관리
베이스/바운드 관리	문맥 교환 시 올바르게 베이스/바운드 설정
예외 처리	예외가 발생할 때 실행할 코드;
	범법 프로세스를 종료하는 것이 가능성이 높은 처리

〈그림 18.4〉 **동적 재배치 : 운영체제 책임**

야 한다. 운영체제는 부팅할 때 특권 명령어를 사용하여 이 핸들러를 설치한다. 예를 들어, 프로세스가 바운드 밖의 메모리에 접근하려는 경우 CPU는 예외를 발생시킨다. 운영체제는 이러한 예외가 발생할 때 조치를 취할 준비가 되어 있어야 한다. 운영체제의 일반적인 대응은 적대적인 양상을 보인다. 운영체제는 불법 행위를 한 프로세스를 종료시킨다. 운영체제는 자신이 실행되는 컴퓨터를 보호해야 한다. 허가 안 된 메모리에 접근하거나 또는 명령어를 실행하려는 프로세스를 좋아하지 않는다. 잘가라~, 잘못 행동한 프로세스여. 너를 만나서 반가웠어.

그림 18.5는 하드웨어/OS의 상호작용을 타임라인으로 보여준다. 그림은 부팅할 때 컴퓨터를 사용 가능한 상태로 만들기 위하여 운영체제가 무엇을 하는지 보여 준다. 그리고 프로세스(Process A)가 실행을 시작할 때 무슨 일이 일어나는지 보여 준다. 메모리 변환은 운영체제의 개입 없이 하드웨어에 처리된다는 사실에 주목하라. 시간이 흐른 후에 타이머 인터럽트가 발생하고 운영체제는 Process B로 전환한다. Process B는 불법적인 주소에 잘못된 탑재를 실행한다. 그 시점에 운영체제가 개입하여 프로세스를 끝낸 후 B의 메모리를 해제하고 프로세스 테이블에서 해당 항목을 제거하여 정리한다. 그림에서 볼 수 있듯이, 우리는 아직 **제한적 직접 실행**의 기본적인 접근 방식을 따르고 있다. 대부분의 경우, 운영체제는 다만 하드웨어를 적절하게 설정하고 프로세스가 CPU에서 직접 실행할 수 있게 한다. 프로세스가 잘못된 행동을 했을 때에만 운영체제가 개입하여야 한다.

18.6 요약

이 장에서는 주소 변환이라고 알려진 가상 메모리 기법을 통해 제한적 직접 실행의 개념을 확장하였다. **주소 변환**을 사용하면 운영체제는 프로세스의 모든 메모리 접근을 제어할 수 있고, 접근이 항상 주소 공간의 범위 내에서 이루어지도록 보장할 수 있다. 이 기술의 효율성의 열쇠는 하드웨어 지원이다. 하드웨어 지원은 프로세스가 이해하는 메모리인 가상 주소를 실제 메모리 모습인 물리 주소로 변환하며 이 변환을 빠르게 수행한다. 이 모든 것은 재배치된 프로세스에게 투명한 방식으로 이루어진다.

운영체제 @ 부트 **(커널 모드)**	**하드웨어**	
트랩 테이블을 초기화		
	다음의 주소를 기억 시스템 콜 핸들러 타이머 핸들러 불법적인 메모리 접근 핸들러 불법적인 명령어 핸들러	
인터럽트 타이머 시작		
	타이머 시작. X msec 후에 인터럽트	
프로세스 테이블 초기화		
빈 공간 리스트 초기화		

운영체제 @ 실행 **(커널 모드)**	**하드웨어**	**프로그램** **(사용자 모드)**
프로세스 A를 시작하기 위해: 프로세스 테이블의 항목 할당 프로세스 메모리 할당 베이스/바운드 레지스터 설정 return-from-trap (A로)		
	A의 레지스터 복원 **사용자 모드**로 이동 A의 (시작) PC로 분기	
		프로세스 A 실행 명령어 반입
	가상 주소 변환 및 반입 실행	
		명령어 실행
	명시적 탑재/저장이라면 주소가 범위 안 인지 확인 가상 주소 변환 및 탑재/저장 실행	
		…
	타이머 인터럽트 **커널 모드**로 이동 인터럽트 핸들러로 분기	
트랩 처리 **switch()** 루틴 호출 A의 레지스터를 A의 **proc-struct**에 저장(베이스/바운드 포함) B의 레지스터를 B의 **proc-struct**에서 복원(베이스/바운드 포함) **return-from-trap** (B로)		
	B의 레지스터 복원 **사용자 모드**로 이동 B의 PC로 분기	
		프로세스 B 실행 불법적인 탑재 실행
	바운드-밖 탑재 **커널 모드**로 이동 트랩 핸들러로 분기	
트랩 처리 프로세스 B 종료시키기로 결정 B의 메모리 회수 프로세스 테이블의 B의 항목 해제		

〈그림 18.5〉 **제한된 실행 프로토콜(동적 재배치)**

베이스와 바운드(base-and-bound) 또는 동적 재배치로 알려진 가상화의 한 형태를 살펴보았다. 베이스 레지스터를 가상 주소에 더하고 생성된 주소가 바운드를 벗어나는지 검사하기 위한 간단한 하드웨어 회로만 추가하면 되기 때문에 base-and-bound 가상화는 매우 *효율적이다*. base-and-bound 가상화는 또한 *보호* 기능도 제공한다. 운영체제와 하드웨어는 협력하여 프로세스가 자신의 주소 공간 이외의 밖의 메모리는 참조를 할 수 없도록 한다. 보호는 운영체제의 가장 중요한 목표 중의 하나이다. 보호 없이는 운영체제가 컴퓨터를 통제할 수 없다. 프로세스가 자유롭게 메모리를 덮어쓸 수 있다면, 그들은 트랩 테이블을 덮어써 시스템을 장악하는 등의 위험한 일을 쉽게 할 수 있다.

동적 재배치는 비효율적이다. 예를 들어, 그림 18.2에서 볼 수 있듯이, 재배치된 프로세스는 32 KB에서 48 KB까지의 물리 메모리를 사용한다. 그러나 프로세스 스택과 힙이 아주 크지 않기 때문에, 둘 사이의 공간이 단순히 낭비되고 있다. 할당된 영역의 내부 공간이 사용되지 않기 때문에, 즉 단편화가 발생되어 *낭비된다*. 이런 유형의 낭비를 **내부 단편화**(**internal fragmentation**)라고 한다. 현재 접근 방식에서 비록 더 많은 프로세스를 탑재할 수 있는 충분한 물리 메모리가 있더라도, 고정 크기의 슬롯에 주소 공간을 배치해야 하기 때문에 내부 단편화가 발생한다[2]. 물리 메모리의 이용률을 높이고 내부 단편화를 방지하기 위해 더 정교한 기법이 필요하다. 첫 번째 시도는 base-and-bound를 일반화하는 것이다. 이러한 일반화된 베이스-바운드 기법을 **세그멘테이션**(**segmentation**)이라고 부르고, 다음 장에서 논의할 것이다.

2) 다른 해결책은 고정 크기의 스택을 코드 바로 아래에 배치하고 힙을 그 아래에 배치하여 힙이 아래 쪽으로 확장하게 하는 것이다. 그러나 이것은 재귀 호출과 함수 호출의 중첩이 긴 경우 그 구현을 어렵게 만들기 때문에 융통성에 제약을 주고 따라서 이는 피하고 싶다.

참고 문헌

[McG65] **"On Dynamic Program Relocation"**
W.C. McGee
IBM Systems Journal Volume 4, Number 3, 1965, pages 184-199
동적 재배치에 관한 초기 연구뿐 아니라 일부 정적 재배치의 기본에 대해 잘 요약한 논문이다.

[Pil90] **"Relocating loader for MS-DOS .EXE executable files"**
Kenneth D.A. Pillay
Microprocessors & Microsystems archive Volume 14, Issue 7 (September 1990)
MS-DOS를 위한 재배치 로더의 예. 최초는 아니지만 그러한 시스템의 동작에 관한 상대적으로 현대적인 예를 설명한다.

[SS74] **"The Protection of Information in Computer Systems"**
J. Saltzer and M. Schroeder
CACM, July 1974
*논문에서 "베이스와 바운드 레지스터의 개념과 하드웨어-해석 설명자는 분명히 독립적으로 1957년과 1959년에 다양한 목표를 가진 세 개의 프로젝트에서 선보였다. MIT의 McCarthy가 시분할 시스템을 구현하기 위한 메모리 보호 시스템의 일부로 베이스-바운드를 제안하였다. IBM은 Stretch (7030) 컴퓨터 시스템의 신뢰성 있는 다중프로그래밍을 허용하기 위한 기법으로서 독립적으로 베이스-바운드 레지스터를 개발하였다. Burroughs에서는 R. Barton이 하드웨어-해석 설명자가 B5000 컴퓨터 시스템의 고수준 언어의 명칭 범위 규칙을 직접적으로 지원한다고 제안하였다." 우리는 이 인용을 Mark Smotherman의 멋진 역사 페이지 [**S04**]에서 발견하였다. 더 자세한 사항은 그 페이지를 참조하도록.*

[Wah+93] **"Efficient Software-based Fault Isolation"**
Robert Wahbe, Steven Lucco, Thomas E. Anderson, and Susan L. Graham
SOSP '93
프로그램에서 메모리 참조를 범위 내에 두기 위하여 하드웨어의 도움 없이 컴파일러의 지원을 사용할 수 있는 방법에 관한 굉장한 논문. 이 논문은 메모리 참조의 고립을 위한 소프트웨어 기술에 관한 새로운 관심을 불러 일으켰다.

숙제

프로그램 `relocation.py`는 베이스와 바운드 레지스터를 가진 시스템에서 주소 변환이 어떻게 일어나는지 볼 수 있게 한다. 자세한 내용은 README를 참조하시오.

문제

1. 시드 1, 2 및 3을 가지고 실행하고 프로세스에 의해 생성된 각 가상 주소가 범위 내에 있는지 계산하라. 바운드 안에 있다면 주소 변환을 수행하라.
2. 다음과 같은 플래그를 주고 실행하라: `-s 0 -n 10`. 생성된 모든 가상 주소가 범위 안에 있는 것을 보장하기 위해서는 -l을 어떤 값으로 설정해야 하는가?
3. 다음과 같은 플래그를 주고 실행하라: `-s 1 -n 10 -l 100`. 주소 공간 전체가 여전히 물리 메모리에 들어가려면 설정할 수 있는 바운드 레지스터의 최댓값은 얼마인가?
4. 더 큰 주소 공간(`-a`)과 물리 메모리(`-p`)를 설정하여 위 문제와 동일하게 실행시켜 보아라.
5. 바운드 레지스터의 값이 변함에 따라 임의로 생성된 가상 주소 중 유효한 주소의 비율은 얼마인가? 다른 랜덤 시드를 가지고 실행한 결과를 그래프로 나타내시오. 값의 범위는 0부터 주소 공간의 최대 크기로 한다.

19

세그멘테이션

지금까지 프로세스 주소 공간 전체를 메모리에 탑재하는 것을 가정해 왔다. 베이스와 바운드 레지스터를 사용하면 운영체제는 프로세스를 물리 메모리의 다른 부분으로 쉽게 재배치할 수 있다. 이러한 형태의 주소 공간에 대해 재미있는 사실을 발견했을 것이다. 스택과 힙 사이에 사용되지 않는 큰 공간이 존재한다.

그림 19.1에서 볼 수 있듯이 스택과 힙 사이의 공간은 사용되지 않더라도 주소 공간을 물리 메모리에 재배치할 때 물리 메모리를 차지한다. 베이스와 바운드 레지스터 방식은 메모리 낭비가 심하다. 또한, 주소 공간이 물리 메모리보다 큰 경우 실행이 매우 어렵다. 이런 측면에서 볼 때 베이스와 바운드 방식은 유연성이 없다.

핵심 질문: 대용량 주소 공간을 어떻게 지원하는가

스택과 힙 사이에 잠재적으로 큰 빈 영역이 존재하는 주소 공간을 어떻게 지원하는가? 우리의 예에서 사용하는 작은 주소 공간에서는 낭비는 그렇게 심각하지 않다. 그러나 크기가 4 GB인 32비트 주소 공간을 상상해 보라; 통상 프로그램은 단지 수 메가바이트만 사용함에도 불구하고 주소 공간 전체가 메모리에 탑재되어야 한다.

19.1 세그멘테이션: 베이스/바운드(base/bound)의 일반화

이 문제를 해결하기 위한 아이디어가 탄생했으니, 바로 **세그멘테이션**(**segmentat-ion**)이다. 상당히 오래된 아이디어로서 적어도 1960년대 초까지 거슬러 올라간다 [Hol61; Gre62]. 아이디어는 간단하다. MMU 안에 오직 하나의 베이스와 바운드 쌍만 존재하는 것이 아니라 주소 공간의 논리적인 **세그멘트**(**segment**) 마다 베이스와 바운드 쌍이 존재한다. 세그멘트는 특정 길이를 가지는 연속적인 주소 공간이다. 우리가 기준으로 삼은 주소 공간에는 코드, 스택, 및 힙의 세 종류의 세그멘트가 있다. 세그멘테이션을 사용하면 운영체제는 각 세그멘트를 물리 메모리의 각기 다른 위치에 배치할 수 있고, 사용되지 않는 가상 주소 공간이 물리 메모리를 차지하는 것을 방지할 수 있다.

예를 들어 보자. 그림 19.1의 주소 공간을 물리 메모리에 배치하려고 한다. 각 세그멘트의 베이스와 바운드 쌍을 이용하여 세그멘트들을 독립적으로 물리 메모리에 배치할

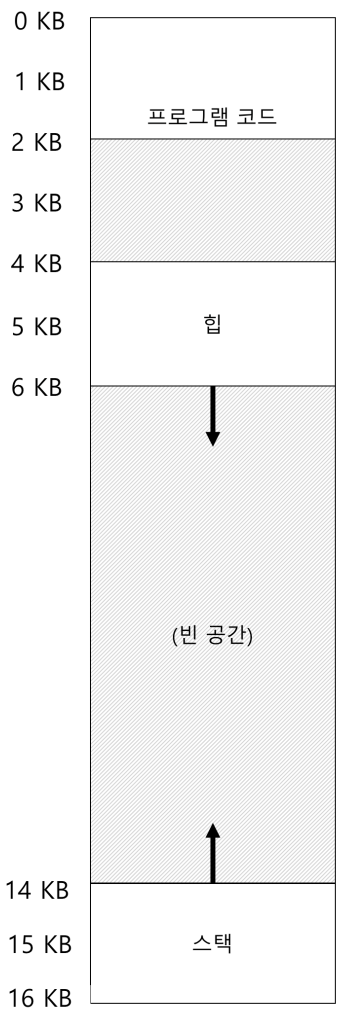
수 있다. 그림 19.2를 보자. 64 KB의 물리 메모리에 3개의 세그멘트와 운영체제용으로 예약된 16 KB 영역이 존재한다.

그림에서 볼 수 있듯이, 사용 중인 메모리에만 물리 공간이 할당된다. 사용되지 않은 영역이 많은 대형 주소 공간을(우리는 때때로 **드문드문 사용되는 주소 공간**(**sparse address space**)이라고 부름) 수용할 수 있다.

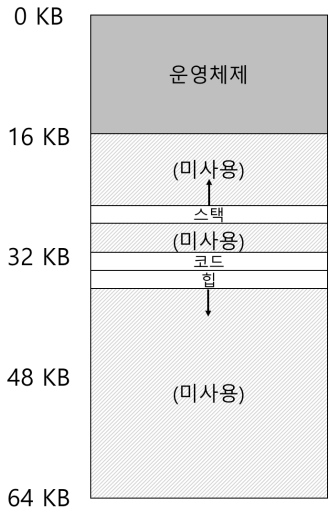
세그멘트 지원을 위한 MMU 하드웨어 구조는 예상한 것과 같다. 이 예제의 경우 3쌍의 베이스와 바운드 레지스터 집합이 필요하다. 그림 19.3은 앞의 예에 해당하는 각 레지스터의 값을 보여준다. 각 바운드 레지스터는 세그멘트의 크기를 저장한다.

그림에서 코드 세그멘트가 물리 주소 32 KB에 배치되고 크기는 2 KB이며, 힙 세그멘트가 34 KB에 배치되고 역시 크기는 2 KB라는 것을 알 수 있다.

그림 19.1의 주소 공간을 사용하여 주소 변환을 해 보자. 가상 주소 100 번지를



〈그림 19.1〉 **주소 공간 (복습)**



〈그림 19.2〉 **물리 메모리에 세그멘트 배치하기**

세그멘트	베이스	크기
코드	32 KB	2 KB
힙	34 KB	2 KB
스택	28 KB	2 KB

〈그림 19.3〉 **세그멘트 레지스터의 값**

여담: 세그멘트 폴트

용어 Segment Fault는 세그멘트 사용 시스템에서 불법적인 주소 접근 시 발생한다. 재미있게도 이 용어는 세그멘트에 대한 지원이 전혀 없는 컴퓨터에서도 여전히 사용된다는 사실이다. 이 경우에는 코드의 오류 원인을 알 수도 없다.

참조한다고 가정하자. 가상 주소 100번지는 코드 세그멘트에 속한다. 참조가 일어나면 하드웨어는 베이스 값에 이 세그멘트의 *오프셋*(이 경우, 100)을 더해 물리 주소는 100+32 KB 또는 32868이 된다. 그 후, 주소가 범위 내에 있는지 검사하고(100은 2 KB보다 작다), 범위 내에 있을 경우, 물리 메모리 주소 32868를 읽는다.

가상 주소 4200의 힙을 살펴보자(다시 그림 19.1을 보시오). 주의할 점이 있다. 가상 주소 4200을 힙의 베이스(34 KB)에 더하면 물리 주소 39016을 얻지만 이 주소는 올바른 물리 주소가 아니다. 먼저 힙 안에서의 *오프셋*, 즉 주소가 참조하는 바이트가 *이 세그멘트* 시작으로부터 몇 번째 바이트인지를 얻어야 한다. 힙은 가상 주소 4 KB(4096)에서 시작하기 때문에 오프셋 4200은 실제로는 4200 빼기 4096, 즉 104가 된다. 이 오프셋(104)을 베이스 레지스터의 물리 주소(34 KB)에 더해 원하는 결과 34920을 얻게

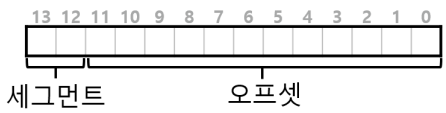
된다.

만일 힙의 마지막을 벗어난 7KB와 같은 잘못된 주소를 접근하려고 한다면 어떠한 일이 벌어질까? 하드웨어가 그 주소가 범위를 벗어났다는 것을 감지하고 운영체제에 트랩을 발생시킨다. 운영체제는 아마도 문제의 프로세스를 종료시킬 가능성이 크다. 이제 모든 C 프로그래머가 두려워하는 유명한 용어의 기원을 알게 되었다: **세그멘트 위반**(**segment violation**) 또는 **세그멘트 폴트**(**segment fault**).

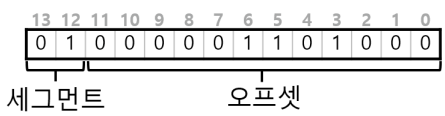
19.2 세그멘트 종류의 파악

하드웨어는 변환을 위해 세그멘트 레지스터를 사용한다. 하드웨어는 가상 주소가 어느 세그멘트를 참조하는지 그리고 그 세그멘트 안에서 오프셋은 얼마인지를 어떻게 알 수 있는가?

한 가지 일반적인 접근법, 가상 주소의 최상위 몇 비트를 기준으로 주소 공간을 여러 세그멘트로 나누는 것이다. 이 기법은 VAX/VMS 시스템에서 사용되었다 [LL82]. 위의 예에서는 3개의 세그멘트가 있다. 주소 공간을 세그멘트로 나누기 위해서는 2비트가 필요하다. 세그멘트를 표시하기 위해 가상 주소 14비트 중 최상위 2비트를 사용하는 경우 가상 주소의 모양은 다음과 같이 보일 것이다.



우리의 예에서 최상위 2비트가 00이면, 하드웨어는 가상 주소가 코드 세그멘트를 가리킨다는 것을 알고, 따라서 코드 세그멘트의 베이스와 바운드 쌍을 사용하여 주소를 정확한 물리 메모리에 재배치한다. 최상위 2비트가 01이면 하드웨어는 주소가 힙 세그멘트라는 것을 인지하여 힙의 베이스와 바운드를 사용한다. 이해를 돕기 위해서 앞에서 사용한 힙에 해당하는 가상 주소(4200)를 변환해 보자. 가상 주소 4200에 해당하는 이진 형식은 다음과 같다.



그림에서 볼 수 있듯이 최상위 2비트(01)는 하드웨어에게 참조하는 세그멘트의 종류를 알려준다. 하위 12비트는 세그멘트 내의 오프셋이다. `0000 0110 1000` 또는 16진수 `0x068` 또는 10진수로 `104`. 하드웨어는 세그멘트 레지스터를 파악하는 데 처음 2비트를 이용하고 세그멘트 오프셋으로 다음 12비트를 취한다. 오프셋에 베이스 레지스터 값을 더하여 하드웨어는 최종 물리 주소를 계산한다. 오프셋은 바운드 검사도 쉽게 만든다. 오프셋이 바운드보다 작은지 여부만 검사하면 된다. 그렇지 않으면 주소가

잘못된 것이다. 베이스와 바운드 쌍을 배열 형식으로 저장할 경우(세그멘트당 하나의 항목), 원하는 물리 주소를 얻기 위하여 다음과 같은 작업을 하게 된다.

```
1 // 14 bit VA중의 상위 2 bit를 얻어옴
2 Segment = (VirtualAddress & SEG_MASK) >> SEG_SHIFT
3 // 이제 오프셋을 얻어옴
4 Offset = VirtualAddress & OFFSET_MASK
5 if (Offset >= Bounds[Segment])
6     RaiseException(PROTECTION_FAULT)
7 else
8     PhysAddr = Base[Segment] + Offset
9     Register = AccessMemory(PhysAddr)
```

우리는 현재 예를 기준으로 위 코드에서 사용된 상수 값들을 정할 수 있다. `SEG_MASK`는 `0x3000`, `SEG_SHIFT`는 12, 그리고 `OFFSET_MASK`는 0xFFF로 지정된다.

세그멘트 종류를 나타내는 데 최상위 2비트를 사용하고 주소 공간에는 세 개의 세그멘트(코드, 힙, 스택)만 존재하기 때문에 지정 가능한 세그멘트 하나는 미사용으로 남는다. 즉, 전체 주소 공간의 1/4은 사용이 불가능하다. 이 문제를 해결하기 위해 일부 시스템은 코드와 힙을 하나의 세그멘트에 저장하고 세그멘트 선택을 위해 1비트만 사용한다 [LL82].

특정 주소의 세그멘트를 하드웨어적으로 파악하는 다른 방법들이 있다. **묵시적(implicit)** 접근 방식에서는 주소가 어떻게 형성되었나를 관찰하여 세그멘트를 결정한다. 예를 들어, 주소가 프로그램 카운터에서 생성되었다면(즉, 명령어 반입) 주소는 코드 세그멘트 내에 있을 것이다. 주소가 스택 또는 베이스 포인터에 기반을 둔다면 주소는 스택 세그멘트 내에 있다. 다른 주소는 모두 힙에 있어야 한다.

19.3 스택

지금까지 주소 공간의 중요한 구성 요소인 스택을 다루지 않았다. 앞의 그림에서 스택은 물리 주소 28 KB에 배치되어 있지만 한 가지 아주 중요한 차이가 있다. 다른 세그멘트들과는 *반대 방향으로 확장된다*는 것이다. 물리 메모리 28 KB에서 시작해서 26 KB까지 차지한다, 가상 주소 16 KB에서 14 KB에 해당한다. 다른 방식의 변환이 필요하다.

첫 번째, 간단한 하드웨어가 추가로 필요하다. 베이스와 바운드 값뿐 아니라 하드웨어는 세그멘트가 어느 방향으로 확장하는지도 알아야 한다. 예를 들어, 하나의 비트를 사용하여 주소가 커지는 쪽(양의 방향)으로 확장하면 1, 작아지는 쪽(음의 방향)으로 확장하면 0으로 설정할 수 있다. 그림 19.4는 이 사실을 반영하여 하드웨어가 관리해야 하는 정보를 표현하고 있다.

하드웨어는 세그멘트가 반대 방향으로 늘어날 수 있다는 것을 알고 있기 때문에, 그런 가상 주소에 대해서는 다른 방식으로 변환한다. 이 과정을 이해하기 위하여 스택에 해당하는 가상 주소를 예로 들어 변환해 보자.

이 예에서 가상 주소 15 KB에 접근하려고 한다고 가정하자. 이 주소는 물리 주소 27 KB에 매핑되어야 한다. 이 가상 주소를 이진 형태로 바꾸면 `11 1100 0000 0000` (16진수 `0x3C00`)이 된다. 하드웨어는 상위 2비트 (11)를 사용하여 세그멘트를 지정한다. 이를 고려하면 3 KB의 오프셋이 남는다. 올바른 음수 오프셋을 얻기 위해서 3 KB

세그멘트	베이스	크기	순방향으로 증가?
코드	32 KB	2 KB	1
힙	34 KB	2 KB	1
스택	28 KB	2 KB	0

〈그림 19.4〉 **세그멘트 레지스터 (반대 방향 확장 지원 기능 포함)**

에서 세그멘트 최대 크기를 빼야 한다. 이 예에서는 세그멘트의 최대 크기가 4 KB이고 따라서 올바른 오프셋은 3 KB에서 4 KB를 뺀 -1 KB이다. 이 음수 오프셋(-1 KB)을 베이스(28 KB)에 더하면 올바른 물리 주소 27 KB를 얻게 된다. 바운드 검사는 음수 오프셋의 절댓값이 세그멘트의 크기보다 작다는 것을 확인하여 계산할 수 있다.

19.4 공유 지원

세그멘테이션 기법이 발전함에 따라 시스템 설계자들은 간단한 하드웨어 지원으로 새로운 종류의 효율성을 성취할 수 있다는 것을 깨달았다. 구체적으로, 메모리를 절약하기 위해 때로는 주소 공간들 간에 특정 메모리 세그멘트를 **공유**하는 것이 유용하다. 특히, **코드 공유**가 일반적이며, 현재 시스템에서도 광범위하게 사용 중이다.

공유를 지원하기 위해, 하드웨어에 **protection bit**의 추가가 필요하다. 세그멘트마다 protection bit를 추가하여 세그멘트를 읽거나 쓸 수 있는지 혹은 세그멘트의 코드를 실행시킬 수 있는지를 나타낸다. 코드 세그멘트를 읽기 전용으로 설정하면 주소 공간의 독립성을 유지하면서도, 여러 프로세스가 주소 공간의 일부를 공유할 수 있다. 각 프로세스는 여전히 자신의 전용 메모리를 사용하고 있다고 생각하지만 운영체제는 이 변경이 불가능하도록 설정된 메모리 영역을 비밀리에 공유시켜 그러한 환상을 유지토록 한다.

하드웨어(및 운영체제)가 유지하는 부가 정보의 예가 그림 19.5에 나와 있다. 코드 세그멘트는 읽기 및 실행으로 설정되어 같은 물리 세그멘트가 여러 가상 주소 공간에 매핑될 수 있다.

세그멘트	베이스	크기	양수 방향으로 증가?	보호
코드	32 KB	2 KB	1	읽기-실행
힙	34 KB	2 KB	1	읽기-쓰기
스택	28 KB	2 KB	0	읽기-쓰기

〈그림 19.5〉 **세그멘트 레지스터 값 (보호 정보 포함)**

protection bit를 사용하면 앞서 언급한 하드웨어 알고리즘이 수정되어야 한다. 가상 주소가 범위 내에 있는지 확인하는 것 이외에 특정 액세스가 허용되는지를 확인해야

한다. 사용자 프로세스가 읽기 전용 페이지에 쓰기를 시도하는 경우 또는 실행 불가 페이지에서 실행하려고 하면 하드웨어는 예외를 발생시켜서 운영체제가 위반 프로세스를 처리할 수 있게 해야 한다.

19.5 소단위 대 대단위 세그멘테이션

우리 예제의 대부분은 지금까지 소수의 세그멘트(즉, 코드, 스택, 힙)만을 지원하는 시스템에만 주로 초점을 맞추었다. 우리는 이 세그멘테이션을 **대단위(coarse-grained)**라고 생각할 수 있다. 주소 공간을 비교적 큰 단위의 공간으로 분할하기 때문이다. 일부 초기 시스템(예를 들어, Multics [CV65; DD68])은 주소 공간을 작은 크기의 공간으로 잘게 나누는 것이 허용되었기 때문에, **소단위(fine-grained)** 세그멘테이션이라고 부른다.

많은 수의 세그멘트를 지원하기 위해서는 여러 세그멘트의 정보를 메모리에 저장할 수 있는 세그멘트 테이블 같은 하드웨어가 필요하다. **세그멘트 테이블**을 이용하면 매우 많은 세그멘트를 손쉽게 생성하고 융통성 있게 세그멘트를 사용할 수 있다. 예를 들어, Burroughs B5000과 같은 초창기 시스템은 수천 개의 세그멘트를 지원하였고, 컴파일러가 코드나 데이터를 여러 세그멘트로 분할할 경우 운영체제와 하드웨어가 이를 지원할 수 있게 하였다 [RK68]. 당시의 생각은 소단위 세그멘트로 관리하는 것이 운영체제가 사용 중인 세그멘트와 미사용인 세그멘트를 구분하여 메인 메모리를 더 효율적으로 활용할 수 있다는 것이었다.

19.6 운영체제의 지원

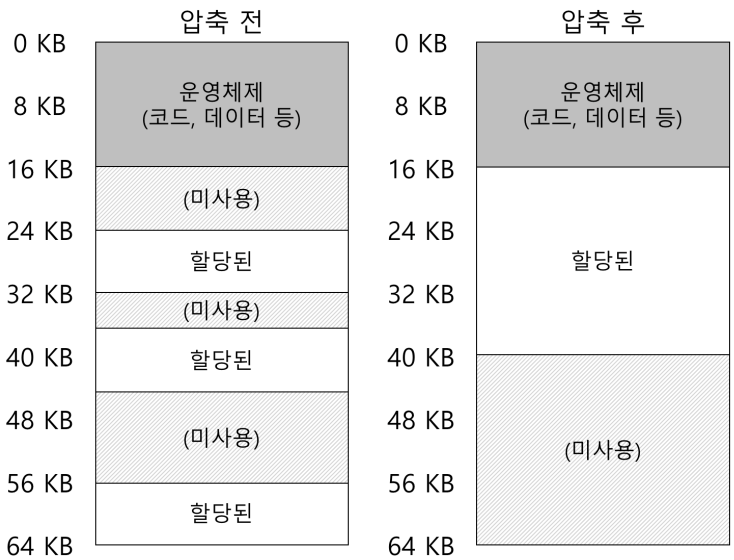
이제 세그멘트의 기본 동작은 이해하고 있어야 한다! 시스템이 각 주소 공간 구성 요소를 별도로 물리 메모리에 재배치하기 때문에 전체 주소 공간이 하나의 베이스-바운드 쌍을 가지는 간단한 방식에 비해 물리 메모리를 엄청나게 절약할 수 있다. 구체적으로는 스택과 힙 사이의 사용하지 않는 공간에 물리 메모리를 할당할 필요가 없기 때문에 같은 크기의 물리 메모리에 더 많은 주소 공간을 탑재할 수 있다.

세그멘테이션은 새로운 많은 문제를 제기한다. 해결해야 할 새로운 운영체제 문제를 우선 설명한다. 처음 문제는 오래된 문제이다. 문맥 교환 시 운영체제는 어떤 일을 해야 하는가? 아마 이미 상상하고 있으리라 믿는다. 세그멘트 레지스터의 저장과 복원이다. 각 프로세스는 자신의 가상 주소 공간을 가지며, 운영체제는 프로세스가 다시 실행하기 전에 레지스터들을 올바르게 설정해야 한다.

두 번째, 더욱 중요한 문제는 미사용 중인 물리 메모리 공간의 관리이다. 새로운 주소 공간이 생성되면 운영체제는 이 공간의 세그멘트를 위한 비어있는 물리 메모리 영역을 찾을 수 있어야 한다. 이전에 우리는 각 주소 공간의 크기가 동일하다고 가정했다. 물리 메모리는 프로세스가 탑재될 슬롯의 집합이라고 생각될 수 있었다. 지금은 프로세스가 많은 세그멘트를 가질 수 있고, 각 세그멘트는 크기가 다를 수 있다.

일반적으로 생길 수 있는 문제는 물리 메모리가 빠르게 작은 크기의 빈 공간들로 채워진다는 것이다. 이 작은 빈 공간들은 새로이 생겨나는 세그멘트에 할당하기도 힘들

거니와 기존 세그멘트를 확장하는 데에도 도움이 되지 않는다. 우리는 이 문제를 **외부 단편화(external fragmentation)**라고 부른다 [Ran69]. 그림 19.6(왼쪽)을 보시오.



〈그림 19.6〉 **압축 전과 압축 후의 메모리 상태**

이 예에서 새로운 프로세스가 생성되어 20 KB를 할당하려고 한다고 가정하자. 위 예에서 24 KB의 빈 공간이 존재하기는 하지만 하나의 연속된 공간이 아니라 세 개의 청크(chunk)로 나누어져 있다. 운영체제는 20 KB의 요청을 충족시킬 수 없다.

이 문제의 해결책 중 한 가지는 기존의 세그멘트를 정리하여 물리 메모리를 **압축(compact)**하는 것이다. 예를 들어, 운영체제는 현재 실행 중인 프로세스를 중단하고, 그들의 데이터를 하나의 연속된 공간에 복사하고, 세그멘트 레지스터가 새로운 물리 메모리 위치를 가리키게 하여, 자신이 작업할 큰 빈 공간을 확보할 수 있다. 이렇게 함으로써 운영체제는 새로운 할당 요청을 충족시킬 수 있다. 하지만 세그멘트 복사는 메모리에 부하가 큰 연산이고 일반적으로 상당량의 프로세서 시간을 사용하기 때문에 압축은 비용이 많이 든다. 압축 작업 후의 물리 메모리가 그림 19.6(오른쪽)에 나와 있다.

간단한 방법은 빈 공간 리스트를 관리하는 알고리즘을 사용하는 것이다. 빈 공간 관리 알고리즘은 할당 가능한 메모리 영역들을 리스트 형태로 유지한다. **최적 적합(best-fit)**, **최악 적합(worst-fit)**, **최초 적합(first-fit)** 및 **버디 알고리즘(buddy algorithm)**과 같은 고전적인 알고리즘을 포함하여 말 그대로 수백 개의 방식이 존재한다. 이 중에서 최적 적합은 빈 공간 리스트에서 요청된 크기와 가장 비슷한 크기의 공간을 할당한다 [Knu68]. 이러한 알고리즘에 대해 더 알고 싶다면 Wilson 등의 논문이 좋은 출발점이 될 것이다 [RWi+95]. 아니면 추후 이 책에서 기본적인 내용이 나올 때까지 기다려도 무방하다. 알고리즘이 아무리 정교하게 동작한다고 해도 외부 단편화는 여전히 존재한다. 좋은 알고리즘은 외부 단편화를 가능한 줄이는 것이 목표이다.

팁: 1000개의 해결책이 존재한다면, 최선의 해결책은 존재하지 않는다

외부 단편화를 최소화하기 위한 매우 다양한 알고리즘이 존재한다는 사실은 중요한 의미를 가진다: 외부 단편화를 해결하는 유일한 "최선"책은 존재하지 않는다. 우리는 합리적인 선택을 하고 그 해결책이 충분히 좋기를 희망할 뿐이다. 유일한 진정한 해결책은 가변 길이 할당을 허용하지 않음으로써 아예 문제가 생기지 않게 하는 것이다. 이후의 장에서 이 해결책을 살펴볼 것이다.

19.7 요약

세그멘테이션은 많은 문제를 해결하며, 메모리 가상화를 효과적으로 실현할 수 있다. 단순한 동적 재배치를 넘어 세그멘테이션은 주소 공간 상의 논리 세그멘트 사이의 큰 공간에 대한 낭비를 피함으로써 드문드문 사용되는 주소 공간(sparse address space)을 지원할 수 있다. 세그멘테이션에 필요한 산술 연산은 쉽고 하드웨어 구현에 적합하기 때문에 속도가 빠르다. 변환 오버헤드도 최소이다. 코드 공유의 장점도 부가적으로 발생한다. 코드가 별도의 세그멘트에 존재한다면 그러한 코드는 실행 중인 여러 프로그램 사이에서 공유될 수 있다.

그러나 살펴본 것처럼 세그멘트의 크기가 일정하지 않기 때문에 몇 가지 문제가 발생한다. 논한 바와 같이 첫째는 외부 단편화이다. 세그멘트는 가변 크기이기 때문에 빈 공간들의 크기 역시 모두 다르며 메모리 할당 요청을 충족시키는 것이 어려울 수 있다. 비교적 정교한 알고리즘 [RWi+95]을 사용하거나 주기적으로 메모리를 압축할 수는 있지만, 외부 단편화 문제는 가변 길이 할당의 태생적인 문제이기 때문에 회피하기 어렵다.

두 번째, 더욱 중요한 문제는 세그멘테이션이 아직 일반적인 드문드문 사용되는 주소 공간을 지원할 만큼 충분히 유연하지 못하다는 것이다. 예를 들어, 크기가 크지만 드문드문 사용되는 힙이 하나의 논리적인 세그멘트에 배정되어 있다고 할 때 이 힙에 접근하기 위해서는 힙 전체가 여전히 물리 메모리에 존재해야 한다. 다시 말해서, 주소 공간이 사용되는 모델과 이를 지원하기 위한 세그멘테이션의 설계 방법이 정확히 일치하지 않는다면, 세그멘테이션은 제대로 동작하지 않는다. 우리는 무언가 새로운 해결책을 찾아야 한다. 찾을 준비 되었나?

참고 문헌

[CV65] **"Introduction and Overview of the Multics System"**
F.J. Corbato and V.A. Vyssotsky
Fall Joint Computer Conference, 1965
Fall Joint Computer Conference에서 발표된 Multics에 관한 다섯 논문 중 하나. 그날 그 방의 보이지 않는 관찰자였으면!

[DD68] **"Virtual Memory, Processes, and Sharing in Multics"**
Robert C. Daley and Jack B. Dennis
Communications of the ACM, Volume 11, Issue 5, May 1968
Multics에서 동적 링킹을 수행하는 방법에 관한 초기 논문. 동적 링킹은 시대를 앞선 아이디어였다. 동적 링킹은 20년이 지난 X-windows 라이브러리가 요구하면서 다시 등장하였다. 어떤 이는 이 커다란 X11 라이브러리가 UNIX의 초기 버전에서 동적 링킹 지원을 제거한 것에 대한 MIT의 복수다라고 말한다.

[Gre62] **"Fact Segmentation"**
M.N. Greenfield
Proceedings of the SJCC, Volume 21, May 1962
세그멘테이션에 관한 또 다른 초기 논문. 너무 일찍 출간되었기 때문에 다른 연구에 대한 참고 문헌이 없다.

[Hol61] **"Program Organization and Record Keeping for Dynamic Storage"**
A.W. Holt
Communications of the ACM, Volume 4, Issue 10, October 1961
세그멘테이션과 그의 사용에 관한 믿을 수 없을 정도로 초기 논문이면서 읽기 어려운 논문.

[09] **"Intel 64 and IA-32 Architectures Software Developer's Manuals"**
Intel, 2009
URL: `http://www.intel.com/products/processor/manuals`
세그멘테이션에 관한 부분을 읽으려고 시도해 보라 (볼륨 3a의 3장). 머리를 손상시킬 것이다, 적어도 아주 조금이라도.

[Knu68] **"The Art of Computer Programming: Volume I"**
Donald Knuth
Addison-Wesley, 1968
Knuth는 그의 초창기 책 "The Art of Computer Programming"으로 뿐 아니라 그의 조판 시스템 TeX로 유명하다. TeX는 오늘까지 전문가에 의해 사용되는 강력한 조판 도구이다. 그리고 사실 바로 이 책을 조판한 도구이다. 그의 알고리즘에 관한 시리즈는 현재 컴퓨팅 시스템의 근간을 이루는 많은 알고리즘의 훌륭한 초창기 참고 문헌이다.

[Lam83] **"Hints for Computer Systems Design"**
Butler Lampson
ACM Operating Systems Review, 15:5, October 1983
컴퓨터 시스템의 설계에 관한 Lampson의 그 유명한 조언. 언젠가 한 번은 읽어야 할 논문이자 아마 여러 번 읽어야 할 논문일 것이다.

[LL82] **"Virtual Memory Management in the VAX/VMS Operating System"**
Henry M. Levy and Peter H. Lipman
IEEE Computer, Volume 15, Number 3 (March 1982)
설계에 매우 많은 상식이 담긴 고전적인 메모리 관리 시스템. 나중에 자세히 공부할 것이다.

[RK68] **"Dynamic Storage Allocation Systems"**
B. Randell and C.J. Kuehner
Communications of the ACM Volume 11(5), pages 297-306, May 1968
페이징과 세그멘테이션의 차이에 관한 좋은 개괄. 다양한 컴퓨터에 관한 약간의 역사적 논의도 포함하고 있다.

[Ran69] **"A note on storage fragmentation and program segmentation"**
Brian Randell
Communications of the ACM Volume 12(7), pages 365-372, July 1969
단편화를 논의한 초창기 논문 중의 하나.

[RWi+95] **"Dynamic Storage Allocation: A Survey and Critical Review"**
Paul R. Wilson, Mark S. Johnstone, Michael Neely, and David Boles
In International Workshop on Memory Management Scotland, United Kingdom, September 1995
메모리 할당기에 관한 훌륭한 요약 논문.

[Smo04] **"System Call Support"**
Mark Smotherman
URL: `http://people.cs.clemson.edu/%CB%9Cmark/syscall.html`
시스템 콜 지원에 관한 깔끔한 역사. Smotherman은 인터럽트와 컴퓨팅 역사의 다른 재미있는 측면과 같은 주제에 관한 초기 역사를 수집하였다. 자세한 사항에 관해서는 웹 페이지를 참조하라.

숙제

이 프로그램은 세그멘테이션 시스템에서 주소 변환이 어떤 식으로 일어나는지 보여준다. 자세한 내용은 README를 참조하시오.

문제

1. 우선 몇 주소를 변환하기 위해 작은 주소 공간을 사용하여 보자. 여기에 여러 가지 랜덤 시드와 함께 간단한 매개변수의 집합이 있다. 이때 주소를 변환할 수 있을까?

```
segmentation.py -a 128 -p 512 -b 0 -l 20 -B 512 -L 20 -s 0
segmentation.py -a 128 -p 512 -b 0 -l 20 -B 512 -L 20 -s 1
segmentation.py -a 128 -p 512 -b 0 -l 20 -B 512 -L 20 -s 2
```

2. 이제 우리가 구축한 이 작은 주소 공간을 이해하고 있는지 알아보자(위의 질문에서 사용한 매개변수를 사용하여). 세그멘트 0의 합법적인 가상 주소의 최댓값은 무엇인가? 세그멘트 1의 합법적인 가상 주소의 최저값은 무엇인가? 이 전체 주소 공간의 잘못된 주소 중 가장 큰 주소와 가장 작은 주소는 무엇인가? 마지막으로, 플래그 `-A`와 함께 실행한 `segmentation.py`의 정확성을 어떻게 확인할 수 있을까?

3. 16바이트 주소 공간과 128바이트의 물리 메모리가 있다고 하자. 시뮬레이터가 지정된 주소 스트림에 대해 다음과 같은 변환 결과를 생성하기 위해서는 베이스와 바운드를 어떤 값으로 설정해야 하는가: 유효, 유효, 위반, ..., 위반, 유효, 유효? 다음과 같은 매개변수를 가정한다.

```
segmentation.py -a 16 -p 128
        -A 0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15
        --b0 ? --l0 ? --b1 ? --l1 ?
```

4. 무작위로 생성된 가상 주소 중 약 90%가 유효한 시스템에서(즉, 세그멘테이션 위반이 아닌) 문제를 만들고 싶다고 가정하자. 그런 결과를 내려면 시뮬레이터를 어떻게 설정해야 하는가? 어떤 매개변수가 중요한가?

5. 모든 가상 주소가 유효하지 않도록 시뮬레이터를 실행할 수 있는가? 어떻게?

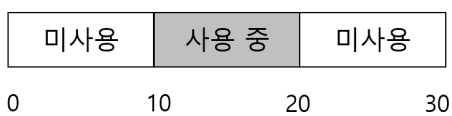
20

빈 공간 관리

이 장에서는 메모리 가상화 논의에서 약간 우회하여 모든 메모리 관리 시스템의 근본적인 측면을 논의한다. 여기서 메모리 관리 시스템이란 프로세스 힙의 페이지를 관리하는 `malloc` 라이브러리일 수도 있고 혹은 프로세스의 주소 공간의 일부분을 관리하는 운영체제 자체일 수도 있다. 구체적으로, **빈 공간 관리**에 관련된 문제를 논의할 것이다.

문제를 구체화해 보자. **페이징** 개념을 논의할 때 보겠지만 빈 공간 관리는 쉬울 수 있다. 관리하고 있는 공간이 고정 크기의 단위로 나누어져 있는 경우 특히 쉽다. 그런 경우 그냥 이 고정 크기 단위의 리스트를 유지하면 된다. 클라이언트가 그 중 하나를 요청하면 첫 번째 항목을 반환하면 된다.

빈 공간 관리가 더 어렵고 흥미로운 경우는 관리하는 공간이 가변-크기 빈 공간들의 집합으로 구성되어 있는 경우이다. 이 경우는 `malloc()`과 `free()`에서처럼 사용자-수준 메모리-할당 라이브러리에서, 그리고 **세그멘테이션**으로 물리 메모리를 관리하는 운영체제에서 발생한다. 어느 경우에도 **외부 단편화**가 존재한다. 빈 공간은 다양한 크기의 작은 조각으로 분할되어 결국 단편화된다. 빈 공간들의 전체 크기가 요청된 것보다 크더라도 하나의 연속된 영역이 존재하지 않으면 요청은 실패할 수 있다.



그림은 이러한 문제의 예를 보여 준다. 이 경우 빈 공간의 전체 크기는 20바이트이다. 불행히도, 10바이트 짜리 두 조각으로 나누어져 있다. 그 결과, 20바이트의 빈 공간이 있지만 15바이트의 요청은 실패한다. 이것이 이 장에서 해결하고자 하는 문제이다.

핵심 질문: 빈 공간을 어떻게 관리하는가

가변 크기의 요구를 충족시켜야 할 때, 빈 공간은 어떻게 관리되어야 하는가? 단편화를 최소화하기 위해 어떤 전략을 사용할 수 있는가? 여러 대안들의 시간과 공간의 오버헤드는 어떻게 되는가?

20.1 가정

이 논의의 대부분은 사용자 수준 메모리 할당 라이브러리에 존재하는 메모리 할당기의 발전 역사에 초점을 맞출 것이다. 이 내용들은 Wilson의 저서에 잘 정리되어 있다 [RWi+95]. 더 자세한 사항을 알려면 원본 문서를 읽어 보기 바란다[1].

`malloc()`과 `free()`에서 제공하는 것과 같은 기본 인터페이스를 가정한다. 구체적으로 `void *malloc (size_t size)`는 응용 프로그램이 요청한 바이트 수를 나타내는 변수 `size`를 받아들인다. 이 함수는 요청된 크기와 같거나 큰 영역을 가리키는, 타입이 없는 또는 C 언어의 용어로 `void` 포인터를 반환한다. 대응되는 루틴 `void free(void *ptr)`는 포인터를 인자로 전달받고 해당 영역을 해제한다. 인터페이스의 의미에 주의하라. 공간을 해제할 때 사용자는 라이브러리에게 크기 정보를 전달하지 않는다. 라이브러리는 포인터만으로 해제하고자 하는 메모리 영역의 크기를 파악해야 한다. 크기를 알아내는 방법에 대해 나중에 논의할 것이다.

이 라이브러리가 관리하는 공간은 역사적으로 힙(`heap`)으로 불리며, 힙의 **빈 공간**을 관리하는 데는 일반적인 링크드리스트가 사용된다. 이 자료 구조는 영역 내의 모든 빈 청크에 대한 주소를 갖고 있다. 물론, 이 자료 구조는 반드시 리스트일 필요는 없고 빈 공간들을 표현할 수 있는 자료 구조면 충분하다.

우리는 위에서 언급한 것처럼 **외부 단편화** 방지에 특히 중점을 두겠다. 물론, **내부 단편화** 문제도 있을 수 있다. 할당기가 요청한 크기보다 더 큰 메모리 청크를 할당할 경우, 요청되지 않은 사용되지 않는 공간에 대해서는 할당 청크의 내부에서 낭비가 일어났기 때문에 내부 단편화라고 간주된다. 할당 공간 낭비의 또 다른 예라고 할 수 있다. 논의를 단순화하기 위하여, 그리고 두 유형 중 더 흥미롭기 때문에 외부 단편화에 초점을 맞출 것이다.

우리는 또한 클라이언트에게 할당된 메모리는 다른 위치로 재배치될 수 없다고 가정한다. 예를 들어, 프로그램이 `malloc()`을 호출하여 힙의 일부 영역에 대한 포인터를 받으면, 그 메모리 영역은 대응하는 `free()`를 통하여 반환될 때까지 프로그램이 소유하게 되고 라이브러리에 의해 다른 위치로 옮겨질 수 없다. 단편화 해결에 유용하게 사용되는 빈 공간의 **압축**은 이 경우에는 사용이 불가능하다[2]. 운영체제가 세그멘트를 구현할 때는 단편화를 해결하기 위하여 압축을 사용할 수 있다. 이에 대해서는 **세그멘테이션**에 관해 논의한 장에서 언급하였다.

20.2 저수준 기법들

세부 정책에 대해 자세히 설명하기 전에 먼저 대부분의 할당기에서 사용되는 일반적인 기법에 대해 논의한다. 첫 번째로 **분할(splitting)**과 **병합(coalescing)**의 기초에 대해서

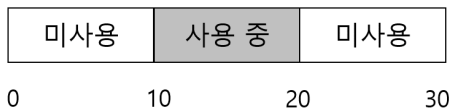
1) 거의 80 페이지에 달하기 때문에 관심을 끌 것이다!

2) C 프로그램에게 메모리 청크(chunk)를 가리키는 포인터를 전달하면, 그 영역을 가리키는 포인터를 모두 알아내는 것은 어려운 일이다. 다른 변수 혹은 심지어 실행 중에 레지스터에 저장되어 있을 수도 있기 때문이다. Strongly-typed 언어나 쓰레기 수집(garbage collection)을 하는 언어에서는 해당되지 않을 수 있다. 이러한 언어에서는 단편화 방지를 위하여 메모리 압축을 사용할 수 있다.

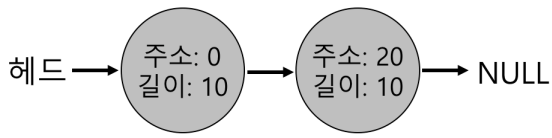
알아본다. 두 번째로 할당된 영역의 크기를 빠르고 상대적으로 쉽게 파악할 수 있는 방법을 설명한다. 마지막으로, 빈 공간과 사용 중인 공간을 추적하기 위해 빈 공간 내에 간단한 리스트를 구현하는 방법에 대해 설명할 것이다.

분할과 병합

빈 공간 리스트는 힙에 있는 빈 공간들의 집합이다. 30바이트의 힙이 있다고 가정하자.

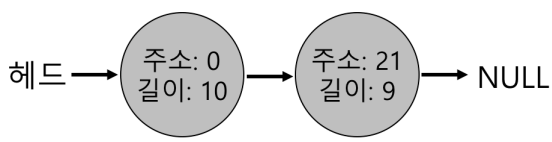


이 힙의 빈 공간 리스트에는 2개의 원소가 있다. 하나는 첫 번째 10바이트의 빈 세그멘트(바이트 0-9)를 설명하고 다른 하나는 나머지 빈 세그멘트(바이트 20-29)를 표현한다.



앞에서 언급한 바와 같이, 10바이트를 초과하는 모든 요청은 실패하여 NULL을 반환할 것이다. 요청한 크기에 해당하는 메모리 청크가 없기 때문이다. 10바이트에 대한 요청은 둘 중 하나의 빈 청크를 사용하여 쉽게 충족된다. 그러나 10바이트보다 *적은* 요청에 대해서는 어떤 일이 일어날까?

메모리를 1바이트만 요청했다고 가정하자. 이 경우 할당기는 **분할(splitting)**로 알려진 작업을 수행한다. 요청을 만족시킬 수 있는 빈 청크를 찾아 이를 둘로 분할한다. 첫 번째 청크는 호출자에게 반환되고, 두 번째 청크는 리스트에 남게 된다. 따라서 위의 예에서 1바이트 요청이 발생하고 할당기가 리스트의 두 번째 원소를 사용하여 요청을 충족시키기로 했다고 가정하자. `malloc()`은 20(1바이트가 할당된 영역의 주소)을 반환하고 최종 빈 리스트는 다음과 같이 될 것이다.

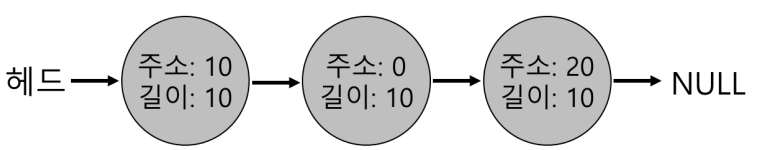


기본적인 리스트의 모습은 바뀌지 않았다는 것을 알 수 있다. 유일한 변경 사항은 빈 공간이 이제 20이 아니라 21에서 시작한다는 것과 빈 공간의 길이가 이제 9라는

것이다[3]. 요청이 특정 빈 청크의 크기보다 작은 경우 분할 기법을 사용한다.

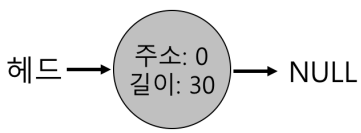
분할에 당연히 동반되는 기법은 빈 공간의 **병합**(**coalescing**)이다. 위의 예를 다시 한 번 보자(빈 공간 10바이트, 사용 중 10바이트, 또 하나의 빈 공간 10바이트).

응용 프로그램이 `free(10)`을 호출하여 힙의 중간에 존재하는 공간을 반환할 때 어떤 일이 일어나는가? 깊게 생각하지 않고 이 빈 공간을 다시 리스트에 추가할 경우 다음과 같은 모양이 될 것이다.



무슨 문제가 있는지 살펴보자. 자 이제 힙 전체가 비어 있지만, 10바이트 길이의 청크 3개로 나누어져 있다. 사용자가 20바이트를 요청하는 경우 단순한 리스트 탐색은 그와 같은 빈 청크를 발견하지 못하고 실패를 반환한다.

할당기가 이 문제를 방지하기 위하여 할 수 있는 일은 메모리 청크가 반환될 때 빈 공간들을 병합하는 것이다. 아이디어는 간단하다: 메모리 청크를 반환할 때 해제되는 청크의 주소와 바로 인접한 빈 청크의 주소를 살펴본다. 새로 해제된 빈 공간이 왼쪽의 빈 청크와 바로 인접해 있다면(혹은 이 예에서처럼 양쪽으로) 그들을 하나의 더 큰 빈 청크로 병합한다. 병합 이후의 최종 리스트는 다음과 같은 모양이 될 것이다.



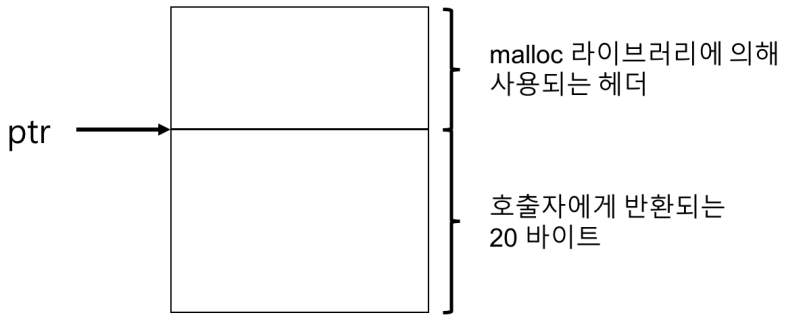
이 상태는 할당이 한 번도 일어나지 않은 최초의 힙 리스트의 모양이다. 병합 기법을 사용하여 할당기가 커다란 빈 공간을 응용 프로그램에게 제공할 수 있다는 것을 더 보장할 수 있다.

할당된 공간의 크기 파악

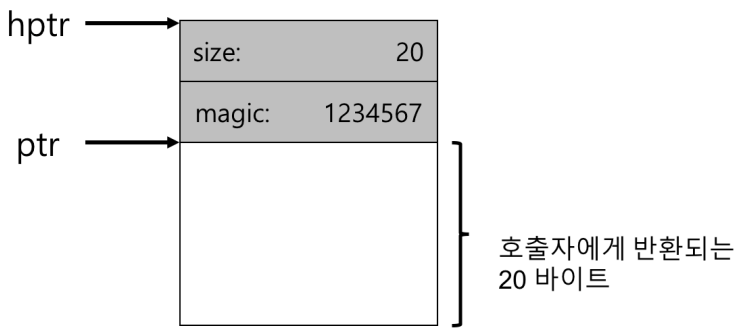
`free(void *ptr)` 인터페이스는 크기를 매개변수로 받지 않는다는 것을 알고 있을 것이다. 포인터가 인자로 전달되면 `malloc` 라이브러리는 해제되는 메모리 영역의 크기를 신속히 파악하여 그 공간을 빈 공간 리스트에 추가시킬 수 있다고 가정한다.

이 작업을 위해 대부분의 할당기는 추가 정보를 **헤더**(**header**) 블럭에 저장한다. 헤더 블럭은 메모리에 유지되며 보통 해제된 청크 바로 직전에 위치한다. 예를 살펴보자(그림 20.1). 이 예에서는 `ptr`이 가리키는 크기 20바이트의 할당된 블럭을 검토하고

3) 이 논의는 헤더가 없다고 가정한다. 비현실적인 가정이지만 당분간 논의를 간단하게 하기 위해 가정한다.



〈그림 20.1〉 **할당된 영역과 헤더**



〈그림 20.2〉 **특정 값이 저장된 헤더**

있다. 사용자는 `malloc()`을 호출하고 그 결과를 ptr에 저장하였다고 가정하자. 예를 들어, `ptr = malloc (20);`.

헤더는 적어도 할당된 공간의 크기는 저장해야 한다(이 경우 20). 또한, 해제 속도를 향상시키기 위한 추가의 포인터, 부가적인 무결성 검사를 제공하기 위한 매직 넘버, 및 기타 정보를 저장할 수 있다. 다음과 같이 할당 영역의 크기와 매직 넘버를 저장하는 간단한 헤더를 가정하자.

```
typedef struct __header_t {
    int size;
    int magic;
} header_t;
```

위의 예제는 그림 20.2에서 볼 수 있는 것과 비슷하다. 사용자가 `free(ptr)`을 호출하면 라이브러리는 헤더의 시작 위치를 파악하기 위해 간단한 포인터 연산을 한다.

```
void free(void *ptr) {
    header_t *hptr = (void *)ptr - sizeof(header_t);
    ...
```

헤더를 가리키는 포인터를 얻어 내면, 라이브러리는 매직 넘버가 기대하는 값과 일치하는지 비교하여 **안전성 검사(sanity check)**를 실시한다(`assert(hptr->magic == 1234567)`). 그리고 새로 해제된 영역의 크기를 간단한 수학을 통해 계산한다(즉,

헤더의 크기를 영역의 크기에 더함). 주의할 점이 있다. 빈 영역의 크기는 헤더 크기 더하기 사용자에게 할당된 영역의 크기가 된다. 사용자가 N 바이트의 메모리 청크를 요청하면 라이브러리는 크기 N의 빈 청크를 찾는 것이 아니라 빈 청크의 크기 N 더하기 헤더의 크기인 청크를 탐색한다.

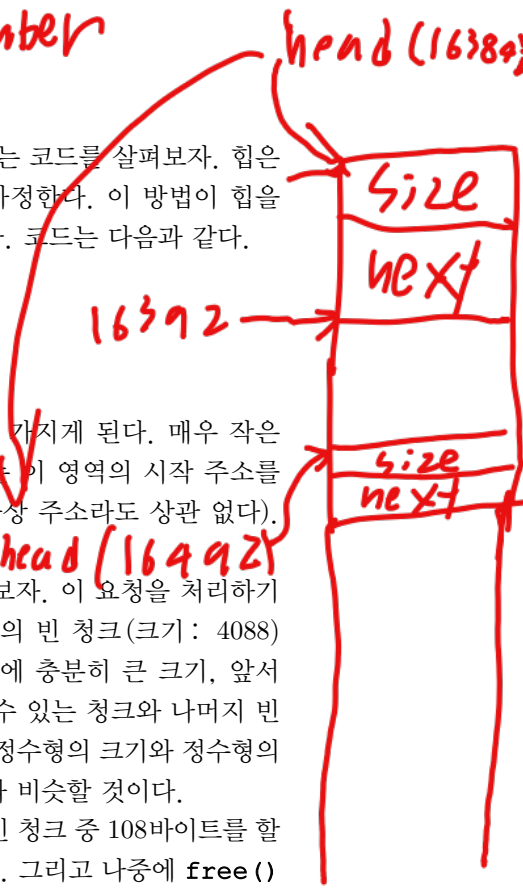
빈 공간 리스트 내장

지금까지 우리는 간단한 빈 공간 리스트의 개념만을 다루었다. 이러한 리스트를 빈 공간에 어떻게 구현할 수 있는가?

보통 새로운 노드를 위한 공간이 필요할 때 `malloc()`을 호출한다. 불행하게도 메모리 할당 라이브러리 루틴에서는 이것이 불가능하다. 대신 빈 공간 내에 리스트를 구축해야 한다.

4096바이트 크기의 메모리 청크가 있다고 하자. 즉, 힙의 크기는 4KB이다. 이를 빈 공간 리스트로 관리하기 위해서 먼저 리스트를 초기화해야 한다. 처음에 리스트는 4096(빼기 헤더 크기) 길이의 항목 하나를 가지고 있다. 이 리스트 노드의 설명은 다음과 같다.

```
typedef struct __node_t {
    int size;
    struct __node_t *next;
} node_t;
```



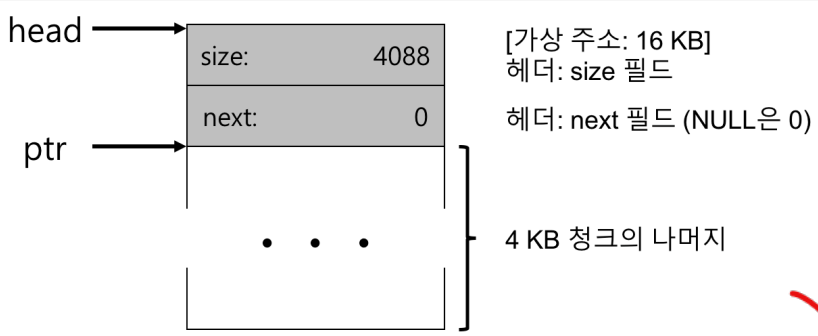
힙을 초기화하고 힙에 빈 공간 리스트의 첫 번째 원소를 넣는 코드를 살펴보자. 힙은 시스템 콜 mmap()을 호출하여 얻어진 영역에 구축된다고 가정한다. 이 방법이 힙을 구축하기 위한 유일한 방법은 아니지만, 이 예에서는 충분하다. 코드는 다음과 같다.

```
// mmap()이 빈 공간의 청크에 대한 포인터를 반환
node_t *head = mmap(NULL, 4096, PROT_READ|PROT_WRITE,
                    MAP_ANON|MAP_PRIVATE, -1, 0);
head->size = 4096 - sizeof(node_t);
head->next = NULL;
```

이 코드의 실행 후 리스트는 크기 4088의 항목 하나만을 가지게 된다. 매우 작은 힙이지만, 우리의 논의를 위해서는 충분하다. `head` 포인터는 이 영역의 시작 주소를 담고 있다. 영역의 시작 주소를 16KB라고 가정하자(아무 가상 주소라도 상관 없다). 이때 힙의 모양은 그림 20.3에서 보는 것과 비슷할 것이다.

자, 이제 100바이트 메모리 청크가 요청되었다고 생각해 보자. 이 요청을 처리하기 위해 라이브러리는 먼저 충분한 크기의 청크를 찾는다. 하나의 빈 청크(크기: 4088)만이 존재하기 때문에 이 청크가 선택된다. 요청을 처리하기에 충분히 큰 크기, 앞서 말한 것처럼 빈 영역의 크기 더하기 헤더의 크기를 충족시킬 수 있는 청크와 나머지 빈 청크 두 개로 **분할**한다. 헤더의 크기를 8바이트라고 가정하면(정수형의 크기와 정수형의 매직 넘버), 이제 힙의 공간의 모습은 그림 20.4에 보이는 것과 비슷할 것이다.

100바이트에 대한 요청이 오면 라이브러리는 기존 하나의 빈 청크 중 108바이트를 할당하고 할당 영역을 가리키는 포인터(그림의 `ptr`)를 반환한다. 그리고 나중에 `free()`



〈그림 20.3〉 **하나의 빈 청크만을 가진 힙**

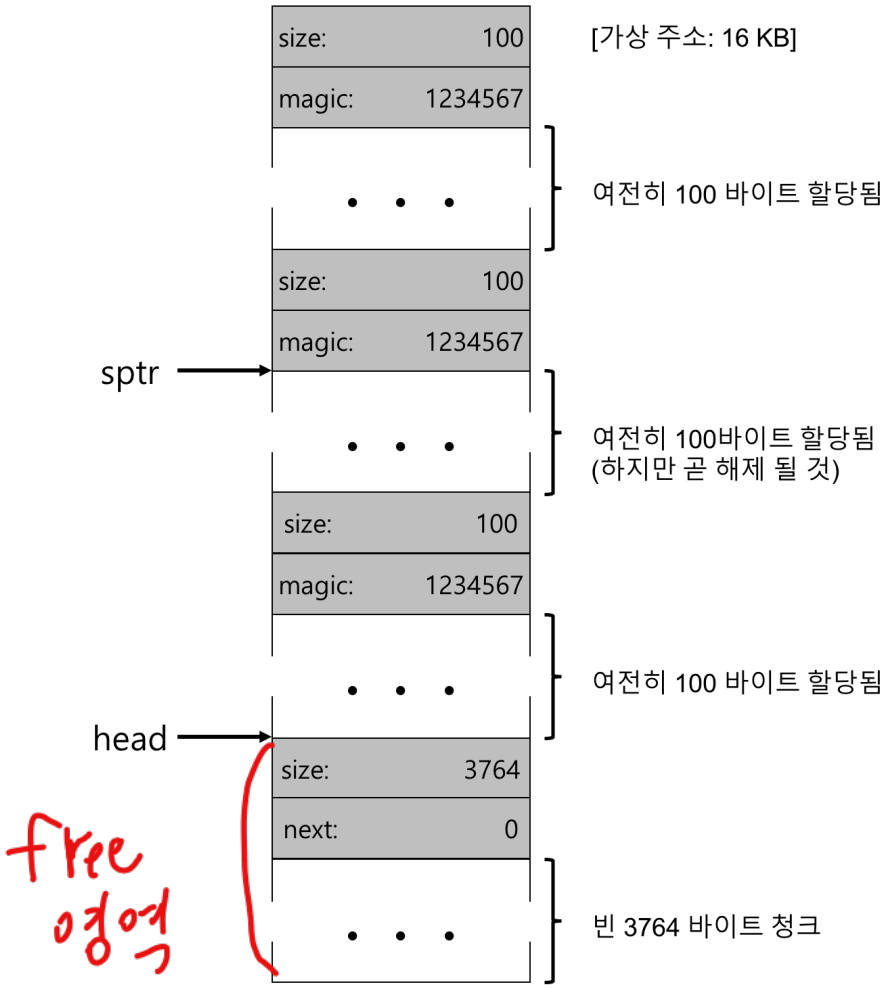


〈그림 20.4〉 **힙 : 한 번의 할당 후**

에서 사용할 수 있도록 할당된 공간 직전 8바이트에 헤더 정보를 넣는다. 그런 후 하나 남은 빈 노드를 3980바이트(4088-108)로 축소한다.

자, 이제 100바이트씩(또는 헤더 포함 108바이트) 할당된 3개의 공간이 존재하는 힙을 살펴보자. 이 힙의 모습이 그림 20.5와 같다.

그림에서 볼 수 있듯이 힙의 시작 부분 324바이트가 현재 할당되어 있다. 3개의 헤더와 호출 프로그램에 의해 사용 중인 3개의 100바이트 영역을 볼 수 있다. 빈 공간 리스트는 여전히 `head`가 가리키는 하나의 노드로 구성되어 있지만 세 번의 분할 이후 3764바이트로 축소된 모습이다. 프로그램이 `free()`를 통해 일부 메모리를 반환하면 어떤 일이 벌어질까?

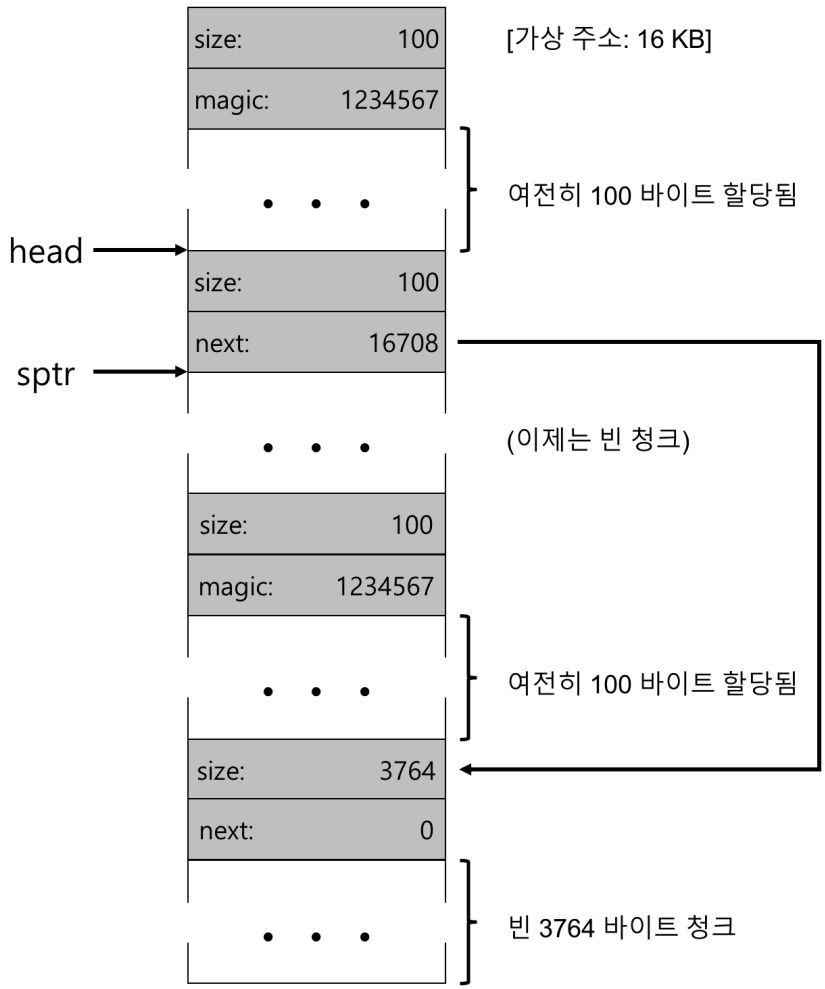


〈그림 20.5〉 **3개의 할당 청크를 가진 빈 공간**

이 예제에서 응용 프로그램은 `free(16500)`을 호출하여 할당 영역 중 가운데 청크를 반환한다. 16500은 메모리 영역의 시작 주소 16384, 이전 메모리 청크의 크기 108, 해제되는 청크의 헤더 8바이트를 모두 더해서 나온 값이다. 그림에서 이 값은 `sptr`이 나타낸다.

라이브러리는 신속히 빈 공간의 크기를 파악하고, 빈 청크를 빈 공간 리스트에 삽입한다. 빈 공간 리스트의 헤드 쪽에 삽입한다고 가정하면 공간의 모양은 그림 20.6과 같다.

이제 빈 공간 리스트의 첫 번째 원소는 작은 빈 청크(100바이트 크기이며 리스트의 헤드가 가리킴)이고 두 번째 원소는 큰 빈 청크(3764바이트)이다. 드디어 우리 리스트가 하나 이상의 원소를 가진다! 불행하지만 흔히 일어나는 단편화가 발생하였다.



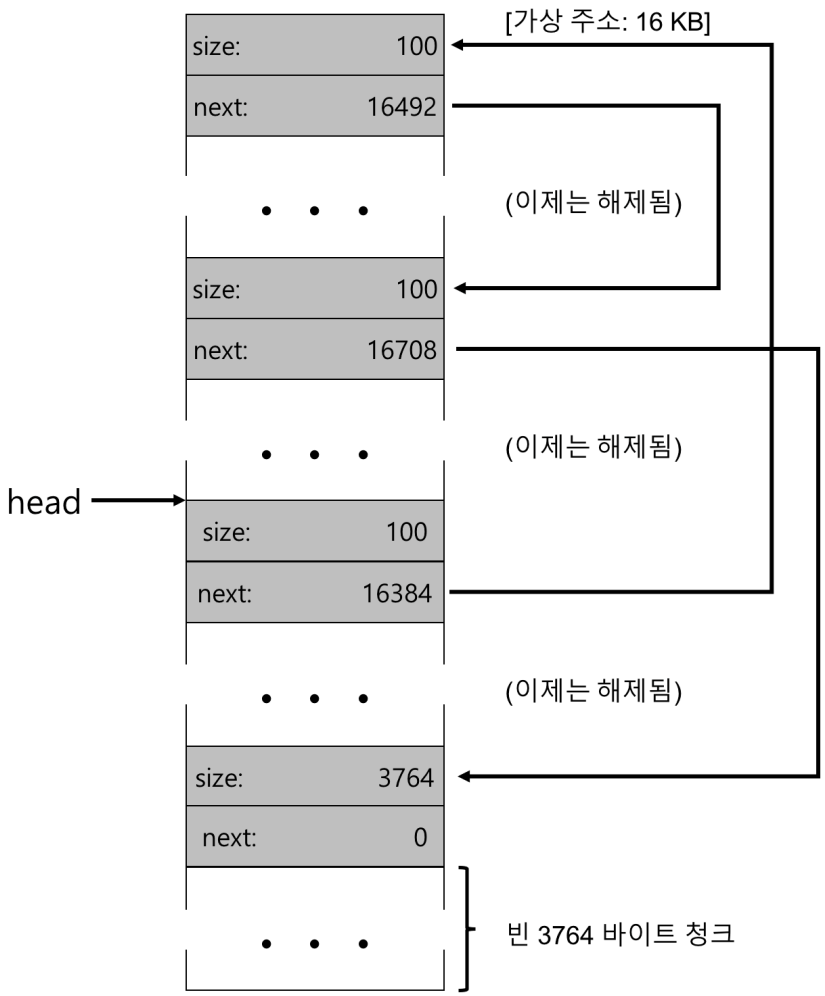
〈그림 20.6〉 **2개의 할당 청크를 가진 빈 공간**

마지막으로 마지막 2개의 사용 중인 청크가 해제된다고 하자. 병합이 없다면, 작은 단편으로 이루어진 빈 공간 리스트가 될 것이다(그림 20.7 참조).

그림에서 볼 수 있듯이, 우리는 지금 큰 난관에 봉착했다. 왜? 우리는 리스트를 **병합**하지 않았다. 모든 메모리가 비어 있긴 하지만, 전부 조각나 있기 때문에 한 청크가 아니라 단편으로 이루어진 메모리처럼 보인다. 해결책은 간단하다: 리스트를 순회하면서 인접한 청크를 **병합**하면 된다. 병합이 완료되면 힙은 전체 하나의 큰 청크가 된다.

힙의 확장

이제 마지막 주제를 다루도록 하겠다. 힙 공간이 부족한 경우에 어떻게 할 것인가? 가장 쉬운 방법은 단순히 실패를 반환하는 것이다. 어떤 경우에는 이 방법이 유일한 대안이며,



〈그림 20.7〉 **병합되지 않은 빈 공간 리스트**

따라서 `NULL`을 반환하는 것은 훌륭한 접근법이다. 너무 낙담하지 마라! 당신은 노력했고 실패했지만 잘 싸운거다.

대부분의 전통적인 할당기는 적은 크기의 힙으로 시작하여 모두 소진하면 운영체제로부터 더 많은 메모리를 요청한다. 할당기는 힙을 확장하기 위하여 특정 시스템 콜(예, 대부분의 UNIX 시스템에서는 `sbrk`)을 호출한다. 그런 후 확장된 영역에서 새로운 청크를 할당한다. `sbrk` 요청을 수행하기 위해 운영체제는 빈 물리 페이지를 찾아 요청 프로세스의 주소 공간에 매핑한 후, 새로운 힙의 마지막 주소를 반환한다. 이제부터 더 큰 힙을 사용할 수 있고 요청은 성공적으로 충족될 수 있다.

20.3 기본 전략

일부 기법에 대해 간략히 살펴보았으므로, 빈 공간 할당을 위한 기본 전략에 대해 살펴보자. 기본 전략들은 여러분들이 스스로 생각해 낼 수 있을만큼 매우 간단하다. 읽기 전에 한 번 생각해 보라. 모든 대안을 생각해 보았는지 확인하라. 아니면 새로운 정책을 생각했을 수도 있다.

이상적인 할당기는 속도가 빠르고 단편화를 최소로 해야 한다. 불행하게도 할당과 해제 요청 스트림은 무작위, 결국 프로그래머에 의해 결정되기 때문에, 어느 특정 전략도 잘 맞지 않는 입력을 만나면 성능이 매우 좋지 않을 수 있다. 우리는 최선의 정책을 설명하는 것이 아니라 몇 가지 기본 정책에 대해 이야기하고 각각의 장점과 단점을 논의한다.

최적 적합(Best Fit)

최적 적합 전략은 매우 간단하다. 먼저 빈 공간 리스트를 검색하여 요청한 크기와 같거나 더 큰 빈 메모리 청크를 찾는다. 그 후, 후보자 그룹 중에서 가장 작은 크기의 청크를 반환한다. 이 청크는 최적 청크라고 불린다. 최소 적합이라고도 불릴 수 있다. 빈 공간 리스트를 한 번만 순회하면 반환할 정확한 블럭을 찾을 수 있다.

최적 적합의 배경은 간단하다. 사용자가 요청한 크기에 가까운 블럭을 반환함으로써 최적 적합은 공간의 낭비를 줄이려고 노력한다. 그러나 이에는 비용이 수반된다. 정교하지 않은 구현은 해당 빈 블럭을 찾기 위해 항상 전체를 검색해야 하기 때문에 엄청난 성능 저하를 초래한다.

최악 적합(Worst Fit)

최악 적합은 최적 적합의 반대 방식이다. 가장 큰 빈 청크를 찾아 요청된 크기 만큼만 반환하고 남는 부분은 빈 공간 리스트에 계속 유지한다. 최악 적합은 최적 적합 방식에서 발생할 수 있는 수많은 작은 청크 대신에 커다란 빈 청크를 남기려고 시도한다. 그러나 다시 한 번 항상 빈 공간 전체를 탐색해야 하기 때문에 이 방법 역시 높은 비용을 지불해야 한다. 설상가상으로 대부분의 연구에서 엄청난 단편화가 발생하면서 오버헤드도 여전히 크다는 것을 보이고 있다.

최초 적합(First Fit)

최초 적합은 간단하게 요청보다 큰 첫 번째 블럭을 찾아서 요청만큼 반환한다. 먼저와 같이 남은 빈 공간은 후속 요청을 위해 계속 유지된다.

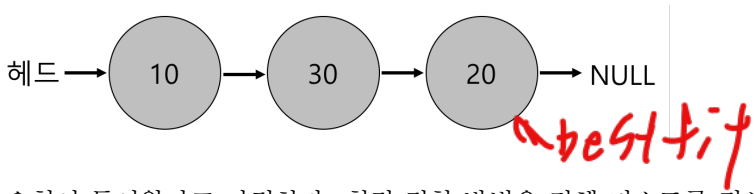
최초 적합은 속도가 빠르다는 것이 장점이다. 원하는 블럭을 찾기 위해 항상 빈 공간 리스트 전체를 탐색할 필요가 없다. 그러나 때때로 리스트의 시작에 크기가 작은 객체가 많이 생길 수 있다. 따라서 할당기가 빈 공간 리스트의 순서를 관리하는 방법이 쟁점이다. 한 가지 방법은 **주소-기반 정렬**(**address-based ordering**)을 사용하는 것이다. 리스트를 주소로 정렬하여 병합을 쉽게 하고, 단편화로 감소시킨다.

다음 적합(Next Fit)

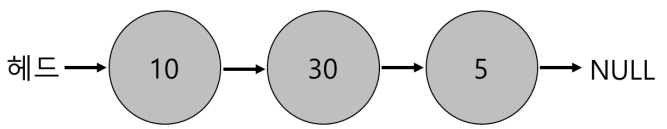
항상 리스트의 처음부터 탐색하는 대신 **다음 적합** 알고리즘은 마지막으로 찾았던 원소를 가리키는 추가의 포인터를 유지한다. 아이디어는 빈 공간 탐색을 리스트 전체에 더 균등하게 분산시키는 것이다. 리스트의 첫 부분에만 단편이 집중적으로 발생하는 것을 방지한다. 이러한 접근 방식은 전체 탐색을 하지 않기 때문에 최초 적합의 성능과 비슷하다.

예제

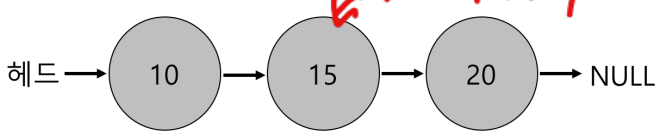
위에서 언급한 전략이 동작하는 몇 가지 예를 보도록 하겠다. 크기가 각각 10, 30, 및 20인 세 개의 원소를 가진 빈 공간 리스트를 생각해 보자(여기서는 헤더와 다른 세부사항은 무시하고 전략이 동작하는 방법에 대해서만 초점을 맞춘다).



크기 15인 요청이 들어왔다고 가정하자. 최적 적합 방법은 전체 리스트를 검색하여 요청을 만족시킬 수 있는 청크 중 가장 작은 20을 선택한다. 할당 후 빈 공간 리스트는 다음과 같다:



이 예에서와 같이, 최적 적합의 경우에는 종종 작은 빈 청크가 남는다. 최악 적합 방식은 유사하지만 대신 가장 큰 청크를 찾는 데 이 예에서는 30이다. 할당 후 빈 공간 리스트는 다음과 같다.



이 예에서 최초 적합 전략은 최악 적합 전략과 같은 결과를 도출한다. 요청을 충족시킬 수 있는 첫 번째 빈 블럭을 찾았다. 차이점은 탐색 비용이다. 최적 적합과 최악 적합은 리스트 전체를 탐색한다. 최초 적합은 단지 적합한 청크를 발견할 때까지만 빈 청크를 검사하기 때문에 탐색 비용이 감소된다.

살펴본 예들은 할당 정책들의 동작을 피상적으로 간단히 보여 준다. 전략에 대한 심도있는 이해를 위해서는 실제 워크로드를 대상으로 그리고 더 복잡한 할당기 동작(예, 병합)에 대한 더 상세한 분석이 필요하다.

20.4 다른 접근법

위에서 설명한 기본적인 접근 방식 외에도 메모리 할당을 향상시키기 위한 기술과 알고리즘이 제안되었다. 그 중 몇 가지를 여기서 소개한다. 단순한 최적 적합 할당에 그치지 않고 그 이상에 대해서 생각해 보도록 하는 것이 목적이다.

개별 리스트

한 동안 유행했던 방법은 별도의 **개별 리스트**(**segregated list**)를 사용하는 것이다. 기본적인 아이디어는 간단하다. 특정 응용 프로그램이 한두 개 자주 요청하는 크기가 있다면, 그 크기의 객체를 관리하기 위한 별도의 리스트를 유지하는 것이다. 다른 모든 요청은 더 일반적인 메모리 할당기에게 전달된다.

이 방법의 장점은 분명하다. 특정 크기의 요청을 위한 메모리 청크를 유지함으로써 단편화 가능성을 상당히 줄일 수 있다. 요청된 크기의 청크만이 존재하기 때문에 복잡한 리스트 검색이 필요하지 않으므로 할당과 해제 요청을 신속히 처리할 수 있다.

다른 좋은 아이디어처럼, 이 방법은 시스템에 새로운 문제를 야기한다. 예를 들어, 지정된 크기의 메모리 풀과 일반적인 풀에 얼마만큼의 메모리를 할당해야 하는가? 특수 목적 할당기인 **슬랩 할당기**(**slab allocator**)는 이 문제를 더 나은 방법으로 해결한다. 슬랩 할당기는 뛰어난 엔지니어인 Jeff Bonwick에 의해 Solaris 커널을 위하여 설계되었다 [Bon94].

커널이 부팅될 때 커널 객체를 위한 여러 **객체 캐시**(**object cache**)를 할당한다. 커널 객체란 락, 파일 시스템 아이노드 등 자주 요청되는 자료 구조들을 일컫는다. 객체 캐시는 지정된 크기의 객체들로 구성된 빈 공간 리스트이고 메모리 할당 및 해제 요청을 빠르게 서비스 하기 위해 사용된다. 아이노드들로 구성된 객체 캐시가 있고, 락 구조만을 담고 있는 객체 캐시도 있다. 기존에 할당된 캐시 공간이 부족하면 상위 메모리 할당기에게 추가 **슬랩**을 요청한다. 요청의 전체 크기는 페이지 크기의 정수배이다. 반대로, 슬랩 내 객체들에 대한 참조 횟수가 0이 되면 상위 메모리 할당기는 이 슬랩을 회수할 수 있다. VM 시스템이 더 많은 메모리를 필요할 때 실제 회수가 일어난다.

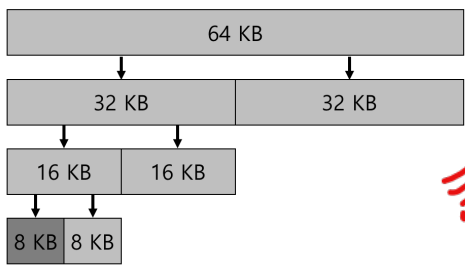
슬랩 할당 방식은 빈 객체들을 사전에 초기화된 상태로 유지한다는 점에서 개별 리스트 방식보다 우수하다. Bonwick은 자료 구조의 초기화와 반납에는 많은 시간이 소요된다는 것을 발견하였다 [Bon94]. 반납된 객체들을 초기화된 상태로 리스트에 유지하여 슬랩 할당기는 객체당 잦은 초기화와 반납의 작업을 피할 수 있어서 오버헤드를 현저히 감소시킨다.

여담: 위대한 엔지니어는 정말 위대하다

여기서 얘기한 슬랩 할당기뿐 아니라 ZFS라는 놀라운 파일 시스템의 개발을 주도한 Jeff Bonwick 같은 엔지니어는 실리콘 밸리의 심장이다. 거의 모든 위대한 작품이나 기술 뒤에는 재주, 능력과 헌신이 평균 이상인 인간(또는 인간의 소그룹)이 있었다. Facebook의 Mark Zuckerberg는 "예외적인 사람들은 보통 사람보다 조금 우수한 것이 아니다. 그들은 100배 우수하다."라고 말한다. 이것이 왜 아직도 오늘날 하나 또는 두 사람이 세상의 모습을 영원히 바꾸는 회사를 시작할 수 있는 이유이다(Google, Apple 또는 Facebook을 생각해 보라). 열심히 노력하면 당신도 그러한 "100×" 사람이 될지도 모른다. 그런 사람이 못 된다면 그런 사람과 함께 일을 하라. 대부분의 사람이 한 달 동안 배우는 것보다 더 많은 것을 하루 동안에 배우게 될 것이다. 그것도 실패하면 슬퍼하라.

버디 할당

빈 공간의 합병은 할당기의 매우 중요한 기능이기 때문에 합병을 간단히 하는 방법들이 설계되었다. 하나의 좋은 예가 **이진 버디 할당기(binary buddy allocator)**이다 [Kno65]. 빈 메모리는 처음에 개념적으로 크기 2^N 인 하나의 큰 공간으로 생각된다. 메모리 요청이 발생하면, 요청을 충족시키기에 충분한 공간이 발견될 때까지(그리고 더 분할하면 공간이 너무 작아져서 요청을 만족시킬 수 없을 때까지) 빈 공간을 2개로 계속 분할한다. 이 시점에서 요청된 블럭이 사용자에게 반환된다. 64 KB 빈 공간에서 7 KB 블럭을 할당하는 예가 있다.



이 예에서 가장 왼쪽의 8 KB 블럭이 할당되고(짙은 회색으로 표시) 사용자에게 반환된다. 이 방식은 2의 거듭제곱 크기 만큼의 블럭만 할당할 수 있기 때문에 **내부 단편화**로 고생할 수 있다는 점에 유의하라.

버디 할당의 아름다움은 블럭이 해제될 때에 있다. 8 KB 블럭을 빈 공간 리스트에 반환하면 할당기는 "버디" 8 KB가 비어 있는지 확인한다. 비어 있다면 두 블럭을 병합하여 16 KB 블럭으로 만든다. 할당기는 다음 16 KB의 버디가 비어 있는지 확인한다. 비어 있다면 이 두 블럭을 다시 합병한다. 이 재귀 합병 과정은 트리를 따라 전체 빈 공간이 복원되거나 버디가 사용 중이라는 것이 밝혀질 때까지 계속 올라간다.

버디 할당이 잘 작동하는 이유는 특정 블럭의 버디를 결정하는 것이 쉽다는 데 있다. 앞에서 빈 공간에 존재하는 블럭의 주소를 한 번 생각해 보자. 각 버디 쌍의 주소는 오직

한 비트만 다르다. 어느 위치의 비트가 다른가는 버디 트리의 수준에 따라 달라진다. 이제 이진 버디 할당 기법의 동작에 대한 기본 사항을 알게 되었다. 더 자세한 사항에 대해서는 Wilson의 요약 문서를 보라 [RWi+95].

기타 아이디어

앞에서 설명한 접근 방식들의 한 가지 문제점은 **확장성**이다. 빈 공간들의 개수가 늘어남에 따라 리스트 검색이 매우 느려질 수 있다. 좀 더 정교한 할당기는 복잡한 자료 구조를 사용하여 이 비용을 줄인다. 단순함과 성능을 교환한다. 균형 이진 트리(balanced binary tree), 스플레이 트리(splay tree), 또는 부분 정렬 트리(partially ordered tree)가 좋은 예이다 [RWi+95].

현대의 시스템은 멀티프로세서 및 멀티쓰레드로 작동된다. 멀티프로세서를 위해 할당기를 최적화하는 노력들이 많이 있었다. Berger [Ber+00]와 Evans [Eva06]가 좋은 예이다. 자세한 내용에 대해서는 그들을 참조하라.

이들은 오랜 시간 개발한 수천 개의 할당기 아이디어 중 두 개에 해당한다. 관심이 있다면 직접 읽어 보기 바란다. 그럴 수 없다면 실제는 어떤가를 이해하기 위해 `glibc`의 할당기가 어떻게 작동하는지에 대해서 읽어라 [Spl15].

20.5 요약

이 장에서 가장 기본적인 형태의 메모리 할당기를 논의하였다. 이러한 할당기는 많은 곳에 존재한다. 우리가 작성한 모든 C 프로그램뿐 아니라 운영체제에도 존재한다. 여느 시스템과 마찬가지로, 이러한 시스템을 구축하는 데 많은 선택 사항들이 있다. 할당기에 들어오는 워크로드를 정확히 이해할수록, 그 워크로드에 대해 더 잘 작동하도록 조정할 수 있다. 다양한 워크로드에 대해 빠르고 효율적이고 확장성이 좋은(scalable) 할당기를 만드는 일은 현대 컴퓨터 시스템의 숙제이다.

참고 문헌

[Ber+00] **"Hoard: A Scalable Memory Allocator for Multithreaded Applications"**
Emery D. Berger, Kathryn S. McKinley, Robert D. Blumofe, and Paul R.Wilson
ASPLOS-IX, November 2000
Berger와 동료들의 멀티프로세서 시스템을 위한 훌륭한 할당기. 단지 흥미로운 논문을 넘어서 실제 사용 중인 할당기.

[Bon94] **"The Slab Allocator: An Object-Caching Kernel Memory Allocator"**
Jeff Bonwick
USENIX '94
운영체제 커널을 위한 할당기를 구축하는 방법과 자주 사용되는 특정 객체의 크기에 대해 특화시키는 방법의 훌륭한 예에 관한 멋진 논문.

[Eva06] **"A Scalable Concurrent malloc(3) Implementation for FreeBSD"**
Jason Evans
April 2006
URL: `http://people.freebsd.org/%CB%9Cjasone/jemalloc/bsdcan2006/jemalloc.pdf`
멀티프로세서에서 사용될 실제 현대의 할당기 구축 방법에 관한 상세한 관찰 . "jemalloc" 할당기는 현재 FreeBSD, NetBSD, Mozilla Firefox, 및 Facebook에서 널리 사용 중에 있다.

[Kno65] **"A Fast Storage Allocator"**
Kenneth C. Knowlton
Communications of the ACM, Volume 8, Number 10, October 1965
버디 할당에 대한 표준 참고 문헌. 이상한 사실 : Knuth는 이 아이디어에 대한 공을 Knowlton이 아니라 노벨상 수상 경제학자인 Harry Markowitz에 돌린다. 또 다른 이상한 사실 : Knuth는 그의 모든 이메일을 비서를 통하여 주고 받는다. 그는 직접 이메일을 보내지 않고 비서에게 보낼 이메일을 말하고 비서가 전송 작업을 수행한다. 마지막 Knuth에 관한 사실 : 그는 이 책을 조판하는 데 사용된 도구인 TeX을 만들었다. 그것은 환상적인 소프트웨어 작품이다[4].

[RWi+95] **"Dynamic Storage Allocation: A Survey and Critical Review"**
Paul R.Wilson, Mark S. Johnstone, Michael Neely, and David Boles International Workshop on Memory Management
Kinross, Scotland, September 1995
메모리 할당의 여러 측면에 관한 훌륭하고 지대한 영향을 준 조사. 이 짧은 장 안에 모든 것을 담기에는 너무 자세한 내용.

[Spl15] **"Understanding glibc malloc"**
Sploitfun
February, 2015
https://sploitfun.wordpress.com/2015/02/10/understanding-glibc-malloc/에는 ***`glibc malloc`****이 어떻게 동작하는지를 놀라울 정도로 자세하고 깊이 있게 연구한 내용을 담고 있는 매우 멋진 문서.*

4) 실제로는 LaTeX을 사용하였다. LaTeX은 Lamport가 TeX에 추가 기능으로 만든 것으로 거의 같다고 할 수 있다.

숙제

`malloc.py` 프로그램은 이 장에서 논의한 간단한 빈 공간 할당기의 동작을 탐색할 수 있게 한다. 그 기본 동작에 대한 자세한 사항은 README를 참조하시오

문제

1. 먼저 무작위 할당과 해제를 생성하기 위하여 `-n 10 -H 0 -p BEST -s 0` 플래그를 주고 실행시켜라. `alloc()`과 `free()`가 무엇을 반환할지 예측할 수 있는가? 각 요청 후의 빈 공간 리스트의 상태를 추측할 수 있는가? 시간이 지남에 따라 빈 공간 리스트에 대해 무엇을 알 수 있는가?
2. 빈 공간 리스트를 탐색하기 위하여 최악 적합 정책을 사용할 때 결과는 어떻게 달라지는가(`-p WORST`)? 무엇이 바뀌는가?
3. 최초 적합을 사용한 경우는 어떤가(`-p FIRST`)? 최초 적합을 사용하면 속도 향상은 어떻게 되는가?
4. 위 질문에 대해, 몇몇 정책의 경우 리스트의 정렬 순서가 빈 위치를 찾는 데 걸리는 시간에 영향을 준다. 다른 빈 공간 리스트의 순서(`-l ADDRSORT`, `-l SIZESORT+`, `-l SIZESORT-`)를 사용하여 정책과 리스트 순서가 서로 어떤 영향을 주는지 관찰하라.
5. 빈 공간의 합병은 매우 중요할 수 있다. 무작위 할당의 개수를 늘려라(`-n 1000`). 시간이 지날수록 요청의 개수가 많아지면 어떤 현상이 발생하는가? 병합이 있는 채로 그리고 없는 채로 실행하라(`-C` 플래그가 없거나 있거나). 출력에 어떤 차이를 볼 수 있는가? 각각의 경우의 시간이 지남에 따라 빈 공간 리스트의 크기는 얼마나 커지는가? 이 경우 리스트의 정렬 순서가 중요한가?
6. 할당 비율 `-P`를 50 이상으로 변경하면 어떤 일이 일어날까? 100에 가까울수록 할당에는 어떤 일이 생기는가? 0에 가까워진다면?
7. 심하게 단편화된 빈 공간을 만들어 내려면 어떤 종류의 요구를 생성할 수 있는가? 단편화된 리스트를 생성하기 위하여 `-A` 플래그를 사용하라. 다른 정책과 옵션이 빈 공간 리스트의 구조에 어떤 영향을 주는지 관찰하라.

21

페이징: 개요

운영체제는 거의 모든 공간 관리 문제를 해결할 때 두 가지 중 하나를 사용한다. 첫 번째 방법은 우리가 가상 메모리의 **세그멘테이션**에서 보았듯이, *가변 크기의* 조각들로 분할하는 것이다. 불행하게도, 이 해결책은 태생적인 문제를 가지고 있다. 공간을 다양한 크기의 청크로 분할할 때 공간 자체가 **단편화**(**fragmented**) 될 수 있고, 할당은 점점 더 어려워진다.

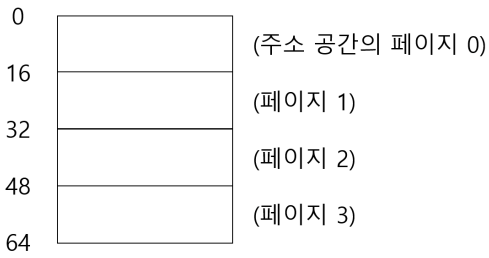
두 번째 방법인 공간을 *동일 크기의* 조각으로 분할하는 것을 고려해 볼 필요가 있다. 가상 메모리에서 이를 **페이징**(**paging**)이라 부르고 이 아이디어의 태생은 초기의 중요한 시스템인 Atlas까지 거슬러 올라간다 [Kil+62; Lav78]. 프로세스의 주소 공간을 몇 개의 가변 크기의 논리 세그멘트 (예, 코드, 힙, 스택)로 나누는 것이 아니라 고정 크기의 단위로 나눈다. 이 각각의 고정 크기 단위를 **페이지**(**page**)라고 부른다. 상응하여 물리 메모리도 **페이지 프레임**(**page frame**)이라고 불리는 고정 크기의 슬롯의 배열이라고 생각한다. 이 프레임 각각은 하나의 가상 메모리 페이지를 저장할 수 있다. 우리의 문제는 다음과 같다.

핵심 질문: 페이지를 사용하여 어떻게 메모리를 가상화할 수 있을까
세그멘테이션의 문제점을 해결하기 위해 페이지를 사용하여 어떻게 메모리를 가상화할 수 있는가? 기본적인 기법은 무엇인가? 공간과 시간 오버헤드를 최소로 하면서 그 기법을 잘 동작하게 만들기 위한 방법은 무엇인가?

21.1 간단한 예제 및 개요

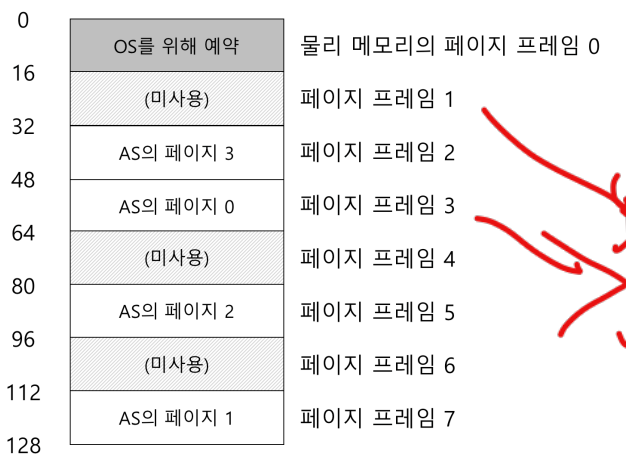
이 방법을 명확히 이해하기 위해 간단한 예를 통해 설명해 보자. 그림 21.1은 총 크기가 64바이트이면서 4개의 16바이트 페이지로 구성된 (가상 페이지 0, 1, 2, 4) 작은 주소 공간의 예를 보여준다. 물론 실제 주소 공간은 훨씬 커서 32비트의 경우 4 GB, 64비트의 경우에는 그보다 훨씬 크다[1].

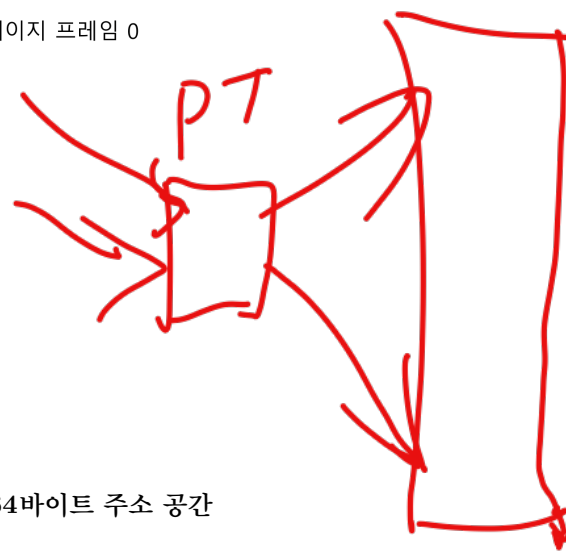
1) 64비트 주소 공간은 상상하기 어려울 정도로 엄청나게 크다. 비유가 도움이 될 것이다. 32비트 주소 공간을 테니스 코트라고 생각하면 64비트 주소 공간은 유럽만하다(!).



〈그림 21.1〉 **간단한 64바이트 주소 공간**

물리 메모리는 그림 21.2에 나타낸 바와 같이, 고정 크기의 슬롯들로 구성된다. 이 경우 8개의 페이지 프레임, 총 128바이트의 비현실적으로 작은 물리 메모리이다. 그림에서 볼 수 있듯이, 가상 주소 공간의 페이지들은 물리 메모리 전체에 분산 배치되어 있다. 그림은 또한 운영체제가 자기자신을 위해서 물리 메모리의 일부를 사용하는 것도 보여 준다.





〈그림 21.2〉 **128바이트 물리 메모리에 탑재된 64바이트 주소 공간**

앞으로 보겠지만, 페이징은 이전 방식에 비해 많은 장점을 가지고 있다. 아마도 가장 중요한 개선은 *유연성*일 것이다. 페이징을 사용하면 프로세스의 주소 공간 사용 방식과는 상관없이 효율적으로 주소 공간 개념을 지원할 수 있다. 예를 들어, 힙과 스택이 어느 방향으로 커지는가, 어떻게 사용되는가에 대한 가정을 하지 않아도 된다.

또 다른 장점은 페이징이 제공하는 빈 공간 관리의 *단순함*이다. 예를 들어, 운영체제가 우리의 작은 64바이트 주소 공간을 8페이지 물리 메모리에 배치하기를 원한다고 할 때, 운영체제는 비어 있는 네 개의 페이지만 찾으면 된다. 아마 이를 위해 운영체제는 모든 비어 있는 페이지의 **빈 공간 리스트**를 유지하고 리스트의 첫 네 개 페이지를 선택할 것이다. 이 예에서, 운영체제는 주소 공간의 페이지 0을 물리 프레임 3에, 가상 페이지 1을 물리 프레임 7, 페이지 2를 프레임 5, 그리고 페이지 3을 프레임 2에 배치하였다.

페이지 프레임 1, 4, 6은 현재 비어 있다.

주소 공간의 각 가상 페이지에 대한 물리 메모리 위치 기록을 위하여 운영체제는 *프로세스 마다* **페이지 테이블**(**page table**)이라는 자료 구조를 유지한다. 페이지 테이블의 주요 역할은 주소 공간의 가상 페이지 **주소 변환**(**address translation**) 정보를 저장하는 것이다. 각 페이지가 저장된 물리 메모리의 위치가 어디인지 알려준다. 위의 간단한 예제의 경우(그림 21.2) 페이지 테이블은 다음과 같은 4개의 항목을 가지게 될 것이다(가상 페이지 0 → 물리 프레임 3), (VP 1 → PF 7), (VP 2 → PF 5) 및 (VP 3 → PF 2).

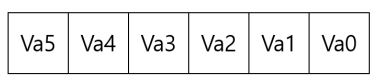
페이지 테이블은 프로세스마다 존재한다는 사실을 숙지해야 한다(우리가 논의하는 대부분의 페이지 테이블 구조는 *프로세스 마다* 존재하는 구조이다. **역 페이지 테이블**(**inverted page table**)이라는 예외적인 기법도 있다). 위의 예에서 다른 프로세스를 실행해야 한다면 운영체제는 이 프로세스를 위한 다른 페이지 테이블이 필요하다. 새 프로세스의 가상 페이지는 *다른* 물리 페이지에 존재하기 때문이다(공유 중인 페이지가 없다면).

이제 주소 변환 준비가 되었다. 작은 주소 공간(64바이트)을 가진 프로세스가 다음 메모리 접근을 수행한다고 가정하자.

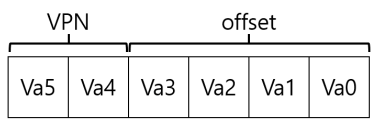
```
movl <virtual address>, %eax
```

구체적으로 주소 `<virtual address>`의 데이터를 `eax` 레지스터에 탑재하는 데에 집중하자(그 전에 일어나는 명령어 반입 단계는 무시하자).

프로세스가 생성한 가상 주소의 **변환**을 위해 먼저 가상 주소를 **가상 페이지 번호**(**virtual page number**, **VPN**)와 페이지 내의 **오프셋** 2개의 구성 요소로 분할한다. 이 예에서는 가상 주소 공간의 크기가 64바이트이기 때문에 가상 주소는 6비트가 필요하다($2^6 = 64$). 가상 주소를 개념적으로 다음 그림과 같이 생각할 수 있다.



이 그림에서 Va5는 가상 주소의 최상위 비트이며, Va0은 최하위 비트를 나타낸다. 우리는 페이지 크기(16바이트)를 알고 있기 때문에, 다음과 같이 가상 주소를 나눌 수 있다:

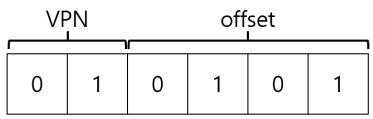


페이지 크기는 64바이트의 주소 공간에서 16바이트이다. 따라서 4페이지를 선택할 수 있어야 하고 따라서 주소의 최상위 2비트가 그 역할을 한다. 우리는 2비트 가상 페이지 번호 (VPN)를 가지게 된다. 나머지 비트는 페이지 내에서 우리가 원하는 바이트의 위치를 나타낸다. 이것을 오프셋이라고 부른다.

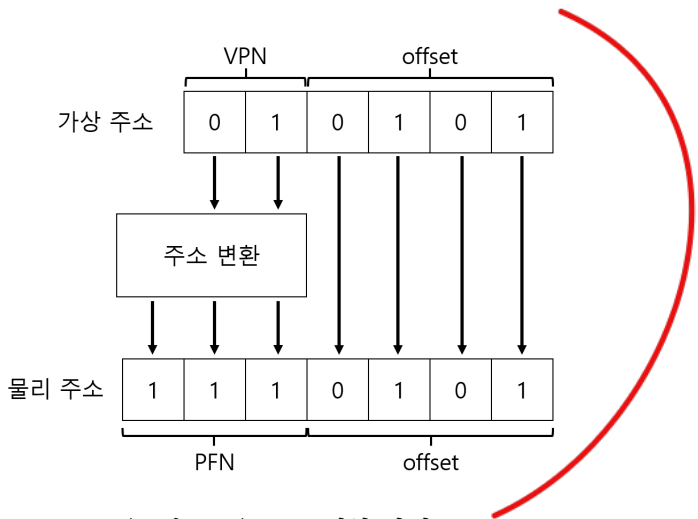
프로세스가 가상 주소를 생성하면 운영체제와 하드웨어가 의미있는 물리 주소로 변환한다. 예를 들어, 위 탑재 명령어의 가상 주소가 21이라고 하자.

```
movl 21, %eax
```

"21"을 이진 형식으로 변환하면 "010101"을 얻고, 이 가상 주소를 검사하고 가상 페이지 번호와 오프셋으로 나눈다.



따라서 가상 주소 "21"은 가상 페이지 "01"(또는 1)의 5번째("0101"번째) 바이트이다. 이 가상 페이지 번호를 가지고 페이지 테이블의 인덱스로 사용하여 가상 페이지 1이 어느 물리 프레임에 저장되어 있는지 찾을 수 있다. 위의 페이지 테이블에서 **물리 프레임 번호(physical frame number, PFN)** 혹은 **물리 페이지 번호(physical page number, PPN)**는 7(이진수 111)이다. VPN을 PFN으로 교체하여 가상 주소를 변환할 수 있다. 그런 후에 물리 메모리에 탑재 명령어를 실행한다(그림 21.3).



〈그림 21.3〉 **주소 변환 과정**

오프셋은 동일하다(즉, 변환되지 않는다)는 것에 주의하라. 오프셋은 페이지 *내에서의* 우리가 원하는 위치를 알려 주기 때문이다. 최종적으로 계산된 물리 주소는 1110101(십진수 117)이며, 이곳이 탑재할 데이터가 저장된 정확한 위치이다(그림 21.2).

이 개요를 염두에 두고, 이제 페이징에 관해 몇 가지 기본적인 질문을 해 보자(그리고 대답도 할 수 있길 바라면서). 페이지 테이블은 어디에 저장되는가 페이지 테이블의 내용은 무엇이며 테이블의 크기는 얼마인가? 페이징은 시스템을 (너무) 느리게 만들지는

않는가? 이들과 다른 질문에 대해서도 이제부터 부분적이라도 대답할 것이다. 계속 읽기 바란다.

21.2 페이지 테이블은 어디에 저장되는가

페이지 테이블은 매우 커질 수 있다. 우리가 이전에 논의했던 작은 세그멘트 테이블이나 베이스-바운드 쌍에 비해서 말이다. 예를 들어, 4 KB 크기의 페이지를 가지는 전형적인 32비트 주소 공간을 상상해 보자. 이 가상 주소는 20비트 VPN과 12비트 오프셋으로 구성된다(1 KB 페이지 크기를 위해 10비트가 필요하며, 4 KB 페이지를 위해서는 두 비트만 추가하면 된다).

20비트 VPN은 운영체제가 각 프로세스를 위해 관리해야 하는 변환의 개수가 2^{20}이라는 것을 의미한다(어림잡아 백만이다). 물리 주소로의 변환 정보와 다른 필요한 정보를 저장하기 위하여 **페이지 테이블 항목(page table entry, PTE)** 마다 4바이트가 필요하다고 가정하면, 각 페이지 테이블을 저장하기 위하여 4 MB의 메모리가 필요하게 된다. 이것은 꽤 커다란 크기이다. 프로세스 100개가 실행 중이라고 가정하자. 주소 변환을 위해서 운영체제가 400 MB의 메모리를 필요로 하는 것을 의미한다. 컴퓨터가 Giga byte 단위의 메모리를 가지고 있는 현재라도 변환을 위해서 이런 큰 청크를 사용한다는 것은 매우 비정상적이다. 64비트 주소 공간을 위한 페이지 테이블의 크기가 얼마나 클지에 대해서는 생각하기조차 싫다.

페이지 테이블이 매우 크기 때문에 현재 실행 중인 프로세스의 페이지 테이블을 저장할 수 있는 회로를 MMU 안에 유지하지 않을 것이다. 대신 각 프로세스의 페이지 테이블을 *메모리에* 저장한다. 당분간 페이지 테이블은 운영체제가 관리하는 물리 메모리에 상주한다고 가정하자. 나중에 운영체제 메모리 자체의 많은 부분이 가상화될 수 있다는 것을 알게 될 것이다. 페이지 테이블은 운영체제 가상 메모리에 저장할 수 있고 심지어 디스크에 스왑될 수 있다. 그러나 현재로선 매우 혼란스럽기 때문에 일단 무시한다. 그림 21.4에 운영체제 메모리 영역(PFN 0)에 페이지 테이블이 존재한다. 그림에 작은 변환 정보들이 보이는가?

21.3 페이지 테이블에는 실제 무엇이 있는가

페이지 테이블 구성에 대해 살펴보자. 페이지 테이블은 가상 주소(또는 실제로는 가상 페이지 번호)를 물리 주소(물리 프레임 번호)로 매핑(mapping)하는 데 사용되는 자료 구조이다. 임의의 자료 구조도 사용 가능하다. 가장 간단한 형태는 **선형 페이지 테이블(linear page table)**이다. 단순한 배열이다. 운영체제는 원하는 물리 프레임 번호(PFN)를 찾기 위하여 가상 페이지 번호(VPN)로 배열의 항목에 접근하고 그 항목의 페이지 테이블 항목(PTE)을 검색한다. 지금은 이 간단한 선형 구조를 사용한다고 가정한다. 이후의 장에서 페이징과 관련된 문제를 해결하기 위한 고급 자료 구조를 사용할 것이다.



〈그림 21.4〉 **예: 커널 물리 메모리에 존재하는 페이지 테이블**

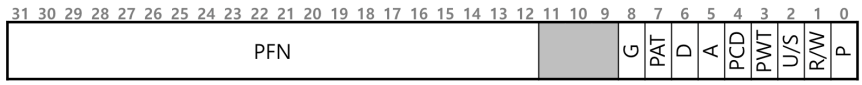
각 PTE에는 심도있는 이해가 필요한 비트들이 존재한다. **Valid bit**는 특정 변환의 유효 여부를 나타내기 위하여 포함된다. 예를 들어, 프로그램이 실행을 시작할 때 코드와 힙이 주소 공간의 한쪽에 있고 반대쪽은 스택이 차지하고 있을 것이다. 그 사이의 모든 미사용 공간은 **무효(invalid)**로 표시되고, 프로세스가 그런 메모리를 접근하려고 하면 운영체제에 트랩을 발생시킨다. 운영체제는 그 프로세스를 종료시킬 확률이 높다. Valid bit는 할당되지 않은 주소 공간을 표현하기 위해 반드시 필요하다. 주소 공간의 미사용 페이지를 모두 표시함으로써 이러한 페이지들에게 물리 프레임을 할당할 필요를 없애 대량의 메모리를 절약한다.

페이지가 읽을 수 있는지, 쓸 수 있는지, 또는 실행될 수 있는지를 표시하는 **protection bit**가 있다. Protection bit가 허용하지 않는 방식으로 페이지에 접근하려고 하면 운영체제에 트랩을 생성한다.

중요하지만 당장은 다루지 않는 몇 가지 다른 비트가 있다. **Present bit**는 이 페이지가 물리 메모리에 있는지 혹은 디스크에 있는지(즉, **스왑 아웃**되었는지) 가리킨다. 이런 기본적인 기법들은 나중에 물리 메모리보다 더 큰 주소 공간을 지원하기 위하여 주소 공간의 일부를 **스왑**하는 방법에 대해 배울 때 좀 더 자세히 이해할 것이다. 스와핑은 운영체제가 드물게 사용되는 페이지를 디스크로 이동시켜 물리 메모리를 비울 수 있게 한다. **dirty bit** 또한 일반적인데, 메모리에 반입된 후 페이지가 변경되었는지 여부를 나타낸다.

reference bit(또는 **accessed bit**)는 때때로 페이지가 접근되었는지를 추적하기 위해 사용된다. 또한, 어떤 페이지가 인기가 있는지 결정하여 메모리에 유지되어야 하는 페이지를 결정하는 데에도 유용하다. 이 정보는 페이지 교체에 매우 중요하다. **페이지 교체**에 대해서는 이후의 장에서 매우 자세하게 다룬다.

그림 21.5는 x86 아키텍처의 페이지 테이블 항목을 보여 준다 [Int09]. 이 항목은 다음과 같은 비트들을 가진다. Present bit(P), 이 페이지에 쓰기가 허용되는지 결정하



〈그림 21.5〉 **x86 페이지 테이블 항목(PTE)**

는 읽기/쓰기 비트(R/W), 사용자 모드 프로세스가 페이지에 액세스 할 수 있는지를 결정하는 사용자/슈퍼바이저 비트(U/S), 이 페이지에 대한 하드웨어 캐시의 동작을 결정하는 몇몇 비트(PWT, PCD, PAT 및 G), reference bit(A)와 dirty bit(D), 그리고 마지막으로 페이지 프레임 번호(PFN) 자체.

x86 페이징 지원에 대한 자세한 내용은 Intel Architecture Manuals [Int09]를 참조하기 바란다. 한 가지 미리 숙지할 사항이 있다. 매뉴얼을 읽는 것은 (매우 유익하고 운영체제에서 페이지 테이블을 사용하는 코드를 작성하는 사람에게는 반드시 필요한 작업이지만) 처음에는 쉽지 않다. 인내와 열정이 있어야 한다.

21.4 페이징: 너무 느림

페이지 테이블의 크기가 메모리 상에서 매우 크게 증가할 수 있다. 페이지 테이블로 인해 처리 속도가 저하될 수 있다. 간단한 명령어를 예로 들어 보자.

```
movl 21, %eax
```

주소 21에 대한 참조만 고려하고 명령어 반입에 대해서는 고려하지 않기로 하자. 이 예에서 하드웨어가 주소 **변환**을 담당한다고 가정한다. 원하는 데이터를 가져 오기 위해, 먼저 시스템은 가상 주소(21)를 정확한 물리적 주소(117)로 변환해야 한다. 주소 117에서 데이터를 반입하기 전에 시스템은 프로세스의 페이지 테이블에서 적절한 페이지 테이블 항목을 가져와야 하고, 변환을 수행한 후, 물리 메모리에서 데이터를 탑재한다.

이렇게 하기 위해서, 하드웨어는 현재 실행 중인 프로세스의 페이지 테이블의 위치를 알아야 한다. 당분간 하나의 **페이지 테이블 베이스 레지스터(page table base register)**가 페이지 테이블의 시작 주소(물리 주소)를 저장한다고 가정한다. 원하는 PTE의 위치를 찾기 위해 하드웨어는 다음과 같은 연산을 수행한다.

```
VPN = (VirtualAddress & VPN_MASK) >> SHIFT
PTEAddr = PageTableBaseRegister + (VPN * sizeof(PTE))
```

이 예제에서 VPN_MASK는 `0x30`(16진수 `30` 또는 이진수 `110000`)으로 설정되고, 전체 가상 주소에서 VPN 비트만 골라낸다. SHIFT는 4로 설정되고(오프셋 비트 수), 올바른 정수 가상 페이지 번호를 형성하기 위해 VPN 비트를 오른쪽으로 이동시킨다. 가상 주소 21(`010101`)을 마스킹 하면 `010000`이 되고; 쉬프트는 01 또는 우리가 원하는 가상 페이지 1로 변환한다. 우리는 이 값을 페이지 테이블 베이스 레지스터가 가리키는 PTE 배열에 대한 인덱스로 사용한다.

이 물리 주소가 알려지면 하드웨어는 메모리에서 PTE를 반입할 수 있고, PFN을 추출하고, 가상 주소의 오프셋과 연결하여 원하는 물리 주소를 만든다. 구체적으로, PFN

을 SHIFT 만큼 왼쪽으로 쉬프트 하고 오프셋과 논리적 OR 연산을 하여 최종 주소를 형성한다고 생각할 수 있다.

```
offset = VirtualAddress & OFFSET_MASK
PhysAddr = (PFN << SHIFT) | offset
```

마지막으로, 하드웨어는 메모리에서 원하는 데이터를 가져와서 eax 레지스터에 넣을 수 있다. 이제 프로그램은 메모리로부터 값을 성공적으로 탑재하였다!

정리를 위하여, 각 메모리 참조 시 일어나는 세부 동작을 살펴보자. 그림 21.6은 기본적인 방식을 보여준다. 모든 메모리 참조에 대해(명령어 반입이건, `load` 또는 `store` 명령어의 실행이든) 먼저 페이지 테이블에서 변환 정보를 반입해야 하기 때문에 반드시 한 번의 추가적인 메모리 참조가 필요하다. 엄청난 양의 작업이다. 메모리 참조는 비용이 비싸고 이 경우에 프로세스는 2배 이상 느려진다.

```
1  // 가상 주소에서 VPN 추출
2  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
3  // 페이지 테이블 항목(PTE)의 주소 형성
4  PTEAddr = PTBR + (VPN * sizeof(PTE))
5  // PTE 반입
6  PTE = AccessMemory(PTEAddr)
7  // 프로세스가 페이지를 접근할 수 있는지 확인
8  if (PTE.Valid == False)
9      RaiseException(SEGMENTATION_FAULT)
10 else if (CanAccess(PTE.ProtectBits) == False)
11     RaiseException(PROTECTION_FAULT)
12 else
13     // 접근 가능하면 물리 주소 만들고 값 가져오기
14     offset = VirtualAddress & OFFSET_MASK
15     PhysAddr = (PTE.PFN << PFN_SHIFT) | offset
16     Register = AccessMemory(PhysAddr)
```

〈그림 21.6〉 **페이징을 사용한 메모리 접근**

이제 우리가 반드시 해결해야 할 *두 개의* 진짜 문제를 알게 되었다. 하드웨어와 소프트웨어의 신중한 설계 없이는 페이지 테이블로 인해 시스템이 매우 느려질 수 있으며 너무 많은 메모리를 차지한다. 페이징이 메모리 가상화에 필요한 훌륭한 해결책처럼 보이지만 먼저 이 두 가지 중요한 문제가 해결되어야 한다.

21.5 메모리 트레이스

마치기 전에, 간단한 메모리 액세스 예를 통해 페이징을 사용했을 때 발생하는 모든 메모리 접근을 살펴보자. 우리가 관심있는 코드는(파일 `array.c`의 C 언어로 된) 다음과 같다.

```
1 int array[1000];
2 ...
3 for (i = 0; i < 1000; i++)
4     array[i] = 0;
```

우리는 `array.c`를 컴파일하고 다음과 같이 실행한다.

여담: 자료 구조—페이지 테이블

현대 운영체제의 메모리 관리 서브시스템에서 가장 중요한 자료 구조 중 하나는 **페이지 테이블**이다. 일반적으로, 페이지 테이블은 **가상-물리 주소 변환**(**virtual-to-physical address translation**)을 저장하여 주소 공간의 각 페이지의 물리 메모리 위치를 알 수 있게 한다. 각 주소 공간은 이런 변환을 필요로 하기 때문에 페이지 테이블은 프로세스마다 하나씩 존재한다. 페이지 테이블의 정확한 구조는 하드웨어(예전 시스템)에 의해 결정되거나 융통성 있게 운영체제에 의해 관리된다(현대 시스템).

```
prompt> gcc -o array array.c -Wall -O
prompt> ./array
```

물론, 이 코드(단순히 배열을 초기화한다)가 어떤 메모리 접근을 생성할지에대해서 진짜로 이해하려면 우리는 몇 가지 사실을 더 알아야 한다(또는 가정해야 한다). 먼저, 루프 안에서 배열을 초기화하기 위해 어떤 어셈블리 명령어를 사용하는지 보기 위하여 결과 이진 파일을 **디스어셈블**(**disassemble**) 해야만 한다(Linux에서는 `objdump`를, Mac에서는 `otool`을 사용하여). 여기 결과 어셈블리 코드가 있다.

```
0x1024 movl $0x0,(%edi,%eax,4)
0x1028 incl %eax
0x102c cmpl $0x03e8,%eax
0x1030 jne 0x1024
```

x86을 조금 알고 있다면, 위 코드는 이해하기 아주 쉽다[2]. 첫 번째 명령어는 값 `0($0x0)`을 가상 메모리 주소로 옮긴다. 0 값이 저장될 가상 메모리 주소는 `%edi`의 값을 `%eax`의 4배에다 더해서 계산된다. `%edi`는 배열의 시작 주소를 저장하고, `%eax`는 배열 인덱스(`i`)를 저장한다. 배열이 정수 배열이고 정수의 크기는 4바이트이기 때문에 4를 곱한다.

두 번째 명령어는 `%eax`에 저장된 배열 인덱스를 증가시키고, 세 번째 명령어는 `%eax`의 값과 16진수 `0x03e8` 또는 십진수 `1000`을 비교한다. 비교 결과 아직 두 값이 같지 않다면(`jne` 명령어가 검사하는 것), 네 번째 명령어는 루프의 상단으로 다시 분기한다.

이 명령어 시퀀스가 (가상 및 물리 수준 모두에서) 어떤 메모리 접근을 생성하는지 이해하기 위해서, 코드와 배열의 가상 메모리의 주소와 페이지 테이블의 위치에 대해서 몇 가지 가정을 해야 한다.

이 예에서 크기가 64 KB인(비현실적으로 적은) 가상 주소 공간을 가정한다. 또 페이지 크기는 1 KB로 가정한다.

우리가 알아야 할 것은 페이지 테이블의 내용과 위치이다. 선형(배열 기반) 페이지 테이블이고 물리 주소 1 KB (1024)에 위치한다고 가정하자.

이 예에서 신경 써야 할 몇 개의 페이지 테이블의 페이지가 있다. 첫째, 코드가 상주하는 가상 페이지가 있다. 페이지 크기가 1 KB이기 때문에, 가상 주소 1,024는

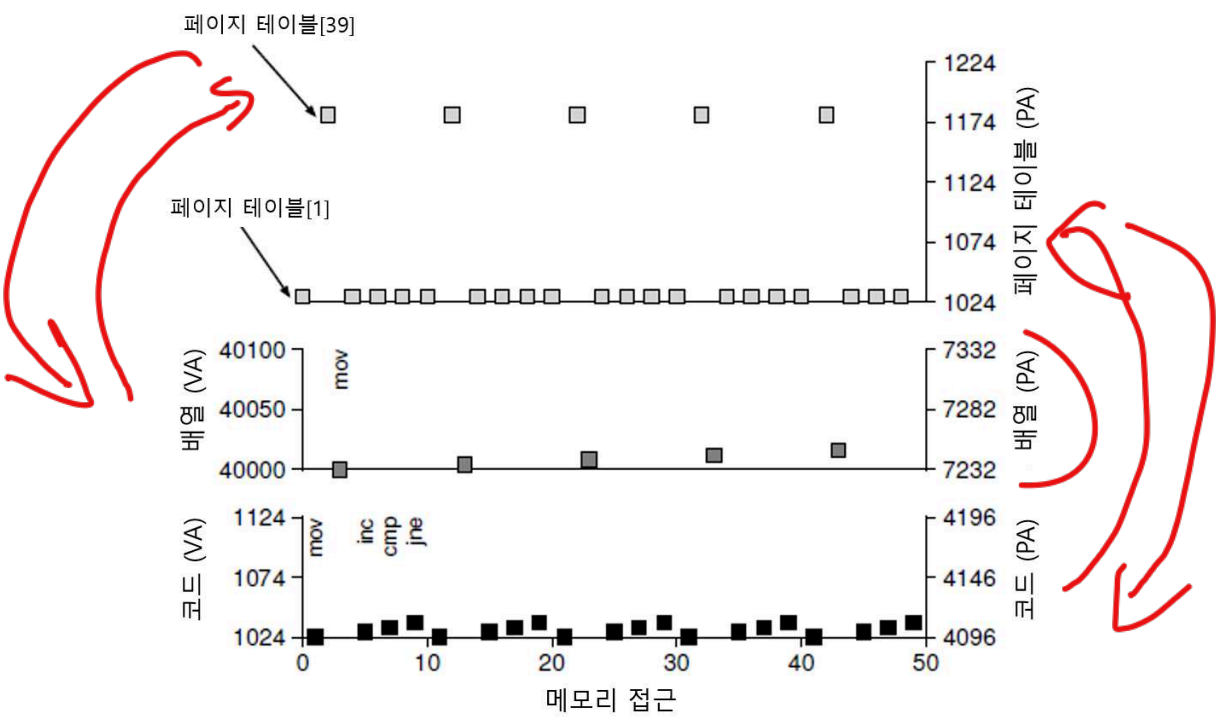
2) 여기서 약간의 부정행위를 하고 있다. 간단하게 하기 위해 각 명령어의 길이는 4바이트라고 가정한다. 실제로는 x86은 가변 길이 명령어를 사용한다.

가상 주소 공간의 두 번째 페이지(VPN=1, VPN=0이 첫 번째 페이지이기 때문에)에 상주한다. 이 가상 페이지가 물리 프레임(4)에 매핑된다고 가정하자(VPN 1 → PFN 4).

다음으로 배열 자체가 있다. 크기는 4000바이트(1000개의 정수)이고, 가상 주소 40000에서 44000까지(마지막 바이트 포함하지 않음) 존재한다고 가정한다. 이 범위에 해당하는 가상 페이지는 VPN=39 … VPN=42이다. 우리는 이 페이지에 대한 매핑이 필요하다. 이 가상-대-물리 주소 매핑이 다음과 같다고 가정하자: (VPN 39 → PFN7), (VPN 40 → PFN 8), (VPN 41 → PFN 9), (VPN 42 → PFN 10).

이제 프로그램의 메모리 참조를 살펴볼 준비가 되었다. 프로그램이 실행되면, 각 명령어의 반입 시에 메모리가 두 번 참조한다: 명령어의 위치를 파악하기 위해 페이지 테이블 접근 한 번 그리고 명령어 자체에 한 번. 게다가 `mov` 명령어는 메모리 참조를 한 번 한다. 이 명령도 먼저 페이지 테이블 접근 한 번(배열의 가상 주소를 올바른 물리 메모리 주소로 변환하기 위하여), 다음 배열 자체를 접근하기 위하여 한 번의 참조가 필요하다.

처음 다섯 번의 루프 반복에 대해 전체 과정이 그림 21.7에 나와 있다. 가장 아래쪽 그래프가 명령어 메모리 참조다(가상 주소가 왼쪽, 물리 주소가 오른쪽). 중앙의 그래프는 배열에 대한 접근이다(가상 주소가 왼쪽, 물리 주소가 오른쪽). 마지막으로, 맨 위의



〈그림 21.7〉 **가상(및 물리) 메모리 추적**

페이지 테이블을 읽어 물리 메모리를 읽는다

그래프는 페이지 테이블 메모리 접근이다(물리 주소만, 이 예에서 페이지 테이블은 물리적 메모리만 존재하기 때문에). 전체 트레이스에서, x축은 루프가 처음 다섯 번 반복되는 과정에서의 메모리 접근을 보인다. 루프당 10번의 메모리 접근이 존재한다. 네 번의 명령어 반입, 한 번의 메모리 갱신, 그리고 이러한 네 번의 반입과 한 번의 명시적인 갱신을 위한 주소 변환을 위한 총 다섯 번의 페이지 테이블 접근 등이다.

이 그래프를 이해할 수 있는지 확인하라. 구체적으로 처음 다섯 번의 루프를 넘어서 계속 실행되면 무엇이 바뀌는가? 어떤 새로운 메모리가 접근되는가? 어떻게 그것을 알아낼 수 있을까?

이것은 가장 간단한 예(몇 줄 안되는 C 코드)이지만, 실제 응용 프로그램의 메모리 동작이 얼마나 복잡한 일인지 알았을 것이다. 걱정하지 마라. 앞으로 더 복잡해진다. 우리가 앞으로 소개할 기법은 기존 기법들을 더 복잡하게 할 것이다. 미안![3]

21.6 요약

메모리 가상화에 대한 해결책으로 **페이징**을 소개하였다. 페이징은 이전 방식에 비해 많은 장점을 가지고 있다(세그멘테이션 등과 같은). 먼저 페이징은 (설계에 의해) 메모리를 고정 크기의 단위로 나눈다. 둘째, 가상 주소 공간의 드문 사용을 허용한다.

페이징을 제대로 구현하지 못하면 컴퓨터가 매우 느려지고(페이지 테이블 접근을 위한 많은 추가적 접근) 뿐만 아니라 메모리 낭비(유용한 응용 데이터 대신 페이지 테이블로 가득 참)까지 초래한다. 제대로 동작하는 페이징 시스템 고안을 위해서는 더 많은 노력을 해야 한다. 다음 두 장은 어떻게 할 수 있는지 보여줄 것이다.

3) 우리가 진짜로 미안한 건 아니다. 그러나 말이 된다면 미안하지 않은 것에 대해 미안하다.

참고 문헌

[Int09] **"Intel 64 and IA-32 Architectures Software Developer's Manuals"**
Intel
URL: `http://www.intel.com/products/processor/manuals`
특히 "Volume 3A: System Programming Guide Part 1"과 "Volume 3B: System Programming Guide Part 2"에 관심을 두라.

[Kil+62] **"One-level Storage System"**
T. Kilburn, D.B.G. Edwards, M.J. Lanigan, and F.H. Sumner
IRE Trans. EC-11, 2 (1962), pp. 223-235
(Reprinted in Bell and Newell, "Computer Structures: Readings and Examples" McGraw-Hill,New York, 1971).
Atlas는 메모리를 고정 크기의 페이지로 나누는 아이디어에 관해 선도적이었고 많은 의미에서 현대 컴퓨터 시스템에서 볼 수 있는 메모리 관리 형식의 초창기 형태였다.

[Lav78] **"The Manchester Mark I and atlas: a historical perspective"**
S.H. Lavington
Communications of the ACM archive Volume 21, Issue 1 (January 1978), pp. 4-12 Special issue on computer architecture
이 논문은 몇 가지 중요한 컴퓨터 시스템 개발의 역사의 일부를 회고한다. 미국에서는 때때로 잊어버리지만 이 새로운 아이디어의 대부분은 해외에서 왔다.

숙제

이 과제에서는 `paging-linear-translate.py`라는 간단한 프로그램을 사용하게 될 것이다. 이 프로그램을 이용하여 선형 페이지 테이블을 이용한 단순한 가상-대-물리 주소 변환이 어떻게 작동하는지를 이해하고 있는지확인할 수 있다. 자세한 사항은 README를 참조하시오.

문제

1. 변환을 하기 전에 매개변수가 주어졌을 때 선형 페이지 테이블이 크기를 변경하는 방법을 연구하기 위해 시뮬레이터를 사용해 보자. 매개변수가 바뀜에 따라 선형 페이지 테이블의 크기를 계산하시오. 추천하는 입력은 아래와 같다. `-v` 플래그를 사용하여 얼마나 많은 페이지 테이블 항목이 채워지는지 알 수 있다.

   먼저, 주소 공간이 커짐에 따라 선형 페이지 테이블 크기의 변화를 이해하기 위하여:

   ```
   paging-linear-translate.py -P 1K -a 1m -p 512m -v -n 0
   paging-linear-translate.py -P 1K -a 2m -p 512m -v -n 0
   paging-linear-translate.py -P 1K -a 4m -p 512m -v -n 0
   ```

   그런 다음 페이지 크기가 커짐에 따라 선형 페이지 테이블 크기의 변화를 이해하기 위하여:

   ```
   paging-linear-translate.py -P 1k -a 1m -p 512m -v -n 0
   paging-linear-translate.py -P 2k -a 1m -p 512m -v -n 0
   paging-linear-translate.py -P 4k -a 1m -p 512m -v -n 0
   ```

   어느 것 하나라도 실행하기 전에 예상 경향에 대해 생각해 보라. 주소 공간이 커지면 페이지 테이블 크기는 어떻게 변하는가? 페이지 크기가 커지는 경우는? 일반적으로 큰 페이지 크기를 사용하면 안 되는 이유는 무엇인가?

2. 자 이제 변환을 해 보자. 몇 가지 작은 예부터 시작해서 `-u` 플래그를 사용하여 주소 공간에 할당된 페이지 개수를 변경해 보라. 예를 들면:

   ```
   paging-linear-translate.py -P 1k -a 16k -p 32K -v -u 0
   paging-linear-translate.py -P 1k -a 16k -p 32K -v -u 25
   paging-linear-translate.py -P 1k -a 16k -p 32K -v -u 50
   paging-linear-translate.py -P 1k -a 16k -p 32K -v -u 75
   paging-linear-translate.py -P 1k -a 16k -p 32K -v -u 100
   ```

   각 주소 공간에 할당된 페이지의 비율을 높인다면 어떤 일이 벌어지는가?

3. 그러면 몇 가지 무작위 배정을 시도하자. 몇 가지 다른 (그리고 때로는 정말 어이 없는) 주소 공간 매개변수를 사용해 보라:

   ```
   paging-linear-translate.py -P 8  -a 32   -p 1024 -v -s 1
   paging-linear-translate.py -P 8K -a 32k  -p 1m   -v -s 2
   paging-linear-translate.py -P 1m -a 256m -p 512m -v -s 3
   ```

이 조합 중 어느 조합이 비현실적인가? 이유는?

4. 다른 문제를 시험해 보기 위해 프로그램을 사용하라. 프로그램이 더 이상 작동하지 않는 제약을 찾을 수 있는가? 예를 들어, 만약 주소 공간의 크기가 물리 메모리보다 *크다면* 어떤 일이 벌어지는가?

22

페이징: 더 빠른 변환 (TLB)

페이징은 상당한 성능 저하를 가져올 수 있다. 페이징은 프로세스 주소 공간을 작은 고정된 크기(즉, 페이지)로 나누고 각 페이지의 실제 위치(매핑 정보, mapping information)를 메모리에 저장한다. 매핑 정보를 저장하는 자료 구조를 페이지 테이블이라 한다. 매핑 정보 저장을 위해 큰 메모리 공간이 요구된다. 무엇보다 중요한 것은 가상 주소에서 물리 주소로의 주소 변환을 위해 메모리에 존재하는 매핑 정보를 읽어야 한다는 사실이다. 페이지 테이블 접근을 위한 메모리 읽기 작업은 엄청난 성능 저하를 유발한다. 모든 로드/스토어 (load/store) 명령어 실행이 추가적인 메모리 읽기를 수반하는 상황을 가정해 보라. 정말 머리 아픈 일이 아닐 수 없다. 우리의 질문은 다음과 같다.

핵심 질문: 주소 변환 속도를 어떻게 향상할까
주소 변환을 어떻게 빨리할 수 있을까? 페이징에서 발생하는 추가 메모리 참조를 어떻게 피할 수 있을까? 어떤 하드웨어가 추가로 필요하지? 운영체제가 어떤 식으로 개입해야 할까?

운영체제의 실행 속도를 개선하려면, 도움이 필요하다. 대부분의 경우 하드웨어로부터 도움을 받는다. 하드웨어는 운영체제의 오랜 친구! 주소 변환을 빠르게 하기 위해서 우리는 **변환-색인 버퍼(translation-lookaside buffer)** 또는 **TLB**라고 하는 부르는 것(역사적인 이유는 [CP78]을 참고)을 도입한다[CG68; Cou95]. TLB는 칩의 **메모리 관리부(memory-management unit, MMU)**의 일부다. 자주 참조되는 가상 주소-실주소 변환 정보를 저장하는 하드웨어 캐시이다. **주소-변환 캐시(address-translation cache)**가 좀 더 적합한 명칭이다. 가상 메모리 참조 시, 하드웨어는 먼저 TLB에 원하는 변환 정보가 있는지를 확인한다. 만약 있다면 페이지 테이블(모든 변환 정보를 갖고 있음)을 통하지 *않고* 변환을 (빠르게) 수행한다. 실질적으로 TLB는 페이징 성능을 엄청나게 향상시킨다. TLB를 도입함으로써, 페이징이 "사용 가능"한 가상 메모리 기법이 된다[Cou95].

22.1 TLB의 기본 알고리즘

그림 22.1은 가상 주소 변환이 이루어지는 과정을 대략적으로 나타내고 있다. 그림 22.1에서는 주소 변환부가 단순한 **선형 페이지 테이블**(**linear page table**, 즉, 배열로 이루어진 페이지 테이블)과 **하드웨어로 관리되는 TLB**(하드웨어가 페이지 테이블 접근에 대한 대부분의 책임을 관리한다. 아래에 더 자세히 설명한다)로 구성되어 있다.

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)   // TLB 히트
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else // TLB 미스
11     PTEAddr = PTBR + (VPN * sizeof(PTE))
12     PTE = AccessMemory(PTEAddr)
13     if (PTE.Valid == False)
14         RaiseException(SEGMENTATION_FAULT)
15     else if (CanAccess(PTE.ProtectBits) == False)
16         RaiseException(PROTECTION_FAULT)
17     else
18         TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
19         RetryInstruction()
```

〈그림 22.1〉 **TLB 제어 흐름 알고리즘**

하드웨어 부분의 알고리즘은 다음과 같이 동작한다. 먼저, 가상 주소(그림 22.1에서 1번 라인)에서 가상 페이지 번호(virtual page number, VPN)을 추출한 후, 해당 VPN의 TLB 존재 여부를 검사한다(라인 2). 만약 존재하면 **TLB 히트**이고 TLB가 변환 값을 갖고 있다는 것을 뜻한다. 성공! 이제 해당 TLB 항목에서 페이지 프레임 번호(page frame number, PFN)를 추출할 수 있다. 해당 페이지에 대한 접근 권한 검사가 성공하면(4번 라인), 그 정보를 원래 가상 주소의 오프셋과 합쳐서 원하는 물리 주소(PA)를 구성하고, 메모리에 접근할 수 있다(5-7번 라인).

TLB에 변환 정보가 존재하지 않는다면 (**TLB 미스**) 할 일이 많다. 이 예제에서는 하드웨어가 변환 정보를 찾기 위해서 페이지 테이블에 접근하며(11-12번 라인), 프로세스가 생성한 가상 메모리 참조가 유효하고 접근 가능하다면(13, 15번 라인), 해당 변환 정보를 TLB로 읽어들인다(18번 라인). 매우 시간이 많이 소요되는 작업이다. 페이지 테이블 접근을 위한 메모리 참조 때문이다(12번 라인). TLB가 갱신되면 하드웨어는 명령어를 재실행한다. 이번에는 TLB에 변환 정보가 존재함으로, 메모리 참조가 빠르게 처리된다.

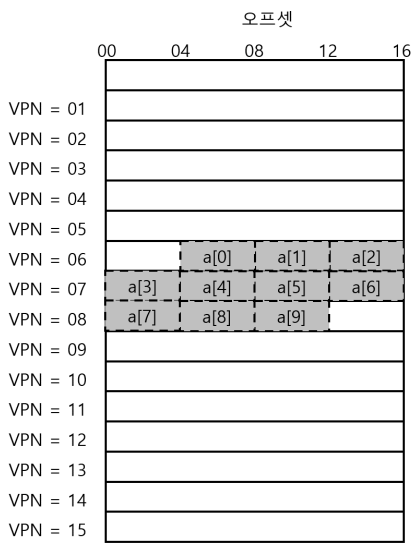
모든 캐시의 설계 철학처럼, TLB 역시 "주소 변환 정보가 대부분의 경우 캐시에 있다"(즉, 캐시에서 히트)라는 가정을 전제로 만들어졌다. TLB는 프로세싱 코어와 가까운 곳에 위치하고 있고, 매우 빠른 하드웨어로 구성되기 때문에, 주소 변환 작업은 그다지 부담스러운 작업이 아니다. 미스가 발생하는 경우, 페이징 비용이 커진다. 페이지 테이블을 접근하여 변환 정보를 찾아야 한다. 메모리 참조(또는 페이지 테이블이

복잡하다면 더 많이)가 추가된다. 이 상황이 자주 일어나면 프로그램은 상당히 느려지게 된다. 메모리 접근 연산은 다른 CPU연산(예, 더하기, 곱하기 등)에 비해 매우 시간이 오래 걸린다. TLB 미스가 많이 발생할수록 메모리 접근 횟수가 많아진다. TLB 미스가 발생하는 경우를 최대한 피해야 한다.

22.2 예제: 배열 접근

TLB 작동 과정을 좀 더 명확히 알아보자. 간단한 가상 주소 트레이스를 대상으로 TLB로 인한 성능 개선을 알아보자. 이 예제에서는 가상 주소 100번지부터 10개의 4바이트 크기의 정수 배열이 존재한다. 가상 주소 공간은 8비트이며, 페이지 크기는 16바이트이다. 가상 주소는 4비트 VPN(16개의 가상 페이지들을 표현)과 4비트 오프셋(각 페이지는 16바이트 크기를 가짐)으로 구성된다.

그림 22.2는 16개의 페이지로 구성된 가상 주소 공간을 도식적으로 표현하고 있다. 배열의 첫 항목(`a[0]`)은 (VPN=06, 오프셋=04)에서 시작한다. 세 개의 4바이트 정수가 페이지에 들어갈 수 있다. 배열은 다음 페이지(VPN=07)에서 계속된다. `a[3]` ... `a[6]`의 네 항목이 존재한다. 배열의 마지막 세 개 항목은 (`a[7]` ... `a[9]`)는 주소 공간 (VPN=08)에 시작하는 다음 페이지에 들어 있다.



〈그림 22.2〉 **예제: 작은 주소 공간 내의 배열**

배열 원소의 합을 구하는 간단한 코드를 살펴보자. C 코드는 다음과 같다.

```
int sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i];
}
```

메모리 접근을 단순화하기 위하여 변수 `i`, `sum` 그리고 명령어를 위한 메모리 접근은 무시한다. 정수 배열에 대한 메모리 접근만 보기로 하겠다. 첫 번째 배열의 항목 (a[0])이 접근된다. 가상 주소 100번이다. 하드웨어는 VPN을 추출한다. VPN이 06번이다. 하드웨어는 TLB에서 해당 VPN을 검색한다. TLB가 완전히 초기화되어 있다 가정하자. 첫 접근이다. TLB 미스가 발생한다. 미스가 발생하면 해당 VPN 06번에 대한 물리 페이지 번호를 찾아, TLB를 갱신한다.

다음은 a[1]을 읽는다. a[1]을 읽을 때에는 상황이 좋다. TLB 히트다! 배열의 두 번째 항목은 첫 번째 항목과 같은 페이지에 존재한다. 첫 번째 항목을 읽을 때, 이미 해당 페이지를 접근하였기 때문에, 필요한 변환 정보가 이미 TLB에 탑재되어 있다. 두 번째 항목을 읽을 때에는 TLB 히트가 발생한다. a[2]에 대한 접근도 같은 식으로 TLB 사용에 성공한다. 이 항목도 a[0], a[1]과 같은 페이지에 있기 때문이다.

불행히도 a[3]을 읽을 때에는 상황이 안 좋다. TLB 미스다. 하지만, 그 다음 항목들 (a[4] ... a[6])은 메모리 내에 같은 페이지에 있기 때문에 TLB 히트가 된다.

a[7]을 접근할 때 마지막 TLB 미스가 발생한다. 하드웨어는 페이지 테이블을 참고하여 가상 페이지에 해당하는 물리 메모리를 파악한 후에 TLB를 갱신한다. 마지막 두 개의 접근(a[8]와 a[9])은 이미 갱신된 TLB에서 주소 변환 정보를 얻는다. 두 번의 TLB 히트가 발생한다.

배열 원소를 읽는 TLB 동작을 정리해 보면 **미스**, 히트, 히트, **미스**, 히트, 히트, 히트, **미스**, 히트, 히트가 된다. 히트 횟수를 총 접근 횟수로 나누어 얻는 **TLB 히트 비율**은 70%가 된다. 그렇게 높은 수치는 아니지만 (히트율이 거의 100%이면 얼마나 좋을까?), 0은 아니니까 이 또한 놀랍다. 배열이 처음으로 접근되었지만, TLB는 **공간 지역성**(**spatial locality**)으로 인해서 성능을 개선할 수가 있다. 배열의 항목들이 페이지 내에서 서로 인접해 있기 때문에, 페이지에서 첫 번째 항목을 접근할 때만 TLB 미스가 발생한다.

이 예제에서 보는 바와 같이 페이지 크기는 TLB의 효용성에 매우 중요한 역할을 한다. 페이지 크기가 두 배가 되면(16이 아니라 32바이트), TLB 미스 횟수가 더 줄어든다. 일반적인 경우 페이지는 4KB이다. 예제와 같이, 정수 배열을 연속적으로 접근하는 프로그램 같은 경우, TLB 사용은 큰 성능 개선 효과를 가져올 것이다. 페이지 접근 시 한 번의 미스만 발생하기 때문이다.

TLB 성능에 관한 마지막 주요 사항이다. 만약 예제 프로그램이 루프 종료 후에도 배열을 사용한다면, 성능은 더욱 개선될 것이다. 모든 주소 변환 정보가 TLB에 탑재되어 있기 때문이다. TLB가 모든 주소 변환 정보를 저장할 정도로 충분히 크다면, 히트, 히트, 히트, 히트, 히트, 히트, 히트, 히트, 히트, 히트를 얻는다. 이 경우에서는, **시간 지역성**(**temporal locality**)으로 인해 TLB의 히트율이 높아진다. 시간 지역성이란 한번 참조된 메모리 영역이 짧은 **시간** 내에 재 참조되는 현상을 일컫는다. 다른 캐시와 마찬가지로 TLB의 성공 여부는 프로그램의 공간 지역성과 시간 지역성 존재 여부에 달려 있다. 만약 프로그램이 공간 혹은 시간 지역성을 보이는 경우, TLB 사용 효과가 더욱 두드러지게 나타날 것이다. 실제로 많은 프로그램들이 공간 지역성이나 시간 지역성을 띄고 있다.

팁: 가능하면 캐싱을 사용하자

캐싱은 컴퓨터 시스템에서 사용되는 가장 근본적인 성능 개선 기술들 중 하나이다. "일반적인 경우를 빠르게 (make the common case fast)" 하기 위해 오랜 시간 적용되어 온 방법이다[HP06]. 하드웨어 캐시 사용의 근본 취지는 명령과 데이터 참조에 있어서 **지역성(locality)**을 활용하는 것이다. 일반적으로 지역성에는 두 가지가 있다. **시간 지역성(temporal locality)**과 **공간 지역성(spatial locality)**이 그것이다. 시간 지역성은 최근에 접근된 명령어 또는 데이터는 곧 다시 접근될 확률이 높다는 사실에 근거한다. for나 while 등의 반복문을 생각해 보자. 반복문에 사용되는 변수나 명령어들은 일정 시간 동안 반복적으로 접근된다. 공간 지역성은 프로그램이 메모리 주소 x를 읽거나 쓰면, x와 인접한 메모리 주소를 접근할 확률이 높다는 사실에 근거한다. 배열을 순차적으로 읽는 프로그램이 공간 지역성을 갖는 대표적인 예이다. 어떤 프로그램이 시간 혹은 공간 지역성을 내재하느냐 하는 것은 공식이나 규칙으로 명확히 결정할 수 있는 성질의 것이 아니다. 그저 "대충" 판단할 수밖에 없다. 이를 영어로는 Rule of Thumb이라 한다. Rule Of Thumb의 어원은 각자 찾아보도록 하자.

모든 하드웨어 캐시의 목적은 필요한 메모리 내용을 매우 빠른 CPU 칩 내의 메모리에 위치시키고, 접근 지역성을 최대한 활용하는 것이다. 이 원리는 명령어 캐시, 데이터 캐시, 그리고 주소 변환 캐시 등(TLB) 모든 하드웨어 캐시에 적용된다. CPU가 메모리 내용을 참조할 때, (느린) 메모리를 직접 접근하는 것이 아니라, 우선 캐시에 사본이 있는지 먼저 확인한다. 만약 캐시에 원하는 내용이 존재하면 프로세서는 원하는 내용을 빠르게 접근할 수 있으며(즉, 몇 CPU 사이클 내에), 메모리를 접근하기 위해 비싼 시간(수 nsec)을 쓰지 않아도 된다.

이 시점에서 한 가지 의문점이 등장할 것이다. 캐시(TLB와 같은)가 그렇게 좋다면, 왜 초대용량 캐시를 제작하여 모든 내용을 저장하지 않을까? 단순한 질문이지만, 실질적으로는 매우 복잡한 문제이다. 간단히 답을 하겠다. 빠르게 할려면 크기가 작아야 한다. 크게 만들면 느려진다. 빠른 캐시를 원한다면 빛의 속도와 같은 문제나 다른 물리적 제약 조건들이 의미가 있을 정도로 작아야만 한다. 크기가 커지면 느려질 수밖에 없다. 캐시가 "캐시"역할을 하기 위해서는 작아야만 하는 것이다. 캐시에 관한 한, 우리가 풀어야 할 숙제는 "작은" 캐시를 어떻게 "잘" 사용하는가이다.

22.3 TLB 미스는 누가 처리할까

TLB 미스의 처리는 어디서 담당할까? 두 가지 방법이 있다. 하드웨어와 소프트웨어(운영체제)이다. 과거 하드웨어는 복잡한 명령어들로 구성되어 있었다. 이를 **CISC**(complex-instruction set computers)라고 통칭했다. 하드웨어 엔지니어들은 엉뚱한 방법쓰기를 좋아하는[1] 운영체제 개발자들의 특성 때문에 그들을 완전히 신뢰할 수가 없었다. 이런 연유로 TLB 미스를 하드웨어가 처리하도록 설계했다. 이를 위해서 하드웨어가 페이지 테이블에 대한 명확한 정보를 가지고 있어야 한다. 메모리 상 위치(그림 22.1의 11

1) 역자 주: 저자는 sneaky라는 표현을 썼음.

번째 줄의 **page-table base register**를 통해서)와 *정확한 형식을* 파악하고 있어야 한다. 미스 발생 시 하드웨어는 다음과 같은 일을 한다: (i) 페이지 테이블에서 원하는 페이지 테이블 엔트리를 찾고, (ii) 필요한 변환 정보를 추출하여, (iii) TLB를 갱신한 후, (iv) TLB 미스가 발생한 명령어를 재실행한다. 인텔 x86 CPU가 **하드웨어로 관리되는 TLB**의 대표적인 예다. x86 CPU는 **멀티 레벨 페이지 테이블**(**multi-level page table**)을 사용한다(다음 장에 자세한 설명이 나와 있다). CR3 레지스터가 페이지 테이블 주소를 갖고 있다 [Int09].

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)      // TLB 히트
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset   = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else                      // TLB 미스
11     RaiseException(TLB_MISS)
```

〈그림 22.3〉 **TLB 제어 흐름 알고리즘(운영체제가 관리)**

RISC(reduced instruction set computing)는 CISC보다 최근에 등장한 컴퓨터 구조이다. MIPS R10K[Hei93]나 Sun의 SPARC v9[WG00]이 대표적 예다. RISC 기반 컴퓨터는 **소프트웨어 관리 TLB**(**software-managed TLB**)를 사용한다. RISC 기반 컴퓨터에서 TLB 미스를 처리하는 과정은 다음과 같다. TLB에서 주소 찾는 것이 실패하면, 하드웨어는 예외(exception) 시그널을 발생시킨다(그림 22.3의 11번째 줄). 예외 시그널을 받은 운영체제는 명령어 실행을 잠정 중지하고, 실행 모드를 커널 모드로 변경하여, 커널 코드 실행을 준비한다. 실행 모드를 커널 모드로 변경하는 작업의 핵심은 커널 주소 공간을 접근할 수 있도록 특권 레벨(privilege level)로 상향 조정하는 것이다. 커널 모드로 변경이 되면 **트랩 핸들러**(**trap handler**)를 실행한다. 이때 실행되는 트랩 핸들러는 TLB 미스의 처리를 담당하는 운영체제 코드이다. 이 트랩 핸들러는 페이지 테이블을 검색하여 변환 정보를 찾고, TLB 접근이 가능한 "특권" 명령어(privileged instruction)를 사용하여 TLB를 갱신한 후에 리턴한다. 트랩 핸들러에서 리턴되면 하드웨어가 명령어를 재실행한다. 트랩 핸들러가 TLB를 갱신했으므로 이제는 TLB 히트가 날 것이다.

두 가지 중요한 사항을 다시 짚어보자. 첫 번째, TLB 미스를 처리하는 트랩 핸들러는 시스템 콜 호출 시 사용되는 트랩 핸들러와의 차이가 있다. 시스템 콜 호출의 경우는 트랩 핸들러에서 리턴 후 시스템 콜을 호출한 명령어의 "다음" 명령어를 실행한다. 일반적인 프로시저 콜과 동일하다. 프로시저 콜이 리턴되면, 프로시저를 호출한 다음 라인부터 실행이 시작된다. TLB의 경우는 다르다. TLB 미스 처리의 경우, 트랩에서 리턴하면 트랩을 발생시킨 명령을 "다시" 실행해야 하며, 재실행 시에는 TLB에서 히트가 발생한다. 트랩이 발생하면 운영체제는 트랩 핸들러가 종료되었을 때 다시 실행을 계속할 명령어 주소(Program Counter 값)를 저장한다. 중요한 사실을 알 수 있다. 운영체제는

트랩 발생의 원인에 따라 현재 명령어의 PC값 혹은 다음 명령어의 PC값을 저장해야 한다.

두 번째, TLB 미스 핸들러를 실행할 때, TLB 미스가 무한 반복되지 않도록 주의를 해야 한다. TLB 미스 핸들러를 접근하는 과정에서 TLB 미스가 발생하는 상황이다. 이를 위해 다양한 해법들이 존재한다. 예를 들면, TLB 미스 핸들러를 물리 메모리에 위치시키는 것도 한 방법이다. TLB 미스 핸들러의 주소는 핸들러의 '물리' 주소로 표시된다. 이 경우 해당 TLB 미스 핸들러는 **unmap**되어 있으며 주소 변환이 필요없다. 다른 방법으로 TLB의 일부를 핸들러 코드 주소를 저장하는 데 영구히 할당하는 것이다. 이렇게 되면 TLB 핸들러는 항상 TLB에서 히트된다. 이를 **연결**(**wired**) 변환이라 한다.

TLB를 소프트웨어로 관리하는 방식의 주된 장점은 *유연성이다*. 운영체제는 하드웨어 변경없이 페이지 테이블 구조를 자유로이 변경할 수 있다. 또 다른 장점은 단순함이다. TLB 제어 흐름에서 보는 것과 같이(그림 22.3의 11번째 줄과 그림 22.1의 11-19번째 줄을 비교) 미스가 발생하였을 때 하드웨어는 별로 할 일이 없다. 예외가 발생하면 운영체제의 TLB 미스 핸들러가 나머지 일을 처리한다.

22.4 TLB의 구성: 무엇이 있나?

하드웨어 TLB의 구성을 좀 더 상세히 살펴보자. 일반적인 TLB는 32, 64, 또는 128개의 엔트리를 가지며, **완전 연관**(**fully associative**) 방식으로 설계된다. 완전 연관 방식에서 변환 정보는 TLB 내에 어디든 위치할 수 있으며, 원하는 변환 정보를 찾는 검색은 TLB 전체에서 병렬적으로 수행된다. TLB의 구성은 아래와 같다.

```
VPN | PFN | 다른비트들
```

변환 정보 저장 위치에 제약이 없도록, 각 항목마다 가상 페이지 번호(VPN)와 물리 페이지 번호(PFN)가 있다. 하드웨어 측면에서 보자면, TLB는 **완전 연관** 캐시이다. 변환 주소를 찾을 때, 하드웨어는 TLB의 각 항목을 동시에 검색한다.

TLB 항목에서 VPN과 PFN을 제외한 "다른 비트들"에 대해서도 눈여겨 볼 필요가 있다. TLB는 일반적으로 **valid bit**를 갖고 있다. 이 비트는 특정 항목이 유효한 변환 정보를 갖고 있는지 여부를 나타낸다. **보호**(**protection**) 비트라는 것도 있다. 보호 비트들은 페이지가 어떻게 접근될 수 있는지를 나타낸다. 그 쓰임새는 페이지 테이블에서와 같다. 예를 들어, 코드 페이지들에 대한 변환은 *읽기와 실행이라고* 표기가 되어 있으며 힙 페이지들은 *읽기와 쓰기라고* 표기되어 있을 수 있다. 그 외에, **주소 공간 식별자**(**address-space identifier**), **더티 비트**(**dirty bit**) 등도 있다. 좀 더 자세한 정보는 다음을 참고하자.

22.5 TLB의 문제: 문맥 교환

TLB를 사용하게 되면 프로세스 간 (주소 공간들로 인해) 문맥 교환 시, 새로운 문제가 등장한다. 구체적으로 살펴보자. TLB에 있는 가상 주소와 실제 주소 간의 변환 정보는

여담: RISC 대 CISC

1980년대에 컴퓨터 구조 분야에 큰 전투가 있었다. 전투의 한 쪽은 **복잡 명령어 컴퓨터**(**complex-instruction set computing**)를 대표하는 **CISC** 진영이었고, 다른 반대편은 **축소 명령어 컴퓨터**(**reduced instruction set computing**)를 대표하는 **RISC** 진영이었다[PS81]. RISC 측에서는 버클리의 David Patterson과 스탠퍼드의 John Hennessy가 선봉장이었다 (이 둘은 잘 알려진 책의 공동 저자이기도 하다[HP06]). 하지만, 정작 John Cocke가 RISC에 대한 초기 연구를 진행한 공로를 인정받아 튜링상을 수상하였다[CM00].

CISC 명령어 집합은 많은 수의 명령어로 구성되어 있으며, 각 명령들은 강력한 기능을 가지는 것이 특징이다. 예를 들어, 두 개의 포인터와 길이를 인자로 하여 목적한 주소로 문자열 복사하는 것을 들 수가 있다. CISC의 설계 배경은 어셈블리어 명령어로 상위 수준 작업을 실행하고, 코딩을 쉽게 하며, 코드를 간단하게 하는 데 있다.

RISC 진영의 입장은 정반대이다. RISC 진영의 핵심 논리는 "어셈블리 명령어는 컴파일러를 위한 것이다."라는 사실이다. 컴파일러는 몇 가지의 단순한 어셈블리 명령어만 있으면 고성능의 코드를 생성해 낼 수 있다. RISC 지지자들은 명령어 집합을 작고 간단하게 설계하여, 즉 하드웨어를 최대한 단순화하여(특히 마이크로코드를), 빠른 컴퓨터를 만들 것을 주장하였다.

초기에 좋은 성능의 RISC 칩들이 발표되었다[BC91]. 이들은 기술 발전에 큰 영향을 미쳤다. 많은 논문이 발표되었고, 회사도 몇 개 생겨났다(예, MIPS와 Sun). 시간이 지남에 따라 인텔과 같은 CISC 제조사들이 프로세서 핵심에 RISC 기술을 도입하였다. 예를 들어, RISC와 유사한 형식의 마이크로 명령어들을 개발하고, CISC 명령어를 마이크로 명령어로 전처리하는 파이프라인 단계들을 도입하였다. 이러한 기술 혁신과 반도체 공정 집적도의 향상에 따라, 오늘날까지도 CISC 기반 프로세스가 경쟁력을 유지할 수 있었다. 지금은, RICS와 CISC 간의 장단점 논쟁이 거의 의미가 없어졌다. 두 종류의 프로세서 모두 빠르게 동작하도록 제작된다.

그것을 탑재시킨 프로세스에서만 유효하다. 다른 프로세스들에게는 의미가 없다. 새로운 프로세스에서는 이전에 실행하던 프로세스의 변환 정보를 사용하지 않도록 주의해야 한다.

예제를 하나 살펴보자. 하나의 프로세스 (P1)가 실행 중이라고 하자. 이 프로세스는 TLB가 자신에게 유효한 변환 정보를 캐싱하고 있다고 가정한다. 즉, P1의 페이지 테이블의 내용을 갖고 있다고 가정한다. 이 예제에서는 P1의 10번째 가상 페이지가 물리 프레임 100에 매핑되어 있다고 하자.

이 예제에서는 또 다른 프로세스 P2가 있다고 가정하고 운영체제가 곧 문맥 교환을 수행하기로 결정하여 이 프로세스를 실행시키려고 한다. P2의 10번째 가상 페이지는 물리 프레임 170에 매핑되어 있다고 가정하자. 두 프로세스의 항목들이 TLB에 존재한다고 하면 TLB의 내용은 다음과 같이 될 것이다.

위의 TLB에 문제가 있다. VPN 10에 대한 변환 정보가 두 개 존재하는 것이다. 10번 VPN이 PFN 100(P1) 과 PFN 170(P2)으로 변환될 수 있다. 하지만, 어떤 프로세스를

여담: TLB valid bit ≠ 페이지 테이블 valid bit

흔한 실수 중 하나는 TLB의 valid bit와 페이지 테이블의 valid bit를 혼동하는 것이다. 페이지 테이블에서 어떤 페이지 테이블 항목(page-table entry, PTE)이 "무효"로 표시되어 있다면 해당 페이지는 프로세스에 할당되지 않았다는 것을 의미한다. 정상적인 프로그램은 해당 페이지를 접근하지 않는다. 유효하지 않은 페이지를 접근할 경우, 운영체제는 트랩을 발생시킨다. 일반적인 경우, 유효하지 않는 페이지를 접근한 프로세스는 kill 된다.

반면에, TLB의 valid bit는 TLB에 탑재되어 있는 해당 변환 정보가 유효한지를 나타내기 위해 사용된다. 예를 들어, 시스템이 시작되면 어떤 변환 정보도 아직 캐시되지 않았기 때문에 일반적으로 TLB의 모든 항목은 "무효"로 초기화되어 있다. 가상 메모리가 초기화되고, 프로그램들이 실행을 시작하여 자신의 가상 주소 공간을 접근하게 되면, TLB는 서서히 채워지게 된다. TLB는 곧 유효한 항목들로 채워지게 된다.

이제 곧 살펴보게 되겠지만, TLB의 valid bit는 문맥 전환할 때도 상당히 유용하다. 문맥 전환 시, 시스템은 모든 TLB 항목을 "무효"로 세팅하여, 새로운 프로세스가 이전 프로세스의 변환 정보를 사용하는 것을 원천적으로 차단한다.

VPN	PFN	valid	prot
10	100	1	rwx
--	---	0	---
10	170	1	rwx
--	---	0	---

위한 항목인지 알 길이 없다. TLB가 정확하고 효율적으로 멀티 프로세스 간의 가상화를 지원하기 위해서는 추가적 기능이 필요하다. 핵심은 다음과 같다.

핵심 질문: 문맥 교환 시 TLB 내용을 어떻게 관리하는가

문맥 교환 시 실행될 프로세스에게는 이전 프로세스가 사용한 TLB 정보는 의미가 없다. 하드웨어 또는 운영체제는 이러한 문제를 해결하기 위해 무엇을 해야 하는가?

이 문제는 여러 가지 해법이 있을 수 있다. 한 방법은 문맥 교환을 수행할 때 다음 프로세스가 실행되기 전에 기존 TLB 내용을 비우는 것이다. 소프트웨어 기반의 시스템에서는 특별한(그리고 특권을 갖는) 하드웨어 명령어을 사용하여 이 목적을 달성할 수 있다. 하드웨어로 관리되는 TLB는 페이지 테이블 베이스 레지스터가 변경될 때 비우기를 시작할 수 있다(운영체제는 문맥 교환을 할 때 PTBR을 어쨌든 변경해야 한다). 둘 중 어느 경우든 비우는 작업은 모든 valid bit를 0으로 설정을 하는 것이다.

문맥 교환할 때마다 TLB를 비우면, 잘못된 변환 정보를 사용하는 상황을 방지할

수 있다. 하지만, 이 방식은 공짜가 아니다. 새로운 프로세스가 실행될 때, 데이터와 코드 페이지에 대한 접근으로 인한 TLB 미스가 발생하게 된다. 문맥 교체가 빈번히 발생한다면, 이 또한 성능에 큰 부담을 가져올 수 있다.

이 부담을 개선하기 위해 몇몇 시스템에서는 문맥 교환이 발생하더라도 TLB의 내용을 보존할 수 있는 하드웨어 기능을 추가하였다. TLB 내에 **주소 공간 식별자(address space identifier, ASID)** 필드를 추가하는 것이 그것이다. ASID는 **프로세스 식별자(process identifier, PID)**와 대략적으로 유사하다. 단, ASID는 좀 더 적은 비트를 갖고 있다(예, ASID는 8비트를 갖고 있지만 PID는 32비트를 갖고 있다).

위에서 사용한 TLB 예제에 ASID 정보를 추가하면 프로세스들이 TLB의 공간을 공유할 수가 있다. 다음은 ASID 필드를 추가한 TLB의 모습을 나타낸다.

VPN	PFN	valid	prot	ASID
10	100	1	rwx	1
--	---	0	---	-
10	170	1	rwx	2
--	---	0	---	-

주소 공간 식별자를 사용할 경우, 프로세스 별로 TLB 변환 정보를 구분할 수 있다. 올바른 주소 변환을 위해서 하드웨어는 현재 어떤 프로세스가 실행 중인지 파악하고 있어야 한다. 이를 위해, 문맥 전환 시, 운영체제는 새로운 ASID 값을 정해진 레지스터에 탑재한다.

추가적으로 TLB의 두 항목이 매우 유사한 경우를 생각해 볼 수도 있다. 이 예제에서는 두 개의 다른 VPN을 갖는 두 개의 다른 프로세스들의 두 항목이 동일한 물리 페이지를 가리키고 있다.

VPN	PFN	valid	prot	ASID
10	101	1	r-x	1
--	---	0	---	-
50	101	1	r-x	2
--	---	0	---	-

이러한 경우는 두 개의 프로세스들이 하나의 페이지(예, 코드 페이지)를 공유하고 있을 때 발생할 수가 있다. 이 예제에서는 프로세스 1이 물리 페이지 101을 프로세스 2와 공유하고 있다. P1은 이 페이지를 자신의 주소 공간의 10번째 페이지에 매핑하고 있으며 P2는 자신의 주소 공간의 50번째 페이지에 매핑하고 있다. 코드 페이지를 공유하는 것은 (바이너리에서든 또는 공유 라이브러리이든) 사용되는 물리 페이지의 수를 줄일 수 있기 때문에 유용하다. 그리고 공유 페이지를 사용하면 메모리 부하도 줄일 수 있다.

22.6 이슈: 교체 정책

모든 캐시가 그러하듯이 TLB에서도 **캐시 교체**(**cache replacement**) 정책이 매우 중요하다. TLB에 새로운 항목을 탑재할 때, 현재 존재하는 항목 중 하나를 **교체** 대상으로 선정해야 한다. 어느 것을 선택해야 할까?

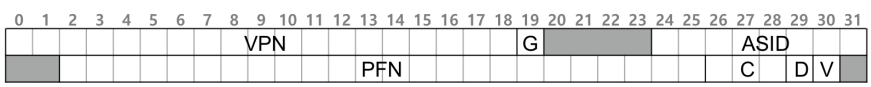
핵심 질문: TLB 교체 정책은 어떻게 설계하는가
TLB에 새로운 항목을 추가할 때 어떤 항목을 교체해야 할까? 목표는 **미스율**을 줄여 (또는 **히트 비율**을 증가시켜서) 성능을 개선하는 것이다.

이 내용은 디스크와 메모리 간의 페이지 스와핑 부분에서 상세히 다루도록 하고, 여기서는 몇 개의 일반적인 정책들만 요약하도록 하겠다. 한 가지 흔한 방법은 가장 오래 전부터 사용되었던 **최저 사용 빈도**(**least-recently-used**, **LRU**) 항목을 교체하는 것이다. LRU는 메모리 참조 패턴에서의 지역성을 최대한 활용하는 것이 목적이다. 사용되지 않은지오래된 항목일수록, 앞으로도 사용될 가능성이 작으며, 교체 대상으로 적합하다는 가정에 근거한다. 또 다른 일반적인 방법은 랜덤(random) 정책이다. 랜덤 정책에서는 교체 대상을 무작위로 정한다. 랜덤 교체 정책은 때때로 잘못된 결정을 내릴 수도 있다. 하지만, 랜덤 교체 정책은 구현이 간단하고 예상치 못한 예외 상황의 발생을 피할 수 있다는 장점이 있다. 예를 들어 LRU와 같은 "합리적인" 정책은 n 개의 변환 정보를 저장할 수 있는 TLB가 $n+1$ 페이지들에 대해서 반복문을 수행하는 프로그램에서 사용될 경우 최악의 TLB 미스를 생성하게 된다. 그 이유는 독자들이 한 번 상상해 보기 바란다. LRU는 모든 접근에 대해 미스를 유발한다. 그에 반해 랜덤의 경우에는 훨씬 더 잘 동작한다.

22.7 실제 TLB

마지막으로 실제 TLB가 어떻게 생겼는지 간략히 살펴보자. MIPS R4000을 예로 들겠다. MIPS R4000은 소프트웨어로 관리되는 TLB를 사용한다. 비교적 최근 시스템이다. MIPS의 간략화된 TLB 항목은 그림 22.4와 같다.

MIPS R4000은 32비트 주소 공간에 4KB 페이지를 지원한다. 일반적인 가상 주소 공간을 사용하기 때문에 VPN은 20비트 그리고 오프셋으로 12비트를 갖을 것이라 예측할 수 있지만, 아니다. TLB에서 보는 바와 같이 19비트가 VPN에 할당되어 있다. 전체 주소 공간의 절반만 사용자 주소 공간으로 할당되어 있기(나머지 반은 커널이 사용한다.)



〈그림 22.4〉 **MIPS의 TLB 항목**

때문에, VPN에 19비트가 할당되어 있다. 물리 프레임 번호(PFN)로 24비트가 할당되어 있다. 64 GB의 (물리) 주 메모리(2^{24}개의 4 KB 페이지들) 지원이 가능하다.

MIPS의 TLB에는 몇 가지 중요한 비트들이 더 있다. 전역(global) 비트 (G)다. 이 비트는 프로세스들 간에 공유되는 페이지들을 위해 사용된다. 전역 비트가 설정되어 있으면 ASID는 무시된다. 8비트 길이의 ASID 필드가 있다. 운영체제는 이 부분을 보고 주소 공간들을 서로 구분한다(위에서 설명한 것과 같이). 여기서 재미있는 질문을 할 수 있다. 만일 256(2^8)개 이상의 프로세스들이 동시에 실행된다면? 마지막으로 3비트 길이의 일관성(coherence, C) 비트를 볼 수 있다. 이 비트를 보고 페이지가 하드웨어에 어떻게 캐시되어 있는지를 판별한다 (이 부분은 이 글의 주제의 범위 밖의 내용이다). 더티(dirty) 비트는 페이지가 갱신되면 세팅되며(나중에 이것의 활용을 살펴볼 것이다), 유효(valid) 비트는 항목에 유효한 변환 정보가 존재하는지를 나타낸다. 그리고 페이지 마스크(page mask) 필드도 있는데 (표현은 안 되었음), 여러 개의 페이지 크기를 지원할 때 사용된다. 대용량 페이지 크기의 유용함에 대해서는 곧 배울 것이다. 마지막으로 회색 처리된 64번째 비트는 사용하지 않는다.

MIPS의 TLB들은 일반적으로 32개 또는 64개의 항목으로 구성된다. 대부분은 사용자 프로세스들이 사용한다. 몇 개는 운영체제를 위해서 예약이 되어 있다. 운영체제에 의해서 연결된(wired) 레지스터가 설정이 될 수도 있으며, 운영체제는 몇 개의 TLB를 예약할지를 하드웨어에게 알린다. 운영체제는 예약된 매핑 영역을 TLB 미스가 발생해서는 안 되는(예, TLB 미스 핸들러) 코드와 데이터를 위해서 사용한다.

MIPS의 TLB는 소프트웨어가 관리하기 때문에 TLB를 갱신하기 위한 명령어가 필요하다. MIPS는 네 개의 명령어를 제공한다. `TLBP`는 어떤 특정 변환 정보가 존재하는지 탐색하는 데 사용이 되며 `TLBR`은 TLB 항목의 내용을 레지스터로 읽어 드리는 데 사용된다. `TLBWI`는 특정 TLB 항목을 교체하며 `TLBWR`은 임의의 TLB 항목을 교체한다. 운영체제는 이 명령어들을 사용하여 TLB의 내용을 관리한다. 이 명령어들은 특별한 **실행권한**(**privileged**)을 갖고 있어야 한다. 만약 사용자 프로세스가 TLB 내용을 변경할 수 있다면 어떻게 될지 상상해 보라(힌트: 거의 무엇이든지 가능하다. 기계의 사용권한을 바꿔서 악의적으로 "운영체제"를 사용할 수 있으며 또는 태양을 없애버릴 수도 있다).

22.8 요약

지금까지 주소 변환을 더 빠르게 처리하기 위한 하드웨어 기법에 대해 살펴보았다. 칩 상의 작은 전용 TLB를 주소 변환 캐시로 사용하여 대부분의 메모리 참조들은 메인 메모리 상의 페이지 테이블을 읽지 않고도 처리가 가능하게 되었다. TLB의 사용으로 일반적인 경우에 프로그램은 메모리 가상화 기능이 없는 것과 동일한 성능을 보일 것이기 때문에 TLB는 운영체제 입장에서 훌륭한 성과일 뿐만 아니라 현대 시스템에서 페이징을 사용하기 위한 필수 요소이다.

하지만, TLB가 모든 프로그램에서 항상 제대로 작동하는 것은 아니다. 특히, 프로그램이 짧은 시간 동안 접근하는 페이지들의 수가 TLB에 들어갈 수 있는 수보다 많다면

팁: RAM은 항상 RAM이 아니다(Culler's Law)

임의-접근 메모리(random-access memory) 또는 **RAM**이라는 용어는 RAM의 어떤 부분이든 다른 부분을 접근하는 것과 같은 속도로 접근할 수 있다는 것을 암시한다. 일반적으로 RAM이 그렇게 동작한다고 생각해도 괜찮다. 왜냐하면, 하드웨어/운영체제의 TLB와 같은 기능 때문이다. 특정 메모리의 페이지를 접근하는 것이 비쌀 수도 있다. 특히, 그 페이지가 현재 TLB에 매핑되어 있지 않다면 더욱 그렇다. 그러므로 **RAM은 항상 RAM이 아니다**라고 하는 구현의 팁을 기억하는 것이 도움이 된다. 때로는, 특히 TLB가 다룰 수 있는 것보다 많은 수의 페이지들을 접근할 때에 주소 공간을 임의 접근하는 것은 심각한 성능 저하를 가져올 수가 있다. 우리의 지도 교수 중의 한 분이셨던 David Culler는 TLB가 여러 성능 문제들의 원인이라고 지적하시곤 하셨다. 그를 위하여 이 법칙의 이름을 **Culler's Law**라고 부른다.

그 프로그램은 많은 수의 TLB 미스를 발생할 것이고 느리게 동작하게 될 것이다. 이러한 현상을 **TLB 범위(TLB coverage)**를 벗어난다고 하며 특정 프로그램들에서는 중요한 문제가 될 수도 있다. 다음 장에서 다룰 한 가지 해법은 더 큰 페이지 크기를 지원하도록 하는 것이다. 더 큰 페이지들을 사용할 경우 TLB의 유효 범위가 늘어날 수가 있다. 더 큰 페이지들을 위한 지원은 **데이터베이스 관리 시스템(database management system, DBMS)**과 같은 프로그램들에 의해서 주로 사용되며, 이러한 프로그램의 자료 구조들은 클 뿐만 아니라 임의적으로 접근된다.

TLB에 관한 또 다른 이슈는 CPU 파이프라인에서 TLB 접근은 병목이 될 수 있다. **물리적으로 인덱스된 캐시(physically-indexed cache)**가 사용될 경우 특히 문제가 된다. 이 캐시에서는 주소 변환이 캐시 접근 *전에* 이루어져야 하는데, 그런 경우에는 상당히 느려질 수가 있다. 이러한 문제 때문에 *가상* 주소로 캐시를 접근하는 다양한 방법들이 고안되었며, 캐시 히트가 발생한 경우에 비싼 변환을 하지 않도록 하였다. **가상적으로 인덱스된 캐시(virtually indexed cache)**는 일부 성능 문제를 해결하지만 새로운 하드웨어 설계 문제들을 수반한다. 자세한 내용은 Wiggins가 쓴 훌륭한 개요을 참고하자[Wig03].

참고 문헌

[BC91] **"Performance from Architecture: Comparing a RISC and a CISC with Similar Hardware Organization"**
D. Bhandarkar and Douglas W. Clark
Communications of the ACM, September 1991
훌륭하고 공정하게 RISC와 CISC를 비교한 글이다. 핵심 요점은 하드웨어가 비슷하다면 RISC가 3배 정도 더 성능이 빠르다는 것이다.

[CP78] **"The architecture of the IBM System/370"**
R.P. Case and A. Padegs
Communications of the ACM. 21:1, 73-96, January 1978
*아마도 **변환 색인 버퍼**(**translation lookasdie buffer**)라는 용어를 처음 쓴 논문일 것이다. 멘체스터 대학에서 지도 시스템을 개발하던 사람들이 사용하던 **색인 버퍼**라고 하는 캐시의 역사적인 명칭에서 따왔다. 주소 변환 정보의 캐시는 이후에 **변환 색인 버퍼**로 바뀌게 되었다. 색인 버퍼라는 용어가 인기를 얻지 못하기는 하였지만, TLB라는 이름은 어떠한 이유에서 계속 사용되고 있다.*

[CM00] **"The evolution of RISC technology at IBM"**
John Cocke and V. Markstein
IBM Journal of Research and Development, 44:1/2
많은 사람들이 첫 번째의 진정한 RISC 마이크로프로세서라고 할 수 있는 IBM 801에 대한 개념과 기술들에 대한 정리이다.

[Cou95] **"The Core of the Black Canyon Computer Corporation"**
John Couleur
IEEE Annals of History of Computing, 17:4, 1995
대단히 흥미로운 이 역사적인 자료에서 Couleur는 1964년에 GE에서 일하면서 TLB를 어떻게 개발하였는지, 그리고 이후에 MIT의 프로젝트 MAC 사람들과 우연하게 협업을 하게 된 이야기를 소개하고 있다.

[CG68] **"Shared-access Data Processing System"**
John F. Couleur and Edward L. Glaser
Patent 3412382, November 1968
주소 변환 정보를 연관 메모리에 저장하는 개념을 다룬 특허이다. 이 개념은 Couleur에 따르면 1964년에 나왔다.

[Hei93] **"MIPS R4000 Microprocessor User's Manual"**
Joe Heinrich
Prentice-Hall, June 1993
URL: `http://cag.csail.mit.edu/raw/documents/R4400_Uman_book_Ed2.pdf`

[HP06] **"Computer Architecture: A Quantitative Approach"**
John Hennessy and David Patterson
Morgan-Kaufmann, 2006
컴퓨터 구조에 관한 훌륭한 책이다. 우리는 고전의 첫판에 대한 이상한 애착을 갖고 있다.

[Int09] **"Intel 64 and IA-32 Architectures Software Developer's Manuals"**
Intel, 2009
URL: `http://www.intel.com/products/processor/manuals`
특히, "Volume 3A: System Programming Guide Part 1"과 "Volume 3B: System Programming Guide Part 2"를 주의 깊게 읽어보자.

[PS81] **"RISC-I: A Reduced Instruction Set VLSI Computer"**
D.A. Patterson and C.H. Sequin
ISCA '81, Minneapolis, May 1981
RISC라는 용어를 소개한 논문으로 성능을 위해서 컴퓨터 칩들을 간단화하는 연구들의 산사태를 시작시켰다.

[Saa92] **"CPU Performance Evaluation and Execution Time Prediction Using Narrow Spectrum Benchmarking"**
Rafael H. Saavedra-Barrera
EECS Department, University of California, Berkeley Technical Report No. UCB/CSD-92-684, February 1992
URL: `www.eecs.berkeley.edu/Pubs/TechRpts/1992/CSD-92-684.pdf`
응용 프로그램의 실행 시간을 예측하는 방법에 대한 훌륭한 학위논문으로서 그 시간을 구성 요소들로 세분화하고 각 요소들의 비용을 알아내는 방식을 사용하였다. 이 연구에서 가장 흥미로운 부분은 (제5장에서 설명하고 있는) 캐시 계층의 세부적 부분들을 측정할 수 있는 도구일 것이다. 거기에 있는 훌륭한 도표들을 꼭 살펴보기 바란다.

[WG00] **"The SPARC Architecture Manual: Version 9"**
David L. Weaver and Tom Germond
SPARC International, San Jose, California, September, 2000
URL: `http://www.sparc.org/standards/SPARCV9.pdf`

[Wig03] **"A Survey on the Interaction Between Caching, Translation and Protection"**
Adam Wiggins
University of New SouthWales TR UNSW-CSE-TR-0321, August, 2003
TLB들이 하드웨어 캐시라고 하는 CPU 파이프라인의 다른 부분들과 상호작용하는 것에 대한 훌륭한 개관이다.

숙제 (측정)

이 숙제에서는 TLB를 접근하는 크기와 비용을 측정할 것이다. 이 개념은 Saavedra-Barrera의 연구[Saa92]에 기초한 것으로 이들이 만든 간단하지만 캐시 계층의 여러 측면을 측정하는 아름다운 기법을 이용한다. 그리고 이 모든 것이 아주 간단한 사용자 수준의 프로그램을 통해 할 수 있다. 좀 더 자세히 알고 싶다면 이 연구를 읽어보도록 하자.

기본 개념은 커다란 자료 구조(예, 배열) 내에서 몇 개의 페이지들을 접근하고 그 접근들에 대한 시간을 측정하는 것이다. 예를 들어서 어떤 기계의 TLB 크기는 4라고 해보자(매우 작기는 하겠지만 이 논의의 목적에 충분히 부합한다). 프로그램을 작성하였는데, 그 프로그램이 네 개 또는 그보다 적은 페이지들을 접근한다면 각 접근들은 TLB 히트가 될 것이며 상대적으로 빠를 것이다. 하지만, 다섯 개 또는 그 이상을 반복문에서 계속 접근한다면 TLB 미스를 만들어서 각 접근은 비용이 갑자기 크게 늘어나게 된다.

다음의 간단한 코드에서 배열을 한 번만 접근하는 반복문을 표현하였다.

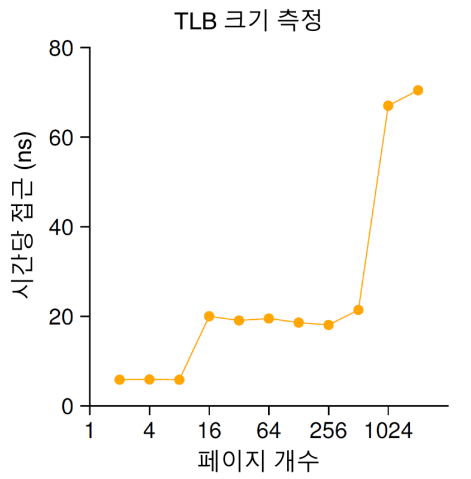
```
int jump = PAGESIZE / sizeof(int);
for (i = 0; i < NUMPAGES * jump; i += jump) {
    a[i] += 1;
}
```

이 반복문에서는 배열 `a`는 페이지당 하나의 정수 형 크기가 갱신이 되며 정해진 `NUMPAGES` 개만큼의 페이지들이 변경된다. 여러 차례 이 반복문이 수행되는 것(이 반복문을 반복하는 또 다른 반복문을 1억 번 수행하거나 또는 이 반복문이 수초 동안 실행될 수 있을 만큼의 반복을 수행한다)을 측정하는 것을 통해 각 접근이 얼마나 오랜 시간이 걸리는지 (평균적으로) 측정할 수 있다. `NUMPAGES`가 커짐에 따라서 비용이 뛰는 정도를 살펴보면 일 단계 TLB의 크기가 얼마나 큰지를 알 수 있으며 이 단계 TLB가 존재하는지를 판단할 수 있으며(존재한다면 얼마나 큰지도 함께), 그리고 TLB 히트와 미스가 성능에 어떤 영향을 줄 수 있는지에 대한 감을 잡을 수가 있다.

그림 22.5에 예시 그래프가 있다. 그래프에서 볼 수 있듯이, 몇 개의 페이지들을 접근할 때(8개 또는 그 이하) 평균 접근 시간은 약 5 nsec가 걸렸다. 16개 또는 그 이상의 페이지들이 접근되었을 때는 접근마다 약 20 nsec로 갑자기 비용이 늘어났다. 그 다음으로 비용이 늘어나는 구간은 1024개 이상의 페이지들을 접근할 때이며 각 접근은 약 70 nsec가 소요된다. 이 데이터에서 이 단계 TLB 계층이 있다고 결론지을 수 있으며 첫 번째는 상당히 작다(8개에서 16개의 항목을 담을 수 있는 것으로 보임). 두 번째는 좀 더 크지만 더 느리다(약 512개를 담을 수 있음). 일 단계 TLB의 히트들과 미스들을 비교해 보면 그 차이가 상당한 것을 알 수 있으며 약 14배 정도 차이나는 것을 볼 수 있다. TLB 성능은 중요하다!

문제

1. 시간을 재기위해서는 `gettimeofday()`를 통해서 만들 수 있는 타이머를 사용해야 할 것이다. 이 타이머는 얼마나 정확한가? 시간을 정확하게 재기 위해서는 타이머가



〈그림 22.5〉 **TLB 크기와 미스의 비용 파악하기**

얼마나 길어야 하는가?(반복문 내에서 몇 번이나 페이지를 접근해야 시간을 정확히 잴 수 있는지를 결정하는 데 도움을 줄 것이다)

2. `tlb.c` 라는 프로그램을 작성하여 각 페이지를 접근하는 데 드는 대략적인 비용을 측정해 보자. 프로그램의 입력 값은 접근해야 하는 페이지들의 수와 시도 횟수이다.

3. 선호하는 스크립트 언어(csh, python 등)를 사용하여 프로그램을 실행시키는 스크립트를 작성하여 접근하는 페이지들의 수를 1부터 수천까지 2의 배수로 증가시키도록 해 보자. 다른 기계들에서 스크립트를 실행하여 데이터를 수집하자. 신뢰할 수 있는 측정치를 얻기 위해서는 몇 번이나 시도해 봐야 하는가?

4. 그 다음으로 결과들을 사용하여 앞서 보인 그래프처럼 만들어 보자. `ploticus`와 같은 툴을 사용하면 된다. 시각화하는 것은 데이터를 좀 더 잘 이해하는 데 도움이 되는데, 왜 그런 것 같은가?

5. 한 가지 주의해야 할 것은 컴파일러 최적화이다. 컴파일러는 프로그램의 다른 부분들이 사용하지 않는 변수를 반복문을 통해 증가시키는 부분을 빼버리는 등의 온갖 교묘한 짓들을 한다. 컴파일러가 작성한 TLB 크기 추정기를 어떻게 수정하여야 컴파일러가 주 반복문을 제거하지 않도록 만들 수 있을까?

6. 또 한 가지 주의해야 할 것은 대부분의 시스템은 CPU가 여러 개이며, 각 CPU는 물론 각각의 TLB 계층을 갖고 있다. 정말 제대로 측정하기 위해서는 스케줄러가 하나의 CPU에서 또 다른 CPU로 이동시키지 못하도록 만들어서 하나의 CPU만을 사용해서 측정해야 할 것이다. 어떻게 그렇게 할 수 있을까? (힌트: 구글에서 "쓰레드 고정하기(pinning a thread)"를 검색해 보자) 이렇게 하지 않았을 때는 어떤 일이 발생할까? 그리고 코드가 한 CPU에서 다른 CPU로 이동할 때 어떤 일이 발생할까?

7. 또 다른 발생할 수 있는 문제는 초기화와 관련되어 있다. 접근하기 전에 배열 `a`를 초기화하지 않는다면, 처음 `a`의 배열을 접근할 경우 한 번도 접근한 적이 없기 때문에

매우 비싼 비용이 들 것이다. 이 부분이 시간을 재는 데에 영향을 줄까? 이러한 잠재적인 비용 요인의 균형을 잡기 위해서 무엇을 할 수 있을까?

23

페이징: 더 작은 테이블

페이징의 두 번째 문제점은 페이지 테이블의 크기이다. 페이지 테이블이 크면 많은 메모리 공간을 차지한다. 배열 형태를 가지는 선형 페이지 테이블(linear page table)을 예로 들어 보자. 전술한 바와 같이[1] 선형 페이지 테이블은 상당히 커질 수가 있다. 페이지 크기가 4 KB(2^{12} 바이트)이고, 페이지 테이블의 각 항목은 4바이트인 32비트 주소 공간(2^{32} 바이트)을 가정해 보자. 주소 공간에는 대략 백만 개($\frac{2^{32}}{2^{12}}$)의 가상 페이지가 존재할 것이다. 여기에 페이지 테이블 항목의 크기를 곱하면 (4바이트 x 백만 개), 페이지 테이블의 크기가 된다. 하나의 페이지 테이블 크기가 약 4 MB가 된다. 또 하나 잊으면 안되는 사실이 있다! 일반적으로 *각 프로세스는 자기 자신의* 페이지 테이블을 갖는다. 백 개의 프로세스가 실행 중이라면 (현대 시스템에서 드물지 않다), 페이지 테이블이 100개 존재하며, 400 MByte의 메모리가 필요하다. 이런 엄청난 메모리 부담을 해결할 수 있는 기술을 모색해 보자. 다양한 해법들이 존재한다. 핵심 질문은 아래와 같이 요약될 수 있다.

핵심 질문: 페이지 테이블을 어떻게 더 작게 만들까

단순한 배열 기반의 페이지 테이블은(흔히 선형 페이지 테이블이라고 불림) 크기가 크며 일반적인 시스템에서 메모리를 과도하게 차지한다. 어떻게 페이지 테이블의 크기를 줄일 수 있을까? 주요 개념들로는 무엇이 있는가? 새로운 자료 구조들은 어떤 비효율성을 갖는가?

23.1 간단한 해법: 더 큰 페이지

페이지 테이블의 크기를 간단하게 줄일 수 있는 방법이 한 가지 있다. 페이지 크기를 증가시키면 된다. 32비트 주소 공간에서, 이번에는 16 KB 페이지를 가정해 보자. 이제는 18비트의 VPN과 14비트의 오프셋을 갖게 된다. 각 PTE (4바이트)의 크기가 모두

1) 기억 못할 수도 있다. 페이징이라는 것이 통제가 잘 안되지 않는가? 해법을 찾아보기 전에 문제가 무엇인지 제대로 이해하고 있는지 확인을 해야 한다. 문제를 이해하면 스스로 해법도 유도해 낼 수가 있다. 여기서 다루는 문제는 명확하다. 간단한 선형(배열 기반의) 페이지 테이블은 너무 크다는 것이다.

여담 : 멀티 페이지 크기

많은 컴퓨터 구조들(예, MIPS, SPARC, x86-64)이 멀티 크기 페이지를 지원한다는 것을 기억하자. 일반적으로, 작은 (4 KB 또는 8 KB) 페이지 크기가 사용된다. 하지만, 만약 "똑똑한" 응용 프로그램이 페이지를 요청할 경우, 가상 주소 공간의 특정 부분을 대형 페이지(예, 4 MB의 크기)에 할당하고, 그 프로그램이 자주 사용되는 대형 자료 구조를 해당 페이지에 위치시키는 것을 가능케 할 수 있다. 대형 페이지를 사용하므로 TLB에서 하나의 항목만 사용한다. 이러한 대형 페이지는 데이터베이스 관리 시스템과 고성능 프로그램에서 흔히 사용된다. 멀티 페이지 크기를 사용하는 주요 이유는 페이지 테이블의 공간을 절약하려는 것은 아니다. TLB 미스를 줄이면서 프로그램이 주소 공간을 접근할 수 있도록 하기 위해서이다. 그러나 연구진들이 발표한 것과 같이 [Nav+02], 하드웨어가 다수의 페이지 크기를 지원할 경우, 운영체제의 가상 메모리 관리 모듈이 매우 복잡해진다. 대형 페이지를 간단히 사용하는 방법으로, 추가 인터페이스를 정의하여, 응용 프로그램이 대형 페이지를 직접 요청할 수 있도록 할 수 있다.

동일하다면, 페이지 테이블에 2^{18} 개의 항목이 있으며, 페이지 테이블의 총 크기는 1 MB가 된다. 기존 페이지 테이블 대비 크기가 1/4로 감소된다(당연하다. 테이블의 크기가 1/4로 감소한 것은 페이지 크기가 정확히 4배 증가했기 때문이다).

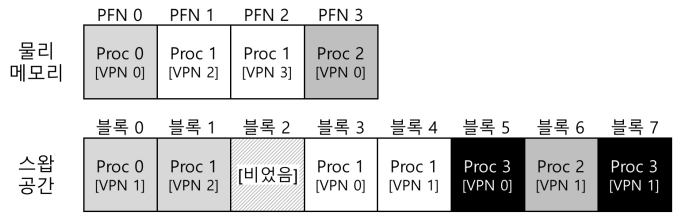
페이지 크기의 증가는 부작용을 수반한다. 가장 큰 문제는 페이지 *내부의* 낭비 공간이 증가하는 것이다. 이를 **내부 단편화**(**internal fragmentation**)라 한다 (할당된 페이지 **내부**에서 낭비가 발생하기 때문이다). 응용 프로그램이 여러 페이지를 할당받았지만, 할당받은 페이지의 일부분만 사용하는 터에, 결국 컴퓨터 시스템의 메모리가 금방 고갈되는 현상이 발생한다. 이런 연유에서, 많은 컴퓨터 시스템들이 비교적 작은 페이지들을 사용한다. 일반적으로 4 KB (x86) 또는 8 KB (SPARCv9)를 사용한다. 페이지 테이블 크기를 감소시키는 우리의 당면 문제는 그리 쉽게 해결되지 않을 것이다.

23.2 하이브리드 접근 방법 : 페이징과 세그멘트

인생의 어떤 일을 함에 있어 좋은 방법이 두 가지 있다면, 그 두 방법을 잘 조합하여, 장점만을 취할 수는 없는지도 검토해 보아야 한다. 두 가지 방법을 조합하는 것을 **하이브리드**(**hybrid**)라고 부른다. 예를 들면, 초콜릿과 피넛 버터를 섞어만든 Reese 사의 Peanut Butter Cup [M28]이 있는데, 사람들은 왜 피넛 버터와 초콜릿 중 하나만 먹을까?[2]

Multics의 창시자 Jack Dennis는 Multics 가상 메모리 시스템을 구축하는 데 있어, 비슷한 아이디어가 떠올랐다 [M07]. Dennis는 페이징과 세그멘테이션을 결합하여 페이지 테이블 크기를 줄이는 아이디어를 내었다. 선형 페이지 테이블의 동작을 면밀히 분석해 보면, 페이징과 세그멘테이션을 효과적으로 결합할 수 있다는 사실을 발견할 수 있다. 힙과 스택에서 실제로 전체 공간 중 작은 부분만 사용되는 경우를 생각해 보자.

2) 역자 주 : 체중이 불기는 하겠다.



〈그림 23.1〉 **1 KB 페이지들로 이루어진 16 KB 주소 공간**

PFN	valid	prot	present	dirty
10	1	r-x	1	0
--	0	---	-	-
--	0	---	-	-
--	0	---	-	-
23	1	rw-	1	1
--	0	---	-	-
--	0	---	-	-
--	0	---	-	-
--	0	---	-	-
--	0	---	-	-
--	0	---	-	-
--	0	---	-	-
--	0	---	-	-
--	0	---	-	-
--	0	---	-	-
28	1	rw-	1	1
4	1	rw-	1	1

〈그림 23.2〉 **16 KB 주소 공간을 위한 페이지 테이블**

1 KB 크기의 페이지를 갖는 16 KB의 주소 공간을 예로 들겠다(그림 23.1). 이 주소 공간을 위한 페이지 테이블은 그림 23.2에 나타나 있다.

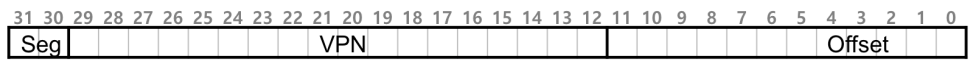
이 예제에서 한 개의 코드 페이지(VPN 0)가 물리 페이지 10번, 그리고 한 개의 힙 페이지 (VPN 4)가 23번 물리 페이지에 매핑되어 있다. 가상 주소 공간의 끝부분에 두 개의 스택 페이지(VPN 14와 15)가 물리 페이지 28번과 4번에 매핑되어 있다. 그림에서 보는 바와 같이, 페이지 테이블 *대부분이* 비어있다. 엄청난 낭비가 발생한다. 16 KB 크기의 아주 작은 주소 공간에서 발생한 상황이다. 32비트 주소 공간용 페이지 테이블에서 비슷한 상황이 발생할 것을 상상해 보자. 골치 아픈 일이다.

결합 방식을 생각해 보자. 프로세스의 전체 주소 공간을 위해 하나의 페이지 테이블을 두는 대신, 논리 세그멘트마다 따로 페이지 테이블을 두면 어떨까? 이 예제에서는 세 개의 페이지 테이블이 있을 수 있다. 코드, 힙 그리고 스택 세그멘트에 대해 페이지 테이블을 각각 두는 것이다.

세그멘테이션에서는 세그멘트의 물리 주소 시작 위치를 나타내는 **베이스**(**base**) 레지스터, 그리고 크기를 나타내는 **바운드**(**bound**) 또는 **리미트**(**limit**) 레지스터가 있다. 우리의 결합 방식에서도 MMU에 비슷한 구조를 사용한다. 베이스 레지스터는 세그멘트 시작 주소를 가리키는 것이 아니라 세그멘트의 *페이지 테이블의 시작 주소*를 갖는다. 바운드 레지스터는 페이지 테이블의 끝을 나타내기 위해서 사용한다.

명확하게 하기 위해 간단한 예를 들어 보자. 4 KB 페이지를 갖는 32비트 가상 주소 공간이 4개의 세그멘트로 나뉘어 있다고 가정하자. 이 예제에서는 세 개의 세그멘트만 사용하도록 하겠다. 하나는 코드를 위해서, 다른 하나는 힙을 위해서 그리고 또 하나는 스택을 위해서 사용한다.

소속 세그멘트를 나타내기 위해 상위 두 비트를 사용한다. 미사용 세그멘트는 00으로, 01은 코드를 그리고 10은 힙, 스택은 11을 나타낸다고 가정하자. 가상 주소는 다음과 같이 표현될 수 있다.



하드웨어에 세 개의 베이스/바운드 레지스터 쌍이 코드와 힙 그리고 스택을 위해서 존재한다고 가정한다. 실행 중인 프로세스에서, 각 세그멘트의 베이스 레지스터는 각 세그멘트 페이지 테이블의 시작 물리 주소를 갖게 된다. 이 시스템에서 모든 프로세스들은 세 개의 페이지 테이블을 갖는다. 문맥 교환 시, 이 레지스터들은 새로 실행되는 프로세스의 페이지 테이블의 위치값으로 변경된다.

TLB 미스가 발생하면(하드웨어 기반 TLB를 가정한다. 즉, TLB 미스를 하드웨어가 처리한다), 하드웨어는 세그멘트 비트(SN)을 사용하여 어떤 베이스와 바운드 쌍을 사용할지 결정한다. 하드웨어는 그 레지스터에 들어 있는 물리 주소를 VPN과 다음과 같은 형식으로 조작하여 페이지 테이블 항목(PTE)의 주소를 얻는다.

```
SN            = (VirtualAddress & SEG_MASK) >> SN_SHIFT
VPN           = (VirtualAddress & VPN_MASK) >> VPN_SHIFT
AddressOfPTE = Base[SN] + (VPN * sizeof(PTE))
```

동작 순서가 눈에 익숙하다. 앞에서 살펴보았던 선형 페이지 테이블의 작동과 거의 동일하다. 유일한 차이가 있다면 하나의 페이지 테이블 베이스 레지스터를 사용하는 대신 세 개 중의 하나의 세그멘트 베이스 레지스터를 사용하는 것이다.

하이브리드 기법에서 핵심은 세그멘트마다 바운드 레지스터가 따로 존재한다는 것이다. 각 바운드 레지스터의 값은 세그멘트의 최대 유효 페이지의 개수를 나타낸다. 예를 들어, 첫 세 개의 페이지들(0, 1, 그리고 2)을 코드 세그멘트로서 사용 중이라면, 코드 세그멘트 페이지 테이블은 세 개의 항목만 할당을 받을 수 있을 것이고 바운드 레지스터는 3으로 설정된다. 해당 세그멘트의 범위가 넘어가는 곳에 대한 메모리 접근은 예외를 발생시키고, 해당 프로세스는 종료될 것이다. 이와 같은 방식으로, 하이브리드 기법은 선형 페이지 테이블에 비해 메모리 사용을 개선시킬 수 있다. 스택과 힙 사이의 할당되지 않은 페이지들은 페이지 테이블 상에서 (유효하지 않다는 것을 표시하기 위해서) 더 이상 공간을 차지하지 않는다.

팁: 하이브리드를 사용하자

두 개의 괜찮은 그리고 상반되는 아이디어가 있다면, 두 방식의 장점을 모두 가져갈 수 있도록 두 방식을 혼합하는 하이브리드를 만들 수 있는지 늘 살펴보아야 한다. 하이브리드 옥수수종을 예로 들면 자연적으로 자라나는 종에 비해 좀 더 강인하다고 알려져 있다. 물론, 모든 하이브리드 기법들이 좋은 아이디어는 아니다. 지돈크(Zeedonk), 또는 존키(Zonkey)는 얼룩말과 당나귀 사이에서 태어난 동물이다. 그런 생물이 존재하는지 믿지 못하겠다면 찾아보자. 놀랄 것이다.

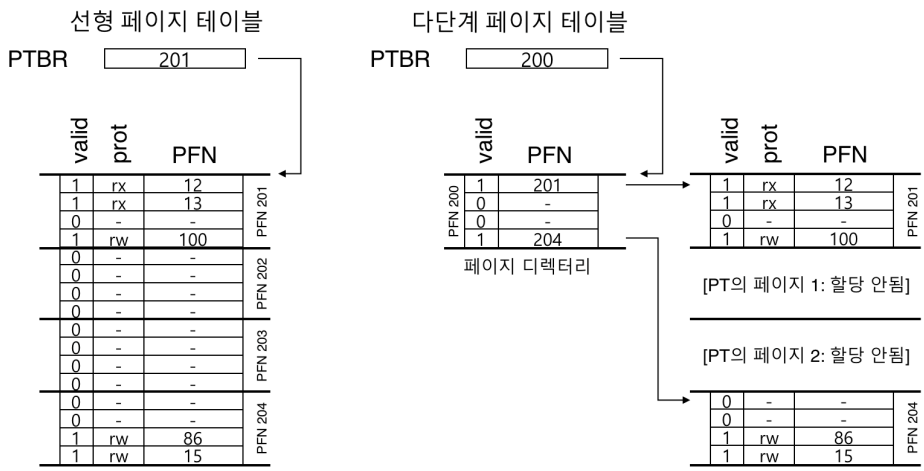
하지만, 이 기법 역시 문제가 없는 것은 아니다. 첫째, 여전히 세그멘테이션을 사용해야 한다; 이 전에 언급했듯이, 세그멘테이션은 주소 공간의 사용에 있어 특정 패턴을 가정하기 때문에 우리가 원하는 만큼은 유연하지가 못하다. 큰 공간을 커버하지만, 드문드문 사용되는(sparsely used) 힙의 경우에는 여전히 페이지 테이블의 낭비를 면치 못할 수가 있다. 둘째로, 하이브리드 기법은 외부 단편화를 유발한다. 하이브리드 방식에서는 페이지 테이블 크기에 제한이 없으며 다양한 크기를 갖는다. 물론, 페이지 테이블의 크기는 페이지 테이블 항목 크기의 정수배가 되어야 한다. 때문에, 메모리 상에서 페이지 테이블용 공간을 확보하는 것이 더 복잡하다. 이런 이유로, 페이지 테이블 크기를 감소시키는 더 나은 방법을 찾는 노력들이 지속되었다.

23.3 멀티 레벨 페이지 테이블

세그멘테이션을 사용하지 않고 페이지 테이블 크기를 줄이는 방법에 대해서 생각해 보자. 어떻게 하면 사용하지 않는 주소 공간을 페이지 테이블에서 제거할 수 있을까? 이번에 소개할 기법은 **멀티 레벨 페이지 테이블**이다. **멀티 레벨 페이지 테이블**에서는 선형 페이지 테이블을 트리 구조로 표현한다. 매우 효율적이기 때문에 많은 현대 시스템에서 사용되고 있다(예, x86 [BO10]). 이 기법을 좀 더 상세히 설명하도록 하겠다.

멀티 레벨 페이지 테이블의 기본 개념은 간단하다. 먼저, 페이지 테이블을 페이지 크기의 단위로 나눈다. 그 다음, 페이지 테이블의 페이지가 유효하지 않은 항목만 있으면, 해당 페이지를 할당하지 않는다. **페이지 디렉터리**(**page directory**) 라는 자료 구조를 사용하여 페이지 테이블 각 페이지의 할당 여부와 위치를 파악한다. 페이지 디렉터리는 페이지 테이블을 구성하는 각 페이지의 존재 여부와 위치 정보를 가지고 있다.

그림 23.3의 예제를 보자. 좌측 그림은 전형적인 선형 페이지 테이블이다. 페이지 테이블의 중앙부에 해당하는 주소 공간은 사용되고 있지 않다. 그러나 페이지 테이블에서 항목들이 할당되어 있다(페이지 테이블의 가운데 두 페이지). 우측은 동일한 주소 공간을 다루는 멀티 레벨 페이지 테이블이다. 페이지 디렉터리에는 두 개의 유효한 페이지가 있다(첫 번째와 마지막). 유효 페이지 두 개는 메모리에 존재한다. 이 예를 통해 멀티 레벨 페이지 테이블의 동작을 좀 더 쉽게 알 수 있다. 선형 페이지 테이블에서 사용되었던 페이지들이 더 이상 필요없고, 페이지 디렉터리를 이용하여 페이지 테이블의 어떤 페이지들이 할당되었는지를 관리한다.



〈그림 23.3〉 **선형(좌) 그리고 멀티 레벨(우) 페이지 테이블**

간단한 2단계 테이블에서, 페이지 디렉터리의 각 항목은 페이지 테이블의 한 페이지를 나타낸다. 페이지 디렉터리는 **페이지 디렉터리 항목**(**page directory entries**, **PDE**)들로 구성된다. 각 항목(PDE)의 구성은 페이지 테이블의 각 항목(Page Table Entry)과 유사하다. **유효**(**valid**) 비트와 **페이지 프레임 번호**(**page frame number**, **PFN**)를 갖고 있다. 실제 구현에 따라 추가 구성 요소가 존재할 수 있다. 하지만, PTE의 유효 비트와 PDE의 유효 비트는 약간 다르다. PDE 항목이 유효하다는 것은, 그 항목이 가리키고 있는(PFN을 통해서) 페이지들 중 최소한 하나가 유효하다는 것을 의미한다. 즉, PDE가 가리키고 있는 페이지 내의 최소한 하나의 PTE의 valid bit가 1로 설정되어 있다. 만약 PDE의 항목이 유효하지 않다면(즉, 0이라면), PDE는 실제 페이지가 할당되어 있지 않은 것이다.

멀티 레벨 페이지 테이블은 이제까지 언급된 다른 기법들에 비해 몇 가지 장점이 있다. 첫째, 멀티 레벨 테이블은 사용된 주소 공간의 크기에 비례하여 페이지 테이블 공간이 할당된다. 그렇기 때문에, 보다 작은 크기의 페이지 테이블로 주소 공간을 표현할 수 있다.

두 번째, 페이지 테이블을 페이지 크기로 분할함으로써 메모리 관리가 매우 용이하다. 페이지 테이블을 할당하거나 확장할 때, 운영체제는 free 페이지 풀에 있는 빈 페이지를 가져다 쓰면 된다. 멀티 레벨 페이징을 단순한 선형 페이지 테이블 방식과 비교해 보자. 선형 페이지 테이블의 각 항목은 해당 가상 페이지의 물리 페이지 주소를 가지고 있다(즉, 디스크로 스왑되지 않는다). 선형 페이지 테이블은 연속된 물리 메모리 공간을 차지한다. 큰 페이지 테이블 (4 MB라고 하자)의 경우, 해당 크기의 연속된 빈 물리 메모리를 찾는 것이 쉽지 않다. 멀티 레벨 페이징에서는 페이지 디렉터리를 사용하여 각 페이지 테이블 페이지들의 위치를 파악한다. 페이지 테이블의 각 페이지들이 물리 메모리에 산재해 있더라도 페이지 디렉터리를 이용하여 그 위치를 파악할 수 있으므로, 페이지 테이블을 위한 공간 할당이 매우 유연하다.

팁: 시간과 공간 간의 절충점에 대해 이해하자

자료 구조 설계 시, 구현에서 시간과 공간의 소요 시간을 적절히 절충(**time-space-trade-offs**)해야 한다. 일반적으로 자료 구조에 접근 속도를 향상시키려면, 해당 구조를 위해 공간을 더 사용해야 한다.

한 가지 유의해야 할 사항이 있다. 멀티 레벨 테이블에는 추가 비용이 발생한다. TLB 미스 시, 주소 변환을 위해 두 번의 메모리 로드가 발생한다(페이지 디렉터리와 PTE 접근을 위해 각각 한 번씩). 선형 페이지 테이블에서는 한 번의 접근만으로 주소 정보를 TLB로 탑재한다. 멀티 레벨 테이블은 시간(페이지 테이블 접근 시간)과 공간(페이지 테이블 공간)을 상호 절충(**time-space-trade-offs**)한 예라 할 수 있다. 페이지 테이블 크기를 줄이는 데 성공하였으나, 대신 메모리 접근 시간이 증가했다. TLB 히트 시(대부분 메모리 접근은 TLB 히트이다) 성능은 동일하지만, TLB 미스 시에는 두 배의 시간이 소요된다.

또 하나의 단점은 *복잡도*이다. 페이지 테이블 검색이 단순 선형 페이지 테이블의 경우보다 더 복잡해진다. 검색을 하드웨어로 구현하느냐 혹은 운영체제로 구현하느냐 여부와는 무관하다. 대부분의 경우, 성능 개선이나 부하 경감을 위해, 우리는 보다 복잡한 기법을 도입한다. 멀티 레벨 페이지 테이블의 경우에는 메모리 자원의 절약을 위해, 페이지 테이블 검색을 좀 더 복잡하게 만들었다.

멀티 레벨 페이징 예제

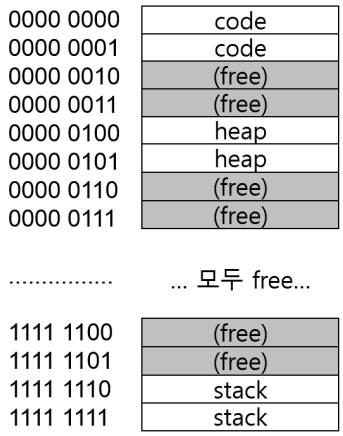
멀티 레벨 페이지 테이블의 개념을 이해하기 위해서 예제를 하나 살펴보자. 64바이트 페이지를 갖는 16 KB 크기의 작은 주소 공간을 생각해 보자. 14비트 가상 주소 공간이다. VPN에 8비트, 페이지 오프셋에 6비트가 필요하다. 선형 페이지 테이블은 2^8(256)개 엔트리로 구성된다. 주소 공간에서 작은 부분만 사용된다 하더라도, 선형 페이지 테이블의 크기는 변하지 않는다. 그림 23.4는 이 구조를 갖는 주소 공간의 예시를 나타낸다.

이 예제에서는, 가상 페이지 0과 1은 코드, 가상 페이지 4와 5는 힙 그리고 가상 페이지 254와 255는 스택으로 사용된다. 주소 공간의 나머지 페이지들은 미사용 중이다.

이 주소 공간을 2단계 페이지 테이블로 구성해 보자. 선형 페이지 테이블을 페이지 단위로 분할한다. 전체 테이블은(이 예제에서) 총 256개의 항목을 갖고 있다는 것을 상기하자. 각 PTE는 4바이트라 가정한다. 페이지 테이블의 크기는 1 KB (256×4 바이트)이다. 페이지가 64바이트라고 하면 1 KB의 페이지 테이블은 16개의 64바이트 페이지들로 분할된다. 각 페이지에는 16개의 PTE가 있다.

이제 VPN으로부터 페이지 디렉터리 인덱스를 추출하고, 페이지 테이블의 각 페이지 위치를 파악하는 법을 살펴보자. 페이지 디렉터리, 페이지 테이블의 페이지들 모두 항목의 배열이라는 것을 기억해야 한다. VPN을 이용하여 인덱스를 구성하는 법만 찾으면 된다.

먼저 페이지 디렉터리의 인덱스를 만들어보자. 예제의 작은 페이지 테이블은 256

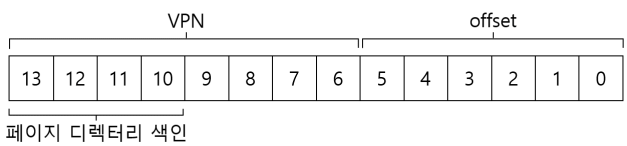


〈그림 23.4〉 **64바이트 페이지들로 이루어진 16 KB 주소 공간**

팁 : 복잡도를 주의하자

시스템 설계자들은 시스템 복잡도의 증가를 주의해야 한다. 좋은 시스템 개발자는 주어진 작업을 처리하기 위한 최소한의 복잡도를 갖는 시스템을 만든다. 예를 들어, 디스크 공간이 풍부하다면 공간 사용을 최소화하기 위한 파일 시스템을 설계해서는 안 된다. 마찬가지로, 프로세서가 빠르다면 어떤 작업을 처리하기 위해 운영체제 내에 이해하기 쉬운 모듈을 작성하는 것이 CPU에 최적화되어 있고 치밀하게 짜여진 코드를 사용하는 것보다 더 좋다. 너무 성급하게 최적화한 코드는 다른 형태에서 불필요한 복잡도를 추가하지 않도록 예의주시하자. 그러한 시스템은 더 이해하기 어려우며 관리와 디버깅을 어렵게 만든다. 앙투안 드 생텍쥐페리(Antoine de Saint-Exupery)는 이런 유명한 말을 남겼다. "완벽함은 무엇인가 더 추가할 것이 없을 때 얻어지는 것이 아니라 더 이상 뺄 것이 없을 때 마침내 얻어진다." 그가 기록하지 않은 부분은 "완벽함에 대해서 이야기 하는 것이 실제로 달성하는 것보다 더 쉽다."는 것이다.

개의 항목으로 16개의 페이지로 나뉘어 있다. 페이지 디렉터리는 페이지 테이블의 각 페이지마다 하나씩 있어야하기 때문에 총 16개의 항목이 있어야 한다. 결과적으로 VPN의 4개의 비트를 사용하여 디렉터리를 구성하며, 여기서는 VPN의 상위 4비트를 다음과 같이 사용한다.



VPN에서 **페이지-디렉터리 인덱스**(**page-directory index**, 짧게 `PDIndex`라고 하

자)를 추출하고 나면 `PDEAddr = PageDirBase + (PDIndex * sizeof(PDE))` 라는 간단한 식을 사용하여 페이지-디렉터리 항목(page-directory entry, PDE)의 주소를 찾을 수 있다. 이렇게 페이지 디렉터리가 구성이 되며, 이것을 활용하여 계속해서 주소 변환 과정을 분석하도록 한다.

페이지-디렉터리의 해당 항목이 무효(invalid)라고 표시되어 있으면, 이 주소 접근은 유효하지 않다. 예외가 발생한다. 해당 PDE가 유효하다면 추가 작업을 해야 한다. 구체적으로 살펴보자. 이 페이지 디렉터리 항목이 가리키고 있는 페이지 테이블의 페이지에서 원하는 페이지 테이블 항목(page table entry, PTE)을 읽어 들이는 것이 목표다. 이 PTE를 찾기 위해서 VPN의 나머지 비트들을 사용한다.



이 **페이지-테이블 인덱스**(**page-table index**, 짧게 `PTIndex`라고 하자)는 페이지 테이블 자체 인덱스로 사용된다. PTE의 주소를 다음과 같이 계산한다. `PTEAddr = (PDE.PFN << SHIFT) + (PTIndex * sizeof(PTE))`.

PTE의 주소를 생성하기 위해서는 페이지-디렉터리 항목에서 얻은 페이지-프레임 번호(page-frame number, PFN)를 먼저 좌측 쉬프트 연산하고 그 값을 페이지 테이블 인덱스에 합산한다.

이 모든 것이 제대로 작동하는지를 보기 위해, 멀티 레벨 페이지 테이블에 실제 값들을 넣은 후에 하나의 가상 주소를 변환해 보자. 이 예제를 위해 **페이지 디렉터리**에서부터 시작해 보자(그림 23.5의 좌측).

이 그림에서 각 페이지 디렉터리 항목(PDE)은 주소 공간의 페이지 테이블의 페이지에 대해 무엇인가를 기술하고 있다. 이 예제에서는 주소 공간에 두 개의 유효한 영역이 있으며(처음과 끝에), 그리고 그 사이는 무효한 매핑들이 존재한다.

물리 페이지 100에 (페이지 테이블의 0번째 페이지의 물리 프레임 번호), 페이지 테이블의 첫 16개 항목이 존재한다. 이 항목들은 가상 주소 공간의 첫 16개 페이지에 대한 물리 주소를 가진다. 그림 23.5의 가운데 열이 해당 페이지의 내용이다.

페이지 테이블의 첫 번째 페이지에는 첫 16개의 VPN과 물리 페이지 주소가 있다. 이 예제에서는 VPN 0과 1이 유효하고(코드 세그멘트), 4와 5도 유효하다(힙). 그러므로 테이블에는 각 페이지들의 매핑 정보가 있다. 그 외의 나머지 엔트리들은 무효로 표기되어 있다.

PFN 101에 다른 유효 페이지들에 대한 정보가 들어있다. 이 페이지에는 가상 주소 공간의 마지막 16개의 VPN에 대한 매핑이 담겨 있다. 그림 23.5의 우측에 상세 내용이 있다.

이 예제에서는, VPN 254와 255가 유효 페이지이다. 이들은 스택에 해당한다. 이 예제를 통해서 멀티 레벨 인덱스 구조가 공간을 얼마큼 절약할 수 있는지를 확인할 수

페이지 디렉터리		PT의 페이지 (@PFN:100)			PT의 페이지 (@PFN:101)		
PFN	**유효한가?**	**PFN**	**유효**	**prot**	**PFN**	**유효**	**prot**
100	1	10	1	r-x	---	-	---
---	0	23	1	r-x	---	0	---
---	0	--	0	---	---	0	---
---	0	--	0	---	---	0	---
---	0	80	1	rw-	---	0	---
---	0	59	1	rw-	---	0	---
---	0	--	0	---	---	0	---
---	0	--	0	---	---	0	---
---	0	--	0	---	---	0	---
---	0	--	0	---	---	0	---
---	0	--	0	---	---	0	---
---	0	--	0	---	---	0	---
---	0	--	0	---	---	0	---
---	0	--	0	---	---	0	---
---	0	---	0	---	55	1	rw-
101	1	---	0	---	45	1	rw-

〈그림 23.5〉 **페이지 디렉터리와 페이지 테이블의 일부**

있다. 선형 페이지 테이블의 열여섯 개의 페이지들을 모두 할당하는 대신 단지 세 개만 할당하였다. 페이지 디렉터리를 위해서 한 페이지, 그리고 유효한 매핑 정보를 갖고 있는 페이지 테이블 내의 두 부분을 위해 두 페이지를 할당하였다. 더 큰 (32비트 또는 64비트) 주소 공간에서는 더 많은 공간이 절약된다.

마지막으로 최종 주소 변환을 해 보자. VPN 254의 0번째 바이트를 가리키는 주소는 다음과 같다. `0x3F80` 또는 이진수로 `11 1111 1000 0000` 이다.

페이지 디렉터리 내에서 각 항목을 가르키기 위해 VPN의 상위 4비트를 사용하였다. `1111`은 페이지 디렉터리의 마지막 엔트리다(0부터 시작했다면 15번째가 된다). 페이지 디렉터리의 15번째 엔트리에는 물리 프레임 주소 `101`이 저장되어 있다. VPN의 다음의 4비트를 (`1110`)을 인덱스로 사용하여, 페이지 테이블의 해당 페이지에서 원하는 PTE를 찾는다. 1110는 101번 페이지의 16개 항목 중에 마지막에서 두 번째 항목이다. 여기에 55가 저장되어 있다. 종합하면 가상 주소 페이지 254 (`1111 1110`)는 물리 페이지 55에 존재한다. 오프셋 000000과 PFN 55(또는 헥사로 `0x37`)를 결합하여 다음과 같이 물리 주소를 구할 수 있으며, 해당 메모리를 접근하는 데 사용된다. `PhysAddr = (PTE.PFN << SHIFT) + offset = 00 1101 1100 0000 = 0x0DC0`.

이제 페이지 디렉터리를 사용하여 페이지 테이블의 페이지를 가리키는 2단계 페이지 테이블에 대한 개념이 어느 정도 이해되었을 것이다. 곧 구체화하겠다. 2단계 페이지 테이블이 충분하지 않을 때가 있다!

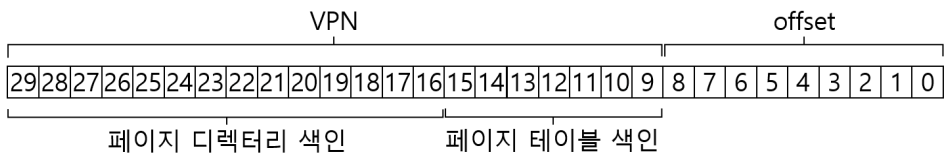
2단계 이상 사용하기

지금까지는 멀티 레벨 페이지 테이블은 페이지 디렉터리와 페이지 테이블의 2개 단계를 가정하였다. 경우에 따라서 트리의 단계를 더 증가시키는 것도 가능하다(그리고 사실, 그래야 할 필요가 있다).

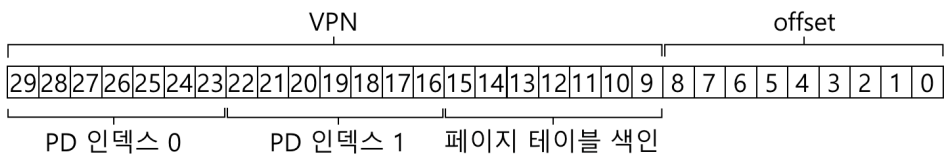
간단한 예제를 통해 2단계 이상의 멀티 레벨 테이블에 대해 알아보자. 이 예제에서는 512바이트 페이지와 30비트 가상 주소 공간을 가정한다. 가상 주소는 21비트의 가상 페이지 번호와 9비트의 오프셋을 갖게 된다.

멀티 레벨 페이지 테이블의 목적은 페이지 테이블의 모든 분할된 부분들이 단일 페이지 크기에 맞도록 하는 것이다. 만약 페이지 디렉터리가 너무 커지면, 어떻게 될까?

멀티 레벨 테이블에서 몇 단계를 둘지 정하기 위해서는 먼저 한 페이지에 몇 개의 페이지 테이블 항목을 저장할 수 있을지를 계산해야 한다. 페이지 크기가 512바이트이고 PTE의 크기가 4바이트라고 가정하면 한 페이지에 128개의 PTE를 넣을 수 있다. 페이지 테이블의 페이지를 인덱스로 쓰려면, VPN의 하위 7비트($\log_2 128$)가 필요하다.



페이지 디렉터리를 위해서 몇 개의 비트가 남았는지를 이 그림에서 알 수 있다. 14개의 비트가 남는다. 2단계 페이지를 사용한다면, 페이지 디렉터리에 2^{14}개의 항목이 있게 된다. 페이지 디렉터리를 위해서 128 페이지 분량의 연속된 메모리가 필요하다. 페이지 테이블을 페이지 단위로 나누어 배치할 수 있도록 하는 멀티 레벨 페이지 테이블의 근본 취지가 훼손된 셈이다.



이 문제를 해결하기 위해서, 페이지 디렉터리 자체를 멀티 페이지들로 나누어서 트리의 단계를 늘리도록 한다. 그리고 페이지 디렉터리의 페이지들을 가리킬 수 있도록 그 위에 새로운 페이지 디렉터리를 추가한다. 결과적으로 다음과 같이 가상 주소를 분할할 수 있다.

이제 가상 주소의 최상위 비트들을(그림에서 `PD Index 0`) 사용하여 상위 단계의 페이지 디렉터리에서 엔트리를 찾는다. 이 인덱스를 사용하여 상위 단계 페이지 디렉터리에서 페이지 디렉터리 항목을 가져온다. 만약 유효하다면, 상위 단계 페이지 디렉터리에서 얻은 물리 주소와 두 번째 단계의 페이지 디렉터리 인덱스(`PD Index`

1)를 결합하여 페이지 테이블 인덱스가 존재한 물리 페이지를 구한다. 해당 페이지가 유효할 경우, 최종적으로, PTE 주소는 2번째 단계의 페이지 디렉터리 항목에서 얻은 페이지 테이블의 물리 주소와 페이지 테이블 인덱스를 결합하여 구한다. 실제 주소를 구하는 데 드는 작업이 상당하다. 멀티 레벨 페이지 테이블 사용 시 수반되는 작업이기도 하다.

변환 과정: TLB를 기억하자

2단계 페이지 테이블 사용 시, 전체 주소 변환 과정을 알고리즘 형태로 요약 정리해 보자 (그림 23.6). 이 그림은 모든 메모리 참조에 대해 하드웨어가 어떤 식으로 동작하는지를 나타낸다(하드웨어 기반 TLB를 가정).

그림에서 볼 수 있듯이, 복잡한 멀티 레벨 페이지 테이블 접근을 거치기 전에, 우선 TLB를 검사한다. 히트가 되면 페이지 테이블을 *참조없이* 물리 주소를 직접 구성한다. TLB 미스 시에만, 멀티 레벨 페이지 테이블의 모든 단계를 거쳐 물리 주소를 구하게 된다. 이 알고리즘을 통해 TLB 미스 발생 시, 전통적인 2단계 페이지 테이블의 주소 계산 비용을 볼 수 있다. 주소 변환을 위해 두 번의 추가 메모리 접근이 발생한다.

23.4 역 페이지 테이블

좀 더 획기적인 공간 절약 방법으로 **역 페이지 테이블(inverted page table)**이 있다. 이 방법에서는 여러 개의 페이지 테이블(시스템의 프로세스당 하나씩) 대신 시스템에 단

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)       // TLB 히트
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else                       // TLB 미스
11     // 먼저, 페이지 디렉터리 항목을 얻어야 함
12     PDIndex = (VPN & PD_MASK) >> PD_SHIFT
13     PDEAddr = PDBR + (PDIndex * sizeof(PDE))
14     PDE = AccessMemory(PDEAddr)
15     if (PDE.Valid == False)
16         RaiseException(SEGMENTATION_FAULT)
17     else
18         // PDE 가 유효함: 이제 페이지 테이블에서 PTE를 가져오자
19         PTIndex = (VPN & PT_MASK) >> PT_SHIFT
20         PTEAddr = (PDE.PFN << SHIFT) + (PTIndex * sizeof(PTE))
21         PTE = AccessMemory(PTEAddr)
22         if (PTE.Valid == False)
23             RaiseException(SEGMENTATION_FAULT)
24         else if (CanAccess(PTE.ProtectBits) == False)
25             RaiseException(PROTECTION_FAULT)
26         else
27             TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
28             RetryInstruction()
```

〈그림 23.6〉 **멀티 레벨 페이지 테이블의 제어 흐름**

하나의 페이지 테이블만 둔다. 페이지 테이블은 *물리 페이지*를 가상 주소 상의 페이지로 변환한다. 역 페이지 테이블의 각 항목은 해당 물리페이지를 사용 중인 프로세스 번호, 해당 가상 페이지 번호를 갖고 있다.

페이지 테이블의 목적은 가상 주소를 물리 주소로 변환하는 것이다. 역 페이지 테이블에서는 주소 변환을 위해 전체 테이블을 검색해서 원하는 가상 주소 페이지를 갖는 항목을 찾아야 한다. 순차 탐색은 느리다. 탐색 속도 향상을 위해 주로 해시 테이블을 사용한다. PowerPC는 이 구조를 사용하는 예이다 [JM98].

좀 더 일반적인 시각에서 보자면, 역 페이지 테이블 역시 하나의 자료 구조일 뿐이다. 자료 구조로 다양한 시도를 할 수 있다. 예를 들면 작게도 만들고 크게도 만들 수 있으며 느리게도 빠르게도 만들 수가 있다. 멀티 레벨과 반전된 페이지 테이블들은 할 수 있는 다양한 방법 중 두 가지 예일 뿐이다.

23.5 페이지 테이블을 디스크로 스와핑하기

마지막으로 중요한 가정을 해제하겠다. 이제까지는 페이지 테이블이 커널이 소유하고 있는 물리 메모리 영역에 존재한다고 가정하였다. 페이지 테이블 크기 축소를 위해 많은 시도를 하더라도, 여전히 모든 페이지 테이블을 메모리에 상주시키기에는 양이 너무 클 수도 있다. 그렇기 때문에 어떤 시스템들은 페이지 테이블들을 **커널 가상 메모리**에 존재시키고, 시스템의 메모리가 부족할 경우, 페이지 테이블들을 디스크로 **스왑**(**swap**)하기도 한다. 이 부분에 대해서는 페이지들을 메모리에 탑재하고 제거하는 방법을 이해한 후에, 다시 (구체적으로 VAX/VMS에 대한 사례 연구를 다루는 장에서) 상세히 다루도록 한다.

23.6 요약

페이지 테이블이 실제로 어떻게 구성되었는지를 보았다. 단순한 선형 배열을 사용하는 구조뿐만 아니라 좀 더 복잡한 자료 구조의 형태도 살펴보았다. 테이블을 위한 자료 구조에는 시간과 공간이라는 모순적 선택 사항이 존재한다. 공간을 많이 소모하는 테이블 구조를 사용할수록 TLB 미스의 처리속도가 빨라지고, 공간을 작게 차지하는 테이블 구조를 사용하면 상황은 반대가 된다. 주어진 제약 조건들을 적절히 고려하여 적합한 자료 구조를 결정해야 한다.

주기억 장치 용량이 작았었던 과거 시스템(많은 과거의 시스템들과 같은)의 경우, 소형 자료 구조의 사용이 현명한 선택이었다. 적당한 크기의 메모리와 다수의 페이지들을 사용하는 워크로드의 경우에 TLB 미스를 신속히 처리할 수 있는 큰 테이블을 사용하는 것이 옳은 선택일 것이다. 소프트웨어로 관리되는 TLB의 경우에는, 전체 자료 구조를 운영체제 개발자가 임의로 그리고 혁신적으로 개발, 그리고 개선할 수 있다. 어떤 새로운 자료 구조가 있을까? 그 새로운 구조는 어떤 문제를 해결하는가? 잠들기 전에 이러한 질문들을 해 보라. 그리고 운영체제 개발자만 꿀 수 있는 큰 꿈을 꾸자.

참고 문헌

[BO10] **"Computer Systems: A Programmer's Perspective"**
Randal E. Bryant and David R. O'Hallaron
Addison-Wesley, 2010
아직 멀티 레벨 페이지 테이블에 대한 좋은 일차 참고 문헌을 찾아야 한다. 하지만 Bryant와 O'Hallaron이 쓴 이 엄청난 교재는 멀티 레벨 페이지 테이블을 사용했던 초기의 시스템 중의 하나 였던 x86의 세부적인 내용을 다루고 있다. 그리고 갖고 있을만한 좋은 책 중에 하나이기도 하다.

[JM98] **"Virtual Memory: Issues of Implementation"**
Bruce Jacob and Trevor Mudge
IEEE Computer, June 1998
여러 다른 시스템들과 각각의 시스템의 메모리 가상화 기법에 대한 탁월한 조사로서 x86과 PowerPC, MIPS 그리고 다른 컴퓨터 구조에 대한 설명이 상세히 나와 있다.

[LL82] **"Virtual Memory Management in the VAX/VMS Operating System"**
Hank Levy and P. Lipman
IEEE Computer, Vol. 15, No. 3, March 1982
운영체제의 고전인 VMS의 실제 가상 메모리 관리자에 대한 아주 훌륭한 논문이다. 너무나 훌륭하기 때문에 앞으로의 몇 장은 이 논문을 기반으로 우리가 여태까지 배운 가상 메모리에 대해서 다시 살펴보도록 하겠다.

[M28] **"Reese's Peanut Butter Cups"**
Mars Candy Corporation.
이 훌륭한 당과류는 1928년에 헤리 버넷 리즈(Harry Burnett Reese)에 의해서 만들어진 것으로 보인다. 낙농업에 종사했던 그는 Milton S. Hershey의 선적 감독 중의 하나였다. 위키페디아에서 적힌 것으로는 그렇다. 그게 사실이라면, Hershey와 Reese는 여느 두 명의 초콜릿 부호들이 그런 것처럼 서로 꼴도 보기 싫었을 것이다.

[M07] **"Multics: History"**
URL: `http://www.multicians.org/history.html`
운영체제 역사상 가장 영향력 있었던 시스템 중의 하나인 Multics 시스템에 대한 방대한 양의 역사를 소개하는 놀라운 웹사이트이다. 내용 중 한 구절을 인용한다. "MIT의 Jack Dennis는 Multics가 처음 개발될 때 영향력 있는 구조적 개념들을 제안하는 공헌을 하였다. 그의 공헌 중에는 페이징과 세그멘테이션을 병합하는 개념도 포함된다."(섹션 1.2.1 중에서)

[Nav+02] **"Practical, Transparent Operating System Support for Superpages"**
Juan Navarro, Sitaram Iyer, Peter Druschel, and Alan Cox
OSDI '02, Boston, Massachusetts, October 2002
*현대의 운영체제에 큰 페이지나 **슈퍼페이지**들을 포함하기 위해 필요한 모든 상세 정보들을 나타낸 좋은 논문이다. 그렇지만 생각하는 만큼 그렇게 쉬운 것은 아니다.*

숙제

멀티 레벨 페이지 테이블이 어떻게 동작하는지를 이해하고 있는지를 확인하기 위한 재미있는 작은 숙제이다. 앞 문장에서 "재미"있다는 표현이 적합한지에 대해서는 논란이 있는 것은 사실이다. 놀랍지 않겠지만, 프로그램의 이름은 `paging-multilevel-translate.py`이다. README의 상세한 설명을 참고하자.

문제

1. TLB 미스에 대해서 하드웨어가 검색을 한다는 가정 하에서 선형 페이지 테이블을 사용하면 페이지 테이블의 위치를 찾기 위해 하나의 레지스터가 필요하다. 2단계 페이지 테이블을 사용하는 경우에는 몇 개의 레지스터가 필요한가? 3단계 페이지 테이블을 사용하는 경우는 몇 개가 필요한가?
2. 시뮬레이터를 이용하여 변환을 수행하여 보자. 이때 랜덤 시드는 0과 1 그리고 2를 사용하여 보자. `-c` 플래그를 사용하여 답이 맞는지 확인해 보자. 각 검색을 수행하기 위해서 몇 번의 메모리 참조가 필요한가?
3. 캐시 메모리가 어떻게 동작하는지 이해했다고 했을 때, 캐시를 사용하면 페이지 테이블에 대한 메모리 참조는 어떻게 동작할까? 캐시 히트가 많이 생겨날까(그래서 빠르게 접근될 것인가)? 또는 많은 미스를 만들어 낼까(그러므로 느리게 접근될 것인가)?

24

물리 메모리 크기의 극복 : 메커니즘

지금까지 가상 주소 공간이 비현실적으로 작아서 모두 물리 메모리에 탑재가 가능한 것으로 가정하였다. 사실 실행 중인 프로세스의 *전체* 주소 공간이 메모리에 탑재된 것으로 가정하고 있었다. 이제 그 가정을 완화한다. 우리는 다수 프로세스들이 동시에 각자 큰 주소 공간을 사용하고 있는 상황을 가정한다.

이를 위해, **메모리 계층**에 레이어의 추가가 필요하다. 지금까지는 모든 페이지들이 물리 메모리에 존재하는 것을 가정하였다. 하지만 큰 주소 공간을 지원하기 위해서 운영체제는 주소 공간 중에 현재는 크게 필요하지 않은 일부를 보관해 둘 공간이 필요하다. 일반적으로 그 공간은 메모리 공간보다 더 크며, 더 느리다(만약 더 빠르다면 그것을 메모리로 사용할 것이다. 그렇지 않을까?). 현대 시스템에서는 보통 **하드 디스크 드라이브**가 이 역할을 담당한다. 이제 메모리 계층에서 크고 느린 하드 디스크 드라이브가 가장 하부에 위치하고, 그 위에 메모리가 있다. 이렇게 하면 우리의 질문은 다음과 같이 변형된다.

핵심 질문 : 물리 메모리 이상으로 나아가기 위해서 어떻게 할까
운영체제는 어떻게 크고 느린 장치를 사용하면서 마치 커다란 가상 주소 공간이 있는 것처럼 할 수 있을까?

한 가지 짚고 넘어가야 할 것은, 왜 프로세스에게 굳이 "큰" 주소 공간을 제공해야 하는가이다. 이에 대한 답은 다시 한 번 편리함과 사용 용이성이다. 주소 공간이 충분히 크면, 프로그램의 자료 구조들을 위한 충분한 메모리 공간이 있는지 걱정하지 않아도 된다. 필요 시 메모리 할당을 운영체제에게 요청하기만 하면 된다. 운영체제가 이러한 "가상" 환경을 제공하면, 인생이 편해진다. 이와 대비되는 기법으로 과거 시스템에서 사용되던 **메모리 오버레이**(**memory overlay**)가 있다. 이 시스템에서는 프로그래머가 코드 또는 데이터의 일부를 수동으로 메모리에 탑재/제거했다 [Den97]. 상상만 해도 힘들다. 함수 호출이나 데이터 접근 시, 코드 또는 데이터를 먼저 메모리에 탑재해야 한다. 헉!

스왑 공간이 추가되면 운영체제는 실행되는 각 프로세스들에게 큰 가상 메모리가 있는 것 같은 환상을 줄 수 있다. 멀티프로그래밍 시스템(컴퓨터의 사용률을 높이기

여담: 저장 장치 기술들

I/O 장치의 동작 메커니즘은 추후 깊게 살펴보도록 하겠다(I/O 장치들에 대한 장을 보자). 조금만 기다리기 바란다. 느린 장치가 꼭 하드 디스크 드라이브일 필요는 없다. 좀 더 현대적인 플래시 기반의 SSD일 수도 있다. 그것에 대해서도 다룰 것이다. 이 시점에서는 대형 가상 공간을 위해, "크고 느린 저장 장치가 있다." 라고만 가정하겠다.

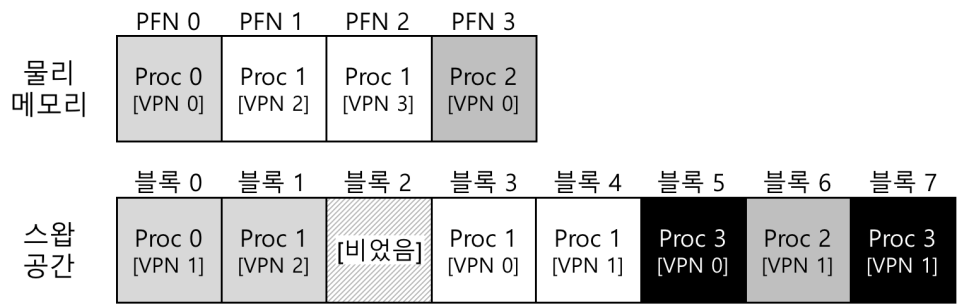
위해 "동시에" 여러 프로그램들을 실행시키는 시스템)이 발명되면서 많은 프로세스들의 페이지를 물리 메모리에 전부 저장하는 것이 불가능하게 되었다. 그래서 일부 페이지들을 스왑 아웃하는 기능이 필요하게 되었다. 멀티프로그래밍과 사용 편의성 등의 이유로 실제 물리 메모리보다 더 많은 용량의 메모리가 필요하게 되었다. 이것이 현대 Virtual Memory(가상 메모리)의 역할이다. 이제 그 방식을 배워보도록 하겠다.

24.1 스왑 공간

가장 먼저할 일은 디스크에 페이지들을 저장할 수 있는 일정 공간을 확보하는 것이다. 이 용도의 공간을 **스왑 공간(swap space)**이라고 한다. 스왑 공간이라 불리는 이유는 메모리 페이지를 읽어서 이곳에 쓰고(*swap out*), 여기서 페이지를 읽어 메모리에 탑재시키기(*swap in*) 때문이다. 스왑 공간의 입출력 단위는 페이지라고 가정한다. 운영체제는 스왑 공간에 있는 모든 페이지들의 **디스크 주소**를 기억해야 한다.

스왑 공간의 크기는 매우 중요하다. 시스템이 사용할 수 있는 메모리 페이지의 최대수를 결정하기 때문이다. 일단 문제를 단순화하기 위해, 스왑 공간은 *매우* 크다고 가정하자.

간단한 예제(그림 24.1)를 보도록 하자. 물리 메모리와 스왑 공간에는 각각 4개의 페이지와 8개의 페이지를 위한 공간이 존재한다. 이 예에서는 세 개의 프로세스(Proc 0, Proc 1, 그리고 Proc 2)가 물리 메모리를 공유하고 있다. 이들 세 프로세스는 몇 개의 유효한 페이지들만 메모리에 올려 놓았으며 나머지 페이지들은 디스크에 스왑 아웃되어 있다. 네 번째 프로세스(Proc 3)의 모든 페이지들은 디스크로 스왑 아웃되어 있기 때문에



〈그림 24.1〉 **물리 메모리와 스왑 공간**

현재 실행 중이 아닌 것이 분명하다. 스왑 영역에 하나의 블럭이 비어있다. 예제를 통해, 스왑 공간을 이용하면, 시스템에 실제 물리적으로 존재하는 메모리 공간보다 더 많은 공간이 존재하는 것처럼 가장할 수 있다는 것을 알 수 있다.

스왑 공간에만 스왑을 할수 있는 것은 아니라는 사실에 유념하자. 예를 들어, 프로그램을 실행한다고 하자(예, `ls` 또는 당신이 컴파일한 `main` 프로그램). 이 실행프로그램의 페이지들은 디스크에 존재한다. 프로그램이 실행되면 각 페이지들은 메모리로 탑재된다(프로그램 실행 시 한 번에 모두 탑재되거나 또는 필요 시 한 페이지씩 탑재할 수 있다). 물리 메모리에 추가 공간을 확보해야 할 때, 코드영역의 페이지들이 차지하는 물리 페이지는 즉시 다른 페이지가 사용할 수 있다. 코드가 저장되어 있는 파일 시스템 영역이 스왑 목적으로 사용되는 셈이다. 해당 페이지들은 디스크에 원본이 있으므로, 언제든지 다시 스왑-인이 가능하기 때문이다.

24.2 Present Bit

디스크에 스왑 공간을 확보했으니 이제 페이지 스왑을 위한 기능을 다룰 차례다. 하드웨어 기반의 TLB를 사용하는 시스템을 가정하자.

먼저 메모리가 참조되는 과정을 상기해 보자. 프로세스가 가상 메모리 참조를 생성한다(명령어 탑재나 데이터 접근 등). 하드웨어는 메모리에서 원하는 데이터를 가져오기 전에, 우선 가상 주소를 물리 주소로 변환한다.

하드웨어는 먼저 가상 주소에서 VPN을 추출한 후에 TLB에 해당 정보가 있는지 검사한다(**TLB 히트**). 만약 히트가 되면 물리 주소를 얻은 후에 메모리로 가져온다. 매우 빠르다. 대부분의 경우가 여기에 해당하기를 바란다(추가적인 메모리 접근이 필요 없다).

만약 VPN을 TLB에서 찾을 수 없다면(즉, **TLB 미스**), 하드웨어는 페이지 테이블의 메모리 주소를 파악하고(**페이지 테이블 베이스 레지스터**를 사용), VPN을 인덱스로 하여 원하는 **페이지 테이블 항목**(**PTE**)을 추출한다. 해당 페이지 테이블 항목이 유효하고, 관련 페이지가 물리 메모리에 존재하면 하드웨어는 PTE에서 PFN 정보를 추출하고, 그 정보를 TLB에 탑재한다. TLB 탑재 후 명령어를 재실행한다. 이번에는 TLB에서 히트된다. 여기까지는 순조롭다.

페이지가 디스크로 스왑되는 것을 가능케 하려면, 많은 기법들이 추가되어야 한다. 특히, 하드웨어가 PTE에서 해당 페이지가 물리 메모리에 존재하지 않는다는 것을 표현해야 한다. 하드웨어는(또는 소프트웨어가 관리하는 TLB 기법인 경우에는 운영체제가) **present bit**를 사용하여 각 페이지 테이블 항목에 어떤 페이지가 존재하는지를 표현한다. Present 비트가 1로 설정되어 있다면, 물리 메모리에 해당 페이지가 *존재한다*는 것이고 위에 설명한 대로 동작한다. 만약 그 비트가 0으로 설정되어 있다면, 메모리에 해당 페이지가 존재하지 않고 디스크 어딘가에 존재한다는 것을 나타낸다. 물리 메모리에 존재하지 않는 페이지를 접근하는 행위를 일반적으로 **페이지 폴트**(**page fault**)라 한다.

페이지 폴트가 발생하면, 페이지 폴트를 처리하기 위해 운영체제로 제어권이 넘어간다. **페이지 폴트 핸들러**(**page-fault handler**) 가 실행된다. 그 세부내용을 다음에서

여담: 스와핑 기술 그리고 다른 것들

가상 메모리 시스템의 용어들은 운영체제와 사용하는 기기에 따라서 약간 혼란스럽고 변종이 많다. 예를 들어 **페이지 폴트**는 페이지 테이블 참조 시 발생가능한 다양한 종류의 오류를 의미할 수 있다. 이 오류에는 지금 여기서 우리가 다루고 있는 종류의 오류, 즉 페이지-존재하지-않음 오류, 포함할 수도 있지만 때로는 불법적인 메모리 접근을 나타낼 수도 있다. 사실, 분명하게 합법적인 접근을 "오류"라고 부르는 것이 이상하다. 실제로는 **페이지 미스**라고 불려야 할 것이다. 그렇지만 대체적으로 사람들이 어떤 프로그램에서 "페이지 폴트"가 발생하였다고 말할 때 의미하는 것은 운영체제가 디스크로 스왑 아웃한 가상 주소 공간의 한 부분을 접근하였다는 것이다.

이러한 동작이 "오류"라고 불리게 된 이유를 운영체제에 이것을 다루기 위해 적용한 기법과 관계가 있다고 추정한다. 일반적이지 않은 어떤 일이 발생한다면, 즉 하드웨어가 어떻게 다뤄야 할지 모르는 어떤 일이 발생한다면, 상황 개선을 기대하며 하드웨어는 제어권을 운영체제에게 넘긴다. 프로세스가 원하는 페이지가 메모리에 없다. 하드웨어는 "예외"를 발생시키며, 그 이후부터는 운영체제가 처리를 담당한다. 프로세스가 불법적인 일을 할 때 그를 처리하는 방법과 동일하기 때문에, 이 동작을 "오류"라고 해도 그리 이상하지 않다.

살펴보도록 하자.

24.3 페이지 폴트

TLB 미스의 처리 방법에 따라, 두 종류의 시스템이 있었다. 하드웨어 기반의 TLB(하드웨어가 페이지 테이블을 검색하여 원하는 변환 정보를 찾는 것)와 소프트웨어 기반의 TLB(운영체제가 처리하는 것)이다. 둘 중 어느 것이던, 페이지 폴트가 발생하면, 운영체제가 그 처리를 담당한다. 운영체제의 **페이지 폴트 핸들러**가 그 처리 메커니즘을 규정한다. 페이지 폴트 핸들러, 이름 참 잘 지었다. 거의 대부분의 시스템들에서 페이지 폴트는 소프트웨어적으로 처리된다. 하드웨어 기반의 TLB도 페이지 폴트 처리는 운영체제가 담당한다.

만약 요청된 페이지가 메모리에 없고, 디스크로 스왑되었다면, 운영체제는 해당 페이지를 메모리로 스왑해 온다. 자연적으로 등장하는 질문이 있다. 원하는 페이지의 위치를 어떻게 파악할지? 많은 시스템들에서 해당 정보—즉 해당 페이지의 스왑 공간상에서의 위치—를 페이지 테이블에 저장한다. 운영체제는 PFN과 같은 PTE비트들을 페이지의 디스크 주소를 나타내는 데 사용할 수 있다. 페이지 폴트 발생 시, 운영체제는 페이지 테이블 항목에서 해당 페이지의 디스크 상 위치를 파악하여, 메모리로 탑재한다.

디스크 I/O가 완료가 되면 운영체제는 해당 페이지 테이블 항목(PTE)의 PFN 값을 탑재된 페이지의 메모리 위치로 갱신한다. 이 작업이 완료되면 페이지 폴트를 발생시킨 명령어가 재실행된다. 재실행으로 인해 TLB 미스가 발생될 수 있다. TLB 미스 처리 과정에서 TLB 값이 갱신된다(이를 피하기 위한 대안으로 페이지 폴트를 처리 시 함께

여담: 페이지 폴트를 하드웨어로 처리하지 않는 이유

TLB를 다뤄본 경험에서 우리는 하드웨어 설계자들이 운영체제가 하는 일을 신뢰하고 싶어하지 않는다는 것을 알았다. 어째서 운영체제에게 페이지 폴트 처리를 맡길까? 몇 가지 중요한 이유들이 있다. 첫째는 페이지 폴트로 인한 디스크 접근이 느리기 때문이다. 페이지 폴트 처리에 시간이 소요된다 하더라도, 디스크 작업 자체가 너무 느리기 때문에 소프트웨어를 실행하는 추가 부담은 절대적으로 작다. 둘째로 페이지 폴트 처리를 위해서는 하드웨어가 스왑 공간의 구조, 디스크에 I/O를 요청하는 방법, 그리고 그 외에 하드웨어가 파악하지 못하고 있는 세부적인 것들을 모두 알고 있어야 한다. 성능과 간편성이라는 두 가지 장점으로, 페이지 폴트 처리를 운영체제가 담당하게 되었고, 하드웨어 설계자도 이것을 만족하게 되었다.

TLB를 갱신하도록 할 수도 있다). 최종적으로, 마지막 재실행 시에 TLB에서 주소 변환 정보를 찾게 되고, 이를 이용하여 물리 주소에서 원하는 데이터나 명령어를 가져온다.

I/O 전송 중에는 해당 프로세스가 **차단된**(**blocked**) 상태가 된다는 것을 유의해야 한다. 페이지 폴트 처리 시 운영체제는 다른 프로세스들을 실행할 수 있다. I/O 실행은 매우 시간이 많이 소요되기 때문에 한 프로세스의 I/O 작업(페이지 폴트)과 다른 프로세스의 실행을 **중첩**(**overlap**)시키는 것은 멀티 프로그램된 시스템에서 하드웨어를 최대한 효율적으로 사용하는 방법 중 하나다.

24.4 메모리에 빈 공간이 없으면?

위 설명에서, 스왑 공간으로부터 페이지를 가져오기 위한 (**page-in**) 여유 메모리가 충분하다는 것을 가정했다. 항상 이런 경우만 있는 것은 아니다. 메모리에 여유 공간이 없을 수도 있다(또는 거의 다 찼을 수도 있다). 탑재하고자 하는 새로운 페이지(들)를 위한 공간을 확보하기 위해 하나 또는 그 이상의 페이지들을 먼저 **페이지 아웃**(**page out**)하려고 할 수도 있다. **교체**(**replace**) 페이지를 선택하는 것을 **페이지 교체 정책**(**page-replacement policy**)이라고 한다.

잘못된 선택은 프로그램의 성능에 큰 악영향을 미친다. 좋은 페이지 교체 정책을 만들기 위해 많은 노력들이 있었다. 잘못된 선택을 할 경우, 프로그램이 메모리 속도로 실행되는 것이 아니라 디스크와 비슷한 속도로 동작할 수도 있기 때문이다. 현재 기술에서 볼 때, 프로그램이 10,000 또는 100,000 배 더 느리게 실행된다는 것을 의미한다. 페이지 교체 정책은 면밀히 검토되어야 한다. 다음 장에서 각종 다른 정책들에 대해서 자세히 다룬다. 현재로서는 페이지 교체 정책이라는 것이 존재한다는 사실과 그 기법이 이전에 다룬 스와핑과 함께 동작한다는 점만 인지하고 넘어간다.

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)          // TLB 히트
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else                          // TLB 미스
11     PTEAddr = PTBR + (VPN * sizeof(PTE))
12     PTE = AccessMemory(PTEAddr)
13     if (PTE.Valid == False)
14         RaiseException(SEGMENTATION_FAULT)
15     else
16         if (CanAccess(PTE.ProtectBits) == False)
17             RaiseException(PROTECTION_FAULT)
18         else if (PTE.Present == True)
19             // 하드웨어를 기반으로 한 TLB를 가정함
20             TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
21             RetryInstruction()
22         else if (PTE.Present == False)
23            RaiseException(PAGE_FAULT)
```

〈그림 24.2〉 **페이지 오류 제어 흐름의 알고리즘(하드웨어)**

24.5 페이지 폴트의 처리

이제 메모리 접근이 이루어지는 전체 과정을 파악할 수 있게 되었다. "프로그램이 메모리에서 데이터를 가져올 때 어떤 일이 발생하는가?"라는 질문을 받았을 때, 다양한 상황에 대해 심도 있는 설명이 가능하게 된 것이다. 그림 24.2와 그림 24.3은 페이지 폴트 처리의 상세한 과정을 나타낸다. 첫 그림은 하드웨어를 통한 주소 변환을 나타낸다. 두 번째 그림은 페이지 폴트 발생 시 운영체제의 동작(소프트웨어)을 나타낸다.

```
1  PFN = FindFreePhysicalPage()
2  if (PFN == -1)                    // 비어있는 페이지 못 찾음
3      PFN = EvictPage()             // 교체 알고리즘 실행
4      DiskRead(PTE.DiskAddr, pfn)   // 대기 (I/O를 기다리기)
5      PTE.present = True            // 존재한다고 페이지 테이블에 갱신
6      PTE.PFN = PFN                 // 비트와 변환 (PFN)
7      RetryInstruction()            // 명령어 재시도
```

〈그림 24.3〉 **페이지 오류 제어 흐름의 알고리즘(소프트웨어)**

그림 24.2의 하드웨어 처리 과정에서, TLB 미스 발생 시, 세 가지의 중요한 경우가 있다는 것을 알 수 있다. 첫째는 페이지가 **존재**하며 **유효**한 경우이다(18-21 라인). 이 경우에는 TLB 미스 핸들러가 PTE에서 PFN을 가져와서 (여러 차례) 명령어를 재시도한다(이때에는 TLB 히트가 된다). 두 번째 경우에는(22-23 라인) 페이지가 유효하지만 존재하지 않는 경우다. 페이지 폴트 핸들러가 반드시 실행되어야 한다. 프로세스가 사용할 수 있는 제대로 된 페이지이기는 하지만(유효한 경우이기 때문에) 물리 메모리에 존재하지 않기 때문이다. 세 번째, 마지막으로 페이지가 유효하지 않는 경우다. 프로그램 버그 등으로 잘못된 주소를 접근하는 경우의 처리를 나타낸다(13-14 라인). 이 경우에는 PTE의 다른 비트는 의미가 없다. 하드웨어는 이 무효한 접근이 운영체제의 트랩 핸들

러에 의해서 처리되도록 한다. 이때 문제를 일으킨 프로세스는 종료될 수가 있다.

그림 24.3에서 운영체제가 페이지 폴트를 처리하는 과정을 대략적으로 볼 수 있다. 먼저 운영체제는 탑재할 페이지를 위한 물리 프레임을 확보한다. 만약 여유 프레임이 없다면, 교체 알고리즘을 실행하여 메모리에서 페이지를 내보내고 여유 공간을 확보한다. 물리 프레임을 확보한 후, I/O 요청을 통해 스왑 영역에서 페이지를 읽어 온다. 마지막으로 이 느린 작업이 완료되면 운영체제는 페이지 테이블을 갱신하고 명령어를 재시도한다. 재시도를 하게 되면 TLB 미스가 발생하며, 또 한 번의 재시도를 할 때에 TLB 히트가 된다. 그제서야 하드웨어는 원하는 것을 접근할 수 있게 된다.

24.6 교체는 실제 언제 일어나는가

이제까지의 설명에서는 메모리에 여유 공간이 고갈된 후에 교체 알고리즘이 작동하는 것을 가정하였다. 이 방법은 그리 효율적이지 않다. 뿐만 아니라, 다양한 이유로 인해, 운영체제는 항상 어느 정도의 여유 메모리 공간을 확보하고 있어야 한다.

메모리에 항상 어느 정도의 여유 공간을 비워두기 위해서, 대부분의 운영체제들은 여유 공간에 관련된 **최댓값**(**high watermark**, HW)과 **최솟값**(**low watermark**, LW)을 설정하여 교체 알고리즘 작동에 활용한다. 동작 방식은 다음과 같다. 운영체제가 여유 공간의 크기가 최솟값보다 작아지면 여유 공간 확보를 담당하는 백그라운드 쓰레드가 실행된다. 이 쓰레드는 여유 공간의 크기가 최댓값에 이를 때까지 페이지를 제거한다. 이 백그라운드 쓰레드는 일반적으로 **스왑 데몬**(**swap daemon**) 또는 **페이지 데몬**[1] (**page daemon**)이라고 불린다. 충분한 여유 메모리가 확보되면 이 백그라운드 쓰레드는 슬립 모드로 들어간다.

일시에 여러 개를 교체하면 성능 개선이 가능하다. 예를 들어 많은 시스템들은 다수의 페이지들을 **클러스터**(**cluster**)나 **그룹**(**group**)으로 묶어서 한 번에 스왑 파티션에 저장함으로써 디스크의 효율을 높인다 [LL82]. 디스크 관련 부분에서 좀 더 자세히 다루겠지만, 클러스터링은 디스크의 탐색(seek)과 회전 지연(rotational delay)에 대한 오버헤드를 경감시켜 성능을 상당히 높일 수 있다.

백그라운드 페이징 쓰레드를 사용하기 위해서는 그림 24.3의 과정이 약간 변경되어야 한다. 교체를 직접 수행하는 대신 알고리즘은 사용할 수 있는 페이지들이 있는지를 단순히 검사만 한다. 만약 없다면 백그라운드 페이징 쓰레드에게 여유 페이지들이 필요하다고 알려준다. 쓰레드가 페이지들을 비운 후에 원래의 쓰레드를 다시 깨워서 원하는 페이지를 불러들일 수 있도록 하며 계속 작업을 진행할 수 있도록 한다.

1) ‘‘데몬(daemon)”이라는 단어는 일반적으로 “데몬(demon)”으로 발음되는데, 이것은 유용한 어떤 동작을 하는 백그라운드 쓰레드 또는 프로세스를 나타내는 오래된 용어이다. (또 다시!) 이 용어의 근원은 Multics에서 시작되었다 [CS94].

팁: 백그라운드에서 작업을 하자

할 일이 있을 때, 그 일을 **백그라운드**로 처리하고, 여러 가지를 묶어서 한 번에 처리하는 것이 효율적일 수 있다. 운영체제는 자주 백그라운드에서 작업을 처리한다. 예를 들어, 많은 시스템들은 파일을 쓸 때에 데이터를 디스크에 실제로 쓰기 전에 메모리의 버퍼에 먼저 쓴다. 그렇게 하면 여러 가지 이득을 얻을 수가 있다. 디스크가 여러 쓰기 요청을 받지 않기 때문에 스케줄을 좀 더 잘할 수 있게 되어 디스크의 효율을 높일 수 있다. 또한, 응용 프로그램은 쓰기가 빠르게 완료된 것처럼 생각하도록 만들기 때문에 쓰기에 대한 지연 시간이 짧아진다. 디스크로 쓰기가 즉시 전달될 필요가 없을 수도 있기 때문에(파일이 삭제된 경우) 작업량이 줄어들 수도 있다. 백그라운드 작업은 대체적으로 시스템이 **유휴 상태**에 있을 때 진행된다. 시스템이 놀고 있는 시간을 잘 활용할 수 있게 된다. 하드웨어를 더 잘 활용할 수 있게 된다 [Gol+95].

24.7 요약

이 장에서는 시스템에 실제 존재하는 물리 메모리의 크기보다 더 많은 메모리를 사용하기 위한 개념을 소개하였다. 이를 위해서 메모리에 특정 페이지가 존재하는지를 알리기 위한 **present bit**와 좀 더 복잡한 페이지 테이블 구조가 필요하다. 운영체제는 **페이지 폴트(page fault)**를 처리하기 위해서 **페이지 폴트 핸들러(page-fault handler)**를 실행시킨다. 핸들러는 원하는 페이지를 디스크에서 메모리로 전송하기 위해 메모리의 일부 페이지들을 먼저 교체하여 새롭게 스왑되서 들어올 페이지를 위한 공간을 만드는 조치를 취한다.

중요하게(그리고 놀랍게!) 이 모든 작업은 프로세스가 인지하지 못하는 상황에서 처리된다는 것을 기억해야 한다. 프로세스가 보기에는 자신의 개별적인 연속된 가상 메모리를 접근하는 것처럼 보인다. 실제로 페이지들은 물리 메모리 임의의(불연속적인) 위치에 배치되며, 때로는 디스크에서 가져와야 할 때도 있다. 일반적인 경우의 메모리 접근 작업이 빠르다고 기대하지만, 어떤 경우 여러 번의 디스크 작업 후에 처리가 되기도 한다. 최악의 경우에는 하나의 명령어를 처리하는 간단한 작업이라 할지라도 수 millisecond 가 걸려야 완료될 수도 있다.

참고 문헌

[CS94] **"Take Our Word For It"**
F. Corbato and R. Steinberg
URL: `http://www.takeourword.com/TOW146/page4.html`
Richard Steinberg는 "컴퓨팅에 사용되고 있는 데몬이라는 단어의 기원에 대해서 누군가 물어보았어요. 내가 조사한 내용에 따르면, 1963년에 IBM 7094를 사용하는 프로젝트 MAC 의 당신의 팀원들이 처음 그 단어를 사용한 것으로 보입니다." 라고 썼다. 그에 대해 Corbato 교수가 다음과 같이 답신을 하였다. "물리와 열역학(내 전공이 물리학이에요)에서 사용하는 Maxwell 의 데몬에서 영감을 얻어서 데몬을 사용하기 시작했어요. Maxwell의 데몬은 상상의 요원으로 서로 다른 속도를 갖는 분자들을 정렬하는 일을 위해 백그라운드에서 지지치 않고 돕는 것이죠. 시스템의 허드렛일을 지치지 않고 작업하는 백그라운드 프로세스들을 설명하기 위해 데몬이라는 멋진 이름을 붙여서 사용하기 시작했습니다."

[Den97] **"Before Memory Was Virtual"**
Peter Denning
From In the Beginning: Recollections of Software Pioneers, Wiley, November 1997
가상 메모리와 작업 세트를 개발한 선구자들 중의 하나로서 훌륭한 역사적인 자료이다.

[Gol+95] **"Idleness is not sloth"**
Richard Golding, Peter Bosch, Carl Staelin, Tim Sullivan, and JohnWilkes
USENIX ATC '95, New Orleans, Louisiana
여러 가지 예를 들어서 유휴 시간을 시스템에서 잘 활용하는 방법을 설명한 재미있고 읽기 쉬운 논문이다.

[LL82] **"Virtual Memory Management in the VAX/VMS Operating System"**
Hank Levy and P. Lipman
IEEE Computer, Vol. 15, No. 3, March 1982
클러스터링이 사용된 것을 나타내는 가장 첫 문서는 아니지만 동작하는 방식을 쉽게 설명한 글이다.

영문 추가 내용 및 본문 보완

이 통합본의 1-9쪽은 제공된 한국어 PDF를 그대로 유지했다. 10쪽에는 영문과 대조하여 확인한 누락·번역 보완 사항을, 11-12쪽에는 한국어판에 없는 측정 실습과 문제 1-7의 전체 번역을 실었다. 영문 제21장은 한국어 제24장에 해당한다. 아래 쪽수는 각 원본 PDF의 쪽수이다.

1. 페이지를 읽어 온 뒤 present 비트도 갱신한다

대응 위치: 영문 5쪽 / 한국어 4쪽, 24.3절

디스크 I/O가 완료되면 운영체제는 페이지 테이블을 갱신하여 **해당 페이지가 메모리에 존재한다고 표시한다.** 또한 페이지 테이블 항목(PTE)의 PFN 필드를 새로 읽어 온 페이지의 물리 메모리 위치로 갱신하고, 명령어를 재실행한다. 한국어 본문에는 PFN 갱신은 설명되어 있지만, 이 문장에 해당하는 present 비트 갱신 설명이 빠져 있다.

2. 백그라운드 쓰기의 이점: 여러 요청을 한 번에 처리

대응 위치: 영문 8쪽 / 한국어 8쪽, 백그라운드 작업 팁

디스크는 **여러 쓰기 요청을 한 번에 받아** 이 요청들을 더 효율적으로 스케줄링할 수 있으므로, 디스크의 효율이 높아질 수 있다. 한국어판의 "여러 쓰기 요청을 받지 않기 때문에"는 영문의 의미와 다르므로 이 문장으로 읽는다.

3. 요약에서의 페이지 교체는 필요할 때 수행한다

대응 위치: 영문 8쪽 / 한국어 8쪽, 요약

페이지 폴트 핸들러는 원하는 페이지를 디스크에서 메모리로 옮기도록 처리하며, **필요하면 먼저** 메모리의 일부 페이지를 교체하여 새로 들어올 페이지의 공간을 만든다. 영문의 "perhaps first replacing"은 항상 먼저 교체한다는 뜻이 아니다.

4. 참고 문헌의 짧은 덧붙임

대응 위치: 영문 9쪽 / 한국어 9쪽, [LL82]

해당 문헌 설명의 끝에는 "우리는 이 논문을 참 자주 인용한다!"라는 문장이 추가로 있다.

그림 24.2와 24.3의 코드 표기 보완

한국어 PDF에 그대로 인쇄된 LaTeX 명령 `\gndx{...}`은 알고리즘의 일부가 아니다. 아래는 영문의 해당 행을 정상 표기로 옮긴 것이다.

그림 24.2, 7행

```
Register = AccessMemory(PhysAddr)
```

그림 24.2, 12행

```
PTE = AccessMemory(PTEAddr)
```

그림 24.3, 4행

```
DiskRead(PTE.DiskAddr, PFN) // I/O 완료까지 대기
```

측정 실습: vmstat와 메모리 사용

추가 번역 | 영문 10-11쪽의 Homework (Measurement)

이 실습에서는 새로운 도구 `vmstat`를 소개하고, 이 도구로 메모리, CPU, I/O 사용량을 파악하는 방법을 알아본다. 아래 실습과 문제를 시작하기 전에 함께 제공되는 `README`를 읽고 `mem.c`의 코드를 살펴보자.

1. CPU 사용량 관찰

먼저 같은 컴퓨터에 연결된 터미널 창 두 개를 열어, 각 창에서 서로 다른 프로그램을 쉽게 실행할 수 있도록 하자. 한 창에서 `vmstat 1`을 실행한다. 이 명령은 컴퓨터의 사용 통계를 1초마다 보여 준다. 출력 내용을 이해할 수 있도록 매뉴얼 페이지, 함께 제공된 `README`, 그리고 필요한 다른 자료를 읽어 보자. 아래의 나머지 실습을 진행하는 동안 이 창에서 `vmstat`를 계속 실행해 둔다.

이제 `mem.c` 프로그램을 아주 적은 메모리만 사용하도록 실행하자. `./mem 1`을 입력하면 1 MB의 메모리만 사용한다. `mem`을 실행할 때 CPU 사용 통계는 어떻게 달라지는가? 사용자 시간(user time) 열의 값은 타당한가? `mem`을 여러 개 동시에 실행하면 이 값은 어떻게 달라지는가?

2. 메모리 통계 관찰

`mem`을 실행하면서 메모리 통계를 살펴보자. 두 열에 집중한다. `swpd`(사용 중인 스왑 메모리의 양)와 `free`(유휴 메모리의 양)이다. `./mem 1024`를 실행하여 1024 MB를 할당하고, 두 값이 어떻게 달라지는지 관찰한다. 그런 다음 `Ctrl+C`를 눌러 실행 중인 프로그램을 종료하고, 값이 어떻게 바뀌는지 다시 관찰한다. 무엇을 발견했는가? 특히 프로그램이 종료될 때 `free` 열은 어떻게 달라지는가? `mem`이 종료되면 여유 메모리가 예상한 만큼 늘어나는가?

3. 스왑 입출력 관찰

다음으로 스왑 열인 `si`와 `so`를 살펴보자. 이 열들은 디스크에서 메모리로, 그리고 메모리에서 디스크로 얼마나 많은 스와핑이 발생하는지 보여 준다. 이 값들이 변하게 하려면 `mem`을 많은 메모리를 사용하도록 실행해야 한다.

먼저 Linux 시스템에 여유 메모리가 얼마나 있는지 확인한다. 예를 들어 `cat /proc/meminfo`를 입력한다. `/proc` 파일 시스템과 이곳에서 확인할 수 있는 정보의 종류는 `man proc`으로 알아볼 수 있다. `/proc/meminfo`의 첫 항목 중 하나는 시스템의 전체 메모리 용량이다.

메모리가 약 8 GB라고 가정하자. 먼저 `mem 4000`(약 4 GB)를 실행하고 스왑 입출력 열을 관찰한다. 0이 아닌 값이 나타나는가? 다음으로 5000, 6000 등의 값을 차례로 사용해 보자. 첫 번째 루프와 비교하여 두 번째 루프와 그 이후의 루프에 들어갈 때 이 값들은 어떻게 달라지는가? 두 번째, 세 번째, 그리고 이후의 루프에서 스왑 인과 스왑 아웃되는 데이터의 총량은 각각 얼마인가? 이 수치들은 타당한가?

측정 실습: 문제 4-7

영문 추가 내용 한국어 번역 (계속)

4. CPU와 블록 I/O 통계 비교

앞에서와 같은 실험을 반복하되, 이번에는 CPU 사용률과 블록 I/O 통계 등 다른 통계를 관찰한다. `mem`이 실행되는 동안 이 값들은 어떻게 달라지는가?

5. 메모리에 들어가는 경우와 스와핑하는 경우의 성능

이제 성능을 살펴보자. `mem`의 입력으로 메모리에 충분히 들어갈 수 있는 값을 선택한다. 예를 들어 시스템 메모리가 8 GB라면 4000을 선택한다. 루프 0과 그 이후의 루프 1, 2 등은 각각 얼마나 오래 걸리는가?

이번에는 물리 메모리 용량을 충분히 초과하는 크기를 선택한다. 역시 메모리가 8 GB라고 가정하면 12000을 선택할 수 있다. 이 경우 각 루프는 얼마나 오래 걸리는가? 대역폭 수치는 어떻게 다른가? 지속적으로 스와핑할 때와 모든 데이터가 메모리에 충분히 들어갈 때의 성능은 얼마나 다른가?

`mem`이 사용하는 메모리의 크기를 x축으로, 그 메모리에 접근하는 대역폭을 y축으로 하는 그래프를 그릴 수 있는가? 마지막으로, 모든 데이터가 메모리에 들어가는 경우와 그렇지 않은 경우 각각에서 첫 번째 루프의 성능은 이후 루프들의 성능과 어떻게 다른가?

6. 스왑 공간의 한계

스왑 공간은 무한하지 않다. `swapon` 도구에 `-s` 옵션을 주면 사용 가능한 스왑 공간을 확인할 수 있다. 사용 가능한 스왑 용량을 넘어서는 것처럼 보일 정도로 점점 더 큰 값을 주어 `mem`을 실행하면 어떤 일이 발생하는가? 메모리 할당은 어느 시점에서 실패하는가?

7. 심화: 스왑 장치에 따른 성능

마지막으로, 심화 실습을 할 수 있다면 `swapon`과 `swapoff`로 시스템이 서로 다른 스왑 장치를 사용하도록 설정해 보자. 자세한 내용은 매뉴얼 페이지를 읽어 보자. 서로 다른 하드웨어를 사용할 수 있다면, 전통적인 하드 디스크, 플래시 기반 SSD, 심지어 RAID 배열을 스왑 장치로 사용할 때 스와핑 성능이 어떻게 달라지는지 확인하자. 새로운 장치로 스와핑 성능을 얼마나 향상시킬 수 있는가? 메모리에 직접 접근하는 성능에 얼마나 가까워질 수 있는가?

번역 메모: 문제 2의 원문은 swpd를 "사용 중인 가상 메모리의 양"이라고 표현한다. 이 번역에서는 vmstat의 열 이름에 맞추어 "사용 중인 스왑 메모리의 양"으로 명확히 했다. 8 GB와 각 입력값은 원문에서 제시한 예시이며, 실습 문제의 명령과 수치는 원문을 유지했다. swpd 정의 확인: procps-ng vmstat(8) 매뉴얼.

25

물리 메모리 크기의 극복 : 정책

가상 메모리 관리자의 입장에서 비어 있는 메모리가 많을수록 일은 쉬워진다. 페이지 폴트가 발생하면 빈페이지 리스트에서 비어 있는 페이지를 찾아서 폴트를 일으킨 페이지에게 할당하면 된다. 오, 운영체제 씨 축하합니다! 또 다시 해냈군요.

불행하게도 빈 메모리 공간이 거의 없으면 일이 더 복잡해진다. 그런 경우 운영체제는 **메모리 압박**(**memory pressure**)을 해소하기 위해 다른 페이지들을 강제적으로 페이징 아웃(paging out)하여 활발히 사용 중인 페이지들을 위한 공간을 확보한다. **내보낼**(**evict**) 페이지(또는 페이지들) 선택은 운영체제의 **교체 정책**(**replacement policy**) 안에 집약되어 있다. 과거의 시스템들은 물리 메모리의 크기가 작았기 때문에 초기 가상 메모리 시스템의 가장 중요한 역할 중 하나가 바로 이 교체 정책이었다. 최소한, 좀 더 알아둘 만한 가치가 있는 정책이다. 우리가 살펴볼 핵심 문제는 다음과 같다.

핵심 질문 : 내보낼 페이지는 어떻게 결정하는가

운영체제는 어떻게 메모리에서 내보낼 페이지(또는 페이지들)를 결정할 수 있을까? 이 결정은 시스템의 교체 정책에 의해서 내려진다. 교체 정책은 보편 타당한 원칙들을 (다음에서 설명함) 따르지만 코너 케이스를 피하기 위한 수정 사항들도 포함되어 있다.

25.1 캐시 관리

정책에 대해서 논하기 전에 먼저 해결하고자 하는 문제에 대해서 좀 더 상세히 설명하도록 하겠다. 시스템의 전체 페이지들 중 일부분만이 메인 메모리에 유지된다는 것을 가정하면, 메인 메모리는 시스템의 가상 메모리 페이지를 가져다 놓기 위한 **캐시**로 생각될 수 있다. 이 캐시를 위한 교체 정책의 목표는 **캐시 미스**의 횟수를 최소화하는 것이다. 즉, 디스크로부터 페이지를 가져오는 횟수를 최소로 만드는 것이다. 한편으로 **캐시 히트** 횟수를 최대로 하는 것도 목표라고 할 수 있다. 즉, 접근된 페이지가 메모리에 이미 존재하는 횟수를 최대로 하는 것이다.

캐시 히트와 미스의 횟수를 안다면 프로그램의 **평균 메모리 접근 시간**(**average memory access time**, **AMAT**)을 계산할 수 있다(컴퓨터 아키텍트가 하드웨어

캐시의 성능을 측정할 때 사용하는 미터법이다 [HP06]). 히트와 미스의 정보를 안다면 프로그램의 AMAT는 다음과 같은 식으로 계산할 수가 있다.

$$AMAT = (P_{Hit} \cdot T_M) + (P_{Miss} \cdot T_D) \tag{25.1}$$

여기서 T_M은 메모리 접근 비용을 나타내고, T_D는 디스크 접근 비용, P_{Hit}는 캐시에서 데이터 항목을 찾을 확률을 나타내며(히트), P_{Miss}는 캐시에서 데이터를 못찾을 확률(미스)을 나타낸다. P_{Hit}와 P_{Miss}는 각각 0.0에서 1.0 사이의 값을 가지며 $P_{Miss} + P_{Hit} = 1.0$을 만족한다.

예를 들어, 어떤 컴퓨터가 4 KB의 작은 메모리를 가지고 있고, 페이지의 크기는 256 바이트라고 하자. 가상 주소는 4비트 VPN(최상위 비트)과 8비트 오프셋(최하위 비트)의 두 부분으로 구성된다. 그러므로 이 예제에서 한 프로세스가 2^4 또는 총 16개의 가상 페이지들에 접근할 수 있다. 프로세스는 가상 주소 0x000, 0x100, 0x200, 0x300, 0x400, 0x500, 0x600, 0x700, 0x800, 0x900의 메모리를 가리키고 있다. 이 가상 주소들은 주소 공간의 첫 열 개 페이지들의 첫 번째 바이트를 나타낸다(각 가상 주소들의 첫 16진수 숫자가 페이지 번호를 나타낸다).

또한, 가상 페이지 3을 제외한 모든 페이지가 이미 메모리에 있다고 가정하자. 주어진 순서의 메모리 참조는 다음과 같은 동작을 보일 것이다: 히트, 히트, 히트, 미스, 히트, 히트, 히트, 히트, 히트, 히트. **히트율(hit rate)**을(메모리에 참조 대상을 찾은 백분율) 계산하면 10개의 참조 중에서 9개가 메모리에 있었기 때문에 90%가 된다($P_{Hit} = 0.9$). **미스율(miss rate)**은 당연하게 10%($P_{Miss} = 0.1$)가 된다.

AMAT를 계산하기 위해서는 메모리를 접근하는 비용과 디스크를 접근하는 비용을 알아야 한다. 메모리를 접근하는 비용(T_M)이 약 100 nsec라고 하고 디스크를 접근하는 비용(T_D)이 약 10 μsec라고 가정할 때 AMAT은 $0.9 \cdot 100ns + 0.1 \cdot 10ms$ 이고 계산하면 $0.9ns + 1ms$ 또는 1.0009 msec 혹은 약 1 μsec가 된다. 만약 히트율이 99.9%가 되었었다면 결과는 확연히 달라진다. 즉, AMAT가 10.1 microseconds 혹은 대략 100배 더 빨라진다. 히트율이 100%에 가까워질수록 AMAT는 100 nsec에 가까워진다.

이 예제에서 볼 수 있듯이 현대 시스템에서는 디스크 접근 비용이 너무 크기 때문에 아주 작은 미스가 발생하더라도 전체적인 AMAT에 큰 영향을 주게 된다. 컴퓨터가 디스크 속도 수준으로 느리게 실행되는 것을 방지하기 위해서는 당연히 미스를 최대한 줄여야 한다. 이를 위해서는 좋은 정책을 잘 만드는 것이다. 이제 그 방법을 알아보자.

25.2 최적 교체 정책

교체 정책의 동작 방식을 잘 이해하기 위해서 **최적** 교체 정책(The Optimal Replacement Policy)과 비교하는 것이 좋다. 최적 정책(optimal replacement policy)은 수년 전 Belady에 의해 개발되었다 [Bel66](그는 원래 MIN이라고 불렀다). 최적 교체 정책은 미스를 최소화한다. Belady는 *가장 나중에 접근될* 페이지를 교체하는 것이 최적이며, 가장 적은 횟수의 미스를 발생시킨다는 것을 증명하였다. 이 정책은 간단하지만 구현하기는 어려운 정책이다.

팁 : 최적의 방법과 비교하는 것은 유용하다

최적의 방법은 비현실적이지만 시뮬레이션이나 다른 연구 목적에서 비교 대상으로 매우 유용하다. 새롭게 개발된 알고리즘이 80%의 히트율을 가진다고 주장하는 것은 의미가 없다. 주관적이기 때문이다. 그렇지만 최적 기법이 82%의 히트율을 가진다고 부가 설명하면(그러면서 새로운 방법이 최적의 결과에 매우 근접했다는 것을 보이면), 결과를 더 의미 있게 만들고, 받아들이는 사람이 맥락을 이해할 수 있게 된다. 어떤 연구를 수행하든 정답을 알고 있다면 가능한 성능 개선이 어느 정도인지 가늠할 수 있다. 또한 가장 이상적인 경우와 충분히 근접해지면 성능 개선 시도를 언제 중단할 수 있는지 알 수 있다 [Arp03].

최적 기법이 직관적으로 쉽게 이해되었기 바란다. 이렇게 생각해 보자. 만약 페이지 하나를 내보내야 한다면 지금부터 가장 먼 시점에 필요하게 될 페이지를 버리는 것이 좋지 않을까? 핵심은 가장 먼 시점에 필요한 페이지보다 캐시에 존재하는 다른 페이지들이 더 중요하다는 것이다. 이 주장이 참인 이유는 간단하다. 가장 먼 시점에 접근하게 될 페이지보다 다른 페이지들을 먼저 참조할 것이기 때문이다.

간단한 예제를 통해 최적 기법의 동작을 살펴보자. 프로그램이 0, 1, 2, 0, 1, 3, 0, 3, 1, 2, 1의 순서대로 가상 페이지들을 접근한다고 하자. 캐시에 세 개의 페이지를 저장할 수 있다고 가정할 때, 그림 25.1은 최적의 기법의 동작을 보여준다.

접근	히트/미스?	내보냄	결과적인 캐시 상태
0	미스		0
1	미스		0, 1
2	미스		0, 1, 2
0	히트		0, 1, 2
1	히트		0, 1, 2
3	미스	2	0, 1, 3
0	히트		0, 1, 3
3	히트		0, 1, 3
1	히트		0, 1, 3
2	미스	3	0, 1, 2
1	히트		0, 1, 2

〈그림 25.1〉 **최적의 교체 정책의 흐름**

그림에서 다음과 같은 동작을 볼 수 있다. 캐시는 처음에 비어 있는 상태로 시작하기 때문에 첫 세 번의 접근은 당연히 미스이다. 이러한 종류의 미스는 때로는 **최초 시작 미스**(**cold-start miss**) 또는 **강제 미스**(**compulsory miss**)라고 한다. 그 다음에 페이지 0과 1을 참조하면 캐시에서 히트된다. 마지막으로 또 다른 미스를 만나게 된다 (페이지 3). 그렇지만 이때는 캐시가 가득 차있기 때문에 교체 정책이 실행되어야 한다!

여담: 캐시 미스의 종류

컴퓨터 구조의 분야에서는 미스를 종류에 따라 분류하는 것이 때때로 유용하다. 그 분류는 세 가지로 강제 미스, 용량 미스, 그리고 충돌 미스가 있으며 이것들은 **세 개의 C**라고 부르기도 한다 [Hil87]. **강제 미스**(**compulsory miss**, 또는 **최초 시작 미스**(**cold-start miss**) [EF78])는 캐시가 비어 있었고 그 항목을 처음으로 참조하기 때문에 발생한다. **용량 미스**(**capacity miss**)는 캐시의 공간이 다찼기 때문에 새로운 항목을 캐시에 넣기 위해서 어떤 항목을 내보내야 할 때 발생한다. 세 번째 종류의 미스는(충돌 미스(conflict miss))는 **세트 연관 매핑**(**set-associativity**)이라고 불리는 것 때문에 하드웨어에서 발생한다. 페이지 캐시는 **완전 연관 매핑**(**fully-associative**)으로 되어 있기 때문에 운영체제에서는 발생하지 않는다. 즉, 메모리에 페이지를 탑재하는 위치에 제한이 없다는 말이다. 상세한 내용은 H&P에 나와 있다 [HP06].

이때 다음과 같은 질문이 나온다. 어떤 페이지를 교체해야 할까? 최적 기법의 경우 캐시에 현재 탑재되어 있는 각 페이지들(0, 1, 그리고 2)의 미래를 살펴본다. 그러면 0은 거의 즉시 접근이 될 것이며 1은 약간 후에, 그리고 2는 가장 먼 미래에 접근이 될 것이란 것을 알 수 있다. 그러므로 최적 기법은 쉬운 선택을 하면 된다. 페이지 2를 내보내고 결과적으로 캐시에는 페이지 0, 1, 3이 남게 된다. 그 후의 세 번의 참조는 히트된다. 그 다음으로 오래 전에 내보냈던 페이지 2를 만나서 또 한 번의 미스를 경험하게 된다. 여기서 최적 기법은 다시 한 번 캐시 내의 각 페이지들(0, 1, 그리고 3)의 미래를 검사한다. 그리고 페이지 1만 아니면(바로 다음에 접근될 예정이라) 어느 페이지를 내보내도 괜찮다는 것을 알게 된다. 페이지 0을 선택하는 것도 괜찮은 결정이었을 수도 있지만, 예제에서는 페이지 3을 내보냈다. 최종적으로 페이지 1이 히트되고 종료된다.

캐시의 히트율을 계산할 수 있다. 캐시 히트가 6번 미스가 5번이었으므로 히트율은 $\frac{Hits}{Hits+Misses}$으로 계산되며 $\frac{6}{6+5}$ 또는 54.5%가 된다. 강제 미스들(즉, 해당 페이지가 처음 미스된 경우)을 제외한 히트율을 계산한다면 85.7%의 히트율을 얻는다.

불행하게도 스케줄링 정책을 만들 때도 보았지만, 미래는 일반적으로 미리 알 수 없다. 그렇기 때문에 범용 운영체제[1]에서는 최적 기법의 구현은 불가능하다. 내보낼 페이지를 결정하기 위한 실제적이고 배포가 가능한 정책을 만들기 위해 다른 방법을 찾는 것에 집중할 것이다. 최적 기법은 비교 기준으로만 사용될 것이며, 이를 통해 "정답"에 얼마나 가까운지 알 수 있다.

25.3 간단한 정책: FIFO

초기의 많은 시스템들은 최적의 방법에 도달하려는 복잡한 시도를 피하고 대신 매우 간단한 교체 정책을 채용하였다. 예를 들면, 일부 시스템에서는 **FIFO**(먼저 들어온 것이 먼저 나간다, 선입선출) 교체 방식을 사용하였다. 이 방법은 시스템에 페이지가 들어오면

1) 만약 할 수 있다면, 우리에게 꼭 알려주기 바란다! 우리는 다같이 부자가 될 수 있다. 또는, 상온 핵융합의 방법을 "발견했다"던 과학자들처럼 엄청난 멸시와 조롱을 받게 될 것이다 [FP89].

접근	히트/미스?	내보냄	결과적인 캐시 상태
0	미스		선입 0
1	미스		선입 0, 1
2	미스		선입 0, 1, 2
0	히트		선입 0, 1, 2
1	히트		선입 0, 1, 2
3	미스	0	선입 1, 2, 3
0	미스	1	선입 2, 3, 0
3	히트		선입 2, 3, 0
1	미스	2	선입 3, 0, 1
2	미스	3	선입 0, 1, 2
1	히트		선입 0, 1, 2

〈그림 25.2〉 **FIFO 정책의 흐름**

큐에 삽입되고, 교체를 해야 할 경우 큐의 테일에 있는 페이지가("먼저 들어온" 페이지) 내보내진다. FIFO는 매우 구현하기 쉽다는 장점을 가진다.

예제로 사용한 메모리 참조의 흐름에서 FIFO가 어떻게 동작하는지 살펴보자(그림 25.2). 처음에는 마찬가지로 페이지 0, 1과 2에 대한 세 개의 강제 미스로 시작하고, 0과 1이 다음에 히트된다. 다음으로 페이지 3이 참조되지만 미스를 일으킨다. FIFO를 사용할 때 교체 결정은 쉽다. 순서상 "첫 번째"로 들어온 페이지인 0을 선택하면 된다(그림에서 캐시 상태는 FIFO 순서로 정렬되었다. 처음 들어온 페이지가 좌측에 있다). 불행하게도 다음 접근은 페이지 0이라서 또 다른 미스를 만들어내고 교체가(페이지 1) 필요하다. 그 후에는 페이지 3에서 히트가 되지만 1과 2는 미스가 되며, 끝으로 3은 히트된다.

최적의 경우와 비교하면 FIFO는 눈에 띄게 성능이 안 좋다. 히트율은 36.4%가 된다(또는 강제 미스를 제외하면 57.1%가 된다). FIFO는 블럭들의 중요도를 판단할 수가 없다. 페이지 0이 여러 차례 접근이 되었더라도 단순히 메모리에 먼저 들어왔다는 이유로 FIFO는 페이지 0을 내보낸다.

25.4 또 다른 간단한 정책: 무작위 선택

또 다른 유사한 교체 정책은 무작위 방식이다. 이 방식은 메모리 압박이 있을 때 페이지를 무작위로 선택하여 교체한다. 무작위 선택 방식은 FIFO와 유사한 성질을 가지고 있다. 구현하기 쉽지만 내보낼 블럭을 제대로 선택하려는 노력을 하지 않는다. 앞선 예제에서 사용한 메모리 참조를 사용하여 무작위 선택 정책이 어떻게 동작하는지 살펴보자(그림 25.3을 보자).

물론 무작위 선택 정책은 선택할 때 얼마나 운이 좋은지(또는 운이 나쁜지)에 전적으로 의존한다. 위에서 사용한 참조 열에서는 무작위 선택 방식이 FIFO 보다는 약간 더 좋은 성능을 보이며 최적의 방법보다는 약간 나쁜 성능을 보인다. 무작위 선택 방식의

여담: Belady's Anomaly

Belady(최적 기법의)와 그의 동료들은 예상과는 약간 다르게 동작하는 흥미로운 경우를 발견하였다 [BNS69]. 그 메모리 참조 흐름은 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5이다. 그들이 연구 중이던 교체 정책은 FIFO였다. 흥미로운 부분: 캐시의 크기를 3에서 4개의 페이지로 증가시킬 때 캐시 히트율은 어떻게 변할까.

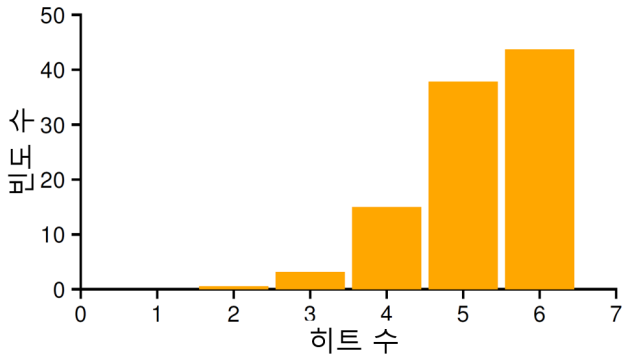
일반적으로 캐시의 크기가 커지면 캐시 히트율이 증가하는(더 좋아지는) 것을 기대할 것이다. 그런데 FIFO를 사용하는 경우에는 더 안 좋아진다! 히트와 미스를 직접 계산해서 확인해 보라. 이 이상한 현상은 일반적으로 **Belady's anomaly**라고 부른다(그의 동료들은 유감이겠지만).

LRU와 같은 다른 정책들은 이와 같은 문제를 겪지 않는다. 왜 그런지 추측할 수 있겠는가? 사실 LRU는 **스택 특성**(**stack property**)이라고 알려진 특성을 가진다 [Mat+70]. 이 특성을 가지는 알고리즘들의 경우에는 캐시 크기가 $N+1$로 증가하게 되면 자연적으로 캐시 크기 N일 때의 내용을 포함하게 되어 있다. 그러므로 캐시의 크기가 증가하면 히트율은 그대로 유지되거나 향상된다. 여러 정책들 중 FIFO와 랜덤 정책은 이 스택 특성을 따르지 않고 따라서 변칙적인 행동을 보이기 쉽다.

접근	히트/미스?	내보냄	결과적인 캐시 상태
0	미스		0
1	미스		0, 1
2	미스		0, 1, 2
0	히트		0, 1, 2
1	히트		0, 1, 2
3	미스	0	1, 2, 3
0	미스	1	2, 3, 0
3	히트		2, 3, 0
1	미스	3	2, 0, 1
2	히트		2, 0, 1
1	히트		2, 0, 1

〈그림 25.3〉 **무작위 선택 정책의 흐름**

일반적인 성능을 알아보기 위해 실험을 수천 번 반복해 볼 수 있다. 그림 25.4는 서로 다른 랜덤 시드를 가지는 실험을 만 번 수행했을 때 무작위 선택 방식이 몇 번의 히트를 얻었는지를 나타낸다. 보는 바와 같이 어떤 때는(40%가 살짝 넘는 경우들) 무작위 선택 방식이 최적 기법만큼 좋은 성능을 보이며 예제에서 6번의 히트를 얻었다. 때로는 매우 안 좋은 성능을 보이며, 2번의 히트 또는 그 이하를 얻었다. 무작위 선택 방식의 동작은 그때그때에 따라 달라진다.



〈그림 25.4〉 **10,000번 실행할 때의 무작위 선택 정책의 성능**

25.5 과거 정보의 사용: LRU

불행하게도 FIFO 또는 무작위 선택 방식처럼 단순한 정책들은 중요한 페이지들을 혹은 바로 다시 참조하게 될 것들을 내보낼 수 있다는 비슷한 문제를 겪게 된다. FIFO는 먼저 가져온 페이지를 먼저 내보낸다. 그 페이지에 중요한 코드 또는 자료 구조가 들어 있어다. 곧 다시 필요하다고 해도, 먼저 들어온 페이지라면 내보낸다. FIFO와 무작위 선택 방식 그리고 그와 유사한 정책들은 최적 기법의 성능을 따라갈 수가 없기 때문에 좀 더 정교한 방식이 필요하다.

스케줄링 정책에서와 같이 미래에 대한 예측을 위해서 *과거 사용 이력을* 활용해 보자. 예를 들어 어떤 프로그램이 가까운 과거에 한 페이지를 접근했다면 가까운 미래에 그 페이지를 다시 접근하게 될 확률이 높다.

페이지 교체 정책이 활용할 수 있는 과거 정보 중 하나는 **빈도수(frequency)**이다. 만약 한 페이지가 여러 차례 접근되었다면, 분명히 어떤 가치가 있기 때문에 교체되면 안될 것이다. 좀 더 자주 사용되는 페이지의 특징은 접근의 **최근성(recency)**이다. 더 최근에 접근된 페이지일수록 가까운 미래에 접근될 확률이 높을 것이다.

이러한 류의 정책은 **지역성의 원칙(principle of locality)**이라고 부르는 특성에 기반을 둔다 [Den70]. 이 원칙은 프로그램의 행동 양식을 관찰하여 얻은 결과이다. 이 원칙이 말하는 것은 단순하다. 프로그램들은 특정 코드들과 자료 구조를 상당히 빈번하게 접근하는 경향이 있다는 것이다. 코드의 예로는 반복문 코드를 들 수 있으며, 자료 구조로는 그 반복문에 의해 접근되는 배열을 예로 들 수 있다. 과거의 현상을 보고 어떤 페이지들이 중요한지 판단하고, 내보낼 페이지를 선택할 때 중요한 페이지들은 메모리에 보존하는 것이다.

그리하여 과거 이력에 기반한 교체 알고리즘 부류가 탄생하게 되었다. **Least-Frequently-Used(LFU)** 정책은 가장 적은 빈도로 사용된 페이지를 교체한다. **Least-Recently-Used(LRU)** 정책은 가장 오래 전에 사용하였던 페이지를 교체한다. 이러한 알고리즘들은 기억하기 쉽다. 이름을 알면 정확하게 그 알고리즘이 어떻게 동작하는지

알 수 있다. 이름이 가지는 탁월한 성질이 아닐 수 없다.

LRU 방식을 좀 더 이해하기 위해서 앞서 사용한 참조 패턴에서 예제에서 어떻게 동작하는지 알아보자. 그림 25.5에 그 결과가 있다. 그림을 보면 상태를 유지하지 않는 랜덤이나 FIFO 정책에 비해 LRU가 과거 정보를 사용하여 더 좋은 성능을 보임을 알 수 있다. 이 예제에서 LRU는 처음으로 페이지를 교체해야 할 때가 되었을 때 페이지 2를 내보낸다. 왜냐하면 0과 1이 최근에 사용되었기 때문이다. 그 후에는 페이지 0을 교체한다. 1과 3이 더 최근에 접근되었기 때문이다. 두 경우 모두 LRU는 과거 정보를 활용하였고, 결과적으로 이후의 참조들에서 히트가 발생하였다는 것을 보면 결정이 옳았다는 것을 알 수 있다. 이 예제에서 LRU가 최적 기법과 같은 정도 수준의 성능을 얻을 수 있는 최고의 결과를 보여주고 있다[2].

접근	히트/미스?	내보냄	결과적인 캐시 상태	
0	미스		LRU	0
1	미스		LRU	0, 1
2	미스		LRU	0, 1, 2
0	히트		LRU	1, 2, 0
1	히트		LRU	2, 0, 1
3	미스	2	LRU	0, 1, 3
0	히트		LRU	1, 3, 0
3	히트		LRU	1, 0, 3
1	히트		LRU	0, 3, 1
2	미스	0	LRU	3, 1, 2
1	히트		LRU	3, 2, 1

〈그림 25.5〉 **LRU 정책의 흐름**

이와 반대되는 **Most-Frequently-Used**(**MFU**)와 **Most-Recently-Used**(**MRU**)와 같은 알고리즘도 존재하지만 대부분의 경우(모든 경우는 아님!), 이러한 정책들은 잘 동작하지 않는다. 대부분의 프로그램들에서 관찰되는 지역성 접근 특성을 사용하지 않고 오히려 무시하기 때문이다.

25.6 워크로드에 따른 성능 비교

지금까지 살펴본 정책들 동작을 더 잘 이해하기 위해 몇 가지 예제들을 더 살펴보자. 이번에는 짧은 흐름 대신 좀 더 복잡한 워크로드를 살펴볼 것이다. 하지만 이러한 워크로드조차도 상당히 간단화된 것이기 때문에 제대로된 연구를 위해서는 응용 프로그램의 트레이스 자료를 사용해야 할 것이다.

첫 번째 살펴볼 워크로드에서는 지역성이 없다. 그 말은 접근되는 페이지들의 집합에서 페이지가 무작위적으로 참조된다는 것을 의미한다. 이 예제에서는 100개의

2) 결과가 조작된 것이기는 하다. 하지만 주장을 증명하기 위해서는 약간의 조작도 필요하다.

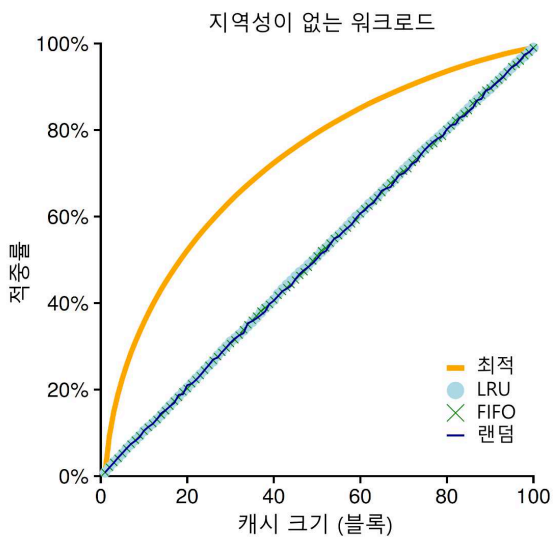
여담: 지역성의 종류

프로그램들이 보이는 지역성에는 두 가지 종류가 있다. 첫 번째는 **공간 지역성(spatial locality)**이다. 이것은 어떤 페이지 P가 접근되었다면 그 페이지 주변의 페이지들($P-1$ 또는 $P+1$이라 하자)이 참조되는 경향이 있다는 것을 말한다. 두 번째는 **시간 지역성(temporal locality)**으로 가까운 과거에 참조되었던 페이지는 가까운 미래에 다시 접근되는 경향이 있다는 것을 나타낸다. 하드웨어 캐시는 명령어와 데이터 그리고 주소 변환 등의 여러 계층에 구현되어 있으며, 프로그램은 이러한 지역성의 도움을 받아 더 빠르게 실행된다. 하드웨어 캐싱에서 지역성의 존재에 대한 가정은 매우 중요한 역할을 한다.

물론 **지역성의 원칙(principle of locality)**은 원칙이라는 이름을 가지고 있지만 모든 프로그램들이 따라야 하는 전혀 변경할 수 없는 엄격한 법칙은 아니다. 실제로, 어떤 프로그램들은 메모리(또는 디스크)를 무작위적으로 접근하며, 지역성이 명확하지 않거나 어떤 지역성도 보이지 않는다. 어떤 종류의 캐시든 설계할 때 지역성을 고려하는 것이 좋지만, 성공을 보장해주는 것은 아니다. 컴퓨터 시스템의 설계에 있어 어느 정도의 유용함이 입증된 결과일 뿐이다.

페이지들이 일정 시간 동안 계속 접근하는 워크로드를 사용한다. 접근되는 페이지는 무작위적으로 선택되며 페이지들이 총 10,000번 접근된다. 실험에서는 캐시의 크기를 매우 작은 것부터(한 페이지) 모든 페이지들을 담을 수 있을 정도의 크기까지(100페이지) 증가시켰으며, 이를 통해 각 정책이 캐시 크기에 따라 어떻게 동작하는지 살펴보았다.

그림 25.6에서는 최적의 방법과 LRU, 랜덤 그리고 FIFO 방식을 사용하였을 때의 실험 결과를 나타낸다. 그림의 y 축은 각 정책이 달성한 히트율을 나타내며 x 축은 앞서

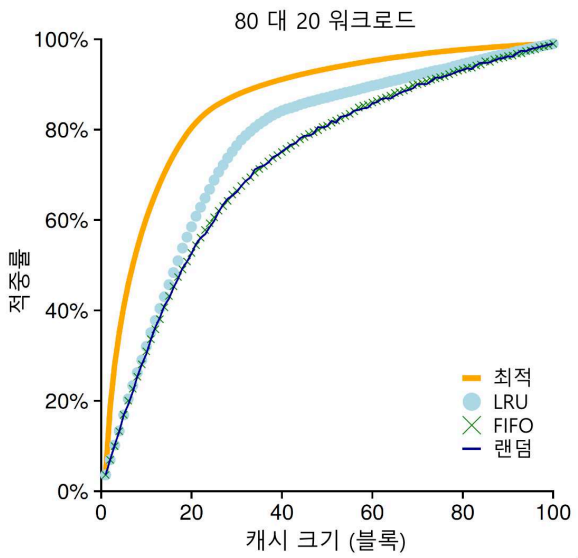


〈그림 25.6〉 **지역성이 없는 워크로드**

설명한 캐시 크기의 변화를 나타낸다.

그림에서 몇 가지 결론을 얻을 수 있다. 먼저, 워크로드에 지역성이 없다면 어느 정책을 사용하든 상관이 없다. LRU와 FIFO 그리고 무작위 선택 정책은 모두 동일한 성능을 보이며, 히트율은 정확히 캐시의 크기에 의해서 결정된다. 두 번째, 캐시가 충분히 커서 모든 워크로드를 다 포함할 수 있다면 역시 어느 정책을 사용하든지 상관 없다. 참조되는 모든 블럭들이 캐시에 들어갈 수 있으면 모든 정책들은(무작위 선택마저도) 히트율이 100%에 도달한다. 마지막으로, 최적 기법이 구현 가능한 기타 정책들보다 눈에 띄게 더 좋은 성능을 보인다. 미래를 알 수 있다면 교체 작업을 월등히 잘할 수 있다.

다음으로 살펴볼 워크로드는 "80 대 20" 워크로드라고 부른다. 20%의 페이지들에서("인기 있는" 페이지) 80%의 참조가 발생하고 나머지 80%의 페이지들에 대해서 20%의 참조만("비인기" 페이지) 발생한다. 이 워크로드에서는 마찬가지로 총 100개의 페이지들이 있다. "인기 있는" 페이지들에 대한 참조가 실험 시간의 대부분을 차지한다. 그림 25.7에는 각 정책들이 이 워크로드에서 어떻게 동작하는지 보여주고 있다.



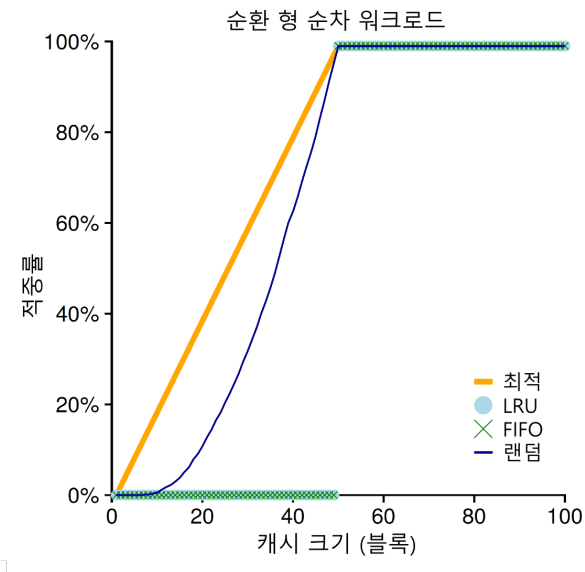
〈그림 25.7〉 **80 대 20 워크로드**

그림에서 볼 수 있듯이 랜덤과 FIFO 정책이 상당히 좋은 성능을 보이지만, 인기 있는 페이지들을 캐시에 더 오래두는 경향이 있는 LRU가 더 좋은 성능을 보인다. 인기 있는 페이지들이 과거에 빈번하게 참조되었기 때문에 그 페이지들은 가까운 미래에 다시 참조되는 경향이 있기 때문이다. 최적 기법은 여전히 더 좋은 성능을 보이고 있으며, 이는 LRU의 과거 정보가 완벽하지는 않다는 것을 보여준다.

이런 의문이 들 수도 있다. 무작위 선택 방식과 FIFO 방식을 개선하여 만든 LRU가 그렇게 대단한 것인가? 그 대답은 항상 그렇듯이 "상황에 따라 다르다" 이다. 만약 각

미스로 인해서 매우 비싼 값을 치러야 한다면(드물게 일어나는 일이 아니다) 히트율이 약간만 증가하더라도(미스율이 줄어들면) 성능에 큰 차이를 만들어 낼 수 있다. 만약 미스로 인한 영향이 그렇게 크지 않다면, LRU로 얻을 수 있는 장점은 당연히 그렇게 중요하지 않게 된다.

마지막으로 하나의 워크로드를 더 살펴보자. 이 워크로드를 "순차 반복" 워크로드라고 부른다. 이 워크로드는 50개의 페이지들을 순차적으로 참조한다. 0번 페이지를 참조하고 1번을 참조하고 ..., 49번째의 페이지를 참조한 후에 다시 처음으로 돌아가서 그 접근 순서를 반복한다. 50개의 개별적인 페이지들을 총 10,000번 접근한다. 그림 25.8에서 보이는 마지막 도표는 각 정책들이 이 워크로드에서 어떤 성능을 보이는지 나타낸다.



〈그림 25.8〉 **순환 형 워크로드**

여러 응용 프로그램에서 흔하게 볼 수 있는 이 워크로드는(데이터베이스와 같은 상업용 응용 프로그램들을 포함하여 [CD85]) LRU와 FIFO 정책에서 가장 안좋은 성능을 보인다. 순차 반복 워크로드에서 이 알고리즘들은 오래된 페이지들을 내보낸다. 불행하게도 워크로드의 반복적인 특성으로 인해서 오래된 페이지들은 정책들이 캐시에 유지하려고 선택한 페이지들보다 먼저 접근된다. 실제로, 캐시의 크기가 49라고 할지라도 50개의 페이지들을 순차 반복하는 워크로드에서는 캐시 히트율이 0%가 된다. 흥미롭게도 무작위 선택 정책은 최적의 경우에 못 미치기는 하지만 눈에 띄게 좋은 성능을 보인다. 무작위 선택 정책의 히트율은 최소한 0%는 아니다. 이렇게 무작위 선택 정책은 몇 가지 좋은 특성을 가지고 있다는 것을 알 수 있다. 그 중 한 가지는 이상한 코너 케이스가 발생하지 않는다는 것이다.

25.7 과거 이력 기반 알고리즘의 구현

살펴본 것처럼 중요한 페이지들을 내보낼 수도 있는 단순한 FIFO와 무작위 선택 정책들보다 LRU와 같은 알고리즘이 더 좋은 성능을 보인다. 하지만 과거 정보에 기반을 둔 정책은 새로운 문제점이 있다. 이런 정책을 어떻게 구현할 것인가?

LRU를 예로 들어 보자. 이를 완벽하게 구현하기 위해서는 많은 작업을 해야 한다. 특히, 각 *페이지 접근마다*(즉, 명령어 탑재나 로드 또는 저장하려는 각 메모리 접근) 해당 페이지가 리스트에 가장 앞으로 이동하도록(즉, MRU 측으로) 자료 구조를 *갱신*해야 한다. FIFO 방식과 비교해 보자. FIFO 리스트는 어떤 페이지의 *제거*(가장 먼저 들어온 페이지를 제거하기) 또는 새로운 페이지의 삽입 시에만(가장 나중에 들어온 쪽으로) 접근된다. LRU에서는 어떤 페이지가 가장 최근에 또는 가장 오래 전에 사용되었는지를 관리하기 위해서 *모든 메모리 참조 정보를* 기록해야 한다. 세심한 주의 없이 정보를 기록하면 성능이 크게 떨어질 수 있다.

이 작업을 좀 더 효율적으로 하는 방법은 약간의 하드웨어 지원을 받는 것이다. 예를 들어 페이지 접근이 있을 때마다 하드웨어가 메모리의 시간 필드를 갱신하도록 할 수 있다(예를 들어, 이 정보를 각 프로세스의 페이지 테이블에 포함시키거나, 물리 페이지마다 한 항목씩 할당된 배열로 보관할 수도 있다). 이렇게 하여 페이지가 접근되면 하드웨어가 시간 필드를 현재 시간으로 설정한다. 페이지 교체 시에 운영체제는 가장 오래 전에 사용된 페이지를 찾기 위해 시간 필드를 검사한다.

시스템의 페이지 수가 증가하면 페이지들의 시간 정보 배열을 검색하여 가장 오래 전에 사용된 페이지를 찾는 것은 매우 고비용의 연산이 된다. 4GB의 메모리를 가지는 현대의 기기에서 4KB 페이지들로 나눈 것을 상상해 보라. 그런 기계는 백만 개의 페이지들을 가지고 있다. LRU 정책의 교체 대상 페이지를 찾는 데 현대의 CPU 속도라 할지라도 오랜 시간이 걸리게 된다. 다음과 같은 질문이 떠오른다. 가장 오래된 페이지를 꼭 찾아야만 할까? 대신 비슷하게 오래된 페이지를 찾아도 되지 않을까?

핵심 질문: LRU 교체 정책을 구현하는 방법
완벽한 LRU를 구현하는 것은 비싼 비용이 든다는 것을 전제로, 어떻게 하면 LRU에 가까운 선택을 하고 유사한 행동을 보이는 정책을 구현할 수 있을까?

25.8 LRU 정책 근사하기

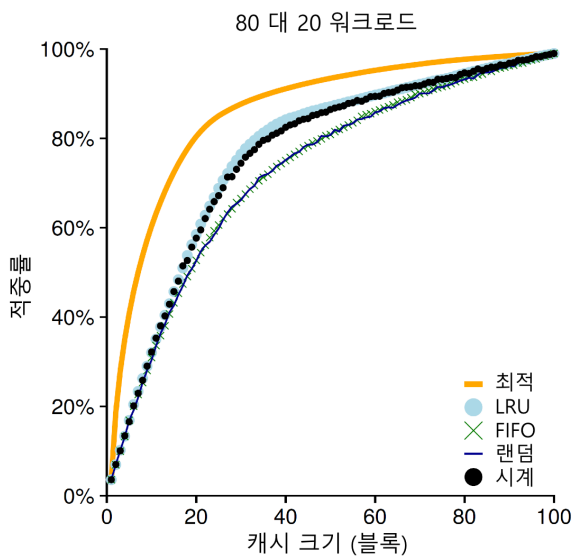
LRU 정책에 가까운 구현이 가능한가라는 질문에 대한 답은 "예"이다. 연산량이라는 관점에서 볼 때, LRU를 "근사"하는 식으로 만들면 구현이 훨씬 쉬워진다. 실제로 현대의 많은 시스템이 이런 방식을 택하고 있다. 이 개념에는 **use bit**(때로는 **reference bit**라고도 불린다)라고 하는 약간의 하드웨어 지원이 필요하다. 이 기법은 페이징을 최초로 적용한 Atlas one-level store 시스템에 처음으로 구현되었다 [Kil+62]. 시스템의 각 페이지마다 하나의 use bit가 있으며, 이 use bit는 메모리 어딘가에 존재한다(구현에

따라 프로세스마다 가지고 있는 페이지 테이블에 있을 수도 있고 또는 어딘가에 배열의 형태로 존재할 수도 있다). 페이지가 참조될 때마다(즉, 읽히거나 기록되면) 하드웨어에 의해서 use bit가 1로 설정된다. 하드웨어는 이 비트를 절대로 지우지 않는다(즉, 0으로 설정하지 않는다). 0으로 바꾸는 것은 운영체제의 몫이다.

운영체제는 LRU에 가깝게 구현하기 위해서 use bit를 어떻게 활용할까? 여러 가지 방법이 있을 수 있지만, 간단한 활용법이 **시계 알고리즘**(**clock algorithm**) [Cor69]에서 제시되었다. 시스템의 모든 페이지들이 환형 리스트를 구성한다고 가정하자. **시계 바늘**(**clock hand**)이 특정 페이지를 가리킨다고 해 보자(어떤 것이든 상관이 없다). 페이지를 교체해야 할 때 운영체제는 현재 바늘이 가리키고 있는 페이지 P의 use bit가 1인지 0인지 검사한다. 만약 1이라면 페이지 P는 최근에 사용되었으며 바람직한 교체 대상이 *아니라*는 것을 뜻한다. P의 use bit는 0으로 설정되고(지워짐) 시계 바늘은 다음 페이지 $P+1$로 이동한다. 알고리즘은 use bit가 0으로 설정되어 있는, 즉 최근에 사용된 적이 없는, 페이지 찾을 때까지 반복된다(최악의 경우에는 모든 페이지들이 사용된 적이 있어서 모든 페이지들을 모두 탐색하면서 use bit를 전부 0으로 설정해야 할 수도 있다).

이 방법 외에도 use bit를 사용하여 LRU를 근사하는 다른 방법들이 존재한다. 주기적으로 use bit를 지우고, 교체 대상 페이지 선택을 위해 use bit가 1과 0인 페이지를 구분할 수 있으면 된다. Corbato의 알고리즘은 미사용 페이지를 찾기 위해 매번 모든 메모리를 검사하지 않아도 되는 좋은 특성을 가진 하나의 초기 방법일 뿐이다.

변형된 시계 알고리즘의 동작이 그림 25.9에 나와 있다. 이 변형 방식은 교체할 때 페이지들을 랜덤하게 검사한다. reference bit가 1로 설정되어 있는 페이지를 만나게 되면 비트를 지운다(즉, 0으로 설정한다). reference bit가 0으로 설정되어 있는 페이지를 만나면 그 페이지를 교체 대상으로 선정한다. 완벽한 LRU 만큼 좋은 성능을 보이지는



〈그림 25.9〉 **시계 기법을 사용한 80 대 20 워크로드**

않지만 과거 정보를 고려하지 않는 다른 기법들에 비해서는 성능이 더 좋다.

25.9 갱신된 페이지(Dirty Page)의 고려

많은 곳에서 실제 사용되고 있는 시계 알고리즘에 대한 추가 개선 사항을 논의하고자 한다. 이 사항은 Corbato [Cor69]가 처음 제안하였으며, 운영체제가 교체 대상을 선택할 때 메모리에 탑재된 이후에 변경되었는지를 추가적으로 고려하는 것이다. 이것이 필요한 이유는 다음과 같다.

만약에 어떤 페이지가 **변경**(**modified**)되어 **더티**(**dirty**) 상태가 되었다면, 그 페이지를 내보내기 위해서는 디스크에 변경 내용을 기록해야 하기 때문에 비싼 비용을 지불해야 한다. 만약 변경되지 않았다면(그러므로 깨끗한 상태(clean)), 내보낼 때 추가 비용은 없다. 해당 페이지 프레임은 추가적인 I/O 없이 다른 용도로 재사용될 수 있다. 때문에 어떤 VM 시스템들은 더티 페이지 대신 깨끗한 페이지를 내보내는 것을 선호한다.

이와 같은 동작을 지원하기 위해서 하드웨어는 **modified bit**(**더티 비트**라고도 불림)를 포함해야 한다. 페이지가 변경될 때마다 이 비트가 1로 설정되므로 페이지 교체 알고리즘에서 이를 고려하여 교체 대상을 선택한다. 예를 들어, 시계 알고리즘은 교체 대상을 선택할 때 사용되지 않은 상태이고 깨끗한, 두 조건을 모두 만족하는 페이지를 먼저 찾도록 수정된다. 이러한 조건을 만족시키는 페이지를 찾는 데 실패하면, 수정되었지만 한동안 사용되지 않았던 페이지를 찾는다.

25.10 다른 VM 정책들

페이지 교체 정책만이 VM 시스템이 채택하는 유일한 정책은 아니다(그 중에 가장 중요한 것이기는 하다). 예를 들어, 운영체제는 언제 페이지를 메모리로 불러들일지 결정해야 한다. 이 정책은 운영체제에게 몇 가지 다른 옵션을 제공한다. 이 정책은 Denning이 불렀던 것처럼 때로는 **페이지 선택**(**page selection**) 정책이라고 불린다 [Den70]).

운영체제는 대부분의 페이지를 읽어 들일 때 **요구 페이징**(**demand paging**) 정책을 사용한다. 이 정책은 말 그대로 "요청된 후 즉시", 즉 페이지가 실제로 접근될 때 운영체제가 해당 페이지를 메모리로 읽어 들인다. 운영체제는 어떤 페이지가 곧 사용될 것이라는 것을 대략 예상할 수 있기 때문에 미리 메모리로 읽어 들일 수도 있다. 이와 같은 동작을 **선반입**(**prefetching**)이라고 하며 성공할 확률이 충분히 높을 때에만 해야 한다. 예를 들어, 어떤 시스템은 코드 페이지 P가 메모리에 탑재되면 $P+1$이 접근될 확률이 높기 때문에 P가 탑재될 때 $P+1$도 함께 탑재되어야 한다고 가정할 수 있을 것이다.

또 다른 정책은 운영체제가 변경된 페이지를 디스크에 반영하는 데 관련된 방식이다. 한 번에 한 페이지씩 디스크에 쓸 수 있지만, 많은 시스템은 기록해야 할 페이지들을 메모리에 모은 후, 한 번에 (더 효율적으로) 디스크에 기록한다. 이와 같은 동작을 **클러스터링**(**clustering**) 또는 단순하게 쓰기 **모으기**(**grouping** of writes)라고 부르며 효과적인 동작 방식이다. 왜냐하면 디스크 드라이브는 여러 개의 작은 크기의 쓰기

요청을 처리하는 것보다 하나의 큰 쓰기 요청을 더 효율적으로 처리할 수 있는 특성을 갖고 있기 때문이다.

25.11 쓰래싱(Thrashing)

마치기 전에 마지막으로 한 가지 질문에 대해 다루고자 한다. 메모리 사용 요구가 감당할 수 없을 만큼 많고 실행 중인 프로세스가 요구하는 메모리가 가용 물리 메모리 크기를 초과하는 경우에 운영체제는 어떻게 해야 하는가? 이런 경우 시스템은 끊임없이 페이징을 할 수밖에 없고, 이와 같은 상황을 **쓰래싱**(**thrashing**)이라고 부른다 [Den70].

몇몇 초기 운영체제들은 쓰래싱이 발생했을 때, 이의 발견과 해결을 위한 상당히 정교한 기법들을 가지고 있었다. 예를 들면, 다수의 프로세스가 존재할 때, 일부 프로세스의 실행을 중지시킨다. 실행되는 프로세스의 수를 줄여서 나머지 프로세스를 모두 메모리에 탑재하여 실행하기 위해서이다. **워킹 셋**(**working set**)이란 프로세스가 실행 중에 일정 시간 동안 사용하는 페이지들의 집합이다. 일반적으로 **진입 제어**(**admission control**)라고 알려져 있는 이 방법은 많은 일들을 엉성하게 하는 것보다는 더 적은 일을 제대로 하는 것이 나을 때가 있다고 말한다. 현대 컴퓨터 시스템에서 뿐만 아니라 실제 삶에서도 종종 부딪치는 상황이기도 하다(아, 슬프다).

일부 최신 시스템들은 메모리 과부하에 대하여 좀 더 과감한 조치를 취하기도 한다. 예를 들면, 일부 버전의 Linux는 메모리 요구가 초과되면 **메모리 부족 킬러**(**out-of-memory killer**)를 실행시킨다. 이 데몬은 많은 메모리를 요구하는 프로세스를 골라 죽이는, 그다지 교묘하지 않은 방식으로 메모리 요구를 줄인다. 메모리 요구량을 줄이는 데는 성공적일지 몰라도, 이 방법은 문제가 될 수 있다. 예를 들어, 만약 X 서버를 죽이게 되면 화면이 필요한 모든 응용 프로그램들을 쓸모없게 만든다.

25.12 요약

모든 현대 운영체제 VM 시스템의 구성 요소인 페이지 교체 정책들과 기타 다른 정책들에 대해 다루었다. 현대 시스템들은 시계 알고리즘과 같은 순수한 LRU 근사 방법에 몇 가지 성능 향상을 위한 기능을 추가하였다. 예를 들어, **탐색 내성**(**scan resistance**)은 ARC [MM03]와 같은 다수의 현대 알고리즘의 중요한 요소이다. 탐색 내성이 있는 알고리즘은 대개는 LRU와 유사하게 동작하지만 LRU가 최악의 경우에 보이는 행동을 방지하였다. 최악의 현상은 순차 반복 워크로드 예에서 보았다. 페이지 교체 알고리즘의 진화는 계속된다.

지금까지 언급했던 알고리즘의 중요성은 점차 퇴색되고 있다. 메모리 접근 시간과 디스크 접근 시간의 차이가 점차 증가하고 있기 때문이다. 디스크에 페이징 하는 비용이 너무 비싸기 때문에 빈번한 페이징을 감당할 수 없다. 과도한 페이징에 대한 최적의 해결책은 아주 간단하다. 그냥 더 많은 메모리를 구입해라.

참고 문헌

[Arp03] **"Run-Time Adaptation in River"**
Remzi H. Arpaci-Dusseau
ACM TOCS, 21:1, February 2003
저자 중 한 명이 River라 불리던 시스템에서 했던 학위논문 연구의 정리이다. 시스템 설계자에게는 이상적인 경우와 비교하는 것이 중요한 기법이라는 것을 확실하게 배웠던 연구.

[BNS69] **"An Anomaly in Space-time Characteristics of Certain Programs Running in a Paging Machine"**
L.A. Belady, R.A. Nelson, and G.S. Shedler
Communications of the ACM, 12:6, June 1969
Belady's anomaly라고 알려진 짧은 메모리 참조 순서에 대한 소개. Nelson과 Shedler는 이 이름에 대해서 어떻게 생각할지 궁금하다.

[Bel66] **"A Study of Replacement Algorithms for Virtual-Storage Computer"**
Laszlo A. Belady
IBM Systems Journal 5(2): 78-101, 1966
정책이 보일 수 있는 최적의 행동을 계산하는 간단한 방법(MIN 알고리즘)을 소개하는 논문.

[CD85] **"An Evaluation of Buffer Management Strategies for Relational Database Systems"**
Hong-Tai Chou and David J. DeWitt
VLDB '85, Stockholm, Sweden, August 1985
여러 일반적인 데이터베이스 접근 패턴을 대상으로 사용할 수 있는 여러 버퍼링 전략에 대한 유명한 데이터베이스 논문이다. 좀 더 일반적인 교훈은 이렇다. 워크로드의 특성을 잘 알고 있으면 운영체제에서 보통 사용되는 범용 정책들보다 더 잘 동작하는 정책으로 가다듬을 수 있다.

[Cor69] **"A Paging Experiment with the Multics System"**
F.J. Corbato
Included in a Festschrift published in honor of Prof. P.M. Morse, MIT Press, Cambridge, MA, 1969
시계 알고리즘에 관한 최초의 그리고 엄청 찾기 어려운 참고 문헌이다. 비록 use bit를 활용한 첫 사례는 아니더라도. 우리를 위해서 이 논문을 찾아준 MIT의 H. Balakrishnan에게 감사한다.

[Den70] **"Virtual Memory"**
Peter J. Denning
Computing Surveys, Vol. 2, No. 3, September 1970
가상 메모리 시스템에 대한 Denning이 쓴 초창기의 유명한 개관.

[EF78] **"Cold-start vs. Warm-start Miss Ratios"**
Malcolm C. Easton and Ronald Fagin
Communications of the ACM, 21:10, October 1978
캐시가 완전히 비어 있는 상태로부터 측정한 미스(cold-start miss)와 캐시를 일단 가득 채운 후 측정한 미스(warm-start miss)에 관한 훌륭한 논의.

[FP89] **"Electrochemically Induced Nuclear Fusion of Deuterium"**
Martin Fleischmann and Stanley Pons
Journal of Electroanalytical Chemistry, Volume 26, Number 2, Part 1, April, 1989

작은 금속이 담겨 있는 물로 거의 무한한 전력을 손쉽게 생성할 수 있는 방법을 제공하여 세상에 혁명을 가져올 뻔한 아주 유명한 논문이다. 불행하게도, Pon과 Fleischmann이 게재하고 광고한 결과는 재현이 불가능한 것으로 판명 났으며, 이 두 선의의 과학자들은 평판이 떨어졌다(그리고 당연히 조롱거리가 되었다). 이 결과로 인해서 유일하게 행복했던 사람은 공동으로 작업을 하였지만 이 논문에서 이름이 제외되었던 Marvin Hawkins이다. 그렇게 Marvin은 20세기의 가장 커다란 과학적 실수에 자신의 이름이 엮이는 걸 피할 수 있었다.

[HP06] **"Computer Architecture: A Quantitative Approach"**
John Hennessy and David Patterson
Morgan-Kaufmann, 2006
컴퓨터 구조에 대한 훌륭하고 놀라운 책이다. 반드시 읽어라!

[Hil87] **"Aspects of Cache Memory and Instruction Buffer Performance"**
Mark D. Hill
Ph.D. Dissertation, U.C. Berkeley, 1987
Mark Hill은 그의 학위 논문 연구에서 3C를 소개하였고 그 이후에 H&P[HP06]에 포함되면서 대중적인 인기를 얻게 되었다. 책의 구절을 인용하면 다음과 같다. "미스를 구분하는 것이 유익하다는 것을 발견하였다... 미스의 원인에 따라 직관적으로 세 가지 그룹으로 나누었다(페이지 49)."

[Kil+62] **"One-level Storage System"**
T. Kilburn, D.B.G. Edwards, M.J. Lanigan, and F.H. Sumner
IRE Trans. EC-11:2, 1962
Atlas가 use bit를 가지고 있기는 했지만 페이지 개수가 매우 적었다. 그렇기 때문에 대용량 메모리에서 use bit를 탐색하는 문제는 저자들이 풀었던 문제에는 포함되지 않았다.

[Mat+70] **"Evaluation Techniques for Storage Hierarchies"**
R.L. Mattson, J. Gecsei, D.R. Slutz, and I.L. Traiger
IBM Systems Journal, Volume 9:2, 1970
이 논문은 주로 캐시 계층 구조를 어떻게 효율적으로 시뮬레이션 하는가를 다룬다. 분명히 그 분야의 고전이라 할 수 있으며, 여러 교체 알고리즘의 일부 특성에 관한 훌륭한 논의를 담고 있기도 하다. 스택 특성을 이용하면 다양한 크기의 캐시들을 한 번에 시뮬레이션 할 수 있는 이유를 생각해 낼 수 있겠는가?

[MM03] **"ARC: A Self-Tuning, Low Overhead Replacement Cache"**
Nimrod Megiddo and Dharmendra S. Modha
FAST 2003, February 2003, San Jose, California
일부 시스템에서 사용하고 있는 ARC라고 하는 새로운 정책을 포함하여 교체 알고리즘에 대한 훌륭한 최근의 논문이다. 2014년 FAST'14 학회에서 storage systems community가 선정한 "Test of Time" 상의 수상 논문이다.

숙제

`paging-policy.py`라는 시뮬레이터는 여러 페이지 교체 정책들을 대상으로 실험해 볼 수 있게 한다. 자세한 내용은 README를 읽어보자.

문제

1. 다음 인자를 주고 랜덤 주소를 생성하라: `-s 0 -n 10`, `-s 1 -n 10`, 그리고 `-s 2 -n 10`. FIFO 정책에서 LRU 정책으로 그리고 OPT 정책으로 변경하라. 생성된 주소 흐름의 모든 주소에 대해 히트인지 미스인지를 판별하라.
2. 캐시 크기가 5인 경우에 대해서 FIFO, LRU 그리고 MRU 정책을 사용할 때 최악의 주소 참조 스트림을 생성하라(최악의 참조 스트림은 가능한 최대 개수의 미스를 발생시킨다). 최악의 참조 스트림의 경우 성능을 극적으로 향상시켜 OPT의 성능에 근접하게 만들려면 캐시의 크기는 얼마나 커야 하는가?
3. 랜덤한 추적 기록을 생성하라(python 또는 perl을 사용하라). 이런 추적 기록을 적용할 때 여러 정책이 어떻게 동작할 것이라고 예상하는가?
4. 얼마간 지역성을 보이는 추적 기록를 생성하라. 그런 추적 기록을 어떻게 생성할 수 있을까? 생성된 추적 기록대로 메모리 참조가 일어날 때 LRU의 성능은 어떤가? RAND는 LRU에 비해서 얼마나 더 좋은 성능을 보이는가? CLOCK은 어떤 성능을 보이는가? 시계 비트의 수를 변경하였을 때 CLOCK은 어떤 성능을 보이는가?
5. Valgrind와 같은 프로그램을 사용하여 실제 응용 프로그램으로부터 정보를 추출하여 가상 페이지 참조 스트림을 생성하라. 예를 들어, `valgrind --tool=lackey --trace-mem=yes ls` 명령어를 실행시키면 `ls` 프로그램에 의해서 참조된 모든 명령과 데이터에 대한 거의 완벽한 참조 추적 기록을 출력할 것이다. 이 정보를 시뮬레이터에서 사용하기 위해서는 먼저 가상 메모리 참조를 가상 페이지 번호 참조로 변환해야 한다. 가상 주소의 오프셋 부분을 제거한 후(masking off), 오프셋 비트 수만큼 오른쪽으로 시프트하면 된다. 당신의 응용 프로그램의 경우, 참조 요청의 대부분을 만족시키는 캐시의 크기는 얼마인가? 캐시의 크기가 증가할 때 응용 프로그램의 작업 집합의 변화를 그래프로 나타내어라.

대조 번역 · 학습 보충본

물리 메모리 크기의 극복: 정책

영어 22장 · 한국어 구판 25장 | 추가·개정 부분 번역

영어: vm-beyondphys-policy.pdf (18쪽, V1.10) / 한국어: 22_vm-beyondphys-policy.pdf (18쪽, V0.90)

두 PDF의 본문, 그림 22.1-22.9와 25.1-25.9, 팁·여담·각주, 참고문헌, 연습문제 5개를 마지막 페이지까지 대조했다. 새 절이 통째로 추가된 것은 아니다. 추가된 문장과 설명, 개정된 AMAT 식, 새 참고문헌 2개를 아래에 수록했다. 이미 번역된 본문과 그림은 반복하지 않는다.

쪽수는 각 첨부 PDF의 첫 페이지를 1쪽으로 센다. 본문 쪽수와도 일치한다. 절 번호는 영어판을 기준으로 쓰고 한국어판 대응 위치를 함께 표시했다. [신규]는 대응 내용 없음, [부분 누락]은 기존 문단에 빠진 설명, [개정]은 같은 주제의 내용 변경을 뜻한다. 대조 설명과 검산은 번역 본문과 구분했다.

수록 내용

1. 평균 메모리 접근 시간(AMAT)의 개정

2. 기존 문단에 빠진 설명

3. 요약의 추가 설명과 SSD에 따른 변화

4. 새 참고문헌 2개

5. 대조 중 확인한 기존 번역의 방향·번호 차이

한국어 원본 전체에 이어지는 추가 번역

앞부분: 원본 18쪽 / 뒷부분: 보충 4쪽 / 이 파일: 총 22쪽

1. 평균 메모리 접근 시간(AMAT)의 개정

[개정] 영어 §22.1, 1-2쪽, 식 (22.1) ↔ 한국어 §25.1, 2쪽, 식 (25.1)

대조: 한국어판은 P_Hit·T_M + P_Miss·T_D를 사용하지만 영어판은 미스가 발생해도 메모리 접근 비용 T_M을 항상 부담하는 모델로 바뀌었다. 이 부분은 통째로 새로 생긴 절이 아니라 공식과 설명의 개정이다. 기존 예제의 주소열과 히트·미스열은 그대로이므로 반복하지 않는다.

이 값들을 알면 프로그램의 평균 메모리 접근 시간은 다음과 같이 계산할 수 있다.

$$\mathbf{AMAT = T_M + P_{Miss} \times T_D}$$

여기서 T_M은 메모리 접근 비용, T_D는 디스크 접근 비용, P_{Miss}는 캐시에서 데이터를 찾지 못할 확률, 즉 미스 확률을 나타낸다. P_{Miss}는 0.0에서 1.0 사이의 값을 가진다. 때로는 확률 대신 미스율을 백분율로 표현한다. 예를 들어 미스율이 10%이면 P_{Miss} = 0.10이다. 데이터를 메모리에서 접근하는 비용은 항상 지불한다는 점에 유의하자. 미스가 발생하면 디스크에서 데이터를 가져오는 비용도 추가로 지불해야 한다.

메모리 접근 비용 T_M을 약 100나노초, 디스크 접근 비용 T_D를 약 10밀리초라고 하자. 미스율이 10%인 경우 AMAT는 100 ns + 0.1 × 10 ms이다. 이는 100 ns + 1 ms, 즉 1.0001 ms이며 약 1밀리초이다. 히트율이 99.9%, 즉 P_{Miss} = 0.001이라면 결과는 크게 달라진다. AMAT는 10.1마이크로초로, 대략 100배 빠르다. 히트율이 100%에 가까워지면 AMAT는 100나노초에 가까워진다.

검산: 100 ns = 0.0001 ms = 0.1 µs. 따라서 10% 미스에서는 1.0001 ms, 0.1% 미스에서는 10.1 µs가 맞다. 두 값의 비는 약 99.02이다. 한국어판에 적힌 10 µsec, 0.9 ns, 1.0009 msec, 약 1 µsec는 영어 신판의 조건·계산과 맞지 않으므로 위 값을 사용한다.

같은 절의 표현 변경: 영어 예제의 4 KB는 물리 메모리 크기가 아니라 작은 가상 주소 공간의 크기이다. 따라서 한국어판의 "4KB의 작은 메모리"는 "4 KB의 작은 주소 공간"으로 읽는다. 256바이트 페이지, VPN 4비트, 오프셋 8비트, 가상 페이지 2^4 = 16개는 두 판이 대응한다.

2. 기존 문단에 빠진 설명

[부분 누락] 영어 §22.6, 9쪽 ↔ 한국어 §25.6, 10쪽

워크로드에 지역성이 없으면 **현실적으로 구현 가능한 교체 정책 중 어떤 것을 쓰는지는 큰 차이를 만들지 않는다.**

대조: 한국어판의 "어느 정책을 사용하든 상관이 없다"에 빠진 한정은 "현실적으로 구현 가능한 정책"이다. 이어지는 LRU·FIFO·Random의 성능 설명과 OPT의 우수성은 이미 번역되어 있다. 이 한정을 넣어 OPT까지 성능이 같다고 읽히지 않게 한다.

연습문제 3의 실행 절차

[부분 누락·개정] 영어 Homework, 18쪽, 문제 3 ↔ 한국어 숙제, 18쪽, 문제 3

즉, Python을 사용하여 임의의 주소들을 출력하는 스크립트를 작성하고, 그 출력을 시뮬레이터에 입력할 수 있다.

한국어판은 "python 또는 perl을 사용하라"라고만 적고 있다. 위 문장은 영어판 괄호 안에 명시된 출력 생성·시뮬레이터 입력 절차를 보충한다. 랜덤 트레이스를 생성하고 정책들의 성능을 예상하라는 문제 본문은 중복 수록하지 않았다.

3. 요약의 추가 설명과 SSD에 따른 변화

[부분 누락·개정·신규] 영어 §22.12, 15쪽 ↔ 한국어 §25.12, 15쪽

한국어판의 마지막 문단은 디스크 페이징 비용 때문에 교체 알고리즘의 중요성이 줄어들며 메모리를 더 사는 것이 낫다는 설명으로 끝난다. 영어판은 이 경향을 과거의 현상으로 서술하고, 아래 설명과 SSD 관련 새 문단을 덧붙인다. ARC와 탐색 내성에 관한 첫 문단은 이미 한국어판에 있다.

쉽게 말해, 교체 알고리즘이 아무리 훌륭하더라도 교체가 빈번하면 시스템은 견디기 어려울 만큼 느려졌다. 따라서 최선의 해결책은 간단했다. 지적으로는 만족스럽지 않을지 몰라도, 메모리를 더 사는 것이었다.

그러나 훨씬 빠른 저장 장치, 예를 들어 플래시 기반 SSD의 최근 혁신은 이러한 성능 비율을 다시 바꾸었고, 페이지 교체 알고리즘 연구의 부흥을 이끌었다. 이 분야의 최근 연구는 [SS10, W+21]을 참조하라.

판본 안내: "최근"은 첨부 영어 원문의 서술 시점을 따른 표현이다. 이 자료는 영어 신판의 추가 내용을 번역한 것이며, 2026년의 저장 장치·운영체제 동작을 새로 조사한 설명은 아니다.

4. 새 참고문헌 2개

[신규] 영어 References, 17쪽 ↔ 한국어 참고문헌, 16-17쪽에 대응 항목 없음

[SS10] Mohit Saxena, Michael M. Swift. "FlashVM: Virtual Memory Management on Flash." USENIX ATC'10, 2010년 6월, 미국 보스턴.

제목 번역: "FlashVM: 플래시에서의 가상 메모리 관리." 위스콘신 대학교의 동료들이 플래시를 페이징에 사용하는 방법을 다룬 초기의 훌륭한 논문이다. 흥미로운 점 하나는 플래시 기반 장치의 본질적인 특성인 마모를 시스템이 고려해야 한다는 것이다. 관심이 있다면 이 책 뒤에서 플래시 기반 SSD에 관해 더 읽어 보라.

[W+21] Kan Wu, Zhihan Guo, Guanzhou Hu, Kaiwei Tu, Ramnatthan Alagappan, Rathijit Sen, Kwanghyun Park, Andrea Arpaci-Dusseau, Remzi Arpaci-Dusseau. "The Storage Hierarchy is Not a Hierarchy: Optimizing Caching on Modern Storage Devices with Orthus." FAST'21, 온라인 개최.

제목 번역: "저장 계층은 계층이 아니다: Orthus를 이용한 현대 저장 장치의 캐싱 최적화." 학회는 온라인으로 열렸다. 코로나19 덕분이다. 현대 장치의 캐싱 접근법에 관한 저자들 자신의 연구이다. 페이지 교체와 직접 관련이 있는가? 어쩌면 그렇지 않을 수도 있다. 하지만 재미있는 논문이다.

참고문헌 대조: 나머지 13개 항목은 모두 대응한다. [AD03]↔[Arp03], [B66]↔[Bel66], [C69]↔[Cor69], [D70]↔[Den70], [H87]↔[Hil87], [KE+62]↔[Kil+62], [M+70]↔[Mat+70]은 인용 키만 달라 같은 문헌으로 처리했다. Easton·Fagin의 Cold-start vs. Warm-start Miss Ratios도 두 판의 [EF78] 항목으로 대응하며 추가하지 않았다.

5. 대조 중 확인한 기존 번역의 방향·번호 차이

대조 주석 | 아래 항목은 누락된 새 절이 아니며, 문장 전체를 다시 번역하지 않았다.

영어 §22.3, 5쪽의 FIFO 예제는 마지막에 페이지 **1**이 히트한다. 한국어 §25.3, 5쪽의 "끝으로 3은 히트된다"는 **"끝으로 1은 히트된다"**로 읽는다. 두 판의 표에 적힌 마지막 접근은 모두 1이다.

영어 18쪽 연습문제 4는 **"LRU는 RAND보다 얼마나 더 나은가?"**라고 묻는다. 한국어 18쪽의 "RAND는 LRU에 비해서 얼마나 더 좋은 성능을 보이는가?"는 비교 방향이 반대다. 나머지 문제 4의 질문들은 대응한다.

이 주석은 비교 방향과 번호를 명확히 하기 위한 것이며 추가 번역 분량으로 세지 않는다. 이미 있는 본문·표·문제를 다시 수록하지 않았다.

전체 대조표: 마지막 페이지까지 확인

영 = 영어 PDF 쪽수 / 한 = 한국어 PDF 쪽수. 대응 내용은 번역 분량에 넣지 않았다. 장 번호·인용 키·배치 차이는 내용 누락과 구별했다.

원문 절·대응 위치	쪽수	대조 결과
서론·핵심 질문	영 1 / 한 1	대응. 새 본문 없음.
§22.1 ↔ §25.1	영 1-2 / 한 1-2	AMAT 식·정의·계산 개정. 자료 1.
§22.2 ↔ §25.2	영 2-4 / 한 2-4	본문·그림 1·팁·미스 종류·각주 1 대응.
§22.3 ↔ §25.3	영 4-5 / 한 4-5	그림 2·본문 대응. 마지막 페이지 번호 주석.
§22.4 ↔ §25.4	영 6 / 한 5-7	그림 3-4 대응. Belady 여담은 영 5 / 한 6.
§22.5 ↔ §25.5	영 7-8 / 한 7-8	그림 5·MFU/MRU·각주 2 대응.
§22.6 ↔ §25.6	영 8-10 / 한 8-11	그림 6-8·지역성 여담 대응. 한정 설명 보충.
§22.7 ↔ §25.7	영 11-12 / 한 12	LRU 구현·핵심 질문 대응.
§22.8 ↔ §25.8	영 12-13 / 한 12-14	Atlas·use bit·시계 알고리즘·그림 9 대응.
§22.9 ↔ §25.9	영 13 / 한 14	더티 페이지 설명 대응.
§22.10 ↔ §25.10	영 14 / 한 14-15	요구 페이징·선반입·클러스터링 대응.
§22.11 ↔ §25.11	영 14 / 한 15	쓰래싱·진입 제어·OOM killer 대응.
§22.12 ↔ §25.12	영 15 / 한 15	설명 일부 보충 및 SSD 새 문단. 자료 3.
참고문헌	영 16-17 / 한 16-17	13개 대응. SS10·W+21 신규. 자료 4.
숙제: 문제 1-5	영 18 / 한 18	5개 모두 대응. 문제 3 절차·문제 4 방향 표시.

완료 확인: 영어·한국어 본문, 그림·도식, 팁·여담, 각주, 참고문헌과 장 말미를 대조했다. 번역문은 추가된 내용과 개정 부분만 수록하며, 반복되는 기존 한국어 설명은 해당 위치만 안내한다.

26

VAX/VMS 가상 메모리 시스템

가상 메모리에 대한 학습을 마치기 전에, VAX/VMS 운영체제 [LL82]의 잘 만들어진 가상 메모리 관리자에 대해 상세하게 살펴보자. 여기서는 이전의 장에서 살펴보았던 몇 가지 개념들이 완전한 메모리 관리자 내에 어떤 식으로 적용되었는지 설명할 것이다.

26.1 배경

VAX-11 미니컴퓨터의 구조는 1970년대 말에 **Digital Equipment Corporation**(**DEC**)에 의해서 소개되었다. DEC는 미니컴퓨터 시대에 컴퓨터 산업계의 일류 회사였다. 불행하게도 PC의 등장과 수차례의 잘못된 경영 판단으로 인해 서서히(하지만 분명하게) 사라졌다 [Chr03]. 이 구조는 VAX-11/780과 그 보다는 낮은 성능의 VAX-11/750을 포함한 컴퓨터에 구현되었다.

VAX/VMS(또는 단순하게 VMS)라고 알려진 운영체제의 주요한 아키텍트는 Dave Cutler로서 이후에는 Microsoft Windows NT 개발을 이끌었다 [CS93]. VMS는 다양한 종류의 기계들에서 실행되어야 한다는 문제를 안고 있었다. VMS가 설치될 컴퓨터는 저가의 VAX들부터 엄청난 고가의 고사양 기계들도 있었다. VMS 운영체제는 다양한 종류의 시스템들에서 동작하는(그것도 잘 동작하는) 기법들과 정책들을 필요로 했다.

핵심 질문: 일반론의 저주(The Curse of Generality)에서 어떻게 방지할까
운영체제는 종종 "일반론의 저주"라고 알려진 문제를 가진다. 운영체제는 다양한 응용 프로그램들과 시스템 부류를 범용으로 지원해야 할 책임이 있다. 이 저주의 결과는 운영체제가 어느 하나의 기계도 제대로 지원할 수 없게 된다는 것이다. VMS의 경우, 그 저주는 현실이 되었다. VAX-11 구조는 여러 다른 형태로 구현되었기 때문이다. 운영체제가 다양한 시스템에서 효과적으로 실행되려면 어떻게 구현되어야 하는가?

VMS는 컴퓨터의 구조적 결함을 소프트웨어로 보완한 훌륭한 사례다. 운영체제가 이상적인 개념과 환상을 제공하기 위해 하드웨어에 의존하지만, 하드웨어가 모든 것을 제대로 해내지 못할 경우도 있다. VAX 하드웨어에서 해당 사례들을 몇 가지 볼 것이다.

하드웨어 결함에도 불구하고, 시스템이 효과적으로 작동하기 위해 VMS 운영체제가 무엇을 하였는지 볼 것이다.

26.2 메모리 관리 하드웨어

VAX-11은 프로세스마다 512바이트 페이지 단위로 나누어진 32비트 가상 주소 공간을 제공한다. 가상 주소는 23비트 VPN과 9비트 오프셋으로 구성되어 있다. VPN의 상위 두 비트는 페이지가 속한 세그멘트를 나타내기 위해서 사용되었다. 이 시스템은 앞서 살펴보았던 페이징과 세그멘테이션의 하이브리드 구조를 갖고 있다.

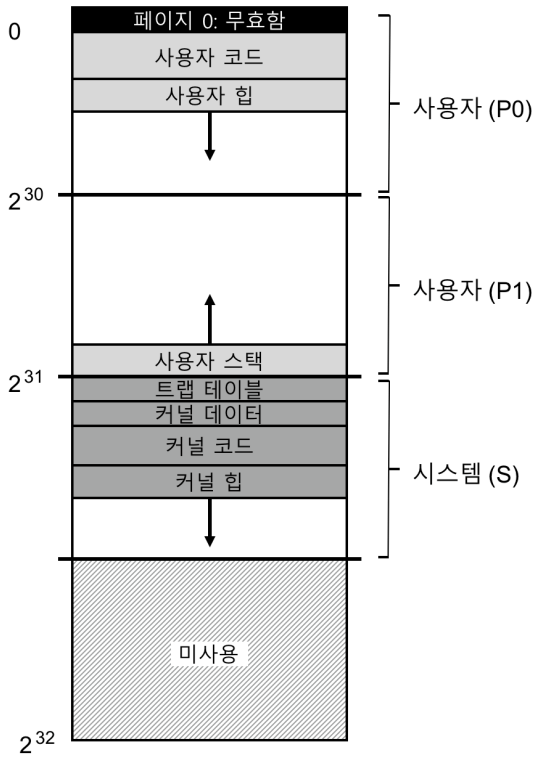
주소 공간의 하위 절반은 "프로세스 공간"으로 알려져 있으며 각 프로세스마다 다르게 할당된다. 프로세스 공간의 첫 번째 절반(`P0`으로 알려져 있음)에 사용자 프로그램과 힙(heap)이 존재한다. 힙은 주소가 큰 쪽으로 증가한다. 프로세스 공간의 두 번째, 즉 큰 쪽 절반은(`P1`) 주소가 작은 방향으로 증가하는 스택(stack)이 존재한다. 주소 공간의 상위 절반은 그 중 반만 사용되며 시스템 공간(`S`)으로 불린다. 운영체제의 보호된 코드와 데이터가 이곳에 존재하며, 이 방식으로 여러 프로세스가 운영체제를 공유한다.

VMS 설계자들의 주요 고민 중 하나는 믿을 수 없을 정도로 작은 VAX 하드웨어의 페이지 크기였다(512바이트). 역사적인 이유로 페이지 크기가 결정되었지만 선형 페이지 테이블의 크기가 지나치게 커진다는 것이 근본적인 문제였다. VMS 설계자들의 첫 목표 중에 하나는 VMS가 페이지 테이블 저장을 위해 메모리를 소진하는 것을 막는 것이었다.

이 시스템은 페이지 테이블로 인한 메모리 압박의 정도를 경감시키기 위한 두 가지 방법을 사용하였다. 첫째, VAX-11은 사용자 주소 공간을 두 개의 세그멘트로 나누어 프로세스마다 (P0과 P1) 각 영역을 위한 페이지 테이블을 가지도록 하였다. 스택과 힙 사이의 사용되지 않은 주소 영역을 위한 페이지 테이블 공간이 필요 없게 되었다. 베이스와 바운드 레지스터는 예상하는 그대로의 방식으로 사용된다. 베이스 레지스터는 해당 세그멘트의 페이지 테이블의 주소를 담고 있으며 바운드는 그 크기를 나타낸다(즉, 페이지 테이블 항목들의 수).

둘째로, 운영체제는 사용자 페이지 테이블들을(`P0`과 `P1`, 즉 프로세스마다 두 개) 커널의 가상 메모리에 배치하여 메모리 압박을 더 줄일 수 있었다. 페이지 테이블을 할당하거나 크기를 키울 때 커널은 자신의 가상 메모리, 세그멘트 `S` 내에 공간을 할당한다. 메모리가 고갈되면, 커널은 페이지 테이블의 페이지들을 디스크로 스왑하여 물리 메모리를 다른 용도로 사용할 수 있게 한다.

커널 가상 메모리에 페이지 테이블을 넣으면 주소 변환 과정이 훨씬 더 복잡해진다. 예를 들어, `P0`과 `P1` 내의 가상 주소를 변환하기 위해서는 하드웨어가 먼저 페이지 테이블(그 프로세스의 `P0` 또는 `P1` 페이지 테이블)에서 해당 페이지 테이블 항목을 찾아야 한다. 그러나 이 과정 중에 하드웨어는 시스템의 페이지 테이블(물리 메모리에 존재)을 먼저 검색해야 할 수도 있다. 변환이 완료되면 하드웨어는 페이지 테이블 페이지의 주소를 알게 되며, 최종적으로 원하는 메모리 접근에 대한 주소를 알게 된다. 다행스럽게도 이 모든 과정은 VAX의 하드웨어 TLB에서 빠르게 처리된다.



〈그림 26.1〉 **VAX/VMS 주소 공간**

26.3 실제 주소 공간

VMS 학습의 좋은 점은 실제 주소 공간의 구축 상황을 볼 수 있다는 것이다(그림 26.1). 지금까지는 사용자 코드, 사용자 데이터 그리고 사용자 힙을 위한 간단한 주소 공간을 가정하였지만 그림에서 본 것과 같이 실제 주소 공간은 훨씬 더 복잡하다.

예를 들어, 코드 세그멘트는 절대로 페이지 0에서 시작하지 않는다. 대신 이 페이지는 접근 불가능 페이지로 마킹되어 있으며 널-포인터(null-pointer) 접근을 검출할 수 있게 한다. 주소 공간 설계 시 한 가지 고려해야 할 사항은 효과적인 디버깅 지원 여부이다. 접근 불가능한 페이지 0은 그런 지원의 한 형태이다.

좀 더 중요한 사실은 커널의 가상 주소 공간이(즉, 커널의 자료 구조와 코드) 사용자 주소 공간의 일부라는 것이다. 문맥 교환이 발생하면 운영체제는 `P0`과 `P1` 레지스터를 다음 실행될 프로세스의 페이지 테이블을 가리키도록 변경한다. 하지만, `S` 베이스와 바운드 레지스터는 변경하지 않기 때문에 결과적으로 "동일한" 커널 구조들이 각 사용자 주소 공간에 매핑된다.

몇 가지 이유로 커널은 여러 주소 공간들로 매핑된다. 그러한 구조를 택하면 커널이 동작이 쉬워진다. 예를 들어 운영체제가 사용자 프로그램으로부터(예, `write()` 시스템 콜에서) 포인터를 전달 받았다면 그 포인터로부터 데이터를 자신의 구조로 그냥

여담: 널 포인터 접근은 왜 세그멘테이션 오류를 발생시키나

널 포인터 역참조(null-pointer dereference)를 할 때 정확히 어떤 일이 발생하는지 이제 잘 이해하고 있을 것이다. 프로세스는 다음과 같은 동작을 통해서 가상 주소 0 번지를 만들어 낸다.

```
int *p = NULL;  // p = 0 으로 설정
*p = 10;        // 가상 주소 0 에 10을 저장 시도
```

하드웨어는 TLB에서 VPN(여기서도 0) 검색을 시도하지만 TLB 미스를 경험한다. 페이지 테이블이 검색되며 VPN 0에 대한 항목이 무효로 표시되어 있는 것을 알게 된다. 무효한 접근이 발생하였으므로 운영체제로 제어가 전달된다. 그러면 운영체제는 해당 프로세스를 종료할 것이다(Unix 시스템에서는 이런 폴트에 대해서 대응할 수 있도록 프로세스에게 시그널이 전달된다. 시그널을 캐치하지 않으면 프로세스는 종료되게 된다).

복사하면 된다. 쉽다. 운영체제는 접근하는 데이터가 어디에서 오는지 고려할 필요 없이 자연스럽게 작성되고 컴파일될 수 있다. 만약 커널이 전부 물리 메모리에만 존재한다면 페이지 테이블의 페이지들을 디스크로 스왑하는 등의 작업은 상당히 어려웠을 것이다. 커널이 자체 주소 공간을 가진다면 사용자 프로그램들과 커널 간의 데이터 이동이 매우 복잡하고 고통스러운 일이 될 것이다. 이제는 보편적으로 사용되는 이러한 방법 덕택에 커널은 비록 보호된 영역이기는 하지만 응용 프로그램에게 마치 라이브러리처럼 보인다.

이 주소 공간에 관한 마지막 이슈는 보호와 관련 있다. 운영체제는 응용 프로그램이 운영체제의 데이터나 코드를 읽거나 쓰는 것을 분명히 원하지 않는다. 운영체제의 자료를 보호하기 위해서는 하드웨어가 페이지 별로 보호 수준을 다르게 설정할 수 있어야 한다. 이를 위해 VAX는 페이지 테이블의 protection bit(protection bits)에 보호 수준을 지정한다. 특정 페이지를 접근하기 위해서 필요한 CPU의 권한 수준이 기록된다. 시스템 데이터와 코드는 사용자의 데이터와 코드보다 더 높은 보호 수준으로 지정된다. 사용자 코드가 더 높은 보호 수준의 자료를 접근하려고 시도하면 운영체제로 트랩이 걸릴 것이다. 그리고 (예상한대로) 트랩을 일으킨 프로세스는 종료된다.

26.4 페이지 교체

VAX의 페이지 테이블 항목(PTE)은 다음과 같은 비트들을 가지고 있다. 유효(valid) 비트, 보호 필드(protection field, 4비트), 변경(modify 또는 더티(dirty)) 비트, 운영체제가 사용하기 위해 예약해 놓은 필드(5비트), 그리고 마지막으로 물리 메모리 페이지의 위치를 저장하기 위한 물리 프레임 번호(PFN)가 있다. 예리한 독자라면 **reference bit**가 없다는 것을 파악했을 것이다. VMS 교체 알고리즘은 어떤 페이지가 자주 사용 중인지를 하드웨어 지원 없이 판단하여야 한다.

개발자들은 **메모리 호그(memory hog)**에 대해서도 고민하였다. 메모리 호그는 메모리를 너무 많이 사용하는 프로그램을 의미한다. 지금까지 우리가 살펴보았던 대부분의

정책들은 메모리를 많이 소비하는 프로세스에 대한 대비책이 없다. 예를 들어, LRU는 프로세스 간에 공정한 메모리 분배는 고려하지 않는 전역적인(global) 정책이다.

세그멘트된 FIFO

위의 두 가지 문제를 해결하기 위해 개발자들은 **세그멘트된 FIFO** 교체 정책을 제안하였다 [TL81]. 개념은 간단하다. 각 프로세스는 **상주 집합 크기**(**resident set size**, **RSS**)라고 불리는 메모리에 유지할 수 있는 최대 페이지 개수를 지정 받는다. 각 페이지들은 FIFO 리스트에 보관되며, 페이지 개수가 RSS보다 커지면 "제일 먼저 들어왔던" 페이지가 쫓겨난다. FIFO는 하드웨어의 지원이 필요 없기 때문에 구현이 간단하다.

순수한 FIFO 성능은 앞서 보았던 것처럼 그리 좋지는 않다. FIFO의 성능을 개선하기 위해서 VMS는 전역 *클린-페이지 프리 리스트*(*global clean-page free list*)와 *더티-페이지 리스트*(*dirty-page list*)라고 하는 두 개의 **second-chance list**를 도입하였다. 이 리스트는 전역 자료 구조이다. Second-chance list는 메모리에서 제거되기 전에 페이지가 보관되는 리스트이다. 프로세스 `P`가 자신의 RSS를 넘긴다면 자신의 FIFO에서 페이지가 제거된다. 제거된 페이지가 클린 상태라면(수정 안된) 클린-페이지 리스트에, 더티 상태라면 (변경된) 더티-페이지 리스트에 추가된다.

다른 프로세스 `Q`에 빈 페이지가 필요하면 전역 클린 리스트에서 첫 번째 프리 페이지를 꺼낸다. 원래의 프로세스 `P`가 해당 페이지가 회수되기 전에 그 페이지에 대해 폴트를 발생시키면, `P`는 프리(또는 더티) 리스트에서 페이지를 가져가서 다시 사용하게 된다. 이런 식으로 디스크 접근을 피한다. 전역 second-chance list의 크기가 클수록 세그멘트된 FIFO 알고리즘은 LRU와 유사하게 동작한다 [TL81].

페이지 클러스터링

또 다른 최적화 기법은 VMS의 작은 페이지 크기를 극복할 수 있게 하였다. 페이지 크기가 작을수록 스왑할 때에 디스크 I/O가 비효율적이 된다. 디스크는 전송 단위가 클수록 성능이 좋기 때문이다. 스왑할 때 I/O의 효율을 개선하기 위해 VMS는 몇 가지 최적화 기법을 도입하였다. 그 중에서 가장 중요한 것은 **클러스터링**(**clustering**)이다. 클러스터링 기법을 써서 VMS는 전역 더티 리스트에 있는 페이지들을 작업 묶음을 만들어서 한 번에 디스크로 보낸다(그렇게 하여 페이지를 클린 상태로 만든다). 운영체제는 스왑 공간 어디에든 페이지들을 자유롭게 배치할 수 있기 때문에 페이지들을 그룹으로 묶어서 쓰는 것이 가능하다. 쓰기 횟수는 줄이고 한 번에 쓰는 양은 늘려서 성능을 향상시키기 때문에 클러스터링은 대부분의 현대 시스템에서 사용된다.

26.5 그 외의 VM 기법들

VMS는 이제는 표준화가 된 기법 두 가지를 더 가지고 있다. **요청 시 0으로 채우기**(**demand zeroing**)와 **쓰기-시-복사**(**copy-on-write**)가 그 두 가지이다. 이제 이 "**게으른**" 최적화 기법들에 대해 설명한다.

여담: reference bit 에뮬레이트하기

사용 중인 페이지를 찾기 위해서 하드웨어 reference bit가 꼭 필요한 것은 아니라는 것을 알았다. 사실, 1980년대 초에 Babaoglu와 Joy는 VAX의 protection bit가 reference bit를 에뮬레이트할 수 있다는 것을 보였다 [BJ81]. 기본적인 개념은 이렇다. 시스템에서 활발하게 사용 중인 페이지를 알고 싶다면 페이지 테이블의 모든 페이지들을 접근 불가능으로 표시해 놓는다(하지만 프로세스 접근 가능 정보를 별도의 페이지 테이블 항목에 보관한다). 프로세스가 페이지를 접근하면 운영체제로 트랩이 발생할 것이다. 운영체제는 페이지가 정말 접근 가능한지 검사한다. 가능하다면 해당 페이지를 정상적인 보호 상태로 되돌린다(예, 읽기 전용 또는 읽기-쓰기). 페이지 교체 시점에 운영체제는 접근 불가능으로 남아 있는 페이지들을 확인하여 최근에 사용되지 않은 페이지들이 어떤 것들인지를 파악한다.

reference bit "에뮬레이션"은 오버헤드를 줄이는 것이 핵심이다. 운영체제는 너무 자주 페이지를 접근 불가능으로 표시하면 안 된다. 오버헤드가 증가한다. 그렇다고 표시를 너무 주저하면 모든 페이지들이 사용된 상태로 남게 될 것이다. 운영체제는 여전히 어떤 페이지를 제거해야 할지 모르게 된다.

VMS(그리고 대부분의 현대의 시스템들)가 사용하는 기법의 한 형태는 페이지들을 **demand zeroing**이다. 이해를 위해 힙 등의 주소 공간에 페이지를 추가하는 예를 생각해 보자. 단순한 구현에서는 힙에 페이지를 추가하는 요청이 오면 운영체제는 물리 메모리에서 페이지를 찾아 0으로 채운다(보안을 위해 필요하다. 그렇지 않으면 다른 프로세스가 이전에 사용했던 페이지의 내용을 볼 수 있게 된다!). 그런 후에 주소 공간에 그 페이지를 매핑한다(즉, 물리 페이지를 원하는 대로 참조할 수 있도록 페이지 테이블을 설정한다). 하지만 이 경우 프로세스가 해당 페이지를 사용하지 않는다면 너무 많은 비용을 지불하는 셈이다.

Demand zeroing의 경우 페이지가 주소 공간에 추가되는 시점에는 거의 하는 일이 없다. 페이지 테이블에 접근 불가능 페이지라고 표기하고 항목을 추가한다. 프로세스가 추가된 페이지를 읽거나 쓸 때 운영체제로 트랩이 발생한다. 트랩을 처리하면서 운영체제는 demand zeroing 할 페이지라는 것을 알게 된다(일반적으로는 페이지 테이블 항목의 "운영체제를 위해 예약된" 부분에 일정 비트로 표기되어 있다). 이 시점에서 운영체제는 물리 페이지를 0으로 채우고 프로세스의 주소 공간으로 매핑하는 등의 필요한 작업을 한다. 프로세스가 해당 페이지를 전혀 접근하지 않는다면 이 모든 작업을 피할 수 있으며, 이것이 바로 demand zeroing의 장점이다.

VMS에서 찾아 볼 수 있는 또 다른 멋진 최적화 방법은(마찬가지로 거의 대부분의 모든 현대의 운영체제에서 찾아 볼 수 있는) **copy-on-write**(**COW**)이다. 개념 자체는 TENEX 운영체제까지 거슬러 올라가며 [Bob+72], 개념도 간단하다. 운영체제가 한 주소 공간에서 다른 공간으로 페이지를 복사할 필요가 있을 때, 복사를 하지 않고 해당 페이지를 대상 주소 공간으로 매핑하고[1] 해당 페이지의 페이지 테이블 엔트리를 양쪽

1) 역자 주: 즉, 대상 페이지 테이블의 해당 페이지 엔트리가 원본의 물리 위치를 가리키게 한다. 하나의

팁: 게으름을 피우자

게으름을 피우는 것이 인생에서나 운영체제에서 덕목이 될 수도 있다. 게으름을 피우면 일을 미룰 수가 있는데, 운영체제의 경우에는 몇 가지 이유로 유익하다. 먼저 일을 미루면 현재 작업의 지연 시간을 줄일 수 있게 되어 응답성을 개선할 수 있다. 예를 들어 대부분의 경우 운영체제가 파일에 대한 쓰기가 성공하였다고 즉시 보고 하지만, 실제 디스크 쓰기는 나중에 백그라운드로 처리된다. 둘째 그리고 더 중요한 이유는 게으름으로 인해서 작업을 해야 할 필요 자체를 제거할 수 있기 때문이다. 예를 들어서 파일이 지워질 때까지 쓰기를 지연하면 쓰기 자체가 필요 없어진다. 게으름은 삶에서도 좋다. 예를 들면, 운영체제 프로젝트를 미루면, 해당 프로젝트가 가지고 있던 명세의 버그를 반 친구들이 고쳐줄 때가 있기 때문이다. 하지만, 프로젝트가 취소되는 경우는 거의 없기 때문에 게으름을 피우는 것은 문제가 될 수 있다. 프로젝트 기한이 지난 뒤에 제출하게 되고, 성적이 떨어지고, 교수님도 슬프게 된다. 교수님들을 슬프게 만들지 말자!

주소 공간에서 읽기 전용으로 표시한다. 만약 양쪽 주소 공간이 페이지를 읽기만 한다면 더 이상의 조치는 필요 없게 되며, 운영체제는 실제로 데이터 이동 없이 빠른 복사를 할 수 있게 된다.

두 주소 공간 중에 하나가 페이지 쓰기를 시도한다면, 운영체제로 트랩을 발생한다. 운영체제는 그때 해당 페이지가 COW 페이지라는 것을 파악한다. 그런 후에 새로운 페이지를 (게으르게) 할당하고, 데이터로 채우고, 이 새로운 페이지 폴트를 일으킨 페이지의 주소 공간에 매핑한다. 이제 프로세스는 독자적인 페이지의 사본을 가지게 되고 하던 일을 계속한다.

COW는 여러 가지 이유로 유용하다. 공유 라이브러리들을 여러 프로세스들의 주소 공간에 copy-on-write로 매핑하여 메모리 공간을 절약할 수 있다. Unix 시스템에서는 `fork()`와 `exec()`의 시맨틱 때문에 COW는 훨씬 더 중요하다. 기억하겠지만 `fork()`는 호출자의 주소 공간과 정확히 동일한 사본을 생성한다. 주소 공간이 크면 복사본을 만드는 시간이 오래 걸리고 짧은 시간에 많은 양의 데이터를 접근해야 한다. 더 심각한 것은, 뒤 따르는 `exec()` 호출에 의해서 대부분의 주소 공간이 덮어 쓰이게 된다. `exec()`은 곧 실행될 프로그램의 주소 공간으로 호출한 프로세스의 주소 공간의 내용을 덮어 쓴다. copy-on-write `fork()`를 수행하게 되면 운영체제는 상당한 불필요한 복사를 피할 수 있으며, 성능을 개선하면서 정확한 시맨틱을 유지할 수 있다.

26.6 요약

전체 가상 메모리 시스템에 대하여 처음부터 끝까지 살펴보았다. 대부분의 기본적인 기법과 정책들을 잘 이해하고 있었으므로, 상세 사항 대부분을 이해하기 쉬웠을 것이다.

물리 페이지를 두 개의 페이지 테이블이 가리키는 것이다.

좀 더 상세한 내용은 훌륭한(그리고 짧은) Levy와 Lipman의 논문에 나와 있다 [LL82]. 읽어보기를 권한다. 이번 장에 대한 원전을 볼 수 있는 좋은 기회이다.

가능하다면 Linux와 다른 현대 시스템을 읽어 최신 기술에 대해서 학습하기 바란다. 어렵지 않게 쓰여진 책을 포함하여 [BC05] 상당한 읽을 내용들이 있다. 놀랄만한 한 가지 사실: VAX/VMS와 같이 오래된 논문들에 나와 있는 고전적인 개념들이 아직도 현대 운영체제의 개발에 영향을 주고 있다.

참고 문헌

[BJ81] **"Converting a Swap-Based System to do Paging in an Architecture Lacking Page-Reference Bits"**
Ozalp Babaoglu and William N. Joy
SOSP '81, Pacific Grove, California, December 1981
reference bit를 에뮬레이트하기 위해서 기존의 보호 기법을 활용하는 방법을 다룬 영리한 개념을 소개하는 논문. 버클리 시스템 배포(Berkeley Systems Distribution) 또는 BSD라고 부르는 자체적인 Unix 버전을 개발하던 그룹에서 나온 아이디어. 이 그룹은 Unix 발전 특히 가상 메모리와 파일 시스템 그리고 네트워크 분야 개발에 엄청난 영향을 끼쳤다.

[Bob+72] **"TENEX, A Paged Time Sharing System for the PDP-10"**
Daniel G. Bobrow, Jerry D. Burchfiel, Daniel L. Murphy, and Raymond S. Tomlinson
Communications of the ACM, Volume 15, March 1972
수많은 좋은 개념들을 고안해 낸 초기의 시분할 운영체제이다. Copy-on-write는 그 중에 하나일 뿐이다. 현대 운영체제의 많은 다른 측면에 영감을 준 시스템. 이 책의 프로세스 관리, 가상 메모리 및 파일 시스템도 그 중 하나다.

[BC05] **"Understanding the Linux Kernel (Third Edition)"**
Daniel P. Bovet and Marco Cesati
O'Reilly Media, November 2005
Linux에 대하여 찾아 볼 수 있는 많은 책들 중 하나. 많은 책들이 금세 구닥다리가 되기는 하지만 많은 기본적인 개념들은 여전히 유지되며 읽을 가치가 충분하다.

[Chr03] **"The Innovator's Dilemma"**
Clayton M. Christenson
Harper Paperbacks, January 2003
디스크 드라이브 산업과 새로운 혁신이 기존의 혁신을 어떻게 붕괴시키는지를 보여주는 환상적인 책이다. 경영 전공 학생들뿐만 아니라 컴퓨터 과학자들에게도 읽기 좋은 책이다. 거대하고 성공적인 기업들이 어떻게 완전히 망하는지에 대한 통찰을 제공한다.

[CS93] **"Inside Windows NT"**
Helen Custer and David Solomon
Microsoft Press, 1993
Windows NT에 대한 책. 시스템의 꼭대기부터 바닥까지 원하는 것 이상보다 더 자세하게 설명한다. 그렇지만 진지하게, 이 책은 상당히 좋은 책이다.

[LL82] **"Virtual Memory Management in the VAX/VMS Operating System"**
Henry M. Levy and Peter H. Lipman
IEEE Computer, Volume 15, Number 3 (March 1982)
이번 장의 대부분의 내용을 다루는 원전을 읽어라. 간결하게 쉽게 읽을 수 있다. 대학원에 진학할 예정이라면 특히 중요하다. 대학원에서 하는 일이라고는 논문을 읽고, 연구하고, 더 많은 논문들을 읽고, 좀 더 연구하고, 결국에는 논문도 쓰고, 그런 후 연구를 더 하고. 그러나 재미있다.

[TL81] **"Segmented FIFO Page Replacement"**
Rollins Turner and Henry Levy
SIGMETRICS '81, Las Vegas, Nevada, September 1981
어떤 워크로드에 대해서는 세그멘트된 FIFO가 LRU의 성능에 근접한다는 것을 보인 짧은 논문.

대조 번역 · 학습 보충본

완전한 가상 메모리 시스템

영어 23장 · 한국어 구판 26장 | Linux 절 전체와 누락 부분

영어: vm-complete.pdf (19쪽, V1.10) / 한국어: 23_vm-vax.pdf (9쪽, V0.90)

한국어 구판은 VAX/VMS를 다루는 26장이다. 영어 신판은 이를 §23.1로 묶고, Linux 가상 메모리 시스템을 다루는 §23.2를 새로 추가했다. 이 자료는 신규 서론, 기존 VAX 설명에 빠진 내용, Linux 절 전체(그림·팁·여담·코드·각주 포함), 요약의 Linux 부분, 신규 참고문헌 8개를 번역한다. 기존 VAX/VMS 설명은 반복하지 않는다.

쪽수는 첨부 PDF의 첫 페이지를 1쪽으로 센다. 절 번호는 영어판 기준이다. §23.1의 번호 없는 소제목들은 한국어 §26.1-26.5와 대응한다. Linux 설명의 "현재·최근" 및 시스템 동작은 첨부 원문의 시대적 서술을 유지했다. 번역과 별도로 넣은 판본·대조 주석은 표시했다.

수록 내용

1. 개정된 서론과 새 핵심 질문

2. VAX/VMS 부분의 세부 보충

3. Linux 가상 메모리 시스템

4. 요약에 추가된 Linux 내용

5. 새 참고문헌 8개

한국어 원본 전체에 이어지는 추가 번역

앞부분: 원본 9쪽 / 뒷부분: 보충 14쪽 / 이 파일: 총 23쪽

1. 개정된 서론과 새 핵심 질문

[신규·개정] 영어 1쪽 ↔ 한국어 1쪽: 짧은 VAX/VMS 도입만 있음

한국어판에도 "앞 장의 개념이 완전한 메모리 관리자 안에서 어떻게 적용되는가"라는 도입은 있다. 아래는 신판에서 확장된 설명만 번역한다.

우리는 페이지 테이블의 여러 설계, TLB와의 상호작용(때로는 운영체제가 직접 처리하기도 한다), 어떤 페이지를 메모리에 남기고 어떤 페이지를 내보낼지 결정하는 전략 등 이러한 시스템의 핵심 요소들을 살펴보았다. 그러나 완전한 가상 메모리 시스템은 성능, 기능, 보안에 관한 수많은 기능을 비롯해 다른 많은 요소들도 포함한다.

핵심 질문: 완전한 가상 메모리 시스템을 어떻게 구축할 것인가

완전한 가상 메모리 시스템을 구현하려면 어떤 기능들이 필요한가? 이 기능들은 성능을 높이고 보안을 강화하거나, 그 밖의 방식으로 시스템을 어떻게 개선하는가?

이를 위해 두 시스템을 다룬다. 첫 번째는 1970년대와 1980년대 초에 개발된 VAX/VMS 운영체제[LL82]에 들어 있는, "현대적" 가상 메모리 관리자의 초기 사례 중 하나이다. 이 시스템의 기법과 접근법 가운데 놀라울 만큼 많은 것이 오늘날까지 살아남았으므로 충분히 공부할 가치가 있다. 50년이나 된 아이디어라도 여전히 알아둘 가치가 있다. 물리학과 같은 대부분의 다른 분야에서는 잘 알려진 생각이지만, 컴퓨터 과학처럼 기술이 주도하는 분야에서는 굳이 말해 주어야 한다.

두 번째는 Linux이다. 이유는 분명할 것이다. Linux는 널리 쓰이는 시스템으로, 휴대전화처럼 작고 성능이 낮은 시스템부터 현대 데이터 센터의 확장성이 뛰어난 멀티코어 시스템에 이르기까지 효과적으로 실행된다. 따라서 이 모든 환경에서 성공적으로 동작할 만큼 가상 메모리 시스템이 유연해야 한다.

두 시스템을 통해 앞 장의 개념들이 완전한 메모리 관리자에 결합되는 모습을 설명한다는 마지막 취지는 한국어 서론에 이미 있어 반복하지 않았다.

2. VAX/VMS 부분의 세부 보충

[부분 누락] 영어 §23.1, 2-7쪽 ↔ 한국어 §26.1-26.5, 1-7쪽

범용성의 저주: 오늘날의 Linux 사례

영어 3쪽 여담 The Curse of Generality ↔ 한국어 1쪽 핵심 질문

이 문제는 오늘날에도 똑같이 현실적이다. Linux는 휴대전화, TV 셋톱박스, 노트북, 데스크톱, 그리고 클라우드 데이터 센터에서 수천 개의 프로세스를 실행하는 고성능 서버에서도 잘 동작해야 한다.

범용 운영체제가 어느 한 설치 환경을 아주 잘 지원하기 어렵다는 일반 설명과 VAX 사례는 한국어판에 있다. Linux 환경을 열거한 위 마지막 문장만 새로 추가되었다.

VAX 복수형에 관한 짧은 괄호 설명

영어 2쪽 VAX/VMS 도입 ↔ 한국어 §26.1, 1쪽

VAXen. 그렇다. 이것이 올바른 복수형이다.

한국어판의 "저가의 VAX들"에 대응하는 원문의 괄호 설명이다. 기술적 새 기법은 아니지만 빠진 짧은 설명도 보충했다.

TLB가 복잡한 페이지 테이블 탐색을 피하게 해 준다

영어 3쪽 Memory Management Hardware 마지막 문장 ↔ 한국어 §26.2, 2쪽 마지막 문장

다행히 VAX의 **하드웨어가 관리하는 TLB**가 이 모든 과정을 더 빠르게 만든다. TLB는 보통, 그러기를 바라지만, 이러한 번거로운 탐색 자체를 피하게 해 준다.

한국어판은 "VAX의 하드웨어 TLB에서 빠르게 처리된다"라고만 적고 있어, 하드웨어 관리와 탐색 우회라는 설명을 보충한다.

메모리 독점 프로그램의 영향

영어 5쪽 Page Replacement ↔ 한국어 §26.4, 4-5쪽

메모리를 많이 사용하는 프로그램은 다른 프로그램들이 실행되기 어렵게 만든다.

memory hog의 정의와 전역 LRU의 불공정한 메모리 분배는 이미 번역되어 있다. 위 문장은 다른 프로그램에 미치는 영향만 보충한다.

참조 비트 에뮬레이션에서 원래의 접근 권한을 보관하는 위치

영어 6쪽 여담 Emulating Reference Bits ↔ 한국어 6쪽 같은 여담

프로세스가 실제로 접근할 수 있는 페이지에 관한 정보는 따로 보관한다. 예를 들어 페이지 테이블 항목의 **"운영체제를 위해 예약된 필드"**에 보관할 수 있다.

한국어판에는 "별도의 페이지 테이블 항목"에 보관한다는 표현이 있지만, 영어판의 예약 필드 예시는 빠져 있다. 에뮬레이션 절차와 오버헤드 설명은 이미 있으므로 반복하지 않는다.

대조상 제외한 표현: 페이지 클러스터링의 large batches는 한국어판의 "작업 묶음"과 대응하며 독립된 새 기법이 아니다. 영어의 힙 downward·스택 upwards는 그림에서의 방향이고, 한국어판의 힙 "큰 주소 쪽"·스택 "작은 주소 쪽"과 같은 배치를 설명하므로 누락으로 처리하지 않았다.

3. Linux 가상 메모리 시스템

[신규 전체 절] 영어 §23.2, 8-17쪽 ↔ 한국어 대응 절 없음

이제 Linux 가상 메모리 시스템의 흥미로운 측면 몇 가지를 살펴보자. Linux의 발전은 실제 운영 환경에서 만난 실제 문제를 해결하는 엔지니어들이 이끌었다. 그 결과 수많은 기능이 차츰 통합되어, 이제는 온전한 기능과 풍부한 특성을 갖춘 가상 메모리 시스템이 되었다.

Linux 가상 메모리의 모든 측면을 설명할 수는 없지만 가장 중요한 것들을 다룰 것이다. 특히 VAX/VMS와 같은 고전적 가상 메모리 시스템을 넘어선 부분에 주목하고, Linux와 이전 시스템들의 공통점도 짚어 보겠다.

여기서는 Intel x86에서 실행되는 Linux에 초점을 맞춘다. Linux는 실제로 여러 프로세서 아키텍처에서 실행되지만, x86의 Linux가 가장 지배적이고 중요한 사용 형태이므로 이를 중심으로 설명한다.

3.1 Linux 주소 공간

영어 8-10쪽, The Linux Address Space / 그림 23.2 / 각주 1

다른 현대 운영체제나 VAX/VMS와 마찬가지로 Linux의 가상 주소 공간[1]은 사용자 부분(사용자 프로그램 코드, 스택, 힙 등)과 커널 부분(커널 코드, 스택, 힙 등)으로 구성된다. 문맥 교환이 일어나면 현재 실행 중인 주소 공간의 사용자 부분은 바뀌지만 커널 부분은 프로세스들 사이에서 동일하다. 사용자 모드에서 실행되는 프로그램은 커널의 가상 페이지에 접근할 수 없다. 커널로 트랩하여 특권 모드로 전환해야만 이러한 메모리에 접근할 수 있다.

[1] 원문 각주 1: 보안 위협으로 최근 변경이 이루어지기 전까지의 설명이다. 이 변경의 자세한 내용은 뒤의 Linux 보안 소절들을 참조하라.

전통적인 32비트 Linux, 즉 32비트 가상 주소 공간을 쓰는 Linux에서 사용자 부분과 커널 부분의 경계는 0xC0000000이다. 이는 주소 공간의 4분의 3 지점이다. 따라서 0부터 0xBFFFFFFF까지는 사용자 가상 주소이고, 나머지 0xC0000000부터 0xFFFFFFFF까지는 커널 가상 주소 공간이다. 64비트 Linux에도 비슷한 분할이 있지만 경계는 약간 다르다. 그림 23.2는 전형적인 주소 공간을 단순화해 나타낸 것이다.

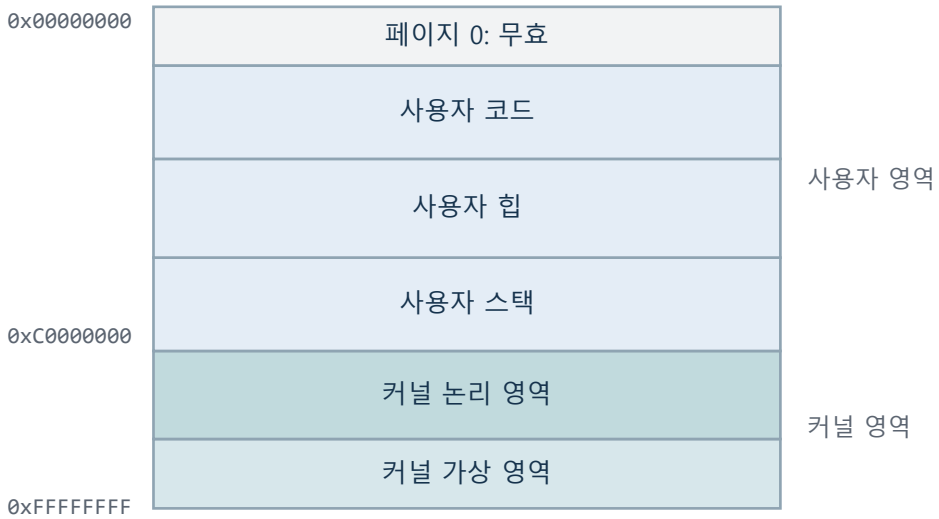


그림 23.2 번역 재도식화: 영역의 실제 크기 비율은 생략

Linux의 약간 흥미로운 특징 하나는 두 종류의 커널 가상 주소를 가진다는 점이다. 첫 번째는 **커널 논리 주소(kernel logical addresses)**[O16]이다. 이는 보통 생각하는 커널의 가상 주소 공간이다. 이런 종류의 메모리를 더 얻으려면 커널 코드는 kmalloc을 호출하면 된다. 페이지 테이블, 프로세스별 커널 스택 등 대부분의 커널 자료 구조가 이곳에 있다. 시스템의 다른 대부분의 메모리와 달리 커널 논리 메모리는 디스크로 스왑할 수 없다.

커널 논리 주소의 가장 흥미로운 특징은 물리 메모리와의 관계이다. 커널 논리 주소와 물리 메모리의 앞부분 사이에는 직접 매핑이 있다. 예를 들어 커널 논리 주소 0xC0000000은 물리 주소 0x00000000으로, 0xC0000FFF는 0x00000FFF로 변환되는 식이다.

이 직접 매핑에는 두 가지 의미가 있다. 첫째, 커널 논리 주소와 물리 주소 사이의 상호 변환이 간단하다. 그래서 이런 주소는 실제 물리 주소인 것처럼 취급되곤 한다. 둘째, 커널 논리 주소 공간에서 연속된 메모리 덩어리는 물리 메모리에서도 연속된다. 따라서 이 부분에서 할당한 메모리는 올바른 동작을 위해 연속된 물리 메모리가 필요한 작업에 적합하다. 예를 들어 직접 메모리 접근(DMA)을 이용한 장치와의 I/O 전송이 그렇다. DMA는 이 책의 세 번째 부분에서 배운다.

다른 종류는 **커널 가상 주소(kernel virtual addresses)**이다. 이런 메모리를 얻으려면 커널 코드는 다른 할당자 vmalloc을 호출한다. 이 함수는 원하는 크기의, 가상 주소상 연속된 영역을 가리키는 포인터를 반환한다. 커널 논리 메모리와 달리 커널 가상 메모리는 보통 물리적으로 연속되지 않는다. 각 커널 가상 페이지가 서로 떨어진 물리 페이지로 매핑될 수 있으므로 DMA에는 적합하지 않다. 대신 할당이 더 쉬워, 큰 연속 물리 메모리 덩어리를 찾기 어려운 대형 버퍼에 사용된다.

32비트 Linux에서 커널 가상 주소가 존재하는 또 다른 이유는 커널이 대략 1 GB보다 많은 메모리를 주소 지정할 수 있게 하기 때문이다. 오래전에는 컴퓨터의 메모리가 이보다 훨씬 적어 1 GB 이상에 접근하는 것이 문제가 되지 않았다. 그러나 기술이 발전하면서 커널이 더 많은 메모리를 사용할 필요가 생겼다. 커널 가상 주소는 물리 메모리와 엄격한 일대일 매핑으로 묶여 있지 않으므로 이를 가능하게 한다. 64비트 Linux로 옮겨 가면서는 커널이 가상 주소 공간의 마지막 1 GB에만 갇히지 않기 때문에 이러한 필요가 덜 절박해졌다.

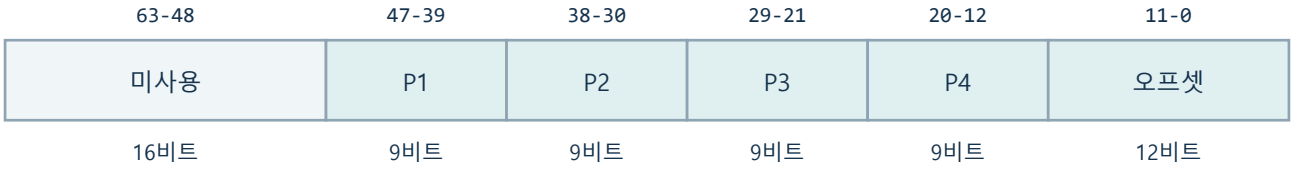
3.2 페이지 테이블 구조

영어 10-11쪽, Page Table Structure / 64비트 주소 도식

x86의 Linux에 초점을 맞추고 있으므로 x86이 제공하는 페이지 테이블 구조를 중심으로 설명한다. 이 구조가 Linux가 할 수 있는 일과 할 수 없는 일을 결정하기 때문이다. 앞서 언급했듯이 x86은 하드웨어가 관리하는 다단계 페이지 테이블을 제공하며, 프로세스마다 하나의 페이지 테이블이 있다. 운영체제는 메모리에 매핑을 설정하고 특권 레지스터가 페이지 디렉터리의 시작을 가리키도록 하면 된다. 나머지는 하드웨어가 처리한다. 예상대로 운영체제는 프로세스 생성·삭제와 문맥 교환 때 관여하며, 각 경우에 하드웨어 MMU가 주소 변환에 올바른 페이지 테이블을 사용하도록 한다.

최근 몇 년간의 가장 큰 변화는 앞에서 짧게 언급한 32비트 x86에서 64비트 x86으로의 이동일 것이다. VAX/VMS에서 보았듯이 32비트 주소 공간은 오래 사용되어 왔다. 그러나 기술이 바뀌면서 결국 프로그램의 실질적인 한계가 되기 시작했다. 가상 메모리는 시스템을 쉽게 프로그래밍하게 해 주지만, 현대 시스템이 수 GB의 메모리를 갖게 되자 32비트로는 그 모든 메모리를 가리킬 수 없었다. 따라서 다음 도약이 필요해졌다.

64비트 주소로 옮겨 가면 예상한 방식으로 x86 페이지 테이블 구조가 달라진다. x86은 다단계 페이지 테이블을 사용하므로, 원문이 설명하는 현재의 64비트 시스템은 4단계 테이블을 쓴다. 다만 가상 주소 공간의 64비트 전체를 아직 사용하는 것은 아니며 하위 48비트만 사용한다. 따라서 가상 주소는 다음과 같이 볼 수 있다.



영어 11쪽 주소 도식 재작성: 인덱스 4개 × 9비트, 오프셋 12비트

그림에서 가상 주소의 상위 16비트는 사용되지 않으며 주소 변환에 관여하지 않는다. 하위 12비트는 4 KB 페이지 크기에 따른 오프셋이므로 변환하지 않고 그대로 사용한다. 남은 가운데 36비트가 주소 변환에 참여한다. P1 부분으로 최상위 페이지 디렉터리를 인덱싱한다. 이후 한 단계씩 변환을 진행하고, 마지막으로 P4가 실제 페이지 테이블 페이지를 인덱싱하여 원하는 페이지 테이블 항목을 얻는다.

시스템 메모리가 더 커지면 이 광대한 주소 공간의 더 많은 부분을 사용할 수 있게 될 것이며, 페이지 테이블 트리는 5단계, 결국에는 6단계 구조로 나아갈 것이다. 생각해 보라. 어떤 데이터가 메모리 어디에 있는지 알아내기 위한 단순한 페이지 테이블 조회에 무려 여섯 단계의 변환이 필요한 것이다.

판본·수식 주석: 위 설명은 첨부 원문이 가정하는 48비트·4단계 모델의 번역이다. 도식의 정확한 분할은 16 + 9 + 9 + 9 + 9 + 12 = 64비트이며, 주소 변환 인덱스는 36비트이다. "상위 16비트 미사용"은 원문의 표현을 유지한 것이다. 5단계·6단계에 관한 문장은 원문의 전망으로 읽는다.

3.3 대형 페이지 지원

영어 11-12쪽, Large Page Support

Intel x86은 표준 4 KB 페이지만이 아니라 여러 페이지 크기를 사용할 수 있게 한다. 특히 최근 설계는 하드웨어에서 2 MB, 심지어 1 GB 페이지까지 지원한다. Linux도 시간이 흐르면서 응용 프로그램이 이러한 페이지를 사용하도록 발전했다. Linux에서는 이를 huge pages, 즉 대형 페이지라고 부른다.

앞서 암시했듯이 대형 페이지에는 여러 이점이 있다. VAX/VMS에서 보았듯이 페이지 테이블에 필요한 매핑 수가 줄어든다. 페이지가 클수록 매핑 수는 적어진다. 그러나 페이지 테이블 항목 수의 감소가 대형 페이지를 사용하는 주된 이유는 아니다. 핵심은 더 나은 TLB 동작과 그에 따른 성능 향상이다.

프로세스가 많은 메모리를 활발히 사용하면 TLB는 주소 변환 정보로 금세 가득 찬다. 그 변환이 4 KB 페이지에 대한 것이면, TLB 미스를 일으키지 않고 접근할 수 있는 메모리 총량은 적다. 그래서 수 GB의 메모리를 갖춘 컴퓨터에서 실행되는 현대의 "대용량 메모리" 워크로드에는 눈에 띄는 성능 비용이 생긴다. 최근 연구에 따르면 일부 응용 프로그램은 실행 사이클의 10%를 TLB 미스 처리에 쓴다[B+13].

대형 페이지는 TLB의 더 적은 슬롯을 사용해 프로세스가 넓은 메모리 영역에 TLB 미스 없이 접근하게 한다. 이것이 주된 장점이다. 다른 이점도 있다. TLB 미스 처리 경로가 더 짧아, 미스가 발생해도 더 빠르게 처리된다. 또한 특정 상황에서는 할당이 상당히 빠를 수 있다. 작지만 때로는 중요한 이점이다.

Linux의 대형 페이지 지원에서 흥미로운 점은 점진적으로 도입했다는 것이다. 처음에 개발자들은 엄격한 성능 요구를 가진 대형 데이터베이스 같은 소수 응용 프로그램에만 이 지원이 중요하다고 판단했다. 그래서 응용 프로그램이 mmap() 또는 shmget() 호출을 통해 대형 페이지 메모리 할당을 명시적으로 요청하도록 했다. 그러면 대부분의 응용 프로그램은 영향을 받지 않고 계속 4 KB 페이지만 쓴다. 요구가 까다로운 일부 응용 프로그램은 이 인터페이스를 사용하도록 바뀌어야 하지만, 그들에게는 수고할 가치가 있었다.

더 최근에는 더 나은 TLB 동작을 필요로 하는 응용 프로그램이 많아지면서 Linux 개발자들이 **투명한 대형 페이지(transparent huge page)** 지원을 추가했다. 이 기능을 켜면 운영체제가 응용 프로그램 수정 없이 대형 페이지를 할당할 기회를 자동으로 찾는다. 보통 2 MB이지만 일부 시스템에서는 1 GB이다.

대형 페이지에도 비용은 있다. 가장 큰 잠재적 비용은 **내부 단편화**, 즉 큰 페이지를 드문드문 사용하는 데서 생긴다. 이런 낭비 때문에 크지만 거의 사용하지 않는 페이지들이 메모리를 채울 수 있다. 스왑을 활성화한 경우에도 대형 페이지와 잘 맞지 않아, 시스템의 I/O 양이 때로는 크게 늘어난다. 다른 상황에서는 할당 오버헤드도 클 수 있다. 전반적으로 한 가지는 분명하다. 오랫동안 시스템에 잘 맞았던 4 KB 페이지 크기는 이제 예전처럼 보편적인 해답이 아니다. 메모리 크기가 커지므로 가상 메모리 시스템의 필수적인 진화 과정에서 대형 페이지와 다른 해법을 고려해야 한다. Linux가 이 하드웨어 기반 기술을 천천히 채택한 것은 다가오는 변화의 증거이다.

팁: 점진적 접근을 고려하자

영어 12쪽, Tip: Consider Incrementalism

살다 보면 혁명가가 되라는 권유를 자주 받는다. 사람들은 "크게 생각해!"라고 말하고 "세상을 바꿔!"라고 외친다. 왜 매력적인지 이해할 수 있다. 어떤 경우에는 큰 변화가 필요하므로 이를 강하게 추진하는 것이 타당하다. 그리고 이런 식으로 시도하면 적어도 더는 당신에게 소리치지 않을지도 모른다.

그러나 많은 경우에는 더 느리고 점진적인 접근이 옳을 수 있다. 이 장의 Linux 대형 페이지 사례는 공학에서의 점진적 접근을 보여 준다. 개발자들은 근본주의자처럼 대형 페이지가 미래의 길이라고 고집하지 않았다. 먼저 특수한 용도를 위한 지원을 도입하고, 장점과 단점을 더 배운 다음, 실제 이유가 생겼을 때에만 모든 응용 프로그램을 위한 더 일반적인 지원을 추가했다.

점진적 접근은 때로 무시당하지만, 느리더라도 신중하고 합리적인 진전을 가져오는 경우가 많다. 시스템을 만들 때는 이런 접근이 바로 필요한 것일 수 있다. 삶에서도 마찬가지일 수 있다.

3.4 페이지 캐시

영어 13쪽, The Page Cache

영구 저장 장치에 접근하는 비용을 줄이기 위해 대부분의 시스템은 적극적인 캐싱을 사용하여 자주 쓰이는 데이터를 메모리에 유지한다. 영구 저장 장치는 이 책의 세 번째 부분에서 다룬다. 이런 점에서 Linux도 전통적인 운영체제와 다르지 않다.

Linux의 페이지 캐시는 통합되어 있으며 세 가지 주요 출처의 페이지를 메모리에 유지한다. 첫째는 메모리 매핑 파일, 둘째는 장치의 파일 데이터와 메타데이터(보통 파일 시스템에 read()와 write() 호출을 보내 접근한다), 셋째는 각 프로세스를 구성하는 힙과 스택 페이지이다. 세 번째는 아래에 이름 있는 파일 대신 스왑 공간이 있어서 때로 **익명 메모리(anonymous memory)**라고 부른다. 이 대상들은 페이지 캐시 해시 테이블에 유지되므로 데이터가 필요할 때 빠르게 찾을 수 있다.

페이지 캐시는 항목이 클린 상태인지(읽었지만 갱신하지 않음), 더티 상태인지(수정됨)를 추적한다. 더티 데이터는 pdflush라고 하는 백그라운드 스레드가 주기적으로 뒷받침 저장소에 쓴다. 파일 데이터는 해당 파일에, 익명 영역은 스왑 공간에 기록한다. 이렇게 수정된 데이터는 결국 영구 저장 장치에 반영된다. 이 백그라운드 작업은 일정 시간이 지난 뒤 또는 더티 페이지가 너무 많다고 판단될 때 실행된다. 두 조건 모두 설정할 수 있다.

때로 시스템의 메모리가 부족해져 Linux는 공간을 확보하려고 어떤 페이지를 내보낼지 결정해야 한다. 이를 위해 Linux는 변형된 **2Q 교체 알고리즘**[JS94]을 사용한다. 이제 그 방법을 설명한다.

기본 생각은 간단하다. 표준 LRU는 효과적이지만 흔한 특정 접근 패턴 때문에 제대로 동작하지 않을 수 있다. 예를 들어 프로세스가 큰 파일을 반복해서 접근하면, 특히 파일이 메모리 크기에 가깝거나 더 크면, LRU는 다른 모든 파일을 메모리에서 밀어낸다. 더 나쁜 점은 이 파일 일부를 메모리에 남겨도 쓸모가 없다는 것이다. 내보내지기 전에 다시 참조되는 일이 없기 때문이다.

Linux의 2Q 교체 알고리즘은 두 리스트를 유지하고 메모리를 둘 사이에 나누어 이 문제를 해결한다. 페이지가 처음 접근되면 한 큐에 들어간다. 원 논문에서는 A1, Linux에서는 **비활성 리스트(inactive list)**라고 한다. 다시 참조되면 다른 큐로 승격된다. 원문은 원 논문에서의 이름을 Aq, Linux에서의 이름을 **활성 리스트(active list)**라고 적는다. 교체가 필요하면 비활성 리스트에서 후보를 고른다. Linux는 활성 리스트의 맨 아래 페이지들을

주기적으로 비활성 리스트로 옮겨, 활성 리스트가 전체 페이지 캐시 크기의 약 3분의 2를 유지하도록 한다 [G04].

이상적으로는 완벽한 LRU 순서로 이 리스트들을 관리하고 싶겠지만 앞 장에서 설명했듯이 비용이 크다. 그래서 다른 많은 운영체제와 마찬가지로 시계 교체와 비슷한 LRU 근사 방법을 쓴다.

이 2Q 방식은 대체로 LRU와 상당히 비슷하게 동작한다. 다만 큰 파일을 순환해서 접근하는 경우 그 페이지들을 비활성 리스트에 가둔다는 점이 특히 다르다. 그런 페이지들은 내보내지기 전에 다시 참조되지 않으므로 활성 리스트에 있는 다른 유용한 페이지들을 밀어내지 않는다.

판본 주석: pdflush, 익명 메모리의 통합 캐시 설명, 2Q와 활성 리스트의 약 2/3 비율은 첨부 원문의 서술을 그대로 번역했다. 또한 원문은 큐 이름을 Aq라고 표기한다. 이 자료에서 오늘날 Linux의 구현으로 임의로 교체하지 않았다.

여담: 어디에나 있는 메모리 매핑

영어 14쪽, Aside: The Ubiquity of Memory-Mapping / pmap 출력

메모리 매핑은 Linux보다 몇 년 앞서 등장했으며 Linux와 다른 현대 시스템의 여러 곳에서 사용된다. 개념은 간단하다. 이미 연 파일 디스크립터에 mmap()을 호출하면 파일 내용이 놓여 있는 것처럼 보이는 가상 메모리 영역의 시작을 가리키는 포인터를 받는다. 이후 이 포인터를 사용하면 단순한 포인터 역참조로 파일의 어느 부분에나 접근할 수 있다.

아직 메모리에 들어오지 않은 메모리 매핑 파일의 부분에 접근하면 페이지 폴트가 발생한다. 그때 운영체제는 해당 데이터를 메모리에 읽어 들이고, 프로세스의 페이지 테이블을 그에 맞게 갱신하여 접근 가능하게 한다. 즉 요구 페이징이다.

일반적인 Linux 프로세스는 main()에서 mmap()을 직접 호출하지 않더라도 모두 메모리 매핑 파일을 사용한다. Linux가 실행 파일의 코드와 공유 라이브러리 코드를 메모리에 올리는 방식 때문이다. 아래는 실행 중인 프로그램의 가상 주소 공간을 어떤 매핑들이 구성하는지 보여 주는 pmap 명령의 출력을 크게 줄인 예이다. 이 예의 프로그램은 셸 tcsh이다. 네 열은 매핑의 가상 주소, 크기, 영역의 보호 비트, 매핑의 출처를 나타낸다.

```
0000000000400000   372K r-x-- tcsh
00000000019d5000  1780K rw--- [anon ]
00007f4e7cf06000  1792K r-x-- libc-2.23.so
00007f4e7d2d0000    36K r-x-- libcrypt-2.23.so
00007f4e7d508000   148K r-x-- libtinfo.so.5.9
00007f4e7d731000   152K r-x-- ld-2.23.so
00007f4e7d932000    16K rw--- [stack ]
```

출력에서 볼 수 있듯이 tcsh 실행 파일의 코드, libc·libcrypt·libtinfo의 코드, 그리고 동적 링커 자체의 코드(ld.so)가 모두 주소 공간에 매핑되어 있다. 익명 영역도 두 개 있다. 두 번째 항목으로 표시된 힙(anon)과 스택(stack)이다. 메모리 매핑 파일은 운영체제가 현대적인 주소 공간을 구성하는 간단하고 효율적인 방법을 제공한다.

3.5 보안과 버퍼 오버플로

영어 14-16쪽, Security And Buffer Overflows / 코드 2개 / 실행 출력 / 각주 2

현대 가상 메모리 시스템(Linux, Solaris, BSD 계열)과 오래된 시스템(VAX/VMS)의 가장 큰 차이는 현대에 보안을 강조한다는 점일 것이다. 보호는 언제나 운영체제의 중요한 관심사였지만, 컴퓨터들이 그 어느 때보다 더 많이 서로 연결된 만큼 개발자들이 교활한 해커의 시스템 장악을 막기 위해 여러 방어책을 구현한 것은 놀랍지 않다.

주요 위협 하나는 **버퍼 오버플로 공격**[2]이다. 일반 사용자 프로그램뿐만 아니라 커널 자체도 공격할 수 있다. 목표 시스템의 버그를 찾아 공격자가 임의의 데이터를 대상의 주소 공간에 주입할 수 있게 하는 것이 핵심이다. 개발자가 입력이 지나치게 길지 않을 것이라고 잘못 가정한 채 입력을 믿고 버퍼에 복사하면서 이런 취약점이 생기기도 한다. 실제 입력은 너무 길어서 버퍼를 넘치게 하고 대상의 메모리를 덮어쓴다. 아래처럼 무해해 보이는 코드가 문제의 원인이 될 수 있다.

```
int some_function(char *input) {
    char dest_buffer[100];
    strcpy(dest_buffer, input); // 앗, 길이 제한 없는 복사!
}
```

[2] 원문 각주 2: 이 주제의 자세한 내용과 관련 링크는 https://en.wikipedia.org/wiki/Buffer_overflow 를 참조하라. “Aleph One”으로도 알려진 보안 해커 Elias Levy의 유명한 글에 대한 참조도 포함되어 있다.

많은 경우 이런 오버플로가 큰 재앙으로 이어지는 것은 아니다. 예를 들어 악의 없이 잘못된 입력을 사용자 프로그램이나 운영체제에 주면 충돌은 일어나겠지만 그 이상의 일은 없을 수 있다. 그러나 악의적인 프로그래머는 버퍼를 넘치는 입력을 정교하게 만들어 자기 코드를 대상 시스템에 주입할 수 있다. 사실상 시스템을 장악하여 자신의 뜻대로 동작하게 만드는 것이다. 네트워크에 연결된 사용자 프로그램을 공격하는 데 성공하면 공격자는 임의의 연산을 실행하거나 침해한 시스템의 연산 자원을 남에게 빌려줄 수도 있다. 운영체제 자체를 공격하는 데 성공하면 더 많은 자원에 접근할 수 있다. 이는 사용자 코드가 커널 접근 권한을 얻는 **권한 상승(privilege escalation)**의 한 형태이다. 짐작했겠지만 모두 나쁜 일이다.

버퍼 오버플로에 대한 첫 번째이자 가장 단순한 방어는 주소 공간의 특정 영역, 예를 들어 스택에 있는 코드의 실행을 막는 것이다. AMD가 자사의 x86에 도입한 **NX(No-eXecute) 비트**가 그런 방어책이다. Intel에도 비슷한 XD 비트가 있다. 페이지 테이블 항목에 이 비트가 설정된 페이지에서 코드를 실행하지 못하게 할 뿐이다. 공격자가 대상의 스택에 주입한 코드가 실행되지 않으므로 문제가 완화된다.

하지만 영리한 공격자는 영리하다. 공격자가 주입한 코드를 명시적으로 추가할 수 없어도 악성 코드는 임의의 명령어 시퀀스를 실행할 수 있다. 가장 일반적인 형태를 **반환 지향 프로그래밍(return-oriented programming, ROP)**[S07]이라고 하며, 참으로 기발하다. ROP는 프로그램의 주소 공간, 특히 방대한 C 라이브러리와 연결된 C 프로그램 안에 수많은 코드 조각이 있다는 관찰에서 출발한다. ROP 용어로는 이를 가젯(gadget)이라고 한다. 공격자는 현재 실행되는 함수의 반환 주소가 원하는 악성 명령어 또는 명령어열을 가리키도록 스택을 덮어쓸 수 있다. 그 명령어열은 반환 명령어로 끝난다. 많은 가젯을 이어 붙이고 각 반환이 다음 가젯으로 이동하게 하면 임의의 코드를 실행할 수 있다. 놀랍다!

이전 형태인 return-to-libc 공격[S+04]을 포함해 ROP를 막으려고 Linux와 다른 시스템은 **주소 공간 배치 무작위화(address space layout randomization, ASLR)**라는 방어를 추가한다. 코드, 스택, 힙을 가상 주소 공간의 고정된 위치에 배치하는 대신 운영체제가 배치를 무작위화한다. 그러면 이 부류의 공격에 필요한 정교한 코드

시퀀스를 만들기가 상당히 어려워진다. 따라서 취약한 사용자 프로그램에 대한 대부분의 공격은 충돌을 일으키지만 실행 중인 프로그램의 제어권을 얻지는 못한다.

흥미롭게도 이 무작위성은 실제로 쉽게 관찰할 수 있다. 다음 코드는 현대 Linux 시스템에서 이를 보여 준다.

```
int main(int argc, char *argv[]) {
    int stack = 0;
    printf("%p\n", &stack);
    return 0;
}
```

이 코드는 스택에 있는 변수의 가상 주소를 출력할 뿐이다. ASLR이 없던 오래된 시스템에서는 매번 같은 값이 나왔을 것이다. 하지만 아래처럼 실행할 때마다 값이 바뀐다.

```
prompt> ./random
0x7ffd3e55d2b4
prompt> ./random
0x7ffe1033b8f4
prompt> ./random
0x7ffe45522e94
```

ASLR은 사용자 수준 프로그램에 매우 유용한 방어책이어서 커널에도 도입되었다. 다소 평범한 이름인 **커널 주소 공간 배치 무작위화(kernel address space layout randomization, KASLR)**라고 한다. 그러나 다음에서 보듯이 커널에는 더 큰 문제들이 있을 수 있다.

3.6 다른 보안 문제: Meltdown과 Spectre

영어 16-17쪽, Other Security Problems: Meltdown And Spectre

이 글을 쓰는 2018년 8월, 시스템 보안의 세계는 서로 관련된 두 가지 새 공격 때문에 뒤집혔다. 첫 번째는 Meltdown, 두 번째는 Spectre이다. 서로 다른 네 연구자·엔지니어 그룹이 거의 같은 시기에 이들을 발견했다. 이 공격들은 컴퓨터 하드웨어와 그 위의 운영체제가 제공하는 근본적인 보호에 깊은 의문을 제기하게 했다. 각 공격을 자세히 설명한 논문은 spectreattack.com을 참조하라. 둘 중 Spectre가 더 문제시된다.

두 공격이 이용하는 공통 약점은 현대 시스템의 CPU가 성능을 높이려고 보이지 않는 곳에서 온갖 복잡한 기법을 수행한다는 데 있다. 문제의 핵심에 있는 기법 중 하나는 **추측 실행(speculative execution)**이다. CPU는 가까운 미래에 어떤 명령어들이 실행될지 추측하고 미리 실행하기 시작한다. 추측이 맞으면 프로그램은 더 빨리 실행된다. 틀리면 CPU는 레지스터 같은 아키텍처 상태에 미친 영향을 되돌린 뒤, 이번에는 올바른 경로로 다시 시도한다.

추측 실행의 문제는 프로세서 캐시, 분기 예측기 등 시스템의 여러 곳에 실행 흔적을 남기는 경향이 있다는 것이다. 공격을 발표한 저자들이 보였듯이, 이런 상태 때문에 메모리 내용이 노출 위험에 놓일 수 있다. MMU가 보호한다고 생각했던 메모리도 마찬가지다.

커널 보호를 강화하는 한 가지 방법은 각 사용자 프로세스에서 커널 주소 공간을 가능한 한 많이 제거하고, 대부분의 커널 데이터를 별도의 커널 페이지 테이블에 두는 것이다. 이를 **커널 페이지 테이블 격리(kernel page-table isolation, KPTI)**[G+17]라고 한다. 커널 코드와 자료 구조를 각 프로세스에 매핑하는 대신 그곳에는 꼭 필요한 최소한만 남긴다. 따라서 커널로 전환할 때는 이제 커널 페이지 테이블로도 바꾸어야 한다. 이는 보안을 높이고 일부 공격 경로를 피하게 해 주지만 성능이라는 비용을 치른다. 페이지 테이블 전환은 비싸다. 보안을 위해 편의성과 성능을 내어주는 것이다.

안타깝게도 KPTI는 위에서 설명한 모든 보안 문제를 해결하지 못하고 일부만 해결한다. 추측 실행을 끄는 것 같은 단순한 해결책도 합리적이지 않다. 시스템이 수천 배 느려질 것이기 때문이다. 따라서 시스템 보안에 관심이 있다면 지금은 흥미로운 시대이다.

이 공격들을 제대로 이해하려면 아마 먼저 더 많이 배워야 할 것이다. 관련 고급 서적에서 현대 컴퓨터 아키텍처를 이해하는 것으로 시작하라. 추측 실행과 그것을 구현하는 데 필요한 모든 기법에 초점을 맞추어라. 앞서 언급한 웹사이트에서 Meltdown과 Spectre 공격을 반드시 읽어 보라. 그 자료에는 추측 실행에 관한 유용한 입문 설명도 있으므로 출발점으로도 괜찮을 것이다. 운영체제에 다른 취약점이 있는지도 공부하라. 어떤 문제가 더 남아 있는지 누가 알겠는가?

판본 주석: "추측 실행을 끄면 수천 배 느려진다"는 원문의 문장을 그대로 옮긴 것이다. 이 자료에서 별도로 측정하거나 검증한 성능 수치는 아니다.

4. 요약에 추가된 Linux 내용

[신규·개정] 영어 §23.3, 17-18쪽 ↔ 한국어 §26.6, 7-8쪽

두 시스템을 처음부터 끝까지 살펴보았다는 신판 요약 중 VAX/VMS 학습과 Levy·Lipman 논문 권고는 한국어판에도 있어 제외했다. 한국어판은 Linux를 따로 읽어 보라고 권하지만, 아래 구체적인 Linux 기법들과 마지막 비유는 없다.

Linux에 대해서도 조금 배웠다. 크고 복잡한 시스템이지만 과거의 좋은 아이디어들을 많이 이어받았다. 그중 상당수는 자세히 논의할 여유가 없었다. 예를 들어 Linux는 fork() 때 페이지를 게으르게 쓰기 시 복사(copy-on-write)하므로 불필요한 복사를 피하여 오버헤드를 줄인다. 또한 /dev/zero 장치의 메모리 매핑을 이용해 페이지를 요청 시 0으로 채우며(demand zeroing), 백그라운드 스왑 데몬 swapd가 페이지를 디스크로 스왑하여 메모리 압박을 줄인다. 실제로 이 가상 메모리 시스템에는 과거에서 가져온 좋은 아이디어들이 가득하고, 자체적인 많은 혁신도 포함되어 있다.

더 배우려면 괜찮은 책들[BC05, G04]을 읽어 보라. 안타깝게도 오래된 책들이지만 직접 읽어 보기를 권한다. 우리가 제공할 수 있는 것은 복잡성의 바다에서 한 방울에 지나지 않기 때문이다. 하지만 어디에선가 시작해야 한다. 바다란 수많은 물방울이 모인 것 말고 또 무엇이겠는가?[M04]

판본 주석: /dev/zero와 swapd라는 표기는 첨부 원문 그대로 유지했다. 한국어 구판의 [BC05] 권고와 겹치는 일반적인 독서 권고는 새 절로 세지 않고, 새 문헌 [G04]와 새 요약 문맥 안에서만 함께 표시했다.

5. 새 참고문헌 8개

[신규] 영어 References, 19쪽 ↔ 한국어 참고문헌, 9쪽에 대응 항목 없음

[B+13] A. Basu, J. Gandhi, J. Chang, M. D. Hill, M. M. Swift. "Efficient Virtual Memory for Big Memory Servers." ISCA'13, 2013년 6월, 이스라엘 텔아비브.

제목 번역: "대용량 메모리 서버를 위한 효율적인 가상 메모리." TLB가 중요하며 대용량 메모리 워크로드에서 사이클의 10%를 소비한다는 사실을 보여 주는 최근 연구이다. 해법은 큰 데이터 집합을 담는 하나의 거대한 세그먼트이다. 앞으로 나아가기 위해 뒤로 돌아가는 것이다!

[G04] M. Gorman. "Understanding the Linux Virtual Memory Manager." Prentice Hall, 2004.

제목 번역: "Linux 가상 메모리 관리자 이해하기." Linux 가상 메모리를 깊이 있게 살펴보지만, 안타깝게도 조금 오래되었다.

[G+17] D. Gruss, M. Lipp, M. Schwarz, R. Fellner, C. Maurice, S. Mangard. "KASLR is Dead: Long Live KASLR." Engineering Secure Software and Systems, 2017.

제목 번역: "KASLR은 죽었다: KASLR 만세." KASLR, KPTI와 그 이상의 내용에 관한 훌륭한 정보를 제공한다. 원문에 제시된 자료: https://gruss.cc/files/kaiser.pdf

[JS94] T. Johnson, D. Shasha. "2Q: A Low Overhead High Performance Buffer Management Replacement Algorithm." VLDB'94, 칠레 산티아고.

제목 번역: "2Q: 낮은 오버헤드와 높은 성능을 갖춘 버퍼 관리 교체 알고리즘." 페이지 교체를 구현하는 간단하지만 효과적인 접근법이다.

[M04] D. Mitchell. "Cloud Atlas." Random House, 2004.

제목 번역: "클라우드 아틀라스." 가장 좋아하는 책을 고르기는 어렵다. 책이 너무 많고 각각 나름의 방식으로 훌륭하다. 하지만 이 책의 저자들이라면 인간의 조건을 다루는 환상적이고 광대한 서사인 "클라우드 아틀라스"를 고르지 않기 어려울 것이다. 이 장 마지막 인용문도 여기서 가져왔다. 당신이 영리하다면, 그리고 우리는 그렇다고 생각한다면, 참고문헌의 이런 알아보기 힘든 논평을 그만 읽고 대신 "클라우드 아틀라스"를 읽어야 한다. 나중에 우리에게 고마워할 것이다.

[O16] A. Ott. "Virtual Memory and Linux." Embedded Linux Conference, 2016년 4월.

제목 번역: "가상 메모리와 Linux." Linux 가상 메모리를 개괄하는 유용한 발표 자료이다. 원문 URL: https://events.static.linuxfound.org/sites/events/files/slides/elc2016mem.pdf

[S07] H. Shacham. "The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls (on the x86)." CCS'07, 2007년 10월.

제목 번역: "뼈 위에 붙은 무고한 살의 기하학: 함수 호출 없는 return-into-libc(x86에서)." return-to-libc를 일반화한 논문이다. 영화 "원초적 본능"에서 Beth Garner 박사는 "그녀는 미쳤어요! 그녀는 천재예요!"라고 말했다. ROP에도 같은 말을 할 수 있겠다.

[S+04] H. Shacham, M. Page, B. Pfaff, E. J. Goh, N. Modadugu, D. Boneh. "On the Effectiveness of Address-space Randomization." CCS'04, 2004년 10월.

제목 번역: "주소 공간 무작위화의 효과에 관하여." return-to-libc 공격과 그 한계를 설명한다. 읽기 시작하되 조심하라. 시스템 보안이라는 토끼굴은 깊다.

전체 대조표: 마지막 페이지까지 확인

영 = 영어 PDF 쪽수 / 한 = 한국어 PDF 쪽수. 대응 내용은 번역 분량에 넣지 않았다. 장 번호·인용 키·배치 차이는 내용 누락과 구별했다.

기존 참고문헌 7개는 재수록하지 않았다. [BB+72]↔[Bob+72], [C03]↔[Chr03], [C93]↔[CS93], [RL81]↔[TL81]은 인용 키가 달라도 같은 문헌이다. [BJ81], [BC05], [LL82]는 두 판에서 키가 같다. BC05의 제목에서 Third Edition이 생략되고 설명이 짧아진 것은 신규 문헌으로 보지 않았다.

원문 절·대응 위치	쪽수	대조 결과
서론·새 핵심 질문	영 1 / 한 1	학습 요소·성능·기능·보안·두 시스템 설명 추가.
§23.1 VAX 도입 ↔ §26.1	영 2 / 한 1-2	역사·DEC·Cutler·하드웨어 결함 대응. 복수형 보충.
범용성의 저주 여담	영 3 / 한 1	일반 설명 대응. Linux 열거 문장 추가.
Memory Management Hardware ↔ §26.2	영 2-3 / 한 2	구조·2개 테이블·스왑 대응. TLB 설명 보충.
A Real Address Space ↔ §26.3	영 3-5 / 한 3-4	그림 23.1↔26.1·널 포인터 여담·코드 대응.
Page Replacement ↔ §26.4	영 5-7 / 한 4-5	RSS·FIFO·클러스터링 대응. hog 영향 보충.
참조 비트 에뮬레이션 여담	영 6 / 한 6	절차 대응. 예약 OS 필드 예시 보충.
Other Neat Tricks ↔ §26.5	영 7-8 / 한 5-7	demand zero·COW·fork/exec·게으름 팁 대응.
§23.2: 도입	영 8 / 한 없음	전체 신규. 자료 3 도입.
Linux Address Space	영 8-10 / 한 없음	전체 신규. 그림 23.2·각주 1 포함.
Page Table Structure	영 10-11 / 한 없음	전체 신규. 64비트 주소 도식 포함.
Large Page Support	영 11-12 / 한 없음	전체 신규. 점진적 접근 팁 포함.
The Page Cache	영 13 / 한 없음	전체 신규. 2Q·클린/더티·회수 포함.
메모리 매핑 여담	영 14 / 한 없음	전체 신규. pmap 출력 7행 포함.
Security And Buffer Overflows	영 14-16 / 한 없음	전체 신규. 코드 2개·출력·각주 2 포함.
Meltdown And Spectre	영 16-17 / 한 없음	전체 신규. 추측 실행·KPTI 포함.
§23.3 ↔ §26.6	영 17-18 / 한 7-8	VAX 요약 대응. Linux 기법·마지막 비유 추가.
참고문헌	영 19 / 한 9	7개 대응, 8개 신규. 자료 5.
장 끝 확인	영 19 / 한 9	두 파일 모두 참고문헌으로 끝남. 별도 숙제 없음.

완료 확인: 영어·한국어 본문, 그림·도식, 팁·여담, 각주, 참고문헌과 장 말미를 대조했다. 번역문은 추가된 내용과 개정 부분만 수록하며, 반복되는 기존 한국어 설명은 해당 위치만 안내한다.